

Multi-Cloud Engineering Program

Manual integral del curso

24 partes · 288 clases · 1.288 horas

De fundamentos de computación a arquitectura de sistemas, AWS, Azure, Google Cloud, Kubernetes, IaC, datos e IA, seguridad, SRE, FinOps, operación y capstones por industria.

Contenido íntegro generado desde las fuentes versionadas del repositorio.

Alcance e integridad

Este manual no es un catálogo. Incluye las guías pedagógicas centrales, los 24 módulos, las 288 lecciones completas y sus 288 evaluaciones. El laboratorio ejecutable y sus plantillas permanecen en el repositorio para conservar su carácter reproducible.

Componente	Cobertura
Partes	24 de 24
Lecciones	288 de 288
Evaluaciones	288 de 288
Horas estimadas	1288
Fuentes	Markdown canónico + curriculum/catalog.json

Índice general

Alcance e integridad	2
Índice general	3
Guías pedagógicas	12
Guía del estudiante	12
Metodología docente	13
Syllabus y cronograma	15
Rúbrica de evaluación	17
Especializaciones e industrias	18
Mapa de certificaciones	19
Bibliografía central y mapa de lectura	20
Parte 00 — Fundamentos de computación, redes y Linux	21
001 — Computación digital y modelo mental de la nube	23
002 — Terminal, sistema de archivos, procesos y variables de entorno	29
003 — Git, GitHub y trabajo reproducible	35
004 — Python, JSON y automatización mínima	41
005 — Redes por capas, TCP/IP, puertos y sockets	47
006 — DNS, HTTP, HTTPS y TLS de extremo a extremo	53
007 — Linux: usuarios, permisos, servicios y logs	59
008 — Virtualización, hipervisores e imágenes	65
009 — APIs REST, autenticación y contratos	72
010 — Responsabilidad compartida y pensamiento de riesgo	78
011 — Costo, energía, capacidad y medición básica	85
012 — Proyecto: servicio local reproducible y observable	91
Parte 01 — Principios, estrategia y adopción cloud	98
013 — Definición NIST y características esenciales de cloud	100
014 — Regiones, zonas de disponibilidad, puntos de presencia y edge	107
015 — IaaS, PaaS, SaaS, CaaS y FaaS	114
016 — Elasticidad, escalabilidad, disponibilidad y resiliencia	120
017 — Tenancy, cuentas, suscripciones, proyectos y jerarquías	127
018 — Identidad, roles, políticas y federación	134
019 — Modelo de responsabilidad compartida por servicio	141
020 — TCO, costos variables, unit economics y FinOps	148
021 — Well-Architected y atributos de calidad	155
022 — Cloud Adoption Framework y modelo operativo	162
023 — Descubrimiento y clasificación de workloads	169
024 — Proyecto: decisión de migración sustentada con ADR	176

Parte 02 — AWS: plataforma esencial	183
025 — Organizations, cuentas, OU, SCP y landing zone	185
026 — IAM, roles, políticas, STS y federación	192
027 — VPC, subredes, rutas, NAT, endpoints y seguridad	200
028 — EC2, AMI, EBS y selección de capacidad	207
029 — Elastic Load Balancing y Auto Scaling	214
030 — S3: objetos, versionado, lifecycle y replicación	221
031 — RDS, DynamoDB y ElastiCache: decisión de datos	228
032 — Lambda, API Gateway y Step Functions	235
033 — SQS, SNS y EventBridge	242
034 — CloudWatch, CloudTrail, Config y Systems Manager	249
035 — KMS, Secrets Manager, WAF y controles de seguridad	256
036 — Proyecto: aplicación de tres capas en AWS	264
Parte 03 — Azure: plataforma esencial	271
037 — Tenant, management groups, suscripciones y resource groups	273
038 — Microsoft Entra ID, RBAC, managed identities y PIM	281
039 — Virtual Network, subredes, NSG, UDR, peering y Private Link	289
040 — Virtual Machines, Scale Sets y Load Balancer	299
041 — Blob Storage, Files, redundancia y lifecycle	308
042 — Azure SQL, Cosmos DB y Azure Cache for Redis	318
043 — App Service, Functions y Container Apps	322
044 — Service Bus, Event Grid y Event Hubs	326
045 — Azure Monitor, Log Analytics y Application Insights	330
046 — Key Vault, Defender for Cloud y Azure Policy	334
047 — Bicep, plantillas y despliegues por alcance	338
048 — Proyecto: aplicación de tres capas en Azure	342
Parte 04 — Google Cloud: plataforma esencial	346
049 — Organización, folders, proyectos, billing y cuotas	348
050 — IAM, service accounts y Workload Identity Federation	352
051 — VPC global, subredes regionales, firewall y Cloud NAT	356
052 — Compute Engine, managed instance groups y load balancing	360
053 — Cloud Storage, clases, lifecycle y replicación	364
054 — Cloud SQL, Spanner, Firestore y Memorystore	368
055 — Cloud Run, Cloud Functions y API Gateway	372
056 — Pub/Sub, Cloud Tasks y Workflows	376
057 — Cloud Logging, Monitoring, Trace y Audit Logs	380
058 — Cloud KMS, Secret Manager y Security Command Center	384
059 — Terraform y despliegues reproducibles en GCP	388
060 — Proyecto: aplicación de tres capas en Google Cloud	392
Parte 05 — Contenedores, Docker y OCI	396

061 — Imágenes, capas, registros y estándar OCI	398
062 — Dockerfile reproducible y builds multi-stage	402
063 — Namespaces, cgroups y runtime de contenedores	406
064 — Volúmenes, bind mounts y persistencia	410
065 — Redes bridge, DNS interno y publicación de puertos	414
066 — Docker Compose y aplicaciones multiservicio	418
067 — Registros, SBOM, firma y procedencia de imágenes	422
068 — Límites, health checks y apagado ordenado	426
069 — Rootless, capabilities, seccomp y secretos	430
070 — Diagnóstico de CPU, memoria, red y filesystem	434
071 — Migración de una aplicación legacy a contenedores	438
072 — Proyecto: stack OCI endurecido y observable	442

Parte 06 — Kubernetes y plataformas administradas 446

073 — API server, etcd, scheduler, controllers y kubelet	448
074 — Pods, ReplicaSets, Deployments y Jobs	452
075 — Services, DNS, Ingress y Gateway API	456
076 — ConfigMaps, Secrets y configuración externa	460
077 — Volumes, PersistentVolumes, CSI y StatefulSets	464
078 — Requests, limits, scheduling y autoscaling	468
079 — Probes, rollouts, rollback y PodDisruptionBudget	472
080 — Namespaces, RBAC, NetworkPolicy y admission	476
081 — Helm, Kustomize y gestión de paquetes	480
082 — Logs, métricas, eventos y depuración	484
083 — EKS, AKS y GKE: similitudes y diferencias	488
084 — Proyecto: plataforma Kubernetes portable	492

Parte 07 — Infraestructura como código y configuración 496

085 — Declarativo, imperativo, idempotencia y convergencia	498
086 — Terraform: HCL, providers, resources y grafo	502
087 — Estado remoto, locking, cifrado y recuperación	506
088 — Módulos, contratos, versiones y composición	510
089 — Variables, outputs, locals y data sources	514
090 — Plan, apply, drift, import y refactor con moved	518
091 — Validación, lint, pruebas y policy as code	522
092 — Secretos y datos sensibles en IaC	526
093 — CloudFormation, Bicep, Pulumi y Terraform	530
094 — Ansible e imagen dorada para configuración	534
095 — Plantillas, golden paths y catálogo interno	538
096 — Proyecto: infraestructura multiambiente promovible	542

Parte 08 — Entrega continua y platform engineering 546

097 — Integración continua, trunk-based development y feedback	548
--	-----

098 — GitHub Actions: workflows, runners, permisos y caché	552
099 — Artefactos inmutables, semver y promoción	556
100 — Pruebas, calidad y puertas de cambio	560
101 — SAST, SCA, secretos, SBOM y firma en pipeline	564
102 — Rolling, blue-green, canary y rollback	568
103 — GitOps con Argo CD o Flux	572
104 — Ambientes efímeros y promoción entre entornos	576
105 — Feature flags y separación deploy-release	580
106 — Platform engineering e Internal Developer Platform	584
107 — Developer experience, DORA y carga cognitiva	588
108 — Proyecto: fábrica de software multi-cloud	592

Parte 09 — Datos, mensajería, serverless e integración 596

109 — Bases relacionales administradas y pooling	598
110 — NoSQL: clave-valor, documento, columnar y grafo	602
111 — Caché, invalidación, TTL y consistencia	606
112 — Object storage, data lake y formatos columnares	610
113 — Colas, entrega, reintentos y dead-letter queues	614
114 — Pub/sub, streams, particiones y orden	618
115 — Arquitectura dirigida por eventos y contratos	622
116 — Sagas, outbox, idempotencia y deduplicación	626
117 — Serverless: límites, cold starts y concurrencia	630
118 — API management, cuotas, versiones y monetización	634
119 — Workflows y orquestación durable	638
120 — Proyecto: pipeline de pedidos orientado a eventos	642

Parte 10 — Observabilidad, SRE y confiabilidad 646

121 — Logs, métricas, trazas y eventos como señales	648
122 — Logging estructurado, correlación y retención	652
123 — Métricas, cardinalidad y modelos RED y USE	656
124 — Tracing distribuido y OpenTelemetry	660
125 — Dashboards, alertas accionables y fatiga	664
126 — SLI, SLO, SLA y presupuesto de error	668
127 — Incidentes, severidad, comando y comunicación	672
128 — Runbooks, playbooks y automatización operativa	676
129 — Capacidad, rendimiento y pruebas de carga	680
130 — Timeouts, retries, backoff, circuit breaker y bulkhead	684
131 — Chaos engineering y game days	688
132 — Proyecto: operación SRE de CloudShop	692

Parte 11 — Seguridad, gobierno, cumplimiento y FinOps 696

133 — Zero Trust y defensa en profundidad	698
134 — Mínimo privilegio, acceso temporal y separación de funciones	702

135 — Segmentación, perímetro, WAF, DDoS y egress	706
136 — Cifrado, KMS, HSM, rotación y envelope encryption	710
137 — Gestión de secretos y credenciales de workloads	714
138 — Vulnerabilidades, imágenes y cadena de suministro	718
139 — CSPM, postura, policy as code y remediación	722
140 — Threat modeling con STRIDE y attack paths	726
141 — Cumplimiento, residencia, privacidad y evidencia	730
142 — FinOps: showback, chargeback, budgets y anomalías	734
143 — Optimización de costo, capacidad y sostenibilidad	738
144 — Proyecto: landing zone con guardrails	742

Parte 12 — Arquitectura cloud-native y sistemas distribuidos 746

145 — Requisitos, restricciones y atributos de calidad	748
146 — Twelve-Factor App y configuración cloud-native	752
147 — DDD, bounded contexts y ownership de datos	756
148 — Monolito modular, microservicios y función	760
149 — CAP, PACELC y consistencia por operación	764
150 — Replicación, particionado y consenso	768
151 — Fallos parciales y patrones de resiliencia	772
152 — Service discovery, malla y comunicación	776
153 — Contratos API, compatibilidad y evolución	780
154 — Multi-tenancy, aislamiento y noisy neighbor	784
155 — Rendimiento, costo, seguridad y operabilidad	788
156 — Proyecto: revisión de arquitectura con ADR	792

Parte 13 — Multi-cloud, híbrido, migración y recuperación 796

157 — Motivaciones y anti-patrones de multi-cloud	798
158 — Portabilidad, capas de abstracción y lock-in	802
159 — Federación de identidad entre nubes	806
160 — Conectividad, tránsito, DNS y service discovery	810
161 — Replicación de datos, soberanía y costos de egress	814
162 — Observabilidad y operación entre proveedores	818
163 — Terraform multi-provider y separación de estados	822
164 — Flotas Kubernetes y políticas comunes	826
165 — Nube híbrida, edge y conectividad privada	830
166 — Backup, RTO, RPO y patrones de disaster recovery	834
167 — Las 7R de migración y oleadas	838
168 — Proyecto: continuidad activa-pasiva entre nubes	842

Parte 14 — Plataformas avanzadas, capstones y carrera 846

169 — Landing zones empresariales y vending de cuentas	848
170 — Gobierno federado y policy as code a escala	852
171 — Platform as a Product y roadmap de capacidades	856

172 — Modelo operativo FinOps y economía unitaria	860
173 — Madurez SRE y confiabilidad organizacional	864
174 — Arquitectura de seguridad cloud empresarial	868
175 — Workloads de IA, GPU, datos y MLOps multi-cloud	872
176 — Edge, IoT y procesamiento desconectado	876
177 — Soberanía digital y confidential computing	880
178 — Capstone: descubrimiento y diseño	884
179 — Capstone: implementación y operación	888
180 — Capstone: defensa, portafolio y plan profesional	892

Parte 15 — Arquitectura de sistemas e ingeniería de requisitos 896

181 — Requisitos funcionales, restricciones y atributos de calidad	898
182 — Contexto, contenedores, componentes y código con C4	902
183 — Acoplamiento, cohesión, modularidad y fronteras	906
184 — Arquitectura monolítica, modular y de microservicios	910
185 — Disponibilidad, confiabilidad y análisis de puntos de fallo	914
186 — Capacidad, latencia, throughput y teoría de colas	918
187 — Consistencia, particiones, relojes y consenso	922
188 — Contratos de API, eventos y compatibilidad evolutiva	926
189 — Modelado de amenazas y arquitectura de confianza cero	930
190 — ADRs, fitness functions y gobierno de decisiones	934
191 — Architecture review y comunicación con stakeholders	938
192 — Proyecto: arquitectura completa de CloudShop	942

Parte 16 — Redes cloud avanzadas, conectividad híbrida y edge 946

193 — CIDR, subnetting y planificación IP a escala	948
194 — Routing, BGP, tránsito y propagación de rutas	952
195 — DNS autoritativo, recursivo, split-horizon y DNSSEC	956
196 — Balanceo L4/L7, proxies, TLS y gestión de certificados	960
197 — CDN, caché, origin shielding y edge compute	964
198 — VPN, Direct Connect, ExpressRoute e Interconnect	968
199 — Transit Gateway, Virtual WAN y Network Connectivity Center	972
200 — Private endpoints, service networking y egress control	976
201 — Service mesh, mTLS y gestión de tráfico este-oeste	980
202 — eBPF, flow logs, packet capture y diagnóstico	984
203 — SD-WAN, 5G, IoT y operación desconectada	988
204 — Proyecto: red multi-región y multi-cloud	992

Parte 17 — AWS: arquitectura, automatización y operación en producción 996

205 — Hosting progresivo con Amplify, S3 y CloudFront	998
206 — OIDC de GitHub y GitLab hacia AWS sin secretos	1002
207 — SAM, Lambda, API Gateway y despliegue serverless	1006
208 — DynamoDB por patrones de acceso y single-table design	1010

209 — Cognito, JWT authorizers, WAF y defensa en profundidad	1014
210 — EventBridge, SQS, DLQ, replay e idempotencia	1018
211 — CloudWatch, X-Ray y observabilidad como código	1022
212 — ECR, ECS Fargate, ALB y autoscaling	1026
213 — EKS, IRSA, GitOps y operación de clúster	1030
214 — Budgets, Cost Explorer, etiquetado y FinOps automatizado	1034
215 — Multi-región, Route 53, failover y game day	1038
216 — Proyecto: CloudShop productivo en AWS	1042

Parte 18 — Azure: arquitectura empresarial y operación en producción 1046

217 — Enterprise-scale landing zones y management groups	1048
218 — Entra ID, workload identity, PIM y Conditional Access	1052
219 — Hub-spoke, Virtual WAN, Private Link y DNS privado	1056
220 — Bicep, deployment stacks y Azure Verified Modules	1060
221 — App Service, Functions y Container Apps en producción	1064
222 — AKS, workload identity, ingress y GitOps	1068
223 — Azure SQL, Cosmos DB y consistencia distribuida	1072
224 — Service Bus, Event Grid y Event Hubs	1076
225 — Azure Monitor, Application Insights y OpenTelemetry	1080
226 — Defender for Cloud, Policy y Sentinel	1084
227 — Cost Management, Advisor, resiliencia y Chaos Studio	1088
228 — Proyecto: CloudShop productivo en Azure	1092

Parte 19 — Google Cloud: arquitectura de datos y operación en producción 1096

229 — Resource Manager, folders, Shared VPC y guardrails	1098
230 — Workload Identity Federation, IAM Conditions y PAM	1102
231 — Red global, load balancing, PSC y Cloud DNS	1106
232 — Terraform, Infrastructure Manager y policy validation	1110
233 — Cloud Run, Functions, API Gateway y Workflows	1114
234 — GKE Autopilot, Workload Identity y Config Sync	1118
235 — Cloud SQL, Spanner, Firestore y Bigtable	1122
236 — BigQuery, Dataflow, Dataproc y gobernanza de datos	1126
237 — Pub/Sub, Eventarc y entrega exactamente-una-vez	1130
238 — Cloud Operations, Trace y OpenTelemetry	1134
239 — SCC, VPC Service Controls, KMS y FinOps	1138
240 — Proyecto: CloudShop productivo en Google Cloud	1142

Parte 20 — Plataformas cloud de datos, analítica, IA y agentes 1146

241 — Lakehouse, warehouse, mesh y contratos de datos	1148
242 — Ingesta batch, CDC y streaming	1152
243 — Orquestación, calidad, lineage y observabilidad de datos	1156
244 — Feature stores, training pipelines y experiment tracking	1160
245 — Serving online, batch inference y escalado de modelos	1164

246 — MLOps, registro, promoción, drift y rollback	1168
247 — Modelos fundacionales, tokens, embeddings y RAG	1172
248 — Bedrock, Azure AI Foundry y Vertex AI	1176
249 — Agentes, tools, memoria, permisos y guardrails	1180
250 — Evaluación de IA, red teaming y observabilidad	1184
251 — Privacidad, gobernanza, sostenibilidad y costo de IA	1188
252 — Proyecto: asistente operativo de CloudShop	1192

Parte 21 — Operación cloud, automatización y respuesta a incidentes 1196

253 — Inventario, etiquetado, CMDB y ownership	1198
254 — Patching, imágenes doradas y gestión de configuración	1202
255 — Backups, restore testing, vaults e inmutabilidad	1206
256 — Administración remota sin SSH permanente	1210
257 — Alertas, on-call, escalamiento y comunicación	1214
258 — Triage de red, cómputo, datos y dependencias	1218
259 — Runbooks ejecutables y auto-remediation	1222
260 — Change management, ventanas y rollback	1226
261 — Game days, chaos engineering y aprendizaje	1230
262 — Capacity planning, cuotas y gestión de demanda	1234
263 — AIOps, automatización asistida y límites humanos	1238
264 — Proyecto: centro de operaciones de CloudShop	1242

Parte 22 — Especializaciones, certificaciones y práctica profesional 1246

265 — Ruta Cloud Engineer y mapa de competencias	1248
266 — Ruta DevOps y Delivery Engineer	1252
267 — Ruta Platform Engineer	1256
268 — Ruta Site Reliability Engineer	1260
269 — Ruta Cloud Security Engineer	1264
270 — Ruta FinOps Practitioner	1268
271 — Ruta Cloud Data y AI Engineer	1272
272 — Ruta Cloud Solutions Architect	1276
273 — Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps	1280
274 — Preguntas de escenario y estrategia de examen	1284
275 — Portafolio, evidencia, README y entrevista de sistemas	1288
276 — Proyecto: defensa técnica ante panel	1292

Parte 23 — Capstones por industria y defensa final 1296

277 — Capstone retail: comercio multi-región	1298
278 — Capstone financiero: pagos, auditoría y recuperación	1302
279 — Capstone salud: privacidad e interoperabilidad	1306
280 — Capstone sector público: soberanía y continuidad	1310
281 — Capstone media: streaming y distribución global	1314
282 — Capstone industria: IoT, edge y operación desconectada	1318

283 — Capstone SaaS: multi-tenancy y unit economics	1322
284 — Capstone datos e IA: plataforma gobernada	1326
285 — Game day integrado y respuesta a incidentes	1330
286 — Revisión Well-Architected multi-proveedor	1334
287 — Paquete de evidencia, costos y riesgos residuales	1338
288 — Defensa final, retrospectiva y plan de 12 meses	1342

Guías pedagógicas

Guía del estudiante

1. Ejecuta el diagnóstico y elige una ruta; no omitas prerequisites que no puedas demostrar.
2. En cada clase: predice, ejecuta, provoca fallo, recupera y conserva evidencia.
3. Cada seis partes presenta un checkpoint. Repite la práctica si una dimensión queda bajo 70%.
4. Usa cuentas autorizadas, identidad temporal, presupuesto y destrucción obligatoria.
5. La graduación requiere capstone, game day, paquete de evidencia y defensa oral.

Metodología docente

Propósito

El programa convierte servicios cloud en decisiones de ingeniería comprensibles. Cada clase debe dejar al estudiante pudiendo explicar, ejecutar, observar y defender una decisión.

Secuencia central

1. Problema: escenario, usuario, restricciones y consecuencias del fallo.
2. Modelo mental: mecanismo neutral antes del nombre comercial.
3. Demostración: el docente verbaliza decisiones y señales.
4. Práctica guiada: laboratorio local, determinista y sin costo.
5. Transferencia: adaptación a AWS, Azure, Google Cloud o entorno híbrido.
6. Evidencia: artefacto, salida, prueba negativa y limitación.
7. Reflexión: trade-off, error frecuente y condición de revisión.

Modelo de sesión de 120 minutos

Momento	Minutos	Actividad
Apertura	10	Objetivo, caso y activación de prerrequisitos
Modelo mental	20	Una idea fuerte y sus límites
Demostración	20	Recorrido corto con decisiones verbalizadas
Práctica guiada	30	Ejecución y lectura de evidencia
Reto	25	Variación, fallo o alternativa
Revisión	10	Pares aplican criterio de aceptación
Cierre	5	Síntesis, error frecuente y siguiente dependencia

Regla de herramientas e IA

El estudiante puede consultar documentación, buscadores y asistentes, pero debe seguir:

1. formular el problema;
2. declarar el supuesto;
3. consultar;
4. ejecutar o contrastar;
5. explicar con evidencia;
6. registrar límites y fuente.

Una respuesta generada sin verificación no es evidencia de dominio.

Intervenciones docentes

Cuando el grupo va lento, se reduce el escenario y se conserva el mismo contrato de evidencia. Cuando va rápido, se añade fallo parcial, límite de costo o segundo proveedor. Cuando aparece ansiedad por la cantidad de servicios, se vuelve al mecanismo neutral y a la pregunta que resuelve. Cuando el laboratorio falla, el error se usa para practicar lectura de logs, hipótesis y descarte sistemático.

Evidencias mínimas

- comando o cambio ejecutado;
- salida o estado observado;
- interpretación en palabras propias;
- prueba negativa;
- limitación;
- decisión y condición de rollback.

Syllabus y cronograma

Carga total

El programa contiene 288 clases. Una clase regular estima 4 horas entre lectura, sesión, laboratorio y reto; los cierres y las tres clases finales de capstone estiman 8 horas. Carga total aproximada: 1.288 h.

Parte	Tema	Clases	Horas aprox.	Semanas a 10 h
00	Fundamentos técnicos	12	52	5–6
01	Estrategia y adopción	12	52	5–6
02	AWS	12	52	5–6
03	Azure	12	52	5–6
04	Google Cloud	12	52	5–6
05	Contenedores	12	52	5–6
06	Kubernetes	12	52	5–6
07	IaC	12	52	5–6
08	Entrega y plataforma	12	52	5–6
09	Datos e integración	12	52	5–6
10	Observabilidad y SRE	12	52	5–6
11	Seguridad, gobierno y FinOps	12	52	5–6
12	Arquitectura distribuida	12	52	5–6
13	Multi-cloud, híbrido y DR	12	52	5–6
14	Avanzado y capstones	12	60	6
15–23	Arquitectura, proveedores productivos, IA, operaciones y capstones	108	500	50
	Total	288	1.288	129

Ritmos sugeridos

Modalidad	Dedicación	Duración
Autodidacta compatible con trabajo	10 h/semana	18 meses
Bootcamp parcial	20 h/semana	9 meses
Inmersivo	35 h/semana	5–6 meses
Ruta especializada	8–12 h/semana	4–8 meses

Dependencias

- Parte 00 precede todo trabajo práctico.
- Parte 01 precede el estudio de proveedores.
- Se requiere dominar al menos uno de 02, 03 o 04 antes de 05–08.
- Partes 05 y 07 preceden 06 y 08 respectivamente.
- Partes 09–11 pueden estudiarse en paralelo después de 08.
- Parte 12 integra 09–11; 13 asume 12; 14 cierra todas las rutas.

Evaluación

Cada clase se aprueba con nivel B o superior. Una parte requiere 80 % de clases aprobadas y su proyecto integrador reproducible. El capstone final pondera arquitectura, implementación, operación, seguridad/costo y defensa profesional.

Rúbrica de evaluación

Reto por clase

Nivel	Descripción	Puntos
A — competente	Cumple el criterio completo, reproduce resultados y defiende trade-offs.	3
B — en desarrollo	Cumple lo esencial; una dimensión requiere mayor profundidad.	2
C — inicial	Entrega parcial, no reproducible o con errores conceptuales.	1
0	Sin evidencia.	0

Se aprueba una clase con B o superior. La evidencia debe provenir de ejecución, inspección o fuente primaria; una captura sin contexto no demuestra por sí sola el resultado.

Laboratorio

Criterio	Peso	A — competente
Corrección técnica	25	Configuración y conclusiones correctas
Evidencia	20	Afirmaciones conectadas a salidas observables
Reproducibilidad	15	Otra persona repite el recorrido
Prueba negativa	15	Denegación o fallo demuestra la frontera
Seguridad y costo	15	Permisos, datos y unidades explícitas
Comunicación	10	Decisión clara para audiencia técnica y ejecutiva

Capstone final

Dimensión	Peso
Requisitos, arquitectura y ADR	20
Infraestructura y entrega reproducible	20
Confiabilidad, observabilidad e incidentes	20
Seguridad, gobierno y FinOps	20
Pruebas, documentación y defensa	20

Un capstone aprobado debe sobrevivir una demostración de fallo, reconstruirse desde el repositorio y distinguir claramente evidencia local, sandbox y producción.

Integridad académica

Se permite colaboración y asistencia de IA declarada. El estudiante debe poder explicar, modificar y defender todo artefacto entregado. Secretos reales, datos personales o recursos no autorizados invalidan la entrega.

Especializaciones e industrias

Las rutas de Cloud Engineer, DevOps, Platform, SRE, Security, FinOps, Data/AI y Architect terminan en evidencia pública: diseño, código, prueba de fallo, costo, runbook y defensa. Los capstones cubren retail, finanzas, salud, sector público, media, industria, SaaS y datos/IA.

Mapa de certificaciones

El programa alinea objetivos, no garantiza aprobación. Los dominios se versionan y deben contrastarse con las guías oficiales vigentes.

Ruta	Partes principales
AWS SAA/DVA/SOA	02, 08, 10, 11, 17
Azure AZ-104/AZ-305	03, 10, 11, 18
Google ACE/PCA	04, 10, 11, 19
CKA/CKAD/CKS	05, 06, 08, 11
FinOps Practitioner	01, 11, 21, 22

Bibliografía central y mapa de lectura

La bibliografía pauta secuencia y profundidad; no se reproduce su contenido. Las ediciones y enlaces comerciales pueden cambiar, por lo que se recomienda verificar catálogo editorial.

Partes	Área	Obras principales
00	Sistemas, redes y herramientas	Ward, How Linux Works · Kurose/Ross, Computer Networking · Chacon/Straub, Pro Git
01, 13	Estrategia y multi-cloud	Hohpe, Cloud Strategy · Storment/Fuller, Cloud FinOps
02	AWS	AWS Well-Architected Framework · Culkin/Zazon, AWS Cookbook · Wittig/Wittig, Amazon Web Services in Action
03	Azure	Azure Architecture Center · Modi, Azure for Architects · Microsoft Press, Exam Ref AZ-305
04	Google Cloud	Google Cloud Architecture Framework · Sullivan, Professional Cloud Architect Study Guide
05	Contenedores	Poulton, Docker Deep Dive · Rice, Container Security · Davis, Cloud Native Patterns
06	Kubernetes	Burns/Beda/Hightower, Kubernetes: Up and Running · Ibryam/Huß, Kubernetes Patterns
07	IaC	Morris, Infrastructure as Code · Brikman, Terraform: Up & Running · Geerling, Ansible for DevOps
08	Entrega y plataforma	Humble/Farley, Continuous Delivery · Forsgren/Humble/Kim, Accelerate · Skelton/Pais, Team Topologies
09, 12	Datos y distribuidos	Kleppmann, Designing Data-Intensive Applications · Hohpe/Woolf, Enterprise Integration Patterns
10	SRE	Beyer et al., Site Reliability Engineering y The Site Reliability Workbook · Majors et al., Observability Engineering
11	Seguridad y FinOps	Dotson, Practical Cloud Security · Estrin, Cloud Security Handbook · Storment/Fuller, Cloud FinOps
12	Arquitectura	Davis, Cloud Native Patterns · Newman, Building Microservices · Nygard, Release It!
14	Organización y carrera	Skelton/Pais, Team Topologies · Richards/Ford, Fundamentals of Software Architecture

Fuentes primarias obligatorias

Además de libros, cada implementación debe citar documentación oficial vigente de AWS, Microsoft Azure, Google Cloud, CNCF/Kubernetes, OpenTelemetry, Terraform/OpenTofu y el estándar OCI correspondiente. Registra URL, versión o fecha de consulta y evita asumir que un límite, precio o nombre comercial permanece estable.

Método de lectura

1. Lee la clase para obtener vocabulario y preguntas.
2. Consulta el capítulo señalado por la parte.
3. Contrasta el mecanismo con documentación oficial.
4. Ejecuta el laboratorio y registra evidencia.
5. Escribe una conclusión propia y una limitación.

Parte 00 — Fundamentos de computación, redes y Linux

Programa · Índice completo · Parte 01 →

Nivel: inicial · Clases: 12 · Duración sugerida: 6–8 semanas

Construir el vocabulario y las destrezas técnicas que la nube da por supuestas.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
001	Computación digital y modelo mental de la nube	foundation	4
002	Terminal, sistema de archivos, procesos y variables de entorno	shell	4
003	Git, GitHub y trabajo reproducible	git	4
004	Python, JSON y automatización mínima	automation	4
005	Redes por capas, TCP/IP, puertos y sockets	network	4
006	DNS, HTTP, HTTPS y TLS de extremo a extremo	network	4
007	Linux: usuarios, permisos, servicios y logs	linux	4
008	Virtualización, hipervisores e imágenes	virtualization	4
009	APIs REST, autenticación y contratos	api	4
010	Responsabilidad compartida y pensamiento de riesgo	security	4
011	Costo, energía, capacidad y medición básica	finops	4
012	Proyecto: servicio local reproducible y observable	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- How Linux Works — Brian Ward.
- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Pro Git — Chacon y Straub.

001 — Computación digital y modelo mental de la nube

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: foundation · Estado: EXECUTABLECORE

Propósito

Construir el modelo mental que sostiene todo el programa: qué ejecuta realmente una máquina, por qué la latencia y no la velocidad del procesador domina el diseño de un sistema distribuido, y qué cambia exactamente cuando ese cómputo se alquila en lugar de comprarse. Sin este modelo, «la nube» se aprende como un catálogo de nombres comerciales que caduca cada dieciocho meses.

Resultados de aprendizaje

Al finalizar podrás:

1. Situar una operación en la jerarquía de memoria y estimar su coste en órdenes de magnitud, de 1 ns a 150 ms.
2. Aplicar las cinco características esenciales de NIST SP 800-145 para decidir si un servicio es realmente cloud o solo hosting renombrado.
3. Distinguir IaaS, PaaS y SaaS como fronteras de responsabilidad operativa, no como niveles de comodidad.
4. Calcular un presupuesto de latencia extremo a extremo y detectar qué componente lo consume.
5. Explicar por qué la elasticidad cambia la unidad económica de la infraestructura: de activo amortizado a gasto por consumo.

Conceptos centrales

Concepto	Comprensión verificable
latencia	Tiempo entre emitir una petición y recibir la primera respuesta. Está acotada por la velocidad de la luz en fibra (200.000 km/s), así que ninguna optimización de software reduce el mínimo físico entre dos ciudades.
ancho de banda	Volumen transferido por unidad de tiempo. Se compra; la latencia no. Añadir ancho de banda no acelera una petición pequeña, solo permite más peticiones simultáneas.
elasticidad	Capacidad de aprovisionar y liberar recursos en minutos siguiendo la demanda. Es lo que convierte capacidad ociosa en coste evitado; sin ella, la nube es un centro de datos con factura mensual.
multi-tenencia	Varios clientes compartiendo el mismo hardware con aislamiento lógico. Explica a la vez el precio (se amortiza entre muchos) y la clase de riesgo (vecino ruidoso, escapes del hipervisor).
modelo de servicio	Dónde se traza la línea entre lo que administra el proveedor y lo que sigue siendo tuyo. IaaS te deja el sistema operativo; PaaS, el runtime; SaaS, solo los datos y la configuración.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    subgraph fisico["Lo que existe de verdad"]
        CPU["CPU · registros y caché<br/>1-4 ns"] --> RAM["Memoria principal<br/>~100 ns"]
        RAM --> SSD["Almacenamiento NVMe<br/>~16 µs"]
        SSD --> RED["Red del centro de datos<br/>~0,5 ms ida y vuelta"]
        RED --> WAN["Enlace intercontinental<br/>~150 ms ida y vuelta"]
    end
    subgraph alquiler["Lo que se alquila"]
        IAAS["IaaS · máquina y disco"] --> PAAS["PaaS · runtime gestionado"]
        PAAS --> SAAS["SaaS · solo datos y configuración"]
    end
    fisico -->|"virtualización y<br/>multi-tenencia"| alquiler
```

Desarrollo

1. La jerarquía de memoria manda sobre el resto del diseño

Un procesador moderno ejecuta del orden de 10¹⁰ instrucciones por segundo y por núcleo, pero pasa la mayor parte del tiempo esperando datos. Los órdenes de magnitud que hay que tener memorizados, popularizados por Jeff Dean en Latency Numbers Every Programmer Should Know:

referencia a caché L1	~1 ns	
referencia a caché L2	~4 ns	
referencia a memoria principal	~100 ns	(100x L1)
lectura aleatoria en SSD NVMe	~16 µs	(160x RAM)
ida y vuelta dentro del centro de datos	~0,5 ms	(31x SSD)
búsqueda en disco mecánico	~2 ms	
ida y vuelta California - Países Bajos	~150 ms	(300x centro de datos)

Entre el primer y el último renglón hay un factor de 150 millones. Esto tiene una consecuencia de diseño que se repetirá en las 288 clases: mover el cómputo hacia los datos casi siempre gana a mover los datos hacia el cómputo. Cuando en la parte 16 se estudien CDN y edge, y en la parte 12 la colocación de réplicas, la razón será siempre esta tabla.

2. La latencia tiene un suelo físico y el ancho de banda no

La luz viaja a 299.792 km/s en el vacío, pero en fibra óptica el índice de refracción la frena a unos 200.000 km/s. Madrid–Nueva York son 5.760 km en línea recta:

mínimo teórico ida	= 5.760 km / 200.000 km/s = 28,8 ms
mínimo teórico ida+vuelta	= 57,6 ms
medido en la práctica	~ 85-100 ms

La diferencia entre 57,6 y 90 ms son enrutamiento no geodésico, colas en routers y conmutación. Puedes optimizar esa parte; los 57,6 ms no. Ninguna inversión hace que una petición Madrid–Nueva York baje de 58 ms. Por eso la respuesta a un problema de latencia rara vez es un servidor más rápido: es una réplica más cerca, una caché, o menos idas y vueltas.

El ancho de banda se comporta al revés: es una mercancía que se compra. Duplicar el caudal no acorta una petición de 2 KB, pero permite atender el doble de ellas.

3. Qué define a la nube según NIST, y qué no

La definición operativa la fija el NIST SP 800-145 (Mell y Grance, 2011) con cinco características esenciales. Un servicio que no cumple las cinco es hosting, por mucho que se anuncie como cloud:

Característica	Prueba concreta
Autoservicio bajo demanda	¿Puedes aprovisionar sin abrir un ticket ni hablar con nadie?
Acceso amplio por red	¿Se consume por protocolos estándar desde cualquier cliente?
Agrupación de recursos	¿El proveedor reasigna capacidad entre clientes de forma opaca?
Elasticidad rápida	¿Escala y libera en minutos, no en semanas?
Servicio medido	¿Se factura por consumo observable y auditable?

La quinta es la que más cuesta interiorizar y la que más aparece en la factura: si el recurso se mide, cada decisión de arquitectura es también una decisión de coste. Un bucle mal escrito en un centro de datos propio es capacidad desperdiciada; el mismo bucle en cloud es una línea en la factura del mes siguiente.

4. Los modelos de servicio son fronteras de responsabilidad

IaaS, PaaS y SaaS no son «niveles de facilidad». Son dónde se corta la responsabilidad operativa, y esa línea determina a quién despiertan de madrugada:

Capa	On-premise	IaaS	PaaS	SaaS
Datos y acceso	tú	tú	tú	tú
Aplicación	tú	tú	tú	proveedor
Runtime y middleware	tú	tú	proveedor	proveedor
Sistema operativo	tú	tú	proveedor	proveedor
Virtualización, servidores, red física	tú	proveedor	proveedor	proveedor

Observa la primera fila: los datos y el control de acceso nunca cambian de dueño. Ese es el núcleo del modelo de responsabilidad compartida que se estudia en la clase 010, y el origen de la mayoría de incidentes públicos atribuidos a «fallos del proveedor» que en realidad fueron configuraciones del cliente.

5. Elasticidad: el cambio económico, no el técnico

La virtualización existe desde los años sesenta (CP-40 de IBM, 1967). Lo que la nube añadió no fue la técnica sino el modelo económico: pasar de CAPEX amortizado a OPEX medido.

Supón un servicio con pico de 100 servidores durante 3 horas al día y 20 el resto:

Compra: $100 \text{ servidores} \times 24 \text{ h} \times 30 \text{ días} = 72.000 \text{ servidor-hora contratadas}$
 Uso real: $(100 \times 3 + 20 \times 21) \times 30 = 21.600 \text{ servidor-hora útiles}$
 utilización = $21.600 / 72.000 = 30 \%$

En compra pagas el 100 % y usas el 30 %. Con elasticidad pagas cerca del 30 %, a un precio unitario mayor. El punto de equilibrio depende de esa relación, no de la moda: si tu carga es plana y predecible, comprar suele salir más barato. Multi-cloud y elasticidad se justifican por requisitos medibles, nunca por defecto.

Ejemplo trabajado

CloudShop necesita responder una página de producto en menos de 300 ms al percentil 95. El equipo desglosa el presupuesto de latencia para un usuario en Santiago de Chile y un despliegue en la región us-east-1 (Virginia), a unos 7.400 km:

resolución DNS (con caché fría)	40 ms	
handshake TLS 1.3 (1 RTT sobre ~120 ms)	120 ms	
petición HTTP ida y vuelta	120 ms	
consulta a base de datos en la misma región	8 ms	
renderizado en servidor	25 ms	

total	313 ms	✗ incumple el SLO

El componente dominante no es la aplicación —33 ms entre consulta y render— sino los 240 ms de red. Optimizar el código no salva el SLO: aunque el render bajara a 0 ms, quedarían 288 ms.

Dos intervenciones sobre la red:

CDN en Santiago para el HTML (RTT 10 ms)		
DNS cacheado	2 ms	
TLS al borde (1 RTT sobre 10)	10 ms	
HTTP al borde	10 ms	
origen solo para datos (1 RTT)	120 ms	
consulta + render	33 ms	

total	175 ms	✓ cumple

La decisión no fue «usar un CDN» sino reconocer que 240 de los 313 ms eran físicos y solo se atacan acercando el punto de terminación. El coste añadido del borde se justifica contra los 138 ms recuperados; si el SLO hubiera sido 500

ms, el gasto no tendría defensa.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/001-computacion-digital-y-modelo-mental-de-la-nube/lab.py
```

El laboratorio selecciona el motor de práctica foundation y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mapa-de-componentes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un mapa con fronteras, entradas, salidas y supuestos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye mapa-de-componentes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El sistema es lento y se responde comprando instancias más grandes	Se confundió latencia con capacidad de cómputo	Desglosa el presupuesto de latencia por componente antes de escalar; si domina la red, más CPU no cambia nada.
«Migramos a la nube» pero la factura supera al centro de datos anterior	Se replicó una carga plana sin usar elasticidad	Calcula la utilización real; si es estable y alta, la nube solo compensa por otros motivos (alcance, servicios gestionados, continuidad).
Se asume que el proveedor protege los datos	Se leyó el modelo de servicio como nivel de comodidad y no como frontera de responsabilidad	Sitúa cada capa en la tabla de responsabilidad; datos y acceso siguen siendo tuyos en IaaS, PaaS y SaaS.
Una prueba local va rápida y en producción no	En local todo cabe en RAM y la red es de 0 ms	Mide con las latencias reales entre zonas y regiones; la jerarquía de memoria del entorno de pruebas no es la de producción.
Se llama cloud a un servidor alquilado por meses	No se aplicaron las cinco características de NIST	Comprueba autoservicio, elasticidad y medición; si faltan, las decisiones de coste y escala no aplicarán.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuántos órdenes de magnitud separan una referencia a caché L1 de una ida y vuelta intercontinental, y qué decisión de arquitectura se deriva de esa diferencia?
2. Un servicio tarda 400 ms y el equipo propone duplicar la CPU. ¿Qué medición pedirías antes de aprobar el gasto?
3. ¿Cuál de las cinco características de NIST es la que convierte cada decisión técnica en una decisión de coste, y por qué?
4. En un despliegue PaaS, ¿qué sigue siendo responsabilidad tuya y qué pasa a ser del proveedor?
5. Una carga usa el 85 % de su capacidad las 24 horas. ¿Qué argumento económico queda a favor de la nube, si es que queda alguno?

Referencias

- Mell, P. y Grance, T. (2011). The NIST Definition of Cloud Computing, SP 800-145. National Institute of Standards and Technology. <<https://doi.org/10.6028/NIST.SP.800-145>>
- Dean, J. (2009). Latency Numbers Every Programmer Should Know — cifras de referencia de la jerarquía de memoria. <<https://static.googleusercontent.com/media/research.google.com/en//people/jeff/stanford-295-talk.pdf>>
- Kurose, J. y Ross, K. (2021). Computer Networking: A Top-Down Approach, 8.^a ed., cap. 1.4 — retardo, pérdida y throughput.
- Barroso, L., Hölzle, U. y Ranganathan, P. (2018). The Datacenter as a Computer, 3.^a ed. — economía y utilización del centro de datos. <<https://doi.org/10.2200/S00874ED3V01Y201809CAC046>>
- Gray, J. (2008). Distributed Computing Economics. ACM Queue 6(3) — por qué conviene mover el cómputo hacia los datos. <<https://doi.org/10.1145/1394127.1394131>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 001 Computación digital y modelo mental de la nube

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega mapa-de-componentes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.

- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

002 — Terminal, sistema de archivos, procesos y variables de entorno

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: shell · Estado: EXECUTABLECORE

Propósito

Dominar la terminal como interfaz primaria de operación cloud: qué es realmente un proceso, cómo el kernel le entrega su entorno, y por qué los descriptores de fichero y las variables de entorno explican la mitad de los incidentes de despliegue. Toda la automatización posterior —contenedores, CI/CD, runbooks— es este modelo repetido a escala.

Resultados de aprendizaje

Al finalizar podrás:

1. Describir el ciclo `fork` → `exec` → `wait` y qué hereda un proceso hijo de su padre.
2. Predecir qué variables de entorno ve un proceso según cómo se lanzó, y por qué un servicio `systemd` no ve tu `.bashrc`.
3. Redirigir `stdout` y `stderr` por separado y explicar por qué `2>&1 >f` no equivale a `>f 2>&1`.
4. Interpretar un código de salida y una señal de terminación para diagnosticar sin adivinar.
5. Componer una tubería que resuelva un problema real de operación sin scripts intermedios.

Conceptos centrales

Concepto	Comprensión verificable
proceso	Instancia en ejecución de un programa, con su propio espacio de direcciones, tabla de descriptores y entorno. El kernel lo identifica por PID y lo relaciona con su padre por PPID.
descriptor de fichero	Entero que indexa la tabla de ficheros abiertos del proceso. Por convención 0 es <code>stdin</code> , 1 <code>stdout</code> y 2 <code>stderr</code> ; el resto se asignan por orden. Redirigir es duplicar entradas de esa tabla, no mover datos.
variable de entorno	Par clave-valor copiado del padre al hijo en el momento del <code>exec</code> . La copia es unidireccional: un hijo no puede modificar el entorno del padre, lo que explica por qué <code>cd</code> debe ser un builtin del shell.
código de salida	Entero de 0 a 255 que devuelve un proceso al terminar. 0 significa éxito; 1-125 son errores del programa; 126 y 127 indican no ejecutable y no encontrado; 128+N significa terminado por la señal N.
señal	Notificación asíncrona del kernel a un proceso. <code>SIGTERM</code> (15) pide terminar y puede atraparse; <code>SIGKILL</code> (9) no es atrapable. Esa diferencia es la base del apagado ordenado en contenedores.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    S["shell (PID 4210)"] -->|fork()| H["hijo: copia del padre<br/>mismo entorno y descriptores"]
    H -->|exec()| P["nuevo programa<br/>conserva PID, descriptores y entorno"]
    P --> R["termina"]
    R -->|exit(0)| OK["código 0 · éxito"]
    R -->|exit(n)| ERR["código n · error del programa"]
    R -->|señal N| SIG["código 128+N"]
    OK --> W["shell recoge con wait()"]
    ERR --> W
    SIG --> W
```

Desarrollo

1. Un proceso nace por copia y se transforma por reemplazo

En UNIX no existe «lanzar un programa» como operación única. Son dos llamadas:

1. `fork()` duplica el proceso actual. El hijo recibe una copia del espacio de direcciones, la tabla de descriptores y el entorno. Devuelve 0 en el hijo y el PID del hijo en el padre; ese es el único modo que tiene cada uno de saber quién es.
2. `exec()` reemplaza la imagen del proceso por otro programa conservando el PID, los descriptores abiertos y el entorno.

```
$ echo $$          # PID del shell actual
4210
$ bash -c 'echo $$; echo $PPID'
4877               # el hijo tiene PID nuevo
4210               # y recuerda a su padre
```

La consecuencia práctica aparece en cuanto se automatiza: un proceso solo puede heredar hacia abajo. Por eso `cd` no puede ser un binario —cambiaría el directorio de un hijo que muere inmediatamente— y por eso exportar una variable en un script no la deja disponible en la terminal que lo invocó.

2. El entorno se copia en el `exec`, no se consulta en vivo

El entorno es un vector de cadenas `CLAVE=valor` que se pasa al `exec`. Se copia una vez, en ese instante. Un proceso que lleva tres días corriendo tiene el entorno del momento en que arrancó, aunque el fichero de configuración haya cambiado.

```
$ VAR=solo_este_comando env | grep VAR    # prefijo: solo para esa invocación
VAR=solo_este_comando
$ VAR=local; env | grep VAR               # sin export: no llega al hijo
$ export VAR=heredable; env | grep VAR    # con export: sí llega
VAR=heredable
```

Esto explica el incidente más repetido en despliegues: un servicio gestionado por `systemd` no lee `./bashrc` ni `./profile`. Esos ficheros los interpreta el shell al iniciar sesión de forma interactiva, y `systemd` no arranca un shell de login. El comando funciona a mano y falla como servicio, con la misma imagen y el mismo binario. La variable hay que declararla en la unidad (`Environment=` o `EnvironmentFile=`), y en contenedores en el `ENV` del `Dockerfile` o el manifiesto.

3. Redirección: se duplican descriptores, y el orden importa

> no «envía» la salida a ningún sitio: hace que el descriptor 1 apunte al mismo fichero abierto que otro. Como es una asignación secuencial, el orden cambia el resultado:

```
# stderr se copia al destino ACTUAL de stdout (la terminal) y luego stdout va al fichero
$ comando 2>&1 > salida.log          # errores a la terminal, salida al fichero

# stdout va al fichero y DESPUÉS stderr se copia a ese mismo destino
$ comando > salida.log 2>&1          # ambos al fichero
```

Es la causa clásica de un log que «pierde» los errores. En Bash moderno, `&>` fichero hace lo correcto sin ambigüedad.

La separación `stdout/stderr` no es cosmética: es un contrato. `stdout` lleva el resultado destinado a la siguiente etapa de la tubería; `stderr` lleva diagnóstico destinado al operador. Un programa que escribe avisos en `stdout` rompe cualquier tubería que lo consuma.

4. Códigos de salida y señales: el lenguaje del diagnóstico

El código de salida es el único canal estructurado que un proceso tiene para decir qué pasó. Sus rangos están convenidos:

Código	Significado	Aparece cuando
0	Éxito	Todo bien
1-125	Error del programa	Fallo de lógica o validación
126	Encontrado pero no ejecutable	Falta el bit de ejecución
127	Orden no encontrada	PATH incorrecto — típico en contenedores
128+N	Terminado por la señal N	137 = 128+9 (SIGKILL), 143 = 128+15 (SIGTERM)

El 137 merece memorizarse: en Kubernetes casi siempre significa que el contenedor excedió su límite de memoria y el OOM killer lo mató. En la parte 06 aparecerá con ese nombre; aquí ya sabes de dónde sale el número.

La diferencia entre SIGTERM y SIGKILL define el apagado ordenado: el orquestador envía SIGTERM, espera un plazo de gracia (30 s por defecto en Kubernetes) y solo entonces envía SIGKILL. Un proceso que ignora SIGTERM pierde las conexiones en vuelo.

5. Tuberías: procesos concurrentes, no secuenciales

`a | b` no ejecuta `a` y después `b`. Arranca ambos a la vez y conecta el descriptor 1 de `a` con el 0 de `b` mediante un búfer del kernel de 64 KB. Cuando el búfer se llena, el escritor se bloquea; cuando se vacía, el lector se bloquea. Es control de flujo gratuito, y permite procesar ficheros mayores que la memoria disponible.

```
# Los cinco procesos que más memoria residente consumen
$ ps -eo pid,rss,comm --sort=-rss | head -6

# Las 10 IP con más peticiones en un log de acceso
$ awk '{print $1}' access.log | sort | uniq -c | sort -rn | head -10
```

El código de salida de una tubería es el del último comando, lo que oculta fallos intermedios. En scripts de operación esto es una trampa seria:

```
$ false | true; echo $?
0                                # el fallo desaparece
$ set -o pipefail
$ false | true; echo $?
1                                # ahora se propaga
```

`set -euo pipefail` al principio de cada script de despliegue evita que un fallo silencioso se declare éxito.

Ejemplo trabajado

Un despliegue de CloudShop funciona a mano y falla como servicio. El binario es el mismo y la imagen también. El operador ejecuta el diagnóstico en orden:

```
$ ./cloudshop-api
escuchando en :8080                                # a mano funciona

$ sudo systemctl start cloudshop
$ systemctl show cloudshop -p ExecMainStatus
ExecMainStatus=127
```

127 = orden no encontrada. No es un fallo de la aplicación: es que algo del PATH no está. Se compara el entorno de ambos contextos:

```
$ echo $PATH
/home/ops/.local/bin:/usr/local/bin:/usr/bin:/bin

$ sudo systemctl show cloudshop -p Environment
Environment=
$ sudo systemd-run --quiet --pipe /usr/bin/env | grep ^PATH
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

La diferencia es /home/ops/.local/bin, que el shell interactivo añade desde /.profile y systemd no. El binario auxiliar que invoca el servicio vive ahí.

Se corrige declarando la dependencia en la unidad, no exportando en un perfil:

```
[Service]
Environment="PATH=/usr/local/bin:/usr/bin:/bin"
ExecStart=/opt/cloudshop/bin/cloudshop-api

$ sudo systemctl restart cloudshop; systemctl show cloudshop -p ExecMainStatus
ExecMainStatus=0
```

La lección no es el arreglo sino el método: el código 127 acotó el problema a resolución de ruta antes de leer una sola línea de la aplicación. Un diagnóstico sin ese dato habría empezado por los logs de la aplicación, donde no había nada que ver.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/002-terminal-sistema-de-archivos-
procesos-y-variables-de-entorno/lab.py
```

El laboratorio selecciona el motor de práctica shell y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa bitacora-de-comandos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una bitácora reproducible de comandos y resultados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye bitacora-de-comandos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El comando funciona en la terminal y falla como servicio o en el contenedor	El entorno interactivo carga perfiles que systemd y Docker no leen	Declara las variables en la unidad (Environment=) o en el ENV de la imagen; nunca dependas de .bashrc.
El fichero de log no contiene los errores	Se escribió 2>&1 > f, que copia stderr al destino anterior de stdout	Usa > f 2>&1 o &> f; el orden de la redirección es una asignación secuencial.
Un script de despliegue termina en éxito pese a que un paso falló	El código de salida de una tubería es solo el del último comando	Empieza los scripts con set -euo pipefail.
El contenedor muere con código 137 y no hay error en la aplicación	128+9: fue SIGKILL, casi siempre el OOM killer por exceder el límite de memoria	Revisa el límite de memoria y el consumo real; el problema es de recursos, no de código.
El servicio pierde peticiones en vuelo en cada despliegue	El proceso no atrapa SIGTERM y se le aplica SIGKILL al agotar el plazo de gracia	Atrapa SIGTERM, deja de aceptar conexiones nuevas y drena las abiertas antes de salir.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué cd tiene que ser un builtin del shell y no puede ser un binario en /usr/bin?
2. Un proceso lleva tres días ejecutándose y alguien cambia una variable en /etc/environment. ¿La ve el proceso? ¿Por qué?
3. ¿Qué diferencia práctica hay entre comando 2>&1 > f y comando > f 2>&1?
4. Un contenedor termina con código 143 y otro con 137. ¿Cuál se apagó ordenadamente y cuál no?
5. ¿Qué añade set -o pipefail que set -e por sí solo no cubre?

Referencias

- The Open Group (2018). POSIX.1-2017, System Interfaces: fork, execve, wait.
<<https://pubs.opengroup.org/onlinepubs/9699919799/functions/fork.html>>
- Kerrisk, M. signal(7) — Linux manual pages: catálogo de señales y semántica de SIGTERM y SIGKILL.
<<https://man7.org/linux/man-pages/man7/signal.7.html>>
- Kerrisk, M. (2010). The Linux Programming Interface, caps. 24-27 — creación de procesos y ejecución de programas.
- systemd (2024). systemd.exec(5) — cómo se construye el entorno de un servicio.
<<https://www.freedesktop.org/software/systemd/man/systemd.exec.html>>
- Ward, B. (2021). How Linux Works, 3.^a ed., caps. 1-2 — procesos, dispositivos y arranque.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 002 Terminal, sistema de archivos, procesos y variables de entorno

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega bitacora-de-comandos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

003 — Git, GitHub y trabajo reproducible

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: git · Estado: EXECUTABLECORE

Propósito

Entender Git como un grafo dirigido acíclico de instantáneas inmutables direccionadas por contenido, no como una lista de parches. Esa diferencia explica por qué rebase, revert y reset hacen lo que hacen, y es la base de todo lo que viene después: GitOps, entrega continua, trazabilidad de auditoría y reproducibilidad de un despliegue.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cómo el hash SHA-1/SHA-256 de un commit se deriva de árbol, padres, autor y mensaje, y qué implica para la integridad del historial.
2. Distinguir reset --soft, --mixed y --hard por lo que hacen sobre HEAD, índice y árbol de trabajo.
3. Elegir entre merge, rebase y revert según si la rama es compartida y si el historial debe ser auditable.
4. Recuperar trabajo aparentemente perdido usando reflog, entendiendo por qué existe la ventana de recuperación.
5. Diseñar un flujo de ramas trazable que permita responder «qué código exacto había en producción el martes a las 14:00».

Conceptos centrales

Concepto	Comprensión verificable
objeto	Unidad inmutable de almacenamiento en Git, nombrada por el hash de su contenido. Hay cuatro tipos: blob (contenido de fichero), tree (directorio), commit (instantánea con metadatos) y tag anotado.
commit	Objeto que apunta a un árbol completo, a cero o más padres, y lleva autor, fecha y mensaje. No guarda un diff: guarda el estado entero. Los diffs se calculan al vuelo comparando árboles.
índice	Área intermedia entre el árbol de trabajo y el repositorio, también llamada staging. Es un fichero binario que lista qué versión de cada ruta entrará en el próximo commit.
referencia	Puntero mutable a un commit. Las ramas y HEAD son referencias; por eso crear una rama cuesta 41 bytes y es una operación instantánea sea cual sea el tamaño del repositorio.
reflog	Registro local de todos los valores por los que ha pasado cada referencia. Permite recuperar commits que ya no alcanza ninguna rama, durante 90 días por defecto.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart RL
    subgraph objetos["Base de objetos · inmutable"]
        C3["commit c3f9a1<br/>árbol + padre + autor"] --> C2["commit 8b2e40"]
        C2 --> C1["commit 1a7dd3"]
        C3 --> T["tree raíz"]
        T --> B1["blob README"]
        T --> B2["blob main.py"]
    end
    subgraph refs["Referencias · mutables"]
        MAIN["main"] --> C3
        HEAD["HEAD"] --> MAIN
        TAG["v2.0.0"] --> C2
    end
```

Desarrollo

1. Git guarda instantáneas, no diferencias

La intuición equivocada más extendida es que un commit almacena «lo que cambió». Almacena el árbol completo del proyecto en ese instante. Los ficheros que no cambiaron no se duplican: el árbol nuevo reutiliza los mismos objetos blob, porque están nombrados por el hash de su contenido.

```
$ git cat-file -p HEAD
tree a4f2c81e9b3d5a7c2f8e1b6d4a9c3e7f2b8d5a1c
parent 8b2e40d7c1a5f9e3b6d2a8c4f7e1b9d3a5c8e2f4
author Vladimir Acuña <...> 1785546717 -0300
committer Vladimir Acuña <...> 1785546717 -0300
```

Optimiza el README principal

El hash del commit se calcula sobre exactamente ese texto. Cambiar una coma del mensaje, la fecha o el padre produce un hash distinto. De ahí se sigue la propiedad que sostiene toda auditoría: no se puede alterar historia pasada sin cambiar los identificadores de todo lo que vino después. Si alguien reescribe un commit de hace un mes, los 200 commits posteriores cambian de hash y cualquier clon lo detecta.

2. Tres zonas y el comando que las mueve

Todo el modelo operativo de Git cabe en tres zonas y una tabla:

```
árbol de trabajo → índice (staging) → repositorio (objetos)
git add ■■■■■■■■      git commit ■■■■■■■■
```

git reset es un único comando que actúa sobre las tres según el modificador, y confundirlos es la causa habitual de trabajo perdido:

Comando	HEAD	Índice	Árbol de trabajo	Uso típico
reset --soft <c>	mueve	intacto	intacto	Rehacer el mensaje o agrupar commits
reset --mixed <c>	mueve	reinicia	intacto	Deshacer un add (por defecto)
reset --hard <c>	mueve	reinicia	descarta	Descartar todo; es el único destructivo

Solo --hard toca el árbol de trabajo. Los cambios no confirmados que destruye no están en ningún objeto, así que el reflog no los recupera: nunca llegaron a existir para Git.

3. Merge, rebase y revert: tres respuestas a preguntas distintas

No son alternativas de estilo. Responden a preguntas diferentes:

- merge crea un commit con dos padres. Preserva la historia tal como ocurrió y no cambia hashes existentes. Es la única opción segura sobre una rama que otros ya tienen.
- rebase reescribe: toma tus commits y los vuelve a aplicar sobre otra base, creando commits nuevos con hashes nuevos. Produce historia lineal y legible, pero si la rama es compartida, todo el que la tenga verá divergencia.
- revert crea un commit nuevo que deshace los efectos de otro. No borra nada. Es lo único aceptable para deshacer algo que ya está en producción o en una rama protegida.

La regla operativa, conocida como golden rule of rebasing: no reescribas historia que otros puedan tener. En la práctica: rebase libremente en tu rama local antes del primer push; después, merge o revert.

Para un repositorio con requisitos de auditoría —los de la parte 11— revert es obligatorio: deja constancia de que hubo un error y de cuándo se corrigió. Un reset --hard seguido de push --force borra esa evidencia.

4. El reflog: por qué casi nada se pierde de verdad

Cuando una rama deja de apuntar a un commit, ese commit queda huérfano: sigue en la base de objetos pero no lo alcanza ninguna referencia. El reflog registra cada movimiento de cada referencia, así que aún se puede llegar a él.

```
$ git reset --hard HEAD~3          # "perdí" tres commits
$ git reflog
c3f9a1 HEAD@{0}: reset: moving to HEAD~3
7d2b8e HEAD@{1}: commit: añade validación de contratos
$ git reset --hard HEAD@{1}       # recuperados
```

Los objetos huérfanos sobreviven hasta que git gc los recoge: 90 días para los alcanzables desde el reflog y 30 para el resto, según gc.reflogExpire. Dos matices que importan en operación:

1. El reflog es estrictamente local. No se clona ni se envía. Un push --force que destruye commits en el servidor no deja reflog para quien clone después.
2. Solo registra lo que llegó a ser un commit. Cambios en el árbol de trabajo que nunca se confirmaron no aparecen.

5. Trazabilidad: del commit al artefacto desplegado

La pregunta que toda auditoría acaba haciendo es: ¿qué código exacto estaba corriendo cuando ocurrió el incidente? Responderla exige una cadena sin huecos:

```
commit (sha) → build (id) → artefacto (digest) → despliegue (revisión) → instante
```

Cada eslabón debe ser inmutable y verificable. Por eso en las partes 05 y 08 se insistirá en referenciar imágenes por digest (sha256:...) y no por etiqueta: una etiqueta como :latest o :v2.0 es una referencia mutable —igual que una rama de Git— y puede apuntar a contenido distinto mañana.

El equivalente en Git es el tag anotado, que sí es un objeto con su propio hash, autor y mensaje, frente al tag ligero que es solo un puntero:

```
$ git tag -a v2.0.0 -m "Release 2.0.0"    # objeto, firmable y con metadatos
$ git tag v2.0.0                          # solo una referencia más
```

Para releases, el tag anotado y firmado (-s) es el que sostiene la afirmación «esto es lo que publicamos».

Ejemplo trabajado

Un despliegue del martes rompió el checkout de CloudShop y hay que responder tres preguntas: qué se desplegó, quién lo aprobó y cómo se revierte sin perder la evidencia.

Paso 1 — identificar el commit exacto que estaba en producción:

```
$ git log --format='%h %ad %an %s' --date=iso -3 v2.4.1
7d2b8e 2026-07-28 14:02:11 -0300 Ana Ruiz   Cambia el cálculo de impuestos
1a7dd3 2026-07-28 11:40:02 -0300 Luis Vega Añade índice a pedidos
8b2e40 2026-07-27 17:15:44 -0300 Ana Ruiz   Actualiza dependencias
```

Paso 2 — confirmar que el artefacto desplegado corresponde a ese commit. La etiqueta no basta; se compara el digest:

```
$ git rev-parse v2.4.1
7d2b8e91c4a2f7d5b3e8a1c6f9d2b4e7a5c8f1d3
$ kubectl get deploy cloudshop -o jsonpath='{...image}'
registry/cloudshop@sha256:4f2a9c...
$ crane config registry/cloudshop@sha256:4f2a9c... | jq -r '.config.Labels."org.opencontainers.image.revision"'
7d2b8e91c4a2f7d5b3e8a1c6f9d2b4e7a5c8f1d3      # coincide
```

Paso 3 — aislar el cambio culpable con búsqueda binaria sobre el historial:

```
$ git bisect start v2.4.1 v2.4.0
$ git bisect run ./tests/checkout_smoke.sh
7d2b8e91 is the first bad commit
```

Con 3 commits bastan 2 pruebas; con 1.000 bastarían 10, porque bisect es logarítmico: $\lceil \log_2(1000) \rceil = 10$.

Paso 4 — revertir sin borrar:

```
$ git revert 7d2b8e -m "Revierte el cálculo de impuestos: rompe el checkout"
[main 9e4c2a] Revert "Cambia el cálculo de impuestos"
```

El historial conserva ahora el error y su corrección. Un `reset --hard 1a7dd3 && push --force` habría dejado producción igual de sana y la auditoría sin nada que leer: no habría constancia de que el fallo existió, ni de cuánto tardó en detectarse.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/003-git-github-y-trabajo-reproducible/
lab.py
```

El laboratorio selecciona el motor de práctica git y produce `labresult.json`. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee `exercise.steps` y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de `negativetest` y explica la señal observada.
4. Materializa repositorio-reproducible en `evidence/` usando la plantilla indicada.
5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un historial pequeño, legible y verificable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye repositorio-reproducible para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Tras un rebase sobre una rama compartida, el equipo ve commits duplicados y conflictos repetidos	Se reescribieron hashes que otros ya tenían	Rebase solo antes del primer push; sobre ramas compartidas usa merge.
Se perdió trabajo con <code>reset --hard</code> y el reflog no lo recupera	Los cambios nunca se confirmaron, así que no existe objeto que recuperar	Confirma o usa <code>git stash</code> antes de cualquier operación destructiva.
Nadie puede decir qué código había en producción durante el incidente	El artefacto se referenció por etiqueta mutable en vez de por digest	Referencia imágenes por sha256: y graba el commit en una etiqueta OCI del artefacto.
El historial de la rama principal no permite auditar una corrección urgente	Se usó <code>reset --hard</code> y <code>push --force</code> en lugar de <code>revert</code>	Deshaz siempre con <code>revert</code> en ramas protegidas; deja el error visible junto a su corrección.
Un <code>git tag</code> no lleva autor ni fecha y no se puede firmar	Se creó un tag ligero, que es solo un puntero	Usa <code>git tag -a</code> (o <code>-s</code> para firmarlo) en cualquier release.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Si un commit guarda el árbol completo, ¿por qué un repositorio con 10.000 commits no ocupa 10.000 veces el tamaño del proyecto?
2. ¿Cuál de los tres modos de `reset` puede destruir trabajo que el reflog no recuperará, y por qué precisamente ese?
3. Un compañero ya hizo pull de tu rama. ¿Qué te impide hacer rebase sobre ella y qué usarías en su lugar?
4. ¿Por qué desplegar `imagen:v2.0` no permite responder qué código corría el martes, y qué referencia lo permitiría?
5. Con 1.024 commits entre una versión buena y una mala, ¿cuántas pruebas necesita `git bisect` en el peor caso?

Referencias

- Chacon, S. y Straub, B. (2014). Pro Git, 2.^a ed., cap. 10 «Git Internals» — objetos, referencias y modelo de almacenamiento. <<https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>>
- Git (2024). `git-reset(1)` — tabla oficial del efecto de cada modo sobre HEAD, índice y árbol de trabajo. <<https://git-scm.com/docs/git-reset#resetpathspect>>
- Git (2024). `git-bisect(1)` — búsqueda binaria automatizada sobre el historial. <<https://git-scm.com/docs/git-bisect>>
- Open Container Initiative (2024). Image Specification — anotaciones estándar, incluida `org.opencontainers.image.revision`. <<https://github.com/opencontainers/image-spec/blob/main/annotations.md>>
- Git (2024). `gitrevisions(7)` — sintaxis de `HEAD@{n}`, y `^` para navegar el grafo. <<https://git-scm.com/docs/gitrevisions>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 003 Git, GitHub y trabajo reproducible

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.

2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega repositorio-reproducible con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

004 — Python, JSON y automatización mínima

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: automation · Estado: EXECUTABLECORE

Propósito

Usar Python y JSON como el pegamento mínimo de la operación cloud: leer una respuesta de API, validar su forma antes de confiar en ella y producir salida estructurada que otro programa pueda consumir. Casi todos los lab.py del programa emiten JSON, y casi toda automatización real consiste en encadenar contratos de este tipo.

Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir los tipos de JSON de los de Python y anticipar las conversiones que rompen datos (enteros grandes, None, claves no textuales).
2. Explicar por qué los números en coma flotante de JSON no representan dinero y qué usar en su lugar.
3. Validar una estructura recibida antes de usarla, en vez de confiar en que la API cumple su documentación.
4. Producir salida determinista: mismas entradas y misma semilla producen bytes idénticos.
5. Diferenciar un fallo esperado, que se maneja, de uno inesperado, que debe propagarse con contexto.

Conceptos centrales

Concepto	Comprensión verificable
serialización	Convertir estructuras en memoria a una secuencia de bytes transportable. JSON solo admite seis tipos: objeto, array, cadena, número, booleano y null; todo lo demás debe traducirse explícitamente.
contrato de datos	Acuerdo explícito sobre qué campos existen, de qué tipo son y cuáles son obligatorios. Sin él, un cambio en el productor rompe al consumidor en producción y no en pruebas.
determinismo	Propiedad por la que las mismas entradas producen exactamente la misma salida. Exige orden estable de claves, semilla fija en cualquier aleatoriedad y ausencia de marcas de tiempo en la salida comparada.
idempotencia	Propiedad por la que repetir una operación no cambia el resultado respecto de ejecutarla una vez. Es lo que hace seguro reintentar tras un error de red.
excepción	Mecanismo para señalar que una operación no puede completarse. Capturar Exception de forma genérica convierte un fallo diagnosticable en uno silencioso.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    API["Respuesta HTTP<br/>bytes"] --> D["json.loads()"]
    D --> V{"¿cumple el contrato?"}
    V -->|"no"| E["error con contexto:<br/>campo, tipo esperado, recibido"]
    V -->|"sí"| L["lógica de negocio"]
    L --> S["json.dumps(sort_keys=True)"]
    S --> O["salida determinista<br/>comparable byte a byte"]
```

Desarrollo

1. JSON y Python no comparten sistema de tipos

El mapeo parece directo hasta que deja de serlo:

JSON	Python al leer	Trampa
object	dict	Las claves siempre acaban siendo str
array	list	Las tuplas se serializan como array y vuelven como lista
number	int o float	No hay distinción en JSON: la decide el punto decimal
true/false	bool	—
null	None	—

La conversión no es reversible:

```
>>> import json
>>> json.loads(json.dumps({1: "a", (2, 3): "b"}))
{'1': 'a', '2, 3': 'b'}      # las claves se volvieron cadenas
>>> json.loads(json.dumps((1, 2)))
[1, 2]                      # la tupla ya no es tupla
```

En una automatización que lee un estado, lo modifica y lo vuelve a escribir, esto significa que el ciclo no es una identidad. Si el consumidor compara claves numéricas, el segundo ciclo falla y el primero no.

2. Los números de JSON no sirven para dinero

JSON no define precisión: la mayoría de implementaciones usan IEEE 754 de doble precisión, que es binario. Los decimales que no son suma de potencias de dos no tienen representación exacta:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

Aplicado a una factura cloud con 43.200 líneas de consumo horario, el error se acumula. La corrección es no usar coma flotante para valores exactos:

```
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2") == Decimal("0.3")
True
```

Hay un segundo límite, y es de interoperabilidad: JavaScript representa todos los números como double, así que solo garantiza enteros exactos hasta $2^{53}-1 = 9.007.199.254.740.991$. Un identificador de 64 bits enviado como número JSON pierde precisión al pasar por cualquier consumidor JavaScript. Por eso las APIs serias envían los identificadores grandes como cadena; en la parte 09 se verá la misma decisión en los sistemas de mensajería.

3. Validar antes de confiar

El error de automatización más caro es asumir que la respuesta tiene la forma documentada. Una API puede devolver 200 con un cuerpo distinto del esperado, y el fallo aparece tres capas más abajo, sin contexto:

```
# frágil: revienta en otro sitio, con un KeyError sin pistas
costo = respuesta["data"]["billing"]["total"]

# explícito: falla donde está el problema y dice cuál es
def leer_costo(respuesta: dict) -> Decimal:
    for clave in ("data", "billing"):
        if clave not in respuesta:
            raise ValueError(f"falta '{clave}' en la respuesta; claves: {sorted(respuesta)}")
        respuesta = respuesta[clave]
    bruto = respuesta.get("total")
    if not isinstance(bruto, str):
        raise TypeError(f"'total' debía ser cadena decimal, llegó {type(bruto).__name__}")
    return Decimal(bruto)
```

La diferencia práctica: el primero produce `KeyError: 'billing'` a las 3 de la madrugada; el segundo dice qué campo falta y qué llegó en su lugar. En un runbook —parte 21— esa distinción es la diferencia entre cinco minutos y dos horas.

4. Determinismo: la propiedad que hace verificable un laboratorio

Los 288 laboratorios de este programa emiten un contrato JSON que debe ser byte a byte idéntico con la misma semilla. Tres condiciones lo garantizan:

```
import json, random

random.seed(42) # 1. aleatoriedad reproducible
resultado = {"decision": "...", "evidencia": [...]}
print(json.dumps(resultado,
                  sort_keys=True, # 2. orden estable de claves
                  ensure_ascii=False)) # 3. codificación estable
```

Sin sortkeys, Python conserva el orden de inserción y dos ejecuciones que construyen el diccionario por caminos distintos producen bytes distintos con el mismo contenido. Sin semilla, no hay repetición posible.

Lo que nunca debe entrar en una salida comparada es la hora actual: convierte cualquier comparación en un falso negativo. Si hace falta registrar el instante, va en un fichero aparte o en un campo excluido de la comparación.

5. Fallos esperados frente a fallos inesperados

Un error de red al llamar a una API es esperado: ocurre, y el código debe reintentar. Un TypeError porque una función recibió una lista donde esperaba un diccionario es inesperado: es un defecto, y ocultarlo lo hace indetectable.

```
# destruye el diagnóstico: cualquier fallo se vuelve None
try:
    datos = pedir(url)
except Exception:
    datos = None

# maneja lo esperado y deja propagar lo demás
for intento in range(5):
    try:
        datos = pedir(url)
        break
    except (TimeoutError, ConnectionError) as e:
        espera = 2 ** intento + random.uniform(0, 1) # 1, 2, 4, 8, 16 s + jitter
        log.warning("intento %d falló (%s); reintento en %.1f s", intento + 1, e, espera)
        time.sleep(espera)
else:
    raise RuntimeError(f"{url} no respondió tras 5 intentos")
```

El retroceso exponencial con jitter no es un detalle: sin el término aleatorio, mil clientes que fallan a la vez reintentan a la vez y mantienen caído el servicio que intentan usar. Es el thundering herd, y reaparece en las partes 10 y 21.

Ejemplo trabajado

Un script de FinOps suma el consumo horario de CloudShop y su total no cuadra con la factura del proveedor. La diferencia es de 0,37 USD sobre 8.412,55 — pequeña, pero impide cerrar el mes.

El script original:

```
total = 0.0
for linea in json.loads(respuesta)["lineItems"]:
    total += linea["cost"] # cost llega como número JSON
print(f"{total:.2f}")
```

Reproducción del error con las 720 líneas de un mes:

```
>>> valores = [0.0117] * 720 # coste horario de una instancia pequeña
>>> sum(valores)
8.423999999999999
>>> float(Decimal("0.0117") * 720)
8.424
```

Sobre una línea el error es de 10⁻⁴; sobre 43.200 líneas de consumo mensual —60 recursos × 720 horas— el error acumulado alcanza el orden de los céntimos. No es un fallo del proveedor: es que la suma se hizo en binario.

Corrección, sumando en decimal y validando el tipo de entrada:

```
from decimal import Decimal, ROUND_HALF_UP

total = Decimal("0")
```

```

for n, linea in enumerate(json.loads(respuesta)["lineItems"]):
    bruto = linea["cost"]
    if isinstance(bruto, float):
        raise TypeError(f"línea {n}: 'cost' llegó como float; exige cadena decimal")
    total += Decimal(str(bruto))
print(total.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP))

8412.55          # cuadra con la factura

```

La comprobación de tipo es deliberada: convierte un error silencioso de céntimos en un fallo ruidoso en la primera línea. Si el proveedor cambia el formato y empieza a enviar cost como número, el script se para en vez de producir un total plausible pero incorrecto.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/004-python-json-y-automatizacion-minima/lab.py
```

El laboratorio selecciona el motor de práctica automation y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa script-de-diagnostico en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un script idempotente con salida estructurada. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye script-de-diagnostico para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un total monetario difiere en céntimos de la factura oficial	Se sumó en coma flotante IEEE 754, que no representa decimales exactos	Usa Decimal construido desde cadena y redondea solo al final.
Un identificador de 64 bits llega alterado al frontend	JavaScript solo garantiza enteros exactos hasta $2^{53}-1$	Transporta los identificadores grandes como cadena JSON.
Dos ejecuciones con la misma semilla producen ficheros distintos	El orden de claves depende del orden de inserción, o hay una marca de tiempo en la salida	Serializa con <code>sortkeys=True</code> y saca la hora de la salida comparada.
El script falla con <code>KeyError</code> en un punto que no tiene relación con la causa	Se accedió a campos anidados sin validar la forma recibida	Valida presencia y tipo en el borde, y falla con un mensaje que diga qué faltaba.
Tras un incidente, mil clientes reintentan a la vez y el servicio no levanta	Retroceso sin jitter: todos reintentan sincronizados	Añade un término aleatorio al retroceso exponencial.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué `json.loads(json.dumps(d))` puede no devolver `d`? Da dos casos concretos.
2. Un endpoint devuelve `"id": 9007199254740993`. ¿Qué le ocurre a ese valor en un consumidor JavaScript y cómo se evita?
3. ¿Qué tres condiciones debe cumplir un laboratorio para que su salida sea comparable byte a byte?
4. ¿Cuándo es correcto capturar una excepción y cuándo hacerlo oculta un defecto?
5. Sin jitter, ¿qué le ocurre a un servicio que se recupera si mil clientes usan el mismo retroceso exponencial?

Referencias

- Bray, T., ed. (2017). RFC 8259: The JavaScript Object Notation (JSON) Data Interchange Format — sección 6 sobre los límites de precisión numérica. <<https://www.rfc-editor.org/rfc/rfc8259#section-6>>
- IEEE (2019). Standard for Floating-Point Arithmetic (IEEE 754-2019). <<https://doi.org/10.1109/IEEESTD.2019.8766229>>
- Goldberg, D. (1991). What Every Computer Scientist Should Know About Floating-Point Arithmetic. ACM Computing Surveys 23(1). <<https://doi.org/10.1145/103162.103163>>
- Python Software Foundation (2024). decimal — Decimal fixed-point and floating-point arithmetic. <<https://docs.python.org/3/library/decimal.html>>
- Brooker, M. (2015). Exponential Backoff and Jitter. AWS Architecture Blog — datos comparados de las variantes de jitter. <<https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 004 Python, JSON y automatización mínima

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.

2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega script-de-diagnostico con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

005 — Redes por capas, TCP/IP, puertos y sockets

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Entender por qué una red se organiza en capas y qué garantiza cada una, para poder diagnosticar sin adivinar. Cuando en la parte 16 aparezcan VPC, tablas de rutas, NAT y balanceadores, todo será este modelo aplicado; y cuando un servicio «no responda», sabrás en qué capa mirar antes de tocar nada.

Resultados de aprendizaje

Al finalizar podrás:

1. Situar un síntoma en la capa correcta y elegir la herramienta de diagnóstico que corresponde a esa capa.
2. Explicar qué garantiza TCP que UDP no, y qué cuesta esa garantía en tiempo de establecimiento.
3. Identificar una conexión por su cuádrupla y deducir cuántas conexiones simultáneas admite un cliente contra un mismo destino.
4. Interpretar los estados de un socket —LISTEN, SYNSENT, TIMEWAIT— para distinguir un puerto cerrado de un filtrado.
5. Calcular el throughput máximo de una conexión TCP a partir de su ventana y su latencia.

Conceptos centrales

Concepto	Comprensión verificable
encapsulación	Cada capa envuelve los datos de la superior con su propia cabecera y no interpreta el contenido. Es lo que permite cambiar de medio físico sin tocar la aplicación.
socket	Extremo de una comunicación, identificado por protocolo, IP y puerto. El sistema operativo lo expone como descriptor de fichero, de ahí que read y write funcionen sobre él igual que sobre un archivo.
cuádrupla	IP origen, puerto origen, IP destino y puerto destino. Identifica unívocamente una conexión TCP; dos conexiones pueden compartir tres de los cuatro valores pero no los cuatro.
MTU	Tamaño máximo de trama que un enlace transporta sin fragmentar. 1500 bytes en Ethernet estándar; los túneles restan espacio para su propia cabecera y son la causa habitual de conexiones que se establecen pero se cuelgan al transferir.
ventana de recepción	Bytes que el receptor declara poder aceptar sin confirmar. Junto con la latencia determina el techo de velocidad de una conexión TCP, independientemente del ancho de banda contratado.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

Flujo de razonamiento

```
flowchart TB
    A["Aplicación · HTTP, DNS<br/>diagnóstico: curl, dig"] --> T["Transporte · TCP, UDP<br/>puertos, fiabilidad<br/>diagnóstico: ss, nc"]
    T --> R["Red · IP<br/>direccionamiento y rutas<br/>diagnóstico: ip route, traceroute"]
    R --> E["Enlace · Ethernet, ARP<br/>MTU y trama"]
    E --> F["Físico · fibra, radio"]
    F --> A
```

F --> A

Desarrollo

1. Cada capa garantiza una cosa y solo una

El valor del modelo por capas no es taxonómico: es que acota dónde buscar. Cada capa promete algo concreto y no se hace cargo de lo demás:

Capa	Garantiza	No garantiza	Herramienta
Aplicación	Semántica del mensaje	Que llegue	curl -v, dig
Transporte (TCP)	Orden, integridad, entrega	Tiempo de llegada	ss -tnp, nc -vz
Transporte (UDP)	Nada más que multiplexar por puerto	Orden ni entrega	nc -u
Red (IP)	Mejor esfuerzo hacia un destino	Orden, entrega ni ruta estable	ip route, traceroute
Enlace	Entrega dentro de un mismo segmento	Nada fuera de él	ip neigh, ping con MTU

La regla de diagnóstico es de abajo hacia arriba: si la capa 3 no alcanza el destino, leer logs de la aplicación es tiempo perdido. En la parte 21, los runbooks de triaje seguirán exactamente este orden.

2. TCP paga con tiempo lo que ofrece en garantías

TCP establece la conexión antes de enviar un solo byte útil, con el saludo de tres vías:

```
cliente ■■■SYN■■■■■■■■■■■ servidor
cliente ■■■SYN+ACK■■■■■■■■■ servidor
cliente ■■■ACK■■■■■■■■■■■ servidor      ya se pueden enviar datos
```

Eso es un RTT completo antes del primer byte. Con Madrid–Virginia a 90 ms de ida y vuelta:

```
TCP                      90 ms
TLS 1.2 (2 RTT)          180 ms
-----
total antes del primer byte 270 ms

TCP                      90 ms
TLS 1.3 (1 RTT)          90 ms
-----
total                    180 ms
```

Por eso TLS 1.3 no es una mejora cosmética: elimina un RTT completo de cada conexión nueva. Y por eso la reutilización de conexiones (keep-alive, pooling) importa tanto: amortiza ese coste entre muchas peticiones. Un cliente que abre y cierra conexión por petición paga 180 ms de peaje cada vez.

UDP no establece nada: el primer datagrama ya lleva datos. A cambio, la aplicación se hace cargo del orden, la pérdida y la congestión. Es la elección correcta para DNS, telemetría y voz, donde llegar tarde es peor que no llegar.

3. Puertos, cuádruplas y el límite real de conexiones

El puerto no identifica una conexión: identifica un extremo. La conexión es la cuádrupla completa, y por eso mil clientes pueden hablar con el puerto 443 del mismo servidor sin ambigüedad: difieren en IP y puerto de origen.

El límite aparece al revés, cuando un cliente abre muchas conexiones al mismo destino. Solo puede variar su puerto de origen, y el rango efímero de Linux es:

```
$ cat /proc/sys/net/ipv4/ip_local_port_range
32768■■60999          # 28.232 puertos disponibles
```

Un proxy o un pod que llama a un mismo backend tiene un techo de 28.232 conexiones simultáneas, y en la práctica bastante menos porque cada cierre deja el puerto en TIMEWAIT durante 2×MSL (60 s en Linux):

conexiones/s sostenibles $\approx 28.232 / 60 \text{ s} \approx 470$ por segundo

Superado ese ritmo aparece EADDRNOTAVAIL y el síntoma es «la aplicación falla bajo carga sin error de la aplicación». La solución no es más CPU: es reutilizar conexiones. Este cálculo reaparecerá en las partes 06 y 16 al dimensionar NAT gateways, que tienen exactamente el mismo límite por IP.

4. El estado del socket dice qué falla

ss muestra en qué punto del protocolo se atascó la conexión, y cada estado apunta a una causa distinta:

```
$ ss -tnp state all '( dport = :443 )'
State      Recv-Q Send-Q Local Address:Port Peer Address:Port
ESTAB      0      0      10.0.1.5:51234      93.184.216.34:443
SYN-SENT   0      1      10.0.1.5:51240      10.0.9.9:443
TIME-WAIT  0      0      10.0.1.5:51201      93.184.216.34:443
```

Síntoma	Estado observado	Causa probable
Rechazo inmediato	Nada, ECONNREFUSED	Puerto cerrado: nadie escucha, pero la IP responde
Cuelgue hasta timeout	SYN-SENT persistente	Filtrado: un firewall descarta en silencio
Conecta y se cuelga al transferir	ESTAB con Send-Q creciendo	MTU: el saludo cabe, los datos no
Falla solo con carga	Muchos TIME-WAIT	Agotamiento de puertos efímeros

La distinción entre rechazado y filtrado es la más útil de todas: rechazado significa que llegaste al host y el servicio no está; filtrado significa que no llegaste. Son dos equipos distintos los que arreglan cada cosa.

5. El producto ancho de banda × retardo pone el techo

Una conexión TCP no puede ir más rápido que lo que permite su ventana dividida por el tiempo de ida y vuelta. El emisor manda una ventana y espera confirmación antes de continuar:

$$\text{throughput}_{\text{máximo}} = \text{ventana} / \text{RTT}$$

Con la ventana clásica de 64 KB y un enlace transatlántico de 90 ms:

$$65.536 \text{ bytes} / 0,090 \text{ s} = 728.178 \text{ B/s} \approx 5,8 \text{ Mbit/s}$$

Da igual que el enlace sea de 10 Gbit/s: una sola conexión no pasará de 5,8 Mbit/s. Para llenar 1 Gbit/s con ese RTT hace falta:

$$\text{ventana} = 1.000.000.000 \text{ bit/s} \times 0,090 \text{ s} / 8 = 11,25 \text{ MB}$$

De ahí el window scaling de RFC 7323, que permite ventanas de hasta 1 GB, y de ahí que las transferencias intercontinentales usen varias conexiones en paralelo. Cuando alguien diga «contratamos más ancho de banda y no mejoró», la respuesta suele estar en esta fórmula.

Ejemplo trabajado

El servicio de pagos de CloudShop falla de forma intermitente contra el proveedor externo. Los logs de la aplicación solo dicen «timeout». El operador diagnostica por capas, de abajo hacia arriba.

Capa 3 — ¿se alcanza el destino?

```
$ ip route get 203.0.113.40
203.0.113.40 via 10.0.0.1 dev eth0 src 10.0.1.5
$ traceroute -n 203.0.113.40 | tail -2
7  198.51.100.9  12.4 ms
8  203.0.113.40  14.1 ms      # alcanzable
```

Capa 4 — ¿se establece la conexión?

```
$ nc -vz 203.0.113.40 443
Connection to 203.0.113.40 443 port [tcp/https] succeeded!
```

Conecta. Así que no es ruta ni firewall. El fallo es intermitente y bajo carga, así que se mira el estado de los sockets durante el pico:

```
$ ss -tan state time-wait | wc -l
27891
$ ss -tan state all '( dport = :443 )' | grep -c ESTAB
412
$ dmesg | tail -1
TCP: request_sock_TCP: Possible SYN flooding...
$ cat /proc/sys/net/ipv4/ip_local_port_range
32768 60999
```

27.891 puertos en TIMEWAIT sobre 28.232 disponibles. El servicio agota el rango efímero:

```

disponibles          = 60.999 - 32.768 = 28.232
en TIME_WAIT         = 27.891  (98,8 %)
libres               =    341
ritmo sostenible     = 28.232 / 60 s ≈ 470 conexiones/s
ritmo observado en pico      ≈ 610 conexiones/s

```

El servicio abre una conexión TCP nueva por cada pago y la cierra. A 610 pagos por segundo pide 140 conexiones/s más de las que el sistema puede reciclar, y los connect() empiezan a devolver EADDRNOTAVAIL, que el cliente HTTP reporta como «timeout».

La corrección es reutilizar conexiones, no tocar el sistema operativo:

```

sesion = requests.Session()          # pool persistente con keep-alive
adapter = HTTPAdapter(pool_maxsize=64)
sesion.mount("https://", adapter)

$ ss -tan state time-wait | wc -l
743                                # de 27.891 a 743

```

64 conexiones reutilizadas sustituyen a 610 por segundo. Subir net.ipv4.tcp_twreuse habría escondido el síntoma un tiempo; el problema era de diseño del cliente, y estaba a dos capas de donde los logs decían.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/005-redes-por-capas-tcp-ip-puertos-y-sockets/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa captura-y-mapa-de-red en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye captura-y-mapa-de-red para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El servicio falla solo bajo carga y sin error propio de la aplicación	Agotamiento de puertos efímeros por abrir una conexión nueva por petición	Usa un pool con keep-alive; el techo es 28.000 puertos y 60 s de TIMEWAIT por cierre.
La conexión se establece pero se cuelga al transferir datos	MTU insuficiente en un túnel: el saludo cabe en paquetes pequeños y los datos no	Prueba con ping -M do -s para hallar la MTU real y ajusta MSS clamping.
Se contrató más ancho de banda y la transferencia no mejoró	El techo lo pone ventana/RTT, no el caudal del enlace	Habilita window scaling o paraleliza conexiones; calcula el producto ancho de banda × retardo.
Se depuran logs de la aplicación durante una hora y el problema era de red	Se diagnosticó de arriba hacia abajo	Verifica ruta y establecimiento de conexión antes de leer la aplicación.
No se distingue si el puerto está cerrado o filtrado	Se interpretó cualquier fallo de conexión como lo mismo	Rechazo inmediato es puerto cerrado; cuelgue en SYN-SENT es filtrado. Son equipos distintos.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuántos RTT median entre el primer paquete y el primer byte útil con TCP + TLS 1.3, y cuántos con TLS 1.2?
2. Un pod abre conexiones a un solo backend. ¿Cuál es su techo teórico de conexiones simultáneas y de dónde sale ese número?
3. Con ventana de 64 KB y RTT de 200 ms, ¿cuál es el throughput máximo de una conexión, y cambia si contratas 10 Gbit/s?
4. ¿Qué diferencia operativa hay entre un ECONNREFUSED inmediato y un cuelgue en SYN-SENT?
5. Una conexión se establece y se cuelga al enviar un cuerpo grande. ¿Qué capa sospechas y con qué comando lo confirmas?

Referencias

- Kurose, J. y Ross, K. (2021). Computer Networking: A Top-Down Approach, 8.^a ed., caps. 3-4 — transporte y capa de red.
- Postel, J., ed. (1981). RFC 793: Transmission Control Protocol — saludo de tres vías y máquina de estados. <<https://www.rfc-editor.org/rfc/rfc793>>
- Borman, D. et al. (2014). RFC 7323: TCP Extensions for High Performance — window scaling y el producto ancho de banda × retardo. <<https://www.rfc-editor.org/rfc/rfc7323>>
- Rescorla, E. (2018). RFC 8446: The Transport Layer Security (TLS) Protocol Version 1.3 — saludo de 1-RTT. <<https://www.rfc-editor.org/rfc/rfc8446>>
- Kerrisk, M. ss(8) y tcp(7) — estados de socket y parámetros del núcleo de Linux. <<https://man7.org/linux/man-pages/man7/tcp.7.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 005 Redes por capas, TCP/IP, puertos y sockets

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega captura-y-mapa-de-red con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

006 — DNS, HTTP, HTTPS y TLS de extremo a extremo

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Seguir una petición HTTPS desde el nombre hasta el byte: quién resuelve el nombre y con qué caché, qué negocia TLS y qué demuestra un certificado, y qué significan realmente los códigos y cabeceras de HTTP. Es el camino que recorren todas las peticiones del programa, y donde se originan la mayoría de incidentes de disponibilidad percibida.

Resultados de aprendizaje

Al finalizar podrás:

1. Trazar una resolución DNS completa e identificar en qué caché se queda un cambio que «no se propaga».
2. Explicar qué demuestra un certificado TLS y qué no, y por qué el cifrado no implica confianza en el otro extremo.
3. Elegir un TTL de DNS en función de la ventana de conmutación que necesita tu plan de continuidad.
4. Distinguir un 502 de un 503 y de un 504 para saber a qué componente apunta cada uno.
5. Diseñar una política de caché HTTP que permita invalidar sin esperar a que expire.

Conceptos centrales

Concepto	Comprensión verificable
resolutor recursivo	Servidor que hace el trabajo de consultar la cadena de autoridades en nombre del cliente y guarda el resultado en caché. Es quien realmente decide durante cuánto tiempo verás una respuesta antigua.
TTL	Segundos que un registro DNS puede permanecer en caché. Fija el suelo del tiempo de conmutación: con TTL de 3600, un cambio de IP tarda hasta una hora en verse, sin importar lo rápido que lo publiques.
SNI	Extensión de TLS que envía el nombre del host solicitado en claro dentro del saludo, para que un servidor con muchos certificados sepa cuál presentar. Al ir en claro, revela qué sitio visitas aunque el contenido vaya cifrado.
cadena de confianza	Secuencia de firmas desde el certificado del servidor hasta una raíz que el cliente ya considera fiable. Un certificado válido demuestra control del nombre, no honestidad del operador.
caché condicional	Mecanismo por el que el cliente pregunta «¿ha cambiado desde esta versión?» con If-None-Match. Si no cambió, el servidor responde 304 sin cuerpo, ahorrando la transferencia pero no el viaje.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    C["cliente"] -->|1 ¿A de shop.ejemplo?"| R["resolutor recursivo"]
    R -->|2| ROOT["raíz · NS de .cl"]
    R -->|3| TLD["TLD · NS de ejemplo.cl"]
    R -->|4| AUT["autoritativo · A = 203.0.113.7"]
    R -->|5 respuesta + TTL"| C
    C -->|6 TCP + TLS: SNI, certificado"| S["servidor 203.0.113.7"]
    S -->|7 HTTP: método, cabeceras, código"| C
    R -.->|caché: sirve sin pasos 2-4 hasta que expira el TTL"| C
```

Desarrollo

1. DNS: la caché que decide cuánto dura tu error

Una resolución completa recorre la jerarquía, pero casi ninguna lo hace: el resolutor responde desde caché. Esa caché es la que gobierna la realidad operativa.

```
$ dig +trace shop.ejemplo.cl A | tail -4
ejemplo.cl.      172800  IN  NS   ns1.ejemplo.cl.
shop.ejemplo.cl. 300    IN  A    203.0.113.7      # TTL de 300 s

$ dig shop.ejemplo.cl A +noall +answer
shop.ejemplo.cl. 187    IN  A    203.0.113.7      # quedan 187 s en caché
```

El TTL es una decisión de arquitectura de continuidad, no un parámetro técnico:

TTL	Coste en consultas	Ventana de conmutación	Cuándo
60 s	Alto	≤ 1 min	Antes de una migración planificada
300 s	Medio	≤ 5 min	Servicios con failover DNS
3600 s	Bajo	≤ 1 h	Registros estables
86400 s	Mínimo	≤ 24 h	NS, MX, registros de verificación

La regla operativa: baja el TTL antes de necesitarlo. Bajarlo a 60 s en mitad de un incidente no sirve, porque los resolutores siguen sirviendo la respuesta anterior con su TTL original. Hay que bajarlo con al menos un TTL antiguo de antelación.

Y hay un límite que no controlas: muchos resolutores públicos y sistemas operativos imponen mínimos propios e ignoran TTL muy bajos. DNS es control de tráfico con granularidad de minutos, nunca de segundos; por eso en la parte 13 el failover regional no se apoya solo en DNS.

2. Qué demuestra un certificado y qué no

TLS resuelve tres problemas distintos y conviene no confundirlos:

1. Confidencialidad: nadie en el camino lee el contenido.
2. Integridad: nadie lo modifica sin que se detecte.
3. Autenticación del servidor: hablas con quien dice el nombre.

El certificado solo aporta el tercero, y de forma acotada: prueba que alguien demostró control sobre ese nombre de dominio ante una autoridad en la que tu cliente confía. No dice nada sobre la honestidad del operador ni la seguridad de su código. Un sitio de phishing con certificado válido es exactamente eso: cifrado, íntegro y fraudulento.

```
$ openssl s_client -connect shop.ejemplo.cl:443 -servername shop.ejemplo.cl </dev/null 2>/dev/null \
| openssl x509 -noout -subject -issuer -dates
subject=CN=shop.ejemplo.cl
issuer=C=US, O=Let's Encrypt, CN=R11
notBefore=Jul  2 00:00:00 2026 GMT
notAfter=Sep 30 23:59:59 2026 GMT
```

El -servername es SNI y es obligatorio: sin él, un servidor con varios certificados presenta el que tenga por defecto y el diagnóstico induce a error. Como SNI viaja en claro, un observador de red sabe qué sitio visitas aunque no vea el contenido; ECH (Encrypted Client Hello) es la respuesta en curso a esa fuga.

3. Los códigos HTTP dicen quién falló

Los códigos 5xx no son intercambiables: cada uno apunta a un componente distinto de la cadena, y confundirlos alarga los incidentes.

Código	Significado	Quién falló
500	Error interno	La aplicación de origen: excepción no controlada
502	Puerta de enlace incorrecta	El proxy habló con el origen y recibió basura o nada

Código	Significado	Quién falló
503	Servicio no disponible	El origen se declara saturado o en mantenimiento
504	Tiempo agotado en la puerta	El proxy esperó y el origen no respondió a tiempo

La distinción práctica entre 502 y 504: en el 502 el origen respondió algo inválido o cerró la conexión, así que hay que mirar sus logs; en el 504 no respondió nada dentro del plazo, así que hay que mirar su latencia y saturación. Buscar excepciones en el origen ante un 504 no lleva a ningún sitio, porque probablemente la petición sigue procesándose.

El 503 tiene un matiz que casi nadie usa: admite la cabecera `Retry-After`, que le dice al cliente cuándo volver. Sin ella, los clientes reintentan de inmediato y agravan la saturación que causó el 503 — el mismo thundering herd de la clase 004.

4. Caché HTTP: el viaje que no se hace

La petición más rápida es la que no ocurre. HTTP ofrece dos niveles, y elegir mal el primero condena a soportar contenido obsoleto:

```
Cache-Control: public, max-age=31536000, immutable
```

Para recursos con huella en el nombre (app.4f2a9c.js): un año, sin revalidar. Como el nombre cambia con el contenido, invalidar es publicar un nombre nuevo; nunca hay que purgar nada.

```
Cache-Control: no-cache
ETag: "7d2b8e91"
```

Para documentos que cambian (index.html): no-cache no significa «no almacenar», significa «almacena pero revalida antes de usar». El cliente pregunta con `If-None-Match: "7d2b8e91"` y recibe 304 Not Modified sin cuerpo si no cambió: se ahorra la transferencia, no el viaje.

Para prohibir el almacenamiento de verdad hace falta no-store, y solo tiene sentido en respuestas con datos personales o tokens.

El error clásico es servir index.html con max-age largo: entonces una corrección urgente no llega hasta que expire, y no hay forma de forzarlo desde el servidor. Por eso el patrón que se repetirá en las partes 16 y 17 es siempre el mismo: HTML corto y revalidable, activos largos e inmutables con huella en el nombre.

5. El coste completo de una petición en frío

Sumando lo anterior, la primera petición a un origen lejano paga varios peajes secuenciales:

resolución DNS (caché fría)	1 RTT al resolutor + cadena
establecimiento TCP	1 RTT
saludo TLS 1.3	1 RTT
petición y respuesta HTTP	1 RTT

mínimo	≈ 4 RTT

Con un RTT de 90 ms son 360 ms antes de mostrar nada, y ninguna optimización del servidor los reduce. Las palancas son todas de red:

- Reutilizar la conexión elimina TCP y TLS de las peticiones siguientes.
- Acercar la terminación (CDN, edge) reduce el RTT que multiplica a todo lo demás.
- 0-RTT de TLS 1.3 permite enviar datos en el primer paquete al reconectar, con la contrapartida de que esos datos son vulnerables a repetición: solo vale para peticiones idempotentes.

Es el mismo cálculo de la clase 001, ahora con los protocolos concretos que lo producen.

Ejemplo trabajado

CloudShop migra su frontend a una IP nueva. El equipo publica el cambio a las 10:00 y a las 11:30 sigue habiendo usuarios llegando al servidor viejo. Se acusa al proveedor de DNS.

Primero se comprueba qué publica la autoridad, sin pasar por caché:

```
$ dig @ns1.ejemplo.cl shop.ejemplo.cl A +noall +answer
shop.ejemplo.cl. 3600 IN A 203.0.113.44      # correcto, ya es la IP nueva
```

La autoridad está bien. Ahora qué ven los resolutores públicos:

```
$ dig @8.8.8.8 shop.ejemplo.cl A +noall +answer
shop.ejemplo.cl. 1187 IN A 203.0.113.7          # IP VIEJA, 1187 s restantes
$ dig @1.1.1.1 shop.ejemplo.cl A +noall +answer
shop.ejemplo.cl. 942 IN A 203.0.113.7
```

El TTL era 3600 s. Cuando se publicó el cambio, los resolutores ya tenían la respuesta anterior cacheada, cada uno con su reloj propio:

publicación del cambio	10:00	
peor caso: un resolutor cacheó a	09:59	→ expira a 10:59
ventana teórica de convergencia		→ hasta 11:00
observado a 11:30		→ hay resolutores con TTL de 3600 s que refrescaron a 10:30

La aritmética explica el residual: cada resolutor que refresca justo antes del cambio arrastra una hora más. El tiempo de convergencia no es el TTL: es hasta dos veces el TTL si el cambio coincide con el peor momento de refresco.

Lo que debió hacerse, con un día de antelación:

D-1 09:00	bajar TTL de 3600 a 60	(empieza a propagarse)
D-1 10:00	todos los resolutores tienen ya TTL de 60	
D 10:00	cambiar la IP	(converge en ≤ 2 min)
D 12:00	restaurar TTL a 3600	

En el incidente real, la mitigación no fue DNS sino red: se mantuvo el servidor viejo respondiendo con un 308 Permanent Redirect hacia el nuevo hasta que la caché expiró.

La lección: DNS no es un conmutador. Es una caché distribuida cuyo tiempo de reacción se decide con un TTL, y ese TTL hay que bajarlo antes de necesitarlo. Por eso los planes de continuidad de la parte 13 no dependen de DNS para conmutaciones de segundos.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/006-dns-http-https-y-tls-de-extremo-a-extremo/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa traza-de-solicitud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye traza-de-solicitud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.

- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se cambia una IP y horas después sigue llegando tráfico al servidor antiguo	El TTL alto estaba cacheado en los resolutores antes del cambio	Baja el TTL con al menos un TTL de antelación; cuenta hasta dos veces el TTL como ventana de convergencia.
openssl sclient muestra un certificado que no corresponde al sitio	Se omitió -servername, así que el servidor presentó su certificado por defecto	Incluye siempre SNI con -servername al diagnosticar hosts virtuales.
Se buscan excepciones en el origen ante un 504 y no aparece ninguna	El 504 significa que el origen no respondió a tiempo, no que fallara	Ante 504 mira latencia y saturación del origen; ante 502, sus logs de error.
Una corrección urgente del HTML no llega a los usuarios	Se sirvió el documento con max-age largo y no hay forma de invalidar desde el servidor	HTML con no-cache y ETag; activos inmutables con huella en el nombre.
Un 503 provoca una avalancha de reintentos que empeora la saturación	La respuesta no incluyó Retry-After y los clientes reintentaron de inmediato	Devuelve Retry-After y aplica retroceso con jitter en el cliente.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Con un TTL de 3600 s, ¿cuál es el peor caso de convergencia tras cambiar una IP, y por qué no es una hora?
2. ¿Qué demuestra exactamente un certificado TLS válido, y qué afirmación sobre el sitio no respalda?
3. Un proxy devuelve 502 y otro 504. ¿En cuál mirarías los logs del origen y en cuál sus métricas de latencia?
4. ¿Por qué no-cache no impide almacenar, y qué cabecera sí lo impide?
5. ¿Cuántos RTT tiene una petición HTTPS en frío y cuáles se eliminan reutilizando la conexión?

Referencias

- Mockapetris, P. (1987). RFC 1035: Domain Names — Implementation and Specification.
<<https://www.rfc-editor.org/rfc/rfc1035>>
- Rescorla, E. (2018). RFC 8446: TLS 1.3 — saludo de 1-RTT, 0-RTT y sus riesgos de repetición.
<<https://www.rfc-editor.org/rfc/rfc8446>>
- Fielding, R. y Reschke, J., eds. (2022). RFC 9110: HTTP Semantics — semántica de códigos de estado.
<<https://www.rfc-editor.org/rfc/rfc9110>>
- Fielding, R. et al., eds. (2022). RFC 9111: HTTP Caching — Cache-Control, validadores y respuestas 304.
<<https://www.rfc-editor.org/rfc/rfc9111>>
- Grigorik, I. (2013). High Performance Browser Networking, caps. 1-4 — coste en RTT de DNS, TCP y TLS.
<<https://hpbnp.co/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 006 DNS, HTTP, HTTPS y TLS de extremo a extremo

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega traza-de-solicitud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

007 — Linux: usuarios, permisos, servicios y logs

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: linux · Estado: EXECUTABLECORE

Propósito

Entender el modelo de identidad y privilegio de Linux, y el ciclo de vida de un servicio bajo systemd, porque son la base literal de todo lo que viene: los contenedores de la parte 05 son procesos Linux con espacios de nombres, y el mínimo privilegio de IAM en la parte 11 es esta misma idea trasladada a la nube.

Resultados de aprendizaje

Al finalizar podrás:

1. Descomponer un modo de permisos octal y explicar por qué el bit de ejecución significa cosas distintas en fichero y en directorio.
2. Sustituir el uso de root por capacidades concretas, justificando cuál se necesita y por qué.
3. Definir una unidad de systemd con reinicio, límites y usuario propio, y explicar cada directiva.
4. Consultar logs con journalctl filtrando por unidad, prioridad y ventana temporal.
5. Diagnosticar un servicio que no arranca distinguiendo permisos, dependencias y configuración.

Conceptos centrales

Concepto	Comprensión verificable
UID efectivo	Identificador con el que el núcleo evalúa cada comprobación de permiso. Puede diferir del UID real tras un setuid, y es el que importa para decidir si una operación se permite.
capacidad	Fragmento del poder histórico de root, otorgable por separado. CAPNETBINDSERVICE permite abrir puertos bajo 1024 sin conceder las otras 40 capacidades que root arrastra.
unidad	Objeto que systemd gestiona: servicio, socket, temporizador, punto de montaje o destino. Se declara de forma descriptiva y systemd deduce el orden de arranque a partir de las dependencias.
journal	Registro binario, indexado y estructurado de systemd. Cada entrada lleva metadatos —unidad, PID, UID, prioridad— consultables como campos, no como texto.
umask	Máscara que se resta de los permisos por defecto al crear ficheros. Con umask 022, un fichero nace 644 y un directorio 755; explica por qué lo que creas no es de escritura para el grupo.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    K["núcleo: ¿permitir esta operación?"] --> U{"¿UID efectivo = propietario?"}
    U -->|sí| PU["aplica los 3 bits de usuario"]
    U -->|no| G{"¿pertenece al grupo?"}
    G -->|sí| PG["aplica los 3 bits de grupo"]
    G -->|no| PO["aplica los 3 bits de otros"]
    PU --> D["decisión"]
    PG --> D
    PO --> D
    D -->|si UID efectivo = 0<br/>se omite y se miran capacidades| C["CAP_NET_BIND_SERVICE,<br/>>CAP_CHOWN, CAP_SYS_ADMIN..."]
```

Desarrollo

1. Permisos: tres tríos y una regla que sorprende

Cada objeto del sistema de ficheros tiene propietario, grupo y tres tríos de bits. El núcleo evalúa el primer trío que aplica y se detiene ahí:

```
$ ls -l /opt/cloudshop/config.yaml
-rw-r----- 1 cloudshop cloudshop 412 Aug  1 09:12 config.yaml
# ↑↑↑↑↑↑↑↑
#  rw-  usuario cloudshop: leer y escribir
#    r--  grupo cloudshop: solo leer
#      ---  otros: nada
```

En octal, 640. La consecuencia contraintuitiva: si eres el propietario y el trío de usuario deniega, no te salva pertenecer al grupo. Un fichero 0640 propiedad de ana y grupo ana es ilegible para ana si el primer trío fuera ---, aunque su grupo pueda leerlo.

El bit de ejecución significa cosas distintas según el objeto:

Bit	En fichero	En directorio
r	Leer el contenido	Listar los nombres
w	Modificar el contenido	Crear y borrar entradas
x	Ejecutarlo	Atravesarlo para llegar a su interior

De ahí dos efectos que confunden a diario:

- Un directorio r-- deja ver los nombres pero no consultar los metadatos de lo que contiene: ls funciona y ls -l da «permiso denegado».
- Para borrar un fichero no hace falta permiso sobre el fichero, sino w sobre el directorio que lo contiene. Por eso /tmp lleva el sticky bit (1777): permite a todos crear, pero solo al propietario borrar lo suyo.

2. root no es un usuario: es la ausencia de comprobaciones

Cuando el UID efectivo es 0, el núcleo omite la comprobación de permisos. Eso no es un privilegio más: es no tener ninguno que aplicar. Linux fragmentó ese poder en unas 40 capacidades otorgables por separado:

Capacidad	Permite	Uso legítimo
CAPNETBINDSERVICE	Abrir puertos < 1024	Un servidor web en el 80
CAPCHOWN	Cambiar el propietario	Gestores de paquetes
CAPSYSADMIN	Montar, espacios de nombres...	Casi equivale a root

El caso que se repite: un servicio quiere escuchar en el 443 y alguien lo ejecuta como root «porque si no, no puede». La alternativa correcta cede solo lo necesario:

```
[Service]
User=cloudshop
AmbientCapabilities=CAP_NET_BIND_SERVICE
CapabilityBoundingSet=CAP_NET_BIND_SERVICE
```

Si ese proceso resulta vulnerable, el atacante obtiene la capacidad de abrir puertos bajos y nada más: no puede leer /etc/shadow, ni cargar módulos, ni cambiar propietarios. CAPSYSADMIN es la excepción: concederla equivale prácticamente a conceder root, y verla en un manifiesto debe activar la misma alarma que privileged: true en la parte 06.

3. systemd declara el estado deseado, no los pasos

Una unidad no describe cómo arrancar: describe qué debe cumplirse. systemd deduce el orden a partir de las dependencias y paraleliza el resto.

```
[Unit]
Description=API de CloudShop
After=network-online.target          # orden: después de que haya red
```

```
Wants=network-online.target          # dependencia débil: si falla, seguimos

[Service]
Type=notify                          # el proceso avisa cuando está listo
User=cloudshop
ExecStart=/opt/cloudshop/bin/api
Restart=on-failure                  # no reiniciar si salió con 0
RestartSec=5
StartLimitBurst=5                   # 5 intentos...
StartLimitIntervalSec=300           # ...en 5 minutos, o se rinde

NoNewPrivileges=true                # ningún hijo escala privilegios
ProtectSystem=strict                 # todo el sistema en solo lectura
ProtectHome=true
PrivateTmp=true                      # /tmp propio, aislado
ReadWritePaths=/var/lib/cloudshop    # única excepción de escritura
MemoryMax=512M

[Install]
WantedBy=multi-user.target
```

Dos distinciones que evitan incidentes:

- Requires frente a Wants: Requires arrastra la caída del dependiente si la dependencia falla; Wants no. Usar Requires a la ligera propaga fallos en cascada.
- After no implica dependencia: solo fija orden. Un servicio con After=X pero sin Wants=X arranca aunque X no exista.

StartLimitBurst merece atención: sin él, un servicio que falla al arrancar entra en bucle de reinicio consumiendo CPU. Con él, systemd se rinde y deja el fallo visible en vez de esconderlo tras reintentos infinitos.

4. El journal es estructurado: consúltalo como tal

journalctl no busca texto: filtra campos indexados. Tratarlo con grep desperdicia su ventaja y produce falsos negativos.

```
# Errores y peores de una unidad en la última hora
$ journalctl -u cloudshop -p err --since "1 hour ago"

# Solo el arranque actual, siguiendo en vivo
$ journalctl -u cloudshop -b -f

# Salida estructurada para automatizar
$ journalctl -u cloudshop -o json --since today | jq -r 'select(.PRIORITY<="3") | .MESSAGE'

# Qué escribió un PID concreto, aunque ya no exista
$ journalctl _PID=4877
```

Las prioridades siguen la escala syslog, de 0 (emergencia) a 7 (depuración); -p err incluye 0 a 3. Un servicio que escribe todo como info hace inútil este filtro: la prioridad es parte del contrato del log, igual que stdout y stderr lo eran en la clase 002.

Dos límites operativos que importan: el journal es local y rotatorio, con tamaño acotado por SystemMaxUse. Si el disco se llena, se descartan las entradas más antiguas. Por eso en la parte 10 los logs se envían fuera del host: un log que solo existe en la máquina que falló no sirve para el postmortem.

5. Diagnóstico de un servicio que no arranca

El orden ahorra horas, porque cada paso descarta una familia de causas:

```
$ systemctl status cloudshop          # 1. ¿qué dice systemd?
$ journalctl -u cloudshop -n 50 -p warning # 2. ¿qué dijo el proceso?
$ systemd-analyze verify cloudshop.service # 3. ¿la unidad es válida?
$ sudo -u cloudshop /opt/cloudshop/bin/api # 4. ¿arranca a mano con ese usuario?
$ systemctl show cloudshop -p ExecMainStatus -p ExecMainCode
```

El paso 4 es el que más separa: si a mano funciona con el mismo usuario, el problema está en el entorno de la unidad —PATH, variables, directorio de trabajo, endurecimiento— y no en la aplicación. Es exactamente el fallo de la clase 002.

Los códigos más frecuentes y su lectura inmediata:

ExecMainStatus	Causa habitual
203	ExecStart no existe o no es ejecutable
200	El directorio de trabajo no existe
208	User= no existe
226	El endurecimiento bloqueó una ruta necesaria
1-125	Fallo propio de la aplicación: ahora sí, mira sus logs

Los códigos 200-241 son de systemd, no de tu programa: significan que el proceso nunca llegó a ejecutarse.

Ejemplo trabajado

La API de CloudShop arranca en desarrollo y falla en el servidor nuevo. El log de la aplicación está vacío.

```
$ systemctl status cloudshop
● cloudshop.service - API de CloudShop
   Active: failed (Result: exit-code)
   Process: 8812 ExecStart=/opt/cloudshop/bin/api (code=exited, status=226/NAMESPACE)
```

226/NAMESPACE: no es la aplicación. systemd impidió el arranque por el endurecimiento antes de ejecutar una sola línea. El log vacío ahora se explica: el proceso nunca corrió.

```
$ journalctl -u cloudshop -n 5 -p err
cloudshop.service: Failed to set up mount namespaces:
/var/log/cloudshop: Read-only file system
```

La unidad declara ProtectSystem=strict, que monta todo el sistema en solo lectura, y la aplicación escribe en /var/log/cloudshop, que no está en las excepciones:

```
$ systemctl show cloudshop -p ProtectSystem -p ReadWritePaths
ProtectSystem=strict
ReadWritePaths=/var/lib/cloudshop
```

Hay dos correcciones posibles y no son equivalentes:

```
# A) Ampliar la excepción: mantiene el endurecimiento
ReadWritePaths=/var/lib/cloudshop /var/log/cloudshop

# B) Escribir al journal por stdout: elimina la necesidad
# (se quita la ruta de log de la aplicación; systemd captura stdout)
```

Se elige B. Razón operativa, no estética: un fichero de log local se pierde cuando la instancia se recicla, no rota solo, y llena el disco. Escribir a stdout deja que systemd lo estructure y que el recolector de la parte 10 lo envíe fuera del host. Es además lo que exigirá el contenedor de la parte 05, donde escribir logs a fichero es directamente un antipatrón.

Comprobación final del privilegio real que conserva el servicio:

```
$ systemctl restart cloudshop && systemctl show cloudshop -p ExecMainStatus
ExecMainStatus=0
$ grep CapEff /proc/$(systemctl show -p MainPID --value cloudshop)/status
CapEff:█00000000000000400          # solo CAP_NET_BIND_SERVICE
```

0x400 es el bit 10, exactamente CAPNETBINDSERVICE. Si esa API resultara vulnerable, el atacante hereda el poder de abrir puertos bajos y nada más. Ese es el mismo razonamiento que en la parte 11 se aplicará a un rol de IAM: no «qué necesita para funcionar», sino qué obtiene quien lo comprometa.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/007-linux-usuarios-permisos-servicios-y-logs/lab.py
```

El laboratorio selecciona el motor de práctica linux y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.

2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-auditado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un servicio con identidad, permisos y logs inspeccionados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye servicio-auditado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un servicio se ejecuta como root solo para escuchar en el puerto 443	Se concedió todo el poder para obtener una capacidad concreta	Usa User= propio más AmbientCapabilities=CAPNETBINDSERVICE.
El servicio falla con 226/NAMESPACE y el log de la aplicación está vacío	El endurecimiento de systemd bloqueó una ruta antes de ejecutar el proceso	Los códigos 200-241 son de systemd: revisa ReadWritePaths y ProtectSystem, no el código.
Un servicio que falla al arrancar consume CPU en bucle de reinicio	Restart=always sin límite de intentos	Usa Restart=on-failure con StartLimitBurst y StartLimitIntervalSec.
Tras un incidente no hay logs de la máquina afectada	El journal es local y rotatorio; se perdió con la instancia	Envía los logs fuera del host; el journal sirve para diagnóstico en vivo, no para postmortem.
Un usuario no puede borrar un fichero del que es propietario	Borrar exige w sobre el directorio contenedor, no sobre el fichero	Revisa los permisos del directorio; el sticky bit en /tmp explota justamente esta regla.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Un fichero es 0640, propiedad de ana, grupo ana. ¿Puede ana escribirlo? ¿Y si el trío de usuario fuera --- y el de grupo rw-?
2. ¿Por qué ls funciona y ls -l falla en un directorio con permisos r--?
3. ¿Qué obtiene un atacante que compromete un proceso con CAPNETBINDSERVICE frente a uno que corre como root?

- ¿Qué diferencia práctica hay entre Requires= y Wants=, y cuál propaga fallos en cascada?
- Un servicio devuelve ExecMainStatus=226. ¿Tiene sentido revisar el código de la aplicación? ¿Por qué?

Referencias

- Kerrisk, M. capabilities(7) — catálogo completo de capacidades de Linux y sus implicaciones. <<https://man7.org/linux/man-pages/man7/capabilities.7.html>>
- systemd (2024). systemd.exec(5) — directivas de endurecimiento: ProtectSystem, ReadWritePaths, NoNewPrivileges. <<https://www.freedesktop.org/software/systemd/man/systemd.exec.html>>
- systemd (2024). systemd.service(5) — códigos de salida 200-241 y política de reinicio. <<https://www.freedesktop.org/software/systemd/man/systemd.service.html>>
- systemd (2024). journalctl(1) — filtrado por campos, prioridades y arranque. <<https://www.freedesktop.org/software/systemd/man/journalctl.html>>
- Ward, B. (2021). How Linux Works, 3.^a ed., caps. 6-7 — arranque, systemd y configuración del sistema.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 007 Linux: usuarios, permisos, servicios y logs

Preguntas

- Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
- Identifica una frontera de responsabilidad y su propietario.
- Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
- ¿Qué demuestra el laboratorio y qué no puede demostrar?
- Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega servicio-auditado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

008 — Virtualización, hipervisores e imágenes

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: virtualization · Estado: EXECUTABLECORE

Propósito

Distinguir los tres mecanismos que la industria llama «virtualización» —máquinas virtuales, contenedores y microVM— por lo que realmente aíslan y lo que realmente cuestan. Sin esa distinción, la elección entre EC2, Fargate y Lambda en la parte 17 se hace por moda; con ella, se hace por requisito de aislamiento y perfil de arranque.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar qué aísla un hipervisor que un espacio de nombres no, y qué superficie de ataque queda en cada caso.
2. Comparar tiempo de arranque, sobrecarga y densidad de VM, contenedor y microVM con órdenes de magnitud.
3. Justificar el uso de microVM cuando se ejecuta código de terceros no confiable.
4. Describir cómo el almacenamiento por capas hace que 50 contenedores de la misma imagen ocupen poco más que uno.
5. Reconocer que una imagen es una lista ordenada de capas más un manifiesto, y por qué el digest es su única referencia inmutable.

Conceptos centrales

Concepto	Comprensión verificable
hipervisor	Capa que presenta hardware virtual a varios sistemas operativos huésped. De tipo 1 corre sobre el hardware desnudo; de tipo 2, sobre un sistema anfitrión. El aislamiento lo impone la CPU, no el software.
espacio de nombres	Mecanismo del núcleo Linux que da a un grupo de procesos su propia vista de un recurso: PID, red, montajes, usuarios. Aísla lo que se ve, no lo que se ejecuta: el núcleo sigue siendo compartido.
cgroup	Subsistema que limita y contabiliza CPU, memoria y E/S de un grupo de procesos. Complementa a los espacios de nombres: estos aíslan la vista y aquel reparte el recurso.
microVM	Máquina virtual reducida al mínimo dispositivo emulado para arrancar en decenas de milisegundos. Combina el aislamiento de hardware del hipervisor con un perfil de arranque cercano al del contenedor.
capa	Conjunto de diferencias del sistema de ficheros, inmutable y direccionado por su digest. Las imágenes que comparten una capa base la almacenan y descargan una sola vez.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    subgraph VM["Máquina virtual"]
        HW1["hardware"] --> HYP["hipervisor"]
        HYP --> K1["núcleo huésped A"] --> P1["procesos"]
        HYP --> K2["núcleo huésped B"] --> P2["procesos"]
    end
    end
    subgraph CT["Contenedores"]
        HW2["hardware"] --> KS["núcleo COMPARTIDO"]
        KS --> NS1["ns + cgroup A"] --> C1["procesos"]
        KS --> NS2["ns + cgroup B"] --> C2["procesos"]
    end
    end
    KS -->|"un fallo aquí<br/>alcanza a todos"| CT
```

Desarrollo

1. Dos aislamientos que no son comparables

La diferencia decisiva cabe en una frase: una máquina virtual tiene su propio núcleo; un contenedor comparte el del anfitrión.

En una VM, el huésped ejecuta su núcleo y el hipervisor le presenta hardware virtual. El aislamiento lo hace cumplir la propia CPU mediante extensiones de virtualización (Intel VT-x, AMD-V): para escapar hay que romper el hipervisor, que expone una interfaz reducida.

En un contenedor no hay huésped. Son procesos normales del anfitrión con una vista restringida por espacios de nombres y un reparto impuesto por cgroups. La superficie de ataque es toda la interfaz de llamadas al sistema del núcleo Linux: más de 300 llamadas, algunas con historial de vulnerabilidades de escalada.

```
$ docker run --rm alpine uname -r
6.8.0-45-generic          # el núcleo del anfitrión, no de la imagen
$ uname -r
6.8.0-45-generic          # idéntico
```

La imagen aporta bibliotecas y binarios; el núcleo siempre es el del anfitrión. De ahí tres consecuencias que reaparecerán en la parte 05: no se pueden ejecutar contenedores Windows sobre un núcleo Linux; un contenedor no puede cargar módulos; y una vulnerabilidad del núcleo afecta simultáneamente a todos los contenedores de esa máquina.

2. El coste del aislamiento, en números

Los órdenes de magnitud, no las cifras exactas, son lo que hay que retener:

	Máquina virtual	Contenedor	microVM (Firecracker)
Arranque	30-60 s	50-200 ms	125 ms
Sobrecarga de memoria	512 MB-2 GB	1-10 MB	< 5 MB
Densidad por host	decenas	miles	miles
Aislamiento	hardware	núcleo compartido	hardware
Núcleo	propio	del anfitrión	propio, reducido

Una VM arranca lento porque hace lo mismo que una máquina física: firmware, cargador, núcleo, init, servicios. Un contenedor solo ejecuta un proceso más con una vista distinta.

Firecracker rompe la disyuntiva: elimina casi todos los dispositivos emulados —sin BIOS, sin USB, sin VGA— y deja solo lo mínimo. Arranca en unos 125 ms con menos de 5 MB de sobrecarga, y mantiene el aislamiento por hardware. Por eso AWS lo usa bajo Lambda y Fargate: ejecutan código de clientes distintos en el mismo host y el aislamiento por núcleo compartido no sería aceptable.

La regla de decisión: si ejecutas tu código, el contenedor basta. Si ejecutas código de terceros que no controlas, necesitas frontera de hardware.

3. Espacios de nombres y cgroups: qué aísla cada cosa

Un contenedor no es una primitiva del núcleo. Es la combinación de varios mecanismos independientes que se pueden usar por separado:

Espacio de nombres	Aísla
pid	La tabla de procesos: dentro, tu proceso es el PID 1
net	Interfaces, rutas, reglas de firewall y puertos
mnt	El árbol de montajes visible
uts	Nombre de host y de dominio
ipc	Colas de mensajes y memoria compartida
user	El mapeo de UID: root dentro puede ser un UID sin privilegios fuera
cgroup	La vista de la propia jerarquía de cgroups

Se ven directamente:

```
$ docker run --rm -d --name demo alpine sleep 300
$ pid=$(docker inspect -f '{{.State.Pid}}' demo)
$ sudo ls -l /proc/$pid/ns/
lrwxrwxrwx net -> 'net:[4026532501]'      # espacio propio
lrwxrwxrwx pid -> 'pid:[4026532503]'
lrwxrwxrwx user -> 'user:[4026531837]'    # el MISMO del anfitrión
```

Ese último renglón es el hallazgo importante: por defecto el espacio de usuarios no está aislado, así que el UID 0 dentro del contenedor es el UID 0 del anfitrión. Si el proceso escapa, escapa como root. Activar user remapea root a un UID sin privilegios y es la mitigación más rentable frente a una fuga.

Los cgroups son la otra mitad: sin un límite de memoria, un contenedor consume la del anfitrión y el OOM killer mata al que más memoria use, que puede ser otro contenedor inocente. Ese es el origen del código 137 de la clase 002.

4. Capas: por qué 50 contenedores caben en el disco de uno

Una imagen es una lista ordenada de capas inmutables, cada una nombrada por el digest de su contenido, más un manifiesto que las enumera. Al arrancar, el runtime apila esas capas en solo lectura y añade encima una capa de escritura propia del contenedor, mediante copy-on-write con OverlayFS.

```
$ docker image inspect cloudshop:2.4 -f '{{range .RootFS.Layers}}{{println .}}{{end}}'
sha256:8e012198eba1...      # base: alpine 3.20
sha256:4f2a9c73b8d5...      # dependencias
sha256:7d2b8e91c4a2...      # aplicación
```

Si 50 contenedores usan esta imagen, las tres capas base se almacenan una vez. Cada contenedor solo añade lo que escribe:

```
imagen                      180 MB   (compartida por los 50)
capa de escritura por contenedor ~2 MB
-----
50 contenedores              180 + 50×2 = 280 MB
50 VM con el mismo software  50 × 180 = 9.000 MB
```

De aquí sale una regla de construcción que dominará la parte 05: ordena las capas de menos a más volátil. Si copias el código antes de instalar dependencias, cualquier cambio de una línea invalida la capa de dependencias y obliga a reconstruirla y redescargarla entera.

Y de aquí sale también por qué el digest importa: una etiqueta como :2.4 es mutable —puede reapuntarse mañana— mientras que sha256:... identifica exactamente ese contenido. Es la misma distinción entre rama y commit de la clase 003.

5. Qué elegir, y con qué criterio

La decisión se toma con dos preguntas, en este orden:

1. ¿De quién es el código que se ejecuta?

- Propio o auditado → contenedor.
- De terceros, sin auditar, o multi-cliente → frontera de hardware: VM o microVM.

2. ¿Qué perfil de arranque exige la carga?

- Larga vida, estado en disco, núcleo específico → VM.
- Efímera, escalado por ráfagas, sin estado → contenedor o microVM.

Situación	Elección	Por qué
API propia con escalado horizontal	Contenedor	Arranque rápido, densidad alta, código confiable
Ejecutar plugins de clientes	microVM	Frontera de hardware entre inquilinos
Base de datos con disco y ajuste de núcleo	VM	Control del núcleo y del almacenamiento
Trabajo por lotes de minutos	Contenedor	Sobrecarga de arranque despreciable
Función invocada miles de veces por segundo	microVM	Aislamiento sin pagar 30 s de arranque

Lo que no es criterio válido: «los contenedores son más modernos». Un contenedor y una VM resuelven problemas de aislamiento distintos, y el coste de equivocarse no aparece en las pruebas: aparece cuando alguien escapa del núcleo compartido.

Ejemplo trabajado

CloudShop quiere ofrecer reglas de descuento programables por sus comercios: cada uno sube un fragmento de código que se ejecuta en cada compra. El equipo propone contenedores «porque ya usamos Kubernetes». Se evalúa con las dos preguntas.

Pregunta 1 — ¿de quién es el código? De terceros y sin auditar. El aislamiento por núcleo compartido pone en juego toda la interfaz de llamadas al sistema:

```
$ docker run --rm alpine sh -c 'grep -c . /proc/kallsyms; ls /proc/sys/kernel | wc -l'
# la superficie visible es la del núcleo del anfitrión
```

Se cuantifica el riesgo con datos, no con opinión. Escapes históricos por CVE de contenedores conocidos: CVE-2019-5736 (reescritura de runc desde el contenedor), CVE-2022-0492 (escalada por cgroups v1), CVE-2024-21626 (fuga de descriptor en runc). Todos permitieron ejecución en el anfitrión desde dentro.

Pregunta 2 — ¿qué perfil de arranque? Una regla se ejecuta por compra: hasta 600 invocaciones por segundo en pico, de unos 20 ms cada una. Una VM clásica queda descartada por aritmética:

```
arranque de VM      ≈ 40 s
trabajo útil        ≈ 0,02 s
sobrecarga          = 40 / 0,02 = 2.000x    inviable
```

Y el modelo de preaprovisionamiento tampoco cierra:

```
600 inv/s × 0,02 s = 12 ejecuciones concurrentes en media
VM preaprovisionadas para pico (×3)      = 36 VM permanentes
memoria: 36 × 1 GB de sobrecarga          = 36 GB solo de hipervisor
```

Con microVM:

```
arranque de microVM ≈ 0,125 s
sobrecarga          = 0,125 / 0,02 = 6,25x    asumible con reutilización
memoria: 36 × 5 MB   = 180 MB
```

180 MB frente a 36 GB, con el mismo aislamiento de hardware. Ese factor de 200 es lo que hace viable el producto.

Decisión registrada: microVM por comercio, reutilizada entre invocaciones del mismo comercio y destruida al cambiar de comercio. El contenedor se descarta no por rendimiento —sería mejor— sino porque el requisito es ejecutar código no confiable de inquilinos distintos, y ahí el núcleo compartido no es una frontera aceptable.

Límite explícito de la decisión: si en el futuro el código dejara de ser de terceros —por ejemplo, si se pasara a un lenguaje de reglas restringido que no permite ejecución arbitraria— la pregunta 1 cambiaría de respuesta y el contenedor volvería a ser correcto.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/008-virtualizacion-hipervisores-e-  
imagenes/lab.py
```

El laboratorio selecciona el motor de práctica virtualization y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa comparativa-de-aislamiento en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una comparación medida de aislamiento y consumo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye comparativa-de-aislamiento para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se ejecuta código de terceros en contenedores compartiendo núcleo	Se confundió aislamiento de vista con aislamiento de seguridad	Para código no confiable usa frontera de hardware: microVM o VM dedicada.
Un contenedor comprometido da acceso root al anfitrión	El espacio de nombres de usuario no estaba activado, así que UID 0 dentro es UID 0 fuera	Activa el remapeo de usuarios y ejecuta el proceso con un UID sin privilegios.
Un contenedor consume toda la memoria y el núcleo mata a otro distinto	No había límite de cgroup: el OOM killer elige por consumo, no por culpa	Define límites de memoria en todos los contenedores, no solo en los sospechosos.
Cada cambio de una línea de código reconstruye y redescarga cientos de MB	El código se copió antes de instalar dependencias e invalida la capa de estas	Ordena las capas de menos a más volátil: dependencias primero, código al final.
Un despliegue reproducible deja de serlo sin que nadie cambie nada	Se referenció la imagen por etiqueta mutable en vez de por digest	Fija imagen@sha256:... en todo lo que deba ser reproducible.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué aísla un hipervisor que un espacio de nombres no, y qué superficie de ataque queda en cada caso?
2. ¿Por qué no se pueden ejecutar contenedores Windows sobre un núcleo Linux, si la imagen trae todas sus bibliotecas?
3. Una carga se invoca 600 veces por segundo y dura 20 ms. ¿Por qué una VM clásica es inviable y una microVM no?
4. Por defecto, ¿qué UID del anfitrión corresponde a root dentro de un contenedor, y qué mitigación lo cambia?
5. ¿Por qué 50 contenedores de la misma imagen no ocupan 50 veces su tamaño en disco?

Referencias

- Agache, A. et al. (2020). Firecracker: Lightweight Virtualization for Serverless Applications. USENIX NSDI — cifras de arranque y sobrecarga. <<https://www.usenix.org/conference/nsdi20/presentation/agache>>
- Kerrisk, M. namespaces(7) — los siete espacios de nombres de Linux y su semántica. <<https://man7.org/linux/man-pages/man7/namespaces.7.html>>
- Kerrisk, M. cgroups(7) — límites, contabilidad y jerarquía de control de recursos. <<https://man7.org/linux/man-pages/man7/cgroups.7.html>>
- Open Container Initiative (2024). Image Specification — manifiesto, capas y direccionamiento por digest. <<https://github.com/opencontainers/image-spec/blob/main/spec.md>>
- Rice, L. (2020). Container Security, caps. 4-8 — espacios de nombres, capacidades y vectores de escape.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 008 Virtualización, hipervisores e imágenes

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega comparativa-de-aislamiento con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.

- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

009 — APIs REST, autenticación y contratos

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: api · Estado: EXECUTABLECORE

Propósito

Diseñar y consumir APIs HTTP tratando el contrato como la unidad que de verdad se despliega. Toda la nube se opera por API —la consola es solo un cliente más— y las decisiones de esta clase sobre idempotencia, paginación, autenticación y versionado son las que hacen que una automatización sobreviva a un reintento, a un fallo parcial y a un cambio del proveedor.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir el método HTTP correcto a partir de sus propiedades de seguridad e idempotencia, no de la costumbre.
2. Hacer idempotente una operación que por naturaleza no lo es, mediante clave de idempotencia.
3. Distinguir autenticación de autorización y explicar qué prueba un token y qué no.
4. Paginar un conjunto grande sin perder ni duplicar elementos cuando la colección cambia durante el recorrido.
5. Evolucionar un contrato sin romper a los consumidores existentes, sabiendo qué cambio es compatible y cuál no.

Conceptos centrales

Concepto	Comprensión verificable
método seguro	El que no altera el estado del servidor: GET, HEAD, OPTIONS. Permite que proxies y navegadores lo repitan o lo precarguen sin consecuencias, lo que a su vez prohíbe usar GET para operaciones con efecto.
idempotencia	Propiedad por la que N ejecuciones idénticas dejan el mismo estado que una. PUT y DELETE lo son por definición; POST no, y por eso necesita una clave explícita para poder reintentarse.
clave de idempotencia	Identificador único que el cliente genera y envía para que el servidor reconozca un reintento y devuelva el resultado original en vez de repetir el efecto.
token portador	Credencial que autoriza a quien la presente, sin más prueba. Quien lo roba lo puede usar: de ahí que su vida deba ser corta y su transporte siempre cifrado.
cursor	Marca opaca de posición en una colección, estable frente a inserciones y borrados. Sustituye al desplazamiento numérico, que salta o repite elementos cuando la colección cambia entre páginas.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

Flujo de razonamiento

```
flowchart LR
  C["cliente"] -->|"POST /pagos<br/>Idempotency-Key: 7d2b8e"| A["API"]
  A --> K{"¿clave ya vista?"}
  K -->|"no"| E["ejecuta y guarda<br/>clave → resultado"]
  K -->|"sí"| R["devuelve el resultado<br/>guardado, sin repetir"]
  E --> O1["201 Created"]
  R --> O2["200 con el mismo cuerpo"]
  O1 -->|"timeout de red:<br/>el cliente reintenta"| C
```


Desarrollo

1. Los métodos tienen propiedades, no solo nombres

Elegir el método no es cuestión de estilo: sus propiedades determinan qué pueden hacer con la petición los proxies, los navegadores y las bibliotecas de reintento.

Método	Seguro	Idempotente	Consecuencia práctica
GET	Sí	Sí	Cacheable; un proxy puede repetirlo sin avisar
HEAD	Sí	Sí	Como GET sin cuerpo; útil para comprobar existencia
PUT	No	Sí	Reintentable sin riesgo: sustituye el recurso entero
DELETE	No	Sí	Reintentable: borrar dos veces deja el mismo estado
POST	No	No	No reintentable sin protección adicional
PATCH	No	No garantizada	Depende del formato del parche

La fila de GET explica un incidente clásico: exponer GET /pedidos/42/cancelar parece cómodo hasta que un precargador del navegador, un rastreador o un proxy lo visitan y cancelan pedidos que nadie pidió cancelar. Si tiene efecto, no puede ser GET.

Y la fila de POST explica el problema central de esta clase: es el único método común que no se puede reintentar con seguridad, y es justo el que se usa para crear pagos, pedidos y recursos.

2. Idempotencia: la propiedad que hace segura una red poco fiable

Un cliente envía POST /pagos, la red se corta antes de la respuesta y el cliente no sabe si el pago se ejecutó. Solo tiene dos opciones malas: reintentar y arriesgarse a cobrar dos veces, o no reintentar y arriesgarse a no cobrar.

La solución es que el cliente genere un identificador único por intención y lo repita en cada reintento:

```
POST /pagos HTTP/1.1
Idempotency-Key: <uuid v4 que genera el cliente>
Content-Type: application/json

{"pedido": "A-1042", "importe": "149.90", "moneda": "CLP"}
```

El servidor guarda la clave junto al resultado. Al recibirla de nuevo:

1. Si no la ha visto, ejecuta y persiste clave → resultado en la misma transacción que el efecto.
2. Si ya la vio y terminó, devuelve el resultado original sin repetir el cobro.
3. Si ya la vio y sigue en curso, responde 409 Conflict.

El punto 1 es donde suele fallar la implementación: si el registro de la clave y el cobro no son atómicos, existe una ventana en la que el cobro ocurrió y la clave no se guardó, y el reintento cobra otra vez. La clave debe persistirse en la misma transacción que el efecto que protege.

Es el mismo mecanismo que en la parte 09 aparecerá como deduplicación en sistemas de mensajería, y por la misma razón: la entrega «exactamente una vez» no existe a nivel de red; se construye con idempotencia en el receptor.

3. Autenticación no es autorización

Son dos preguntas distintas y se resuelven en momentos distintos:

- Autenticación: ¿quién eres? Se resuelve una vez y produce una credencial.
- Autorización: ¿puedes hacer esto sobre este recurso? Se resuelve en cada petición.

Saltarse la segunda produce la vulnerabilidad más común de las APIs, catalogada como BOLA (Broken Object Level Authorization, primer puesto del OWASP API Security Top 10):

```
# vulnerable: autentica pero no autoriza sobre el objeto
@app.get("/pedidos/{id}")
def ver(id: str, usuario = Depends(authenticar)):
    return db.pedidos.get(id)          # cualquiera autenticado ve CUALQUIER pedido
```

```
# correcto: la autorización es por objeto
@app.get("/pedidos/{id}")
def ver(id: str, usuario = Depends(authenticar)):
    pedido = db.pedidos.get(id)
    if pedido is None or pedido.cliente_id != usuario.cliente_id:
        raise HTTPException(404) # 404, no 403: no revelar existencia
    return pedido
```

Devolver 404 en vez de 403 es deliberado: un 403 confirma que el recurso existe y permite enumerar identificadores ajenos. Es la misma lógica por la que un formulario de acceso no debe decir «esa contraseña es incorrecta» frente a «ese usuario no existe».

Los códigos correctos: 401 significa «no sé quién eres» y admite reintento con credenciales; 403 significa «sé quién eres y no puedes», y reintentar no ayuda.

4. Paginación: por qué el desplazamiento numérico pierde datos

GET /pedidos?offset=100&limit=50 parece razonable y falla en cuanto la colección cambia durante el recorrido. Con orden descendente por fecha:

```
t0 el cliente lee offset=0..49 (elementos 1-50)
t1 se insertan 3 pedidos nuevos al principio
t2 el cliente lee offset=50..99
  → los elementos 48, 49 y 50 se han desplazado y se REPITEN
  → si en vez de insertar se hubieran borrado 3, se PERDERÍAN
```

Y hay un segundo problema, de coste: OFFSET 100000 obliga a la base de datos a leer y descartar cien mil filas antes de devolver la página. El coste crece linealmente con la profundidad.

La paginación por cursor usa una marca estable —la clave del último elemento— en vez de una posición:

```
GET /pedidos?limit=50
{"datos": [...], "siguiente": "eyJ0IjoimjAyNi0wOC0wMVQwOT0xMiJ9"}

GET /pedidos?limit=50&cursor=eyJ0IjoimjAyNi0wOC0wMVQwOT0xMiJ9
```

La consulta pasa a ser WHERE (creado, id) < (:t, :id) ORDER BY creado DESC, id DESC LIMIT 50, que usa índice y cuesta lo mismo en la página 1 que en la 10.000. El cursor debe ser opaco para el cliente: si se documenta su estructura, se convierte en parte del contrato y ya no se puede cambiar.

El orden debe incluir un desempate único (id): sin él, dos filas con la misma marca de tiempo pueden repetirse o perderse en el corte de página.

5. Evolucionar sin romper: qué cambio es compatible

Un contrato desplegado tiene consumidores que no controlas. La distinción práctica:

Cambio	¿Compatible?	Por qué
Añadir un campo opcional a la respuesta	Sí	Los clientes ignoran lo que no conocen
Añadir un parámetro opcional	Sí	Su ausencia mantiene el comportamiento
Añadir un valor nuevo a un enumerado	No	Los clientes con switch exhaustivo fallan
Renombrar o eliminar un campo	No	Rotura directa
Cambiar un tipo (número → cadena)	No	Rotura de deserialización
Hacer obligatorio un campo opcional	No	Rotura para quien no lo enviaba
Endurecer una validación	No	Peticiones antes válidas empiezan a fallar

La fila del enumerado es la que más sorprende: parece aditiva, pero un consumidor que hace exhaustivo sobre los valores conocidos falla al recibir uno nuevo. Por eso los contratos maduros documentan desde el principio que los enumerados pueden crecer y exigen un caso por defecto.

Cuando el cambio es incompatible, versionar. Y la regla que importa no es dónde poner la versión —ruta, cabecera o tipo de medio— sino cuánto tiempo mantienes la anterior viva y cómo avisas: una versión sin fecha de retirada publicada no es una versión, es deuda.

El campo Deprecation y Sunset (RFC 8594) permiten anunciarlo en la propia respuesta, para que el cliente se entere antes de romperse.

Ejemplo trabajado

Durante una promoción, 1 de cada 400 clientes de CloudShop aparece cobrado dos veces. El importe duplicado siempre coincide, y los dos cargos distan menos de 30 segundos.

La hipótesis es reintento sobre POST no idempotente. Se confirma en los logs:

```
$ jq -r 'select(.ruta=="/pagos") | [.trace, .pedido, .estado, .ms] | @tsv' pagos.jsonl | sort | head -4
4f2a9c A-1042 timeout 30021
7d2b8e A-1042 201 412
```

Dos trazas distintas para el mismo pedido: la primera agotó el plazo del cliente a los 30 s, pero el servidor la completó—el cargo existe— y el cliente reintentó con una petición nueva.

La aritmética del incidente:

peticiones de pago en la promoción	48.000
timeout del cliente	30 s
p99,8 de latencia del proveedor	31 s
peticiones que superan el timeout	$\approx 0,25\% \rightarrow 120$
de ellas, completadas en el servidor	$\approx 100\% \rightarrow 120$ cobros dobles
observado	118

Cuadra: el problema no es la tasa de error del proveedor, es que el timeout del cliente es menor que la cola de latencia del servidor. Subir el timeout reduce la frecuencia pero no elimina la clase de fallo: siempre habrá una petición que se corte después de tener efecto.

Corrección con clave de idempotencia, generada por el cliente una vez por intención y reutilizada en los reintentos:

```
clave = str(uuid.uuid4()) # fuera del bucle: la MISMA en cada reintento
for intento in range(3):
    try:
        r = sesion.post("/pagos", json=cuerpo,
                        headers={"Idempotency-Key": clave}, timeout=35)
        break
    except (TimeoutError, ConnectionError):
        time.sleep(2 ** intento + random.uniform(0, 1))
```

Y en el servidor, con la clave persistida en la misma transacción que el cargo:

```
BEGIN;
INSERT INTO idempotencia (clave, estado) VALUES ($1, 'en_curso')
ON CONFLICT (clave) DO NOTHING;
-- si no insertó ninguna fila, es un reintento: devolver lo guardado
INSERT INTO cargos (pedido, importe) VALUES ($2, $3);
UPDATE idempotencia SET estado='hecho', respuesta=$4 WHERE clave=$1;
COMMIT;
```

Resultado tras el despliegue: 0 cobros dobles en 52.000 pagos de la promoción siguiente, con 96 reintentos que devolvieron el resultado original en vez de repetir el cargo.

El detalle que decide todo es dónde se genera la clave: fuera del bucle de reintento. Generarla dentro produce una clave distinta por intento y deja el sistema exactamente igual de roto, con la ilusión de estar protegido.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/009-apis-rest-autenticacion-y-contratos/lab.py
```

El laboratorio selecciona el motor de práctica api y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.

3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cliente-api-verificado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un contrato versionado con pruebas positivas y negativas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cliente-api-verificado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se ejecutan acciones que nadie solicitó	Una operación con efecto se expuso como GET y la repitió un precargador o un proxy	GET debe ser seguro; usa POST, PUT o DELETE para cualquier cosa con efecto.
Un reintento tras timeout duplica un cobro	POST no es idempotente y no había clave de idempotencia	Genera la clave una vez por intención, fuera del bucle, y persístela en la misma transacción que el efecto.
Un usuario autenticado accede a datos de otro cambiando el identificador de la URL	Se autenticó pero no se autorizó por objeto (BOLA)	Comprueba la propiedad del recurso en cada petición y responde 404, no 403.
Un recorrido paginado repite o pierde elementos	Paginación por desplazamiento sobre una colección que cambia	Usa cursor opaco sobre una clave estable con desempate único.
Añadir un valor a un enumerado rompe consumidores en producción	Se asumió que todo cambio aditivo es compatible	Documenta desde el principio que los enumerados crecen y exige un caso por defecto en el cliente.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué PUT y DELETE se pueden reintentar sin riesgo y POST no?
2. ¿En qué punto exacto debe persistirse la clave de idempotencia para que no quede una ventana de doble efecto?
3. Un usuario autenticado pide un recurso ajeno. ¿Por qué 404 es mejor respuesta que 403?
4. Explica con un ejemplo cómo la paginación por desplazamiento pierde un elemento cuando la colección cambia.

5. ¿Cuáles de estos cambios rompen a un consumidor: añadir un campo opcional, añadir un valor a un enumerado, hacer obligatorio un campo opcional?

Referencias

- Fielding, R. y Reschke, J., eds. (2022). RFC 9110: HTTP Semantics, secs. 9.2.1-9.2.2 — métodos seguros e idempotentes. <<https://www.rfc-editor.org/rfc/rfc9110#section-9.2>>
- Jena, J. et al. (2024). The Idempotency-Key HTTP Header Field, IETF draft — semántica y manejo de reintentos. <<https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/>>
- OWASP (2023). API Security Top 10, API1:2023 Broken Object Level Authorization. <<https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/>>
- Wilde, E. (2019). RFC 8594: The Sunset HTTP Header Field — anunciar la retirada de un contrato. <<https://www.rfc-editor.org/rfc/rfc8594>>
- Nottingham, M. (2022). RFC 9205: Building Protocols with HTTP — buenas prácticas de diseño sobre HTTP. <<https://www.rfc-editor.org/rfc/rfc9205>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 009 APIs REST, autenticación y contratos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cliente-api-verificado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

010 — Responsabilidad compartida y pensamiento de riesgo

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Trazar con precisión dónde termina la responsabilidad del proveedor y empieza la tuya, y convertir esa línea en un método de análisis de riesgo aplicable antes de escribir infraestructura. Es la clase que explica por qué casi ningún incidente público «de la nube» fue del proveedor, y la que fija el criterio con el que se juzgarán todas las decisiones de las 278 clases restantes.

Resultados de aprendizaje

Al finalizar podrás:

1. Situar cualquier control concreto del lado correcto de la línea según el modelo de servicio contratado.
2. Distinguir disponibilidad del servicio, durabilidad del dato y confidencialidad, y qué garantiza el proveedor de cada una.
3. Leer un SLA identificando qué mide, qué excluye y cuál es la compensación real.
4. Aplicar un método de análisis de riesgo que produzca controles verificables en vez de una lista de miedos.
5. Calcular el riesgo residual de una decisión y declararlo explícitamente en vez de dejarlo implícito.

Conceptos centrales

Concepto	Comprensión verificable
responsabilidad compartida	Reparto contractual de controles entre proveedor y cliente. El proveedor responde de la seguridad de la nube; el cliente, de la seguridad en la nube. La línea se mueve con el modelo de servicio, nunca desaparece.
SLA	Compromiso contractual con una métrica, un umbral y una compensación. Casi siempre mide disponibilidad del plano de servicio y excluye lo que más duele: tus datos, tu configuración y tu lucro cesante.
durabilidad	Probabilidad de que un objeto almacenado siga existiendo tras un periodo. Es independiente de la disponibilidad: un dato puede ser durable y estar inaccesible, o estar disponible y haber sido borrado por ti.
riesgo residual	Lo que queda después de aplicar los controles. No es un fallo del análisis: es el resultado esperado. Un diseño honesto lo nombra y dice quién lo acepta.
radio de impacto	Alcance de lo que se rompe cuando algo falla. Reducirlo —por celdas, cuentas o regiones— suele ser más barato y más eficaz que intentar evitar el fallo.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    subgraph tuyo["Siempre tuyo · en los tres modelos"]
        D["Datos y su clasificación"]
        I["Identidades y permisos"]
        CFG["Configuración que eliges"]
    end
    subgraph movil["La línea se mueve con el modelo"]
        AP["Aplicación"] --> RT["Runtime"] --> SO["Sistema operativo"]
    end
    subgraph prov["Siempre del proveedor"]
        HW["Hardware, red física, energía"]
        DC["Seguridad física del centro de datos"]
    end
    tuyo -.->|IaaS: S0 tuyo<br/>PaaS: runtime del proveedor<br/>SaaS: solo datos y config| movil
    movil -.-> prov
```

Desarrollo

1. Lo que nunca cambia de lado

El modelo de servicio mueve la línea, pero tres cosas permanecen siempre del lado del cliente, en IaaS, en PaaS y en SaaS:

1. Los datos: qué guardas, cómo los clasificas y cuánto tiempo los conservas.
2. Las identidades y sus permisos: quién puede hacer qué.
3. La configuración que eliges: si un bucket es público, lo hiciste público tú.

Esto explica un hecho incómodo: la inmensa mayoría de las brechas atribuidas públicamente a «la nube» fueron configuraciones del cliente, no fallos del proveedor. Almacenamiento expuesto sin autenticación, claves de acceso subidas a un repositorio público, permisos comodín concedidos «temporalmente» hace dos años.

Gartner lleva años sosteniendo la misma proyección: hasta 2025, en torno al 99 % de los fallos de seguridad en la nube serán culpa del cliente. El dato importa menos que su consecuencia de diseño: el trabajo de seguridad no está en elegir proveedor, está en el lado que te toca.

El error simétrico también existe: asumir que nada hace el proveedor. El cifrado del disco físico, la destrucción segura de medios, el control de acceso al centro de datos y el parcheo del hipervisor no son tuyos y no deberías replicarlos.

2. Disponibilidad, durabilidad y confidencialidad son tres cosas distintas

Se confunden a diario y cada una tiene garantías, riesgos y controles diferentes:

Propiedad	Pregunta	Garantía típica	Qué la rompe
Disponibilidad	¿Puedo acceder ahora?	99,9 %-99,99 % en SLA	Caída de zona, saturación, error de red
Durabilidad	¿Seguirá existiendo?	99,999999999 % anual (11 nueves)	Un borrado tuyo, no un fallo de disco
Confidencialidad	¿Solo lo ve quien debe?	No hay SLA	Configuración, permisos, fuga de credenciales

Los once nueves de durabilidad de un almacenamiento de objetos significan que la probabilidad anual de perder un objeto dado por fallo de infraestructura es 10^{-11} . Con 10 millones de objetos:

$$\begin{aligned} \text{pérdida esperada} &= 10.000.000 \times 10^{-11} = 0,0001 \text{ objetos al año} \\ &\approx 1 \text{ objeto cada } 10.000 \text{ años} \end{aligned}$$

Y sin embargo se pierden datos constantemente. La razón es que la durabilidad no protege del borrado autorizado: un DELETE con credenciales válidas, un ciclo de vida mal configurado o un ransomware con tus permisos destruyen el objeto sin que la infraestructura falle en absoluto.

De ahí que la copia de seguridad siga siendo tuya, con dos propiedades que la durabilidad no da: versionado —para volver atrás— e inmutabilidad —para que ni tus propias credenciales puedan borrarla—. Se retoma en la parte 21.

3. Leer un SLA por lo que excluye

Un SLA tiene tres partes y la tercera casi nunca es la que se recuerda:

1. Métrica y umbral: por ejemplo, 99,99 % mensual de disponibilidad del servicio.
2. Compensación: un porcentaje de crédito sobre lo facturado de ese servicio.
3. Exclusiones: lo que no cuenta como indisponibilidad.

La aritmética del umbral, en un mes de 30 días:

99,9 % → 43,2 min de caída permitida al mes
99,95 % → 21,6 min
99,99 % → 4,3 min
99,999 % → 26 s

El salto de 99,9 a 99,99 no es «un poco mejor»: exige reducir la ventana de fallo diez veces, lo que en la práctica obliga a redundancia entre zonas y a automatizar la conmutación, porque ninguna intervención humana cabe en 4,3 minutos.

Las exclusiones habituales, que conviene buscar antes de firmar:

- Indisponibilidad por tu configuración o tu código.
- Ventanas de mantenimiento anunciadas.
- Fuerza mayor y fallos de tu proveedor de red.
- Servicios en vista previa o beta.

Y la exclusión que más importa: la compensación es un crédito de servicio, no una indemnización. Si una caída de 4 horas te cuesta 80.000 USD en ventas, el SLA te devuelve un porcentaje de lo que pagues por ese servicio ese mes —quizá 200 USD—. El SLA no transfiere tu riesgo de negocio: acota el del proveedor. Tu continuidad la pagas tú, con arquitectura.

4. Un método de riesgo que produce controles, no miedos

Una lista de amenazas sin estructura no sirve para decidir. Cinco preguntas, en orden, sí:

1. ¿Qué protegemos? El activo, con su clasificación. «La base de datos» no es un activo; «los datos personales de clientes, categoría alta» sí.
2. ¿De quién y de qué? Actor y capacidad: un atacante externo sin credenciales, un empleado con acceso legítimo, un error de operación, un fallo de infraestructura.
3. ¿Por dónde? El vector concreto: credencial filtrada, permiso excesivo, dependencia comprometida, error de configuración.
4. ¿Qué control lo corta? Preventivo, detectivo o correctivo, y verificable.
5. ¿Qué queda? El riesgo residual, nombrado y aceptado por alguien con autoridad para aceptarlo.

La columna que separa un análisis útil de un documento decorativo es la cuarta: cada control debe tener una prueba. «Cifrado en tránsito» no es un control; «TLS 1.2 mínimo, verificado por una prueba automática que falla el despliegue si se acepta TLS 1.0» sí lo es.

Y los tres tipos no son intercambiables. Un control preventivo que falle en silencio es peor que uno detectivo que avise: el primero da confianza falsa. Por eso todo control preventivo necesita una prueba negativa —comprobar que efectivamente deniega— que es exactamente lo que hacen los lab.py de este programa.

5. Radio de impacto: contener sale más barato que evitar

Los fallos ocurren. La pregunta de diseño no es cómo evitarlos todos, sino qué se rompe cuando ocurren.

Una cuenta, todo junto: 1 credencial comprometida → todo el negocio
Cuentas separadas por entorno: 1 credencial comprometida → un entorno
Cuentas por dominio + entorno: 1 credencial comprometida → un dominio de un entorno

La separación no impide la brecha; acota su alcance, y eso suele ser mucho más barato que intentar hacer la brecha imposible. Es el mismo razonamiento de la clase 007 sobre capacidades: no «qué necesita para funcionar», sino «qué obtiene quien lo comprometa».

Tres preguntas que hay que poder responder de cualquier diseño:

- ¿Qué se rompe si esto falla? Si la respuesta es «todo», el diseño tiene un punto único de fallo aunque el componente sea redundante.
- ¿Cuánto tarda en detectarse? Un fallo silencioso de días es peor que una caída ruidosa de minutos.
- ¿Se puede revertir? Un cambio irreversible exige un nivel de control distinto al de uno reversible.

Estas tres preguntas son el guion de los ADR de la parte 12 y de los game days de la parte 21.

Ejemplo trabajado

Un informe de exposición avisa a CloudShop de que un bucket con facturas de clientes es accesible sin autenticación. El equipo quiere responder «el proveedor garantiza cifrado y once nueves». Se aplica el método.

1. ¿Qué protegemos? 412.000 facturas en PDF con nombre, RUT, dirección y detalle de compra. Clasificación: datos personales, categoría alta. Retención legal: 6 años.
2. ¿De quién? Actor externo sin credenciales. No hace falta más: el objeto es legible por cualquiera con la URL.
3. ¿Por dónde? Se reconstruye el vector:

```
$ aws s3api get-bucket-policy --bucket cloudshop-facturas | jq -r '.Policy' | jq '.Statement[0]'
{"Effect": "Allow", "Principal": "*", "Action": "s3:GetObject", ...}
$ aws cloudtrail lookup-events --lookup-attributes \
  AttributeKey=EventName,AttributeValue=PutBucketPolicy --max-items 1 \
  | jq -r '.Events[0] | "\(.EventTime) \(.Username)"'
2024-11-14T16:22:08Z ci-deploy-role
```

La política la puso un despliegue hace 21 meses. El cifrado en reposo estaba activo todo el tiempo y no sirvió de nada: cifra frente a quien acceda al disco físico, no frente a quien hace una petición autorizada por la política. La durabilidad de once nueves tampoco: garantiza que el objeto no se pierda, no que no se lea.

Cuantificación de la exposición:

```
objetos legibles          412.000
ventana                   21 meses ≈ 638 días
peticiones GET anónimas en los logs  1.847 (de 12 IP distintas)
```

4. ¿Qué controles, y cómo se verifican?

Tipo	Control	Prueba
Preventivo	Bloqueo de acceso público a nivel de cuenta	Prueba negativa: intentar publicar y comprobar que falla
Preventivo	Política de organización que prohíbe Principal: ""	Despliegue de prueba que debe ser rechazado
Detectivo	Alerta sobre cualquier PutBucketPolicy	Ejecutarla y verificar que notifica en < 5 min
Correctivo	Versionado + retención inmutable	Restaurar un objeto borrado en un simulacro

Cada fila tiene una prueba ejecutable. La segunda columna sin la tercera es una intención, no un control.

5. ¿Qué riesgo queda? Los datos estuvieron expuestos 638 días y eso no se puede revertir: hay que asumir que fueron copiados y actuar en consecuencia —notificación a los afectados y a la autoridad, según la normativa aplicable—. Además, un operador con permiso legítimo sigue pudiendo exportarlos; ese riesgo se acota con registro de acceso y revisión periódica, no se elimina.

La conclusión que importa: ni el cifrado ni la durabilidad protegían de esto, porque ninguno de los dos estaba del lado del problema. El control que faltaba era de configuración, y la configuración es siempre del cliente en los tres modelos de servicio.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/010-responsabilidad-compartida-y-pensamiento-de-riesgo/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-de-responsabilidad en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-de-responsabilidad para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se asume que el cifrado del proveedor protege los datos de un acceso indebido	El cifrado en reposo protege frente al disco físico, no frente a una petición autorizada	Separa los controles por propiedad: confidencialidad se protege con permisos y política, no con cifrado en reposo.
Se confía en los once nuevos de durabilidad como estrategia de respaldo	La durabilidad no protege del borrado autorizado, que es la causa real de pérdida	Añade versionado y retención inmutable; son tuyos, no del proveedor.
Tras una caída larga, el SLA compensa una fracción mínima de la pérdida	La compensación es un crédito sobre lo facturado de ese servicio, no una indemnización	Dimensiona la continuidad con arquitectura; el SLA acota el riesgo del proveedor, no el tuyo.
El análisis de riesgo es una lista de amenazas que nadie usa para decidir	No llegó a controles verificables ni nombró el riesgo residual	Exige a cada control una prueba ejecutable y un responsable que acepte lo que queda.
Una credencial comprometida da acceso a todo el negocio	No hay separación de cuentas ni entornos: el radio de impacto es total	Separa por entorno y dominio; contener el alcance es más barato que evitar la brecha.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué tres responsabilidades siguen siendo del cliente en IaaS, PaaS y SaaS por igual?
2. Un almacenamiento con once años de durabilidad pierde datos de un cliente. ¿Cómo es posible sin que falle la infraestructura?
3. ¿Cuántos minutos de caída mensual permite un SLA de 99,99 %, y por qué eso descarta la intervención humana?
4. Un control dice «cifrado en tránsito». ¿Qué le falta para ser verificable?
5. ¿Por qué un control preventivo que falla en silencio es peor que uno detectivo?

Referencias

- AWS (2024). Shared Responsibility Model — reparto de controles por modelo de servicio. <<https://aws.amazon.com/compliance/shared-responsibility-model/>>
- Microsoft (2024). Shared responsibility in the cloud — tabla comparada IaaS/PaaS/SaaS. <<https://learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility>>
- Cloud Security Alliance (2024). Top Threats to Cloud Computing — taxonomía de amenazas con incidentes documentados. <<https://cloudsecurityalliance.org/research/topics/top-threats>>
- Shostack, A. (2014). Threat Modeling: Designing for Security — las cuatro preguntas y el uso de STRIDE.
- NIST (2018). Framework for Improving Critical Infrastructure Cybersecurity v1.1 — funciones identificar, proteger, detectar, responder y recuperar. <<https://doi.org/10.6028/NIST.CSWP.04162018>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 010 Responsabilidad compartida y pensamiento de riesgo

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-de-responsabilidad con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

011 — Costo, energía, capacidad y medición básica

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

Propósito

Convertir el coste en una magnitud de ingeniería con la que se decide, no en una factura que se revisa al final del mes. Aquí se establece la unidad económica que el programa usará en las 277 clases restantes: coste por transacción útil. Sin ella, «optimizar costes» es apagar cosas al azar y esperar que nada se rompa.

Resultados de aprendizaje

Al finalizar podrás:

1. Definir una unidad de coste ligada a algo que el negocio entiende, y calcularla a partir de la factura.
2. Aplicar la ley de Little para dimensionar capacidad a partir de tasa de llegada y tiempo de servicio.
3. Explicar por qué la latencia se dispara antes de llegar al 100 % de utilización, y a partir de qué punto.
4. Comparar modelos de precio —bajo demanda, compromiso, capacidad excedente— con un umbral de utilización calculado.
5. Estimar el coste energético de una carga y situarlo frente a su coste monetario.

Conceptos centrales

Concepto	Comprensión verificable
unidad de coste	Denominador que convierte la factura en una métrica decidible: coste por pedido, por usuario activo, por GB procesado. Sin denominador, un aumento de factura no distingue crecimiento de derroche.
utilización	Fracción del tiempo que un recurso está ocupado, entre 0 y 1. Por encima de 0,7 el tiempo de espera crece de forma no lineal, así que perseguir el 100 % es perseguir una cola infinita.
ley de Little	En un sistema estable, $L = \lambda \cdot W$: el número medio de elementos en el sistema es la tasa de llegada por el tiempo medio de permanencia. No supone ninguna distribución, lo que la hace aplicable casi siempre.
coste amortizado	Coste efectivo de un compromiso repartido en su plazo. Permite comparar un descuento por reserva de 1 o 3 años con el precio bajo demanda sobre la misma base.
PUE	Power Usage Effectiveness: energía total del centro de datos dividida por la que llega al equipamiento informático. 1,0 sería perfecto; los hiperescalares operan cerca de 1,1 y una sala tradicional ronda 1,8.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
F["Factura mensual<br/>USD"] --> U["dividir por<br/>unidad de negocio"]
T["Transacciones útiles<br/>pedidos, usuarios, GB"] --> U
U --> C["Coste unitario<br/>USD / pedido"]
C --> D["¿sube o baja<br/>mes a mes?"]
D -->| "sube con volumen plano" | W["derroche: investigar"]
D -->| "baja con volumen creciente" | E["economía de escala: sano"]
D -->| "plano con volumen creciente" | L["lineal: sin escala, revisar diseño"]
```

Desarrollo

1. Sin denominador, la factura no dice nada

«La factura subió un 40 %» es una observación inútil hasta que se divide por algo. Con denominador, la misma cifra puede significar tres cosas opuestas:

mes 1: 12.400 USD / 620.000 pedidos = 0,0200 USD/pedido
mes 2: 17.360 USD / 980.000 pedidos = 0,0177 USD/pedido ← +40 % factura, -11 % unitario
mes 3: 17.360 USD / 610.000 pedidos = 0,0285 USD/pedido ← +43 % unitario: hay derroche

El mes 2 es crecimiento sano: se factura más porque se vende más, y cada pedido cuesta menos. El mes 3 con la misma factura es una alarma: el volumen cayó y el coste no.

La unidad debe cumplir tres condiciones para servir:

1. La entiende el negocio: pedidos, usuarios activos, GB procesados. No «horas de instancia».
2. Se puede atribuir: existe un mecanismo de etiquetado que asigna cada recurso a un producto o equipo.
3. Se mide igual todos los meses: cambiar el denominador rompe la serie histórica.

La condición 2 es la que exige trabajo previo: sin etiquetado consistente, el 30-40 % de la factura acaba en un cubo de «sin asignar» que nadie reclama y nadie optimiza. Por eso el etiquetado es la primera tarea de FinOps, antes que cualquier optimización.

2. La ley de Little: dimensionar sin adivinar

Para cualquier sistema estable, con independencia de la distribución de llegadas o servicios:

$$L = \lambda \cdot W$$

L = elementos en el sistema (conurrencia)
 λ = tasa de llegada (peticiones por segundo)
W = tiempo medio en el sistema (segundos)

Sirve para dimensionar antes de desplegar. Con 800 peticiones por segundo y 250 ms de latencia media:

$$L = 800 \times 0,250 = 200 \text{ peticiones concurrentes}$$

Si cada trabajador atiende una petición a la vez, hacen falta 200 trabajadores solo para el estado estacionario. Con 40 trabajadores por instancia:

$$\text{instancias} = 200 / 40 = 5 \quad (\text{mínimo teórico, utilización } 1,0)$$

Y ese «mínimo teórico» es exactamente el número que no hay que desplegar, por lo que viene a continuación.

La ley también funciona al revés, para detectar incoherencias: si mides 200 conexiones concurrentes y 800 peticiones por segundo, la latencia media tiene que ser 250 ms. Si tu panel dice 80 ms, alguna de las tres medidas está mal —normalmente porque se está midiendo la media de una distribución con cola larga—.

3. Por qué el 100 % de utilización es una trampa

En un sistema con llegadas aleatorias, el tiempo de espera no crece de forma lineal con la utilización. Para una cola M/M/1:

$$W = W_{\text{servicio}} / (1 - \rho) \quad \rho = \text{utilización}$$

Con un servicio de 100 ms:

ρ	Tiempo total	Multiplificador
0,50	200 ms	2×
0,70	333 ms	3,3×
0,80	500 ms	5×
0,90	1.000 ms	10×
0,95	2.000 ms	20×
0,99	10.000 ms	100×

Entre 0,70 y 0,90 la utilización sube 20 puntos y la latencia se triplica. Ese es el motivo de que el objetivo operativo habitual sea 0,60-0,70 y no 0,95: el 30 % de capacidad «ociosa» no es derroche, es lo que absorbe la varianza de las llegadas.

Aplicado al cálculo anterior:

mínimo teórico ($p = 1,0$) 5 instancias
 objetivo $p = 0,70$ $5 / 0,70 = 7,1 \rightarrow 8$ instancias
 tolerancia a la caída de una zona ($n+1$) $\rightarrow 9$ instancias

Las 4 instancias adicionales sobre el mínimo teórico no son exceso: son el precio de un percentil 95 estable y de sobrevivir a la pérdida de una zona. Optimizar costes recortando ahí es cambiar dinero por incidentes, y el intercambio hay que hacerlo explícito.

4. Modelos de precio: el umbral que decide

Los tres modelos comunes, con el mismo recurso como referencia:

Modelo	Precio relativo	Compromiso	Riesgo
Bajo demanda	1,00	Ninguno	Ninguno
Compromiso 1 año	0,60	Pagas uses o no	Sobredimensionar
Compromiso 3 años	0,40	Ídem, más largo	Cambio tecnológico
Capacidad excedente	0,10-0,30	Ninguno	Interrupción con preaviso corto

El umbral de utilización a partir del cual compensa comprometerse sale de igualar ambos costes:

coste bajo demanda = $u \times 730 \text{ h} \times P$
 coste comprometido = $730 \text{ h} \times 0,60 P$

umbral: $u \times P = 0,60 P \rightarrow u = 0,60$

Con más del 60 % de uso sostenido, el compromiso a un año sale a cuenta. Por debajo, no. Ese único número evita la mayoría de las discusiones sobre reservas.

La capacidad excedente merece su propio criterio: cuesta entre un 70 % y un 90 % menos, pero puede retirarse con un preaviso de dos minutos. Es correcta para trabajos por lotes reanudables, colas asíncronas y entornos de pruebas. Nunca para un componente cuya caída rompa una petición de usuario, salvo que la arquitectura absorba la pérdida sin degradación visible.

5. La energía es el coste que no aparece en la factura

El coste monetario ya incorpora la energía, pero la huella no, y cada vez más organizaciones tienen que declararla.

$\text{energía}_{\text{total}} = \text{potencia}_{\text{TI}} \times \text{horas} \times \text{PUE}$

Para 9 instancias de 80 W durante un mes:

potencia TI = $9 \times 80 \text{ W} = 720 \text{ W} = 0,72 \text{ kW}$
 energía TI = $0,72 \text{ kW} \times 730 \text{ h} = 525,6 \text{ kWh}$
 con PUE 1,12 = $525,6 \times 1,12 = 588,7 \text{ kWh/mes}$

Y su traducción a emisiones depende por completo de dónde se ejecuta:

región con 50 g CO₂e/kWh: $588,7 \times 0,050 = 29,4 \text{ kg CO}_2\text{e/mes}$
 región con 400 g CO₂e/kWh: $588,7 \times 0,400 = 235,5 \text{ kg CO}_2\text{e/mes}$

Un factor de 8 por la misma carga, con la misma factura. La elección de región, que en la clase 001 se justificaba por latencia y en la 010 por soberanía del dato, tiene aquí un tercer eje.

Dos matices honestos: los factores de emisión varían por hora según el mix de generación, y el PUE no incluye el coste de fabricación del hardware. Cualquier cifra de este tipo es una estimación con orden de magnitud útil, no una medición.

Ejemplo trabajado

La factura de CloudShop pasa de 12.400 a 17.360 USD en un mes y dirección pide «bajar costes un 30 %». El equipo empieza calculando el denominador antes de tocar nada.

mes anterior 12.400 USD / 620.000 pedidos = 0,0200 USD/pedido
mes actual 17.360 USD / 980.000 pedidos = 0,0177 USD/pedido

El coste unitario bajó un 11 %. El aumento es crecimiento, no derroche, y ese dato cambia la conversación: recortar un 30 % lineal significaría recortar capacidad mientras el volumen sube un 58 %.

Se desglosa por unidad para encontrar dónde sí hay margen:

cómputo (9 instancias)	6.480	0,0066	37 %
base de datos	4.900	0,0050	28 %
transferencia de salida	3.100	0,0032	18 %
almacenamiento	1.480	0,0015	9 %
observabilidad	1.400	0,0014	8 %

Se revisan las dos partidas mayores con los criterios de la clase.

Cómputo. Utilización medida sobre 30 días: media 0,58, percentil 95 diario 0,71. Por la ley de Little con $\lambda=800/s$ y $W=250$ ms hacen falta 200 concurrentes; a $\rho=0,70$ son 8 instancias y con tolerancia $n+1$, nueve. Están correctamente dimensionadas: no hay recorte sin degradar. Pero la utilización sostenida del 58 % está cerca del umbral de compromiso:

umbral de compromiso a 1 año: $u > 0,60$
medido: 0,58 en media, 0,71 en p95
→ comprometer 6 de las 9 instancias (base estable), dejar 3 bajo demanda
ahorro: $6 \times 720 \text{ USD} \times 0,40 = 1.728 \text{ USD/mes}$ (-27 % del cómputo)

Transferencia de salida. 3.100 USD equivalen a unos 34 TB. Se revisa la tasa de acierto del CDN:

aciertos de caché	61 %
salida desde origen	34 TB

Subir el acierto al 90 % ajustando Cache-Control en los activos con huella —lo de la clase 006— reduce la salida desde origen:

salida estimada $34 \text{ TB} \times (1 - 0,90)/(1 - 0,61) = 8,7 \text{ TB}$
ahorro $\approx 2.300 \text{ USD/mes}$ (-74 % de esa partida)

Resultado combinado:

ahorro total	$1.728 + 2.300 = 4.028 \text{ USD/mes}$	(-23 %)
coste unitario nuevo	$13.332 / 980.000 = 0,0136 \text{ USD/pedido}$	(-32 %)

No se llegó al 30 % de la factura, pero sí se superó el 30 % en coste unitario, que es la métrica que el negocio quería sin saber nombrarla. Y ningún ahorro salió de recortar capacidad: salieron de comprometer la base estable y de dejar de pagar transferencia evitable.

Riesgo residual declarado: el compromiso a un año sobre 6 instancias deja de ser rentable si el volumen cae más de un 40 %. Se acepta con revisión trimestral.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/011-costo-energia-capacidad-y-medicion-basica/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa estimacion-de-costo en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye estimacion-de-costo para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se pide recortar un porcentaje de la factura sin más contexto	No hay unidad de coste, así que no se distingue crecimiento de derroche	Divide la factura por una unidad de negocio antes de decidir; el unitario puede bajar mientras la factura sube.
La latencia se dispara al subir la utilización del 70 % al 90 %	El tiempo de espera crece como $1/(1-p)$, no linealmente	Fija el objetivo de utilización en 0,60-0,70; la capacidad libre absorbe la varianza.
Un compromiso de 3 años se convierte en gasto muerto	Se comprometió capacidad con utilización por debajo del umbral	Compromete solo la base estable, con $u > 0,60$; deja el pico bajo demanda.
Una carga en capacidad excedente rompe peticiones de usuario al ser retirada	Se usó capacidad interrumpible para un componente del camino crítico	Resérvala para trabajos reanudables; el preaviso puede ser de dos minutos.
El 35 % de la factura está en «sin asignar» y nadie lo optimiza	Falta etiquetado consistente, así que no hay atribución posible	Impón etiquetas obligatorias en el aprovisionamiento antes de intentar cualquier optimización.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. La factura sube un 40 % y el coste unitario baja un 11 %. ¿Hay un problema de costes? Justifica.
2. Con 500 peticiones por segundo y 400 ms de latencia media, ¿cuántas peticiones concurrentes hay en el sistema?
3. ¿Por qué desplegar al 95 % de utilización sale más caro que al 70 %, aunque use menos instancias?
4. ¿A partir de qué utilización sostenida compensa un compromiso de un año al 60 % del precio, y de dónde sale ese umbral?
5. Dos regiones tienen el mismo precio. ¿Qué otro criterio, con qué orden de magnitud, puede decidir entre ellas?

Referencias

- Little, J. D. C. (1961). A Proof for the Queuing Formula: $L = \lambda W$. Operations Research 9(3), 383-387. <<https://doi.org/10.1287/opre.9.3.383>>
- Storment, J. R. y Fuller, M. (2023). Cloud FinOps, 2.ª ed., caps. 6-9 — unidad de coste, etiquetado y modelos de compromiso.
- Gunther, N. (2007). Guerrilla Capacity Planning — leyes de escalabilidad y el efecto de la utilización sobre la latencia.
- Barroso, L., Hölzle, U. y Ranganathan, P. (2018). The Datacenter as a Computer, 3.ª ed., cap. 6 — PUE, eficiencia y coste total. <<https://doi.org/10.2200/S00874ED3V01Y201809CAC046>>
- FinOps Foundation (2024). FinOps Framework — capacidades de asignación, previsión y optimización de tasa. <<https://www.finops.org/framework/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 011 Costo, energía, capacidad y medición básica

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega estimacion-de-costo con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

012 — Proyecto: servicio local reproducible y observable

Parte: 00 — Fundamentos de computación, redes y Linux Nivel: inicial · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Integrar las once clases anteriores en un artefacto único: un servicio local que arranca con un comando, se observa, falla de forma controlada y se recupera. Es el punto de partida de CloudShop, el sistema que evolucionará durante las 276 clases restantes hasta operar en tres nubes. Lo que se construye aquí no es un ejercicio: es la línea base contra la que se medirá cada cambio posterior.

Resultados de aprendizaje

Al finalizar podrás:

1. Ensamblar un servicio con comprobaciones de vida y de disponibilidad distinguiendo correctamente sus semánticas.
2. Instrumentar las cuatro señales doradas y justificar por qué la media es insuficiente para la latencia.
3. Provocar un fallo de dependencia y demostrar que el servicio degrada en vez de caer.
4. Establecer una línea base de rendimiento reproducible que sirva de referencia para comparar cambios futuros.
5. Documentar el servicio de modo que otra persona lo levante, lo observe y lo recupere sin conocimiento tácito.

Conceptos centrales

Concepto	Comprensión verificable
liveness	Comprobación de si el proceso debe reiniciarse. Debe fallar solo ante un estado irrecuperable: si comprueba dependencias externas, una caída de la base de datos reinicia todas las réplicas y convierte una degradación en una caída total.
readiness	Comprobación de si el proceso puede recibir tráfico ahora. Sí debe mirar dependencias: al fallar, el balanceador retira la instancia sin matarla, y vuelve a incluirla cuando se recupera.
señales doradas	Latencia, tráfico, errores y saturación. Cuatro métricas que, según el SRE Book, bastan para detectar la mayoría de los problemas de un servicio orientado a peticiones.
degradación elegante	Responder con funcionalidad reducida en vez de fallar por completo cuando una dependencia no esencial no está disponible.
línea base	Medición reproducible del comportamiento actual. Sin ella, cualquier afirmación posterior sobre mejora o regresión es una opinión.

Modelo mental

Antes de alquilar infraestructura remota, aprende a observar una computadora local como un conjunto de procesos, redes, archivos, identidades y recursos medibles.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    CL["cliente"] --> LB["balanceador"]
    LB -->|"/health/ready"| API["servicio CloudShop"]
    API --> DB["base de datos<br>esencial"]
    API -->|"/opcional"| CA["caché"]
    API -->|"/opcional"| REC["recomendaciones"]
    API --> M["/metrics<br>latencia, tráfico,<br>errores, saturación"]
    DB -->|"/cae → ready falla<br>→ se retira del balanceador"| LB
    REC -->|"/cae → responde sin<br>recomendaciones: 200"| CL
```

Desarrollo

1. Liveness y readiness responden a preguntas opuestas

Confundirlas es el error que convierte una degradación en una caída total, y ocurre porque ambas «comprueban si el servicio está bien».

	Liveness	Readiness
Pregunta	¿Hay que reiniciar?	¿Puede recibir tráfico?
Acción al fallar	Matar y reiniciar	Retirar del balanceador
¿Mira dependencias?	Nunca	Sí
Coste de un falso positivo	Reinicio innecesario	Capacidad reducida

```
@app.get("/health/live")
def live():
    return {"status": "ok"} # solo: ¿responde el bucle de eventos?

@app.get("/health/ready")
def ready():
    fallos = []
    if not db.ping(timeout=1.0):
        fallos.append("db") # esencial
    if fallos:
        raise HTTPException(503, {"no_listo": fallos})
    return {"status": "ok", "degradado": estado_opcionales()}
```

El escenario que justifica la regla: la base de datos se cae 40 segundos. Con liveness mirando la base de datos, las 9 réplicas fallan la comprobación a la vez y el orquestador las mata todas; cuando la base vuelve, no hay servicio porque todas están arrancando en frío. Con la separación correcta, las 9 salen del balanceador, siguen vivas, y vuelven a entrar en cuanto la dependencia responde.

Hay una tercera comprobación que conviene desde el principio: startup, para procesos con arranque lento. Sin ella hay que relajar el plazo de liveness para todo el ciclo de vida, lo que retrasa la detección de bloqueos reales.

2. Las cuatro señales doradas, y por qué la media miente

El SRE Book reduce la instrumentación mínima a cuatro señales:

Señal	Qué mide	Cómo se instrumenta
Latencia	Cuánto tarda, separando éxitos de errores	Histograma, no media
Tráfico	Demanda: peticiones/s	Contador
Errores	Tasa de fallo, explícito e implícito	Contador por código
Saturación	Cuán lleno está el recurso más restrictivo	Medidor

La separación de latencia entre éxitos y errores no es un detalle: los errores suelen ser rápidos, así que mezclarlos baja artificialmente la media justo cuando el servicio está peor.

Y la media es insuficiente por sí misma. Con 1.000 peticiones:

```
950 peticiones a 50 ms
50 peticiones a 2.000 ms
```

$media = (950 \times 50 + 50 \times 2000) / 1000 = 147,5 \text{ ms}$ ← parece aceptable
 $p95 = 50 \text{ ms}$
 $p99 = 2.000 \text{ ms}$ ← 1 de cada 100 usuarios espera 2 s

La media de 147 ms no la experimenta nadie: unos ven 50 ms y otros 2 segundos. Por eso los SLO se fijan sobre percentiles, y por eso las métricas se exportan como histograma —que permite calcular percentiles agregando entre instancias— y no como media previamente calculada, que es matemáticamente imposible de agregar.

La saturación es la más olvidada y la más predictiva: es la única que avisa antes de que la latencia y los errores se degraden. Con lo visto en la clase 011, la saturación por encima de 0,7 anticipa el problema que las otras tres señales aún no muestran.

3. Degradar en vez de caer

No todas las dependencias son iguales, y tratarlas igual convierte cualquier fallo en una caída completa. Clasificarlas es una decisión de producto, no técnica:

Dependencia	Clase	Si no está
Base de datos de pedidos	Esencial	503: no se puede servir
Caché	Opcional	Responder más lento desde origen
Recomendaciones	Opcional	Responder sin esa sección
Telemetría	Opcional	Registrar localmente y continuar

```

async def pagina_producto(pid: str):
    producto = await db.producto(pid)          # esencial: si falla, propaga
    recomendados, degradado = [], []
    try:
        recomendados = await asyncio.wait_for(rec.para(pid), timeout=0.15)
    except (asyncio.TimeoutError, ConnectionError):
        degradado.append("recomendaciones")    # opcional: se anota y sigue
    return {"producto": producto, "recomendados": recomendados, "degradado": degradado}
  
```

Tres detalles que hacen que esto funcione de verdad:

1. El plazo es obligatorio. Sin timeout, una dependencia lenta bloquea al llamante y propaga la saturación hacia arriba. Una dependencia opcional sin plazo es una dependencia esencial disfrazada.
2. La degradación se declara en la respuesta. El campo degradado permite que el cliente y el panel sepan que el 200 no es completo.
3. El plazo debe ser menor que el del llamante. Si el usuario espera 300 ms, una dependencia opcional con plazo de 500 ms nunca llega a tiempo y solo añade latencia.

4. Una línea base que se pueda repetir

Sin medición previa, «esto mejoró el rendimiento» es una creencia. La línea base debe fijar carga, duración, entorno y percentiles, y guardarse junto al código:

```
$ hey -z 60s -c 50 -q 20 http://localhost:8080/api/productos/A-1042
```

```

Summary:
  Total:          60.0031 secs
  Requests/sec:  987.4210
  
```

```

Latency distribution:
  50% in 0.0384 secs
  95% in 0.0912 secs
  99% in 0.1847 secs
  
```

```

Status code distribution:
  [200] 59248 responses
  
```

Se registra como contrato comparable:

```

{"escenario": "productos-lectura", "concurrency": 50, "duracion_s": 60,
 "rps": 987.4, "p50_ms": 38.4, "p95_ms": 91.2, "p99_ms": 184.7, "errores": 0}
  
```

Dos verificaciones con lo aprendido en la clase 011. Ley de Little:

$L = \lambda \cdot W = 987 \times 0,0384 = 37,9$ concurrentes

Coherente con los 50 solicitados: el sistema no está saturado, hay holgura. Si L hubiera dado 50 exactos, la concurrencia sería el cuello de botella y la medida estaría limitada por el generador de carga, no por el servicio.

Relación p99/p50 = 184,7/38,4 = 4,8×. Es la métrica que hay que vigilar entre versiones: si sube, la cola se está alargando aunque la media no se mueva.

5. Reproducible significa sin conocimiento tácito

El criterio de aceptación del proyecto no es que funcione en tu máquina: es que otra persona lo levante sin preguntarte nada. Eso exige que el repositorio contenga:

projects/cloudshop/	
■■■■ README.md	qué es, cómo se levanta, cómo se comprueba, cómo se para
■■■■ Dockerfile	entorno de ejecución fijado
■■■■ compose.yaml	el servicio y sus dependencias
■■■■ app.py	el servicio
■■■■ smoke.sh	comprobación de extremo a extremo con código de salida
■■■■ baseline.json	la línea base medida, con su escenario

Y que el recorrido completo quepa en cuatro comandos con salida verificable:

```
$ docker compose up -d
$ ./smoke.sh # sale 0 si todo responde; distinto de 0 si no
$ curl -s localhost:8080/metrics | grep -c '^http_request_duration'
$ docker compose down -v
```

El smoke.sh con código de salida no es cosmético: es lo que permite que este mismo recorrido se ejecute en CI en la parte 08 sin cambiar nada. Un script que imprime «OK» pero siempre sale 0 no verifica nada.

Lo que no debe estar: credenciales reales, rutas absolutas de tu máquina, dependencias instaladas globalmente y no declaradas, y pasos que solo tú conoces. Cada uno de esos es conocimiento tácito, y el conocimiento tácito es lo que hace que un servicio sea irrecuperable cuando la persona que lo sabe está de vacaciones.

Ejemplo trabajado

Se ensambla CloudShop v0 y se comprueba con un fallo provocado que degrada en vez de caer.

Línea base con todas las dependencias sanas:

```
$ docker compose up -d && ./smoke.sh
ready: ok . degradado: []
$ hey -z 30s -c 50 http://localhost:8080/api/productos/A-1042 | grep -E '95%|99%|Requests/sec'
Requests/sec: 987.4210
95% in 0.0912 secs
99% in 0.1847 secs
```

Prueba negativa 1 — cae una dependencia opcional. Se detiene el servicio de recomendaciones:

```
$ docker compose stop recomendaciones
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8080/api/productos/A-1042
200
$ curl -s localhost:8080/api/productos/A-1042 | jq -c '.degradado'
["recomendaciones"]
$ curl -s localhost:8080/health/ready | jq -c '.'
{"status":"ok","degradado":["recomendaciones"]}
```

Sigue respondiendo 200 y declara la degradación. El impacto en latencia se mide, no se supone:

p95 con recomendaciones	91,2 ms	
p95 sin recomendaciones	73,8 ms	← más rápido: ya no espera esa llamada

Prueba negativa 2 — la dependencia opcional no cae, se vuelve lenta. Este es el caso que de verdad rompe sistemas, porque nada falla:

```
$ docker compose exec recomendaciones tc qdisc add dev eth0 root netem delay 800ms
$ hey -z 30s -c 50 http://localhost:8080/api/productos/A-1042 | grep '95%'
95% in 0.0947 secs
```

El p95 apenas se mueve: 94,7 ms frente a 91,2. El plazo de 150 ms corta la llamada lenta y el resultado es equivalente a que estuviera caída. Sin ese plazo, el p95 habría subido a más de 800 ms y la saturación se habría propagado hacia arriba: cada petición mantendría un trabajador ocupado 800 ms, y por la ley de Little la concurrencia necesaria se

multiplicaría por 8.

Prueba negativa 3 — cae la dependencia esencial.

```
$ docker compose stop db
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8080/health/ready
503
$ curl -s -o /dev/null -w '%{http_code}\n' localhost:8080/health/live
200                                # sigue VIVO: no debe reiniciarse
$ docker compose start db && sleep 3
$ curl -s localhost:8080/health/ready | jq -r .status
ok                                # vuelve solo, sin reiniciar
```

Ese 200 en liveness durante la caída es el resultado que valida el diseño: el orquestador retira la instancia del balanceador pero no la mata, así que cuando la base vuelve hay capacidad caliente inmediata. Si liveness hubiera consultado la base, las nueve réplicas habrían muerto a la vez y la recuperación habría añadido el arranque en frío al tiempo de indisponibilidad.

Evidencia registrada en evidence/: baseline.json, las tres pruebas negativas con su salida, y una limitación explícita — la línea base se midió en una sola máquina sin latencia de red entre componentes; los números no son extrapolables a un despliegue distribuido, solo comparables contra sí mismos.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-00-foundations-computing-networking-linux/012-proyecto-servicio-local-reproducible-y-observable/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-local en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye servicio-local para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Cae la base de datos y el orquestador reinicia todas las réplicas a la vez	La comprobación de liveness consulta dependencias externas	Liveness solo mira el propio proceso; las dependencias van en readiness.
El panel muestra latencia media aceptable mientras los usuarios se quejan	La media oculta la cola y mezcla errores rápidos con éxitos lentos	Exporta histogramas, separa éxitos de errores y fija los SLO sobre p95 y p99.
Una dependencia opcional se vuelve lenta y satura todo el servicio	La llamada no tenía plazo, así que la dependencia opcional era esencial de hecho	Toda llamada opcional necesita timeout menor que el presupuesto del llamante.
Nadie puede afirmar si un cambio mejoró o empeoró el rendimiento	No existe una línea base reproducible con escenario y percentiles	Registra carga, duración, entorno y p50/p95/p99 en un fichero versionado.
Solo una persona sabe levantar el servicio	El recorrido depende de conocimiento tácito no escrito	Exige que otra persona lo levante siguiendo solo el README, y corrige lo que le falte.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué liveness no debe consultar la base de datos, y qué ocurre exactamente si lo hace durante una caída de 40 s?
2. Con 950 peticiones a 50 ms y 50 a 2.000 ms, ¿cuál es la media y cuál el p99? ¿Cuál describe mejor la experiencia?
3. Una dependencia opcional se vuelve lenta pero no falla. ¿Qué la convierte de hecho en esencial y cómo se evita?
4. ¿Por qué no se pueden agregar medias de latencia entre instancias, y qué formato de métrica sí lo permite?
5. Tu línea base da 987 rps y p50 de 38 ms con concurrencia 50. Por la ley de Little, ¿estaba saturado el sistema?

Referencias

- Beyer, B. et al., eds. (2016). Site Reliability Engineering, cap. 6 «Monitoring Distributed Systems» — las cuatro señales doradas. <<https://sre.google/sre-book/monitoring-distributed-systems/>>
- Kubernetes (2024). Configure Liveness, Readiness and Startup Probes — semántica y efectos de cada sonda. <<https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/>>
- Nygard, M. (2018). Release It!, 2.^a ed., caps. 4-5 — patrones de estabilidad: timeouts, bulkheads y circuit breaker.
- Prometheus (2024). Histograms and summaries — por qué los histogramas son agregables y los cuantiles precalculados no. <<https://prometheus.io/docs/practices/histograms/>>
- Dean, J. y Barroso, L. (2013). The Tail at Scale. Communications of the ACM 56(2), 74-80. <<https://doi.org/10.1145/2408776.2408794>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 012 Proyecto: servicio local reproducible y observable

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.

2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega servicio-local con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 01 — Principios, estrategia y adopción cloud

← Parte 00 · Índice completo · Parte 02 →

Nivel: inicial-intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Decidir cuándo, por qué y cómo adoptar nube con criterios técnicos y económicos.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
013	Definición NIST y características esenciales de cloud	foundation	4
014	Regiones, zonas de disponibilidad, puntos de presencia y edge	architecture	4
015	IaaS, PaaS, SaaS, CaaS y FaaS	decision	4
016	Elasticidad, escalabilidad, disponibilidad y resiliencia	reliability	4
017	Tenancy, cuentas, suscripciones, proyectos y jerarquías	governance	4
018	Identidad, roles, políticas y federación	iam	4
019	Modelo de responsabilidad compartida por servicio	security	4
020	TCO, costos variables, unit economics y FinOps	finops	4
021	Well-Architected y atributos de calidad	architecture	4
022	Cloud Adoption Framework y modelo operativo	governance	4
023	Descubrimiento y clasificación de workloads	migration	4
024	Proyecto: decisión de migración sustentada con ADR	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Cloud Strategy — Gregor Hohpe.
- Cloud FinOps — Storment y Fuller.
- Accelerate — Forsgren, Humble y Kim.

013 — Definición NIST y características esenciales de cloud

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: foundation · Estado: EXECUTABLECORE

Propósito

Convertir la definición de NIST en una herramienta de decisión y no en un dato de examen. Cada una de las cinco características esenciales genera una consecuencia arquitectónica concreta, y la quinta —el servicio medido— es la que convierte cada decisión técnica del resto del programa en una decisión económica.

Resultados de aprendizaje

Al finalizar podrás:

1. Auditar un servicio contra las cinco características y determinar si es cloud o hosting con nombre nuevo.
2. Derivar de cada característica al menos una consecuencia de diseño verificable.
3. Situar un despliegue en el modelo correcto —público, privado, híbrido o comunitario— por su frontera de control, no por su ubicación.
4. Explicar por qué la agrupación de recursos implica riesgo de vecino ruidoso y qué controles lo acotan.
5. Justificar con la definición por qué «nube privada» y «centro de datos virtualizado» no son sinónimos.

Conceptos centrales

Concepto	Comprensión verificable
autoservicio bajo demanda	Capacidad de aprovisionar sin interacción humana con el proveedor. Su prueba es operativa: si hace falta un ticket o una llamada, no se cumple, y el tiempo de aprovisionamiento deja de ser un parámetro de arquitectura.
agrupación de recursos	Modelo en el que el proveedor reasigna capacidad física entre clientes de forma opaca. Es lo que abarata el servicio y lo que introduce el riesgo de interferencia entre inquilinos.
elasticidad rápida	Aprovisionar y liberar en minutos siguiendo la demanda, en ambas direcciones. Sin la dirección de bajada no hay elasticidad: hay aprovisionamiento rápido, que no genera ahorro.
servicio medido	Medición automática y transparente del consumo, con capacidad de auditarla. Es la característica que convierte la arquitectura en una función de coste.
modelo de despliegue	Quién controla la infraestructura y para quién opera: público, privado, comunitario o híbrido. Se define por la frontera de control y de inquilinos, no por dónde están los servidores.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
S["Servicio candidato"] --> C1{"¿Aprovisionas<br/>sin pedir permiso?"}
C1 -->|"no"| H["Hosting"]
C1 -->|"sí"| C2{"¿Acceso por red<br/>estándar?"}
C2 -->|"no"| H
C2 -->|"sí"| C3{"¿Capacidad agrupada<br/>y reasignada?"}
C3 -->|"no"| H
C3 -->|"sí"| C4{"¿Escala y LIBERA<br/>en minutos?"}
C4 -->|"no"| H
C4 -->|"sí"| C5{"¿Consumo medido<br/>y auditable?"}
C5 -->|"no"| H
C5 -->|"sí"| N["Cumple NIST SP 800-145"]
```

Desarrollo

1. Cinco características, cinco consecuencias de diseño

La definición del NIST SP 800-145 (Mell y Grance, 2011) no es descriptiva: es una lista de comprobación con consecuencias. Un servicio debe cumplir las cinco; fallar una lo saca de la categoría.

Característica	Prueba operativa	Consecuencia de diseño
Autoservicio bajo demanda	¿Aprovisionas sin ticket?	El tiempo de aprovisionamiento pasa de semanas a minutos, así que la capacidad deja de ser una decisión anual
Acceso amplio por red	¿Protocolos estándar, cualquier cliente?	La red es la única frontera: no hay «estar dentro»
Agrupación de recursos	¿El proveedor reasigna sin avisarte?	Tu rendimiento depende de vecinos que no ves
Elasticidad rápida	¿Escala y libera en minutos?	La capacidad ociosa se vuelve coste evitable
Servicio medido	¿Consumo auditable por unidad?	Cada bucle mal escrito aparece en la factura

La columna derecha es la que importa. La cuarta fila tiene un matiz que decide proyectos: elasticidad exige las dos direcciones. Un sistema que escala en 2 minutos pero tarda 3 días en liberar capacidad —porque nadie se atreve a apagar— no produce ahorro. Es aprovisionamiento rápido, no elasticidad, y su economía es la de un centro de datos.

La quinta fila es la que reorganiza la ingeniería: en un centro de datos propio, un algoritmo ineficiente consume capacidad ya pagada; en cloud, consume dinero medible atribuible a un equipo. Ese cambio de incentivo es el origen de FinOps, y por eso la clase 011 estableció la unidad de coste antes de entrar en esta parte.

2. El modelo de despliegue lo define el control, no la ubicación

La confusión más extendida de esta parte: creer que «privado» significa «en mis instalaciones». NIST define los cuatro modelos por para quién opera la infraestructura, no por dónde está:

Modelo	Inquilinos	Puede estar
Público	Cualquiera	Solo en el proveedor
Privado	Una sola organización	En tus instalaciones o en un tercero
Comunitario	Varias organizaciones con requisitos compartidos	En cualquiera de ellas o en un tercero
Híbrido	Combinación con portabilidad de datos entre ellas	Ambos

De ahí dos consecuencias que se discuten mal en los comités:

1. Una nube privada alojada en un tercero sigue siendo privada. El criterio es la exclusividad de inquilino, no la propiedad del edificio.
2. Un centro de datos virtualizado no es una nube privada. Le suele faltar autoservicio (hay que pedir la máquina), elasticidad (no se libera) y medición (no se factura por consumo). Cumple una de cinco.

Y el híbrido tiene un requisito que casi nadie aplica: NIST exige portabilidad de datos y aplicaciones entre las partes. Tener servidores propios y además una cuenta en un proveedor no es híbrido: es tener dos cosas separadas. Sin la

portabilidad, no hay modelo híbrido, hay coexistencia. Esa distinción reaparecerá con consecuencias de coste en la parte 13.

3. Agrupación de recursos: el precio tiene una contrapartida

La agrupación es lo que hace barata la nube: el proveedor amortiza el mismo hardware entre muchos clientes con picos que no coinciden. La contrapartida es el vecino ruidoso: tu rendimiento depende de cargas que no ves ni controlas.

Se manifiesta en tres recursos, con síntomas distintos:

Recurso	Síntoma	Mitigación
CPU	Latencia irregular sin cambio de carga propia	Instancias con núcleos dedicados
E/S de disco	IOPS por debajo de lo esperado a ratos	Volúmenes con IOPS aprovisionadas
Red	Throughput variable entre instancias	Instancias con ancho de banda garantizado

La variabilidad, no la media, es lo que hay que medir. Un proveedor puede cumplir su media anunciada y aun así producir un p99 inaceptable, porque la interferencia es esporádica. Es el mismo argumento de la clase 012 sobre por qué la media miente.

El caso especial es el crédito de CPU de las instancias ráfaga: acumulan crédito mientras están ociosas y lo gastan al trabajar. Cuando se agota, el rendimiento cae a una fracción del nominal —por ejemplo, al 20 %— sin que ninguna métrica de la aplicación explique el desplome. Diagnosticarlo exige mirar el saldo de créditos, que es una métrica del proveedor y no del sistema operativo. Es la primera vez en el programa en que una métrica externa a la máquina es imprescindible para explicar su comportamiento.

4. Auditar un servicio con la definición

La utilidad práctica de NIST es cortar discusiones de marketing. Tres casos frecuentes, auditados:

«Nuestro hosting gestionado es cloud privado.»

Característica	¿Cumple?
Autoservicio	No: hay que abrir ticket
Acceso por red	Sí
Agrupación	Parcial
Elasticidad	No: alta en horas, baja en días
Medición	No: cuota fija mensual

Una de cinco. Es hosting. La consecuencia no es semántica: si compras esperando elasticidad, el ahorro previsto no llegará y el caso de negocio era falso desde el principio.

«Somos híbridos porque tenemos servidores y una cuenta en un proveedor.» Falta la portabilidad de datos y aplicaciones entre ambos. Sin ella no hay conmutación posible, así que tampoco hay la continuidad que se estaba justificando.

«Un servicio SaaS con precio por asiento no es cloud porque no se mide por consumo.» Falso: el asiento es la unidad de medida, y es auditable. NIST exige medición apropiada al tipo de servicio, no facturación por segundo.

El patrón: la definición sirve para verificar que la propiedad por la que estás pagando existe de verdad, no para clasificar por gusto.

5. Qué no dice NIST, y por qué importa

Tan útil como lo que define es lo que deja fuera, porque marca dónde el criterio debe ponerlo el arquitecto:

- No dice nada de seguridad. Un servicio puede cumplir las cinco características y ser inseguro. La seguridad se reparte con el modelo de responsabilidad compartida de la clase 019.

- No dice nada de coste. Cumplir la definición no implica ser barato; implica ser medible. Que salga a cuenta depende de la utilización, como se calculó en la clase 011.
- No dice nada de fiabilidad. No hay ninguna característica sobre disponibilidad. Esa la fija el SLA, que es un documento distinto con exclusiones propias.
- No dice nada de portabilidad, salvo en el modelo híbrido. Cumplir NIST es compatible con un bloqueo total de proveedor.

La última es la más relevante para un programa multi-cloud: la definición no protege del bloqueo de proveedor. Un servicio perfectamente conforme puede usar una API propietaria sin equivalente en otro sitio. La portabilidad es una decisión de arquitectura con coste propio —se estudiará en las partes 12 y 13—, no una propiedad que venga incluida.

Y el documento tiene 14 años. Serverless, contenedores gestionados y funciones no existían como categorías cuando se escribió; encajan sin problema en IaaS/PaaS, pero la frontera entre modelos es hoy más difusa de lo que sugiere la clasificación en tres niveles.

Ejemplo trabajado

El comité de CloudShop evalúa dos ofertas para sacar su plataforma del centro de datos actual, y las dos se presentan como «nube privada». Se auditan contra las cinco características antes de mirar el precio.

Oferta A — proveedor local, «cloud privado gestionado».

autoservicio	ticket con SLA de 4 h laborables	X
acceso por red	VPN y API REST propia	✓
agrupación	hardware dedicado por cliente	X (no hay agrupación)
elasticidad	alta en 4 h, baja con preaviso de 30 d	X
medición	cuota fija mensual + extras	X

Una de cinco. Es alojamiento dedicado. No es que sea mala opción: es que el caso de negocio presentado se apoyaba en ahorro por elasticidad, y esta oferta no puede producirlo por construcción.

Oferta B — hiperescalar, cuenta dedicada con conectividad privada.

autoservicio	API y consola, sin intervención	✓
acceso por red	protocolos estándar	✓
agrupación	multi-inquilino con aislamiento lógico	✓
elasticidad	minutos en ambas direcciones	✓
medición	por segundo, con etiquetas y export	✓

Cinco de cinco. Es nube pública con conectividad privada, no nube privada: comparte hardware con otros inquilinos. La distinción importa porque el requisito regulatorio del cliente exigía «aislamiento de inquilino» y aquí hay agrupación.

Se cuantifica el impacto de la característica que falta en A, con los datos de utilización reales:

capacidad de pico	100 unidades, 3 h/día
capacidad de base	20 unidades, 21 h/día
utilización media	$= (100 \times 3 + 20 \times 21) / (100 \times 24) = 30 \%$

Oferta A (sin elasticidad): se paga el pico las 24 h
 $100 \times 24 \times 30 \times 0,85 \text{ USD} = 61.200 \text{ USD/mes}$

Oferta B (con elasticidad): se paga el consumo
 $(100 \times 3 + 20 \times 21) \times 30 \times 1,00 \text{ USD} = 21.600 \text{ USD/mes}$

39.600 USD/mes de diferencia, y no viene del precio unitario —que es peor en B— sino de la característica 4. A cobra un 15 % menos por unidad y cuesta casi el triple, porque cobra por capacidad reservada.

Decisión y límite explícito: se elige B, y el requisito de aislamiento de inquilino se resuelve con instancias de tenencia dedicada solo para las cargas que lo exigen —un 12 % del total—, no para toda la plataforma. Riesgo residual declarado: la tenencia dedicada elimina la agrupación en esa fracción, así que ahí no habrá ahorro por elasticidad y su coste unitario será el de A.

La lección del ejercicio: las cinco características no son un dato de examen, son las cinco preguntas que revelan si el caso de negocio se sostiene.

Laboratorio guiado

Ejecuta desde la raíz:

python classes/part-01-cloud-principles-strategy-adoption/013-definicion-nist-y-caracteristicas-esenciales-de-cloud/lab.py

El laboratorio selecciona el motor de práctica foundation y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-nist en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un mapa con fronteras, entradas, salidas y supuestos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-nist para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se firma un contrato esperando ahorro por elasticidad y la factura no baja	El servicio escala hacia arriba pero no libera capacidad; falla la característica 4	Exige la prueba en ambas direcciones antes de firmar: aprovisionar y liberar, con tiempos medidos.
Se llama nube privada a un centro de datos virtualizado	Se confundió virtualización con las cinco características	Audita las cinco; virtualizar cumple una y el caso de negocio suele apoyarse en las otras cuatro.
Se declara arquitectura híbrida sin poder conmutar entre las partes	NIST exige portabilidad de datos y aplicaciones, y solo había coexistencia	Si no hay portabilidad demostrada, no cuentes con la continuidad que justificaba el modelo.
El rendimiento cae sin que ninguna métrica del sistema operativo lo explique	Agotamiento de créditos de CPU en una instancia ráfaga: es una métrica del proveedor	Vigila el saldo de créditos junto a las métricas del sistema; la interferencia entre inquilinos no se ve desde dentro.
Se asume que cumplir NIST implica portabilidad entre proveedores	La definición no dice nada de portabilidad salvo en el modelo híbrido	Trata la portabilidad como decisión de arquitectura con coste propio, no como propiedad incluida.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Un servicio escala en 2 minutos y libera capacidad en 3 días. ¿Cumple la característica de elasticidad? ¿Qué consecuencia económica tiene?
2. ¿Puede una nube privada estar alojada en un tercero? ¿Qué criterio decide el modelo de despliegue?
3. ¿Qué le falta a un centro de datos virtualizado para cumplir la definición, y cuántas de las cinco cumple?
4. Un SaaS cobra por asiento y no por segundo. ¿Incumple la característica de servicio medido? Justifica.
5. Nombra dos propiedades que NIST no garantiza y que suelen darse por supuestas al elegir proveedor.

Referencias

- Mell, P. y Grance, T. (2011). The NIST Definition of Cloud Computing, SP 800-145 — las cinco características, tres modelos de servicio y cuatro de despliegue. <<https://doi.org/10.6028/NIST.SP.800-145>>
- Liu, F. et al. (2011). NIST Cloud Computing Reference Architecture, SP 500-292 — actores, roles y sus relaciones. <<https://doi.org/10.6028/NIST.SP.500-292>>
- Hohpe, G. (2020). Cloud Strategy: A Decision-Based Approach — por qué la definición sirve para decidir y no para clasificar.
- Armbrust, M. et al. (2010). A View of Cloud Computing. Communications of the ACM 53(4), 50-58 — obstáculos y oportunidades, incluida la variabilidad de rendimiento. <<https://doi.org/10.1145/1721654.1721672>>
- Barroso, L., Hölzle, U. y Ranganathan, P. (2018). The Datacenter as a Computer, 3.ª ed., cap. 7 — economía de la agrupación de recursos. <<https://doi.org/10.2200/S00874ED3V01Y201809CAC046>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 013 Definición NIST y características esenciales de cloud

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-nist con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.

- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

014 — Regiones, zonas de disponibilidad, puntos de presencia y edge

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Entender la geografía de una nube como una jerarquía de dominios de fallo con latencias y precios distintos entre niveles. Colocar un componente en el nivel equivocado produce o bien una factura de transferencia inesperada, o bien un sistema que cae entero cuando falla un edificio. Esta clase fija el vocabulario y la aritmética que usarán las partes 13, 16 y 17.

Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir región, zona de disponibilidad y punto de presencia por su dominio de fallo, su latencia y su precio de transferencia.
2. Calcular la disponibilidad compuesta de un despliegue en una, dos y tres zonas.
3. Anticipar el coste de transferencia de un diseño antes de desplegarlo, sabiendo qué cruces se cobran.
4. Justificar cuándo el edge aporta y cuándo solo añade complejidad.
5. Reconocer que las zonas son etiquetas por cuenta y qué implica al comparar despliegues entre cuentas.

Conceptos centrales

Concepto	Comprensión verificable
región	Área geográfica con varias zonas independientes. Es la unidad de soberanía del dato y de aislamiento de servicios: un fallo regional es el mayor dominio de fallo que un proveedor reconoce.
zona de disponibilidad	Uno o más centros de datos con energía, refrigeración y red independientes dentro de una región, separados kilómetros pero conectados con latencia de un dígito de milisegundos.
punto de presencia	Ubicación de borde que termina conexiones y sirve contenido cacheado cerca del usuario. No ejecuta tu aplicación completa: reduce el RTT del primer tramo.
dominio de fallo	Conjunto de componentes que caen juntos. El diseño consiste en elegir qué comparte dominio y qué no, porque redundancia dentro del mismo dominio no es redundancia.
transferencia de salida	Tráfico que sale hacia otra zona, región o internet. Es asimétrico: la entrada suele ser gratis y la salida se cobra, lo que convierte la topología en una decisión de coste.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```

flowchart TB
    subgraph reg["Región · dominio de fallo mayor"]
        subgraph az1["Zona A"]
            A1["cómputo"] --- D1["réplica primaria"]
        end
        subgraph az2["Zona B"]
            A2["cómputo"] --- D2["réplica secundaria"]
        end
        subgraph az3["Zona C"]
            A3["cómputo"] --- D3["testigo"]
        end
    end
    POP["Puntos de presencia<br/>decenas por continente"] -->|"RTT 5-20 ms"| reg
    U["usuarios"] --> POP
    az1 <-->|"< 2 ms · se cobra"| az2
    reg <-->|"decenas de ms · se cobra más"| reg2["Otra región"]

```

Desarrollo

1. Tres niveles, tres órdenes de magnitud

La jerarquía no es organizativa: cada salto cambia latencia, dominio de fallo y precio en un orden de magnitud.

Nivel	Latencia entre pares	Cae junto por	Transferencia
Dentro de una zona	< 0,5 ms	Rack, sala	Normalmente gratis
Entre zonas de una región	0,5-2 ms	Desastre regional	Se cobra, precio bajo
Entre regiones	10-150 ms	Nada común	Se cobra, precio alto
Punto de presencia a usuario	5-20 ms	Un PoP concreto	Salida a internet, el más caro

La fila de zonas es la que sostiene la alta disponibilidad: 2 ms es lo bastante bajo para replicar de forma síncrona una base de datos, y lo bastante independiente para que un incendio, un corte eléctrico o una inundación no alcancen a las dos. Ese es exactamente el punto de equilibrio que buscan los proveedores al situarlas: suficientemente lejos para no compartir riesgo físico, suficientemente cerca para no romper la replicación síncrona.

Entre regiones, los 10-150 ms hacen inviable la replicación síncrona: cada escritura pagaría el RTT completo. Por eso la replicación entre regiones es asíncrona, y por eso siempre tiene un RPO mayor que cero. No es una limitación del producto: es la velocidad de la luz de la clase 001 aplicada al diseño de datos.

2. Disponibilidad compuesta: la aritmética de las zonas

Si un componente tiene disponibilidad a y se despliegan n copias en dominios de fallo independientes, la disponibilidad del conjunto es:

$$A(n) = 1 - (1 - a)^n$$

Con una disponibilidad por zona de 99,5 %:

1 zona: $1 - 0,005^1 = 99,5\%$ → 3,6 h de caída al mes
 2 zonas: $1 - 0,005^2 = 99,9975\%$ → 1,1 min
 3 zonas: $1 - 0,005^3 = 99,999988\%$ → 0,3 s

El salto de una a dos zonas divide la indisponibilidad por 200. El de dos a tres, por otras 200 — pero en términos absolutos pasa de 1,1 minutos a 0,3 segundos, una mejora que casi ningún negocio percibe. La tercera zona rara vez se justifica por disponibilidad; se justifica por quórum, que es otra cosa: los sistemas de consenso necesitan mayoría, y con dos zonas no hay mayoría posible si cae una.

Y la fórmula tiene una condición que se incumple a diario: exige independencia. Tres réplicas de cómputo en tres zonas, todas apuntando a una única base de datos en la zona A, tienen la disponibilidad de la zona A. La redundancia se calcula sobre el componente menos redundante, no sobre el más visible.

$$\text{cómputo 3 zonas (99,999988 \%)} \times \text{base de datos 1 zona (99,5 \%)} = 99,5 \%$$

El cómputo redundante no aporta nada mientras la base sea única. Es el error de diseño más común de esta parte.

3. La transferencia es asimétrica y decide topologías

El precio de mover un byte depende de qué frontera cruza, y la asimetría entrada/salida moldea las arquitecturas más de lo que se suele reconocer:

Cruce	Precio orientativo por GB
Entrada desde internet	0,00 USD
Dentro de la misma zona	0,00 USD
Entre zonas de una región	0,01 USD (cada sentido)
Entre regiones	0,02-0,09 USD
Salida a internet	0,05-0,12 USD
Salida vía CDN	0,02-0,085 USD

Dos consecuencias prácticas:

El tráfico entre zonas se cobra en ambos sentidos. Un diseño que reparte peticiones aleatoriamente entre tres zonas hace que dos tercios de las llamadas internas crucen zona. Con 40 TB mensuales de tráfico interno:

reparto aleatorio: $2/3 \times 40 \text{ TB} \times 2 \text{ sentidos} \times 0,01 \text{ USD/GB} = 546 \text{ USD/mes}$
afinidad de zona: $\sim 5 \% \text{ cruza zona} = 41 \text{ USD/mes}$

La afinidad de zona —que una petición se resuelva dentro de la zona donde entró— ahorra un orden de magnitud, a cambio de complicar el balanceo y de perder algo de uniformidad de carga. Es una decisión, no una optimización obvia.

La salida a internet es el precio más alto y el menos vigilado. Un servicio que devuelve respuestas de 200 KB en lugar de 20 KB multiplica por diez la partida más cara de la factura. Comprimir y paginar no son solo mejoras de latencia.

4. Edge: para qué sirve y para qué no

Un punto de presencia reduce el RTT del primer tramo, y eso solo ayuda si el RTT es el componente dominante. Con lo visto en la clase 006, la petición en frío cuesta unos 4 RTT.

Sirve para: contenido estático cacheable, terminación de TLS cerca del usuario, absorción de picos y de ataques volumétricos, y lógica de borde muy corta —reescrituras, redirecciones, comprobación de un token—.

No sirve para: reducir el tiempo de una consulta a la base de datos, ni para nada que necesite estado consistente, ni cuando la respuesta es personalizada y no cacheable. En esos casos el borde añade un salto y empeora la latencia.

sin edge: usuario $\blacksquare\blacksquare 90 \text{ ms} \blacksquare\blacksquare$ origen $= 4 \times 90 = 360 \text{ ms}$
con edge: usuario $\blacksquare\blacksquare 10 \text{ ms} \blacksquare\blacksquare$ PoP $\blacksquare\blacksquare 80 \text{ ms} \blacksquare\blacksquare$ origen
 contenido cacheado en el PoP: $4 \times 10 = 40 \text{ ms} \quad \checkmark$
 contenido NO cacheable: $3 \times 10 + 1 \times 90 + 10 = 130 \text{ ms} \quad \checkmark$ (aún mejor: TLS al borde)
 respuesta dinámica sin reutilizar conexión al origen: puede superar los 360 ms $\times$

La tercera línea es la que sorprende: el edge mejora incluso contenido dinámico si mantiene conexiones calientes hacia el origen, porque ahorra el establecimiento TCP y TLS. Si no las mantiene, añade un tramo y no ahorra nada. Por eso el origen shielding no es un extra: es lo que hace que el borde funcione para contenido no cacheable.

5. Las zonas son etiquetas por cuenta

Un detalle operativo que produce errores difíciles de diagnosticar: en varios proveedores, el nombre de zona que ves —us-east-1a— está mapeado de forma distinta en cada cuenta. La us-east-1a de tu cuenta y la de otra pueden ser centros de datos físicos diferentes.

El mapeo se hizo para repartir la carga: si todos los clientes eligen «la primera zona», la primera zona física se satura. Aleatorizar la correspondencia por cuenta distribuye esa preferencia.

Las consecuencias:

1. Comparar despliegues entre cuentas por nombre de zona no tiene sentido. «Ambos están en 1a» no significa que compartan riesgo ni que estén cerca.
2. Colocar recursos de dos cuentas en la misma zona física —para latencia o para coste de transferencia— exige el identificador estable de zona (use1-az4), no el nombre.

3. Un incidente que el proveedor comunica por zona física afecta a nombres distintos según la cuenta.

Además, no todas las regiones tienen el mismo número de zonas —las hay con 3 y con 6— ni todos los servicios están en todas ellas. Diseñar «para tres zonas» y desplegar en una región de dos rompe el supuesto de disponibilidad calculado antes, sin ningún aviso.

Ejemplo trabajado

CloudShop debe elegir topología para su plataforma de pedidos con dos requisitos: 99,95 % mensual y RPO de 15 minutos. Se evalúan tres opciones con la misma aritmética.

Disponibilidad objetivo traducida a tiempo:

99,95 % mensual $\rightarrow 0,0005 \times 30 \times 24 \times 60 = 21,6$ min de caída permitida

Opción 1 — una zona. Con 99,5 % por zona:

A = 99,5 % $\rightarrow 3,6$ h/mes ✗ incumple por un factor de 10

Opción 2 — dos zonas, base de datos con réplica síncrona.

cómputo: $1 - 0,005^2 = 99,9975$ %

base de datos síncrona entre zonas (latencia 1,4 ms medida)

A compuesta $\approx 99,995$ % $\rightarrow 1,3$ min/mes ✓ cumple con holgura

RPO = 0 (síncrona) ✓

Coste de transferencia entre zonas, con 40 TB mensuales de tráfico interno:

sin afinidad: $2/3 \times 40.000 \text{ GB} \times 2 \times 0,01 = 533$ USD/mes

con afinidad de zona: ~ 5 % = 40 USD/mes

Opción 3 — dos regiones, réplica asíncrona.

A $\approx 99,999$ % ✓

RPO: retraso de replicación medido, p95 = 4,2 min ✓ dentro de los 15

transferencia entre regiones: $40 \text{ TB} \times 0,02 = 800$ USD/mes

coste de capacidad duplicada: ≈ 6.500 USD/mes

Comparación final contra los requisitos:

Opción 1 · 1 zona	99,5 %	0	—	✗
Opción 2 · 2 zonas	99,995 %	0	40 USD	✓
Opción 3 · 2 regiones	99,999 %	4,2 min	7.300 USD	✓

Se elige la opción 2. La 3 cumple mejor y cuesta 180 veces más; la mejora de 1,3 min a 0,3 min mensuales no la percibe ningún cliente y el requisito es 21,6.

Antes de cerrar se verifica el error clásico —redundancia sobre el componente menos redundante—:

```
$ aws rds describe-db-instances --query 'DBInstances[0].[MultiAZ,AvailabilityZone,SecondaryAvailabilityZone]'
[true, "us-east-1a", "us-east-1c"] # la base SÍ cruza zona
$ aws ec2 describe-availability-zones --query 'AvailabilityZones[].[ZoneName,ZoneId]' --output text
us-east-1a use1-az4
us-east-1c use1-az6 # zonas físicas distintas: independientes
```

Se registra el identificador estable, no el nombre, porque el nombre no es comparable entre cuentas.

Límite explícito declarado en el ADR: la opción 2 no sobrevive a un fallo regional completo. Ese riesgo se acepta conscientemente; si el negocio cambia el requisito a continuidad regional, la decisión se revisa y el sobre coste de 7.300 USD/mes pasa a estar justificado. Escribirlo evita que dentro de un año alguien descubra el límite durante un incidente.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/014-regiones-zonas-de-disponibilidad-puntos-de-presencia-y-edge/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.

2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mapa-de-topologia en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye mapa-de-topologia para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se despliega cómputo en tres zonas y el sistema cae igual cuando falla una	La base de datos seguía en una sola zona: la disponibilidad la fija el componente menos redundante	Calcula la disponibilidad compuesta de la cadena completa, no del componente más visible.
Aparece una partida de transferencia inesperada de cientos de dólares	El tráfico interno cruza zonas y se cobra en ambos sentidos	Aplica afinidad de zona para el tráfico este-oeste y mide qué fracción cruza.
Se añade un CDN y la latencia de las respuestas dinámicas empeora	El borde añade un salto y no mantenía conexiones calientes al origen	Activa origin shielding y conexiones persistentes, o no pases el tráfico dinámico por el borde.
Dos cuentas creen estar en la misma zona y no lo están	El nombre de zona se mapea distinto por cuenta	Usa el identificador estable de zona para cualquier comparación o colocación entre cuentas.
Un diseño calculado para tres zonas se despliega en una región que solo tiene dos	Se asumió un número de zonas uniforme entre regiones	Verifica zonas disponibles y servicios ofrecidos por región antes de fijar el modelo de disponibilidad.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Con 99,5 % por zona, ¿cuánta indisponibilidad mensual queda con una, dos y tres zonas? ¿Qué justifica la tercera?

2. Tres réplicas de cómputo en tres zonas contra una base de datos en una sola. ¿Cuál es la disponibilidad del conjunto?
3. ¿Por qué la replicación entre regiones es asíncrona y qué implica eso para el RPO?
4. ¿En qué caso concreto un punto de presencia empeora la latencia de una respuesta dinámica?
5. ¿Por qué no puedes comparar la zona 1a de dos cuentas distintas, y qué identificador sí es comparable?

Referencias

- AWS (2024). Regions, Availability Zones, and Local Zones — identificadores estables de zona y su mapeo por cuenta. <<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-regions-availability-zones.html>>
- Microsoft (2024). Azure regions and availability zones — modelo de zonas y servicios con soporte por región. <<https://learn.microsoft.com/en-us/azure/reliability/availability-zones-overview>>
- Google Cloud (2024). Geography and regions — jerarquía región/zona y ubicación de recursos. <<https://cloud.google.com/docs/geography-and-regions>>
- Beyer, B. et al., eds. (2016). Site Reliability Engineering, cap. 22 «Addressing Cascading Failures» — independencia de dominios de fallo. <<https://sre.google/sre-book/addressing-cascading-failures/>>
- Brewer, E. (2012). CAP Twelve Years Later: How the Rules Have Changed. IEEE Computer 45(2) — por qué la latencia entre regiones fuerza replicación asíncrona. <<https://doi.org/10.1109/MC.2012.37>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 014 Regiones, zonas de disponibilidad, puntos de presencia y edge

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega mapa-de-topología con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

015 — IaaS, PaaS, SaaS, CaaS y FaaS

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Elegir modelo de servicio por lo que cede y lo que retiene, no por comodidad aparente. Cada escalón hacia arriba entrega operación al proveedor y a cambio impone sus límites: cuotas, tiempos máximos, runtimes soportados y una forma concreta de bloqueo. Esta clase da el criterio con el que se decidirá entre EC2, Fargate y Lambda —o sus equivalentes— en las partes 17, 18 y 19.

Resultados de aprendizaje

Al finalizar podrás:

1. Situar una carga en el modelo adecuado a partir de su perfil de ejecución y su tolerancia a límites.
2. Calcular el punto de cruce económico entre función y contenedor permanente para una carga dada.
3. Anticipar qué límites duros impone cada modelo y cuáles no se pueden negociar.
4. Distinguir bloqueo de datos, de API y operativo, y estimar el coste de salida de cada uno.
5. Justificar por qué CaaS y FaaS no son escalones sucesivos sino respuestas a preguntas distintas.

Conceptos centrales

Concepto	Comprensión verificable
arranque en frío	Latencia adicional cuando no hay instancia caliente y hay que crear el entorno de ejecución. Va de decenas de milisegundos a varios segundos según runtime y tamaño del paquete, y solo afecta a una fracción de las invocaciones.
concurrency	Ejecuciones simultáneas. En FaaS es la unidad de escalado y de cuota; en CaaS y IaaS se deriva del número de instancias y de trabajadores por instancia.
bloqueo de proveedor	Coste de migrar a otro proveedor. No es binario: se descompone en datos, API y operación, y cada componente tiene un coste de salida distinto.
plano de control	Parte del servicio que gestiona el ciclo de vida —crear, escalar, actualizar—. Al subir de modelo se cede el plano de control y con él la capacidad de intervenir cuando falla.
granularidad de facturación	Unidad mínima que se cobra: hora, segundo o milisegundo con memoria asignada. Determina si una carga intermitente es barata o ruinosa.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

Flujo de razonamiento

```
flowchart LR
    Q1{"¿Ejecución continua<br/>o por eventos?"}}
    Q1 -->|"continua"| Q2{"¿Necesitas controlar<br/>el sistema operativo?"}}
    Q1 -->|"por eventos<br/>e intermitente"| Q3{"¿Cabe en los límites<br/>de la plataforma?"}}
    Q2 -->|"sí: núcleo, drivers,<br/>agentes"| IAAS["IaaS"]
    Q2 -->|"no"| CAAS["CaaS · contenedores"]
    Q3 -->|"sí: duración, memoria,<br/>tamaño de paquete"| FAAS["FaaS · funciones"]
    Q3 -->|"no"| CAAS
    IAAS -.->|"cedes: parcheo, escalado"| CAAS
    CAAS -.->|"cedes: runtime, capacidad"| FAAS
```

Desarrollo

1. Cada escalón cede operación y recibe límites

Subir de modelo no es «más fácil»: es un intercambio explícito. Se entrega trabajo operativo y se aceptan restricciones que antes no existían.

	IaaS	CaaS	FaaS
Tú operas	SO, parches, escalado, runtime	Imagen y réplicas	Solo el código
El proveedor opera	Hardware, hipervisor	+ SO y orquestación	+ capacidad y escalado
Unidad de escalado	Instancia (minutos)	Tarea (decenas de s)	Invocación (ms)
Facturación	Por segundo, encendido	Por segundo, con tarea viva	Por ms × memoria, solo al ejecutar
Límite duro típico	Cuota de instancias	Cuota de tareas	Duración máxima
Estado local	Persistente	Efímero por tarea	Efímero, no garantizado

La fila de límites duros es la decisiva y la que más tarde se descubre. En FaaS existe un techo de duración por invocación —15 minutos en AWS Lambda, 60 en Cloud Run functions de 2.ª generación, 10 en el plan de consumo de Azure Functions— que no es negociable con soporte. Una carga que hoy tarda 8 minutos y crece un 20 % anual choca contra el techo en tres años, y la migración no será un ajuste de configuración: será reescribir el modelo de ejecución.

La fila de facturación explica el resto: FaaS solo cobra mientras ejecuta. Para una carga que corre 50 ms cada 10 minutos, eso es la diferencia entre pagar 0,4 segundos al día y pagar 24 horas.

2. El punto de cruce económico se calcula, no se opina

La comparación entre función y contenedor permanente tiene un umbral concreto. Con precios orientativos:

FaaS: 0,0000166667 USD por GB-segundo + 0,20 USD por millón de invocaciones

CaaS: ~0,04 USD por hora de 1 vCPU + 2 GB (tarea permanente)

Para una función de 512 MB que tarda 200 ms, con N invocaciones al mes:

$$\begin{aligned}\text{coste FaaS}(N) &= N \times 0,2 \text{ s} \times 0,5 \text{ GB} \times 0,0000166667 + N \times 0,0000002 \\ &= N \times (1,6667e-6 + 2e-7) = N \times 1,8667e-6\end{aligned}$$

$$\text{coste CaaS} = 730 \text{ h} \times 0,04 = 29,20 \text{ USD/mes (fijo)}$$

$$\begin{aligned}\text{umbral: } N &= 29,20 / 1,8667e-6 \approx 15,6 \text{ millones de invocaciones/mes} \\ &\approx 6 \text{ invocaciones por segundo sostenidas}\end{aligned}$$

Por debajo de 6 invocaciones por segundo sostenidas, la función sale más barata; por encima, el contenedor. Y el umbral se desplaza con la duración: si la función tardara 2 segundos en vez de 200 ms, el cruce bajaría a 1,56 millones —0,6 por segundo—, porque el coste de FaaS es lineal en el tiempo de ejecución.

Dos matices que el cálculo simple omite:

- El contenedor permanente necesita al menos dos réplicas para tolerar fallos, así que su coste real se duplica y el umbral sube.
- FaaS escala a cero, así que en entornos de pruebas o cargas estacionales su ventaja es mayor que la que sugiere la media.

3. El arranque en frío importa menos de lo que se teme y más de lo que se mide

El arranque en frío se cita como el argumento contra FaaS y casi siempre se estima mal en ambas direcciones.

Afecta solo a las invocaciones que no encuentran instancia caliente. Con tráfico sostenido, esa fracción es pequeña:

invocaciones/mes	2.000.000	
arranques en frío medidos	12.000	→ 0,6 %
latencia p50 en caliente	45 ms	
latencia p50 en frío	820 ms	

impacto en p50 global: despreciable

impacto en p99: el p99 ES el arranque en frío si supera el 1 %

La segunda línea es la que se olvida: con un 0,6 % de arranques en frío, el p99 global no los ve; con un 1,5 %, el p99 pasa a ser el arranque en frío. La pregunta correcta no es «¿hay arranque en frío?» sino «¿qué percentil de mi SLO cae dentro de esa fracción?».

Los factores que la determinan, por orden de impacto:

1. Tamaño del paquete: hay que descargarlo y descomprimirlo. Un artefacto de 250 MB arranca en frío mucho peor que uno de 5 MB.
2. Runtime: los que necesitan inicializar una máquina virtual —JVM, .NET— pagan más que los interpretados o compilados a nativo.
3. Conectividad a red privada: adjuntar la función a una VPC añadía segundos históricamente; hoy es del orden de decenas de milisegundos, pero conviene medirlo y no asumirlo.

Mitigaciones reales: concurrencia aprovisionada —que elimina el ahorro de escalar a cero y cambia la aritmética anterior—, reducir el paquete, y mantener caliente solo la ruta crítica.

4. Bloqueo: tres componentes con costes de salida distintos

«Bloqueo de proveedor» se usa como una sola cosa y son tres, con coste de salida muy desigual:

Tipo	Ejemplo	Coste de salida
De datos	Formato propietario, egreso de 40 TB	Alto: se paga por GB y lleva tiempo
De API	SDK específico incrustado en el dominio	Medio: reescribir adaptadores
Operativo	Runbooks, formación, herramientas del equipo	El más alto y el menos contabilizado

El orden habitual de preocupación es el inverso al orden de coste. Se discute mucho sobre no usar un servicio propietario y nada sobre que el equipo solo sabe operar una consola.

El bloqueo crece con el modelo de servicio, pero no de forma uniforme:

- IaaS: bajo en API —una máquina virtual es una máquina virtual—, alto en datos si el volumen es grande.
- CaaS: bajo si la imagen es OCI estándar y el orquestador es Kubernetes; alto si se usan extensiones propietarias del proveedor.
- FaaS: alto en API —el modelo de eventos, los disparadores y el formato de entrada son específicos— pero el código de negocio puede aislarse tras un adaptador fino.

La estrategia práctica no es evitar el bloqueo, que es imposible y caro: es saber cuánto cuesta salir y decidir a sabiendas. Un servicio propietario que ahorra 30.000 USD al año con un coste de salida estimado de 40.000 es una decisión razonable; el problema es no haber calculado nunca el segundo número.

5. CaaS y FaaS no son escalones, son respuestas distintas

La presentación habitual como escalera —IaaS, PaaS, CaaS, FaaS, cada uno «más gestionado»— sugiere que FaaS es la meta. No lo es: responden a preguntas diferentes.

CaaS responde a: quiero portabilidad del artefacto y control del entorno de ejecución, sin gestionar sistemas operativos. La imagen OCI corre igual en cualquier sitio; el coste es que sigues decidiendo réplicas, límites y sondas.

FaaS responde a: quiero que la unidad de escalado sea la petición y no pagar por capacidad ociosa. El coste es aceptar los límites de la plataforma y un modelo de eventos específico.

Una carga puede ser mala candidata para FaaS y excelente para CaaS aunque sea moderna y sin estado: basta con que su duración supere el techo, que necesite conexiones persistentes —WebSocket largo— o que su patrón de tráfico sea sostenido y alto, donde el umbral calculado antes favorece al contenedor.

Y al revés: una tarea programada que corre 3 segundos cada hora es ruinosa como contenedor permanente —730 horas facturadas para 0,73 horas de trabajo, un 0,1 % de utilización— y trivial como función.

El criterio es el perfil de ejecución, no la modernidad del modelo.

Ejemplo trabajado

CloudShop tiene tres cargas que hoy corren en máquinas virtuales y quiere decidir modelo para cada una. Se aplican las dos preguntas del árbol y la aritmética.

Carga A — generación de miniaturas al subir una imagen.

perfil	por eventos, 8.400 invocaciones/día, 1,2 s cada una, 512 MB
límites	duración 1,2 s « techo de 15 min ✓ cabe
coste FaaS	$8.400 \times 30 \times 1,2 \text{ s} \times 0,5 \text{ GB} \times 1,6667\text{e-}6 + \text{invocaciones}$ $= 252.000 \times 1,0\text{e-}6 + 0,05 = 0,30 \text{ USD/mes}$
coste CaaS	2 réplicas $\times 29,20 = 58,40 \text{ USD/mes}$
utilización	$252.000 \times 1,2 \text{ s} / (730 \times 3600) = 11,5 \%$

FaaS, por un factor de 195. La utilización del 11,5 % es exactamente el caso que la función resuelve.

Carga B — API de catálogo.

perfil	continua, 610 peticiones/s en pico, 45 ms, 512 MB
umbral calculado:	$\sim 6 \text{ inv/s} \rightarrow 610$ está 100 veces por encima
coste FaaS	$610 \times 2,6\text{e}6 \text{ s/mes} \times 0,045 \text{ s} \times 0,5 \times 1,6667\text{e-}6 \approx 1.190 \text{ USD/mes}$
coste CaaS	por ley de Little: $L = 610 \times 0,045 = 27,5$ concurrentes a $p=0,7 \rightarrow 40$ concurrentes $\rightarrow 4$ tareas de 16 $\rightarrow 4 \times 29,20 = 117 \text{ USD/mes}$

CaaS, por un factor de 10. Tráfico sostenido y alto: el contenedor gana con holgura.

Carga C — procesamiento nocturno de conciliación contable.

perfil	una vez al día, 38 min actuales, crecimiento 20 % anual
límites	38 min > techo de 15 min de FaaS ✗ no cabe y en 3 años serán 66 min
necesita	acceso a un driver de base de datos legado con biblioteca nativa

Dos exclusiones independientes: excede el techo de duración y necesita control del entorno para una biblioteca nativa. La segunda también descarta CaaS con imágenes mínimas sin capacidad de instalar dependencias del sistema.

resultado	IaaS o CaaS con imagen completa; se elige CaaS como tarea programada
coste:	$38 \text{ min/día} \times 30 = 19 \text{ h/mes} \times 0,04 = 0,76 \text{ USD/mes}$
frente a una VM encendida	$730 \text{ h} = 29,20 \text{ USD/mes}$

Decisión final y bloqueo estimado:

A · miniaturas	FaaS	0,30 USD	alto (eventos)	~ 2 días de trabajo
B · API catálogo	CaaS	117,00 USD	bajo (OCI)	~ 0 : imagen portable
C · conciliación	CaaS	0,76 USD	bajo	~ 0

El bloqueo alto de A se acepta explícitamente: el adaptador de eventos son unas 40 líneas y el resto de la lógica es agnóstica. Se registra en el ADR que la lógica de negocio no debe importar tipos del SDK del proveedor, que es lo que convierte dos días en dos meses cuando llega la migración.

Ahorro total frente a las tres máquinas virtuales actuales ($3 \times 29,20 = 87,60 \text{ USD}$, más operación): el coste baja a 118 USD pero la carga B pasa de una VM a cuatro tareas, dimensionadas por la ley de Little en vez de por costumbre. El ahorro real no está en la factura sino en dejar de parchear tres sistemas operativos.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/015-iaas-paas-saas-caas-y-faas/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-de-servicios en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-de-servicios para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una carga migrada a funciones empieza a fallar cuando crece	Se acercó al techo de duración por invocación, que no es negociable	Comprueba el margen contra el techo con la proyección de crecimiento antes de elegir FaaS.
El coste de las funciones supera al de los contenedores que sustituyeron	El tráfico sostenido está por encima del punto de cruce	Calcula el umbral con tu duración y memoria reales; con tráfico alto y continuo, el contenedor gana.
Se activa concurrencia aprovisionada y desaparece el ahorro esperado	La concurrencia aprovisionada elimina el escalado a cero, que era la fuente del ahorro	Aprovisiona solo la ruta crítica y recalcula el umbral con ese coste fijo incluido.
El p99 se dispara aunque el p50 esté bien	La fracción de arranques en frío supera el 1 % y pasa a dominar el percentil del SLO	Mide la fracción real y compárala con el percentil que fija tu SLO, no con la media.
Migrar de proveedor cuesta meses pese a usar contenedores portables	El bloqueo operativo —runbooks, herramientas, formación— no se contabilizó	Estima los tres tipos de bloqueo por separado; el operativo suele ser el mayor.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Una función de 512 MB tarda 200 ms. ¿A partir de cuántas invocaciones por segundo sale más barato un contenedor permanente, y cómo cambia si tardara 2 s?
2. ¿Por qué un 0,6 % de arranques en frío no afecta al p99 y un 1,5 % sí?
3. Una carga sin estado, moderna y de 40 minutos. ¿Es candidata a FaaS? Justifica con el límite concreto.
4. Ordena los tres tipos de bloqueo por coste de salida y explica por qué el orden de preocupación suele ser el inverso.
5. Una tarea corre 3 segundos cada hora. ¿Qué utilización tendría como contenedor permanente y qué modelo corresponde?

Referencias

- Mell, P. y Grance, T. (2011). The NIST Definition of Cloud Computing, SP 800-145 — los tres modelos de servicio originales. <<https://doi.org/10.6028/NIST.SP.800-145>>
- AWS (2024). Lambda quotas — techo de duración, memoria, tamaño de paquete y concurrencia. <<https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html>>
- Google Cloud (2024). Cloud Run functions: quotas and limits — límites de la 1.^a y 2.^a generación. <<https://cloud.google.com/functions/quotas>>
- Jonas, E. et al. (2019). Cloud Programming Simplified: A Berkeley View on Serverless Computing — límites estructurales del modelo de funciones. <<https://doi.org/10.48550/arXiv.1902.03383>>
- Hohpe, G. (2020). Cloud Strategy, cap. sobre bloqueo — descomposición del coste de salida en datos, API y operación.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 015 IaaS, PaaS, SaaS, CaaS y FaaS

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-de-servicios con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

016 — Elasticidad, escalabilidad, disponibilidad y resiliencia

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Separar cuatro palabras que se usan como sinónimos y significan cosas distintas: elasticidad, escalabilidad, disponibilidad y resiliencia. Confundirlas produce sistemas que escalan pero no sobreviven, o que sobreviven pero cuestan el triple. Aquí se establece la aritmética —ley de Amdahl, ley universal de escalabilidad, disponibilidad compuesta— con la que se dimensionará el resto del programa.

Resultados de aprendizaje

Al finalizar podrás:

1. Definir las cuatro propiedades por lo que garantizan y por lo que no, con un ejemplo de sistema que cumple una y falla otra.
2. Aplicar la ley de Amdahl para acotar la ganancia máxima de paralelizar una carga.
3. Explicar por qué a partir de cierto punto añadir nodos empeora el rendimiento, usando la ley universal de escalabilidad.
4. Dimensionar un autoescalado con umbrales, periodos de enfriamiento y margen de arranque que no oscile.
5. Distinguir MTBF de MTTR y justificar por qué reducir el segundo suele ser más barato que aumentar el primero.

Conceptos centrales

Concepto	Comprensión verificable
escalabilidad	Capacidad de aumentar la capacidad al añadir recursos. Es una propiedad del diseño y tiene un techo determinado por la fracción no paralelizable y por el coste de coordinación.
elasticidad	Capacidad de ajustar la capacidad a la demanda de forma automática y en ambas direcciones. Presupone escalabilidad, pero un sistema escalable puede no ser elástico si nadie automatiza el ajuste.
disponibilidad	Fracción del tiempo en que el sistema responde correctamente. Se mide como $MTBF/(MTBF+MTTR)$, lo que revela que hay dos palancas y no una.
resiliencia	Capacidad de seguir prestando servicio, aunque degradado, mientras algo falla, y de recuperarse después. Es lo que hace que un fallo no se convierta en una caída.
coherencia	En la ley universal de escalabilidad, el coste de mantener sincronizados N nodos. Crece con N^2 y es lo que hace que añadir nodos llegue a empeorar el rendimiento total.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
E["Escalabilidad<br/>¿puedo crecer?"] --> EL["Elasticidad<br/>¿crezco y decrezco solo?"]
D["Disponibilidad<br/>¿responde ahora?"] --> R["Resiliencia<br/>¿sobrevive al fallo?"]
EL -->|"sin escalabilidad<br/>no hay nada que ajustar"| E
R -->|"la resiliencia PRODUCE<br/>disponibilidad, no al revés"| D
E --- X{"independientes:<br/>un sistema puede escalar<br/>y no sobrevivir"}
D --- X
```


Desarrollo

1. Cuatro propiedades, cuatro preguntas distintas

El vocabulario se usa mal a diario y cada confusión tiene un coste concreto:

Propiedad	Pregunta	Se mide con	Falla cuando
Escalabilidad	¿Puedo atender más añadiendo recursos?	Rendimiento frente a nodos	Hay un cuello serializado
Elasticidad	¿Se ajusta solo, en ambos sentidos?	Tiempo de reacción y utilización	Solo escala hacia arriba
Disponibilidad	¿Responde ahora?	Tiempo útil / tiempo total	Un componente único cae
Resiliencia	¿Sigue sirviendo mientras falla algo?	Comportamiento bajo fallo inyectado	Todo es esencial

Las combinaciones que existen en la práctica y demuestran que son independientes:

- Escalable pero no disponible: 200 réplicas sin estado contra una única base de datos. Atiende cualquier volumen y cae entera cuando cae la base.
- Disponible pero no elástico: tres zonas sobredimensionadas al triple, siempre encendidas. Nunca cae y desperdicia el 70 % del gasto.
- Elástico pero no resiliente: escala en 90 segundos y toda dependencia es esencial. Si el servicio de recomendaciones tarda, el catálogo entero devuelve 500.
- Resiliente pero no escalable: degrada con elegancia, tiene circuit breakers y plazos, y su cuello de botella serializado le impide pasar de 400 peticiones por segundo hagas lo que hagas.

Cada una necesita un trabajo distinto. Pedir «que escale» cuando el problema es de resiliencia produce más réplicas de algo que sigue cayendo igual.

2. La ley de Amdahl pone el techo

Si una fracción p del trabajo se puede paralelizar y el resto no, la aceleración con N recursos es:

$$S(N) = 1 / ((1 - p) + p/N)$$

$$\text{límite cuando } N \rightarrow \infty: S(\infty) = 1 / (1 - p)$$

La consecuencia es brutal y contraintuitiva:

Fracción paralelizable	Aceleración máxima	Con 16 nodos
50 %	2×	1,88×
90 %	10×	6,4×
95 %	20×	9,1×
99 %	100×	13,9×

Con un 95 % paralelizable, el 5 % serializado limita la ganancia a 20× por muchos nodos que pongas. Y con 16 nodos solo se obtiene 9,1× —un 57 % de eficiencia—: la mitad del gasto no produce rendimiento.

Aplicado a una petición web, la fracción serializada suele estar en sitios concretos y localizables:

- Una transacción que toma un bloqueo sobre una fila muy consultada.
- Un contador global incrementado en cada petición.
- Una secuencia de base de datos para generar identificadores.
- Un límite de tasa implementado con un único contador central.

Encontrar y eliminar el 5 % serializado da más rendimiento que triplicar los nodos. Ese es el trabajo de ingeniería que un autoescalado no puede sustituir.

3. La ley universal de escalabilidad: cuando añadir nodos empeora

Amdahl explica el techo pero no explica algo que se observa en producción: a partir de cierto número de nodos el rendimiento baja. Gunther lo modela añadiendo un término de coherencia:

$$C(N) = N / (1 + \sigma(N - 1) + \kappa N(N - 1))$$

σ = contención (serialización, como en Amdahl)

κ = coherencia (coste de sincronizar entre nodos, crece con N^2)

Con $\sigma = 0,03$ y $\kappa = 0,0001$:

N	C(N)	Eficiencia
10	7,8×	78 %
20	12,7×	64 %
40	17,4×	44 %
57	18,6×	33 % ← máximo
80	18,2×	23 %
120	16,6×	14 %

A partir de 57 nodos, cada nodo adicional reduce el rendimiento total. El término κN^2 es el coste de que todos se pongan de acuerdo: réplicas que sincronizan estado, nodos de caché que se invalidan entre sí, miembros de un clúster que intercambian latidos.

Esto explica un patrón que de otro modo parece magia: partir un clúster de 100 nodos en cuatro de 25 mejora el rendimiento agregado, porque κ actúa dentro de cada partición y no entre ellas. Es el fundamento de la celdas y del particionado que aparecerán en la parte 12.

Ajustar σ y κ a un sistema real requiere medir el rendimiento con varios tamaños de clúster. El valor del modelo no es predecir con precisión, sino saber que el óptimo existe y es finito.

4. Disponibilidad: dos palancas, una mucho más barata

La definición operativa revela algo que la cifra de «nueves» oculta:

$$A = \text{MTBF} / (\text{MTBF} + \text{MTTR})$$

MTBF = tiempo medio entre fallos

MTTR = tiempo medio de reparación

Con MTBF de 30 días y MTTR de 4 horas:

$$A = 720 / (720 + 4) = 99,45 \%$$

Para llegar a 99,95 % hay dos caminos:

A) subir MTBF a 8.000 h (11 meses sin fallos) manteniendo MTTR de 4 h

B) bajar MTTR a 22 min manteniendo MTBF de 30 días

El camino A exige hacer el sistema 11 veces más fiable; el B, detectar y recuperar 11 veces más rápido. El segundo casi siempre es más barato y, sobre todo, es alcanzable: automatizar la conmutación, tener runbooks ejecutables y alertar sobre síntomas en vez de causas.

Esto reordena las prioridades de inversión. Un equipo que gasta en hacer improbable el fallo obtiene rendimientos decrecientes; uno que gasta en detectar rápido y recuperar automáticamente mejora la disponibilidad de forma lineal con el esfuerzo.

Y hay un tercer factor que la fórmula no muestra: reducir el radio de impacto cambia el numerador de la ecuación de negocio. Un fallo que afecta al 5 % de los usuarios durante 4 horas no es equivalente a uno que afecta al 100 %, aunque la disponibilidad medida como tiempo sea idéntica. Por eso las celdas y las implantaciones progresivas mejoran la experiencia sin tocar ni MTBF ni MTTR.

5. Autoescalado que no oscila

Un autoescalado mal configurado produce oscilación: añade capacidad, la métrica baja, retira capacidad, la métrica sube, y el ciclo se repite consumiendo más de lo que ahorra.

Cuatro parámetros y su razón:

umbral de subida	70 %	← por la clase 011: por encima, la latencia se dispara
umbral de bajada	40 %	← histéresis: NO 65 %, o entra en ciclo
enfriamiento	300 s	← > tiempo de arranque + estabilización
margen de arranque	120 s	← lo que tarda una instancia en servir tráfico

La histéresis —la banda entre 40 % y 70 %— es lo que evita el ciclo. Si los umbrales estuvieran en 70 % y 65 %, al añadir una instancia sobre nueve la utilización caería de 71 % a 64 % y dispararía inmediatamente la bajada.

El margen de arranque impone un límite físico a lo que el autoescalado puede resolver: si una instancia tarda 120 segundos en servir tráfico, un pico que se duplica en 30 segundos no se puede absorber escalando. Solo hay tres respuestas posibles:

1. Capacidad preexistente suficiente para el pico (cara pero simple).
2. Encolar y absorber el pico con latencia mayor, no con más capacidad.
3. Rechazar con 429 y Retry-After, protegiendo lo que ya está en curso.

La tercera es la que casi nadie implementa y la que evita el colapso: degradar el servicio a una fracción de los usuarios es preferible a caer para todos. Es resiliencia, no escalabilidad, y por eso este punto pertenece a esta clase y no a la de capacidad.

Ejemplo trabajado

El catálogo de CloudShop sufre degradación en las promociones. La propuesta del equipo es «poner más réplicas»: de 12 a 40. Se comprueba con la aritmética antes de gastar.

Mediciones con distintos tamaños de clúster, a carga saturante:

nodos	rendimiento (rps)	eficiencia
4	3.100	100 %
8	5.520	89 %
12	7.190	77 %
16	8.320	67 %
24	9.410	51 %

Ajustando la ley universal de escalabilidad a estos puntos:

$\sigma \approx 0,042$ (contención)
 $\kappa \approx 0,00035$ (coherencia)

$N_{\text{óptimo}} = \sqrt{((1 - \sigma)/\kappa)} = \sqrt{(0,958/0,00035)} \approx 52$ nodos

$C(52) \approx 11.900$ rps

$C(40) \approx 11.400$ rps

$C(24) \approx 9.410$ rps

Pasar de 12 a 40 nodos cuesta 3,3 veces más y produce 1,59 veces el rendimiento. La eficiencia cae del 77 % al 40 %: se pagan 28 nodos para obtener el trabajo de 11.

Se busca el origen de $\sigma = 0,042$ antes de aceptar ese gasto:

-- consulta más frecuente durante el pico
SELECT stock FROM productos WHERE id = \$1 FOR UPDATE;

Un bloqueo pesimista sobre la fila del producto en promoción. Todas las peticiones del artículo destacado se serializan sobre la misma fila: ese es el 4,2 % que la ley detectaba sin saber dónde estaba.

Corrección: se sustituye por un contador particionado en 16 fragmentos con reconciliación asíncrona, patrón que reaparecerá en la parte 12.

nueva medición tras el cambio:

nodos	rendimiento	eficiencia
12	9.980	92 %
16	12.900	89 %
24	18.100	83 %

$\sigma \approx 0,008$ $\kappa \approx 0,00009$
 $N_{\text{óptimo}} = \sqrt{(0,992/0,00009)} \approx 105$ nodos

Con 16 nodos se obtienen 12.900 rps: más que con 40 nodos del diseño anterior (11.400), a un 40 % del coste.

propuesta original	40	11.400	3,3×	285
tras eliminar el bloqueo	16	12.900	1,3×	806

Se fija el autoescalado con los parámetros calculados:

mínimo 12 · máximo 24 · subida al 70 % · bajada al 40 %
enfriamiento 300 s · margen de arranque medido 118 s

Y se añade lo que el autoescalado no puede resolver: el pico de la promoción se duplica en 25 segundos, por debajo de los 118 de arranque. Se implementa rechazo con 429 y Retry-After al superar el 85 % de utilización, que protege las peticiones en curso en vez de degradar todas.

La lección: la pregunta no era cuántas réplicas, sino cuál era σ . Encontrar el 4,2 % serializado valió más que triplicar el clúster.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/016-elasticidad-escalabilidad-  
disponibilidad-y-resiliencia/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-de-capacidad en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modelo-de-capacidad para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se añaden nodos y el rendimiento total baja	El término de coherencia κN^2 supera la ganancia; se pasó el óptimo	Mide con varios tamaños, ajusta σ y κ , y particiona en clústeres menores en vez de crecer uno solo.
Escalar de 12 a 40 réplicas mejora un 60 % y cuesta un 230 % más	Hay una fracción serializada que la ley de Amdahl limita	Localiza el cuello serializado —bloqueos, contadores, secuencias— antes de escalar.
El autoescalado añade y quita instancias en ciclo continuo	Umbral de subida y bajada demasiado próximos: no hay histéresis	Separa los umbrales (por ejemplo 70 % y 40 %) y fija un enfriamiento mayor que el arranque.
Un pico brusco tumba el servicio pese al autoescalado	El pico crece más rápido que el tiempo de arranque; escalar no llega a tiempo	Añade rechazo con 429 y Retry-After, o capacidad preexistente; el escalado no resuelve picos por debajo del margen de arranque.
Se invierte en fiabilidad y la disponibilidad apenas mejora	Se atacó el MTBF, con rendimientos decrecientes, en vez del MTTR	Reduce el tiempo de detección y recuperación: suele ser más barato y escala linealmente con el esfuerzo.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Con un 95 % del trabajo paralelizable, ¿cuál es la aceleración máxima y qué eficiencia obtienes con 16 nodos?
2. ¿Por qué partir un clúster de 100 nodos en cuatro de 25 puede mejorar el rendimiento agregado?
3. Un sistema tiene MTBF de 30 días y MTTR de 4 h. Da dos caminos para llegar a 99,95 % e indica cuál es más barato.
4. ¿Qué ocurre si los umbrales de subida y bajada del autoescalado están en 70 % y 65 %?
5. Un pico duplica el tráfico en 25 s y una instancia tarda 118 s en servir. ¿Qué tres respuestas quedan y cuál protege mejor a los usuarios en curso?

Referencias

- Amdahl, G. (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. AFIPS Conference Proceedings. <<https://doi.org/10.1145/1465482.1465560>>
- Gunther, N. (2007). Guerrilla Capacity Planning — ley universal de escalabilidad y ajuste de σ y κ .
- Beyer, B. et al., eds. (2016). Site Reliability Engineering, cap. 22 — degradación elegante y rechazo de carga. <<https://sre.google/sre-book/addressing-cascading-failures/>>
- Nygard, M. (2018). Release It!, 2.^a ed., caps. 4-5 — patrones de estabilidad frente a fallo en cascada.
- Fielding, R. y Reschke, J., eds. (2022). RFC 9110, sec. 15.5.29 — código 429 y cabecera Retry-After. <<https://www.rfc-editor.org/rfc/rfc9110#status.429>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 016 Elasticidad, escalabilidad, disponibilidad y resiliencia

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modelo-de-capacidad con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

017 — Tenancy, cuentas, suscripciones, proyectos y jerarquías

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Diseñar la jerarquía de recursos como el primer control de seguridad y de coste, no como una tarea administrativa. La frontera de cuenta es la única barrera que ningún error de permisos atraviesa, y las cuotas viven en ella: dos razones por las que esta decisión, tomada al principio, condiciona los siguientes cinco años.

Resultados de aprendizaje

Al finalizar podrás:

1. Traducir la jerarquía de los tres grandes proveedores a un modelo común de contenedor, política heredada y frontera de aislamiento.
2. Justificar la separación por cuenta frente a la separación por etiquetas usando radio de impacto y cuotas.
3. Explicar por qué una política heredada restrictiva no concede permisos y qué implica al depurar accesos.
4. Anticipar qué límites son por cuenta y cómo un vecino interno puede agotarlos.
5. Diseñar una estructura de organización que soporte crecimiento sin reorganizaciones destructivas.

Conceptos centrales

Concepto	Comprensión verificable
frontera de aislamiento	Límite que un permiso mal concedido no puede atravesar. En los tres grandes proveedores es la cuenta, suscripción o proyecto: dentro se puede fallar en la configuración; fuera hace falta un permiso explícito adicional.
política heredada	Restricción aplicada a un nodo superior de la jerarquía que acota lo máximo que se puede conceder debajo. No concede nada: solo limita el techo de lo que otras políticas pueden otorgar.
cuota	Límite de servicio asociado normalmente a la cuenta y la región. Es un recurso compartido entre todo lo que viva en esa cuenta, y por tanto una razón técnica —no organizativa— para separar.
etiqueta	Par clave-valor sobre un recurso, usado para atribución de coste y automatización. No es una frontera de seguridad: se puede modificar con permisos de escritura sobre el recurso.
unidad organizativa	Agrupador intermedio en la jerarquía que permite aplicar políticas a un conjunto de cuentas. Su diseño debe reflejar cómo se gobierna, no cómo está el organigrama, porque los organigramas cambian más rápido.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
ORG["Organización · raíz"] --> U01["U0 Producción"]
ORG --> U02["U0 No producción"]
ORG --> U03["U0 Sandbox"]
U01 --> C1["cuenta pagos-prod"]
U01 --> C2["cuenta catalogo-prod"]
U02 --> C3["cuenta catalogo-pre"]
U03 --> C4["cuenta lab-ana"]
ORG -->|"política heredada:<br/>prohíbe salir de regiones"| U01
U01 -->|"prohíbe borrar<br/>registros de auditoría"| C1
C1 --- Q["cuotas propias:<br/>no las comparte con C2"]
C2 --- Q2["cuotas propias"]
```

Desarrollo

1. Un modelo común bajo tres vocabularios

Los tres grandes proveedores implementan la misma idea con nombres distintos. Traducirla evita aprender tres veces lo mismo:

Concepto	AWS	Azure	Google Cloud
Raíz	Organización	Tenant de Entra ID	Organización
Agrupador	Unidad organizativa	Grupo de administración	Carpeta
Frontera de aislamiento	Cuenta	Suscripción	Proyecto
Agrupador interno	—	Grupo de recursos	—
Política heredada	SCP	Azure Policy	Restricción de organización

La fila en negrita es la que importa: es donde viven las cuotas, la facturación y el aislamiento real. Todo lo que esté por encima es organización; todo lo que esté por debajo es agrupación conveniente.

Dos diferencias que sí cambian el diseño:

- Azure tiene un nivel intermedio —el grupo de recursos— que agrupa recursos con ciclo de vida común dentro de una suscripción. No es frontera de aislamiento, pero sí unidad de borrado: eliminar el grupo elimina todo lo que contiene, lo que es útil para entornos efímeros y peligroso si se confunde con una frontera.
- Google Cloud permite anidar carpetas en varios niveles, mientras que AWS limita la profundidad de unidades organizativas a cinco. Diseñar jerarquías profundas es posible en uno y no en el otro.

2. Por qué la cuenta y no la etiqueta

La tentación inicial es usar una sola cuenta con etiquetas por entorno y equipo. Es más simple y es incorrecto por tres razones técnicas, no organizativas:

1. Radio de impacto. Una política de permisos mal escrita alcanza todo lo que hay en la cuenta. Con separación:

cuenta única: 1 error de permisos → producción y desarrollo
cuentas separadas: 1 error de permisos → un entorno de un producto

2. Cuotas. Casi todos los límites de servicio son por cuenta y región. Un entorno de pruebas que arranca 200 instancias para un experimento consume la cuota que producción necesitaría para escalar durante un pico. No es hipotético: es el modo de fallo más frecuente de las cuentas compartidas, y no lo previene ningún permiso.

3. Atribución de coste. Una etiqueta puede faltar; una cuenta no. Si el 30 % de los recursos no está etiquetado, ese 30 % del gasto es inatribuible. La cuenta atribuye por construcción.

Y una razón que no es técnica pero pesa: las etiquetas se pueden cambiar con permisos de escritura sobre el recurso. Quien pueda modificar una instancia puede cambiar su etiqueta entorno: produccion a entorno: desarrollo. Una etiqueta no es una frontera de seguridad, es metadato.

Lo que sí resuelven bien las etiquetas: atribución fina dentro de una cuenta, automatización y ciclos de vida. Son complementarias, no alternativas.

3. Las políticas heredadas limitan, no conceden

Es la fuente de confusión más común al depurar accesos. Una política heredada define el techo de lo que se puede conceder por debajo; el permiso efectivo es la intersección:

permiso efectivo = política heredada $\cap$ permiso de identidad $\cap$ política de recurso

De ahí dos consecuencias:

1. Adjuntar una política heredada permisiva no da acceso a nadie. Si nadie tiene un permiso de identidad que lo conceda, no hay acceso.
2. Una denegación en cualquier nivel gana siempre. No hay forma de conceder por debajo lo que se denegó arriba, ni siquiera para el administrador de la cuenta.

Ejemplo de política heredada que impide salir de regiones aprobadas:

```
{
  "Effect": "Deny",
  "NotAction": ["iam:*", "organizations:*", "support:*"],
  "Resource": "*",
  "Condition": {"StringNotEquals": {"aws:RequestedRegion": ["us-east-1", "sa-east-1"]}}
}
```

El NotAction con servicios globales no es un detalle: IAM y facturación no tienen región, así que sin exceptuarlos la política bloquea la propia administración de la cuenta. Es el error que convierte un guardrail en una cuenta inutilizable.

Al depurar «no tengo permiso» hay que revisar los tres niveles, y en ese orden: primero si algo lo deniega arriba —porque ningún cambio abajo lo arreglará—, después el permiso de identidad, y por último la política del recurso.

4. Diseñar la jerarquía por gobierno, no por organigrama

El error estructural más caro: modelar unidades organizativas según el organigrama. Los organigramas se reorganizan cada 12-18 meses; mover cuentas entre unidades cambia las políticas heredadas que se les aplican, y eso puede romper cargas en producción.

El criterio robusto es agrupar por qué política se aplica, que cambia mucho más despacio:

raíz	
■ ■ ■ ■ Producción	políticas estrictas: regiones, borrado, cifrado obligatorio
■ ■ ■ ■ pagos-prod	
■ ■ ■ ■ catalogo-prod	
■ ■ ■ ■ No producción	políticas medias: regiones, límite de gasto
■ ■ ■ ■ catalogo-pre	
■ ■ ■ ■ pagos-pre	
■ ■ ■ ■ Sandbox	políticas laxas, presupuesto duro, borrado automático
■ ■ ■ ■ lab-<persona>	
■ ■ ■ ■ Infraestructura	cuentas compartidas: red, registro de imágenes, auditoría
■ ■ ■ ■ red-central	
■ ■ ■ ■ auditoria	

Si mañana el equipo de pagos se fusiona con el de catálogo, no hay que mover ninguna cuenta: las políticas que se les aplican no dependen de quién las gestiona.

Tres cuentas que conviene separar desde el principio:

- Auditoría: recibe los registros de todas las demás y solo permite escritura desde ellas. Si un atacante compromete una cuenta, no puede borrar la evidencia de otra.
- Red central: la conectividad compartida, para que su ciclo de vida no dependa de un producto.
- Gestión de identidad: donde viven los roles que se asumen hacia las demás.

Y una regla operativa: la cuenta raíz de la organización no debe alojar cargas de trabajo. Es la única que no puede tener políticas heredadas por encima.

5. Cuotas: el límite que se descubre en el peor momento

Las cuotas son el aspecto menos visible de la frontera de cuenta y el que más incidentes causa, porque solo se manifiestan bajo carga.

Categorías, con su comportamiento:

Tipo	Ejemplo	¿Ampliable?
Blanda por cuenta	Número de instancias por región	Sí, con solicitud y días de espera
Dura por cuenta	Cuentas por organización	No, o con excepción especial
Por recurso	Conexiones por base de datos	Depende del tamaño elegido
De tasa	Peticiones por segundo a una API	A veces, con ráfaga limitada

Dos comportamientos que sorprenden:

La ampliación no es instantánea. Solicitar más cuota puede tardar de horas a días. Si se descubre durante un incidente, la cuota no es una palanca disponible: hay que haberla ampliado antes. Por eso forma parte del plan de capacidad de la parte 21 y no de la respuesta a incidentes.

Las cuotas de tasa se aplican al plano de control. Un bucle que consulta el estado de un recurso cada segundo puede agotar el límite de llamadas a la API de gestión y provocar que fallen los despliegues de todo lo demás en esa cuenta, mientras las aplicaciones siguen funcionando con normalidad. El síntoma —«no puedo desplegar pero el servicio va bien»— desconcierta hasta que se conoce la causa.

La consecuencia de diseño: monitoriza el consumo de cuota como una métrica más, con alerta al 80 %. Es de las pocas métricas que predicen un fallo con días de antelación.

Ejemplo trabajado

CloudShop opera todo en una cuenta con etiquetas por entorno. Un despliegue de pruebas deja producción sin capacidad durante el Cyber Monday. Se reconstruye qué pasó y se rediseña.

La secuencia:

```
14:02 un experimento de carga en "desarrollo" arranca 180 instancias
14:09 el autoescalado de producción intenta subir de 16 a 24
14:09 falla: VcpuLimitExceeded
14:11 latencia p95 de producción: 91 ms → 2.400 ms
14:22 alguien identifica el experimento y lo detiene
14:26 producción recupera capacidad
```

24 minutos de degradación. Ningún permiso estaba mal: el experimento tenía derecho a arrancar instancias en su entorno. El problema es que «su entorno» era una etiqueta y la cuota es de la cuenta.

```
$ aws service-quotas get-service-quota --service-code ec2 --quota-code L-1216C47A \
  --query 'Quota.[QuotaName,Value]' --output text
Running On-Demand Standard instances      256

256 vCPU de cuota total en la región
producción en régimen                      64 vCPU (16 instancias × 4)
producción en pico necesario                96 vCPU (24 × 4)
experimento                               180 vCPU
64 + 180 = 244 → quedan 12 vCPU: solo 3 de las 8 instancias que pedía
```

La etiqueta no separaba nada porque las cuotas no leen etiquetas.

Rediseño con separación por cuenta:

```
raíz
  ■■■ Producción          SCP: solo us-east-1 y sa-east-1; prohíbe borrar logs
  ■   ■■■ cloudshop-prod  cuota propia: 256 vCPU
  ■■■ No producción      SCP: mismas regiones; presupuesto 2.000 USD
  ■   ■■■ cloudshop-pre   cuota propia: 128 vCPU
  ■■■ Sandbox            SCP: presupuesto 200 USD; borrado a los 7 días
  ■   ■■■ lab-*           cuota propia: 64 vCPU
  ■■■ Infraestructura
    ■■■ red-central
    ■■■ auditoria        recibe logs; nadie más puede borrarlos
```

Se verifica que la política heredada no rompe la administración:

```
$ aws organizations describe-policy --policy-id p-regiones --query 'Policy.Content' \
  | jq -r '.Statement[0].NotAction'
["iam:*", "organizations:*", "support:*", "budgets:*", "cloudfront:*"]
```

Los servicios globales quedan exceptuados; sin eso, la política habría bloqueado la gestión de identidades de todas las cuentas hijas.

Y se añade la métrica que habría avisado con días de antelación:

```
$ aws cloudwatch put-metric-alarm --alarm-name cuota-vcpu-prod \
  --metric-name ResourceCount --namespace AWS/Usage \
  --threshold 204 --comparison-operator GreaterThanThreshold # 80 % de 256
```

Resultado en el siguiente evento de alto tráfico:

cuota compartida	sí (256 total)	no (256 solo prod)
degradación por vecino	24 min	0 min
gasto de sandbox atribuible	~30 % sin asignar	100 % por cuenta

La separación no impidió que alguien arrancara 180 instancias: impidió que eso afectara a producción. Es exactamente el principio de radio de impacto de la clase 010, aplicado a la frontera que las cuotas respetan.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/017-tenancy-cuentas-suscripciones-proyectos-
y-jerarquias/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa arbol-de-recursos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye arbol-de-recursos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un entorno de pruebas deja sin capacidad a producción	Las cuotas son por cuenta y las etiquetas no las separan	Separa entornos por cuenta, suscripción o proyecto; la etiqueta no es una frontera.
Una política heredada bloquea la administración de la cuenta	La condición de región alcanzó a servicios globales como IAM y facturación, que no tienen región	Excluye los servicios globales con NotAction al restringir por región.
Se concede una política heredada permisiva y nadie obtiene acceso	Las políticas heredadas limitan el techo, no conceden permisos	Concede con permisos de identidad; usa la herencia solo como barrera superior.
Una reorganización de equipos rompe cargas en producción	La jerarquía se modeló según el organigrama y mover cuentas cambió sus políticas heredadas	Agrupar por régimen de gobierno —producción, no producción, sandbox—, que cambia mucho más despacio.
No se puede desplegar aunque las aplicaciones funcionen bien	Se agotó la cuota de tasa del plano de control, normalmente por un bucle de consulta	Monitoriza el consumo de cuota con alerta al 80 % y aplica retroceso en los bucles de consulta.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuál es la frontera de aislamiento real en AWS, Azure y Google Cloud, y por qué las etiquetas no la sustituyen?
2. Un desarrollador arranca 180 instancias en «desarrollo» y producción no puede escalar. ¿Qué mecanismo falló y cuál no?
3. Si adjuntas una política heredada que permite todo, ¿quién obtiene acceso? Justifica con la fórmula del permiso efectivo.
4. ¿Por qué restringir por región puede inutilizar una cuenta, y qué excepción lo evita?
5. ¿Por qué la ampliación de cuota no sirve como respuesta durante un incidente?

Referencias

- AWS (2024). Organizations: service control policies — evaluación de políticas heredadas y servicios globales. <<https://docs.aws.amazon.com/organizations/latest/userguide/orgsmanagepolicies-scps.html>>
- Microsoft (2024). Cloud Adoption Framework: resource organization — grupos de administración, suscripciones y grupos de recursos. <<https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/azure-setup-guide/organize-resources>>
- Google Cloud (2024). Resource hierarchy — organización, carpetas, proyectos y herencia de políticas. <<https://cloud.google.com/resource-manager/docs/cloud-platform-resource-hierarchy>>
- AWS (2024). Organizing your AWS environment using multiple accounts — criterios de separación por cuenta. <<https://docs.aws.amazon.com/whitepapers/latest/organizing-your-aws-environment/organizing-your-aws-environment.html>>
- Beyer, B. et al., eds. (2018). The Site Reliability Workbook, cap. 11 — gestión de capacidad y cuotas. <<https://sre.google/workbook/managing-load/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 017 Tenancy, cuentas, suscripciones, proyectos y jerarquías

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega árbol-de-recursos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

018 — Identidad, roles, políticas y federación

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Construir el modelo de identidad que sostiene toda la seguridad cloud: cómo se evalúa una petición, por qué las credenciales de larga vida son el vector de brecha más frecuente y cómo eliminarlas con federación. Es la clase que hace posible que en las partes 08 y 17 un pipeline despliegue en producción sin que exista ningún secreto que robar.

Resultados de aprendizaje

Al finalizar podrás:

1. Reproducir el orden de evaluación de una petición y predecir el resultado ante políticas en conflicto.
2. Sustituir una clave de acceso permanente por una identidad federada con credenciales temporales.
3. Escribir una condición de confianza que impida que otro repositorio asuma tu rol.
4. Distinguir autenticación, autorización y auditoría, y qué registro responde a cada pregunta forense.
5. Aplicar privilegio mínimo de forma iterativa, partiendo de lo que el uso real demuestra necesario.

Conceptos centrales

Concepto	Comprensión verificable
principal	Entidad que hace una petición: usuario, rol asumido, servicio o carga de trabajo federada. Lo que importa no es quién es, sino qué permisos tiene en el momento exacto de la llamada.
rol	Conjunto de permisos que se asume temporalmente, sin credenciales propias permanentes. Al asumirlo se reciben credenciales con caducidad, lo que acota la ventana de uso de una fuga.
política de confianza	Documento que declara quién puede asumir un rol. Es la puerta: los permisos dicen qué se puede hacer una vez dentro, la confianza dice quién entra.
federación de identidad	Mecanismo por el que un proveedor externo de identidad emite un token que la nube acepta a cambio de credenciales temporales. Elimina la necesidad de almacenar secretos de larga vida.
privilegio mínimo	Conceder solo lo que el uso demuestra necesario. No es un estado inicial sino un proceso: se parte de lo observado y se recorta, porque nadie acierta la lista completa por adelantado.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    P["Petición del principal"] --> D1{"¿Alguna denegación<br/>explícita?"}}
    D1 -->|"sí"| N0["DENEGADA<br/>no hay apelación"]
    D1 -->|"no"| D2{"¿La política heredada<br/>lo permite?"}}
    D2 -->|"no"| N0
    D2 -->|"sí"| D3{"¿Permiso de identidad<br/>o de recurso lo concede?"}}
    D3 -->|"no"| N02["DENEGADA<br/>denegación implícita"]
    D3 -->|"sí"| D4{"¿Se cumplen las<br/>condiciones?"}}
    D4 -->|"no"| N0
    D4 -->|"sí"| SI["PERMITIDA"]
```

Desarrollo

1. El orden de evaluación no es negociable

Toda petición pasa por la misma secuencia, y conocerla convierte «no tengo permiso» de misterio en diagnóstico de tres pasos:

1. Denegación explícita: si algo la deniega, termina ahí. Ninguna concesión posterior la revierte.
2. Barreras heredadas: políticas de organización que acotan el techo. Si no lo permiten, se deniega aunque el permiso de identidad lo conceda.
3. Concesión: algún permiso de identidad o de recurso debe permitirlo explícitamente.
4. Condiciones: origen, hora, etiquetas, MFA, red. Si no se cumplen, se deniega.

Sin una concesión explícita el resultado es denegación implícita, que es la respuesta por defecto. Esa asimetría —todo prohibido salvo lo concedido, y lo denegado no se puede reconceder— es lo que hace el modelo analizable.

La consecuencia práctica al depurar: empieza por buscar denegaciones, no concesiones. Añadir permisos a una identidad que choca contra una denegación de organización no cambia nada, y es donde se pierden las tardes.

El caso de la política de recurso merece atención: cuando el recurso está en otra cuenta, hacen falta las dos —la de identidad en la cuenta que llama y la de recurso en la que responde—. Una sola no basta, y el mensaje de error no siempre distingue cuál falta.

2. Las credenciales de larga vida son el problema, no la solución

Una clave de acceso permanente tiene tres propiedades que la convierten en el vector más explotado:

1. No caduca: sigue siendo válida años después de filtrarse.
2. Es portable: funciona desde cualquier lugar del mundo.
3. Es silenciosa: su uso indebido es indistinguible del legítimo sin análisis de comportamiento.

Los escáneres automáticos de repositorios públicos localizan credenciales expuestas en minutos. El patrón es conocido: un desarrollador sube por error un fichero de configuración, lo borra en el commit siguiente, y la credencial sigue en el historial de Git —por lo visto en la clase 003, los objetos no desaparecen— y ya fue recogida.

La alternativa es no tener nada que robar:

clave permanente: AKIA... + secreto	válida hasta que alguien la revoque
credencial temporal: token de 1 h	inútil después, y ligada a condiciones

La jerarquía de preferencia, de mejor a peor:

Mecanismo	Vida	Dónde vive el secreto
Identidad de carga (metadata)	minutos	En ningún sitio
Federación OIDC	1 h	En ningún sitio: se intercambia un token firmado
Rol asumido con MFA	1-12 h	En la sesión
Clave permanente en gestor de secretos	indefinida	En el gestor
Clave permanente en variable de entorno	indefinida	En todas partes

Las dos primeras son cualitativamente distintas: no hay secreto que filtrar. Las demás solo reducen la probabilidad o la ventana.

3. Federación OIDC: desplegar sin secretos

El mecanismo que elimina las claves de los pipelines. El flujo, sin secretos compartidos en ningún punto:

1. El pipeline pide a su plataforma un token OIDC firmado que declara quién es: repositorio, rama, entorno, ejecución.
2. Lo presenta al proveedor cloud pidiendo asumir un rol.
3. El proveedor verifica la firma contra las claves públicas de la plataforma y comprueba que las declaraciones cumplen la política de confianza.

4. Devuelve credenciales temporales de 1 hora.

La política de confianza es donde se juega la seguridad, y donde se comete el error más peligroso:

```
{
  "Effect": "Allow",
  "Principal": {"Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"},
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
      "token.actions.githubusercontent.com:sub": "repo:miorg/cloudshop:ref:refs/heads/main"
    }
  }
}
```

La condición sobre sub es obligatoria y debe ser precisa. Los dos errores frecuentes:

"sub": "repo:miorg/*" → cualquier repositorio de tu organización
sin condición sobre sub → CUALQUIER repositorio de GitHub del mundo

El segundo caso es una brecha total: basta con que alguien cree un repositorio público, ejecute una acción y pida asumir tu rol. La firma será válida porque viene de la misma plataforma; lo único que lo distingue es la declaración sub.

Usar StringLike con comodines requiere cuidado adicional: repo:miorg/cloudshop: permite cualquier rama y cualquier entorno, incluida una rama que un colaborador externo pueda crear en un fork con permisos de escritura.

4. Privilegio mínimo es un proceso, no un estado

Nadie acierta la lista exacta de permisos por adelantado. Intentarlo produce uno de dos resultados: o se concede de más «por si acaso», o se bloquea el trabajo y alguien acaba adjuntando la política de administrador «temporalmente».

El método que funciona parte de lo observado:

1. Conceder amplio en un entorno NO productivo, con registro activado.
2. Ejercitar todos los caminos: éxito, error, reintento, borrado.
3. Extraer del registro los permisos realmente usados.
4. Generar la política a partir de esa lista.
5. Validar en preproducción y recortar lo que sobre.
6. Repetir cada trimestre: los permisos no usados en 90 días se retiran.

El paso 2 es el que se olvida y el que causa incidentes: los caminos de error usan permisos distintos. Un rol que puede escribir en una cola pero no en su cola de mensajes fallidos funciona perfectamente hasta el primer fallo, y entonces pierde mensajes en silencio.

Los proveedores ofrecen herramientas para el paso 3: analizadores que generan políticas a partir de la actividad registrada, e informes del último acceso por servicio que permiten podar sin adivinar.

Dos límites honestos del método:

- Un permiso no usado en 90 días puede ser el de la recuperación anual ante desastres. Antes de retirar hay que comprobar contra los runbooks, no solo contra el registro.
- Los permisos de solo lectura no son inocuos. s3:GetObject sobre un bucket de facturas es exactamente la brecha de la clase 010. «Solo lectura» describe el verbo, no el impacto.

5. Autenticación, autorización y auditoría responden a preguntas distintas

Las tres se confunden y cada una deja rastro en un sitio diferente. Ante un incidente, saber cuál preguntar ahorra horas:

Pregunta forense	Concepto	Dónde mirar
¿Quién era?	Autenticación	Registro del proveedor de identidad
¿Qué podía hacer?	Autorización	Políticas vigentes en ese momento
¿Qué hizo?	Auditoría	Registro de llamadas a la API

La segunda fila esconde una dificultad real: las políticas cambian. Saber qué permisos tenía un principal en el instante del incidente exige historial de configuración, no el estado actual. Sin él, la reconstrucción es una conjetura.

Y la tercera tiene un requisito que la clase 017 ya anticipó: el registro de auditoría debe vivir en otra cuenta, con escritura permitida y borrado prohibido para todas las demás. Un atacante que compromete una cuenta y puede borrar su propio rastro convierte la auditoría en decorativa.

Tres señales que conviene alertar desde el primer día, porque preceden a casi todo incidente:

- Uso de la identidad raíz de la cuenta.
- Creación de una clave de acceso permanente.
- Cambio en una política de confianza —es cómo se abre la puerta de par en par.

La tercera es la menos vigilada y la más peligrosa: modificar una condición sub de una línea puede convertir un rol seguro en uno asumible por cualquiera, y no genera ninguna alerta si nadie la configuró.

Ejemplo trabajado

El pipeline de CloudShop despliega en producción usando una clave de acceso permanente guardada como secreto del repositorio. Se migra a federación OIDC.

Estado inicial y su riesgo:

```
$ aws iam list-access-keys --user-name ci-deploy
{"AccessKeyMetadata": [{"AccessKeyId": "AKIA...", "CreateDate": "2023-03-14", "Status": "Active"}]}
$ aws iam get-user --user-name ci-deploy --query 'User.PasswordLastUsed'
null
```

antigüedad de la clave	2 años y 5 meses
rotaciones	0
copias conocidas	secreto del repo, portátil de 2 personas, un runbook
ventana de uso si se filtra	ilimitada

Paso 1 — registrar el proveedor OIDC (una sola vez por cuenta):

```
$ aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com
```

Paso 2 — política de confianza, con la condición precisa. Primero se escribe la versión insegura para verla y descartarla:

```
"Condition": {"StringEquals": {""...:aud": "sts.amazonaws.com"}}
```

Se prueba desde un repositorio de laboratorio ajeno a la organización:

```
$ (desde otro repo cualquiera) aws sts assume-role-with-web-identity ...
→ ÉXITO. Cualquier repositorio de GitHub podía asumir el rol.
```

Se corrige acotando el sujeto al repositorio, la rama y el entorno:

```
"Condition": {
  "StringEquals": {
    "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
    "token.actions.githubusercontent.com:sub": "repo:miorg/cloudshop:environment:produccion"
  }
}
```

Se repite la prueba negativa:

desde otro repositorio	→ AccessDenied ✓
desde miorg/cloudshop, rama pre	→ AccessDenied ✓
desde miorg/cloudshop, prod	→ credenciales de 1 h ✓

Paso 3 — recortar los permisos con el uso real. El rol tenía PowerUserAccess. Se extrae lo ejercitado en 30 días de despliegues:

```
$ aws accessanalyzer start-policy-generation --policy-generation-details ...
$ aws iam get-service-last-accessed-details --job-id ... \
  --query 'ServicesLastAccessed[?TotalAuthenticatedEntities>`0`].ServiceNamespace'
["s3", "cloudfront", "ecs", "ecr", "logs"]

permisos concedidos antes ~8.000 acciones (PowerUserAccess)
servicios usados en 30 días 5
```

acciones usadas	34
política final	34 acciones sobre 6 recursos concretos

Se ejercitan también los caminos de error antes de aplicar —el paso que se olvida—: un despliegue que falla necesita `ecs:StopTask` y `logs:PutLogEvents` sobre el grupo de errores, que no aparecían en los despliegues correctos.

Paso 4 — retirar la clave y verificar que nada la usa:

```
$ aws iam update-access-key --user-name ci-deploy --access-key-id AKIA... --status Inactive
# 7 días de observación sin fallos
$ aws iam delete-access-key --user-name ci-deploy --access-key-id AKIA...
$ aws iam delete-user --user-name ci-deploy
```

Resultado:

credencial	permanente, 2,5 años	token de 1 h
secretos almacenados	4 copias conocidas	0
permisos	~8.000 acciones	34 acciones
superficie si se filtra	ilimitada	1 h, solo desde ese repo y entorno

El cambio decisivo no fue reducir permisos: fue que ya no existe nada que filtrar. Un atacante que consiga el token tiene una hora, y solo desde el contexto exacto que la condición sub describe.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/018-identidad-roles-politicas-y-federacion/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce `labresult.json`. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee `exercise.steps` y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de `negativetest` y explica la señal observada.
4. Materializa `matriz-rbac` en `evidence/` usando la plantilla indicada.
5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `matriz-rbac` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Cualquier repositorio externo puede asumir el rol de despliegue	La política de confianza no condiciona la declaración sub	Condiciona siempre sobre repositorio, rama o entorno; sin ello la firma válida basta para entrar.
Se añaden permisos y el acceso sigue denegado	Una denegación explícita o una barrera heredada gana sobre cualquier concesión	Al depurar, busca primero denegaciones y políticas de organización, no concesiones.
El pipeline funciona hasta que un despliegue falla, y entonces pierde información	Los caminos de error usan permisos distintos que no se ejercitaron al generar la política	Ejercita éxito, error, reintento y borrado antes de extraer la política del registro.
Tras un incidente no se puede saber qué permisos tenía el principal	Solo se conserva el estado actual de las políticas, no su historial	Activa historial de configuración; la autorización de ayer no se deduce de la política de hoy.
Un rol de solo lectura provoca una fuga de datos	Se asumió que leer es inocuo; el impacto depende del recurso, no del verbo	Clasifica por sensibilidad del dato: leer facturas no es equivalente a leer métricas.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Una identidad tiene un permiso que concede la acción y una barrera heredada que no la incluye. ¿Cuál gana y por qué?
2. ¿Qué tiene que ocurrir para que un repositorio ajeno pueda asumir tu rol federado, y qué línea lo impide?
3. ¿Por qué generar una política solo con los permisos de los despliegues exitosos deja un fallo latente?
4. ¿Qué registro consultas para responder «qué podía hacer» frente a «qué hizo»?
5. Nombra tres eventos de identidad que conviene alertar desde el primer día y explica por qué el tercero es el menos vigilado.

Referencias

- AWS (2024). Policy evaluation logic — orden de evaluación, denegación explícita e implícita. <<https://docs.aws.amazon.com/IAM/latest/UserGuide/referencepolicievaluation-logic.html>>
- GitHub (2024). About security hardening with OpenID Connect — declaraciones del token y condiciones de confianza. <<https://docs.github.com/en/actions/deployment/security-hardening-your-deployments/about-security-hardening-with-openid-connect>>
- Sakimura, N. et al. (2014). OpenID Connect Core 1.0 — semántica de las declaraciones sub y aud. <<https://openid.net/specs/openid-connect-core-10.html>>
- NIST (2020). Zero Trust Architecture, SP 800-207 — verificación por petición y credenciales de vida corta. <<https://doi.org/10.6028/NIST.SP.800-207>>
- Dotson, C. (2023). Practical Cloud Security, 2.^a ed., caps. 4-5 — gestión de identidad y privilegio mínimo iterativo.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 018 Identidad, roles, políticas y federación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-rbac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

019 — Modelo de responsabilidad compartida por servicio

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Convertir el modelo de responsabilidad compartida de un diagrama de márketing en una matriz operativa por servicio, con controles nombrados y un responsable por cada uno. La clase 010 estableció el principio; aquí se aplica servicio a servicio, que es donde aparecen las zonas grises que nadie reclama hasta que hay un incidente.

Resultados de aprendizaje

Al finalizar podrás:

1. Construir una matriz RACI de controles para un servicio concreto, distinguiendo quién ejecuta y quién responde.
2. Localizar las zonas grises —controles que ambas partes creen del otro— antes de que las encuentre un auditor.
3. Comparar el reparto real entre una base de datos autogestionada y una gestionada, con el detalle de qué se hereda.
4. Interpretar un informe de cumplimiento del proveedor sabiendo qué controles deja explícitamente al cliente.
5. Verificar un control heredado en vez de asumirlo, distinguiendo evidencia de declaración.

Conceptos centrales

Concepto	Comprensión verificable
control heredado	Control que el proveedor ejecuta y del que obtienes el beneficio sin implementarlo. Se hereda el control, no la responsabilidad de comprobar que aplica a tu configuración.
control compartido	Control en el que ambas partes hacen algo distinto sobre la misma capa. El parcheo en IaaS es el ejemplo canónico: el proveedor parchea el hipervisor y tú el sistema operativo huésped.
zona gris	Control que ninguna de las dos partes ejecuta porque cada una supone que lo hace la otra. Es donde se concentran los hallazgos de auditoría y los incidentes evitables.
RACI	Reparto de un control en cuatro papeles: quién lo ejecuta, quién responde por él, a quién se consulta y a quién se informa. La distinción entre ejecutar y responder es la que evita las zonas grises.
informe de cumplimiento	Auditoría independiente sobre los controles del proveedor. Su sección de controles complementarios del usuario enumera lo que deja explícitamente a tu cargo, y es la parte que casi nadie lee.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    subgraph S["Un servicio concreto"]
        C1["Control: cifrado en reposo"]
        C2["Control: parcheo"]
        C3["Control: copia de seguridad"]
        C4["Control: acceso a los datos"]
    end
    end
    C1 --> H["Heredado<br/>el proveedor lo ejecuta"]
    C2 --> CO["Compartido<br/>cada uno una capa"]
    C3 --> V["¿activado por defecto?"]
    C4 --> T["Tuyo siempre"]
    V -->|no| G["ZONA GRIS<br/>existe la función,<br/>nadie la encendió"]
    V -->|sí, con retención X| OK["Heredado, con límite<br/>que hay que verificar"]
```

Desarrollo

1. Tres clases de control, no dos

La presentación habitual reparte los controles en dos columnas y esa simplificación es la que crea las zonas grises. En la práctica hay tres clases:

Clase	Quién ejecuta	Tu obligación
Heredado	Solo el proveedor	Verificar que aplica a tu configuración y obtener la evidencia
Compartido	Ambos, en capas distintas	Ejecutar tu capa y saber dónde está la frontera
Propio	Solo tú	Todo

La columna derecha de la primera fila es la que se ignora. Heredar la seguridad física del centro de datos no exime de comprobar que la región que usas está incluida en el informe de cumplimiento; heredar el cifrado de disco no exime de comprobar que está activado en tu recurso.

El ejemplo canónico de control compartido es el parcheo en IaaS:

```
hipervisor y firmware → proveedor, sin aviso ni ventana que tú controles
sistema operativo     → tú
bibliotecas del sistema → tú
runtime del lenguaje   → tú
dependencias de la app → tú
```

Una vulnerabilidad en el hipervisor la resuelve el proveedor y puede implicar un reinicio de tu instancia con preaviso corto. Una en glibc es tuya, aunque el sistema operativo venga de una imagen del proveedor. La frontera es el hipervisor, no la imagen.

2. Autogestionado frente a gestionado: qué cambia de verdad

Comparar una base de datos instalada por ti con una gestionada, control a control, revela que el reparto no se mueve en bloque:

Control	En una VM	Servicio gestionado
Parche del motor	Tú	Proveedor, en ventana que tú eliges
Cifrado en reposo	Tú lo configuras	Heredado, pero tú lo activas
Copia de seguridad	Tú	Proveedor ejecuta, tú fijas retención
Prueba de restauración	Tú	Tú — nunca la hace el proveedor
Réplica entre zonas	Tú	Proveedor, si lo activas
Esquema y consultas	Tú	Tú
Permisos de acceso	Tú	Tú
Clasificación del dato	Tú	Tú

Dos filas concentran casi todos los incidentes:

La prueba de restauración nunca es del proveedor. Que exista una copia no demuestra que se pueda restaurar, ni cuánto tarda. Una copia sin restauración probada es una suposición, y el momento de descubrirlo no puede ser el incidente. Es el control que más se hereda por error.

Casi todo lo gestionado depende de que lo actives. Cifrado, réplica, retención extendida, registro de auditoría: el proveedor los ofrece, no los impone. La diferencia entre «el servicio soporta X» y «mi instancia tiene X activado» es exactamente el espacio donde vive la zona gris.

Y hay un control que empeora al pasar a gestionado: el acceso del operador del proveedor a los datos. Es un riesgo aceptable y contractualmente acotado, pero existe y hay que nombrarlo en el análisis, no descubrirlo en una auditoría.

3. RACI: separar quién ejecuta de quién responde

La distinción entre ejecutar y responder es la que cierra las zonas grises. Un control puede ejecutarlo el proveedor y seguir siendo tu responsabilidad ante el regulador y ante tus clientes.

Para la copia de seguridad de una base de datos gestionada:

Actividad	Ejecuta	Responde	Evidencia
Tomar la copia	Proveedor	Cliente	Registro de instantáneas
Fijar la retención	Cliente	Cliente	Configuración versionada
Cifrar la copia	Proveedor	Cliente	Atributo del recurso
Probar la restauración	Cliente	Cliente	Informe con RTO medido
Verificar la integridad	Cliente	Cliente	Suma de comprobación tras restaurar

La columna «Responde» es siempre el cliente. Eso no es una carga retórica: significa que ante un fallo de copia no puedes trasladar la consecuencia al proveedor más allá del crédito del SLA, como se calculó en la clase 010.

La columna «Evidencia» es la que convierte la matriz en algo auditable. Un control sin evidencia es una intención: «el proveedor hace copias» no es evidencia; el identificador de la última instantánea y la fecha de la última restauración probada sí lo son.

Hacer esta tabla para los cinco o seis servicios principales de una plataforma lleva unas horas y localiza las zonas grises antes de que las encuentre un auditor o un incidente.

4. Leer un informe de cumplimiento por su última sección

Los informes de auditoría del proveedor —SOC 2 Tipo II, ISO 27001, y equivalentes— se usan mal: se archivan como prueba de que «el proveedor es seguro». Lo que contienen es más útil y más incómodo.

Qué mirar, en orden:

1. Alcance: qué servicios y qué regiones cubre. Un servicio nuevo o una región recién abierta pueden estar fuera, y entonces no hay nada heredado.
2. Periodo: los informes Tipo II cubren un intervalo pasado, típicamente 6 o 12 meses. Un informe cerrado hace ocho meses no dice nada de lo ocurrido después.
3. Excepciones: controles con desviaciones detectadas durante la auditoría, y qué se hizo.
4. Controles complementarios del usuario: la sección decisiva. Enumera lo que el proveedor deja explícitamente a tu cargo para que sus controles funcionen.

La cuarta sección suele contener afirmaciones muy concretas: que configures el cifrado, que gesticiones el ciclo de vida de las credenciales, que revises los permisos, que actives el registro. Son exactamente los controles que después aparecen como hallazgos.

Un informe del proveedor no te certifica. Si tu organización necesita cumplir una norma, necesitas tu propia auditoría; el informe del proveedor cubre su parte del reparto y por eso incluye la lista de lo que espera de ti.

5. Verificar lo heredado en vez de asumirlo

Heredar un control no exime de comprobar que aplica. La diferencia entre declaración y evidencia:

```
# Declaración: la documentación dice que el servicio soporta cifrado
# Evidencia: mi recurso concreto lo tiene activado
$ aws rds describe-db-instances --db-instance-identifier cloudshop-prod \
  --query 'DBInstances[0].[StorageEncrypted,KmsKeyId,BackupRetentionPeriod,MultiAZ]'
[true, "arn:aws:kms:...:key/1a2b3c", 7, true]
```

Cuatro comprobaciones en una línea, y cada una podría haber sido false sin que nada fallara visiblemente. La retención de 7 días es un dato, no una validación: hay que contrastarla con el requisito. Si el requisito legal es 30 días, el control heredado no cumple aunque esté activado.

El patrón general, aplicable a cualquier control heredado:

1. ¿Qué afirma el proveedor? documentación e informe de cumplimiento
2. ¿Aplica a mi recurso? consulta al plano de control
3. ¿Cumple mi requisito concreto? comparación con el número exigido
4. ¿Cómo lo demuestro dentro de un año? evidencia versionada y automatizada

El paso 4 es el que separa una comprobación puntual de un control: si la verificación no está automatizada, dentro de seis meses nadie sabrá si sigue siendo cierta. Convertir estas comprobaciones en pruebas que fallan el despliegue —lo que en la parte 11 será política como código— es lo que hace que un control heredado sea auditable de forma continua y no una foto de un día.

Ejemplo trabajado

Una auditoría de CloudShop devuelve un hallazgo: «no se evidencia la capacidad de restauración de la base de datos de pedidos». El equipo responde que el proveedor hace copias automáticas. Se construye la matriz para verlo.

Estado declarado frente a estado verificado:

```
$ aws rds describe-db-instances --db-instance-identifier cloudshop-prod \
  --query 'DBInstances[0].[BackupRetentionPeriod,PreferredBackupWindow,StorageEncrypted]'
[7, "07:00-07:30", true]
$ aws rds describe-db-snapshots --db-instance-identifier cloudshop-prod \
  --snapshot-type automated --query 'length(DBSnapshots)'
7
```

Las copias existen. El hallazgo no era ese:

tomar la copia	proveedor	cliente	✓ 7 instantáneas
cifrar la copia	proveedor	cliente	✓ StorageEncrypted
fijar retención	cliente	cliente	✓ 7 días
PROBAR LA RESTAURACIÓN	cliente	cliente	✗ nunca se hizo
verificar integridad tras restaurar	cliente	cliente	✗

Dos zonas grises. El equipo heredó «copia de seguridad» completo cuando solo se heredaban dos de sus cinco actividades.

Se ejecuta la restauración por primera vez, en una cuenta de no producción:

```
$ aws rds restore-db-instance-from-db-snapshot \
  --db-instance-identifier cloudshop-restore-test \
  --db-snapshot-identifier rds:cloudshop-prod-2026-08-01-07-05
$ time aws rds wait db-instance-available --db-instance-identifier cloudshop-restore-test
real    38m12s

RTO comprometido en el plan de continuidad    60 min
RTO medido en la restauración                 38 min  ✓ cumple
RPO implícito (copia diaria a las 07:00)     hasta 24 h
RPO comprometido                             15 min  ✗ INCUMPLE
```

El segundo hallazgo aparece al medir, no al auditar. La retención de 7 días cumple el requisito de conservación, pero una copia diaria da un RPO de hasta 24 horas frente a los 15 minutos comprometidos. Nadie lo había notado porque la copia existía y se asumió que eso resolvía el requisito.

Se verifica la integridad, no solo la disponibilidad:

```
-- en el original
SELECT count(*), sum(total) FROM pedidos WHERE creado < '2026-08-01 07:00';
1284471 | 48219338.55
-- en la restaurada
```


1284471 | 48219338.55 ✓ coincide

Correcciones, separando qué es de cada quién:

RPO	activar recuperación a un instante (heredado, hay que ACTIVARLO)	
	→ RPO efectivo: 5 min	✓
prueba	restauración trimestral automatizada en no producción	
	con comparación de recuento y sumas	✓ cliente
evidencia	informe versionado con RTO y RPO medidos	✓ cliente
control	regla que falla el despliegue si un recurso productivo	
	no tiene recuperación a un instante activada	✓ cliente

La última línea es la que convierte el arreglo en un control: sin ella, el próximo recurso que alguien cree volverá a nacer sin la función activada.

La lección: se heredaba el mecanismo y se creía heredar el resultado. El proveedor toma copias; que esas copias satisfagan tu RPO y sean restaurables es una afirmación tuya que exige evidencia tuya.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/019-modelo-de-responsabilidad-compartida-por-servicio/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa raci-de-controles en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye raci-de-controles para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una auditoría rechaza la copia de seguridad pese a existir instantáneas	Nunca se probó la restauración: el proveedor toma la copia, el cliente demuestra que sirve	Automatiza una restauración periódica con verificación de integridad y RTO medido.
El RPO real es de 24 h aunque el plan comprometa 15 min	Se asumió que existir copia equivale a cumplir el RPO	Activa recuperación a un instante y mide el RPO efectivo, no la frecuencia de la copia.
Un recurso nuevo nace sin cifrado ni réplica	Las funciones gestionadas se ofrecen, no se imponen, y no había control que lo exigiera	Convierte la comprobación en política como código que falle el despliegue.
Se presenta el informe de cumplimiento del proveedor como prueba propia	El informe cubre la parte del proveedor y enumera lo que deja al cliente	Lee la sección de controles complementarios del usuario: es tu lista de tareas.
Se hereda un control de una región o servicio que el informe no cubre	No se revisó el alcance ni el periodo del informe	Comprueba servicios, regiones y ventana temporal antes de dar por heredado un control.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué diferencia hay entre control heredado, compartido y propio, y qué obligación te queda en el primero?
2. En una base de datos gestionada, ¿qué actividad de la copia de seguridad nunca ejecuta el proveedor?
3. ¿Por qué la columna «Responde» de la matriz RACI es siempre el cliente, y qué consecuencia práctica tiene?
4. ¿Cuál es la sección más útil de un informe de cumplimiento del proveedor y por qué?
5. Un servicio soporta cifrado en reposo. ¿Qué te falta comprobar antes de darlo por heredado?

Referencias

- AWS (2024). Shared Responsibility Model — controles heredados, compartidos y propios del cliente.
<<https://aws.amazon.com/compliance/shared-responsibility-model/>>
- Microsoft (2024). Shared responsibility in the cloud — reparto por modelo de servicio con tabla de controles.
<<https://learn.microsoft.com/en-us/azure/security/fundamentals/shared-responsibility>>
- Cloud Security Alliance (2024). Cloud Controls Matrix — catálogo de controles con reparto proveedor/cliente.
<<https://cloudsecurityalliance.org/research/cloud-controls-matrix>>
- AICPA (2017). SOC 2 Trust Services Criteria — estructura del informe y controles complementarios del usuario.
<<https://www.aicpa-cima.com/resources/landing/system-and-organization-controls-soc-suite-of-services>>
- Dotson, C. (2023). Practical Cloud Security, 2.^a ed., cap. 1 — construir la matriz de responsabilidad servicio a servicio.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 019 Modelo de responsabilidad compartida por servicio

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.

2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega raci-de-contrôles con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

020 — TCO, costos variables, unit economics y FinOps

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

Propósito

Construir un caso de coste total defendible ante alguien que sabe de finanzas, no solo de infraestructura. La clase 011 dio la unidad de coste; aquí se añade lo que un TCO honesto incluye y casi ningún cálculo de migración recoge: personal, licencias, amortización pendiente, coste de salida y el valor temporal del dinero.

Resultados de aprendizaje

Al finalizar podrás:

1. Enumerar las partidas de un TCO que no aparecen en la factura y estimarlas con órdenes de magnitud.
2. Calcular el valor actual neto de una migración y explicar por qué comparar totales sin descontar engaña.
3. Distinguir coste hundido de coste evitable y por qué el hardware ya comprado no debe entrar en la decisión.
4. Construir un modelo de coste unitario que muestre si la escala mejora la economía o solo la traslada.
5. Presentar el caso con supuestos explícitos y análisis de sensibilidad sobre los dos que más pesan.

Conceptos centrales

Concepto	Comprensión verificable
TCO	Coste total de propiedad: todo lo que cuesta operar una capacidad durante su vida útil, no solo lo que se factura. Incluye personal, licencias, espacio, energía, riesgo y coste de oportunidad.
coste hundido	Gasto ya realizado e irre recuperable. Es irrelevante para decidir el futuro: incluirlo en la comparación es la falacia del coste hundido, y produce decisiones de mantener sistemas solo porque ya se pagaron.
valor actual neto	Suma de flujos futuros descontados a una tasa que refleja el coste del capital. Permite comparar alternativas cuyos gastos ocurren en momentos distintos, que es siempre el caso en una migración.
unit economics	Coste e ingreso por unidad de negocio. Es lo que revela si crecer mejora o empeora el margen, algo que el total absoluto oculta por completo.
coste de salida	Lo que costaría deshacer la decisión: egreso de datos, reescritura, formación y tiempo. Rara vez se calcula y es lo que convierte una decisión reversible en una irreversible.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    F["Factura del proveedor"] --> T["TCO"]
    P["Personal: operación,<br/>guardias, formación"] --> T
    L["Licencias y soporte"] --> T
    E["Espacio, energía,<br/>refrigeración"] --> T
    R["Riesgo: caídas,<br/>brechas, cumplimiento"] --> T
    S["Coste de salida<br/>si hay que deshacer"] --> T
    T --> VAN["descontar a<br/>coste de capital"]
    VAN --> D["Decisión comparable<br/>entre alternativas"]
    H["Hardware ya comprado"] -. "coste HUNDIDO:<br/>NO entra" .-> X[" "]
```

Desarrollo

1. Lo que la factura no incluye suele ser la mitad

Un TCO que solo suma la factura del proveedor subestima sistemáticamente ambos lados de la comparación. Las partidas ausentes, con órdenes de magnitud orientativos:

Partida	En centro de datos propio	En nube
Infraestructura	CAPEX amortizado a 3-5 años	Factura mensual
Personal de operación	1 persona por 100-200 servidores	Menor, pero no cero
Guardias	Rotación completa 24x7	Rotación completa 24x7
Licencias de virtualización	Significativas	Incluidas o distintas
Espacio, energía, refrigeración	Directo	Dentro del precio
Renovación de hardware	Ciclo de 4-5 años	No aplica
Sobreaprovisionamiento	Se compra para el pico	Se paga el consumo
Formación	Menor, tecnología estable	Alta y recurrente
Coste de salida	Bajo	Medio-alto

Dos filas merecen atención porque suelen decidir el resultado:

Personal. Es habitual que sea la mayor partida del centro de datos propio y la más ignorada. Una persona dedicada cuesta, con cargas, del orden de 40.000-90.000 USD anuales según el mercado; tres personas superan a mucha infraestructura. La nube reduce esa partida pero no la elimina: cambia parcheo de sistemas operativos por gestión de identidades, políticas y costes.

Formación. Es la que se olvida en la dirección contraria. Un equipo que domina VMware necesita meses para operar con soltura en un hiperescalador, y el coste no es solo el curso: es la productividad reducida durante la transición y los errores del periodo de aprendizaje.

2. El hardware ya comprado no entra en la decisión

La objeción más frecuente contra una migración es «acabamos de invertir 400.000 USD en servidores». Es la falacia del coste hundido: ese dinero ya se gastó y no se recupera decidas lo que decidas.

Lo que sí entra es lo evitable:

NO entra (hundido):	
compra del hardware ya pagada	400.000 USD
Sí entra (evitable si migras):	
soporte y mantenimiento restantes	38.000 USD/año
energía y espacio	22.000 USD/año
renovación prevista en 2028	180.000 USD
valor de reventa hoy	60.000 USD (ingreso)

La pregunta correcta no es «¿amortizamos lo comprado?» sino «a partir de hoy, ¿qué alternativa cuesta menos?». Si operar lo existente cuesta 60.000 USD anuales evitables y la nube cuesta 45.000, migrar ahorra 15.000 al año por mucho que se hayan pagado 400.000 el año pasado.

Hay dos matices legítimos que no son la falacia:

1. El valor de reventa sí entra, como ingreso, y desaparece con el tiempo: un servidor de tres años vale bastante menos que uno de uno.
2. Un contrato con penalización por cancelación anticipada es coste evitable de la opción de migrar, no coste hundido. Hay que incluirlo.

La distinción es la que permite tener una conversación financiera en vez de una emocional.

3. Descontar: 100.000 hoy no son 100.000 dentro de tres años

Una migración concentra gastos al principio y ahorros después. Comparar los totales sin descontar favorece artificialmente a la alternativa que gasta tarde.

$$VAN = \sum \text{flujo}_t / (1 + r)^t$$

Con una tasa de descuento del 10 % —el coste del capital de la organización— y un horizonte de 5 años:

año	flujo neto	factor	valor actual	
0	-250.000	1,000	-250.000	migración e implantación
1	+85.000	0,909	+77.273	
2	+92.000	0,826	+76.033	
3	+99.000	0,751	+74.380	
4	+106.000	0,683	+72.395	
5	+113.000	0,621	+70.156	

VAN = +120.237 USD				

Sin descontar, la suma sería +245.000. Con descuento, +120.237: el descuento se come la mitad del beneficio aparente, porque los ahorros llegan tarde y el gasto es inmediato.

Dos métricas complementarias que la dirección suele pedir:

periodo de recuperación simple: $250.000 / 85.000 \approx 2,9$ años
tasa interna de retorno (TIR): ≈ 24 %

Si la TIR supera el coste de capital, el proyecto crea valor. Con 24 % frente a 10 %, lo crea con holgura — siempre que los ahorros proyectados se materialicen, que es donde entra el análisis de sensibilidad.

4. Unit economics: la escala puede empeorar el margen

El total absoluto oculta si el negocio mejora al crecer. La descomposición mínima:

margen unitario = ingreso por unidad – coste por unidad

Con los datos de una plataforma real:

pedidos	620.000	980.000	1.740.000
coste de infraestructura	12.400	17.360	26.100
coste unitario	0,0200	0,0177	0,0150
ingreso por pedido	0,4200	0,4100	0,3900
margen unitario	0,4000	0,3923	0,3750

El coste unitario baja un 25 %: hay economía de escala. Pero el margen unitario también baja, porque el ingreso por pedido cae más rápido que el coste. La infraestructura mejora y el negocio empeora, y solo se ve con las dos series juntas.

El patrón contrario también existe y es más peligroso: un coste unitario que sube con el volumen indica que algo escala peor que linealmente —normalmente el término de coherencia de la clase O16, o transferencia entre zonas que crece con el cuadrado de los nodos—.

Tres preguntas que el modelo debe responder:

1. ¿El coste unitario baja, sube o se mantiene al crecer?
2. ¿Qué partida domina, y cambia esa dominancia con la escala?
3. ¿A qué volumen el coste unitario deja de bajar? Es el punto donde la arquitectura actual agota su economía.

5. Supuestos explícitos y sensibilidad sobre los dos que más pesan

Un TCO sin supuestos declarados no es defendible: cualquiera puede rehacerlo con otros números y llegar a la conclusión contraria. La disciplina mínima es listar los supuestos y probar cuánto aguanta la conclusión.

supuestos del modelo	
crecimiento de volumen	+18 % anual
reducción de personal	1,5 personas
utilización media	58 %
tasa de descuento	10 %
horizonte	5 años
inflación de precios cloud	0 % (los proveedores tienden a bajar)

El análisis de sensibilidad se hace sobre los dos supuestos con más peso, no sobre todos:

caso base	+120.237	
crecimiento +9 % en vez de +18 %	+71.400	
sin reducción de personal	-18.900	← cambia el signo
utilización 40 % en vez de 58 %	+148.000	(más ahorro por elasticidad)
descuento 15 %	+82.100	

El caso depende críticamente de un supuesto: la reducción de 1,5 personas. Si no ocurre, el proyecto destruye valor. Eso convierte una decisión técnica en una decisión organizativa, y es exactamente lo que la dirección necesita saber antes de aprobar.

Declararlo tiene una ventaja adicional: convierte el supuesto en un compromiso verificable. Si a los doce meses el personal no se ha reducido, no hay que esperar cinco años para saber que el caso no se cumple.

Ejemplo trabajado

CloudShop evalúa migrar su plataforma del centro de datos actual. El equipo de infraestructura presenta un ahorro del 60 % comparando la factura estimada con el gasto en hardware. Se rehace el cálculo completo.

Cálculo original presentado:

amortización de hardware	95.000 USD/año
factura cloud estimada	38.000 USD/año
"ahorro"	57.000 USD/año (60 %)

Errores detectados: incluye coste hundido, omite personal en ambos lados, ignora licencias, no descuenta y no contempla el coste de salida.

TCO reconstruido, solo con costes evitables desde hoy:

soporte y mantenimiento de hardware	38.000	0
energía, espacio, refrigeración	22.000	0
licencias de virtualización	31.000	0
personal de operación (3,0 → 1,5 FTE)	186.000	93.000
factura del proveedor	0	38.000
sobreaprovisionamiento (util. 30 %) ya incluido	0	evitado
formación primer año	0	24.000
	-----	-----
coste anual evitable	277.000	155.000
ahorro anual		122.000

Más del doble del ahorro que se presentaba —y por razones distintas: no viene del hardware sino del personal y las licencias.

Costes de una sola vez:

migración e implantación (6 meses, 2 personas)	140.000
doble ejecución durante la transición	46.000
reescritura de dos componentes acoplados	64.000
valor de reventa del hardware	-60.000

inversión neta inicial	190.000

Valor actual neto a 5 años, descontando al 10 %:

año 0	-190.000	× 1,000	= -190.000
año 1	+122.000	× 0,909	= +110.909
año 2	+122.000	× 0,826	= +100.826
año 3	+122.000	× 0,751	= +91.660
año 4	+122.000	× 0,683	= +83.327
año 5	+122.000	× 0,621	= +75.752

	VAN = +272.474 USD TIR ≈ 56 %		
	recuperación ≈ 1,6 años		

Sensibilidad sobre los dos supuestos dominantes:

caso base	+272.474	
personal NO baja de 3,0 a 1,5 FTE	-80.100	← el caso se cae
migración cuesta el doble (280.000)	+182.474	
factura cloud un 40 % mayor (53.200)	+214.800	
ambos adversos: sin reducción y +40 % factura	-137.700	

El proyecto depende de una sola variable: la reducción de personal. No de la factura, no del coste de migración. Esa es la conclusión que debe llevarse la dirección, y no aparecía en ninguna parte del cálculo original.

Se declara además el coste de salida, que nadie había estimado:

egreso de 42 TB a 0,09 USD/GB	3.780 USD
reescritura de adaptadores propietarios	~35.000 USD
reformación del equipo	~20.000 USD
coste de deshacer la decisión	~59.000 USD

Recomendación registrada: migrar, condicionado a un plan explícito de recolocación de 1,5 personas, con revisión a los 12 meses. Si a esa fecha el personal no se ha reducido, el caso se reevalúa con el coste de salida ya calculado sobre la mesa.

La diferencia entre el primer cálculo y este no es la cifra: es que el segundo dice de qué depende.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/020-tco-costos-variables-unit-economics-y-finops/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa caso-tco en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye caso-tco para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se rechaza una migración porque «acabamos de comprar el hardware»	Falacia del coste hundido: el gasto ya realizado no se recupera con ninguna decisión	Compara solo costes evitables desde hoy; incluye la reventa como ingreso, no la compra como coste.
El ahorro proyectado nunca aparece en las cuentas	El caso dependía de una reducción de personal que no se ejecutó	Declara los supuestos organizativos como compromisos verificables con fecha de revisión.
Dos alternativas parecen equivalentes y una es claramente peor	Se compararon totales sin descontar, favoreciendo a la que gasta más tarde	Calcula el VAN con la tasa de coste de capital de la organización.
El volumen crece, la factura crece y nadie sabe si eso es bueno	No hay unit economics: falta el coste por unidad junto al ingreso por unidad	Sigue las dos series juntas; el coste unitario puede bajar mientras el margen empeora.
Deshacer una decisión resulta imposible por su coste	El coste de salida nunca se estimó, así que la decisión era irreversible sin saberlo	Calcula egreso, reescritura y reformatación al decidir, no cuando quieras salir.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Se compraron servidores por 400.000 USD el año pasado. ¿Qué parte de esa cifra entra en la decisión de migrar y cuál no?
2. ¿Por qué comparar el total sin descontar favorece a la alternativa que concentra el gasto al final?
3. El coste unitario baja un 25 % y el margen unitario también baja. ¿Qué está ocurriendo y por qué el total lo oculta?
4. ¿Sobre qué supuestos conviene hacer el análisis de sensibilidad y cómo se eligen?
5. Nombra tres partidas de un TCO que no aparecen en ninguna factura y afectan a ambos lados de la comparación.

Referencias

- Storment, J. R. y Fuller, M. (2023). Cloud FinOps, 2.^a ed., caps. 4-6 — modelo de coste unitario y previsión.
- FinOps Foundation (2024). Unit Economics — definición y construcción de métricas por unidad de negocio. <<https://www.finops.org/framework/capabilities/unit-economics/>>
- Brealey, R., Myers, S. y Allen, F. (2020). Principles of Corporate Finance, 13.^a ed., caps. 2-6 — VAN, TIR y costes hundidos.
- Hohpe, G. (2020). Cloud Strategy, cap. sobre economía — por qué el ahorro de la nube rara vez viene de donde se anuncia.
- Barroso, L., Hölzle, U. y Ranganathan, P. (2018). The Datacenter as a Computer, 3.^a ed., cap. 6 — TCO de un centro de datos. <<https://doi.org/10.2200/S00874ED3V01Y201809CAC046>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 020 TCO, costos variables, unit economics y FinOps

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.

2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega caso-tco con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

021 — Well-Architected y atributos de calidad

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Usar los marcos Well-Architected como una lista de preguntas que fuerza a hacer explícitos los compromisos, no como una certificación que se aprueba. Su valor real está en los pilares que se contradicen entre sí: mejorar seguridad suele costar rendimiento, y mejorar coste suele costar fiabilidad. Esta clase enseña a nombrar ese intercambio en vez de fingir que no existe.

Resultados de aprendizaje

Al finalizar podrás:

1. Traducir un requisito de negocio a atributos de calidad medibles con escenario, estímulo y respuesta.
2. Identificar qué pares de pilares se oponen en una decisión concreta y cuantificar el intercambio.
3. Conducir una revisión que produzca riesgos priorizados y no una lista de buenas prácticas genéricas.
4. Distinguir un riesgo alto de uno medio por su impacto y su reversibilidad, no por su gravedad aparente.
5. Convertir los hallazgos en acciones con responsable, fecha y criterio de cierre verificable.

Conceptos centrales

Concepto	Comprensión verificable
atributo de calidad	Propiedad no funcional medible: latencia, disponibilidad, coste unitario, tiempo de recuperación. Se especifica con un escenario completo, porque «rápido» y «seguro» no son requisitos verificables.
pilar	Dimensión de análisis del marco: excelencia operativa, seguridad, fiabilidad, eficiencia de rendimiento, optimización de costes y sostenibilidad. Su utilidad es forzar a mirar las seis, incluidas las incómodas.
escenario de calidad	Formulación de seis partes —fuente, estímulo, artefacto, entorno, respuesta y medida— que convierte una aspiración en algo comprobable.
compromiso	Reconocimiento explícito de que mejorar un atributo empeora otro. Un diseño sin compromisos declarados no es equilibrado: es un diseño cuyos costes aún no se han descubierto.
riesgo alto	Hallazgo cuyo impacto es grave y cuya corrección posterior es cara o destructiva. La reversibilidad, no solo la gravedad, es lo que separa el riesgo alto del medio.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
R["Requisito de negocio"] --> E["Escenario de calidad<br/>fuente · estímulo · respuesta · medida"]
E --> P{"Revisión por los 6 pilares"}
P --> S["Seguridad"]
P --> F["Fiabilidad"]
P --> C["Costes"]
P --> D["Rendimiento"]
S <-->|"se oponen"| D
F <-->|"se oponen"| C
S --> H["Riesgos priorizados<br/>por impacto × reversibilidad"]
F --> H
C --> H
D --> H
H --> A["Acciones con responsable,<br/>fecha y criterio de cierre"]
```

Desarrollo

1. Un atributo sin escenario no es un requisito

«El sistema debe ser rápido y seguro» no se puede verificar, así que no se puede diseñar contra ello. La formulación de seis partes de Bass, Clements y Kazman lo convierte en algo comprobable:

Parte	Ejemplo
Fuente	Un usuario autenticado desde Chile
Estímulo	Solicita la página de un producto
Artefacto	El servicio de catálogo
Entorno	Operación normal, hora punta
Respuesta	Devuelve la página completa
Medida	p95 < 300 ms, medido en el cliente

La última fila es la que hace la diferencia. Y tiene dos detalles que se omiten sistemáticamente:

1. El percentil, porque como se vio en la clase 012 la media no la experimenta nadie.
2. El punto de medición. «300 ms en el servidor» y «300 ms en el cliente» difieren en todo el tramo de red, que en la clase 001 resultó ser el componente dominante. Un SLO medido en el sitio equivocado se cumple mientras los usuarios se quejan.

El mismo rigor aplica a los demás pilares:

disponibilidad	99,95 % mensual medido sobre peticiones con éxito, excluyendo mantenimiento anunciado con 72 h de antelación
coste	≤ 0,018 USD por pedido a volumen de 1 M/mes
recuperación	RTO 60 min y RPO 15 min, verificados trimestralmente
seguridad	ninguna credencial de larga vida en el camino de despliegue

Cada uno tiene número, unidad y método de verificación. Sin los tres, un pilar es una intención.

2. Los pilares se contradicen, y ahí está el trabajo

El error más común al usar el marco es tratarlo como una lista donde todo se puede maximizar. Los pares que se oponen, con el mecanismo concreto:

Par	Por qué chocan	Ejemplo medible
Seguridad ↔ Rendimiento	Cifrado, inspección y verificación añaden latencia	mTLS entre servicios: +3-8 ms por salto
Fiabilidad ↔ Coste	La redundancia se paga aunque no se use	3 zonas: +50 % de cómputo sobre 2
Rendimiento ↔ Coste	La capacidad libre absorbe varianza	p 0,70 en vez de 0,95: +36 % de instancias
Operación ↔ Rapidez	Las puertas de calidad frenan la entrega	Revisión obligatoria: +4 h de plazo medio
Sostenibilidad ↔ Rendimiento	La región limpia puede estar lejos	+40 ms de latencia por 8× menos CO ₂ e

Una revisión útil cuantifica estos intercambios en vez de declararlos. «Hay un trade-off entre seguridad y rendimiento» no ayuda a decidir; «mTLS cuesta 6 ms sobre un presupuesto de 300 y elimina la posibilidad de suplantación este-oeste» sí.

Y hay un pilar que se opone a casi todos: la simplicidad, que no figura en el marco pero debería. Cada mecanismo añadido —una caché, una cola, una región extra— mejora un atributo y empeora la capacidad del equipo de operar y diagnosticar el sistema. Ese coste no aparece en ninguna métrica hasta el primer incidente a las 3 de la madrugada.

3. Priorizar por impacto y reversibilidad

Los marcos clasifican hallazgos en alto, medio y bajo, y la clasificación suele hacerse solo por gravedad. Falta la segunda dimensión: cuánto cuesta arreglarlo después.

$$\text{riesgo} = \text{impacto} \times \text{probabilidad} \times (\text{coste de corregir tarde} / \text{coste de corregir ahora})$$

El tercer factor es el que reordena la lista:

Hallazgo	Impacto	Corregir ahora	Corregir en 2 años	Prioridad
Sin cifrado en tránsito	Alto	2 días	1 semana	Media
Esquema de datos sin particionar	Medio	1 semana	6 meses + migración	Alta
Sin alertas de coste	Medio	1 día	1 día	Baja
Identificadores secuenciales expuestos	Alto	3 días	Reescritura de clientes	Alta

La segunda fila es el patrón clave: un hallazgo de impacto medio con corrección casi irreversible pesa más que uno de impacto alto y arreglo trivial. Las decisiones de datos y de contratos públicos son las que más envejecen: cambiar un esquema con 2 TB en producción o un formato de identificador que ya consumen terceros no es lo mismo que activar una opción.

La regla operativa: decide pronto lo irreversible y tarde lo reversible. Un hallazgo reversible puede esperar a tener más información; uno irreversible hay que resolverlo mientras el coste sigue siendo bajo.

4. Una revisión que produce decisiones, no una lista

Una revisión Well-Architected mal conducida genera 60 recomendaciones genéricas que nadie ejecuta. Bien conducida produce entre 5 y 10 riesgos con dueño.

El guion que funciona:

1. Escenarios primero (30 min)
¿Cuáles son los 5 atributos de calidad con número y método de medición?
Sin esto, la revisión no tiene contra qué contrastar.
2. Recorrido por pilares (2-3 h)
Para cada pregunta: ¿lo hacemos? ¿cómo lo demostramos? ¿qué evidencia hay?
"Sí" sin evidencia se registra como "no verificado", que es distinto de "no".
3. Compromisos explícitos (45 min)
¿Qué decisión mejora un pilar a costa de otro? ¿Está declarada y aceptada?
4. Priorización (30 min)
Impacto × reversibilidad. Máximo 10 riesgos; si salen 40, la revisión está describiendo el estado, no priorizando.
5. Acciones (30 min)
Cada riesgo alto: responsable, fecha, criterio de cierre verificable.

El paso 2 tiene una regla que cambia el resultado: distinguir «no» de «no verificado». Un equipo suele responder «sí, tenemos copias de seguridad»; la pregunta siguiente —«¿cuándo se restauró por última vez?»— convierte muchos síes en no verificados, que es exactamente el hallazgo de la clase 019.

Y el paso 5 exige un criterio de cierre comprobable: «mejorar la seguridad de IAM» no se puede cerrar; «ninguna clave de acceso permanente activa, verificado por una consulta automática semanal» sí.

5. Los marcos coinciden más de lo que su vocabulario sugiere

Los tres grandes proveedores tienen su versión y las tres comparten estructura. Traducirlas evita aprender tres veces lo mismo y permite hacer revisiones multi-proveedor:

Pilar	AWS	Azure	Google Cloud
Operación	Operational Excellence	Operational Excellence	Operational excellence
Seguridad	Security	Security	Security, privacy, compliance
Fiabilidad	Reliability	Reliability	Reliability
Rendimiento	Performance Efficiency	Performance Efficiency	Performance optimization
Coste	Cost Optimization	Cost Optimization	Cost optimization
Sostenibilidad	Sustainability	—	Sustainability

Las diferencias reales son de énfasis, no de fondo: AWS incorporó sostenibilidad como pilar en 2021; Azure la trata dentro de otros; Google agrupa privacidad y cumplimiento con seguridad.

El límite honesto de los tres: están escritos por quien vende la plataforma. Ninguno pregunta si deberías estar en esa nube, si el bloqueo es aceptable o si la alternativa más barata es no migrar. Son excelentes para revisar cómo está construido algo y no sirven para decidir si construirlo.

Para esa segunda pregunta hacen falta los criterios de las clases 013, 015 y 020: definición, modelo de servicio y coste total. Un marco Well-Architected aplicado a una decisión que no debió tomarse produce un sistema impecablemente construido sobre una premisa equivocada.

Ejemplo trabajado

Revisión Well-Architected de la plataforma de pedidos de CloudShop antes de duplicar el volumen. Se sigue el guion y se cuantifican los compromisos.

Paso 1 — escenarios de calidad, con medida y punto de medición:

Q1 latencia p95 < 300 ms medido en el cliente, hora punta
Q2 disponibilidad 99,95 % mensual sobre peticiones con éxito
Q3 coste ≤ 0,018 USD/pedido a 1 M pedidos/mes
Q4 recuperación RTO 60 min · RPO 15 min, verificado cada trimestre
Q5 seguridad cero credenciales de larga vida en el despliegue

Paso 2 — recorrido, separando «no» de «no verificado»:

copias de seguridad automáticas	sí	✓ 7 instantáneas
restauración probada	"sí"	✗ NO VERIFICADO
despliegue sin secretos	sí	✓ OIDC (clase 018)
límites de tasa por cliente	no	—
particionado del esquema de pedidos	no	—
alertas sobre consumo de cuota	no	—

Paso 3 — compromisos, cuantificados:

mTLS entre servicios
seguridad: elimina suplantación este-oeste
rendimiento: +6 ms medidos sobre 3 saltos → 18 ms del presupuesto de 300 (6 %)
DECISIÓN: se adopta. 6 % del presupuesto por eliminar una clase entera de ataque.

tercera zona de disponibilidad
fiabilidad: de 99,995 % a 99,999 % → de 1,3 a 0,3 min/mes
coste: +50 % de cómputo = +58 USD/mes
DECISIÓN: NO. El requisito es 21,6 min/mes; ya se cumple con holgura.

utilización objetivo 0,70 en vez de 0,90
rendimiento: p95 de 91 ms en vez de ~300 ms (clase 011)
coste: +36 % de instancias
DECISIÓN: se adopta. Sin ella Q1 no se cumple.

Paso 4 — priorización por impacto × reversibilidad:

esquema de pedidos sin particionar	medio	1 sem	6 meses+migr	ALTA
------------------------------------	-------	-------	--------------	------

restauración nunca probada	alto	2 días	2 días	ALTA
sin límites de tasa por cliente	alto	3 días	3 días	MEDIA
sin alerta de cuota	medio	1 día	1 día	BAJA

El esquema sin particionar sube a alta pese a impacto medio: hoy la tabla tiene 180 GB y particionar cuesta una semana; con el volumen duplicado durante dos años serán 1,3 TB y la migración exigirá ventana de indisponibilidad. Es el único hallazgo cuya ventana de corrección barata se está cerrando.

La restauración no probada es alta por impacto puro: su coste de corrección no crece, pero el riesgo se materializa en cualquier momento.

Paso 5 — acciones con criterio de cierre:

R1	particionar pedidos por mes	Ana	15-09	pg_partman activo y consulta p95 < 40 ms
R2	restauración automatizada	Luis	22-08	informe trimestral con RT0 medido y recuento verificado
R3	límite de tasa por cliente	Ana	30-09	429 con Retry-After bajo prueba de carga
R4	alerta de cuota al 80 %	Luis	08-08	alarma disparada en simulacro

Resultado: 4 acciones con dueño y criterio comprobable, frente a las 34 recomendaciones genéricas que devolvió la herramienta automática. La diferencia la produjo el paso 1: sin los cinco escenarios con número, no había contra qué contrastar y todo parecía igual de importante.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/021-well-architected-y-atributos-de-calidad/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa revision-well-architected en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye revision-well-architected para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
La revisión produce 40 recomendaciones y no se ejecuta ninguna	Se describió el estado en vez de priorizar; faltaban escenarios contra los que contrastar	Empieza por 5 atributos con número y método de medición; limita la salida a 10 riesgos.
Un SLO de latencia se cumple mientras los usuarios se quejan	Se mide en el servidor y el tramo dominante es la red	Declara el punto de medición en el escenario; medir en el cliente cambia el número por completo.
Un hallazgo de impacto medio se pospone y dos años después cuesta una migración	Se priorizó solo por gravedad, ignorando la reversibilidad	Prioriza por impacto × coste de corregir tarde; decide pronto lo irreversible.
El equipo declara tener un control que en realidad nunca se ejerció	No se distinguió «sí» de «sí, pero no verificado»	Pide la evidencia en la misma pregunta; un sí sin evidencia se registra como no verificado.
Se construye impecablemente un sistema que no debió existir	El marco revisa cómo está hecho algo, no si debía hacerse	Decide primero con definición, modelo de servicio y TCO; el marco viene después.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Convierte «el sistema debe ser rápido» en un escenario de calidad completo, incluyendo el punto de medición.
2. Nombra dos pares de pilares que se opongan y cuantifica el intercambio con un número concreto.
3. ¿Por qué un hallazgo de impacto medio puede tener más prioridad que uno de impacto alto?
4. ¿Qué diferencia hay entre responder «no» y «no verificado» en una revisión, y cómo se distingue?
5. ¿Qué pregunta importante no responde ningún marco Well-Architected, y con qué criterios se responde?

Referencias

- AWS (2024). Well-Architected Framework — seis pilares, preguntas y proceso de revisión.
<<https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html>>
- Microsoft (2024). Azure Well-Architected Framework — pilares y compromisos entre ellos.
<<https://learn.microsoft.com/en-us/azure/well-architected/>>
- Google Cloud (2024). Architecture Framework — pilares y guía de diseño.
<<https://cloud.google.com/architecture/framework>>
- Bass, L., Clements, P. y Kazman, R. (2021). Software Architecture in Practice, 4.^a ed., caps. 3-4 — escenarios de atributos de calidad en seis partes.
- Richards, M. y Ford, N. (2020). Fundamentals of Software Architecture, cap. 2 — «todo en arquitectura es un compromiso».
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 021 Well-Architected y atributos de calidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.

2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega revision-well-architected con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

022 — Cloud Adoption Framework y modelo operativo

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Entender por qué las migraciones fracasan por causas organizativas y no técnicas, y qué modelo operativo hay que construir para que la plataforma no se convierta en un cuello de botella. La ley de Conway no es una curiosidad: es la razón por la que una arquitectura de microservicios impuesta a una organización de silos produce un monolito distribuido.

Resultados de aprendizaje

Al finalizar podrás:

1. Aplicar la ley de Conway en sentido inverso: diseñar equipos para obtener la arquitectura que se quiere.
2. Elegir entre los modelos operativos centralizado, descentralizado y de plataforma según tamaño y madurez.
3. Distinguir un equipo de plataforma que ofrece un producto de uno que se convierte en un servicio de tickets.
4. Secuenciar una adopción por olas con criterios de salida verificables en vez de por fechas.
5. Reconocer las señales tempranas de que el modelo operativo elegido está fallando.

Conceptos centrales

Concepto	Comprensión verificable
ley de Conway	Las organizaciones producen diseños que replican sus estructuras de comunicación. No es una tendencia evitable con disciplina: es una restricción que conviene usar deliberadamente.
carga cognitiva	Cantidad de conocimiento que un equipo debe sostener para operar lo suyo. Cuando supera su capacidad, la calidad cae sin que nadie sea negligente; es el límite real al tamaño del dominio de un equipo.
camino dorado	Trayecto recomendado y bien soportado para hacer algo común. No es obligatorio: compite por ser tan bueno que salirse resulte caro, y esa voluntariedad es lo que lo mantiene honesto.
equipo de plataforma	Equipo cuyo producto son las capacidades internas que otros consumen en autoservicio. Su métrica no es cuántos tickets cierra, sino cuántos deja de recibir.
ola de adopción	Grupo de cargas que se migran juntas con un criterio de salida definido. Avanzar por olas con criterio permite aprender; avanzar por fechas produce deuda que se descubre en la última ola.
equipo habilitador	Equipo que trabaja temporalmente con otros para elevar su capacidad y después se retira. Se distingue del de plataforma en que no deja un servicio permanente, sino conocimiento.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    subgraph mal["Organización en silos"]
        D1["Desarrollo"] -->|ticket| O1["Operaciones"]
        O1 -->|ticket| S1["Seguridad"]
    end
    subgraph bien["Equipos alineados al flujo"]
        E1["Equipo pedidos<br/>construye y opera"]
        E2["Equipo catálogo<br/>construye y opera"]
        P["Plataforma<br/>autoservicio"]
        E1 -->|consume| P
        E2 -->|consume| P
    end
    mal -->|produce| A1["Arquitectura acoplada:<br/>3 equipos = 3 fronteras"]
    bien -->|produce| A2["Servicios independientes<br/>con dueño claro"]
```

Desarrollo

1. La ley de Conway, usada al revés

Conway observó en 1968 que las organizaciones producen diseños que copian sus estructuras de comunicación. La formulación es descriptiva, pero su uso práctico es prescriptivo: si quieres una arquitectura concreta, organiza los equipos con esa forma.

El caso negativo se repite en todas las migraciones grandes:

```
organización: desarrollo → operaciones → seguridad (tres silos, tickets entre ellos)
se pide:      microservicios independientes
se obtiene:   servicios que no se pueden desplegar sin coordinar tres equipos
              = un monolito distribuido, con lo peor de ambos modelos
```

El resultado no es un fallo de ejecución: es exactamente lo que la estructura de comunicación permite construir. Ningún equipo puede desplegar sin los otros dos, así que la frontera real del sistema son los tres equipos, no los servicios.

La maniobra de Conway inversa consiste en cambiar primero la organización:

```
antes: 3 equipos por función → fronteras por capa técnica
después: N equipos por dominio → fronteras por dominio de negocio
```

Un equipo que construye y opera pedidos —incluyendo su base de datos, su despliegue y su guardia— puede producir un servicio independiente porque no necesita hablar con nadie para cambiarlo. Esa autonomía es la condición de la arquitectura, no su consecuencia.

Y el límite es honesto: reorganizar equipos es caro y lento. Por eso conviene hacerlo antes de comprometerse con una arquitectura, no después de descubrir que no encaja.

2. Tres modelos operativos y cuándo falla cada uno

Modelo	Cómo funciona	Escala hasta	Falla cuando
Centralizado	Un equipo aprovisiona todo por petición	5 equipos consumidores	Se convierte en cola: el tiempo de espera domina la entrega
Descentralizado	Cada equipo hace lo suyo sin capa común	10 equipos	Divergencia: 10 formas de desplegar, 10 posturas de seguridad
Plataforma	Capacidades en autoservicio con guardarraíles	Decenas	El equipo de plataforma se convierte en soporte de tickets

Los síntomas de que un modelo agotó su rango son medibles:

```
centralizado agotado: tiempo de espera para un entorno nuevo > 3 días
                     cola de peticiones creciente semana a semana
```

```
descentralizado agotado: n.º de formas distintas de desplegar ≈ n.º de equipos
                       un hallazgo de seguridad exige N correcciones distintas
```

```
plataforma degradada: > 40 % del tiempo del equipo en peticiones puntuales
                     el autoservicio existe y nadie lo usa
```

La tercera fila es la más frecuente y la más silenciosa: un equipo de plataforma que dedica la mayoría de su tiempo a resolver casos particulares ha vuelto al modelo centralizado con otro nombre. La causa suele ser que el camino dorado no cubre los casos reales, así que todos se salen de él y piden ayuda.

No hay un modelo correcto: hay un modelo adecuado al tamaño. Imponer plataforma a una organización de tres equipos crea más trabajo del que ahorra.

3. Carga cognitiva: el límite que no se negocia

Skelton y Pais sitúan la carga cognitiva como la restricción central del diseño de equipos. Un equipo tiene una capacidad finita de conocimiento sostenido, y cuando el dominio la supera la calidad cae sin que nadie sea negligente.

Los tres tipos, y qué hacer con cada uno:

Tipo	Qué es	Acción
Intrínseca	Complejidad del dominio: reglas de negocio	Formación; no se puede eliminar
Extrínseca	Cómo desplegar, configurar redes, obtener permisos	Eliminar con plataforma
Pertinente	Diseñar bien lo propio	Proteger: es donde está el valor

El objetivo de una plataforma es reducir la extrínseca para liberar capacidad para la pertinente. Medirlo es posible:

antes de la plataforma	
crear un entorno nuevo	14 pasos, 3 equipos, 4 días
conocimiento necesario	redes, IAM, DNS, certificados, CI

después	
crear un entorno nuevo	1 comando, 12 minutos
conocimiento necesario	el nombre del entorno

Si un equipo de producto necesita entender redes virtuales, políticas de identidad y emisión de certificados para publicar un servicio, esa carga extrínseca está consumiendo la capacidad que debería ir al dominio. No es que sean malos ingenieros: es que el sistema les pide sostener demasiado.

Y el criterio para dimensionar un dominio: si el equipo no puede explicar todo lo que opera en una pizarra durante 15 minutos, el dominio es demasiado grande.

4. Camino dorado: voluntario y bueno, o no funciona

Un camino dorado es el trayecto recomendado para hacer algo común. Su propiedad definitoria es que es voluntario, y esa voluntariedad es lo que lo mantiene honesto: si nadie lo usa, es que no es bueno, y esa señal es valiosa.

Qué lo hace funcionar:

1. Resuelve el 80 % de los casos, no el 100 %.
Perseguir el 100 % produce una plataforma que tarda años y no llega.
2. Trae seguridad y cumplimiento incorporados.
Usarlo debe ser más fácil QUE saltárselo, y además deja el sistema conforme.
3. Permite salirse, con coste explícito.
Quien se sale asume la operación completa: su propia guardia, su propio parcheo.
4. Se mide por adopción, no por mandato.
Si hay que obligar, el camino no es lo bastante bueno.

El punto 3 es el que evita el fracaso más común. Prohibir salirse convierte la plataforma en un obstáculo y genera infraestructura en la sombra —que es peor, porque nadie la ve—. Permitir salirse con el coste operativo explícito hace que la mayoría elija el camino por interés propio.

Métrica de salud de una plataforma, en una línea:

adopción = equipos que usan el camino dorado / equipos totales

Por debajo del 60 % hay que preguntar a los que no lo usan por qué, no insistir. La respuesta suele ser concreta: no soporta un tipo de carga, la latencia de aprovisionamiento es alta, o el equipo perdió algo que antes controlaba.

5. Olas con criterio de salida, no con fechas

Una adopción por fechas produce que la última ola herede toda la deuda de las anteriores. Por olas con criterio de salida verificable, cada una corrige lo aprendido:

Ola 0 · Fundación cuentas, identidad federada, red, registro de auditoría
salida: una carga trivial desplegada extremo a extremo sin secretos permanentes

Ola 1 · Piloto (1-2 cargas, sin criticidad)
salida: RTO y RPO medidos; coste unitario conocido; runbook probado por alguien ajeno al equipo que lo escribió

Ola 2 · Camino dorado (3-5 cargas similares)
salida: la tercera carga se despliega sin intervención de la plataforma y en menos de la mitad de tiempo que la primera

Ola 3 · Escala
salida: adopción > 60 %; ningún equipo bloqueado esperando a la plataforma

Ola 4 · Cargas difíciles
las que exigen reescritura o tienen dependencias legadas

El criterio de la ola 2 es el más revelador: si la tercera carga sigue necesitando ayuda manual, el camino dorado no existe todavía y escalar solo multiplicará el trabajo manual.

Y la ola 4 va al final por una razón deliberada: las cargas difíciles consumen mucho tiempo y enseñan poco reutilizable. Empezar por ellas —tentador, porque suelen ser las más visibles— agota el presupuesto político antes de tener nada que mostrar.

La señal de que hay que parar y corregir, en cualquier ola: el tiempo por carga no baja. Si migrar la carga número 8 cuesta lo mismo que la número 2, no se está construyendo capacidad reutilizable; se está haciendo el mismo trabajo ocho veces.

Ejemplo trabajado

CloudShop lleva ocho meses migrando. Han pasado 6 de 40 cargas y el equipo de plataforma está saturado. Se diagnostica el modelo operativo antes de pedir más gente.

Mediciones del estado actual:

cargas migradas	6 de 40	
tiempo por carga: nº 1	6 semanas	
tiempo por carga: nº 6	5,5 semanas	← no baja
tickets al equipo de plataforma/semana	47	
tiempo del equipo en tickets	68 %	
formas distintas de desplegar	6	← una por carga
adopción del camino dorado	0 % (no existe)	

Dos señales de agotamiento a la vez. El tiempo por carga no baja: no se está construyendo capacidad reutilizable. Y el 68 % del tiempo en tickets significa que la «plataforma» es un equipo centralizado con otro nombre.

Se revisa la estructura organizativa:

Desarrollo (4 equipos) ■■■ticket■■■ Plataforma (5 personas) ■■■ticket■■■ Seguridad (2)

Tres silos con tickets entre ellos. Por la ley de Conway, la arquitectura resultante replicará esa estructura: ningún servicio podrá desplegarse sin coordinar tres equipos, que es exactamente lo que se observa en las 6 semanas por carga.

Desglose de los 47 tickets semanales:

crear entorno o cuenta	14	sí
permisos de IAM	12	sí, con plantillas
reglas de red y DNS	9	sí
certificados	6	sí
casos genuinamente nuevos	6	no

41 de 47 son repetición. No hace falta más gente: hace falta convertir esos cuatro tipos en autoservicio.

Intervención, en dos frentes a la vez:

ORGANIZATIVO (Conway inverso)

seguridad deja de ser puerta y pasa a equipo habilitador:
escribe las políticas como código, no revisa cada cambio
los 4 equipos de desarrollo asumen la guardia de lo suyo

PLATAFORMA (camino dorado del 80 %)

1 comando crea entorno + cuenta + red + rol federado + certificado
cubre servicios HTTP sin estado: 31 de las 34 cargas restantes
las 3 que no encajan se salen, con su operación explícitamente propia

Resultado seis meses después:

tiempo por carga (nº 20)	5,5 sem	4 días
tickets/semana	47	9
tiempo del equipo en tickets	68 %	18 %
formas de desplegar	6	1 + 3 excepciones
adopción del camino dorado	0 %	84 %
cargas migradas	6	27

Se migraron 21 cargas en seis meses frente a 6 en ocho, con el mismo número de personas. El cuello no era capacidad: era que cada carga se resolvía a mano porque la estructura organizativa lo exigía.

Límite declarado: las 3 cargas fuera del camino dorado —dos con dependencias legadas y una con requisitos de latencia especiales— siguen consumiendo tiempo desproporcionado. Se dejan deliberadamente para el final, y sus equipos asumen su operación completa. Intentar que el camino dorado las cubriera habría retrasado el 84 % restante.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/022-cloud-adoption-framework-y-modelo-operativo/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa backlog-de-adopcion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye backlog-de-adopcion para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se adoptan microservicios y ninguno se puede desplegar de forma independiente	La organización sigue en silos funcionales: la arquitectura replica la comunicación	Reorganiza por dominio antes de comprometerte con la arquitectura; Conway no se evita con disciplina.
El equipo de plataforma dedica más de la mitad del tiempo a tickets	El camino dorado no cubre los casos reales, así que todos se salen y piden ayuda	Clasifica los tickets; si más del 80 % es repetición, conviértelos en autoservicio antes de contratar.
Migrar la carga número 8 cuesta lo mismo que la número 2	No se está construyendo capacidad reutilizable: se repite el mismo trabajo	Fija criterios de salida por ola; si el tiempo por carga no baja, para y construye antes de seguir.
Existe una plataforma en autoservicio y los equipos no la usan	No resuelve sus casos o les quitó control sin compensación	Pregunta a quienes no la usan; por debajo del 60 % de adopción el problema es el producto, no la disciplina.
Se prohíbe salirse del camino dorado y aparece infraestructura en la sombra	La obligatoriedad empuja fuera del radar en vez de fuera del camino	Permite salirse con el coste operativo explícito: guardia y parcheo propios.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué arquitectura produce una organización con tres silos funcionales que pide microservicios, y por qué?
2. ¿Qué tres señales medibles indican que un modelo operativo agotó su rango de tamaño?
3. ¿Cuál de los tres tipos de carga cognitiva debe eliminar una plataforma y cuál debe proteger?
4. ¿Por qué el camino dorado debe ser voluntario, y qué pasa si se hace obligatorio?
5. ¿Por qué las cargas más difíciles se dejan para la última ola en vez de atacarlas primero?

Referencias

- Conway, M. (1968). How Do Committees Invent?. Datamation 14(5), 28-31.
<<https://www.melconway.com/Home/CommitteesPaper.html>>
- Skelton, M. y Pais, M. (2019). Team Topologies — cuatro tipos de equipo, carga cognitiva y maniobra de Conway inversa.
- Forsgren, N., Humble, J. y Kim, G. (2018). Accelerate — evidencia sobre estructura organizativa y rendimiento de entrega.
- Microsoft (2024). Cloud Adoption Framework — fases de estrategia, planificación, preparación y adopción.
<<https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/>>
- AWS (2024). Cloud Adoption Framework — seis perspectivas, incluidas las de personas y gobierno.
<<https://aws.amazon.com/cloud-adoption-framework/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 022 Cloud Adoption Framework y modelo operativo

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega backlog-de-adopcion con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

023 — Descubrimiento y clasificación de workloads

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 4 Laboratorio: migration · Estado: EXECUTABLECORE

Propósito

Levantar un inventario de cargas que sirva para decidir, no para archivar. La mayoría de migraciones empieza con una hoja de cálculo de servidores y fracasa porque los servidores no son la unidad de decisión: lo son las cargas, con sus dependencias, sus datos y sus restricciones. Esta clase produce el insumo del ADR de la clase 024.

Resultados de aprendizaje

Al finalizar podrás:

1. Descubrir dependencias reales por observación de tráfico en vez de por documentación o entrevistas.
2. Clasificar cada carga en una de las siete estrategias de migración con un criterio explícito.
3. Detectar los grupos de cargas que deben moverse juntas y por qué partarlos rompe el sistema.
4. Priorizar por valor y dificultad, situando cada carga en un cuadrante con consecuencia clara.
5. Reconocer los datos que hacen inviable una estrategia antes de comprometerse con ella.

Conceptos centrales

Concepto	Comprensión verificable
carga de trabajo	Conjunto de recursos y código que entrega una capacidad de negocio identificable. Es la unidad de decisión de una migración: los servidores son un detalle de implementación.
grupo de movimiento	Conjunto de cargas tan acopladas que deben migrarse a la vez. Determinarlo mal produce llamadas que cruzan la frontera con latencia y coste inesperados.
descubrimiento pasivo	Identificación de dependencias observando el tráfico real durante un periodo representativo, en lugar de preguntar. Encuentra lo que nadie recuerda y lo que nadie quiere admitir.
gravedad de los datos	Tendencia de las aplicaciones a permanecer cerca de los datos que consumen. Cuanto mayor es el volumen, más caro es moverlo y más atrae a todo lo demás.
las 7 R	Estrategias de migración: retirar, retener, reubicar, rehospedar, replataformar, recomprar y rearquitecturar. Ordenadas de menor a mayor coste y beneficio potencial.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
I["Inventario de cargas"] --> D["Descubrimiento pasivo<br/>30 días de tráfico"]
D --> G["Grupos de movimiento<br/>por acoplamiento observado"]
G --> C["Por cada carga"]
C -->|"sin uso"| R1["Retirar"]
C -->|"restricción legal<br/>o técnica"| R2["Retener"]
C -->|"cambio mínimo"| R3["Rehospedar"]
C -->|"ajuste al gestionado"| R4["Replataformar"]
C -->|"existe SaaS"| R5["Recomprar"]
C -->|"alto valor y<br/>mal encaje"| R6["Rearquitecturar"]
R1 --> P["Priorización<br/>valor × dificultad"]
R3 --> P
R4 --> P
```

Desarrollo

1. La unidad de decisión es la carga, no el servidor

Un inventario de servidores responde «qué tenemos» y no responde ninguna de las preguntas que importan: qué depende de qué, qué se puede mover solo, qué se rompe si se parte.

El registro mínimo por carga, con lo que cada campo decide:

Campo	Decide
Capacidad de negocio que entrega	Si merece migrarse o retirarse
Dependencias entrantes y salientes	Con qué debe moverse
Volumen y clasificación del dato	Coste de traslado y restricciones legales
Perfil de tráfico	Modelo de servicio adecuado (clase 015)
Criticidad y RTO/RPO exigidos	Orden en las olas
Dueño técnico y de negocio	Quién decide y quién acepta el riesgo
Licencias y su portabilidad	Coste oculto que puede invertir el caso

La penúltima fila es la que más se olvida y la que más incidentes causa: una carga sin dueño de negocio identificable no se puede migrar, porque nadie puede aprobar la ventana de indisponibilidad ni aceptar el riesgo. Encontrar esas cargas huérfanas es uno de los resultados más valiosos del inventario, y a menudo revela candidatas a retirada.

La última fila puede invertir un caso de negocio completo: una licencia por núcleo físico que no se puede trasladar, o que en la nube se cuenta de otra forma, convierte una migración rentable en ruinosa. Hay que comprobarlo antes, no después.

2. Descubrir dependencias observando, no preguntando

Preguntar al equipo produce un mapa incompleto por tres razones que no son mala fe: nadie conoce el sistema entero, las integraciones antiguas se olvidan, y algunas conexiones las creó alguien que ya no está.

El descubrimiento pasivo observa el tráfico real:

```
# Conexiones establecidas con su proceso, agrupadas por destino
$ ss -tnp state established \
  | awk 'NR>1 {split($4,l,"."); split($5,r,"."); print r[1]":"r[2], $6}' \
  | sort | uniq -c | sort -rn | head -20
```

Un periodo de al menos 30 días es imprescindible, y la razón es concreta: los procesos de cierre mensual, las conciliaciones y los informes periódicos solo aparecen una vez al mes. Un descubrimiento de una semana pierde exactamente las dependencias que más duelen al fallar, porque son las que nadie ejercita a diario.

Lo que el descubrimiento encuentra y las entrevistas no:

- servicios que nadie sabía que seguían activos
- clientes externos consumiendo una API que se creía interna
- una tarea programada que escribe en una base de datos de otro sistema
- conexiones salientes a proveedores olvidados
- tráfico hacia una IP sin dueño identificable

El último caso es habitual y hay que resolverlo antes de migrar: una dependencia sin dueño es una llamada que dejará de funcionar sin que nadie sepa qué se rompió.

Y una advertencia sobre el método: observar tráfico en producción puede capturar datos sensibles. Hay que limitar la captura a metadatos de conexión —origen, destino, puerto, volumen— y nunca a contenido.

3. Las siete estrategias, con su criterio de elección

Estrategia	Qué es	Cuándo	Coste	Beneficio
Retirar	Apagar	Sin uso medido en 90 días	Mínimo	Alto: elimina coste y riesgo
Retener	Dejar donde está	Restricción legal, técnica o retirada próxima	Nulo	Ninguno, y es correcto
Reubicar	Mover la VM tal cual al hipervisor gestionado	Se necesita salir del centro de datos ya	Bajo	Bajo
Rehospedar	Mover sin cambios	Plazo corto, carga estándar	Bajo	Bajo-medio
Replataformar	Ajustes para usar servicios gestionados	Base de datos o cola sustituibles	Medio	Medio-alto
Recomprar	Sustituir por SaaS	Existe producto y no es diferencial	Medio	Alto si encaja
Rearquitecturar	Rediseñar	Alto valor y mal encaje con el modelo actual	Alto	Alto, con riesgo

Dos errores de selección que se repiten:

Rearquitecturar todo. Es la estrategia más cara y más lenta, y solo se justifica cuando la carga es de alto valor y su arquitectura actual impide obtener el beneficio. Aplicarla a una aplicación interna que usan 40 personas consume el presupuesto que necesitaban veinte cargas más rentables.

No retirar nada. Es la estrategia con mejor relación coste-beneficio y la que menos se usa, porque exige admitir que algo ya no sirve. Un inventario típico encuentra entre un 10 % y un 30 % de cargas sin uso real; migrarlas cuesta dinero y no aporta nada.

La comprobación de retirada debe ser por medición, no por opinión: sin conexiones entrantes durante 90 días, incluido un cierre mensual completo.

4. Grupos de movimiento: qué se rompe al partir

Dos cargas que intercambian mucho tráfico con baja tolerancia a latencia forman un grupo y deben moverse juntas. Separarlas produce un fallo silencioso: todo funciona en las pruebas y se degrada en producción.

La aritmética de partir un grupo mal:

carga A llama a carga B: 45 veces por petición de usuario
latencia en el mismo centro de datos: $0,4 \text{ ms} \rightarrow 45 \times 0,4 = 18 \text{ ms}$
latencia si B se queda y A migra: $28 \text{ ms} \rightarrow 45 \times 28 = 1.260 \text{ ms}$

De 18 ms a 1,26 segundos. El patrón de 45 llamadas por petición —una consulta dentro de un bucle— es invisible mientras la latencia es de 0,4 ms y catastrófico en cuanto sube. La migración no causó el problema de diseño: lo reveló.

Criterio para formar grupos, a partir del descubrimiento:

forman grupo si:
 llamadas por petición > 10, o
 volumen entre ellas > 1 GB/día, o
 comparten transacciones con consistencia fuerte, o
 comparten la misma base de datos

pueden separarse si:
 la comunicación es asíncrona y tolera segundos
 el volumen es bajo
 hay contrato estable entre ambas

La tercera condición es la más rígida: una transacción que abarca dos sistemas no se puede partir entre nubes sin cambiar el modelo de consistencia, que es rearquitecturar y no rehospedar. Descubrirlo durante la migración es descubrirlo tarde.

Y existe una salida intermedia legítima: partir el grupo y aceptar la degradación temporal durante una ventana corta, con las dos partes reunidas al final de la ola. Lo que no funciona es partirlo indefinidamente.

5. La gravedad de los datos decide más que la aplicación

Cuanto mayor es el volumen de datos, más caro es moverlo y más atrae hacia sí a todo lo que lo consume. Esa es la razón por la que las migraciones se planifican desde los datos hacia arriba y no al revés.

El tiempo de traslado por red tiene un suelo aritmético:

$$\text{tiempo} = \text{volumen} / \text{ancho_de_banda_efectivo}$$

40 TB por un enlace de 1 Gbit/s al 70 % de eficiencia:

$$40.000 \text{ GB} \times 8 \text{ bit} / (1 \text{ Gbit/s} \times 0,70) = 457.143 \text{ s} \approx 5,3 \text{ días}$$

Cinco días de transferencia continua, durante los cuales los datos siguen cambiando. Por eso existen dos mecanismos que hay que considerar desde el inventario:

- Transferencia física: dispositivos que el proveedor envía, se cargan y se devuelven. Para decenas de TB suele ser más rápido, y el cálculo es sencillo: si el envío tarda menos que la red, gana.
- Replicación continua con corte final: se replica en caliente durante días o semanas y se hace un corte corto al final. Es lo que permite ventanas de indisponibilidad de minutos en lugar de días.

Y una restricción que no es técnica: los datos con residencia obligatoria no se mueven, por mucho ancho de banda que haya. Detectarlo en el inventario es lo que evita diseñar una arquitectura completa que después resulta ilegal.

La consecuencia práctica sobre el orden de las olas: una carga cuyos datos no se pueden mover fija la posición de todo su grupo. No es la aplicación la que decide dónde va: es el dato.

Ejemplo trabajado

CloudShop inventaría sus 40 cargas antes de decidir el plan de migración. El resultado reordena por completo el plan inicial, que era «mover todo tal cual en seis meses».

Descubrimiento pasivo, 34 días para cubrir un cierre mensual completo:

cargas declaradas por los equipos	40
cargas detectadas con tráfico	43
→ 3 servicios activos que nadie declaró	
cargas declaradas SIN tráfico entrante	7
→ candidatas a retirada	
dependencias declaradas	58
dependencias observadas	91
→ 33 dependencias que nadie recordaba (36 %)	

Las 33 dependencias no documentadas son el hallazgo que justifica el método: una de cada tres conexiones reales no estaba en ningún diagrama.

Clasificación de las 43 cargas:

Retirar	7	sin tráfico entrante en 34 días, incluido cierre
Retener	3	1 por residencia de datos, 2 por retirada en 2027
Rehospedar	18	estándar, plazo corto
Replataformar	9	base de datos a gestionada, cola a gestionada
Recomprar	4	correo interno, gestor documental, wiki, tickets
Rearquitecturar	2	catálogo y pedidos: alto valor y mal encaje

Las 7 retiradas ahorran 41.000 USD anuales sin migrar nada. Es el mejor retorno del proyecto entero y estuvo disponible desde el primer día.

Grupos de movimiento detectados:

G1	pedidos + inventario + pagos	transacciones compartidas	→ INSEPARABLE
G2	catálogo + búsqueda	820 MB/día entre ellos	→ juntos
G3	informes + almacén de datos	lote nocturno	→ separables
G4	15 cargas independientes	sin acoplamiento	→ una a una

Se detecta a tiempo un error del plan original, que ponía inventario en la ola 1 y pedidos en la ola 3:

llamadas de pedidos a inventario por transacción:	45
latencia actual (mismo centro de datos):	0,4 ms → 18 ms totales
latencia si se separan (nube ↔ centro de datos):	28 ms → 1.260 ms
presupuesto de la petición:	300 ms

Habrían roto el SLO por un factor de 4 durante los dos meses entre olas. El plan se corrige: G1 se mueve completo en una sola ventana.

Gravedad de los datos sobre las dos cargas mayores:

almacén de datos históricos 38 TB residencia obligatoria en Chile
→ RETENER; fija la posición de los informes que lo consumen
base de datos de pedidos 2,4 TB sin restricción
→ por red al 70 % de 1 Gbit/s: 7,6 h
→ con replicación continua + corte: ventana de 12 min

Priorización final por valor y dificultad:

7 retiradas	alto	mínima	0	← inmediato
4 recompras SaaS	alto	baja	1	
15 cargas independientes	medio	baja	1-2	
G2 catálogo + búsqueda	alto	media	2	
G1 pedidos + inventario + pagos	alto	ALTA	3	← una ventana
2 rearquitecturas	alto	ALTA	4	
G3 informes	bajo	alta	4	← atado al dato retenido

Cambio respecto al plan inicial: de «40 cargas en 6 meses» a «7 retiradas esta semana, 19 cargas fáciles en 3 meses, y las 3 difíciles con su ventana propia». El inventario no retrasó el proyecto: evitó que la ola 3 rompiera producción y encontró 41.000 USD anuales de ahorro inmediato.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/023-descubrimiento-y-clasificacion-de-workloads/lab.py
```

El laboratorio selecciona el motor de práctica migration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa inventario-de-workloads en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un inventario, dependencias, riesgo y oleadas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye inventario-de-workloads para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Aparecen dependencias desconocidas durante la migración	El mapa se construyó por entrevistas; nadie conoce el sistema completo	Descubrimiento pasivo de al menos 30 días para capturar los procesos mensuales.
Una carga migrada funciona en pruebas y se degrada en producción	Se partió un grupo de movimiento con muchas llamadas por petición	Agrupar por llamadas por petición y volumen; una transacción compartida es inseparable.
Se migran cargas que nadie usa	No se midió el uso real antes de decidir la estrategia	Retira lo que no tenga tráfico entrante en 90 días; es el mayor retorno del proyecto.
El caso de negocio se invierte al llegar la factura de licencias	No se comprobó la portabilidad de las licencias ni cómo se cuentan en la nube	Incluye licencias en el inventario y verifica su modelo de conteo antes de decidir.
Una arquitectura completa resulta inviable por residencia de datos	La restricción legal se descubrió después de diseñar	Clasifica los datos en el inventario; un dato que no se mueve fija la posición de todo su grupo.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué el descubrimiento pasivo necesita al menos 30 días, y qué dependencias se pierden con una semana?
2. Dos cargas hacen 45 llamadas por petición a 0,4 ms. ¿Qué latencia total tendrán si se separan a 28 ms y qué implica para un SLO de 300 ms?
3. ¿Cuál de las siete estrategias tiene mejor relación coste-beneficio y por qué casi nunca se usa?
4. ¿Qué condición hace que dos cargas sean inseparables incluso con poco tráfico entre ellas?
5. 40 TB por un enlace de 1 Gbit/s al 70 % de eficiencia: ¿cuánto tarda y qué dos alternativas existen?

Referencias

- AWS (2024). Migration strategies: the 7 Rs — criterios de elección por estrategia.
<<https://docs.aws.amazon.com/prescriptive-guidance/latest/large-migration-guide/migration-strategies.html>>
- Microsoft (2024). Cloud Adoption Framework: assess workloads — inventario, dependencias y clasificación.
<<https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/plan/discover>>
- McCrory, D. (2010). Data Gravity in the Clouds — por qué los datos atraen a las aplicaciones.
<<https://datagravitas.com/2010/12/07/data-gravity-in-the-clouds/>>
- Google Cloud (2024). Migration to Google Cloud: assess and discover workloads — descubrimiento y agrupación.
<<https://cloud.google.com/architecture/migration-to-gcp-getting-started>>
- Newman, S. (2019). Monolith to Microservices, caps. 2-3 — identificar fronteras y grupos acoplados antes de partir.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 023 Descubrimiento y clasificación de workloads

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.

2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega inventario-de-workloads con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

024 — Proyecto: decisión de migración sustentada con ADR

Parte: 01 — Principios, estrategia y adopción cloud Nivel: inicial-intermedio · Horas estimadas: 8 Laboratorio: capstone
· Estado: EXECUTABLECORE

Propósito

Cerrar la parte produciendo el artefacto que la sostiene: un registro de decisión de arquitectura que integra todo lo anterior —definición, topología, modelo de servicio, identidad, responsabilidad, coste y capacidad— en un documento que alguien pueda cuestionar dentro de tres años. Un ADR no documenta lo decidido: documenta por qué, y sobre todo qué se descartó y bajo qué condición habría que revisarlo.

Resultados de aprendizaje

Al finalizar podrás:

1. Escribir un ADR con contexto, opciones evaluadas, decisión, consecuencias y criterio de revisión.
2. Justificar el descarte de al menos una alternativa con datos y no con preferencia.
3. Declarar las consecuencias negativas de la opción elegida con la misma claridad que las positivas.
4. Definir la condición observable que obligaría a reabrir la decisión.
5. Distinguir una decisión reversible de una que no lo es, y ajustar el rigor del proceso a esa diferencia.

Conceptos centrales

Concepto	Comprensión verificable
ADR	Documento breve e inmutable que registra una decisión de arquitectura con su contexto y consecuencias. Se numera, se versiona junto al código y no se edita: se sustituye por otro que lo supersedes.
estado del ADR	Propuesto, aceptado, rechazado, obsoleto o superseded. La cadena de estados es lo que permite reconstruir la evolución del razonamiento, no solo el resultado final.
decisión irreversible	Aquella cuyo coste de deshacer es un múltiplo del de tomarla. Merece más análisis, más consenso y una condición de revisión explícita; las reversibles merecen lo contrario.
consecuencia	Lo que la decisión hace cierto a partir de ahora, tanto favorable como desfavorable. Un ADR sin consecuencias negativas no fue escrito con honestidad.
condición de revisión	Hecho observable y medible que obligaría a reabrir la decisión. Convierte un documento estático en un compromiso vivo.

Modelo mental

La nube es un modelo operativo de acceso bajo demanda, no un lugar mágico ni el centro de datos de otra persona.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
C["Contexto: fuerzas y restricciones"] --> O["Opciones evaluadas<br/>con datos comparables"]
O --> D["Decisión y su razón"]
D --> CP["Consecuencias positivas"]
D --> CN["Consecuencias NEGATIVAS<br/>lo que empeora"]
D --> CR["Condición de revisión<br/>hecho observable"]
CR -->|"se cumple"| N["Nuevo ADR que<br/>supersede a este"]
N -->|"el original NO se edita"| D
```


Desarrollo

1. Un ADR registra el razonamiento, no el resultado

Dentro de tres años nadie recordará por qué se eligió aquello, y la persona que lo decidió probablemente ya no esté. Lo que se pierde no es la decisión —esa está en el código— sino las restricciones que la hicieron razonable.

Sin ese registro ocurren dos patologías simétricas:

1. Se revierte una buena decisión porque su motivo ya no es visible. Alguien ve una elección que hoy parece subóptima y la cambia, sin saber que respondía a una restricción que sigue vigente.
2. Se mantiene una mala decisión porque nadie sabe si aún aplica. La restricción desapareció hace dos años y el diseño sigue pagándola.

La estructura mínima, en el formato de Nygard:

```
# ADR-0012: Base de datos gestionada multi-zona para pedidos

**Estado:** Aceptado · 2026-08-01
**Decide:** equipo de plataforma · **Acepta el riesgo:** dirección de tecnología

## Contexto
[Las fuerzas en juego, con números. No la solución.]

## Opciones consideradas
[Al menos dos alternativas reales, comparadas con los mismos criterios.]

## Decisión
[Qué se elige y por qué, ligado a los datos del contexto.]

## Consecuencias
[Positivas y NEGATIVAS. Sin las segundas no es honesto.]

## Condición de revisión
[Hecho observable que obligaría a reabrir esto.]
```

Dos reglas que lo mantienen útil: vive junto al código, no en una wiki que se abandona; y no se edita. Si la decisión cambia, se escribe otro ADR que la supere y se enlazan. El historial de razonamiento es el valor, y editar lo destruye.

2. El contexto va con números y sin solución dentro

El error más común es escribir la solución en el contexto: «Necesitamos una base de datos gestionada multi-zona porque...». Eso no es contexto, es la decisión disfrazada, y elimina la posibilidad de evaluar alternativas.

El contexto debe enumerar fuerzas medibles:

```
## Contexto

- La plataforma sirve 980.000 pedidos/mes con crecimiento del 18 % anual.
- Requisito de disponibilidad: 99,95 % mensual → 21,6 min de caída permitida.
- Requisito de recuperación: RTO 60 min, RPO 15 min.
- Presupuesto de infraestructura: ≤ 0,018 USD por pedido.
- El equipo son 4 personas sin especialista en bases de datos.
- Restricción legal: los datos personales no salen del territorio nacional.
- La región disponible tiene 3 zonas.
```

Siete fuerzas, todas verificables, ninguna sugiere una solución. Con este contexto otra persona podría llegar a una decisión distinta y defenderla, que es exactamente la prueba de que el contexto está bien escrito.

La quinta línea merece comentario: las restricciones de equipo son tan legítimas como las técnicas y casi nunca se escriben. «No tenemos especialista en bases de datos» descarta razonablemente opciones que exigirían operarla, y omitirlo hace que la decisión parezca arbitraria a quien lea el documento después.

Y conviene distinguir en el contexto lo que es dato de lo que es supuesto. El crecimiento del 18 % anual es una proyección, no un hecho; marcarlo como supuesto permite que la condición de revisión lo vigile.

3. Las opciones se comparan con los mismos criterios

Un ADR con una sola opción no es una decisión: es un anuncio. Y con opciones evaluadas por criterios distintos, la comparación no significa nada.

La tabla que hace verificable la elección:

Opciones consideradas

Criterio	A: gestionada multi-zona	B: autogestionada en VM	C: gestionada una zona
Disponibilidad	99,995 %	99,5 %	99,9 %
¿Cumple 99,95 %?	Sí	No	No
RPO	5 min	según operación	5 min
Coste mensual	4.900 USD	3.100 USD	3.400 USD
Coste por pedido	0,0050	0,0032	0,0035
Carga operativa	Baja	**Alta: parcheo y réplica**	Baja
Personal necesario	0,2 FTE	**1,0 FTE**	0,2 FTE
Coste real con personal	4.900	**3.100 + 5.200 = 8.300**	3.400

Las dos últimas filas son las que cambian el resultado y las que casi nunca se incluyen. La opción B parece la más barata por infraestructura y es la más cara en total, porque exige una persona dedicada. Sin esa fila, el ADR habría elegido mal y con apariencia de rigor.

El criterio de descarte debe ser explícito y ligado al contexto:

- ****B se descarta**** por dos motivos independientes: no alcanza el requisito de disponibilidad (99,5 % frente a 99,95 %) y exige 1,0 FTE que el equipo de 4 personas no tiene.
- ****C se descarta**** por disponibilidad (99,9 % → 43 min/mes frente a 21,6 permitidos), pese a ser 1.500 USD/mes más barata.

Cada descarte cita el número del contexto que lo justifica. Eso es lo que permite que dentro de tres años alguien compruebe si el motivo sigue vigente.

4. Las consecuencias negativas son la parte honesta

Un ADR que solo enumera ventajas describe una compra, no una decisión. Toda elección de arquitectura empeora algo, y escribirlo tiene dos efectos prácticos: obliga a mirarlo antes de comprometerse, y evita que en el futuro se presente como un fallo lo que fue una elección consciente.

Consecuencias

Positivas

- Disponibilidad de 99,995 %, con margen sobre el requisito.
- RPO de 5 min frente a los 15 exigidos.
- El parcheo del motor deja de consumir tiempo del equipo.

Negativas

- ****Coste por pedido de 0,0050 USD frente a 0,0035 de la opción C****: 43 % más caro por un requisito de disponibilidad que ninguna medición de negocio ha validado todavía.
- ****Bloqueo de proveedor medio****: la conmutación automática y el punto de recuperación son propietarios. Migrar exigiría rediseñar la continuidad.
- ****Pérdida de control sobre la ventana de parcheo****: se elige el intervalo, no la versión ni la fecha exacta.
- ****El operador del proveedor tiene acceso técnico a los datos****, acotado por contrato pero real. Aceptado por dirección de tecnología.

La primera consecuencia negativa es la más valiosa del documento: admite que el requisito que justifica el sobrecoste no está validado. Eso convierte una suposición invisible en algo que alguien puede refutar con datos, y es justo lo que la condición de revisión debe vigilar.

La cuarta muestra otro patrón útil: nombrar un riesgo residual y quién lo acepta. Sin la segunda parte, el riesgo queda huérfano, que es como acaban la mayoría.

5. Ajustar el rigor a la reversibilidad

No todas las decisiones merecen el mismo proceso. Bezos las clasificó en dos tipos y la distinción es operativamente útil:

Tipo	Coste de deshacer	Proceso adecuado
Reversible	Bajo, días	Decide rápido, prueba, corrige
Irreversible	Alto o destructivo	ADR completo, consenso, condición de revisión

Ejemplos del programa, con su coste real de reversión:

reversible	elegir una biblioteca de registro	horas
reversible	umbrales del autoescalado	minutos
semi	modelo de servicio: contenedor o función	semanas
IRREVERSIBLE	esquema y particionado de datos	meses + ventana
IRREVERSIBLE	formato de identificadores públicos	rompe a terceros
IRREVERSIBLE	región donde residen los datos	coste de egreso + legal

Aplicar el proceso pesado a una decisión reversible es la forma más común de que un equipo se vuelva lento sin ganar nada. Aplicar el ligero a una irreversible es cómo se acumulan las restricciones que dentro de dos años nadie puede quitar.

La condición de revisión es lo que mantiene vivo el ADR, y debe ser observable:

```
## Condición de revisión
```

Esta decisión se reabre si ocurre cualquiera de:

1. El coste de la base de datos supera el 35 % de la factura total.
2. Se mide durante dos trimestres que la disponibilidad real del negocio no se ve afectada por caídas menores de 40 min/mes (invalidaría el requisito que justifica el sobre coste).
3. El volumen supera 5 M de pedidos/mes, donde el particionado obliga a revisar el modelo de datos completo.

Cada condición es un número comprobable, no una impresión. La segunda es la más valiosa: describe cómo se demostraría que la propia decisión fue innecesariamente cara, que es la prueba definitiva de que el ADR se escribió con honestidad.

Ejemplo trabajado

ADR-0012 de CloudShop, escrito con todo lo acumulado en la parte 01.

```
# ADR-0012: Base de datos gestionada multi-zona para pedidos
```

```
**Estado:** Aceptado · 2026-08-01
```

```
**Supersede:** ninguno · **Superseded por:** —
```

```
**Decide:** equipo de plataforma · **Acepta el riesgo:** dirección de tecnología
```

```
## Contexto
```

- 980.000 pedidos/mes, crecimiento proyectado del 18 % anual [SUPUESTO].
- Disponibilidad exigida 99,95 % mensual → 21,6 min de caída permitida [DATO].
- RTO 60 min, RPO 15 min, del plan de continuidad vigente [DATO].
- Presupuesto ≤ 0,018 USD/pedido; hoy el total es 0,0177 [DATO].
- Equipo de 4 personas, sin especialista en bases de datos [DATO].
- Datos personales con residencia nacional obligatoria [DATO, legal].
- La región disponible tiene 3 zonas [DATO].

```
## Opciones consideradas
```

```
| Criterio | A: gestionada multi-zona | B: autogestionada | C: gestionada 1 zona |
|---|---|---|---|
| Disponibilidad | 99,995 % | 99,5 % | 99,9 % |
| Caída mensual | 1,3 min | 3,6 h | 43 min |
| ¿Cumple 21,6 min? | Sí | No | **No** |
| RPO | 5 min | variable | 5 min |
| Infraestructura | 4.900 USD | 3.100 USD | 3.400 USD |
| Personal | 0,2 FTE | 1,0 FTE (5.200 USD) | 0,2 FTE |
| **Coste total** | **4.900** | **8.300** | 3.400 |
```

****B se descarta**** por dos motivos independientes: incumple la disponibilidad (3,6 h frente a 21,6 min) y exige 1,0 FTE inexistente. Es además la más cara en total pese a ser la más barata en infraestructura.

****C se descarta**** solo por disponibilidad: 43 min/mes frente a 21,6 permitidos. Es 1.500 USD/mes más barata y sería la elección si el requisito cambiara.

Decisión

Se adopta ****A****: base de datos gestionada con réplica síncrona entre dos zonas y recuperación a un instante activada.

Coste unitario resultante: $4.900 / 980.000 = 0,0050$ USD/pedido; el total sube de 0,0177 a 0,0192 USD/pedido, ****por encima del presupuesto de 0,018****. Se compensa con la optimización de transferencia identificada en la clase 011 (-2.300 USD/mes), que deja el unitario en 0,0169. Ambas acciones van juntas.

Consecuencias

Positivas

- 1,3 min/mes de caída frente a 21,6 permitidos.
- RPO de 5 min frente a 15 exigidos.
- El parcheo del motor sale del trabajo del equipo ($\approx 0,8$ FTE liberados).

Negativas

- ****43 % más cara que C**** por un requisito de disponibilidad que ninguna medición de impacto de negocio ha validado.
- ****Bloqueo medio****: conmutación y punto de recuperación son propietarios. Coste de salida estimado: 3.780 USD de egreso + ~35.000 de rediseño.
- Se elige la ventana de parcheo, no la versión ni la fecha.
- El operador del proveedor tiene acceso técnico a los datos, acotado por contrato. Riesgo aceptado por dirección de tecnología.
- ****No sobrevive a un fallo regional completo.**** Aceptado: la continuidad regional no es requisito hoy.

Condición de revisión

1. El coste de la base supera el 35 % de la factura total.
2. Dos trimestres consecutivos sin impacto de negocio atribuible a caídas menores de 40 min/mes → invalidaría el requisito y haría preferible C.
3. El volumen supera 5 M pedidos/mes → obliga a revisar el particionado.
4. Aparece requisito de continuidad regional → reabre con opción multi-región.

Verificación

- [] Restauración probada con RTO medido, trimestral (clase 019).
- [] Alerta si la retención baja de 7 días o se desactiva el punto de recuperación.
- [] Revisión de las condiciones 1 y 2 en el comité trimestral de arquitectura.

Lo que hace útil a este ADR no es la decisión —A era razonablemente evidente— sino tres cosas que un documento típico omite: la fila de personal que invierte el orden de coste de B, la admisión de que el requisito que justifica el sobrecoste no está validado, y una condición de revisión que describe cómo se demostraría que la decisión fue innecesariamente cara.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-01-cloud-principles-strategy-adoption/024-proyecto-decision-de-migracion-sustentada-con-adr/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa adr-de-migracion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye adr-de-migracion para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se revierte una decisión y se rompe algo que nadie sabía que dependía de ella	El ADR no registró la restricción que la hacía razonable	Escribe el contexto con fuerzas medibles; el código dice qué se hizo, el ADR por qué.
El ADR presenta una sola opción y parece una decisión	Es un anuncio: sin alternativas comparadas no hay razonamiento verificable	Evalúa al menos dos opciones reales con los mismos criterios y cita el número que descarta cada una.
La opción más barata en infraestructura resulta la más cara en total	No se incluyó el coste de personal en la comparación	Añade carga operativa y FTE necesarios como criterio; suele invertir el orden.
Años después nadie sabe si la decisión sigue siendo válida	No hay condición de revisión observable	Define hechos medibles que obliguen a reabrir, incluido uno que demostraría que fue innecesaria.
Un equipo se vuelve lento decidiendo cosas triviales	Se aplica el proceso de decisión irreversible a decisiones reversibles	Clasifica por coste de deshacer: reversible se decide rápido y se corrige; irreversible merece el ADR completo.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué un ADR no se edita cuando la decisión cambia, y qué se hace en su lugar?
2. ¿Cómo distingues un contexto bien escrito de uno que ya contiene la solución?
3. ¿Qué criterio incluido en la comparación invirtió el orden de coste entre las opciones A y B?
4. ¿Qué le falta a un ADR que solo enumera consecuencias positivas?
5. Escribe una condición de revisión que demostraría que la decisión tomada fue innecesariamente cara.

Referencias

- Nygard, M. (2011). Documenting Architecture Decisions — formato original de los ADR. <<https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>>
- Keeling, M. y Runde, J. (2017). Share the Load: Distribute Design Authority with Architecture Decision Records. Agile Alliance. <<https://www.agilealliance.org/resources/experience-reports/distribute-design-authority-with-architecture-decision-records/>>
- Bezos, J. (2016). Letter to Shareholders — decisiones de tipo 1 y tipo 2 según su reversibilidad. <<https://www.sec.gov/Archives/edgar/data/1018724/000119312516530910/d168744dex991.htm>>
- Ford, N., Parsons, R. y Kua, P. (2017). Building Evolutionary Architectures, cap. 2 — funciones de aptitud y decisiones con condición de revisión.
- Richards, M. y Ford, N. (2020). Fundamentals of Software Architecture, cap. 19 — registro y comunicación de decisiones.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 024 Proyecto: decisión de migración sustentada con ADR

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega adr-de-migracion con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 02 — AWS: plataforma esencial

← Parte 01 · Índice completo · Parte 03 →

Nivel: intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar y operar una carga completa en AWS con controles explícitos.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
025	Organizations, cuentas, OU, SCP y landing zone	governance	4
026	IAM, roles, políticas, STS y federación	iam	4
027	VPC, subredes, rutas, NAT, endpoints y seguridad	network	4
028	EC2, AMI, EBS y selección de capacidad	compute	4
029	Elastic Load Balancing y Auto Scaling	reliability	4
030	S3: objetos, versionado, lifecycle y replicación	storage	4
031	RDS, DynamoDB y ElastiCache: decisión de datos	data	4
032	Lambda, API Gateway y Step Functions	serverless	4
033	SQS, SNS y EventBridge	messaging	4
034	CloudWatch, CloudTrail, Config y Systems Manager	observability	4
035	KMS, Secrets Manager, WAF y controles de seguridad	security	4
036	Proyecto: aplicación de tres capas en AWS	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — John Culkin y Mike Zazon.
- Amazon Web Services in Action — Wittig y Wittig.

025 — Organizations, cuentas, OU, SCP y landing zone

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Construir la estructura de cuentas que sostendrá todo lo que se despliegue en AWS durante el resto del programa. Es la decisión más irreversible de la parte: mover cuentas entre unidades organizativas cambia las políticas que se les aplican, y separar más tarde una cuenta compartida exige migrar recursos. Aquí se aplica a AWS lo que la clase 017 estableció de forma neutral.

Resultados de aprendizaje

Al finalizar podrás:

1. Diseñar una jerarquía de unidades organizativas por régimen de gobierno y no por organigrama.
2. Escribir una SCP que restrinja sin inutilizar la cuenta, exceptuando correctamente los servicios globales.
3. Explicar por qué una SCP no concede permisos y qué implica al depurar un acceso denegado.
4. Desplegar una landing zone con cuentas de auditoría, red y seguridad separadas desde el inicio.
5. Verificar que los guardarraíles funcionan mediante prueba negativa, no por inspección de la configuración.

Conceptos centrales

Concepto	Comprensión verificable
SCP	Service Control Policy: política heredada que define el techo máximo de permisos de las cuentas bajo una unidad organizativa. Nunca concede: solo limita lo que otras políticas pueden otorgar.
landing zone	Estructura base de cuentas, red, identidad y registro sobre la que se despliega todo lo demás. Su valor está en existir antes de la primera carga, porque retrofitarla es caro.
cuenta de gestión	La que crea la organización. No admite SCP sobre sí misma, así que no debe alojar cargas: cualquier compromiso ahí no tiene barrera superior que lo contenga.
guardarraíl preventivo	Control que impide la acción antes de que ocurra, normalmente una SCP. Se distingue del detectivo, que avisa después, y exige prueba negativa para demostrar que efectivamente deniega.
cuenta de auditoría	Destino centralizado de registros de todas las demás, con escritura permitida y borrado prohibido. Es lo que evita que quien comprometa una cuenta pueda borrar su propio rastro.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    G["Cuenta de gestión<br/>sin cargas · sin SCP posible"] --> R["Raíz de la organización"]
    R --> S["U0 Security"]
    R --> I["U0 Infrastructure"]
    R --> W["U0 Workloads"]
    R --> SB["U0 Sandbox"]
    S --> AUD["log-archive<br/>solo escritura"]
    S --> SEC["security-tooling"]
    I --> NET["network<br/>conectividad compartida"]
    W --> P["U0 Prod"] --> PA["pagos-prod"]
    W --> NP["U0 No-prod"] --> PB["pagos-dev"]
    R -->|"SCP: regiones permitidas"| W
    P -->|"SCP: prohíbe borrar logs<br/>y desactivar CloudTrail"| PA
```

Desarrollo

1. La cuenta de gestión no aloja cargas

AWS Organizations tiene una asimetría deliberada: la cuenta de gestión no puede tener SCP aplicadas sobre sí misma. Es la única de toda la organización sin barrera superior.

La consecuencia es directa: cualquier recurso que viva ahí opera sin los guardarraíles que protegen a las demás, y cualquier credencial comprometida en esa cuenta tiene el poder de modificar la organización entera —crear cuentas, mover unidades, desactivar políticas—.

Lo que sí pertenece a la cuenta de gestión:

- la propia organización y sus SCP
- la agregación de facturación
- nada más

Y lo que hay que hacer con ella el primer día:

```
# Activar MFA en el usuario raíz y no volver a usarlo
$ aws iam get-account-summary --query 'SummaryMap.AccountMFAEnabled'
1
# Cero claves de acceso del usuario raíz
$ aws iam get-account-summary --query 'SummaryMap.AccountAccessKeysPresent'
0
# Alerta ante cualquier uso del usuario raíz
```

La tercera línea es el control más rentable de toda la landing zone: el uso del usuario raíz es un evento que debería ocurrir una o dos veces al año —para tareas que solo él puede hacer— y cualquier otra aparición merece una llamada telefónica.

AWS Control Tower automatiza buena parte de esta estructura, y conviene saber qué hace por debajo antes de usarlo: crea las unidades organizativas Security y Sandbox, las cuentas de archivo de registros y de auditoría, y aplica un conjunto de guardarraíles. Usarlo sin entender la estructura produce dependencia de la herramienta para operaciones que después hay que hacer a mano.

2. SCP: el orden de evaluación decide el diagnóstico

Una SCP no concede nada. Es un filtro sobre lo que las políticas de identidad pueden otorgar dentro de esa rama de la organización:

permiso efectivo = SCP $\cap$ política de identidad $\cap$ política de recurso $\cap$ límite de permisos

Dos consecuencias operativas:

1. Adjuntar una SCP con Allow: no da acceso a nadie. Si ninguna política de identidad lo concede, no hay permiso.
2. Un Deny en la SCP gana sobre cualquier concesión, incluida la del administrador de la cuenta. No hay escalada posible desde dentro.

Las SCP admiten dos estrategias, y mezclarlas causa confusión:

lista de permitidos (allowlist): quitar FullAWSAccess y enumerar lo permitido
→ muy restrictivo, alto mantenimiento: cada servicio nuevo exige tocar la SCP

lista de denegados (denylist): mantener FullAWSAccess y denegar lo prohibido

→ lo habitual: menos mantenimiento, protege contra lo que se sabe peligroso

El error de diagnóstico más caro es buscar el problema en la política de identidad cuando lo deniega una SCP. El mensaje de error de AWS lo distingue, y conviene leerlo con atención:

```
AccessDenied ... with an explicit deny in a service control policy
AccessDenied ... with an explicit deny in an identity-based policy
AccessDenied ... no identity-based policy allows the action
```

La tercera es denegación implícita —falta conceder—; las dos primeras dicen exactamente qué documento hay que revisar. Añadir permisos ante la primera no cambia nada.

3. Restringir regiones sin romper la cuenta

La SCP más común y la que más cuentas ha inutilizado. Restringir por región parece trivial hasta que se descubre que varios servicios de AWS son globales y sus llamadas se emiten contra us-east-1.

La versión que rompe:

```
{
  "Effect": "Deny",
  "Action": "*",
  "Resource": "*",
  "Condition": {"StringNotEquals": {"aws:RequestedRegion": ["us-east-1", "sa-east-1"]}}
}
```

En cuanto se aplica, deja de funcionar IAM, la facturación, el soporte y Route 53 desde cualquier contexto que no resuelva a esas regiones. Y con Action: "" se bloquea incluso la posibilidad de quitar la política desde la propia cuenta.

La versión correcta excluye los servicios sin región:

```
{
  "Effect": "Deny",
  "NotAction": [
    "iam:*", "organizations:*", "support:*", "sts:*",
    "route53:*", "cloudfront:*", "budgets:*", "ce:*",
    "health:*", "waf:*", "shield:*"
  ],
  "Resource": "*",
  "Condition": {
    "StringNotEquals": {"aws:RequestedRegion": ["us-east-1", "sa-east-1"]}
  }
}
```

Y un matiz sobre sts:: excluirlo es necesario porque el punto de acceso global de STS resuelve a us-east-1; sin la excepción, asumir roles falla desde otras regiones.

Toda SCP debe probarse en una cuenta de sandbox antes de aplicarla a una unidad organizativa productiva. No hay simulador que capture todos los casos, y el coste de equivocarse es una cuenta que no se puede administrar.

4. Cuentas de la base: auditoría, red y seguridad

Tres cuentas que conviene separar desde el primer día, porque retrofitarlas exige mover recursos con estado:

Archivo de registros (log-archive). Recibe CloudTrail, Config y logs de acceso de todas las cuentas. Su política de bucket permite escritura desde la organización y prohíbe el borrado a todo el mundo, incluido su propio administrador:

```
{
  "Effect": "Deny",
  "Principal": "*",
  "Action": ["s3:DeleteObject", "s3:DeleteObjectVersion", "s3:PutBucketPolicy"],
  "Resource": "arn:aws:s3:::org-log-archive/*"
}
```

Con bloqueo de objetos en modo cumplimiento, ni siquiera la cuenta raíz puede borrarlos antes de que expire la retención. Ese es el punto: un atacante que compromete una cuenta no puede borrar la evidencia de lo que hizo.

Red (network). Aloja el Transit Gateway, las zonas privadas de DNS y los endpoints compartidos. Separarla evita que el ciclo de vida de la conectividad dependa de un producto concreto, y permite que el equipo de red tenga permisos ahí sin tenerlos en las cuentas de carga.

Herramientas de seguridad (security-tooling). Cuenta delegada como administradora de GuardDuty, Security Hub y Config. La delegación es importante: permite operar esos servicios en toda la organización sin usar la cuenta de gestión, que es donde no hay barreras.

Las tres tienen algo en común: su valor aparece durante un incidente, y para entonces ya no se pueden crear.

5. Un guardarraíl sin prueba negativa es una intención

Configurar una SCP no demuestra que deniegue. La configuración puede estar adjuntada a la unidad equivocada, tener una condición que nunca se cumple, o quedar anulada por un NotAction demasiado amplio.

La verificación exige intentar la acción y comprobar que falla:

```
# Prueba negativa 1: región no permitida
$ aws ec2 describe-instances --region eu-west-1
An error occurred (UnauthorizedOperation) ... with an explicit deny in a
service control policy                                ✓ deniega

# Prueba negativa 2: desactivar el rastro de auditoría
$ aws cloudtrail stop-logging --name org-trail
An error occurred (AccessDeniedException) ... explicit deny in a service
control policy                                      ✓ deniega

# Prueba positiva: lo que debe seguir funcionando
$ aws iam list-roles --max-items 1 >/dev/null && echo "IAM operativo ✓"
$ aws sts get-caller-identity >/dev/null && echo "STS operativo ✓"
```

Las dos últimas líneas son tan importantes como las dos primeras: una SCP que deniega lo que debe y también lo que no, es un fallo igual de grave, solo que se descubre más tarde y en peor momento.

El simulador de políticas de IAM ayuda pero no basta: no evalúa SCP en todos los escenarios ni captura las llamadas que los servicios hacen entre sí. La única evidencia válida es el intento real desde una cuenta bajo la unidad organizativa correspondiente.

Estas pruebas deben ser automáticas y periódicas, no de una sola vez. Una SCP correcta hoy puede quedar anulada mañana por otra adjuntada a un nivel distinto de la jerarquía, sin que nadie modifique la primera.

Ejemplo trabajado

CloudShop opera en una sola cuenta de AWS y migra a una landing zone. Se ejecuta y se verifica paso a paso.

Estado inicial y sus tres riesgos concretos:

```
$ aws organizations describe-organization 2>&1 | head -1
AWSOrganizationsNotInUseException
$ aws cloudtrail describe-trails --query 'trailList[0].[Name,S3BucketName,IsMultiRegionTrail]' --
output text
cloudshop-trail    cloudshop-logs    False

1. Sin organización: no hay SCP posible, ningún guardarraíl preventivo.
2. CloudTrail en la MISMA cuenta: quien la comprometa borra su rastro.
3. Rastro de una sola región: la actividad en otras regiones es invisible.
```

Paso 1 — organización y unidades por régimen de gobierno, no por equipo:

```
$ aws organizations create-organization --feature-set ALL
$ for uo in Security Infrastructure Workloads Sandbox; do
  aws organizations create-organizational-unit --parent-id r-abcd --name $uo
done
$ aws organizations create-organizational-unit --parent-id ou-work --name Prod
$ aws organizations create-organizational-unit --parent-id ou-work --name NonProd
```

Paso 2 — cuentas de la base:

log-archive	Security	destino de logs, borrado prohibido
security-tooling	Security	administrador delegado de GuardDuty y Config
network	Infrastructure	Transit Gateway y DNS privado
cloudshop-prod	Workloads/Prod	la carga actual, migrada aquí
cloudshop-dev	Workloads/NonProd	

Paso 3 — SCP de regiones, probada antes en Sandbox:

```
$ aws organizations attach-policy --policy-id p-regiones --target-id ou-sandbox
# desde una cuenta de sandbox:
```

```
$ aws ec2 describe-instances --region ap-south-1
AccessDenied ... explicit deny in a service control policy ✓
$ aws iam list-roles --max-items 1 >/dev/null && echo ok
ok ✓ IAM intacto
$ aws sts get-caller-identity >/dev/null && echo ok
ok ✓ STS intacto
```

La primera versión de la SCP sí rompió STS: sin sts: en el NotAction, asumir roles desde sa-east-1 fallaba porque el punto de acceso global resuelve a us-east-1. Se detectó en sandbox, que es exactamente para lo que sirve.

Solo entonces se aplica a Workloads.

Paso 4 — SCP de protección de la evidencia sobre Prod:

```
{
  "Effect": "Deny",
  "Action": [
    "cloudtrail:StopLogging", "cloudtrail:DeleteTrail",
    "config:DeleteConfigurationRecorder", "config:StopConfigurationRecorder",
    "guardduty:DeleteDetector", "guardduty:DisassociateFromMasterAccount"
  ],
  "Resource": "*"
}

$ aws cloudtrail stop-logging --name org-trail # desde cloudshop-prod
AccessDeniedException ... explicit deny in a service control policy ✓
```

Paso 5 — verificación del aislamiento de cuotas, que era el fallo de la clase 017:

```
$ for c in cloudshop-prod cloudshop-dev; do
  printf "%-18s " $c
  aws service-quotas get-service-quota --service-code ec2 \
    --quota-code L-1216C47A --profile $c --query 'Quota.Value' --output text
done
cloudshop-prod      256
cloudshop-dev       128
```

Cuotas independientes: un experimento en desarrollo ya no puede consumir la capacidad de producción.

Resultado, con las pruebas que lo demuestran:

cuentas	1	5	—
guardarraíles preventivos	0	2	prueba negativa ✓
logs borrables por quien compromete la cuenta	sí	no	stop-logging denegado ✓
cuota compartida entre entornos	sí	no	256 y 128 separadas ✓
visibilidad multi-región	no	sí	rastros de organización ✓

Lo que hizo útil el ejercicio no fue la estructura sino las pruebas negativas. La primera SCP parecía correcta por inspección y rompía STS; solo intentar la acción lo reveló.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/025-organizations-cuentas-ou-scp-y-landing-zone/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa landing-zone-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye landing-zone-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Tras aplicar una SCP de regiones deja de funcionar IAM y no se puede quitar la política	Los servicios globales emiten contra us-east-1 y no se exceptuaron	Usa NotAction con iam, organizations, sts, route53, cloudfront y facturación; prueba siempre en sandbox primero.
Se añaden permisos a un rol y el acceso sigue denegado	Lo deniega una SCP, y ninguna concesión posterior la revierte	Lee el mensaje: «explicit deny in a service control policy» indica qué documento revisar.
Un atacante borra los registros de la cuenta que comprometió	CloudTrail escribía en la misma cuenta y nada impedía detenerlo	Cuenta de archivo separada, bloqueo de objetos y SCP que deniegue StopLogging y DeleteTrail.
La cuenta de gestión aloja cargas de trabajo	Es la única sin SCP posible: no tiene barrera superior	Deja en ella solo la organización y la facturación; mueve todo lo demás a cuentas bajo unidades organizativas.
Un guardarraíl configurado no impide la acción que debía impedir	Se verificó por inspección de la configuración y no por intento real	Prueba negativa automatizada y periódica, más prueba positiva de lo que debe seguir funcionando.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué la cuenta de gestión no debe alojar cargas de trabajo?
2. Adjuntas una SCP con Allow: a una unidad organizativa. ¿Quién obtiene acceso y por qué?
3. ¿Qué servicios hay que exceptuar al restringir regiones, y qué ocurre concretamente si olvidas sts:?
4. ¿Qué dos propiedades debe tener la cuenta de archivo de registros para que la evidencia sobreviva a un compromiso?
5. ¿Por qué una prueba negativa no basta sin su prueba positiva correspondiente?

Referencias

- AWS (2024). Organizations: service control policies — evaluación, estrategias y límites.
<<https://docs.aws.amazon.com/organizations/latest/userguide/orgsmanagepoliciesscps.html>>

- AWS (2024). Organizing your AWS environment using multiple accounts — cuentas de la base y unidades recomendadas. <<https://docs.aws.amazon.com/whitepapers/latest/organizing-your-aws-environment/organizing-your-aws-environment.html>>
- AWS (2024). Control Tower: how controls work — guardarraíles preventivos y detectivos. <<https://docs.aws.amazon.com/controltower/latest/controlreference/controls.html>>
- AWS (2024). CloudTrail: security best practices — rastro de organización, validación de integridad y bloqueo de objetos. <<https://docs.aws.amazon.com/awscloudtrail/latest/userguide/best-practices-security.html>>
- AWS (2024). AWS services that work with IAM — qué servicios son globales y cómo se comportan con condiciones de región. <<https://docs.aws.amazon.com/IAM/latest/UserGuide/referenceaws-services-that-work-with-iam.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 025 Organizations, cuentas, OU, SCP y landing zone

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega landing-zone-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

026 — IAM, roles, políticas, STS y federación

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Dominar el modelo de identidad de AWS hasta poder predecir el resultado de una petición sin ejecutarla, y sustituir toda credencial de larga vida por identidad temporal. Es la clase que hace posible que en la parte 17 un pipeline despliegue sin secretos, y la que explica por qué la mayoría de brechas en AWS no son fallos de AWS.

Resultados de aprendizaje

Al finalizar podrás:

1. Reproducir el orden de evaluación de una petición y predecir el resultado ante políticas en conflicto.
2. Escribir una política con condiciones que acoten origen, etiqueta y presencia de MFA.
3. Configurar federación OIDC desde un pipeline con una condición de sujeto que impida la suplantación.
4. Usar límites de permisos para delegar creación de roles sin permitir escalada de privilegios.
5. Diagnosticar un acceso denegado leyendo el mensaje para saber qué documento revisar.

Conceptos centrales

Concepto	Comprensión verificable
rol	Identidad sin credenciales permanentes que se asume temporalmente. Tiene dos documentos: la política de confianza —quién puede asumirlo— y las de permisos —qué puede hacer una vez dentro—.
política de confianza	Documento que declara qué principales pueden asumir el rol y bajo qué condiciones. Es la puerta de entrada, y un error aquí es más grave que uno en los permisos.
límite de permisos	Política adjunta a una identidad que define el techo de lo que puede hacer, aunque otras políticas concedan más. Permite delegar la creación de roles sin permitir escalada.
confused deputy	Ataque en el que un tercero legítimo es inducido a actuar contra un recurso ajeno. Se mitiga con condiciones sobre la cuenta de origen y el identificador externo.
clave de condición	Atributo evaluable de la petición: origen, hora, MFA, etiquetas, VPC. Convierte una política de «puede» en «puede, si además se cumple esto».

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
P["Petición"] --> D1{"¿Deny explícito<br/>en cualquier política?"}
D1 -->|"sí"| N1["DENEGADA · sin apelación"]
D1 -->|"no"| D2{"¿La SCP lo permite?"}
D2 -->|"no"| N2["DENEGADA · revisa la SCP"]
D2 -->|"sí"| D3{"¿Límite de permisos<br/>lo permite?"}
D3 -->|"no"| N3["DENEGADA · revisa el límite"]
D3 -->|"sí"| D4{"¿Política de identidad<br/>o de recurso concede?"}
D4 -->|"no"| N4["DENEGADA · implícita"]
D4 -->|"sí"| D5{"¿Se cumplen las<br/>condiciones?"}
D5 -->|"no"| N1
D5 -->|"sí"| OK["PERMITIDA"]
```

Desarrollo

1. El orden de evaluación, y cómo leer el error

AWS evalúa cada petición en una secuencia fija. Conocerla convierte «no tengo permiso» en un diagnóstico de un minuto:

1. Deny explícito en cualquier política → denegada, sin excepción.
2. SCP de la organización → si no lo permite, denegada.
3. Límite de permisos, si existe → si no lo permite, denegada.
4. Política de identidad o de recurso → alguna debe conceder.
5. Condiciones → deben cumplirse todas.

El mensaje de error dice exactamente dónde mirar, y casi nadie lo lee:

```
"...with an explicit deny in a service control policy"
→ revisa las SCP. Añadir permisos a la identidad no hará nada.

"...with an explicit deny in a permissions boundary"
→ revisa el límite de permisos del rol.

"...with an explicit deny in an identity-based policy"
→ hay un Deny en la política del rol o del usuario.

"...because no identity-based policy allows the action"
→ denegación implícita: falta conceder. Este es el único caso en que
añadir un permiso resuelve el problema.
```

Solo el cuarto caso se arregla concediendo. Los tres primeros exigen tocar otro documento, y perseguirlos con más permisos es la causa habitual de que un rol acabe con AdministratorAccess.

Hay un matiz sobre acceso entre cuentas: cuando el recurso está en otra cuenta hacen falta las dos políticas —la de identidad en la cuenta que llama y la de recurso en la que responde—. Una sola nunca basta, y el mensaje no siempre distingue cuál falta.

2. Condiciones: donde una política deja de ser un cheque en blanco

Un permiso sin condiciones es válido desde cualquier lugar, a cualquier hora y en cualquier contexto. Las claves de condición acotan eso:

```
{
  "Effect": "Allow",
  "Action": ["s3:GetObject", "s3:PutObject"],
  "Resource": "arn:aws:s3:::cloudshop-facturas/*",
  "Condition": {
    "Bool": {"aws:MultiFactorAuthPresent": "true"},
    "StringEquals": {"aws:PrincipalTag/equipo": "finanzas"},
    "IpAddress": {"aws:SourceIp": ["203.0.113.0/24"]},
    "DateLessThan": {"aws:CurrentTime": "2026-12-31T23:59:59Z"}
  }
}
```

Las cuatro condiciones se combinan con Y lógico: todas deben cumplirse. Dentro de una misma clave con varios valores, la relación es O.

Dos condiciones merecen atención especial:

aws:PrincipalTag habilita control de acceso por atributos: en vez de una política por equipo, una sola política que compara la etiqueta del principal con la del recurso. Escala mucho mejor que enumerar.

aws:SourceIp no funciona como se espera con endpoints de VPC. Cuando la llamada viaja por un endpoint, la IP de origen es privada y la condición falla. Para ese caso hay que usar aws:SourceVpce o aws:SourceVpc, y es un fallo que se descubre justo después de endurecer la red.

Y una condición que evita el confused deputy al conceder acceso a un tercero:

```
"Condition": {"StringEquals": {"sts:ExternalId": "identificador-unico-por-cliente"}}
```

Sin ella, un proveedor que gestiona varias cuentas podría ser inducido a actuar contra la tuya usando su propio rol legítimo.

3. Federación OIDC: desplegar sin ningún secreto

El mecanismo que elimina las claves de acceso de los pipelines, aplicado a AWS. El intercambio, sin secreto compartido en ningún punto:

1. El pipeline pide a su plataforma un token OIDC que declara quién es.
2. Llama a sts:AssumeRoleWithWebIdentity presentando ese token.
3. AWS verifica la firma contra las claves públicas del emisor y comprueba que las declaraciones cumplen la política de confianza.
4. Devuelve credenciales temporales de 1 hora.

La política de confianza es donde se juega todo:

```
{
  "Effect": "Allow",
  "Principal": {"Federated": "arn:aws:iam::123456789012:oidc-provider/token.actions.githubusercontent.com"},
  "Action": "sts:AssumeRoleWithWebIdentity",
  "Condition": {
    "StringEquals": {
      "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
      "token.actions.githubusercontent.com:sub": "repo:miorg/cloudshop:environment:produccion"
    }
  }
}
```

Sin la condición sobre sub, cualquier repositorio de GitHub del mundo puede asumir el rol. La firma será válida porque procede del mismo emisor; lo único que distingue a tu repositorio de otro es esa declaración.

Los errores por grado de peligro:

sin condición sobre sub	→ cualquiera, en cualquier parte
"sub": "repo:miorg/*"	→ cualquier repositorio de tu organización
StringLike "repo:miorg/cloudshop:*"	→ cualquier rama, incluida una de un fork
StringEquals "...:environment:produccion"	→ correcto

Usar environment en lugar de ref añade una capa: los entornos de GitHub admiten revisores obligatorios, así que el rol solo es asumible tras una aprobación humana.

4. Límites de permisos: delegar sin permitir escalada

El problema: quieres que los equipos creen sus propios roles sin abrir la puerta a que se concedan AdministratorAccess. Conceder iam:CreateRole y iam:AttachRolePolicy es, de hecho, conceder administrador.

Un límite de permisos define el techo de lo que una identidad puede hacer, con independencia de lo que sus políticas concedan:

```
// Límite: nada fuera de estos servicios, y prohibido tocar IAM salvo lo mínimo
{
  "Version": "2012-10-17",
  "Statement": [
    {"Effect": "Allow", "Action": ["s3:*", "dynamodb:*", "logs:*", "sqs:*"], "Resource": "*"},
    {"Effect": "Deny", "Action": ["iam:*", "organizations:*", "account:*"], "Resource": "*"}
  ]
}
```

```
}
```

Y la política que permite delegar obligando a aplicar ese límite:

```
{
  "Effect": "Allow",
  "Action": ["iam:CreateRole", "iam:AttachRolePolicy"],
  "Resource": "arn:aws:iam::*:role/equipos/*",
  "Condition": {
    "StringEquals": {"iam:PermissionsBoundary": "arn:aws:iam::123456789012:policy/limite-equipos"}
  }
}
```

La condición es la clave: solo se pueden crear roles que lleven el límite adjunto. Sin ella, el equipo crearía roles sin límite y la delegación sería una escalada.

Falta una pieza para cerrarlo del todo:

```
{
  "Effect": "Deny",
  "Action": ["iam:DeleteRolePermissionsBoundary", "iam:PutRolePermissionsBoundary"],
  "Resource": "*"
}
```

Sin esta denegación, el equipo podría crear el rol con límite y quitárselo después. Es el hueco que convierte un diseño correcto en uno inútil, y no es evidente hasta que alguien lo prueba.

5. Privilegio mínimo con las herramientas de AWS

Adivinar la lista de permisos produce o bien exceso o bien bloqueo. AWS ofrece tres herramientas para partir de lo observado:

```
# 1. Qué servicios ha usado realmente un rol
$ aws iam generate-service-last-accessed-details --arn arn:aws:iam::...:role/ci-deploy
$ aws iam get-service-last-accessed-details --job-id ... \
  --query 'ServicesLastAccessed[?TotalAuthenticatedEntities>`0`].[ServiceName,LastAuthenticated]'
```

```
# 2. Generar una política a partir de la actividad de CloudTrail
$ aws accessanalyzer start-policy-generation --policy-generation-details \
  '{"principalArn":"arn:aws:iam::...:role/ci-deploy"}' --cloud-trail-details ...
```

```
# 3. Validar la política antes de aplicarla
$ aws accessanalyzer validate-policy --policy-document file://politica.json \
  --policy-type IDENTITY_POLICY --query 'findings[?findingType==`SECURITY_WARNING`]'
```

La tercera detecta patrones peligrosos que pasan desapercibidos: comodines en el principal, iam:PassRole sin restricción de recurso, o acciones que permiten escalada.

iam:PassRole merece atención propia. Permite entregar un rol a un servicio, y sin restricción de recurso equivale a conceder cualquier permiso que exista en la cuenta: basta con pasar un rol de administrador a una función o a una instancia que tú controlas.

```
// Peligroso
{"Effect": "Allow", "Action": "iam:PassRole", "Resource": "*"}

// Correcto: solo roles concretos y solo al servicio esperado
{
  "Effect": "Allow", "Action": "iam:PassRole",
  "Resource": "arn:aws:iam::*:role/cloudshop-tarea-*",
  "Condition": {"StringEquals": {"iam:PassedToService": "ecs-tasks.amazonaws.com"}}
}
```

Y el límite del método basado en registros, ya señalado en la clase 018: los caminos de error usan permisos distintos. Un rol que escribe en una cola pero no en su cola de mensajes fallidos funciona hasta el primer fallo.

Ejemplo trabajado

El pipeline de CloudShop despliega en producción con una clave de acceso de 2 años y medio y PowerUserAccess. Se migra a OIDC con privilegio mínimo, verificando cada paso.

Estado inicial:

```
$ aws iam list-access-keys --user-name ci-deploy \
  --query 'AccessKeyMetadata[0].[CreateDate,Status]' --output text
2023-03-14T10:22:00Z    Active
$ aws iam list-attached-user-policies --user-name ci-deploy --query 'AttachedPolicies[].PolicyName'
```

```
["PowerUserAccess"]

antigüedad          2 años 5 meses, 0 rotaciones
copias conocidas     secreto del repositorio, 2 portátiles, 1 runbook
permisos             ~8.000 acciones
ventana si se filtra ilimitada
```

Paso 1 — proveedor OIDC y prueba de la política de confianza insegura, para verla fallar:

```
$ aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com --client-id-list sts.amazonaws.com
```

Con la condición solo sobre aud, desde un repositorio de laboratorio ajeno a la organización:

```
$ aws sts assume-role-with-web-identity --role-arn ...:role/ci-deploy-oidc ...
{"Credentials": {...}} ← ÉXITO desde un repositorio ajeno
```

Se corrige acotando el sujeto al entorno:

```
$ aws iam update-assume-role-policy --role-name ci-deploy-oidc \
  --policy-document file://confianza.json # sub: repo:miorg/cloudshop:environment:produccion
```

Pruebas negativas y positiva:

```
desde otro repositorio          → AccessDenied ✓
desde miorg/cloudshop, rama de trabajo → AccessDenied ✓
desde miorg/cloudshop, entorno prod → credenciales de 1 h ✓
```

Paso 2 — recortar permisos con lo observado en 30 días:

```
$ aws accessanalyzer start-policy-generation --policy-generation-details \
  '{"principalArn":"arn:aws:iam::123456789012:user/ci-deploy"}'
$ aws accessanalyzer get-generated-policy --job-id ... \
  --query 'generatedPolicyResult.generatedPolicies[0].policy' | jq -r '.Statement[].Action' | wc -l
34
```

```
permisos concedidos antes  ~8.000 acciones
acciones realmente usadas    34
servicios usados            5 (s3, cloudfront, ecs, ecr, logs)
```

Se ejercitan también los caminos de error antes de aplicar. Aparecen dos acciones más que ningún despliegue correcto usaba:

```
ecs:StopTask          al abortar un despliegue fallido
logs:PutLogEvents     sobre el grupo de errores
```

Paso 3 — validar la política antes de aplicarla:

```
$ aws accessanalyzer validate-policy --policy-document file://ci-deploy.json \
  --policy-type IDENTITY_POLICY \
  --query 'findings[?findingType==`SECURITY_WARNING`].[issueCode,learnMoreLink]' --output text
PASS_ROLE_WITH_STAR_IN_RESOURCE https://docs.aws.amazon.com/...
```

El validador detecta el fallo que la generación automática no evita: iam:PassRole con Resource: "", que permite pasar cualquier rol —incluido uno de administrador— a un servicio controlado por el pipeline. Se acota:

```
{
  "Effect": "Allow", "Action": "iam:PassRole",
  "Resource": "arn:aws:iam::*:role/cloudshop-tarea-*",
  "Condition": {"StringEquals": {"iam:PassedToService": "ecs-tasks.amazonaws.com"}}
}

$ aws accessanalyzer validate-policy ... --query 'findings[?findingType==`SECURITY_WARNING`]'
[] ✓
```

Paso 4 — retirar la credencial, con observación previa:

```
$ aws iam update-access-key --user-name ci-deploy --access-key-id AKIA... --status Inactive
# 7 días sin fallos ni llamadas registradas con esa clave
$ aws iam delete-access-key --user-name ci-deploy --access-key-id AKIA...
$ aws iam delete-user --user-name ci-deploy
```

Resultado:

```
credencial          permanente, 2,5 años  token de 1 h
secretos almacenados 4 copias              0
acciones permitidas  ~8.000                36
iam:PassRole         sin restricción        1 patrón, 1 servicio
superficie si se filtra ilimitada          1 h, solo ese repo y entorno
```

El paso que más aportó fue el validador, no la generación automática: la política generada a partir del uso real contenía un PassRole sin acotar que habría permitido escalar a administrador desde el propio pipeline.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/026-iam-roles-politicas-sts-y-federacion/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa politica-iam-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye politica-iam-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Cualquier repositorio de GitHub puede asumir el rol de despliegue	La política de confianza no condiciona la declaración sub	Condiciona con StringEquals sobre repositorio y entorno; repo:org/ o StringLike con comodín no bastan.
Se conceden más permisos y el acceso sigue denegado	El deny está en una SCP o en un límite de permisos	Lee el mensaje de error: nombra el documento exacto que deniega.
Una política endurecida con aws:SourceIp rompe el acceso desde la VPC	A través de un endpoint de VPC la IP de origen es privada	Usa aws:SourceVpce o aws:SourceVpc para tráfico que viaja por endpoints.
Un equipo con permiso para crear roles se concede administrador	Se delegó iam:CreateRole sin exigir límite de permisos ni impedir retirarlo	Condiciona con iam:PermissionsBoundary y deniega Put/DeleteRolePermissionsBoundary.
Un pipeline con permisos mínimos consigue privilegios de administrador	iam:PassRole con Resource permite pasar cualquier rol a un servicio propio	Acota PassRole por patrón de recurso y por iam:PassedToService.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Un error dice «explicit deny in a service control policy». ¿Sirve de algo añadir permisos al rol? ¿Qué documento revisas?
2. ¿Por qué aws:SourceIp deja de funcionar cuando el tráfico pasa por un endpoint de VPC, y qué clave se usa entonces?
3. ¿Qué dos piezas hacen falta para delegar la creación de roles sin permitir escalada de privilegios?
4. ¿Por qué iam:PassRole con Resource: "" equivale a conceder administrador?
5. ¿Qué distingue exactamente tu repositorio del de un atacante en un intercambio OIDC?

Referencias

- AWS (2024). Policy evaluation logic — orden de evaluación completo con SCP y límites de permisos. <<https://docs.aws.amazon.com/IAM/latest/UserGuide/referencepoliciesevaluation-logic.html>>
- AWS (2024). Global condition context keys — catálogo de claves de condición y su comportamiento. <<https://docs.aws.amazon.com/IAM/latest/UserGuide/referencepoliciescondition-keys.html>>
- AWS (2024). Permissions boundaries for IAM entities — delegación segura de creación de roles. <<https://docs.aws.amazon.com/IAM/latest/UserGuide/accesspoliciesboundaries.html>>
- AWS (2024). IAM Access Analyzer policy validation — comprobaciones de seguridad antes de aplicar una política. <<https://docs.aws.amazon.com/IAM/latest/UserGuide/access-analyzer-policy-validation.html>>
- Hardt, D., ed. (2012). RFC 6749: The OAuth 2.0 Authorization Framework — base del intercambio de token por credenciales. <<https://www.rfc-editor.org/rfc/rfc6749>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 026 IAM, roles, políticas, STS y federación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega politica-iam-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.

- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

027 — VPC, subredes, rutas, NAT, endpoints y seguridad

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Diseñar una VPC entendiendo qué componente decide cada cosa: la tabla de rutas determina por dónde sale el tráfico, el grupo de seguridad quién puede hablar, y la elección entre NAT y endpoint decide una partida de la factura que suele sorprender. Es la base de las clases 028 a 036 y el origen de la mayoría de incidentes de conectividad del programa.

Resultados de aprendizaje

Al finalizar podrás:

1. Planificar un rango CIDR que admita crecimiento y no colisione con la red corporativa ni con futuros pares.
2. Determinar si una subred es pública o privada por su tabla de rutas, no por su nombre.
3. Elegir entre grupo de seguridad y ACL de red sabiendo cuál tiene estado y cuál no.
4. Calcular el ahorro de sustituir tráfico por NAT gateway por endpoints de VPC.
5. Diagnosticar un fallo de conectividad recorriendo las capas en orden y con los registros de flujo.

Conceptos centrales

Concepto	Comprensión verificable
tabla de rutas	Conjunto de reglas que decide por dónde sale el tráfico de una subred. Es lo único que hace pública a una subred: una ruta hacia una puerta de enlace de internet.
grupo de seguridad	Cortafuegos con estado a nivel de interfaz de red. Solo tiene reglas de permiso y la respuesta al tráfico permitido vuelve automáticamente, sin regla de salida que la autorice.
ACL de red	Cortafuegos sin estado a nivel de subred. Admite denegaciones explícitas, y al no tener estado exige regla de vuelta para los puertos efímeros.
NAT gateway	Servicio que permite salida a internet desde subredes privadas. Cobra por hora y por GB procesado, lo que lo convierte en una de las partidas más caras y menos vigiladas.
endpoint de VPC	Ruta privada hacia un servicio de AWS sin pasar por internet. Los de tipo gateway —S3 y DynamoDB— son gratuitos; los de interfaz cobran por hora y por GB, pero menos que el NAT.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    IGW["Internet Gateway"] --> PUB["Subred pública<br/>ruta 0.0.0.0/0 → IGW"]
    PUB --> NAT["NAT Gateway"]
    NAT --> PRIV["Subred privada<br/>ruta 0.0.0.0/0 → NAT"]
    PRIV --> APP["aplicación"]
    APP -->|"gratis, no pasa por NAT"| EPG["Endpoint gateway<br/>S3 · DynamoDB"]
    APP -->|"por hora + GB, más barato que NAT"| EPI["Endpoint de interfaz<br/>ECR · Secrets · SSM"]
    APP -->|"todo lo demás:<br/>0,045 USD/GB"| NAT
    ISO["Subred aislada<br/>sin ruta a 0.0.0.0/0"] --- DB[("base de datos")]
```


Desarrollo

1. Planificar el CIDR antes de crear nada

El rango de una VPC no se puede cambiar; solo se pueden añadir bloques secundarios, con restricciones. Un error aquí se paga durante años.

Tres reglas que evitan los problemas más caros:

1. No solapar con nada con lo que puedas necesitar hablar. Si la red corporativa usa 10.0.0.0/8 y creas la VPC en 10.0.0.0/16, la VPN o el emparejamiento serán imposibles: el enrutamiento no admite solapamiento. Un registro central de rangos asignados es más barato que una migración.
2. Dimensionar con holgura. AWS admite de /16 a /28. Un /16 da 65.536 direcciones y no cuesta nada reservarlo; un /24 se agota en cuanto se añaden zonas o servicios que consumen direcciones.
3. Contar las direcciones que AWS reserva. En cada subred se reservan 5, no 2:

```
Subred 10.20.1.0/24 → 256 direcciones teóricas
.0   dirección de red
.1   puerta de enlace de la VPC
.2   servidor DNS
.3   reservada para uso futuro
.255 difusión
-----
251 direcciones utilizables
```

Un esquema que crece sin colisiones:

VPC produccion		10.20.0.0/16	
pública	zona a	10.20.0.0/20	4.091 utilizables
pública	zona b	10.20.16.0/20	
privada	zona a	10.20.32.0/20	
privada	zona b	10.20.48.0/20	
aislada	zona a	10.20.64.0/20	
aislada	zona b	10.20.80.0/20	
libre		10.20.96.0/19	reservado para crecer

El bloque libre al final no es desperdicio: es lo que permite añadir una tercera zona sin rehacer el esquema.

2. Pública o privada lo decide la tabla de rutas

Los nombres «pública» y «privada» son convención humana. Lo único que hace pública a una subred es una ruta hacia una puerta de enlace de internet:

Subred pública	0.0.0.0/0 → igw-xxxx
Subred privada	0.0.0.0/0 → nat-xxxx
Subred aislada	sin ruta hacia 0.0.0.0/0

De ahí un fallo que aparece con regularidad: una subred llamada «privada» con ruta al IGW es pública, y cualquier recurso con IP pública en ella es accesible desde internet. El nombre no protege nada.

La comprobación es directa:

```
$ aws ec2 describe-route-tables \
  --filters Name=association.subnet-id,Values=subnet-0a1b2c \
  --query 'RouteTables[0].Routes[?DestinationCidrBlock==`0.0.0.0/0`].GatewayId' --output text
igw-0f9e8d      # es PÚBLICA, se llame como se llame
```

Y hay una segunda condición para que una instancia sea alcanzable desde internet: necesita IP pública además de la ruta. Una instancia sin IP pública en una subred pública no es accesible desde fuera, aunque sí puede salir si hay NAT.

La tercera categoría —aislada, sin ruta a internet en absoluto— es donde deben vivir las bases de datos. No es lo mismo que privada: una subred privada puede iniciar conexiones salientes a internet a través del NAT, y eso es exactamente lo que usa un atacante para exfiltrar datos o descargar herramientas.

3. Grupo de seguridad y ACL: con estado y sin estado

La diferencia decisiva:

	Grupo de seguridad	ACL de red
Ámbito	Interfaz de red	Subred entera
Estado	Con estado	Sin estado
Reglas	Solo permitir	Permitir y denegar
Evaluación	Todas las reglas	Por orden de número
Referencia a otro grupo	Sí	No, solo CIDR

Con estado significa que la respuesta al tráfico permitido vuelve automáticamente. Si un grupo permite entrada al 443, la respuesta sale sin necesidad de regla de salida.

Sin estado significa que hay que autorizar ambos sentidos. Este es el error clásico de las ACL:

```
ACL con entrada 443 permitida y salida solo 443:
petición entrante al 443          ✓ permitida
respuesta desde el 443 hacia el
puerto efímero del cliente (52341) ✗ BLOQUEADA
```

La conexión se establece y se cuelga. Hay que permitir la salida hacia el rango efímero 1024-65535, y es la causa habitual de que «el firewall parece bien configurado y no funciona».

La capacidad de referenciar otro grupo de seguridad en vez de un CIDR es lo que hace mantenible el diseño:

```
$ aws ec2 authorize-security-group-ingress --group-id sg-db \
--protocol tcp --port 5432 --source-group sg-app
```

Esto dice «lo que esté en sg-app puede hablar con la base de datos», y sigue siendo cierto cuando las instancias cambian de IP, se escalan o se sustituyen. Un CIDR fijo exige mantenimiento cada vez que la topología cambia.

La recomendación práctica: grupos de seguridad para casi todo, ACL solo para denegaciones amplias —bloquear un rango de IP concreto—, porque su falta de estado las hace fáciles de configurar mal.

4. NAT gateway: la partida cara que nadie mira

Un NAT gateway cobra dos veces: por hora de existencia y por GB procesado. La segunda es la que sorprende.

```
precio por hora          ~0,045 USD → 32,85 USD/mes
precio por GB procesado  ~0,045 USD
```

Con 8 TB mensuales de tráfico saliente hacia servicios de AWS:

```
8.000 GB × 0,045 = 360 USD/mes solo de procesamiento
más 32,85 de la hora, por cada NAT
con un NAT por zona (3 zonas): 98,55 + 360 = 458,55 USD/mes
```

Y lo importante: buena parte de ese tráfico no necesita salir a internet. Descargar imágenes de ECR, leer secretos, escribir logs o hablar con S3 son llamadas a servicios de AWS que un endpoint resuelve por la red interna.

```
tráfico por NAT gateway          0,045 USD
tráfico por endpoint de interfaz  0,010 USD
tráfico por endpoint gateway     0,000 USD ← S3 y DynamoDB
```

El endpoint gateway es gratuito y se instala como una ruta en la tabla; no tener uno para S3 es dinero regalado sin ninguna contrapartida. El de interfaz cuesta 0,01 USD por hora por zona más 0,01 por GB, así que compensa a partir de un volumen modesto:

```
umbral del endpoint de interfaz (1 zona):
7,30 USD/mes de coste fijo / (0,045 - 0,010) USD/GB ≈ 209 GB/mes
```

Por encima de 209 GB mensuales hacia ese servicio, el endpoint sale más barato. Y además del ahorro, aporta seguridad: el tráfico no sale de la red de AWS, lo que permite subredes aisladas que hablan con los servicios que necesitan sin ninguna ruta a internet.

5. Diagnóstico de conectividad, de abajo hacia arriba

El orden ahorra horas porque cada paso descarta una capa completa:

```
# 1. ¿Existe ruta hacia el destino?
$ aws ec2 describe-route-tables --filters Name=association.subnet-id,Values=subnet-xxx \
--query 'RouteTables[0].Routes[].[DestinationCidrBlock,GatewayId,NatGatewayId]' --output text
```

```
# 2. ¿Lo permite la ACL de la subred, en AMBOS sentidos?
$ aws ec2 describe-network-acls --filters Name=association.subnet-id,Values=subnet-xxx \
  --query 'NetworkAcls[0].Entries[?RuleNumber<`32767`]'
```

```
# 3. ¿Lo permite el grupo de seguridad de origen y el de destino?
$ aws ec2 describe-security-groups --group-ids sg-app sg-db \
  --query 'SecurityGroups[][GroupId,IpPermissions[][FromPort]]'
```

```
# 4. Analizador de accesibilidad: evalúa la ruta completa
$ aws ec2 create-network-insights-path --source i-origen --destination i-destino \
  --protocol tcp --destination-port 5432
$ aws ec2 start-network-insights-analysis --network-insights-path-id nip-xxx
```

El paso 4 es el más rentable y el menos usado: evalúa la configuración completa y dice qué componente bloquea, sin necesidad de tráfico real.

Y cuando la configuración parece correcta pero algo falla, los registros de flujo dan el veredicto:

```
$ aws logs filter-log-events --log-group-name /aws/vpc/flowlogs \
  --filter-pattern '[version, account, eni, source, destination, srcport,
                    destport=5432, protocol, packets, bytes, start, end,
                    action=REJECT, status]' --max-items 5
```

Un REJECT indica que una regla lo bloqueó y hay que revisar grupos y ACL. La ausencia total de registros es un diagnóstico distinto y más útil: significa que el paquete nunca llegó a la interfaz, así que el problema está en enrutamiento o resolución de nombres, no en el firewall.

Ejemplo trabajado

La factura de red de CloudShop es de 1.180 USD/mes y nadie sabe explicarla. Además, la base de datos está en una subred llamada «privada». Se auditan ambas cosas.

Parte 1 — de dónde viene el gasto:

```
$ aws ce get-cost-and-usage --time-period Start=2026-07-01,End=2026-08-01 \
  --granularity MONTHLY --metrics UnblendedCost \
  --filter '{"Dimensions":{"Key":"USAGE_TYPE_GROUP","Values":["EC2: NAT Gateway"]}}' \
  --query 'ResultsByTime[0].Total.UnblendedCost.Amount' --output text
842.31
```

842 USD de 1.180 son NAT gateway. Se desglosa qué tráfico lo atraviesa con los registros de flujo:

destino	GB/mes	% del tráfico por NAT
S3 (misma región)	9.140	49 %
ECR (descarga de imágenes)	4.620	25 %
Secrets Manager y SSM	310	2 %
CloudWatch Logs	1.850	10 %
internet real (APIs de terceros)	2.680	14 %
total	18.600 GB	

El 86 % del tráfico que paga NAT no va a internet: va a servicios de AWS de la misma región.

Cálculo del cambio a endpoints:

actual		
3 NAT × 32,85 USD/mes de hora	=	98,55
18.600 GB × 0,045 USD	=	837,00

		935,55

con endpoints		
gateway S3 (gratis)	9.140 GB →	0,00
interfaz ECR 3 zonas × 7,30 + 4.620 × 0,010	=	68,10
interfaz Secrets+SSM 2 svc × 3 × 7,30 + 310 × 0,010	=	46,90
interfaz Logs 3 × 7,30 + 1.850 × 0,010	=	40,40
NAT solo para internet real		
3 × 32,85 + 2.680 × 0,045	=	219,15

		374,55

ahorro: 935,55 – 374,55 = 561 USD/mes (–60 %)

Se verifica el umbral antes de crear cada endpoint de interfaz, para no añadir coste fijo sin retorno:

Secrets Manager: 310 GB/mes
 coste fijo 3 zonas: 21,90 USD
 ahorro por GB: $310 \times (0,045 - 0,010) = 10,85$ USD
 → NO compensa por volumen, pero SÍ por seguridad: permite que la subred aislada lea secretos sin ninguna ruta a internet. Se crea igualmente, con el sobre coste de 11 USD/mes declarado como decisión de seguridad.

Parte 2 — la subred «privada» de la base de datos:

```
$ aws ec2 describe-route-tables \
  --filters Name=association.subnet-id,Values=subnet-0db1 \
  --query 'RouteTables[0].Routes[].[DestinationCidrBlock,GatewayId,NatGatewayId]' --output text
10.20.0.0/16    local    None
0.0.0.0/0      None    nat-0a7f
```

No es pública —sale por NAT, no por IGW— pero sí puede iniciar conexiones salientes a internet. Para una base de datos eso es un camino de exfiltración que no aporta nada:

```
$ aws ec2 create-route-table --vpc-id vpc-0c1d --tag-specifications \
  'ResourceType=route-table,Tags=[{Key=Name,Value=aislada}]'
# sin ruta 0.0.0.0/0; solo local y los endpoints necesarios
$ aws ec2 associate-route-table --route-table-id rtb-nueva --subnet-id subnet-0db1
```

Prueba negativa desde la instancia de base de datos:

```
$ curl -m 5 https://example.com
curl: (28) Connection timed out                ✓ sin salida a internet
$ aws secretsmanager get-secret-value --secret-id cloudshop/db --query Name
"cloudshop/db"                                ✓ sigue accediendo por endpoint
```

Resultado:

coste de red	1.180 USD	619 USD	(−48 %)
tráfico por NAT	18.600 GB	2.680 GB	
salida a internet desde la BD	sí	no	
endpoints	0	5	

El diagnóstico no fue de red sino de rutas. El 86 % del gasto venía de que todo el tráfico interno salía por un componente pensado para internet, y la base de datos tenía un camino de salida que nadie había decidido darle: lo heredó de la tabla de rutas por defecto.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/027-vpc-subredes-rutas-nat-endpoints-y-seguridad/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa vpc-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye vpc-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una subred llamada «privada» resulta accesible desde internet	Su tabla de rutas apunta a la puerta de enlace de internet: el nombre no decide nada	Verifica la ruta 0.0.0.0/0; pública, privada y aislada se distinguen solo por ahí.
Una conexión se establece y se cuelga pese a que el firewall permite el puerto	La ACL de red no tiene estado y falta permitir la vuelta por el rango efímero	Autoriza 1024-65535 en el sentido de retorno, o usa grupos de seguridad, que sí tienen estado.
La factura de NAT gateway es la mayor partida de red	El tráfico hacia servicios de AWS de la misma región atraviesa el NAT y paga 0,045 USD/GB	Endpoint gateway para S3 y DynamoDB (gratis) y de interfaz por encima de 209 GB/mes.
No se puede ampliar el rango de la VPC ni emparejar con la red corporativa	El CIDR se solapa y no se puede cambiar tras la creación	Planifica el rango contra un registro central antes de crear nada; deja bloques libres para crecer.
Una base de datos puede iniciar conexiones salientes a internet	Está en subred privada con ruta al NAT, heredada de la tabla por defecto	Usa subred aislada sin ruta a 0.0.0.0/0 y endpoints para lo que necesite; verifica con prueba negativa.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuántas direcciones utilizables tiene una subred /24 en AWS y por qué no son 254?
2. ¿Qué convierte a una subred en pública, y basta con eso para que una instancia sea alcanzable desde internet?
3. Una ACL permite entrada al 443 y salida solo por el 443. ¿Qué ocurre y por qué?
4. A partir de cuántos GB mensuales compensa un endpoint de interfaz frente al NAT, y de dónde sale ese número?
5. Los registros de flujo no muestran ninguna entrada para una conexión fallida. ¿Qué te dice eso frente a un REJECT?

Referencias

- AWS (2024). VPC user guide: subnets and routing — direcciones reservadas y tablas de rutas.
<<https://docs.aws.amazon.com/vpc/latest/userguide/configure-subnets.html>>
- AWS (2024). Compare security groups and network ACLs — con estado frente a sin estado.
<<https://docs.aws.amazon.com/vpc/latest/userguide/infrastructure-security.html>>
- AWS (2024). AWS PrivateLink and VPC endpoints — tipos de endpoint, precios y restricciones.
<<https://docs.aws.amazon.com/vpc/latest/privatelink/what-is-privatelink.html>>

- AWS (2024). VPC Flow Logs — formato de registro y campos para diagnóstico.
<<https://docs.aws.amazon.com/vpc/latest/userguide/flow-logs.html>>
- AWS (2024). Reachability Analyzer — análisis estático de la ruta entre dos recursos.
<<https://docs.aws.amazon.com/vpc/latest/reachability/what-is-reachability-analyzer.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 027 VPC, subredes, rutas, NAT, endpoints y seguridad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega vpc-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

028 — EC2, AMI, EBS y selección de capacidad

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: compute · Estado: EXECUTABLECORE

Propósito

Elegir capacidad de cómputo con criterio medible en vez de por costumbre. La familia de instancia, el tipo de volumen y el modelo de compra son tres decisiones independientes, y equivocarse en cualquiera produce o bien un cuello de botella invisible o bien una factura que triplica lo necesario.

Resultados de aprendizaje

Al finalizar podrás:

1. Descifrar el nombre de una instancia y deducir familia, generación, procesador y capacidades adicionales.
2. Elegir tipo de volumen a partir de IOPS, throughput y patrón de acceso, no por tamaño.
3. Detectar el agotamiento de créditos de una instancia ráfaga y decidir entre modo ilimitado o cambiar de familia.
4. Calcular el umbral de utilización a partir del cual compensa un plan de ahorro frente a bajo demanda.
5. Construir una AMI reproducible y explicar por qué el estado local es el enemigo de la elasticidad.

Conceptos centrales

Concepto	Comprensión verificable
familia de instancia	Perfil de recursos optimizado para un uso: propósito general, cómputo, memoria, almacenamiento o aceleración. Elegir mal la familia produce que se pague de más en la dimensión que no se usa.
crédito de CPU	Saldo que acumulan las instancias ráfaga mientras están ociosas y gastan al trabajar. Al agotarse, el rendimiento cae a la línea base sin que ninguna métrica de la aplicación lo explique.
IOPS	Operaciones de entrada/salida por segundo. Junto con el throughput y el tamaño de bloque determina el rendimiento real de un volumen: 16.000 IOPS de 4 KB no equivalen a 16.000 de 256 KB.
AMI	Imagen de máquina que define el estado inicial de una instancia. Construirla de forma reproducible es lo que permite sustituir instancias en lugar de repararlas.
instancia efímera	Instancia que puede reclamarse con dos minutos de preaviso a cambio de un descuento de hasta el 90 %. Correcta para trabajo interrumpible; nunca para el camino crítico de una petición.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    Q1{"¿Qué recurso satura?"}
    Q1 -->|"CPU"| C["familia c: cómputo"]
    Q1 -->|"memoria"| R["familia r o x"]
    Q1 -->|"equilibrado"| M["familia m"]
    Q1 -->|"E/S local"| I["familia i o d"]
    Q1 -->|"ráfagas cortas<br/>y baja media"| T["familia t"]
    T --> CR{"¿se agotan<br/>los créditos?"}
    CR -->|"sí"| U["modo ilimitado<br/>o cambiar de familia"]
    C --> P{"¿utilización sostenida?"}
    M --> P
    P -->|"> 60 %"| SP["plan de ahorro"]
    P -->|"< 60 %"| OD["bajo demanda"]
    P -->|"interrumpible"| SPOT["efímeras: -70/90 %"]
```

Desarrollo

1. El nombre de la instancia es una especificación

m7g.2xlarge no es un código arbitrario: cada carácter dice algo, y leerlo evita elegir a ciegas.

```
m      familia: propósito general (c=cómputo, r=memoria, i=E/S, t=ráfaga)
7      generación: cuanto mayor, mejor relación precio-rendimiento
g      procesador: g=Graviton (ARM), i=Intel, a=AMD; sin letra = Intel
.2xlarge tamaño: 8 vCPU y 32 GB
```

Sufijos adicionales que cambian el precio y el comportamiento:

Sufijo	Significa
d	Almacenamiento NVMe local, efímero
n	Ancho de banda de red mejorado
e	Memoria o almacenamiento ampliados
flex	Rendimiento sostenido reducido, más barato

La generación es la palanca más rentable y la más ignorada: pasar de la 5 a la 7 suele dar mejor rendimiento por menos dinero, y el cambio no requiere tocar la aplicación salvo por la arquitectura del procesador.

Sobre Graviton: es ARM, así que exige binarios para arm64. Para lenguajes interpretados o con máquina virtual —Python, Java, Node— el cambio suele ser transparente; para binarios compilados o imágenes de contenedor hay que reconstruir. La relación precio-rendimiento típicamente mejora entre un 20 % y un 40 %, así que la reconstrucción se amortiza rápido en flotas grandes.

El tamaño escala linealmente dentro de la familia: 2xlarge es exactamente el doble de xlarge en vCPU, memoria, red y precio. Eso hace que la aritmética de dimensionado sea directa.

2. Créditos de CPU: el desplome que no aparece en las métricas

Las instancias de la familia t no ofrecen su vCPU completa de forma continua. Tienen una línea base —un porcentaje del rendimiento nominal— y acumulan créditos mientras consumen por debajo de ella.

```
t3.medium: línea base 20 % por vCPU
  ociosa al 5 %   → acumula créditos
  al 80 % de CPU  → gasta créditos
  saldo a cero    → el rendimiento se limita al 20 %
```

El síntoma es característico y desconcierta: la aplicación se vuelve cuatro o cinco veces más lenta sin ningún cambio en el código, la carga ni la configuración. Ninguna métrica del sistema operativo lo explica, porque desde dentro la CPU parece ocupada y disponible.

El diagnóstico exige una métrica del proveedor, no del sistema:

```
$ aws cloudwatch get-metric-statistics --namespace AWS/EC2 \
  --metric-name CPUCreditBalance --dimensions Name=InstanceId,Value=i-0a1b2c \
  --start-time 2026-08-01T00:00:00Z --end-time 2026-08-01T12:00:00Z \
  --period 300 --statistics Minimum --query 'Datapoints[?Minimum<`10`]|length(@)'
```


Diecisiete intervalos con el saldo por debajo de 10 créditos: la instancia lleva horas limitada.

Dos salidas, con consecuencias distintas:

- modo ilimitado permite superar la línea base pagando el exceso.
Correcto para picos ocasionales; ruinoso si la carga es sostenida, porque el sobrecargo puede superar el precio de una instancia normal.
- cambiar familia si la utilización media supera la línea base de forma sostenida, la familia t es la elección equivocada.

La regla: la familia t es para cargas con media baja y picos cortos. Si la media supera la línea base, el descuento desaparece y el riesgo permanece.

3. Volúmenes: IOPS, throughput y tamaño de bloque

Elegir volumen por tamaño es el error más común. Lo que decide el rendimiento son tres números y su interacción:

Tipo	IOPS máx	Throughput máx	Precio relativo	Para
gp3	16.000	1.000 MB/s	1,0	Propósito general; IOPS y throughput independientes del tamaño
gp2	3 por GB, tope 16.000	250 MB/s	1,25	Generación anterior: el rendimiento depende del tamaño
io2	256.000	4.000 MB/s	3-5	Bases de datos exigentes, con durabilidad mayor
st1	500	500 MB/s	0,3	Lectura secuencial grande: logs, big data

La diferencia entre gp2 y gp3 es económicamente relevante: en gp2 hay que sobredimensionar el tamaño para obtener IOPS. Para 6.000 IOPS hacen falta 2.000 GB aunque solo se usen 200. En gp3 se configuran por separado:

$$\begin{aligned}
 \text{gp2 para 6.000 IOPS: } & 2.000 \text{ GB} \times 0,10 \text{ USD/GB} = 200,00 \text{ USD/mes} \\
 \text{gp3 equivalente: } & 200 \text{ GB} \times 0,08 + 3.000 \text{ IOPS extra} \times 0,005 \\
 & = 16,00 + 15,00 = 31,00 \text{ USD/mes}
 \end{aligned}$$

Un factor de 6,5 por la misma capacidad efectiva. Migrar de gp2 a gp3 es de las optimizaciones de mayor retorno y menor riesgo que existen.

Y el detalle que descoloca al medir: una operación de E/S se cuenta hasta 256 KB. Una lectura de 1 MB consume 4 IOPS, no 1. Por eso una carga secuencial agota antes el throughput que las IOPS, y una aleatoria de bloques pequeños agota antes las IOPS:

$$\begin{aligned}
 16.000 \text{ IOPS} \times 4 \text{ KB} &= 64 \text{ MB/s} \leftarrow \text{limita el IOPS} \\
 16.000 \text{ IOPS} \times 256 \text{ KB} &= 4.096 \text{ MB/s} \leftarrow \text{limita el throughput (tope 1.000)}
 \end{aligned}$$

Hay un segundo límite que se olvida: la instancia también tiene un tope de ancho de banda hacia EBS, independiente del volumen. Un gp3 de 1.000 MB/s conectado a una instancia con 600 MB/s de ancho de banda entrega 600.

4. Modelos de compra y el umbral que los separa

Cuatro formas de pagar el mismo cómputo:

Modelo	Descuento	Compromiso	Riesgo
Bajo demanda	0 %	Ninguno	Ninguno
Savings Plan 1 año	28 %	Gasto por hora, no instancia concreta	Sobredimensionar
Savings Plan 3 años	45 %	Ídem	Cambio tecnológico
Efímeras	70-90 %	Ninguno	Reclamación con 2 min de aviso

El umbral de utilización a partir del cual compensa comprometerse sale de igualar costes, como en la clase 015:

$$u \times 730 \text{ h} \times P = 730 \text{ h} \times 0,72 \text{ P} \rightarrow u = 0,72$$

Con más del 72 % de uso sostenido, el plan de un año sale a cuenta. Y hay un matiz que lo hace más atractivo de lo que parece: los Savings Plans de cómputo comprometen gasto por hora, no un tipo de instancia. Se puede cambiar de

familia, de tamaño y de región sin perder el descuento, lo que elimina buena parte del riesgo de sobredimensionar.

Las instancias efímeras merecen su propio criterio. El descuento es enorme y el riesgo concreto: dos minutos de preaviso. Son correctas para trabajo por lotes reanudable, transcodificación, entrenamiento con puntos de control y entornos de prueba. No lo son para nada que sostenga una petición de usuario, salvo que la arquitectura absorba la pérdida sin degradación visible.

La estrategia habitual en una flota:

base estable (60 % de la capacidad)	→ Savings Plan
variación previsible (25 %)	→ bajo demanda
picos y lotes (15 %)	→ efímeras

5. El estado local es el enemigo de la elasticidad

Una instancia que guarda estado en su disco no se puede sustituir: hay que repararla. Y reparar es lo contrario de operar en cloud.

La distinción práctica, con lo que hay que hacer con cada cosa:

logs	→ fuera del host, a un recolector (clase 007)
sesiones	→ almacén compartido, no memoria del proceso
ficheros	→ almacenamiento de objetos o sistema de ficheros compartido
caché	→ externa; si es local, debe poder perderse sin consecuencias
datos	→ base de datos gestionada
configuración	→ parámetros o secretos, inyectados al arrancar

Si todo eso está fuera, una instancia es desechable: se puede terminar y crear otra sin ceremonia. Esa propiedad es la que hacen posibles el autoescalado, los despliegues sin interrupción y el parcheo por sustitución.

Y exige una AMI reproducible. Construirla a mano —arrancar, instalar, guardar imagen— produce una imagen que nadie sabe recrear:

```
# Fragmento de plantilla declarativa
source "amazon-eks" "cloudshop" {
  source_ami_filter {
    filters = { name = "al2023-ami-*-arm64" }
    owners  = ["amazon"]
    most_recent = true
  }
  instance_type = "t4g.small"
  ami_name      = "cloudshop-{{timestamp}}"
}
```

Dos propiedades que hay que exigir a la imagen:

1. Reproducible: la misma plantilla produce una imagen funcionalmente equivalente.
2. Fechada e inmutable: nunca se sobrescribe una AMI; se crea otra y se cambia la referencia. Es la misma lógica de la clase 003 sobre etiquetas mutables.

Y una advertencia de seguridad: una AMI compartida por error es pública para siempre a efectos prácticos. Antes de compartir hay que comprobar que no contiene claves, historial de shell ni datos residuales.

Ejemplo trabajado

El servicio de catálogo de CloudShop se degrada cada día entre las 11:00 y las 14:00. No hay despliegues ni aumento de tráfico en esa franja.

Las métricas de la aplicación no explican nada:

peticiones/s	estables en 340
tasa de error	0,02 %, sin cambio
CPU del sistema	94 % durante la degradación
latencia p95	88 ms → 410 ms

La CPU al 94 % sugiere saturación, pero el tráfico no subió. Se mira la métrica del proveedor:

```
$ aws cloudwatch get-metric-statistics --namespace AWS/EC2 \
  --metric-name CPUCreditBalance --dimensions Name=InstanceId,Value=i-0a1b2c \
  --start-time 2026-08-01T08:00:00Z --end-time 2026-08-01T15:00:00Z \
  --period 3600 --statistics Average --query 'Datapoints[][Timestamp,Average]' --output text | sort
2026-08-01T08:00:00Z    142.0
```

2026-08-01T10:00:00Z	38.0	
2026-08-01T11:00:00Z	0.0	← agotados
2026-08-01T14:00:00Z	0.0	

Créditos agotados a las 11:00. Las instancias son t3.large con línea base del 30 %; al agotar el saldo, el rendimiento se limita a esa fracción:

rendimiento nominal	2 vCPU
línea base 30 %	0,6 vCPU efectivas
factor de degradación	3,3×
latencia observada	88 × 3,3 = 290 ms teórico; medido 410 con encolamiento

Se comprueba la utilización media real para elegir la salida:

utilización media diaria	47 %
línea base de t3.large	30 %

La media supera la línea base, así que la familia t es la elección equivocada: el modo ilimitado cobraría el exceso todo el día. Se dimensiona una alternativa por la ley de Little:

$\lambda = 340$ peticiones/s, $W = 0,088$ s $\rightarrow L = 30$ concurrentes
a $\rho = 0,70 \rightarrow 43$ concurrentes $\rightarrow$ con 16 por instancia, 3 instancias

Comparación de tres opciones para 3 instancias:

3 × t3.large (actual)	6	182,50 USD	no: se limita
3 × t3.large modo ilimitado	6	182,50 + ~95 sobrecargo = 277,50	
3 × m7g.large (Graviton)	6	139,00 USD	sí, sostenido

m7g.large cuesta menos que la opción actual degradada y sostiene el rendimiento. Requiere reconstruir la imagen para arm64; la aplicación es Python y la reconstrucción resultó transparente salvo por una dependencia con extensión nativa.

Se aprovecha para revisar los volúmenes:

```
$ aws ec2 describe-volumes --filters Name=attachment.instance-id,Values=i-0a1b2c \
--query 'Volumes[][VolumeType,Size,Iops]' --output text
gp2      500      1500

gp2 de 500 GB: 1.500 IOPS, uso real 92 GB  $\rightarrow$  se pagaba tamaño para obtener IOPS
gp3 de 120 GB con 3.000 IOPS configurados:
  gp2: 500 × 0,10 = 50,00 USD/mes
  gp3: 120 × 0,08 = 9,60 USD/mes
  (3.000 IOPS entran en la base gratuita)
  ahorro por instancia = 40,40 USD/mes
```

Resultado combinado:

instancias	3 × t3.large	3 × m7g.large	
cómputo	182,50 USD	139,00 USD	
almacenamiento	150,00 USD	28,80 USD	
total	332,50 USD	167,80 USD	(-50 %)
latencia p95 11:00-14:00	410 ms	91 ms	
degradación diaria	3 horas	ninguna	

El diagnóstico no estaba en la aplicación ni en el sistema operativo: estaba en una métrica que solo el proveedor publica. Y la corrección salió más barata que el problema.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/028-ec2-ami-ebs-y-seleccion-de-capacidad/lab.py
```

El laboratorio selecciona el motor de práctica compute y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa instancia-aws en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una selección de capacidad justificada y observable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye instancia-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El rendimiento se desploma a diario sin cambios de código ni de carga	Agotamiento de créditos de CPU en una instancia ráfaga	Vigila <code>CPUCreditBalance</code> ; si la media supera la línea base, cambia de familia en vez de activar modo ilimitado.
Se paga un volumen enorme del que se usa una fracción	En <code>gp2</code> las IOPS dependen del tamaño, así que hay que sobredimensionar	Migra a <code>gp3</code> y configura IOPS y throughput por separado; el ahorro suele superar el 60 %.
Un volumen con 1.000 MB/s entrega bastante menos	La instancia tiene su propio tope de ancho de banda hacia EBS	Comprueba el límite de la instancia además del volumen; el menor de los dos manda.
Una instancia no se puede sustituir sin perder datos	Guarda estado local: sesiones, ficheros o logs	Externaliza todo el estado; una instancia que no es desechable impide el autoescalado y el parcheo por sustitución.
Un trabajo en instancias efímeras pierde horas de progreso	Se usó capacidad interrumpible sin puntos de control	Reserva las efímeras para trabajo reanudable con checkpoints; el preaviso es de dos minutos.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Descompón `c7gn.4xlarge`: ¿qué dice cada parte del nombre?
2. Una instancia `t3` tiene utilización media del 47 % y línea base del 30 %. ¿Conviene modo ilimitado? Justifica.
3. ¿Cuántas IOPS consume una lectura secuencial de 1 MB, y por qué eso cambia qué límite alcanzas antes?
4. ¿Por qué `gp3` con 200 GB puede rendir más que `gp2` con 2.000 GB y costar seis veces menos?
5. ¿A partir de qué utilización sostenida compensa un Savings Plan de un año, y qué riesgo elimina que comprometa gasto y no tipo de instancia?

Referencias

- AWS (2024). Amazon EC2 instance types — nomenclatura, familias y capacidades adicionales.
<<https://docs.aws.amazon.com/ec2/latest/instancetypeguide/instance-types.html>>
- AWS (2024). Burstable performance instances — línea base, créditos y modo ilimitado.
<<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/burstable-performance-instances.html>>
- AWS (2024). Amazon EBS volume types — IOPS, throughput y tamaño de operación de E/S.
<<https://docs.aws.amazon.com/ebs/latest/userguide/ebs-volume-types.html>>
- AWS (2024). Savings Plans — compromiso por gasto horario frente a instancias reservadas.
<<https://docs.aws.amazon.com/savingsplans/latest/userguide/what-is-savings-plans.html>>
- AWS (2024). Spot Instances: interruption notices — preaviso de dos minutos y patrones de uso seguro.
<<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-interruptions.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 028 EC2, AMI, EBS y selección de capacidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega instancia-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

029 — Elastic Load Balancing y Auto Scaling

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Construir una capa elástica que reparta tráfico y ajuste capacidad sin oscilar ni tumbar el servicio al desplegar. Aquí se aplican a AWS los cálculos de la clase 016 —histéresis, margen de arranque, rechazo de carga— y se añade lo que ninguna fórmula anticipa: que el drenado de conexiones y el tipo de comprobación de estado deciden si un despliegue pierde peticiones.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre balanceador de aplicación y de red según protocolo, latencia y necesidad de terminar TLS.
2. Configurar comprobaciones de estado que distingan una instancia arrancando de una averiada.
3. Dimensionar un grupo de autoescalado con umbrales, enfriamiento y periodo de gracia coherentes.
4. Evitar la pérdida de peticiones al reducir capacidad, mediante drenado de conexiones.
5. Diagnosticar un 502 y un 504 del balanceador sabiendo qué componente señala cada uno.

Conceptos centrales

Concepto	Comprensión verificable
grupo de destino	Conjunto de destinos que reciben tráfico, con su propia comprobación de estado y política de enrutamiento. Es la unidad sobre la que actúan el balanceador y el autoescalado.
comprobación de estado	Sondeo periódico que decide si un destino recibe tráfico. Debe reflejar readiness —¿puede servir?— y no liveness, por lo visto en la clase 012.
drenado de conexiones	Periodo durante el cual un destino que se retira deja de recibir peticiones nuevas pero termina las que tiene en curso. Sin él, reducir capacidad corta transacciones a medias.
periodo de gracia	Tiempo que el autoescalado espera antes de evaluar la salud de una instancia recién creada. Corto de más, mata instancias que aún arrancan y entra en bucle.
seguimiento de objetivo	Política de escalado que ajusta capacidad para mantener una métrica en un valor deseado. Gestiona la histéresis automáticamente, a diferencia de las políticas por umbral.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
C["clientes"] --> ALB["Balanceador de aplicación<br/>termina TLS · enruta por ruta"]
ALB --> TG["Grupo de destino<br/>health check /health/ready"]
TG --> I1["instancia 1 · healthy"]
TG --> I2["instancia 2 · healthy"]
TG --> I3["instancia 3 · draining<br/>termina lo que tiene"]
ASG["Auto Scaling"] -->|"crea y retira"| TG
ASG --> M{"métrica objetivo:<br/>peticiones por destino"}
M -->|"por encima"| UP["+1 instancia<br/>gracia 180 s"]
M -->|"por debajo"| DOWN["-1 instancia<br/>drenado 60 s"]
```

Desarrollo

1. Cuatro balanceadores, tres decisiones

	Aplicación (ALB)	Red (NLB)	Gateway (GWLB)
Capa	7 (HTTP)	4 (TCP/UDP)	3 (GENEVE)
Enruta por	Ruta, host, cabecera, método	Puerto	Todo el paquete
Termina TLS	Sí	Sí, opcionalmente	No
IP fija	No	Sí, por zona	No
Latencia añadida	5-10 ms	1 ms	Depende
Preserva IP de origen	En cabecera X-Forwarded-For	En el paquete	Sí

Tres criterios deciden:

Protocolo. Si es HTTP y hace falta enrutar por ruta o host, ALB. Si es TCP, UDP o un protocolo propio, NLB.

IP fija. El NLB tiene una IP estática por zona; el ALB resuelve por DNS y sus IP cambian. Esto importa cuando un cliente externo debe incluir tu servicio en una lista de permitidos: con ALB no hay IP estable que dar.

IP de origen. El NLB preserva la IP del cliente en el propio paquete; el ALB la pone en X-Forwarded-For. Una aplicación que aplica límites de tasa por IP y lee la IP de la conexión verá la del balanceador —y limitará a todos como si fueran uno— salvo que lea la cabecera. Es un fallo que solo aparece bajo ataque.

Y un detalle de coste: ambos cobran por hora y por unidad de capacidad consumida, una métrica compuesta de conexiones nuevas, conexiones activas, ancho de banda y reglas evaluadas. La dimensión que domine determina la factura, y suele ser conexiones nuevas por segundo en servicios que no reutilizan conexión —lo de la clase 005—.

2. La comprobación de estado debe reflejar readiness

Un balanceador retira del reparto lo que falla la comprobación. Por tanto la comprobación debe responder «¿puede servir ahora?» y no «¿está el proceso vivo?».

```
camino          /health/ready      ← no / ni /health/live
intervalo       10 s
plazo           5 s                ← menor que el intervalo
umbral sano     2 comprobaciones  ← vuelve rápido
umbral insano   3 comprobaciones  ← no se va por un fallo aislado
códigos válidos 200
```

La asimetría entre los dos umbrales es deliberada: entrar rápido y salir despacio. Un fallo aislado no debe retirar un destino sano, pero un destino que se recupera debe volver a recibir tráfico cuanto antes.

El tiempo de detección se calcula:

```
detección de fallo = intervalo × umbral insano = 10 × 3 = 30 s
```

Treinta segundos durante los cuales una fracción del tráfico va a un destino averiado. Bajarlo tiene coste: intervalos muy cortos generan carga de sondeo y falsos positivos bajo saturación. Con nueve destinos y 10 segundos son 54 sondeos por minuto, despreciable; con 200 destinos y 5 segundos son 2.400, que ya se nota.

El error que convierte una degradación en caída total, ya visto en la clase 012: si /health/ready comprueba la base de datos y esta cae, los nueve destinos fallan a la vez y el balanceador se queda sin ninguno sano. AWS mitiga esto con el modo fail open —si ningún destino está sano, envía tráfico a todos—, pero apoyarse en eso es apoyarse en un comportamiento de emergencia. Lo correcto es que la comprobación distinga dependencias esenciales de opcionales.

3. Autoescalado: los cuatro parámetros y su interacción

El seguimiento de objetivo es preferible a los umbrales manuales porque gestiona la histéresis solo: se declara el valor deseado de una métrica y el servicio calcula los ajustes.

```
$ aws autoscaling put-scaling-policy --auto-scaling-group-name cloudshop-asg \
  --policy-name seguimiento-peticiones --policy-type TargetTrackingScaling \
  --target-tracking-configuration '{
    "PredefinedMetricSpecification": {
```

```

    "PredefinedMetricType": "ALBRequestCountPerTarget",
    "ResourceLabel": "app/cloudshop-alb/50dc.../targetgroup/cloudshop-tg/6d0e..."
  },
  "TargetValue": 420.0,
  "ScaleInCooldown": 300,
  "ScaleOutCooldown": 60
}'

```

Cuatro decisiones en ese documento:

La métrica. `ALBRequestCountPerTarget` es mejor que la CPU para servicios web: mide demanda directamente y no depende de que el trabajo sea de cómputo. Una carga limitada por E/S nunca dispara un escalado basado en CPU aunque esté saturada.

El valor objetivo. Se deriva de la capacidad medida por destino y del objetivo de utilización de la clase 016:

capacidad por destino a saturación	600 peticiones/s
objetivo de utilización	0,70
valor objetivo	420 peticiones/s

Los enfriamientos asimétricos. Subir rápido (60 s) y bajar despacio (300 s). Retirar capacidad demasiado pronto es lo que produce oscilación, y el coste de sobrar una instancia unos minutos es mucho menor que el de faltar.

El periodo de gracia, que se configura en el grupo:

```

periodo de gracia > tiempo de arranque hasta servir tráfico
medido: 118 s → se fija en 180 s

```

Si la gracia es menor que el arranque, el grupo marca insanas a las instancias nuevas, las termina y crea otras: un bucle que consume dinero y nunca converge.

4. Drenado: por qué reducir capacidad pierde peticiones

Cuando el autoescalado retira una instancia, el balanceador deja de enviarle peticiones nuevas pero puede tener transacciones en curso. Sin drenado, esas se cortan.

```

drenado 0 s    la instancia se termina de inmediato
               → peticiones en vuelo cortadas → errores para el usuario
drenado 60 s   deja de recibir nuevas, termina las que tiene
               → cero errores si ninguna dura más de 60 s

```

El valor debe ser mayor que la petición más larga:

```

$ aws elbv2 modify-target-group-attributes --target-group-arn arn:...:targetgroup/... \
  --attributes Key=deregistration_delay.timeout_seconds,Value=60

```

Y hay una segunda pieza que casi siempre falta: la aplicación debe cooperar. El balanceador deja de enviar tráfico, pero el orquestador envía `SIGTERM` a la instancia. Si el proceso no lo atrapa y cierra de inmediato, el drenado del balanceador no sirve de nada —lo de la clase 002—:

```

def apagar(signum, frame):
    servidor.stop_accepting()      # no aceptar conexiones nuevas
    servidor.wait_for_inflight(55) # terminar las en curso, con margen
    sys.exit(0)

signal.signal(signal.SIGTERM, apagar)

```

Hay un orden que importa y que se equivoca a menudo: el proceso debe primero fallar la comprobación de readiness y solo después dejar de aceptar conexiones. Si deja de aceptar antes de que el balanceador se entere, hay una ventana de segundos en la que el balanceador sigue enviando tráfico a un puerto cerrado, y el usuario recibe un 502.

La secuencia correcta, con sus tiempos:

```

t+0   llega SIGTERM
t+0   /health/ready empieza a devolver 503
t+30  el balanceador ha detectado el fallo (intervalo × umbral) y deja de enviar
t+30  el proceso deja de aceptar conexiones nuevas
t+85  terminan las peticiones en curso
t+85  el proceso sale con 0

```

5. 502 y 504 del balanceador señalan cosas distintas

Los errores del balanceador se distinguen de los de la aplicación en las métricas y en los logs, y confundirlos alarga los incidentes:

Código	Métrica de CloudWatch	Qué ocurrió	Dónde mirar
502	HTTPCodeELB502Count	El destino cerró la conexión o devolvió algo inválido	Logs del destino
503	HTTPCodeELB503Count	No hay destinos sanos	Comprobación de estado
504	HTTPCodeELB504Count	El destino no respondió dentro del plazo	Latencia y saturación del destino

La causa más frecuente de 502 sin ningún error en la aplicación es una discordancia de tiempos de espera:

```
tiempo de espera del balanceador 60 s
tiempo de espera keep-alive del destino 5 s ← MENOR
```

El destino cierra la conexión inactiva a los 5 segundos; el balanceador, que la creía viva, envía una petición por ella y recibe un cierre. El resultado es un 502 intermitente, con más frecuencia cuanto menor sea el tráfico —porque las conexiones se quedan inactivas—.

La regla: el keep-alive del destino debe ser mayor que el tiempo de espera de inactividad del balanceador. Con 60 s en el balanceador, 75 s en el destino.

Y los logs del balanceador dan el detalle que las métricas no:

```
request_processing_time target_processing_time response_processing_time
0.001 -1 -1
```

Un -1 en targetprocessingtime significa que el destino nunca respondió: el problema no está en la aplicación sino en la conexión hacia ella. Distinguir eso de un 59.998 —el destino tardó y agotó el plazo— cambia por completo dónde hay que buscar.

Ejemplo trabajado

Cada despliegue de CloudShop produce entre 200 y 400 errores 502, y el autoescalado oscila creando y destruyendo instancias. Se atacan los dos problemas.

Problema 1 — errores en cada despliegue.

```
$ aws elbv2 describe-target-group-attributes --target-group-arn arn:...:targetgroup/cloudshop-tg \
--query 'Attributes[?Key==`deregistration_delay.timeout_seconds`].Value' --output text
0
```

Drenado a cero: las instancias se terminan con peticiones en vuelo. Se mide la duración de las peticiones para elegir el valor:

```
p50 38 ms
p95 91 ms
p99 185 ms
máx 4,2 s (exportación de informes)
```

Se fija en 60 s, con margen amplio sobre los 4,2 s del peor caso.

Pero al repetir el despliegue siguen apareciendo 502, solo que menos:

```
antes del cambio 340 errores
después 96 errores
```

La aplicación no atrapaba SIGTERM. Se añade el manejador y se comprueba el orden:

```
versión inicial: SIGTERM → cierra el socket inmediatamente
                  el balanceador tarda 30 s en enterarse → 502 durante 30 s
versión correcta: SIGTERM → /health/ready devuelve 503
                        espera 35 s (intervalo 10 × umbral 3 + margen)
                        deja de aceptar y drena lo en curso
```

```
tras el arreglo completo: 0 errores en 5 despliegues consecutivos
```

Problema 2 — oscilación del autoescalado.

```
$ aws autoscaling describe-scaling-activities --auto-scaling-group-name cloudshop-asg \
--max-items 8 --query 'Activities[][StartTime,Description]' --output text
2026-08-01T11:42 Terminating EC2 instance: i-0f3a
```

```
2026-08-01T11:39 Launching a new EC2 instance: i-0f3a
2026-08-01T11:36 Terminating EC2 instance: i-0c8b
2026-08-01T11:33 Launching a new EC2 instance: i-0c8b
```

Crea y destruye cada 3 minutos. La configuración:

```
política          umbral simple: sube al 70 % de CPU, baja al 65 %
enfriamiento      120 s
gracia            60 s
tiempo de arranque medido 118 s
```

Dos fallos independientes:

1. Histéresis insuficiente: con 8 instancias al 71 %, añadir una baja la utilización a 63 % → dispara la reducción → vuelve a 71 %.
2. Gracia (60 s) < arranque (118 s): las instancias nuevas se marcan insanas antes de poder servir y se terminan.

Se sustituye por seguimiento de objetivo sobre peticiones por destino:

```
capacidad medida por destino a saturación 600 peticiones/s
objetivo 0,70                             420 peticiones/s
enfriamiento de subida                     60 s
enfriamiento de bajada                     300 s
periodo de gracia                          180 s (> 118 medidos)
```

Se elige peticiones por destino y no CPU porque el servicio pasa el 60 % del tiempo esperando a la base de datos: la CPU nunca refleja la saturación real.

Resultado en 7 días:

errores 502 por despliegue	340	0
actividades de escalado/día	48	6
instancias en régimen	6-11 osc.	7-9
p95 en hora punta	410 ms	94 ms
coste mensual de cómputo	412 USD	338 USD

El coste bajó un 18 % al dejar de oscilar: cada instancia creada y destruida a los 3 minutos se facturaba igual y no llegaba a servir tráfico útil.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/029-elastic-load-balancing-y-auto-scaling/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-elastico-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye servicio-elastico-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.

- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Cada despliegue produce cientos de 502	Drenado a cero: las instancias se terminan con peticiones en vuelo	Fija el drenado por encima de la petición más larga y atrapa SIGTERM en la aplicación.
Se configura el drenado y siguen apareciendo 502, aunque menos	La aplicación cierra el socket antes de que el balanceador detecte el cambio de estado	Al recibir SIGTERM, falla readiness primero y espera intervalo × umbral antes de dejar de aceptar.
El autoescalado crea y destruye instancias cada pocos minutos	Umbral de subida y bajada demasiado próximos, o gracia menor que el arranque	Usa seguimiento de objetivo y fija la gracia por encima del arranque medido.
El servicio se satura y el autoescalado no reacciona	La métrica es la CPU y la carga está limitada por E/S	Escala por peticiones por destino o por una métrica que refleje la demanda real.
Aparecen 502 intermitentes, más frecuentes con poco tráfico	El keep-alive del destino es menor que el tiempo de espera del balanceador	Configura el keep-alive del destino por encima del plazo de inactividad del balanceador.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿En qué dos casos concretos necesitas un balanceador de red en vez de uno de aplicación?
2. Con intervalo de 10 s y umbral insano de 3, ¿cuánto tarda en detectarse un fallo y qué ocurre entretanto?
3. ¿Por qué los umbrales sano e insano deben ser asimétricos, y en qué dirección?
4. Describe la secuencia correcta desde que llega SIGTERM hasta que el proceso sale sin perder peticiones.
5. Un log del balanceador muestra targetprocessingtime = -1. ¿Qué significa y dónde buscas?

Referencias

- AWS (2024). Application Load Balancer: target groups and health checks — parámetros y semántica. <<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/target-group-health-checks.html>>
- AWS (2024). Deregistration delay — drenado de conexiones y su interacción con el ciclo de vida. <<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/edit-target-group-attributes.html>>
- AWS (2024). Target tracking scaling policies — métricas predefinidas y enfriamientos. <<https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-scaling-target-tracking.html>>
- AWS (2024). Troubleshoot your Application Load Balancer — causas de 502, 503 y 504 y campos del log de acceso. <<https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-troubleshooting.html>>
- Beyer, B. et al., eds. (2016). Site Reliability Engineering, cap. 20 — reparto de carga y comprobaciones de salud. <<https://sre.google/sre-book/load-balancing-datacenter/>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 029 Elastic Load Balancing y Auto Scaling

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega servicio-elastico-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

030 — S3: objetos, versionado, lifecycle y replicación

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

Propósito

Operar almacenamiento de objetos entendiendo qué garantiza y qué no. Los once nuevos de durabilidad no protegen de un borrado autorizado —lo estableció la clase 010— y el modelo de acceso de S3 es el origen de una parte desproporcionada de las brechas públicas atribuidas a la nube. Aquí se convierte todo eso en configuración verificable.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar por qué el versionado y el bloqueo de objetos son controles distintos y qué protege cada uno.
2. Diseñar una política de ciclo de vida que reduzca coste sin romper la recuperación ni el cumplimiento.
3. Calcular si una transición a una clase más fría ahorra, contando el mínimo de días y el coste de recuperación.
4. Bloquear el acceso público en todos los niveles y verificarlo con prueba negativa.
5. Distinguir replicación de copia de seguridad, y por qué la primera propaga los borrados.

Conceptos centrales

Concepto	Comprensión verificable
versionado	Conservar todas las versiones de un objeto, incluidas las borradas mediante un marcador. Protege del error humano, pero no de quien tenga permiso para borrar versiones.
bloqueo de objetos	Retención inmutable que impide borrar o sobrescribir durante un periodo. En modo cumplimiento ni siquiera la cuenta raíz puede saltárselo, que es lo que lo hace útil frente a ransomware.
clase de almacenamiento	Perfil de coste y acceso: estándar, infrecuente, archivo. Las frías cobran menos por GB y añaden coste de recuperación, mínimos de duración y a veces latencia de horas.
marcador de borrado	Versión especial que oculta un objeto sin eliminarlo. Explica por qué un bucket versionado sigue creciendo y costando después de «vaciarlo».
punto de acceso	Endpoint con su propia política, asociado a un bucket. Permite dar permisos acotados por aplicación sin que la política del bucket crezca sin control.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    P["PUT objeto"] --> V1["¿versionado?"]
    V1 -->|si| V1_1["versión nueva<br/>las anteriores se conservan"]
    V1 -->|no| S1["sobrescribe<br/>el anterior se pierde"]
    D["DELETE objeto"] --> V2["¿versionado?"]
    V2 -->|si| M["marcador de borrado<br/>el objeto sigue ahí y se paga"]
    V2 -->|no| X["eliminado"]
    M -->|DeleteObjectVersion| X
    X -->|bloqueo de objetos<br/>en modo cumplimiento| B["IMPOSIBLE hasta<br/>que expire la retención"]
```

Desarrollo

1. Versionado y bloqueo protegen de cosas distintas

Se confunden y cubren riesgos que no se solapan:

Riesgo	Versionado	Bloqueo de objetos
Sobrescritura accidental	✓	✓
Borrado accidental	✓	✓
Borrado deliberado con permisos	✗	✓
Ransomware con credenciales válidas	✗	✓
Corrupción por fallo de disco	✓ (11 nuevos)	✓

Las dos filas centrales son la diferencia. Con versionado, quien tenga `s3:DeleteObjectVersion` borra todas las versiones y el objeto desaparece de verdad. El bloqueo en modo cumplimiento lo impide hasta que expire la retención, y no admite excepción para nadie —incluida la cuenta raíz—.

```
$ aws s3api put-object-lock-configuration --bucket cloudshop-facturas \
  --object-lock-configuration '{"ObjectLockEnabled":"Enabled",
  "Rule":{"DefaultRetention":{"Mode":"COMPLIANCE","Days":2555}}}'
```

Los dos modos no son intercambiables:

GOVERNANCE quien tenga `s3:BypassGovernanceRetention` puede saltárselo.
Útil para protección operativa con escape controlado.

COMPLIANCE nadie puede. Ni el administrador, ni la raíz, ni AWS.
Es el único que protege frente a un atacante con permisos.

Y una advertencia que hay que entender antes de activarlo: el modo cumplimiento no se puede reducir ni desactivar. Un objeto con 7 años de retención ocupará y costará 7 años. Es exactamente lo que se quiere para evidencia legal y una trampa cara si se aplica por defecto a todo un bucket de datos operativos.

El bloqueo exige versionado activado y solo puede habilitarse al crear el bucket, o mediante una solicitud a soporte. Otra razón para decidirlo al principio.

2. El marcador de borrado explica la factura que no baja

En un bucket versionado, DELETE no borra: crea un marcador que oculta el objeto. Las versiones anteriores siguen ahí y siguen facturándose.

```
$ aws s3 rm s3://cloudshop-datos/informe.csv
delete: s3://cloudshop-datos/informe.csv
$ aws s3 ls s3://cloudshop-datos/informe.csv
# vacío: parece borrado
$ aws s3api list-object-versions --bucket cloudshop-datos --prefix informe.csv \
  --query '[Versions[].[Size,VersionId],DeleteMarkers[].VersionId]'
[[[4823901, "3HL4kqt..."], ["Ktr5nQ..."]]]
```

El objeto de 4,8 MB sigue existiendo y pagándose. En un bucket con años de operación, las versiones no vigentes pueden ser la mayor parte del almacenamiento:

```
$ aws s3api list-object-versions --bucket cloudshop-datos --output json \
  | jq '{vigentes: ([.Versions[]|select(.IsLatest)|.Size]|add),
  antiguas: ([.Versions[]|select(.IsLatest|not)|.Size]|add)}'
{"vigentes": 412000000000, "antiguas": 1840000000000}
```

1,84 TB de versiones antiguas frente a 412 GB vigentes: el 82 % del coste es historial.

La solución no es desactivar el versionado —se perdería la protección— sino caducar lo antiguo con ciclo de vida:

```
{
  "Rules": [{
    "ID": "caducar-versiones-antiguas",
    "Status": "Enabled",
    "Filter": {"Prefix": ""},
    "NoncurrentVersionExpiration": {"NoncurrentDays": 90, "NewerNoncurrentVersions": 3},
    "Expiration": {"ExpiredObjectDeleteMarker": true},
    "AbortIncompleteMultipartUpload": {"DaysAfterInitiation": 7}
```

```
}]
}
```

La última regla merece mención aparte: las subidas multiparte abandonadas no aparecen en ningún listado normal y se facturan indefinidamente. Es una partida invisible que en buckets con mucha escritura fallida puede alcanzar cientos de gigabytes.

3. Clases de almacenamiento: el ahorro que a veces no lo es

Clase	USD/GB/mes	Recuperación	Mínimo	Latencia
Standard	0,023	0	—	ms
Intelligent-Tiering	0,023 → 0,0125	0	—	ms
Standard-IA	0,0125	0,01 USD/GB	30 días	ms
Glacier Instant	0,004	0,03 USD/GB	90 días	ms
Glacier Flexible	0,0036	0,01-0,03	90 días	min-horas
Deep Archive	0,00099	0,02 USD/GB	180 días	12 h

Las dos columnas de la derecha son las que invalidan la mitad de las transiciones que se configuran:

El mínimo de duración se factura completo. Un objeto movido a Standard-IA y borrado a los 10 días paga 30. Para datos con vida corta, la transición cuesta más de lo que ahorra.

La recuperación se cobra. Si los datos se leen con frecuencia, el coste de acceso supera el ahorro de almacenamiento:

```
1 TB en Standard:      1.024 × 0,023 = 23,55 USD/mes
1 TB en Standard-IA:   1.024 × 0,0125 = 12,80 USD/mes
ahorro bruto              = 10,75 USD/mes
```

```
umbral de lecturas: 10,75 / 0,01 USD/GB = 1.075 GB/mes
→ si se lee más del 105 % del volumen al mes, Standard-IA sale MÁS CARO
```

Y hay un coste que se olvida: la propia transición cuesta unos 0,01 USD por cada 1.000 objetos. Con 40 millones de objetos pequeños, mover a otra clase cuesta 400 USD de una vez, y si los objetos son de 20 KB el ahorro mensual puede no compensarlo nunca.

De ahí que Intelligent-Tiering sea razonable cuando el patrón de acceso es desconocido: mueve automáticamente según el uso real, sin coste de recuperación, a cambio de una pequeña tarifa de monitorización por objeto que solo compensa con objetos mayores de 128 KB.

4. Bloquear el acceso público en todos los niveles

Las brechas de almacenamiento expuesto siguen ocurriendo porque hay cuatro mecanismos independientes que pueden conceder acceso público, y bloquear uno no bloquea los demás:

1. ACL del bucket
2. ACL del objeto
3. Política del bucket
4. Política del punto de acceso

El bloqueo de acceso público los cubre todos, y debe aplicarse a nivel de cuenta, no solo de bucket:

```
$ aws s3control put-public-access-block --account-id 123456789012 \
  --public-access-block-configuration \
    'BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true'
```

Los cuatro parámetros hacen cosas distintas y hay que entender por qué son cuatro:

```
BlockPublicAcls      impide CREAR ACL públicas nuevas
IgnorePublicAcls     IGNORA las ACL públicas que ya existan
BlockPublicPolicy     impide crear políticas públicas nuevas
RestrictPublicBuckets bloquea el acceso a través de políticas ya existentes
```

Las de «crear» no arreglan lo que ya está mal; las de «ignorar» y «restringir» sí. Aplicar solo las dos primeras deja abierto lo que ya estaba abierto, que es exactamente el caso de la brecha de la clase 010.

La verificación no puede ser por inspección:

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://cloudshop-facturas.s3.amazonaws.com/f-1042.pdf
403
$ aws s3api put-bucket-acl --bucket cloudshop-facturas --acl public-read
AccessDenied ... blocked by the public access block
$ aws s3api get-object --bucket cloudshop-facturas --key f-1042.pdf /dev/null
{"ContentLength": 48219}
```

✓ sin credenciales
✓ no se puede abrir
✓ con credenciales sí

Las tres pruebas juntas: niega sin credenciales, impide abrirlo, y permite el acceso legítimo. Solo la primera no demuestra nada — el objeto podría estar simplemente mal nombrado.

5. Replicación no es copia de seguridad

La confusión es cara. La replicación copia cambios a otro bucket, y un borrado es un cambio:

replicación protege de: pérdida de una región, latencia de lectura lejana
NO protege de: borrado, cifrado por ransomware, corrupción lógica
porque propaga esas operaciones al destino

copia de seguridad protege de: todo lo anterior
porque es un punto en el tiempo aislado del origen

Hay un matiz importante: por defecto la replicación no replica los borrados —los marcadores de borrado no se replican salvo que se active DeleteMarkerReplication—. Eso da una protección parcial, pero no cubre el caso peligroso: si un atacante borra versiones con DeleteObjectVersion, esa operación sí se propaga.

La configuración que sí resiste:

origen versionado + bloqueo de objetos + política que deniega DeleteObjectVersion
destino cuenta DISTINTA, con sus propias credenciales y su propio bloqueo

La cuenta distinta es la pieza decisiva, por lo visto en la clase 017: si el destino está en la misma cuenta, unas credenciales comprometidas alcanzan ambos. Con cuentas separadas, el atacante necesita comprometer dos.

Y como estableció la clase 019, nada de esto vale sin la prueba:

```
$ aws s3api list-object-versions --bucket cloudshop-backup --prefix f-1042.pdf \
--query 'Versions[0].[VersionId,LastModified]' --output text
3sL9mQt... 2026-08-01T07:12:44Z
$ aws s3api get-object --bucket cloudshop-backup --key f-1042.pdf \
--version-id 3sL9mQt... /tmp/r.pdf && sha256sum /tmp/r.pdf
```

Restaurar y comparar la suma de comprobación con el original. Que exista la copia no demuestra que sea recuperable ni que sea correcta.

Ejemplo trabajado

Auditoría del almacenamiento de CloudShop: 2,25 TB facturados, un bucket con facturas de clientes y ninguna prueba de recuperación.

Hallazgo 1 — el 82 % del coste es historial invisible:

```
$ aws s3api list-object-versions --bucket cloudshop-datos --output json \
| jq '{vigentes:(.[.Versions[]|select(.IsLatest)|.Size]|add),
      antiguas:(.[.Versions[]|select(.IsLatest|not)|.Size]|add),
      marcadores:(.DeleteMarkers|length)}'
{"vigentes": 412000000000, "antiguas": 1840000000000, "marcadores": 28417}

vigentes    412 GB × 0,023 = 9,48 USD/mes
antiguas    1.840 GB × 0,023 = 42,32 USD/mes ← el 82 %
```

Se añaden subidas multipart abandonadas, que no salían en ningún listado:

```
$ aws s3api list-multipart-uploads --bucket cloudshop-datos --query 'length(Uploads)'
1247
```

1.247 subidas incompletas, algunas de 2024, facturándose desde entonces.

Ciclo de vida aplicado:

```
{
  "Rules": [
    {
      "ID": "limpieza",
      "Status": "Enabled",
      "Filter": {
        "Prefix": ""
      },
      "NoncurrentVersionExpiration": {
        "NoncurrentDays": 90,
        "NewerNoncurrentVersions": 3
      },
      "Expiration": {
        "ExpiredObjectDeleteMarker": true
      },
      "AbortIncompleteMultipartUpload": {
        "DaysAfterInitiation": 7
      }
    }
  ]
}
```

almacenamiento tras 90 días: 412 GB vigentes + ~180 GB de historial reciente

coste 51,80 → 13,62 USD/mes (-74 %)

Hallazgo 2 — una transición configurada que costaba dinero. El equipo había movido las miniaturas a Standard-IA:

objetos 38.400.000 miniaturas de ~18 KB
volumen 691 GB
lecturas/mes 1.240 GB (las miniaturas se leen constantemente)

Standard: $691 \times 0,023 = 15,89$ USD/mes
Standard-IA: $691 \times 0,0125 + 1.240 \times 0,01 = 8,64 + 12,40 = 21,04$ USD/mes

La transición encarecía un 32 %. Además, los objetos de 18 KB están por debajo del umbral de 128 KB, así que ni Intelligent-Tiering compensaría. Se devuelven a Standard.

Hallazgo 3 — el bucket de facturas.

```
$ aws s3api get-public-access-block --bucket cloudshop-facturas \
  --query 'PublicAccessBlockConfiguration' --output json
{"BlockPublicAcls": true, "IgnorePublicAcls": false,
 "BlockPublicPolicy": true, "RestrictPublicBuckets": false}
```

Dos de cuatro. Impide crear accesos públicos nuevos y no restringe los que ya existen. La política del bucket tenía un Principal: "" de 2024, exactamente el caso de la clase 010.

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://cloudshop-facturas.s3.amazonaws.com/f-1042.pdf
200 ← accesible sin credenciales
```

Corrección en los cuatro parámetros y a nivel de cuenta:

```
$ aws s3control put-public-access-block --account-id 123456789012 \
  --public-access-block-configuration \
    'BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicBuckets=true'
$ curl -s -o /dev/null -w '%{http_code}\n' https://cloudshop-facturas.s3.amazonaws.com/f-1042.pdf
403 ✓
$ aws s3api put-bucket-acl --bucket cloudshop-facturas --acl public-read
AccessDenied ✓ no se puede reabrir
$ aws s3api get-object --bucket cloudshop-facturas --key f-1042.pdf /dev/null >/dev/null && echo ok
ok ✓ acceso legítimo intacto
```

Y se añade retención inmutable, que el versionado por sí solo no daba:

bloqueo de objetos en modo cumplimiento, 2.555 días (7 años de retención legal)
réplica a bucket en cuenta distinta, con su propio bloqueo
restauración probada: objeto recuperado y sha256 idéntico al original ✓

Resultado:

coste de almacenamiento	67,69 USD	29,51 USD	(-56 %)
facturas accesibles sin credencial	sí	no	
borrado de versiones posible	sí	no (cumplimiento)	
restauración probada	nunca	sí, con hash verificado	
subidas abandonadas	1.247	0	

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/030-s3-objetos-versionado-lifecycle-y-replicacion/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa bucket-gobernado-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye bucket-gobernado-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
La factura de almacenamiento no baja después de borrar objetos	En un bucket versionado, DELETE crea un marcador y las versiones siguen facturándose	Ciclo de vida con NoncurrentVersionExpiration y limpieza de marcadores caducados.
Se factura almacenamiento que no aparece en ningún listado	Subidas multiparte abandonadas: invisibles en s3 ls y facturadas indefinidamente	Añade AbortIncompleteMultipartUpload al ciclo de vida.
Mover datos a una clase más fría encarece la factura	El coste de recuperación y el mínimo de duración superan el ahorro por GB	Calcula el umbral de lecturas; con acceso frecuente u objetos pequeños, la clase fría pierde.
Se activa el bloqueo de acceso público y el bucket sigue expuesto	Solo se activaron los parámetros de «crear», no los de «ignorar» y «restringir»	Activa los cuatro y a nivel de cuenta; verifica con petición sin credenciales.
Un borrado malicioso se propaga al bucket réplica	La replicación copia cambios, y borrar versiones es un cambio	Réplica en cuenta distinta con bloqueo de objetos propio; la replicación no sustituye a la copia de seguridad.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué riesgo cubre el bloqueo de objetos que el versionado no cubre, y por qué el modo cumplimiento es el único que sirve ahí?
2. Tras aws s3 rm, el objeto no aparece en el listado. ¿Se está pagando? Justifica.
3. 1 TB en Standard-IA se lee 1,5 TB al mes. ¿Ahorra frente a Standard? Haz el cálculo.
4. ¿Cuál de los cuatro parámetros de bloqueo público cierra un bucket que YA estaba expuesto?
5. ¿Por qué una réplica en la misma cuenta no protege de unas credenciales comprometidas?

Referencias

- AWS (2024). Using versioning in S3 buckets — versiones, marcadores de borrado y facturación.
<<https://docs.aws.amazon.com/AmazonS3/latest/userguide/Versioning.html>>
- AWS (2024). S3 Object Lock — modos gobernanza y cumplimiento, y sus límites.
<<https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html>>
- AWS (2024). Blocking public access to your Amazon S3 storage — semántica de los cuatro parámetros.
<<https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-control-block-public-access.html>>
- AWS (2024). Managing your storage lifecycle — transiciones, mínimos de duración y limpieza de multiparte.
<<https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html>>
- AWS (2024). S3 storage classes — precios, latencias de recuperación y umbrales de tamaño.
<<https://docs.aws.amazon.com/AmazonS3/latest/userguide/storage-class-intro.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 030 S3: objetos, versionado, lifecycle y replicación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega bucket-gobernado-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

031 — RDS, DynamoDB y ElastiCache: decisión de datos

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Elegir motor de datos a partir de los patrones de acceso, no del catálogo del proveedor. Es la decisión más irreversible de la parte —cambiar de motor con terabytes en producción es una migración, no un ajuste— y la que más determina el coste y la latencia del sistema durante años.

Resultados de aprendizaje

Al finalizar podrás:

1. Derivar el modelo de datos desde los patrones de acceso en lugar de normalizar primero y consultar después.
2. Distinguir cuándo una carga necesita transacciones multiclave y cuándo no, con consecuencias de elección.
3. Dimensionar capacidad en DynamoDB y detectar una clave de partición mal elegida antes de que sature.
4. Justificar una caché por el patrón de lectura y explicar sus dos modos de invalidación.
5. Calcular el coste de las tres opciones para una carga concreta y no solo su idoneidad técnica.

Conceptos centrales

Concepto	Comprensión verificable
patrón de acceso	Consulta concreta que la aplicación hará, con su frecuencia y su latencia exigida. En almacenes no relacionales es la entrada del diseño; enumerarlos mal produce un modelo que no se puede consultar.
clave de partición	Atributo que determina en qué partición física vive un elemento. Una clave con pocos valores distintos concentra el tráfico en pocas particiones y produce una partición caliente.
partición caliente	Partición que recibe una fracción desproporcionada del tráfico y satura antes que el resto. Se manifiesta como limitación de tasa mientras la capacidad agregada parece sobrada.
índice secundario global	Proyección de una tabla con otra clave, que permite consultas por atributos distintos. Tiene su propia capacidad y se actualiza de forma asíncrona: es coherente en último término.
invalidación de caché	Mecanismo para que la copia deje de servirse cuando el origen cambia. Los dos modos —caducidad y escritura activa— tienen fallos distintos: datos obsoletos frente a complejidad y acoplamiento.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    Q1{"¿Consultas ad hoc<br/>o conocidas de antemano?"}}
    Q1 -->|"ad hoc, uniones,<br/>agregaciones"| REL["Relacional · RDS"]
    Q1 -->|"conocidas y acotadas"| Q2{"¿Transacciones sobre\nvarias entidades?"}}
    Q2 -->|"sí, frecuentes"| REL
    Q2 -->|"no o puntuales"| Q3{"¿Escala > 10k ops/s\nno crecimiento impredecible?"}}
    Q3 -->|"sí"| NOSQL["DynamoDB"]
    Q3 -->|"no"| REL
    REL --> C{"¿lecturas repetidas\nde lo mismo?"}}
    NOSQL --> C
    C -->|"sí, ratio > 10:1"| CACHE["ElastiCache"]
```

Desarrollo

1. Los patrones de acceso van primero

En un motor relacional se normaliza el modelo y después se consulta como haga falta: el optimizador se encarga. En un almacén de clave-valor no hay optimizador, así que el modelo debe derivarse de las consultas.

La disciplina es enumerar los patrones antes de diseñar nada:

P1	obtener un pedido por su id	12.000/min	< 10 ms
P2	listar los pedidos de un cliente, más recientes	3.400/min	< 30 ms
P3	listar los pedidos de un estado en una fecha	40/min	< 2 s
P4	total facturado por mes	2/día	< 30 s

Con esa lista, la elección se decide sola:

- P1 y P2 son consultas por clave conocida: un almacén de clave-valor las sirve con latencia de un dígito de milisegundos y escala sin límite práctico.
- P3 necesita un índice secundario.
- P4 es una agregación analítica: no pertenece al almacén operativo, y forzarla ahí degrada P1 y P2.

El error clásico es diseñar el modelo relacional primero, migrarlo tal cual a un almacén no relacional y descubrir que P2 requiere escanear la tabla entera. Un escaneo completo en un almacén de clave-valor no es una consulta lenta: es una consulta que cuesta dinero proporcional al tamaño y que empeora cada mes.

Y el criterio inverso también aplica: si los patrones de acceso no se pueden enumerar —porque el negocio hace preguntas nuevas cada semana—, un motor relacional es la elección correcta, y no una concesión.

2. Transacciones: qué se pierde y qué cuesta recuperarlo

Un motor relacional da atomicidad sobre varias tablas con una sola sentencia. En un almacén de clave-valor eso existe pero con límites duros que hay que conocer antes de diseñar:

```
DynamoDB TransactWriteItems:
  hasta 100 elementos por transacción
  máximo 4 MB en total
  consume el DOBLE de capacidad de escritura
  no puede abarcar regiones
```

La consecuencia de diseño: si la operación central del negocio toca cinco entidades y ocurre miles de veces por segundo, el coste doble y el límite de 100 elementos importan. Si ocurre decenas de veces por minuto, no.

La alternativa cuando la transacción no cabe es el patrón de saga: descomponer en pasos con compensación. No es gratis:

```
transacción    todo o nada, garantizado por el motor
saga           pasos independientes con compensación explícita
              → hay estados intermedios visibles
              → la compensación puede fallar y hay que reintentarla
              → el código de compensación se ejercita poco y suele estar mal probado
```

La regla práctica: elegir saga por diseño, no por descubrir tarde que la transacción no cabía. Un sistema que necesita compensación en su camino principal está pagando complejidad permanente por una decisión de motor.

Y una precisión sobre consistencia: DynamoDB ofrece lectura fuertemente consistente en la tabla base —cuesta el doble de capacidad de lectura— pero los índices secundarios globales son siempre coherentes en último término. Una escritura seguida de una consulta por índice puede no ver el cambio. Ese retraso es de milisegundos en régimen normal y puede crecer bajo carga.

3. La clave de partición decide si escala

DynamoDB reparte los datos por el hash de la clave de partición. Una clave con pocos valores distintos concentra el tráfico:

```
clave = estado_pedido    (5 valores: nuevo, pagado, enviado, entregado, cancelado)
→ 5 particiones lógicas para toda la tabla
→ el 80 % del tráfico va a "nuevo"
→ esa partición satura mientras la capacidad agregada parece sobrada
```

El síntoma es característico y desconcierta: `ProvisionedThroughputExceededException` con la capacidad global muy por debajo del límite. La partición está limitada a 3.000 unidades de lectura y 1.000 de escritura por segundo, sea cual sea la capacidad total de la tabla.

Una clave adecuada tiene alta cardinalidad y reparto uniforme:

```
clave = pedido_id (UUID)      → millones de valores, reparto uniforme ✓
clave = cliente_id           → miles de valores, pero un cliente grande
                             puede concentrar tráfico ~
clave = fecha (2026-08-01)    → todo el tráfico del día en una X
```

El segundo caso es el más traicionero: funciona en pruebas y falla cuando un cliente crece. La mitigación es añadir sufijo de dispersión:

```
clave = f"{cliente_id}#{hash(pedido_id) % 10}"
→ reparte cada cliente en 10 particiones
→ coste: consultar todos los pedidos de un cliente exige 10 consultas
```

Es un intercambio explícito: se gana reparto y se pierde simplicidad de consulta. Como todo en esta clase, la decisión debe estar escrita.

Y el modo bajo demanda no elimina el problema: escala automáticamente la capacidad total, pero el límite por partición sigue existiendo. Una clave mal elegida satura igual, solo que la factura crece antes de que se note.

4. Caché: cuándo aporta y cómo se invalida

Una caché solo aporta si el mismo dato se lee muchas más veces de las que se escribe. El indicador es la relación lectura-escritura:

```
ratio < 3:1    la caché añade complejidad y poco beneficio
ratio 10:1     empieza a compensar
ratio > 100:1  claramente rentable
```

Y el ahorro se calcula, no se supone:

```
lecturas/mes          840.000.000
tasa de acierto medida 92 %
lecturas evitadas al origen 772.800.000
coste evitado (0,25 USD por millón de lecturas fuertes) = 193 USD/mes
coste de la caché (cache.r7g.large × 2)                 = 208 USD/mes
```

No compensa por coste con esos números; compensa por latencia, que es otra justificación válida y que hay que declarar como tal:

```
latencia desde el origen 8-12 ms
latencia desde la caché 0,3-0,8 ms
```

Los dos modos de invalidación tienen fallos distintos:

Modo	Cómo	Fallo característico
Caducidad	TTL fijo; se relee al expirar	Datos obsoletos hasta el TTL
Escritura activa	La escritura actualiza caché y origen	Se desincroniza si una de las dos falla

El segundo parece mejor y acopla la escritura a la disponibilidad de la caché. Si la caché no responde, ¿falla la escritura o se queda desincronizada? Ninguna respuesta es buena, y por eso la caducidad suele ser preferible salvo

que la obsolescencia sea inaceptable.

Hay un tercer fallo que afecta a ambos, la estampida: cuando una clave muy consultada caduca, todas las peticiones simultáneas van al origen a la vez. Se mitiga con bloqueo de recálculo o con caducidad aleatorizada —el mismo jitter de la clase 004—.

5. El coste también decide, y a veces al revés

Comparar motores solo por idoneidad técnica lleva a decisiones caras. Los tres modelos de precio son estructuralmente distintos:

RDS	por hora de instancia + almacenamiento + E/S → coste fijo, predecible, independiente del tráfico
DynamoDB	por unidades de lectura y escritura consumidas + almacenamiento → coste proporcional al tráfico, cero si no hay uso
ElastiCache	por hora de nodo → coste fijo

Para una carga con tráfico muy variable, DynamoDB bajo demanda puede ser mucho más barato:

carga con 2 horas de pico al día y el resto casi inactiva	
RDS db.r6g.xlarge Multi-AZ: 730 h × 0,96 USD	= 700,80 USD/mes
DynamoDB bajo demanda:	
escrituras 42 M × 1,25 USD/millón	= 52,50
lecturas 310 M × 0,25 USD/millón	= 77,50
almacenamiento 180 GB × 0,25	= 45,00

	175,00 USD/mes

Pero con tráfico alto y sostenido se invierte:

tráfico sostenido de 4.000 escrituras/s	
DynamoDB bajo demanda: 10.368 M escrituras × 1,25/millón	= 12.960 USD/mes
DynamoDB aprovisionado con autoescalado	≈ 2.600 USD/mes
RDS db.r6g.4xlarge Multi-AZ	≈ 2.800 USD/mes

Bajo demanda cuesta cinco veces más que aprovisionado con tráfico predecible. La regla: bajo demanda para tráfico variable o desconocido; aprovisionado con autoescalado cuando el patrón se estabiliza.

Y una partida que se olvida: en RDS, la E/S se factura aparte en algunas clases de almacenamiento. Una consulta sin índice que escanea millones de filas no solo es lenta: aparece en la factura.

Ejemplo trabajado

CloudShop tiene los pedidos en RDS PostgreSQL. La tabla ha llegado a 180 GB, el p95 de la consulta principal es de 340 ms y la factura de base de datos es de 4.900 USD/mes. Se evalúa el cambio.

Primero, los patrones de acceso reales, medidos en 30 días:

SELECT queryid, calls, mean_exec_time, rows FROM pg_stat_statements ORDER BY calls DESC LIMIT 4;				
P1	SELECT * FROM pedidos WHERE id = \$1	17.280.000	2,1 ms	1
P2	SELECT * FROM pedidos WHERE cliente_id = \$1 ORDER BY creado DESC LIMIT 20	4.896.000	312,0 ms	20
P3	SELECT * FROM pedidos WHERE estado=\$1 AND creado > \$2	57.600	890,0 ms	~4000
P4	agregación mensual por mes	60	8.400 ms	1

P2 domina el problema: 4,9 millones de llamadas diarias a 312 ms. Antes de cambiar de motor se comprueba si es un problema de índice:

```
EXPLAIN (ANALYZE, BUFFERS) SELECT * FROM pedidos
WHERE cliente_id = 8241 ORDER BY creado DESC LIMIT 20;

Index Scan using idx_pedidos_cliente on pedidos (cost=0.56..18402.11)
Rows Removed by Filter: 0
Buffers: shared hit=142 read=8891 ← 8.891 bloques de disco
Execution Time: 309.442 ms
```

El índice existe pero es solo sobre clienteid: PostgreSQL lo usa para filtrar y después ordena 12.400 filas para devolver 20. Se prueba un índice compuesto:

```
CREATE INDEX CONCURRENTLY idx_pedidos_cliente_creado
ON pedidos (cliente_id, creado DESC);
```

Execution Time: 1.284 ms ← de 312 ms a 1,3 ms
 Buffers: shared hit=24 read=0

Un índice compuesto resolvió el 96 % del problema. El motor no era el problema: el modelo de índices sí.

Se reevalúa la situación con ese dato:

p95 global	340 ms	38 ms
E/S facturada	1,2 TB/mes	0,18 TB/mes
coste	4.900 USD	4.130 USD

Ahora sí se compara con la alternativa, con los patrones ya optimizados:

P1 por id	2,1 ms	3 ms
P2 por cliente, ordenado	1,3 ms	4 ms (clave compuesta)
P3 por estado y fecha	890 ms	GSI, ~40 ms
P4 agregación mensual	8.400 ms	NO SOPORTADO
transacciones multiclave	sí	límite 100 elementos
coste mensual estimado	4.130 USD	1.980 USD
migración	—	~3 meses + reescritura

P4 es el bloqueo: la agregación mensual no existe en DynamoDB sin exportar a otro sistema. Y la migración cuesta tres meses para ahorrar 2.150 USD al mes, con recuperación en 4 meses de trabajo de dos personas.

Decisión: quedarse en RDS. Se añaden dos mejoras acotadas:

1. Caché para P1, que es el 78 % del tráfico
 ratio lectura/escritura medido: 47:1 → compensa
 tasa de acierto esperada 90 %
 latencia 2,1 ms → 0,4 ms
 coste: +104 USD/mes
2. P4 a una réplica de lectura, para que la agregación de 8,4 s
 deje de competir con el tráfico operativo
 coste: +380 USD/mes

p95	340 ms	31 ms
coste	4.900 USD	4.614 USD
esfuerzo	—	2 días

La lección: se estaba evaluando un cambio de motor de tres meses para resolver un problema que era un índice mal definido. El análisis de patrones de acceso —que se hizo para elegir motor— acabó descartando el cambio.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/031-rds-dynamodb-y-elasticache-decision-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-datos-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-datos-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una consulta usa el índice y aun así tarda cientos de milisegundos	El índice filtra pero no cubre el orden, así que el motor ordena miles de filas	Índice compuesto que incluya las columnas de ordenación; comprueba con EXPLAIN ANALYZE y los bloques leídos.
DynamoDB limita la tasa con la capacidad global muy por debajo del límite	Partición caliente: la clave tiene poca cardinalidad o un valor concentra el tráfico	Elige una clave de alta cardinalidad o añade sufijo de dispersión, asumiendo el coste de consulta múltiple.
Una escritura seguida de consulta por índice secundario no ve el cambio	Los índices secundarios globales son coherentes en último término	Consulta la tabla base cuando necesites leer lo que acabas de escribir.
El coste de DynamoDB se dispara al estabilizarse el tráfico	Bajo demanda cuesta varias veces más que aprovisionado con carga predecible	Cambia a aprovisionado con autoescalado en cuanto el patrón sea estable.
Al caducar una clave muy consultada, el origen recibe una avalancha	Estampida de caché: todas las peticiones recalculan a la vez	Caducidad aleatorizada o bloqueo de recálculo, igual que el jitter en los reintentos.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué en un almacén de clave-valor los patrones de acceso van antes que el modelo, y qué pasa si se invierte el orden?
2. Una tabla tiene la clave de partición fecha. ¿Qué ocurrirá y por qué el modo bajo demanda no lo resuelve?
3. ¿Qué le cuesta a una transacción de DynamoDB en capacidad, y cuál es su límite de elementos?
4. Con ratio lectura/escritura de 3:1, ¿conviene una caché? ¿Y qué otra justificación podría hacerla válida igualmente?
5. ¿En qué caso bajo demanda es cinco veces más caro que aprovisionado, y por qué?

Referencias

- AWS (2024). DynamoDB best practices: partition key design — cardinalidad, particiones calientes y dispersión.
<<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/bp-partition-key-design.html>>

- AWS (2024). DynamoDB transactions — límites de elementos, tamaño y consumo de capacidad.
<<https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transaction-apis.html>>
- Kleppmann, M. (2017). Designing Data-Intensive Applications, caps. 2-3 — modelos de datos, índices y motores de almacenamiento.
- PostgreSQL (2024). Using EXPLAIN — lectura de planes y coste de E/S.
<<https://www.postgresql.org/docs/current/using-explain.html>>
- AWS (2024). ElastiCache caching strategies — caducidad, escritura activa y mitigación de estampidas.
<<https://docs.aws.amazon.com/AmazonElastiCache/latest/dg/Strategies.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 031 RDS, DynamoDB y ElastiCache: decisión de datos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-datos-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

032 — Lambda, API Gateway y Step Functions

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Construir una API serverless sabiendo dónde están sus límites duros y cuáles son negociables. La clase 015 dio el criterio para elegir el modelo; aquí se aplica a AWS y se añade lo que decide si funciona en producción: la concurrencia como recurso compartido de la cuenta, el arranque en frío frente al percentil del SLO, y por qué una función que llama a una base relacional agota sus conexiones.

Resultados de aprendizaje

Al finalizar podrás:

1. Calcular la concurrencia que consumirá una función a partir de su tasa de invocación y su duración.
2. Anticipar el efecto de una función que agota la concurrencia de la cuenta sobre las demás.
3. Decidir entre concurrencia aprovisionada y optimización del paquete según el percentil del SLO.
4. Resolver el agotamiento de conexiones al llamar a una base de datos relacional desde funciones.
5. Elegir entre orquestación con máquina de estados y coreografía por eventos según trazabilidad y acoplamiento.

Conceptos centrales

Concepto	Comprensión verificable
concurrencia	Invocaciones ejecutándose simultáneamente. Es el recurso limitado de Lambda y se comparte entre todas las funciones de la cuenta y región: 1.000 por defecto.
concurrencia reservada	Porción de la concurrencia de la cuenta apartada para una función. Actúa a la vez como garantía —nadie se la quita— y como techo —no puede superarla—.
concurrencia aprovisionada	Entornos inicializados y mantenidos calientes. Elimina el arranque en frío y elimina también el ahorro de escalar a cero: se paga aunque no se invoque.
proxy de conexiones	Capa que multiplexa conexiones hacia una base de datos relacional. Resuelve que cada entorno de ejecución abra la suya y agote el límite del motor.
máquina de estados	Orquestador que define el flujo de forma explícita, con reintentos, capturas y estado visible. Se opone a la coreografía, donde cada componente reacciona a eventos sin visión global.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

Flujo de razonamiento

```
flowchart TB
  API["API Gateway"] --> L1["Lambda: síncrona<br/>límite 29 s del gateway"]
  API --> SF["Step Functions"]
  SF --> L2["tarea 1"] --> L3["tarea 2"] --> L4["tarea 3"]
  SF -.->|"reintentos, capturas<br/>y estado visible"| SF
  L1 --> RDS["RDS"]
  L1 -.->|"cada entorno abre\nsu conexión"| P["RDS Proxy<br/>multiplexa"]
  P --> RDS
  C["Concurrencia de la cuenta<br/>1.000 por defecto"] -.->|"compartida"| L1
  C -.-> L2
```

Desarrollo

1. La concurrencia es el recurso que se agota, no la CPU

Lambda no escala por CPU ni por memoria: escala creando entornos de ejecución. El número de entornos simultáneos es la concurrencia, y se calcula con la ley de Little de la clase 011:

```
concurrencia = invocaciones por segundo × duración en segundos  
800 inv/s × 0,250 s = 200 concurrentes
```

El límite por defecto es 1.000 por cuenta y región, compartido entre todas las funciones. De ahí el modo de fallo más desconcertante del modelo:

```
función A: procesamiento por lotes, 900 concurrentes durante 10 minutos  
función B: API de pagos, necesita 120  
→ B recibe TooManyRequestsException y devuelve 429 a los usuarios  
→ B no cambió, no tiene errores, y su código está perfecto
```

Es exactamente el problema de vecino ruidoso de la clase 017, dentro de la misma cuenta. Y la solución es la misma: acotar.

```
# Techo para el proceso por lotes: no puede consumir más de 200  
$ aws lambda put-function-concurrency --function-name lotes --reserved-concurrent-executions 200  
# Garantía para la API crítica: 300 apartados, nadie se los quita  
$ aws lambda put-function-concurrency --function-name pagos-api --reserved-concurrent-executions 300
```

La concurrencia reservada tiene un doble filo que hay que entender: es garantía y es techo. La función con 300 reservados nunca bajará de 300 disponibles y nunca pasará de 300, aunque el resto de la cuenta esté ocioso. Reservar de menos convierte la protección en una limitación.

Y hay un límite adicional sobre el ritmo de crecimiento: la concurrencia sube en incrementos de 1.000 por minuto por región tras la ráfaga inicial. Un pico que necesite 5.000 concurrentes en 30 segundos no se puede absorber escalando, igual que en la clase 016.

2. Arranque en frío: comparar con el percentil del SLO

La pregunta útil no es «¿hay arranque en frío?» sino «¿qué fracción de invocaciones lo sufre y dónde cae el percentil de mi SLO?».

invocaciones/mes	2.000.000	
arranques en frío	12.000	→ 0,6 %
latencia p50 en caliente	45 ms	
latencia en frío	820 ms	

SLO al p95 → el 0,6 % no lo alcanza; el p95 sigue siendo ~90 ms ✓
SLO al p99 → el 0,6 % está dentro del 1 % peor; el p99 ES el frío ✗

Factores que determinan la latencia de arranque, por impacto:

tamaño del paquete	250 MB descomprimido tarda mucho más que 5 MB
runtime	JVM y .NET inicializan máquina virtual
inicialización propia	conexiones, carga de configuración, SDK

La inicialización propia es la más controlable y la peor aprovechada. El código fuera del manejador se ejecuta una vez por entorno, no por invocación:

```
# Fuera del manejador: se paga una vez por entorno  
import boto3  
cliente = boto3.client("dynamodb")      # reutilizado en invocaciones siguientes  
config = cargar_configuracion()  
  
def handler(event, context):  
    return cliente.get_item(...)        # sin coste de inicialización
```

Ponerlo dentro del manejador multiplica el coste por cada invocación, no solo por cada arranque.

La concurrencia aprovisionada elimina el frío y cambia la economía por completo:

```
10 entornos aprovisionados de 512 MB:  
10 × 0,5 GB × 730 h × 0,0000097222 USD/GB-s × 3600 = 127,80 USD/mes  
se paga estén o no invocándose
```

Con eso, el cálculo de la clase 015 cambia: ya no se escala a cero, así que el umbral frente a un contenedor permanente se desplaza a favor del contenedor.

3. Funciones y bases de datos relacionales: el choque de modelos

Una función escala a cientos de entornos y cada entorno abre su propia conexión. Un motor relacional tiene un límite de conexiones muy inferior:

conurrencia de la función	400 entornos	
conexiones por entorno	1	
max_connections de la instancia	200	← se agota al 50 % de la conurrencia

El síntoma: FATAL: sorry, too many clients already. Y no se arregla subiendo maxconnections, porque cada conexión de PostgreSQL consume memoria del servidor y a partir de cierto punto degrada el motor entero.

Las tres respuestas, de peor a mejor:

1. Reservar conurrencia por debajo del límite de conexiones
→ funciona y desperdicia la elasticidad que motivó usar funciones
2. Cerrar la conexión al final de cada invocación
→ añade el coste de establecerla a cada llamada (decenas de ms)
3. Proxy de conexiones
→ multiplexa: 400 entornos comparten un pool de 50 conexiones reales

La tercera es la correcta, y tiene un matiz que decide si funciona: el proxy solo puede reutilizar conexiones si la sesión no tiene estado. Una transacción abierta, una tabla temporal o una variable de sesión fuerzan a fijar la conexión a ese cliente hasta que termine, y con suficientes clientes fijados el pool se agota igual.

sin estado de sesión	reutilización alta, el proxy cumple su función
con transacciones largas	fijación de conexión, el proxy no ayuda

Y una alternativa estructural: si la carga es de funciones, un almacén de clave-valor encaja mejor que uno relacional, porque su modelo de acceso es sin conexión persistente. Es la decisión de la clase 031 vista desde el otro lado.

4. Los límites del gateway y cómo se rodean

API Gateway impone restricciones que no dependen de Lambda y que hay que conocer antes de diseñar:

Límite	Valor	Negociable
Tiempo de espera de integración	29 s	No
Tamaño de carga útil	10 MB	No
Peticiones por segundo	10.000 por región	Sí, con solicitud
Ráfaga	5.000	Sí

Los dos primeros son duros y determinan la arquitectura:

29 segundos es menos que los 15 minutos de Lambda. Una función que tarda 2 minutos funciona invocada directamente y falla siempre a través del gateway. El patrón correcto es asíncrono:

```
POST /informes → 202 Accepted, devuelve id de trabajo
                  encola el trabajo
GET /informes/{id} → 200 con estado, o 303 al resultado cuando termina
```

10 MB de carga útil hace inviable subir ficheros grandes por la API. La solución es no pasarlos por ella:

```
POST /subidas → devuelve una URL prefirmada de S3
cliente      → sube directamente a S3, sin límite del gateway
S3           → emite evento que dispara el procesamiento
```

Esto además ahorra dinero: los bytes no atraviesan el gateway ni la función.

Y una decisión que se toma al principio y cuesta cambiar: HTTP API frente a REST API. La primera cuesta aproximadamente un 70 % menos y tiene menos funciones —sin planes de uso, sin caché, sin validación de solicitud—. Empezar por REST API «por si acaso» es una de las formas más comunes de pagar de más sin usar nada de lo que se paga.

5. Orquestación frente a coreografía

Un flujo de varios pasos se puede construir de dos formas, y la elección determina cuánto cuesta diagnosticar un fallo:

	Orquestación (máquina de estados)	Coreografía (eventos)
Flujo	Explícito en un sitio	Emergente, repartido
Estado	Visible y consultable	Hay que reconstruirlo
Reintentos y compensación	Declarativos	En cada componente
Acoplamiento	Mayor: el orquestador conoce los pasos	Menor
Diagnóstico	Se ve dónde falló	Hay que correlacionar trazas
Coste	Por transición de estado	Por evento

La orquestación se declara y el motor se encarga de reintentar:

```
{
  "Type": "Task",
  "Resource": "arn:aws:lambda:...:function:cobrar",
  "Retry": [{
    "ErrorEquals": ["States.TaskFailed"],
    "IntervalSeconds": 2, "MaxAttempts": 3, "BackoffRate": 2.0
  }],
  "Catch": [{"ErrorEquals": ["States.ALL"], "Next": "CompensarReserva"}]
}
```

Esas nueve líneas sustituyen a código de reintento con retroceso exponencial repartido por varios servicios, y —más importante— hacen visible la compensación, que en coreografía suele estar implícita y sin probar.

El criterio práctico:

orquestación	flujos de negocio con compensación, pasos con orden, necesidad de auditar dónde quedó cada ejecución
coreografía	reacciones independientes, muchos consumidores del mismo hecho, equipos que no deben coordinarse para añadir un consumidor

Y sobre coste: Step Functions Standard cobra por transición de estado, lo que en flujos muy repetitivos se acumula. El modo Express cuesta por duración y es mucho más barato para flujos cortos de alto volumen, a cambio de garantía «al menos una vez» en vez de «exactamente una vez» —lo que exige idempotencia en los pasos, como en la clase 009—.

Ejemplo trabajado

La API de pagos de CloudShop empieza a devolver 429 sin que su tráfico haya cambiado, y en el mismo día aparecen errores de conexión contra la base de datos.

Síntoma 1 — 429 sin cambio de tráfico:

```
$ aws cloudwatch get-metric-statistics --namespace AWS/Lambda \
  --metric-name Throttles --dimensions Name=FunctionName,Value=pagos-api \
  --start-time 2026-08-01T09:00:00Z --end-time 2026-08-01T12:00:00Z \
  --period 300 --statistics Sum --query 'sum(Datapoints[0].Sum)'
8412.0
$ aws lambda get-account-settings --query 'AccountLimit.ConcurrentExecutions'
1000
```

Se busca quién consume la concurrencia:

```
$ for f in pagos-api catalogo informes-lotes notificaciones; do
  printf "%-18s " $f
  aws cloudwatch get-metric-statistics --namespace AWS/Lambda \
    --metric-name ConcurrentExecutions --dimensions Name=FunctionName,Value=$f \
    --start-time 2026-08-01T10:00:00Z --end-time 2026-08-01T11:00:00Z \
    --period 3600 --statistics Maximum --query 'Datapoints[0].Maximum'
done
pagos-api          118.0
catalogo           94.0
informes-lotes     742.0      ←
notificaciones     46.0
```

informes-lotes consume 742 de los 1.000. Se lanzó un reproceso histórico esa mañana. La API de pagos no cambió: se quedó sin sitio.

Verificación con la ley de Little de lo que cada una necesita:

```
pagos-api      340 inv/s × 0,32 s = 109 concurrentes → observado 118 ✓
informes-lotes 62 inv/s × 12,0 s = 744                → observado 742 ✓
```

Corrección con techo y garantía:

```
$ aws lambda put-function-concurrency --function-name informes-lotes \
  --reserved-concurrent-executions 200
$ aws lambda put-function-concurrency --function-name pagos-api \
  --reserved-concurrent-executions 250      # 118 observados + margen

con 200 reservados, informes-lotes tarda 3,7× más (62 → 16,7 inv/s efectivas)
y es trabajo por lotes sin plazo: aceptable y declarado
```

Síntoma 2 — conexiones agotadas. Aparece al aumentar la concurrencia:

```
FATAL: sorry, too many clients already

$ aws rds describe-db-parameters --db-parameter-group-name cloudshop-pg \
  --query 'Parameters[?ParameterName==`max_connections`].ParameterValue' --output text
200

concurrencia combinada de las funciones que tocan la base: 118 + 94 + 200 = 412
conexiones disponibles                                     200
→ se agotan al 48 % de la concurrencia
```

Se evalúan las tres salidas:

1. bajar la concurrencia por debajo de 200 → renuncia a la elasticidad
2. cerrar conexión por invocación → +18 ms medidos por llamada
3. proxy de conexiones → multiplexa

Se elige el proxy y se comprueba la condición que decide si sirve:

```
$ aws rds create-db-proxy --db-proxy-name cloudshop-proxy \
  --engine-family POSTGRES --require-tls ...
$ aws cloudwatch get-metric-statistics --namespace AWS/RDS \
  --metric-name DatabaseConnectionsCurrentlySessionPinned \
  --dimensions Name=ProxyName,Value=cloudshop-proxy \
  --period 300 --statistics Maximum --query 'Datapoints[0].Maximum'
3.0
```

Solo 3 conexiones fijadas de 412 clientes: el código no usa transacciones largas ni estado de sesión, así que la multiplexación funciona.

conexiones reales al motor	412 (falla)	47
concurrencia sostenible	~190	412
latencia p95 de pagos-api	340 ms	112 ms
throttles/hora	2.804	0

Los dos síntomas tenían la misma raíz: recursos compartidos sin acotar —concurrencia de la cuenta y conexiones del motor—. Ninguno era un problema de la función que fallaba.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/032-lambda-api-gateway-y-step-functions/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa api-serverless-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye api-serverless-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una función devuelve 429 sin que su tráfico haya cambiado	Otra función consumió la concurrencia compartida de la cuenta	Reserva concurrencia como techo para lo interrumpible y como garantía para lo crítico.
Reservar concurrencia protege la función pero la limita en el pico	La concurrencia reservada es garantía y también techo	Dimensiona el reservado por encima del máximo observado más margen; reservar de menos es limitarse.
Errores de conexión contra la base de datos al escalar la función	Cada entorno abre su conexión y el motor tiene un límite muy inferior	Usa un proxy de conexiones y comprueba la métrica de conexiones fijadas para saber si multiplexa.
Una función de 2 minutos falla siempre a través del gateway	El tiempo de espera de integración es de 29 s y no es negociable	Patrón asíncrono: 202 con identificador de trabajo y consulta posterior del estado.
El p99 se dispara aunque el p50 sea excelente	La fracción de arranques en frío cae dentro del percentil del SLO	Compara la fracción medida con el percentil exigido; reduce el paquete o aprovisiona solo la ruta crítica.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Una función se invoca 500 veces por segundo y dura 400 ms. ¿Cuánta concurrencia consume y qué fracción del límite por defecto?
2. ¿Por qué la concurrencia reservada puede empeorar el rendimiento de la función que pretende proteger?
3. ¿Por qué subir maxconnections no resuelve el agotamiento de conexiones desde funciones?
4. ¿Qué límite del gateway hace imposible una integración síncrona de 2 minutos, y qué patrón lo rodea?
5. ¿Qué garantiza el modo Express de una máquina de estados y qué exige a cambio a los pasos?

Referencias

- AWS (2024). Lambda function scaling and concurrency — límites de cuenta, reserva y ritmo de crecimiento. <<https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html>>
- AWS (2024). Provisioned concurrency — coste y efecto sobre el arranque en frío. <<https://docs.aws.amazon.com/lambda/latest/dg/provisioned-concurrency.html>>
- AWS (2024). Using Amazon RDS Proxy — multiplexación, fijación de sesión y sus causas. <<https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/rds-proxy.html>>
- AWS (2024). API Gateway quotas and important notes — tiempo de espera de 29 s y tamaño de carga útil. <<https://docs.aws.amazon.com/apigateway/latest/developerguide/limits.html>>
- AWS (2024). Step Functions: Standard vs Express workflows — garantías de ejecución y modelo de precio. <<https://docs.aws.amazon.com/step-functions/latest/dg/concepts-standard-vs-express.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 032 Lambda, API Gateway y Step Functions

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega api-serverless-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

033 — SQS, SNS y EventBridge

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Diseñar comunicación asíncrona sabiendo qué garantiza cada servicio y qué hay que construir encima. La entrega «exactamente una vez» no existe en la red —lo estableció la clase 009— así que el receptor debe ser idempotente. Aquí se elige entre cola, tema y bus por acoplamiento y garantías, y se construye la cola de mensajes fallidos que evita perder trabajo en silencio.

Resultados de aprendizaje

Al finalizar podrás:

1. Elegir entre cola, tema y bus a partir del número de consumidores y del acoplamiento deseado.
2. Configurar el tiempo de invisibilidad coherente con la duración del procesamiento y el número de reintentos.
3. Construir una cola de mensajes fallidos con alerta y procedimiento de reproceso.
4. Explicar por qué una cola FIFO limita el rendimiento y cuándo ese coste se justifica.
5. Diagnosticar mensajes procesados varias veces distinguiendo duplicado de entrega de fallo de idempotencia.

Conceptos centrales

Concepto	Comprensión verificable
tiempo de invisibilidad	Periodo durante el cual un mensaje leído no es visible para otros consumidores. Si expira antes de que el consumidor termine, otro lo recibe y el mensaje se procesa dos veces.
cola de mensajes fallidos	Destino de los mensajes que agotaron sus reintentos. Sin ella, un mensaje envenenado se reintenta indefinidamente y bloquea la cola; con ella, el fallo queda aislado y visible.
abanico de salida	Patrón en el que un mensaje llega a varios consumidores independientes. Un tema lo hace en el momento; una cola, no: el mensaje lo consume uno solo.
entrega al menos una vez	Garantía de que un mensaje llegará, posiblemente más de una vez. Es lo que ofrecen las colas estándar, y obliga a que el receptor sea idempotente.
mensaje envenenado	Mensaje que provoca un fallo determinista en el consumidor. Sin cola de fallidos, se reintenta para siempre y consume capacidad que otros mensajes necesitan.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
P["productor"] --> Q{"¿cuántos consumidores<br/>del mismo hecho?"}
```

```
Q -->|"uno"| SQS["Cola · SQS<br/>reparto de trabajo"]
```

```
Q -->|"varios, conocidos"| SNS["Tema · SNS<br/>abanico de salida"]
```

```
Q -->|"varios, con filtro<br/>y enrutamiento"| EB["Bus · EventBridge"]
```

```
SQS --> C1["consumidor"]
```

```
SNS --> Q1["cola A"] --> C2["consumidor A"]
```

```
SNS --> Q2["cola B"] --> C3["consumidor B"]
```

```
C1 -->|"agota reintentos"| DLQ["Cola de fallidos<br/>con alerta"]
```

Desarrollo

1. Cola, tema y bus: tres formas de desacoplar

	Cola (SQS)	Tema (SNS)	Bus (EventBridge)
Consumidores del mismo mensaje	Uno	Todos los suscritos	Los que casen con la regla
Persistencia	Hasta 14 días	Ninguna: si no hay suscriptor, se pierde	Reproducibile con archivo
Filtrado	No	Por atributos	Por contenido completo
Reintentos	Del consumidor	Del suscriptor	Con cola de fallidos por regla
Latencia típica	ms	ms	0,5 s

La primera fila es la distinción fundamental: una cola reparte trabajo, un tema difunde un hecho. Si diez consumidores leen de la misma cola, cada mensaje lo procesa uno solo; eso es correcto para repartir carga y erróneo si los diez deben enterarse.

La segunda fila es la que produce pérdidas silenciosas: SNS no persiste. Si un suscriptor está caído cuando llega el mensaje, lo pierde. Por eso el patrón robusto es siempre tema seguido de cola por consumidor:

```
SNS tema "pedido-creado"
  ■■■ cola facturacion    → consumidor de facturación
  ■■■ cola inventario     → consumidor de inventario
  ■■■ cola analitica      → consumidor de analítica
```

Cada consumidor tiene su propia cola con su propio ritmo, sus propios reintentos y su propia cola de fallidos. Si el de analítica cae dos horas, sus mensajes esperan en su cola sin afectar a los demás.

EventBridge añade filtrado por contenido y enrutamiento declarativo, a cambio de más latencia:

```
{"detail-type": ["pedido-creado"], "detail": {"importe": [{"numeric": [ ">", 1000 ] ] } } }
```

Esa regla entrega solo los pedidos por encima de 1.000. Con SNS habría que filtrar en el consumidor y pagar por procesar lo que se descarta.

2. El tiempo de invisibilidad es la causa de los duplicados

Cuando un consumidor lee un mensaje, este se vuelve invisible durante un tiempo configurable. Si el consumidor no lo borra antes de que expire, el mensaje vuelve a estar disponible y otro lo procesa.

```
tiempo de invisibilidad    30 s
duración del procesado     45 s      ← MAYOR
```

```
t+0   consumidor A lee el mensaje
t+30  expira la invisibilidad; el mensaje reaparece
t+30  consumidor B lo lee y empieza a procesarlo
t+45  A termina y lo borra
t+75  B termina: el efecto se aplicó DOS VECES
```

La regla de dimensionado:

$\text{invisibilidad} \geq \text{duración del p99 del procesamiento} \times \text{margen}$

Con un p99 de 45 s, un valor de 120 s es razonable. Y cuando la duración es muy variable, el consumidor puede extenderla mientras trabaja:

```
$ aws sqs change-message-visibility --queue-url $URL \
  --receipt-handle $RH --visibility-timeout 300
```

El efecto compuesto con el número de reintentos es el que sorprende:

```
invisibilidad 120 s, maxReceiveCount 5
→ un mensaje envenenado tarda  $120 \times 5 = 10$  minutos en llegar a la cola de fallidos
→ durante ese tiempo ocupa capacidad de consumo 5 veces
```

Y el límite superior: la invisibilidad máxima es de 12 horas. Un procesamiento que pueda tardar más no cabe en el modelo y necesita otro diseño —un trabajo asíncrono con estado propio—.

Un último detalle que confunde al depurar: si el consumidor falla y no borra el mensaje, no hace falta esperar a la invisibilidad para reintentarlo si el consumidor la reduce a cero explícitamente al capturar el error. Eso acelera el reintento de fallos transitorios sin tocar la configuración.

3. La cola de mensajes fallidos no es opcional

Sin ella, un mensaje que provoca un fallo determinista se reintenta indefinidamente. Consume capacidad de consumo, llena los logs de errores repetidos y retrasa a los mensajes correctos que están detrás.

```
$ aws sqs set-queue-attributes --queue-url $URL --attributes '{
  "RedrivePolicy": "{\"deadLetterTargetArn\":\"arn:aws:sqs:...:pedidos-dlq\",
    \"maxReceiveCount\": \"5\"}",
  "VisibilityTimeout": "120",
  "MessageRetentionPeriod": "1209600"
}'
```

Tres decisiones en esa configuración:

maxReceiveCount. Demasiado bajo (2) envía a fallidos por errores transitorios de red; demasiado alto (20) tarda horas en aislar un envenenado. Entre 3 y 5 cubre los fallos transitorios sin retrasar el aislamiento.

La retención de la cola de fallidos debe ser mayor que la de origen. Si ambas son de 14 días, un mensaje que llega a fallidos el día 13 solo se conserva un día más. La cola de fallidos debe tener 14 días contados desde su llegada, y por eso conviene revisar que el mensaje no envejezca antes de que alguien lo mire.

La alerta. Una cola de fallidos sin alerta es un cubo donde el trabajo desaparece en silencio, que es peor que no tenerla porque da falsa sensación de control:

```
$ aws cloudwatch put-metric-alarm --alarm-name pedidos-dlq-no-vacia \
  --metric-name ApproximateNumberOfMessagesVisible --namespace AWS/SQS \
  --dimensions Name=QueueName,Value=pedidos-dlq \
  --statistic Maximum --period 300 --threshold 0 \
  --comparison-operator GreaterThanThreshold --evaluation-periods 1
```

El umbral es cero: cualquier mensaje en fallidos merece atención. Y hace falta un procedimiento de reproceso escrito, porque el momento de improvisarlo no puede ser durante un incidente.

4. FIFO: qué garantiza y qué cuesta

Las colas FIFO añaden orden estricto y deduplicación, y ambas cosas se pagan:

	Estándar	FIFO
Rendimiento	Prácticamente ilimitado	300 msg/s, 3.000 con lotes
Orden	Mejor esfuerzo	Estricto dentro del grupo
Entrega	Al menos una vez	Exactamente una vez, ventana de 5 min
Alto rendimiento	—	Hasta 70.000/s con particionado por grupo

La frase clave está en la fila del orden: el orden es estricto dentro de un grupo de mensajes, no en toda la cola. El identificador de grupo es lo que decide el paralelismo:

```
grupo = "global"           → orden total, un solo consumidor efectivo, 300 msg/s
grupo = pedido_id          → orden por pedido, miles de grupos en paralelo
```

La segunda opción es casi siempre la correcta: rara vez se necesita orden global. Lo que se necesita es que los eventos de un mismo pedido lleguen en orden, y para eso el grupo debe ser el identificador del pedido.

La deduplicación tiene un límite temporal que hay que conocer: 5 minutos. Dos mensajes idénticos separados por 6 minutos se entregan ambos. Por eso la deduplicación de FIFO no sustituye a la idempotencia del receptor, solo reduce su frecuencia de activación.

La pregunta antes de elegir FIFO:

```
¿el orden importa de verdad, o solo lo parece?
"el pago debe ir después de la reserva" → sí, importa
"prefiero que lleguen en orden"         → no justifica el coste
```

Muchos sistemas que creen necesitar orden lo que necesitan es idempotencia y detección de eventos fuera de orden —descartar una actualización con marca de tiempo anterior a la ya aplicada—, que escala sin límite.

5. Duplicado de entrega frente a fallo de idempotencia

Cuando aparece un efecto duplicado hay dos causas posibles y se corrigen en sitios distintos:

duplicado de ENTREGA el mismo mensaje se entregó dos veces
→ normal en entrega al menos una vez
→ se corrige con idempotencia en el consumidor

fallo de IDEMPOTENCIA el consumidor no reconoció que ya lo había procesado
→ es un defecto
→ se corrige en la lógica del consumidor

Distinguir las exige registrar el identificador del mensaje:

```
def handler(event, context):
    for registro in event["Records"]:
        mid = registro["messageId"]
        veces = int(registro["attributes"]["ApproximateReceiveCount"])
        log.info("procesando", extra={"message_id": mid, "recepcion": veces})
        if ya_procesado(mid):
            log.info("duplicado descartado", extra={"message_id": mid})
            continue
        procesar(registro)
        marcar_procesado(mid)      # en la MISMA transacción que el efecto
```

El comentario de la última línea es el mismo punto de la clase 009: si marcar y aplicar el efecto no son atómicos, existe una ventana en la que el efecto ocurrió y la marca no, y el reintento duplica.

ApproximateReceiveCount es la métrica que separa los diagnósticos:

contador = 1 y efecto duplicado → fallo de idempotencia: el consumidor falló
contador > 1 y efecto duplicado → duplicado de entrega no manejado
contador > 1 y sin duplicado → funcionando como debe

Y un consejo operativo: alertar cuando ApproximateReceiveCount supere 2 de forma sostenida. No es un fallo por sí mismo, pero indica que algo está reintentando más de lo normal —invisibilidad corta, consumidor lento o errores transitorios frecuentes— y precede a problemas mayores.

Ejemplo trabajado

Durante una promoción, 1.847 pedidos de CloudShop se facturan dos veces y 312 no se facturan en absoluto. Dos síntomas opuestos, causas distintas.

Síntoma 1 — facturación duplicada.

```
$ aws sqs get-queue-attributes --queue-url $URL_FACTURACION \
  --attribute-names VisibilityTimeout ApproximateNumberOfMessagesNotVisible
{"VisibilityTimeout": "30", "ApproximateNumberOfMessagesNotVisible": "412"}
```

Se mide cuánto tarda el consumidor:

p50	8,2 s	
p95	28,4 s	
p99	51,7 s	← mayor que la invisibilidad de 30 s
invocaciones en el pico	42.000	
fracción por encima de 30 s	4,4 %	→ 1.848 mensajes
duplicados observados	1.847	

Cuadra exactamente. El 4,4 % de los mensajes tarda más que la invisibilidad, reaparece y lo procesa otro consumidor. No es un fallo del código: es un parámetro mal dimensionado.

Se comprueba si además había fallo de idempotencia:

```
$ aws logs filter-log-events --log-group-name /aws/lambda/facturacion \
  --filter-pattern "duplicado descartado" --start-time 1785540000000 \
  --query 'length(events)'
0
```

Cero descartes: el consumidor no comprobaba duplicados en absoluto. Los dos arreglos son necesarios y ninguno basta solo.

```
$ aws sqs set-queue-attributes --queue-url $URL_FACTURACION \
  --attributes VisibilityTimeout=180          # 3,5× el p99

# y en el consumidor, marca y efecto en la misma transacción
with db.transaction():
    if db.execute("INSERT INTO procesados(message_id) VALUES(%) "
                  "ON CONFLICT DO NOTHING", [mid]).rowcount == 0:
        return          # ya procesado
    facturar(pedido)
```

Síntoma 2 — 312 pedidos sin facturar.

```
$ aws sqs get-queue-attributes --queue-url $URL_FACTURACION \
  --attribute-names RedrivePolicy
{}
```

No había cola de mensajes fallidos. Se reconstruye qué pasó con los logs:

```
$ aws logs filter-log-events --log-group-name /aws/lambda/facturacion \
  --filter-pattern 'ERROR' --query 'events[0].message' --output text
ERROR ValueError: importe negativo en pedido A-88213
```

312 pedidos de una promoción con descuento superior al importe generaban un valor negativo. El consumidor fallaba, el mensaje volvía a la cola, y así indefinidamente:

```
mensajes envenenados          312
reintentos por mensaje en 4 horas ~480 (cada 30 s)
invocaciones desperdiciadas    ~149.760
```

Además de perder esos pedidos, consumían capacidad que retrasaba a los buenos, lo que agravó el síntoma 1 al alargar los tiempos de procesamiento.

```
$ aws sqs create-queue --queue-name facturacion-dlq \
  --attributes MessageRetentionPeriod=1209600
$ aws sqs set-queue-attributes --queue-url $URL_FACTURACION --attributes '{
  "RedrivePolicy":{"deadLetterTargetArn":"arn:...:facturacion-dlq",
    "maxReceiveCount":"4"}}'
$ aws cloudwatch put-metric-alarm --alarm-name facturacion-dlq-no-vacia \
  --metric-name ApproximateNumberOfMessagesVisible --namespace AWS/SQS \
  --dimensions Name=QueueName,Value=facturacion-dlq \
  --statistic Maximum --period 300 --threshold 0 \
  --comparison-operator GreaterThanThreshold --evaluation-periods 1
```

Y se reprocesan los 312 tras corregir la validación:

```
$ aws sqs start-message-move-task \
  --source-arn arn:aws:sqs:...:facturacion-dlq \
  --destination-arn arn:aws:sqs:...:facturacion
```

Resultado en la promoción siguiente:

pedidos duplicados	1.847	0
pedidos sin facturar	312	0
invocaciones desperdiciadas	149.760	0
p95 de la cola	28,4 s	9,1 s
mensajes en cola de fallidos	n/a	7 (con alerta y reproceso)

Los 7 de la cola de fallidos son el resultado correcto: fallos reales, aislados, visibles y recuperables, en vez de trabajo que desaparece.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/033-sqs-sns-y-eventbridge/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa flujo-eventos-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye flujo-eventos-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un porcentaje de mensajes produce efecto duplicado	El procesamiento supera el tiempo de invisibilidad y el mensaje reaparece	Dimensiona la invisibilidad por encima del p99 del procesamiento, o extiéndela mientras trabajas.
Trabajo que desaparece sin dejar rastro	No hay cola de mensajes fallidos: los envenenados se reintentan hasta caducar	Configura cola de fallidos con maxReceiveCount entre 3 y 5 y alerta con umbral cero.
Un suscriptor pierde mensajes mientras está caído	SNS no persiste: si no hay quien reciba, el mensaje se pierde	Patrón tema seguido de cola por consumidor; cada uno con su ritmo y sus reintentos.
Una cola FIFO no supera 300 mensajes por segundo	Todos los mensajes usan el mismo identificador de grupo, así que no hay paralelismo	Usa el identificador de la entidad como grupo; el orden estricto es por grupo, no por cola.
Aparece un duplicado con ApproximateReceiveCount igual a 1	No es duplicado de entrega: es un fallo de idempotencia en el consumidor	Marca el mensaje como procesado en la misma transacción que el efecto.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué el patrón robusto es tema seguido de cola por consumidor en vez de suscribir consumidores directamente al tema?
2. El procesamiento tiene un p99 de 50 s y la invisibilidad es de 30 s. ¿Qué fracción se duplicará y por qué?
3. Con invisibilidad de 120 s y maxReceiveCount de 5, ¿cuánto tarda un mensaje envenenado en aislarse?
4. ¿Por qué la deduplicación de FIFO no sustituye a la idempotencia del receptor?

5. Ves un efecto duplicado con `ApproximateReceiveCount = 1`. ¿Dónde está el fallo?

Referencias

- AWS (2024). Amazon SQS visibility timeout — semántica, extensión y límite de 12 horas.
<<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html>>
- AWS (2024). Amazon SQS dead-letter queues — política de reenvío y reproceso.
<<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html>>
- AWS (2024). FIFO queues: message groups and throughput — orden por grupo y límites.
<<https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues.html>>
- AWS (2024). EventBridge event patterns — filtrado por contenido y enrutamiento declarativo.
<<https://docs.aws.amazon.com/eventbridge/latest/userguide/eb-event-patterns.html>>
- Hohpe, G. y Woolf, B. (2003). Enterprise Integration Patterns — canal punto a punto, publicación-suscripción y canal de mensajes inválidos.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 033 SQS, SNS y EventBridge

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega flujo-eventos-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

034 — CloudWatch, CloudTrail, Config y Systems Manager

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Construir la evidencia operativa que permite responder tres preguntas distintas durante un incidente: qué está pasando, quién hizo qué y qué cambió. Cada una se responde con un servicio diferente, y confundirlos hace que se busque durante horas en el sitio equivocado. Aquí se aterriza en AWS lo que la clase 012 estableció sobre señales.

Resultados de aprendizaje

Al finalizar podrás:

1. Asignar cada pregunta forense al servicio que la responde y saber qué no responde ninguno.
2. Diseñar métricas y filtros que no multipliquen el coste por la dimensionalidad.
3. Consultar registros de auditoría para reconstruir quién ejecutó una acción y desde dónde.
4. Detectar desviaciones de configuración con reglas que se evalúan de forma continua.
5. Ejecutar una operación en una flota sin abrir puertos ni mantener claves de acceso.

Conceptos centrales

Concepto	Comprensión verificable
cardinalidad	Número de combinaciones distintas de dimensiones de una métrica. Cada combinación es una serie temporal facturada por separado, así que una dimensión con muchos valores multiplica el coste.
métrica de filtro	Métrica derivada de patrones en los logs. Convierte texto en series temporales sin instrumentar el código, a cambio de depender del formato del mensaje.
registro de auditoría	Historial de llamadas a la API con identidad, origen, parámetros y resultado. Responde «quién hizo qué», que ninguna métrica ni log de aplicación responde.
elemento de configuración	Instantánea del estado de un recurso en un momento dado. Permite comparar cómo estaba antes y después de un cambio, que es distinto de saber quién lo hizo.
documento de automatización	Procedimiento declarativo ejecutable sobre una flota, con parámetros y control de errores. Convierte un runbook escrito en uno ejecutable y auditable.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
Q1["¿Qué está pasando<br/>AHORA?"] --> CW["CloudWatch<br/>métricas, logs, alarmas"]
Q2["¿Quién hizo qué?"] --> CT["CloudTrail<br/>llamadas a la API"]
Q3["¿Qué cambió y<br/>cómo estaba antes?"] --> CFG["Config<br/>historial de estado"]
Q4["¿Cómo lo arreglo<br/>sin entrar por SSH?"] --> SSM["Systems Manager<br/>automatización"]
CW --> A["alarma"]
A --> SSM
CT --> AUD[("cuenta de auditoría<br/>borrado prohibido")]
CFG --> AUD
```

Desarrollo

1. Cuatro preguntas, cuatro servicios

Durante un incidente se hacen preguntas distintas y cada una tiene su fuente. Buscar en la equivocada es la causa más común de que un diagnóstico tarde horas:

Pregunta	Servicio	Qué no responde
¿Qué está pasando ahora?	CloudWatch	Quién lo provocó
¿Quién hizo qué y desde dónde?	CloudTrail	Cómo estaba antes
¿Qué cambió y cuál era el estado previo?	Config	Quién lo cambió
¿Cómo actuó sobre la flota?	Systems Manager	—

Las dos filas centrales se confunden constantemente y son complementarias:

CloudTrail dice: "a las 14:22, el rol ci-deploy llamó a ModifyDBInstance desde 203.0.113.40 con estos parámetros"

Config dice: "a las 14:22, la instancia pasó de BackupRetentionPeriod=7 a BackupRetentionPeriod=0"

CloudTrail da el actor y la intención; Config da el estado antes y después. Para reconstruir un incidente hacen falta los dos, y por eso ambos deben escribir en la cuenta de auditoría de la clase 025.

Y hay una pregunta que ninguno responde: por qué. Eso solo está en el ADR, el ticket o la conversación. Es la razón por la que la clase 024 insistía en registrar el razonamiento: la telemetría reconstruye el qué con precisión y nunca el motivo.

2. La cardinalidad multiplica la factura

Cada combinación distinta de dimensiones es una métrica personalizada facturada por separado, a unos 0,30 USD al mes. La aritmética se dispara rápido:

métrica: latencia_peticion

dimensiones: servicio (8) × endpoint (40) × código (12) × instancia (24)

series = $8 \times 40 \times 12 \times 24 = 92.160$

coste = $92.160 \times 0,30 = 27.648$ USD/mes

Casi 28.000 dólares al mes en métricas, por añadir dos dimensiones que parecían útiles. Y la mayoría de esas series no se consulta nunca.

La dimensión culpable suele ser identificable: instancia tiene 24 valores hoy y crecerá con el autoescalado, además de generar series que mueren en cada despliegue. Un identificador efímero nunca debe ser dimensión de una métrica.

sin la dimensión instancia: $8 \times 40 \times 12 = 3.840$ series → 1.152 USD/mes

agrupando códigos en clases (2xx, 4xx, 5xx): $8 \times 40 \times 3 = 960$ → 288 USD/mes

Un factor de 96 respecto al diseño inicial, sin perder capacidad de diagnóstico: si hace falta saber qué instancia concreta falló, eso está en los logs, que se cobran por volumen ingerido y no por combinación.

La regla práctica:

métrica → pocas dimensiones, valores acotados y estables

log → todo el detalle, incluidos identificadores

traza → la relación entre servicios de una petición concreta

Y una forma económica de obtener métricas sin instrumentar: el formato de métrica embebida, que permite emitir un log estructurado del que CloudWatch extrae métricas automáticamente, pagando la ingesta del log en vez de la publicación de cada métrica.

3. Consultar registros de auditoría para reconstruir un incidente

CloudTrail responde quién hizo qué, y la consulta directa por atributos es limitada. Para investigaciones reales conviene usar el lago de datos o consultas sobre el bucket:

```
# Búsqueda rápida por evento, últimos 90 días
```

```
$ aws cloudtrail lookup-events \
```

```
--lookup-attributes AttributeKey=EventName,AttributeValue>DeleteBucketPolicy \
```

```
--query 'Events[].[EventTime,Username,CloudTrailEvent]' --output text | head -3
```

Y el detalle completo del evento, que es donde está lo útil:

```
{
  "eventTime": "2026-08-01T14:22:08Z",
  "userIdentity": {
    "type": "AssumedRole",
    "arn": "arn:aws:sts::123456789012:assumed-role/ci-deploy/despliegue-4821",
    "sessionContext": {"attributes": {"mfaAuthenticated": "false"}}
  },
  "sourceIPAddress": "203.0.113.40",
  "userAgent": "aws-cli/2.15.0",
  "requestParameters": {"bucketName": "cloudshop-facturas"},
  "errorCode": null
}
```

Tres campos que suelen decidir la investigación:

- sessionContext indica si hubo MFA. Una acción sensible sin MFA es un hallazgo por sí misma.
- sourceIPAddress distingue una acción desde el pipeline de una desde un portátil.
- errorCode revela los intentos fallidos, que son la señal más útil de un ataque en curso: alguien probando permisos que no tiene.

Dos límites que conviene conocer: el historial de consulta directa cubre 90 días; más atrás hay que ir al bucket. Y los eventos de datos no se registran por defecto —lecturas de objetos, invocaciones de funciones—: si hacen falta, hay que activarlos explícitamente y su volumen puede ser enorme.

Esa segunda limitación es exactamente lo que impidió, en la brecha de la clase 010, saber cuántas veces se descargaron las facturas: los eventos de gestión estaban registrados y los de datos no.

4. Config: qué cambió y detección continua de desviaciones

Config graba el estado de cada recurso a lo largo del tiempo, lo que permite responder cómo estaba antes de un cambio:

```
$ aws configservice get-resource-config-history \
  --resource-type AWS::RDS::DBInstance --resource-id db-ABCD1234 \
  --query 'configurationItems[0:2].[configurationItemCaptureTime,
    configuration.backupRetentionPeriod]' --output text
2026-08-01T14:22:31Z      0
2026-07-15T09:11:02Z      7
```

Y sobre ese historial se evalúan reglas de forma continua:

```
$ aws configservice put-config-rule --config-rule '{
  "ConfigRuleName": "rds-retencion-minima",
  "Source": {"Owner": "AWS", "SourceIdentifier": "DB_INSTANCE_BACKUP_ENABLED"},
  "InputParameters": "{\"backupRetentionPeriod\": \"7\"}"
}'
```

La distinción con las SCP de la clase 025 importa:

SCP	preventiva: impide que la acción ocurra
regla Config	detectiva: la acción ocurre y se marca como no conforme

Son complementarias y ninguna sustituye a la otra. Hay controles que no se pueden prevenir sin bloquear trabajo legítimo, y ahí la detección con corrección automática es la respuesta:

```
$ aws configservice put-remediation-configurations --remediation-configurations '[{
  "ConfigRuleName": "rds-retencion-minima",
  "TargetType": "SSM_DOCUMENT",
  "TargetId": "AWSConfigRemediation-ModifyRDSInstanceBackupRetention",
  "Automatic": true,
  "MaximumAutomaticAttempts": 3
}]'
```

Una advertencia sobre el coste: Config cobra por elemento de configuración registrado. En un entorno con autoescalado agresivo, cada instancia creada y destruida genera varios elementos, y la factura puede crecer de forma inesperada. Se acota limitando los tipos de recurso grabados a los que de verdad importan.

5. Operar la flota sin puertos abiertos ni claves

Systems Manager permite ejecutar comandos y sesiones interactivas sin abrir el puerto 22, sin claves SSH y sin IP pública. El agente inicia una conexión saliente, así que la instancia puede vivir en una subred aislada —la de la clase 027—.

```
# Sesión interactiva, auditada y sin SSH
$ aws ssm start-session --target i-0a1b2c3d

# Comando en toda una flota por etiqueta
$ aws ssm send-command --document-name AWS-RunShellScript \
  --targets Key=tag:Entorno,Values=produccion \
  --parameters 'commands=["systemctl restart cloudshop"]' \
  --max-concurrency 10% --max-errors 2
```

Los dos últimos parámetros son la diferencia entre una operación y un incidente: max-concurrency limita cuántas instancias se tocan a la vez y max-errors detiene la ejecución si demasiadas fallan. Sin ellos, un comando erróneo se aplica a las 200 instancias simultáneamente.

La ventaja sobre SSH no es la comodidad, son tres propiedades:

1. Auditado: cada sesión y cada comando quedan en CloudTrail
2. Sin credenciales de larga vida: usa el rol de instancia
3. Sin superficie de red: ningún puerto entrante abierto

Y los runbooks se vuelven ejecutables en vez de escritos:

```
schemaVersion: '0.3'
parameters:
  InstanceId: {type: String}
mainSteps:
  - name: DrenarDelBalanceador
    action: aws:executeAwsApi
    inputs: {Service: elbv2, Api: DeregisterTargets, ...}
  - name: EsperarDrenado
    action: aws:sleep
    inputs: {Duration: PT90S}
  - name: Reiniciar
    action: aws:runCommand
    inputs: {DocumentName: AWS-RunShellScript, ...}
```

La diferencia con un runbook en una wiki: este se ejecuta igual a las 3 de la madrugada que en un simulacro, no depende de que alguien recuerde el orden, y su ejecución queda registrada.

Ejemplo trabajado

A las 14:31 el equipo de CloudShop recibe una alerta: la retención de copias de la base de datos de producción es cero. Se reconstruye el incidente completo con los cuatro servicios.

¿Qué está pasando? — CloudWatch dio la alerta, pero no dice más:

```
$ aws cloudwatch describe-alarms --alarm-names config-rds-retencion \
  --query 'MetricAlarms[0].[StateValue,StateUpdatedTimestamp]' --output text
ALARM    2026-08-01T14:31:12Z
```

¿Qué cambió y cómo estaba antes? — Config:

```
$ aws configservice get-resource-config-history --resource-type AWS::RDS::DBInstance \
  --resource-id db-ABCD1234 --limit 2 \
  --query 'configurationItems[].[configurationItemCaptureTime,
    configuration.backupRetentionPeriod,configuration.multiAZ]' --output text
2026-08-01T14:22:31Z    0    true
2026-07-15T09:11:02Z    7    true
```

De 7 días a 0 a las 14:22. Nueve minutos antes de la alerta.

¿Quién lo hizo? — CloudTrail:

```
$ aws cloudtrail lookup-events \
  --lookup-attributes AttributeKey=ResourceName,AttributeValue=db-ABCD1234 \
  --start-time 2026-08-01T14:00:00Z \
  | jq -r '.Events[0].CloudTrailEvent | fromjson
  | [.eventTime, .userIdentity.arn, .sourceIPAddress,
    .requestParameters.backupRetentionPeriod] | @tsv'
```

2026-08-01T14:22:08Z arn:aws:sts::...:assumed-role/ci-deploy/despliegue-4821 203.0.113.40 0

Fue el pipeline, no una persona. Se busca el origen en el cambio de infraestructura:

el módulo de Terraform tenía backup_retention_period sin declarar
→ el proveedor aplicó su valor por defecto: 0
→ el plan mostraba el cambio y nadie lo leyó

Se comprueba si hubo más recursos afectados por la misma causa:

```
$ aws configservice select-resource-config \
  --expression "SELECT resourceId WHERE resourceType='AWS::RDS::DBInstance'
    AND configuration.backupRetentionPeriod = 0"
{"Results": [{"resourceId\":\"db-ABCD1234\""}, {"resourceId\":\"db-EFGH5678\""}]}
```

Dos instancias, no una. La segunda no tenía alerta porque la regla solo cubría producción.

Cuánto tiempo estuvo sin protección:

cambio	14:22:08
detección	14:31:12 (9 min: la regla se evalúa cada 10 min)
corrección	14:38:00
ventana sin copias	15 min 52 s
copias perdidas	ninguna: las anteriores se conservan 7 días desde su creación

Correcciones, separadas por tipo de control:

PREVENTIVO	SCP que deniega ModifyDBInstance con BackupRetentionPeriod=0 probado con prueba negativa: \$ aws rds modify-db-instance --backup-retention-period 0 ... AccessDenied ... explicit deny in a service control policy ✓
DETECTIVO	regla de Config ampliada a todas las cuentas, no solo producción corrección automática que restaura 7 días
ORIGEN	el módulo de Terraform declara el valor explícitamente y una prueba de política rechaza el plan si es menor que 7
OPERATIVO	documento de automatización para verificar retención en la flota, ejecutable en simulacro

Y se detecta un hueco al revisar el registro: los eventos de datos no estaban activados, así que no se puede saber si alguien accedió a las copias durante la ventana. Se activan para los buckets de copias, con el coste declarado:

eventos de datos en 3 buckets: $\sim 2,4 \text{ M eventos/mes} \times 0,10 \text{ USD/100k} = 2,40 \text{ USD/mes}$

Resultado: el incidente se reconstruyó completo en 12 minutos porque cada pregunta tenía su fuente. Sin Config no se habría sabido el estado anterior; sin CloudTrail, que fue el pipeline y no una persona; y sin la consulta agregada, que había una segunda instancia afectada.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/034-cloudwatch-cloudtrail-config-y-systems-manager/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa evidencia-operativa-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye evidencia-operativa-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
La factura de métricas personalizadas es de miles de dólares	Una dimensión de alta cardinalidad multiplica las series facturadas	Saca los identificadores efímeros de las dimensiones; ese detalle va en los logs, que se cobran por volumen.
Tras un incidente no se sabe si alguien accedió a los datos	Los eventos de datos de CloudTrail no se registran por defecto	Actívalos para los recursos sensibles y asume su coste; los de gestión no cubren lecturas de objetos.
Se sabe qué cambió pero no quién, o al revés	Se consultó un solo servicio: Config da el estado y CloudTrail el actor	Usa ambos: son complementarios y ninguno responde la pregunta del otro.
Un comando erróneo se aplica a toda la flota a la vez	Se ejecutó sin limitar concurrencia ni tolerancia a errores	Usa max-concurrency y max-errors en toda ejecución sobre flota.
La factura de Config crece de forma inesperada	Se graban todos los tipos de recurso y el autoescalado genera elementos continuamente	Limita los tipos grabados a los que importan para el cumplimiento y el diagnóstico.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué servicio responde «quién lo hizo» y cuál «cómo estaba antes»? ¿Por qué hacen falta los dos?
2. Una métrica con 4 dimensiones de 8, 40, 12 y 24 valores. ¿Cuántas series genera y cuál eliminarías primero?
3. ¿Qué campo de un evento de auditoría revela un ataque en curso mejor que las acciones exitosas?
4. ¿Qué diferencia hay entre una SCP y una regla de Config, y por qué ninguna sustituye a la otra?
5. ¿Qué tres propiedades aporta una sesión gestionada frente a SSH, más allá de la comodidad?

Referencias

- AWS (2024). CloudWatch: publishing custom metrics and dimensions — cardinalidad y facturación por serie.
<<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/publishingMetrics.html>>
- AWS (2024). CloudWatch Embedded Metric Format — métricas desde logs estructurados.
<<https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/CloudWatchEmbeddedMetricFormat.html>>

- AWS (2024). CloudTrail: logging data events — qué se registra por defecto y qué hay que activar.
<<https://docs.aws.amazon.com/awscloudtrail/latest/userguide/logging-data-events-with-cloudtrail.html>>
- AWS (2024). AWS Config rules and remediation — evaluación continua y corrección automática.
<<https://docs.aws.amazon.com/config/latest/developerguide/evaluate-config.html>>
- AWS (2024). Systems Manager Session Manager — acceso sin puertos abiertos ni claves, con auditoría.
<<https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 034 CloudWatch, CloudTrail, Config y Systems Manager

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega evidencia-operativa-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

035 — KMS, Secrets Manager, WAF y controles de seguridad

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Aplicar defensa en profundidad a la plataforma AWS: cifrado con control de claves propio, secretos que rotan sin desplegar, y filtrado en el borde. La clase 026 cubrió quién puede hacer qué; esta cubre qué ocurre cuando esa barrera falla, que es la única suposición razonable a la hora de diseñar seguridad.

Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir clave gestionada por el proveedor, gestionada por el cliente y material importado, con sus consecuencias.
2. Escribir una política de clave que impida el acceso incluso a quien tenga permisos sobre el recurso cifrado.
3. Rotar un secreto sin desplegar la aplicación, entendiendo la fase de dos versiones.
4. Configurar reglas de borde que corten ataques volumétricos y de aplicación sin bloquear tráfico legítimo.
5. Verificar cada control con prueba negativa, no con inspección de la configuración.

Conceptos centrales

Concepto	Comprensión verificable
cifrado de sobre	Cifrar los datos con una clave de datos y esa clave con una clave maestra. Permite cifrar volúmenes grandes sin llamar al servicio de claves por cada operación y rotar la maestra sin recifrar los datos.
política de clave	Documento que gobierna quién puede usar y administrar una clave. Es independiente de los permisos sobre el recurso cifrado: sin acceso a la clave, el acceso al dato no sirve.
rotación de secretos	Sustitución periódica de una credencial sin intervención humana. Exige una fase en la que dos versiones son válidas, o la rotación corta las conexiones en curso.
regla gestionada	Conjunto de reglas de filtrado mantenido por el proveedor o un tercero. Ahorra escribirlas y exige medirlas en modo cuenta antes de bloquear, porque generan falsos positivos.
modo cuenta	Configuración en la que una regla registra las coincidencias sin bloquear. Es el único modo seguro de introducir reglas nuevas en un servicio con tráfico real.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    U["tráfico"] --> SH["Shield · volumétrico"]
    SH --> WAF["WAF · reglas de aplicación<br/>modo cuenta primero"]
    WAF --> ALB["balanceador"]
    ALB --> APP["aplicación"]
    APP -->|"lee al arrancar\ny en cada rotación"| SM["Secrets Manager"]
    SM -->|"rota sin desplegar"| DB[("base de datos")]
    APP --> S3[("S3 cifrado")]
    S3 -->|"clave de datos"| KMS["KMS · clave del cliente"]
    KMS -->|"la política de clave\npuede denegar aunque\nel permiso de S3 conceda"| S3
```

Desarrollo

1. Tres tipos de clave, tres niveles de control

Tipo	Quién controla	Se puede	Coste
Gestionada por AWS	AWS	Nada: no se ve ni se audita su uso por separado	Gratis
Gestionada por el cliente	Tú	Política propia, rotación, desactivación, auditoría	1 USD/mes + uso
Material importado	Tú, fuera de AWS	Todo lo anterior más control del material	1 USD/mes + uso

La diferencia práctica entre la primera y la segunda es mayor de lo que sugiere la tabla:

clave gestionada por AWS
no puedes escribir su política
no puedes desactivarla
no puedes impedir que otro principal de la cuenta la use
→ el cifrado protege del disco físico y de poco más

clave del cliente
su política es una segunda barrera independiente de la del recurso
se puede desactivar y con ello hacer ilegibles los datos al instante
su uso se audita como evento propio

Esa segunda barrera es lo que convierte el cifrado en un control real. Con clave del cliente, alguien con permiso de lectura sobre el bucket pero sin permiso sobre la clave no puede leer nada:

```
{
  "Sid": "Solo el rol de la aplicación descifra",
  "Effect": "Allow",
  "Principal": {"AWS": "arn:aws:iam::123456789012:role/cloudshop-app"},
  "Action": ["kms:Decrypt", "kms:GenerateDataKey"],
  "Resource": "*",
  "Condition": {"StringEquals": {"kms:ViaService": "s3.sa-east-1.amazonaws.com"}}
}
```

La condición kms:ViaService añade una restricción útil: la clave solo se puede usar a través de S3, no directamente. Alguien que consiga el permiso no puede descifrar objetos que haya copiado a otro sitio.

Y el cifrado de sobre explica por qué esto no arruina el rendimiento: KMS no cifra los datos, cifra la clave de datos que sí los cifra. Un objeto de 5 GB implica una llamada a KMS, no cinco mil.

2. Rotación de claves: lo que rota y lo que no

La rotación automática de una clave de KMS crea material criptográfico nuevo y conserva el anterior:

los datos cifrados con el material antiguo NO se recifran
KMS guarda todas las versiones y usa la correcta al descifrar
las escrituras nuevas usan el material nuevo

Eso tiene dos consecuencias que sorprenden:

1. La rotación no invalida el material antiguo. Si alguien lo obtuvo, sigue sirviendo para lo cifrado antes. Rotar reduce la exposición futura, no la pasada.
2. No hay que hacer nada al rotar. No hay migración, no hay ventana, no hay recifrado. Por eso activarla no tiene coste operativo y no activarla no tiene excusa.

```
$ aws kms enable-key-rotation --key-id 1a2b3c4d --rotation-period-in-days 365
```

```
$ aws kms get-key-rotation-status --key-id 1a2b3c4d --query 'KeyRotationEnabled'
true
```

La acción que sí invalida el acceso es desactivar la clave:

```
$ aws kms disable-key --key-id 1a2b3c4d
# a partir de aquí, ningún descifrado funciona
```

Es la respuesta correcta ante un compromiso confirmado, y por eso la clave debe estar en una cuenta donde el atacante no tenga permisos. Si la clave y los datos comparten cuenta y credenciales, desactivarla no es una opción disponible durante el incidente.

Y el borrado de una clave tiene una salvaguarda deliberada: un periodo de espera de entre 7 y 30 días, irreversible después. Borrar una clave hace ilegibles todos los datos cifrados con ella, para siempre. No hay recuperación, ni por soporte. Es el equivalente criptográfico de un borrado seguro y hay que tratarlo con el mismo cuidado.

3. Rotar secretos sin cortar conexiones

Un secreto que rota mal corta las conexiones activas. La estrategia de cuatro pasos existe para evitarlo:

createSecret	genera la credencial nueva, la guarda como AWSPENDING
setSecret	la aplica en el servicio destino (crea el usuario nuevo)
testSecret	comprueba que funciona
finishSecret	mueve la etiqueta AWSCURRENT a la nueva versión

Durante los pasos 1 a 3 ambas credenciales son válidas. Esa ventana es lo que permite que las conexiones en curso terminen con la antigua mientras las nuevas usan la nueva.

La estrategia de usuario alterno lo lleva más lejos y es la recomendada para bases de datos:

usuario_a	activo, contraseña vigente
usuario_b	se rota este; cuando termina, se conmuta la etiqueta
siguiente rotación:	se rota usuario_a

Así nunca se toca la credencial que la aplicación está usando en ese momento.

El error que anula todo esto: cachear el secreto indefinidamente en la aplicación.

```
# mal: se lee una vez al arrancar y nunca más
SECRETO = obtener_secreto("cloudshop/db")

# bien: caché con caducidad y reintento ante fallo de autenticación
_cache = {"valor": None, "expira": 0}
def secreto():
    if time.time() > _cache["expira"]:
        _cache.update(valor=obtener_secreto("cloudshop/db"), expira=time.time() + 300)
    return _cache["valor"]

def conectar():
    try:
        return db.connect(**secreto())
    except AuthenticationError:
        _cache["expira"] = 0          # forzar relectura: quizá rotó
        return db.connect(**secreto())
```

El bloque except es la pieza que hace la rotación transparente: si la autenticación falla, se relea el secreto antes de rendirse. Sin él, la aplicación falla hasta que alguien la reinicie, y la rotación automática se convierte en una fuente de incidentes programados.

4. Filtrado en el borde: medir antes de bloquear

Las reglas gestionadas de WAF cubren categorías conocidas —inyección SQL, cruce de sitios, entradas maliciosas— y generan falsos positivos con tráfico real. Activarlas en modo bloqueo directamente es una forma fiable de cortar usuarios legítimos.

El procedimiento seguro:

1. Añadir la regla en modo cuenta
2. Observar 7-14 días de tráfico real, incluido un cierre de mes
3. Revisar las coincidencias: ¿son ataques o tráfico propio?
4. Excluir las reglas concretas que producen falsos positivos
5. Pasar a bloqueo

El paso 3 es donde aparecen las sorpresas: un campo de texto libre donde los usuarios escriben comillas o guiones dispara reglas de inyección; una carga de fichero grande dispara reglas de tamaño de cuerpo.

```
$ aws wafv2 get-sampled-requests --web-acl-arn $ACL --rule-metric-name AWSManagedRulesCommonRuleSet \
  --scope REGIONAL --time-window StartTime=...,EndTime=... --max-items 100 \
  | jq -r '.SampledRequests[] | [.Request.URI, .RuleNameWithinRuleGroup] | @tsv' \
  | sort | uniq -c | sort -rn | head -5
```

La regla más útil y la más barata de todas no es de contenido, es de tasa:

```
{
  "Name": "limite-por-ip",
  "Priority": 1,
  "Statement": {"RateBasedStatement": {"Limit": 2000, "AggregateKeyType": "IP"}},
  "Action": {"Block": {}}
}
```

Corta la fuerza bruta y el rastreo agresivo sin necesidad de entender el contenido. Dos matices: la ventana de evaluación es de 5 minutos, así que un pico corto puede pasar; y agregando por IP se penaliza a redes con NAT compartida —una oficina entera sale por una IP—, para lo que conviene agregar por otra clave, como una cabecera de sesión.

Y la capa anterior: la protección volumétrica estándar cubre ataques de red sin coste. El nivel avanzado añade respuesta gestionada y protección de costes frente al escalado provocado por un ataque, que es un riesgo real: un ataque que no tumba el servicio pero dispara el autoescalado produce una factura, no una caída.

5. Cada control necesita su prueba negativa

Igual que en la clase 025, la configuración no demuestra el efecto. Cada control de esta clase tiene su comprobación:

```
# 1. La política de clave deniega a quien no debe
$ aws s3api get-object --bucket cloudshop-facturas --key f-1042.pdf /tmp/x \
  --profile rol-sin-permiso-de-clave
An error occurred (AccessDenied) ... not authorized to perform: kms:Decrypt ✓

# 2. El secreto rotado sigue funcionando sin desplegar
$ aws secretsmanager rotate-secret --secret-id cloudshop/db
$ sleep 60 && curl -s -o /dev/null -w '%{http_code}\n' https://api.cloudshop.cl/health/ready
200 ✓

# 3. La regla de tasa corta el exceso
$ for i in $(seq 1 2500); do curl -s -o /dev/null https://api.cloudshop.cl/productos; done
$ curl -s -o /dev/null -w '%{http_code}\n' https://api.cloudshop.cl/productos
403 ✓

# 4. El tráfico legítimo NO se bloquea
$ curl -s -o /dev/null -w '%{http_code}\n' -X POST https://api.cloudshop.cl/opiniones \
  -d '{"texto":"El envío llegó rápido; muy bien -- lo recomiendo"}'
200 ✓
```

La cuarta es tan importante como las tres primeras y casi nunca se hace: ese texto contiene -- y ;, que las reglas de inyección marcan. Un control que bloquea tráfico legítimo es un incidente igual que uno que deja pasar un ataque, con la diferencia de que se descubre cuando los usuarios se quejan.

Y todas deben ser automáticas y periódicas. Una política de clave correcta hoy puede quedar anulada mañana por otra concesión; una exclusión de WAF puede volverse innecesaria o insuficiente cuando cambia el tráfico.

Ejemplo trabajado

Una revisión de seguridad de CloudShop encuentra tres huecos: cifrado con clave gestionada por AWS, la contraseña de la base de datos en una variable de entorno desde hace 14 meses, y ninguna protección de aplicación en el borde.

Hueco 1 — el cifrado no protegía de nada útil.

```
$ aws s3api get-bucket-encryption --bucket cloudshop-facturas \
  --query 'ServerSideEncryptionConfiguration.Rules[0].ApplyServerSideEncryptionByDefault'
{"SSEAlgorithm": "AES256"} # clave gestionada por AWS
```

Con esa configuración, cualquiera con s3:GetObject lee los objetos: no hay segunda barrera. Se migra a clave del cliente:

```
$ aws kms create-key --description "cloudshop facturas" --policy file://politica-clave.json
```

```
$ aws s3api put-bucket-encryption --bucket cloudshop-facturas \
--server-side-encryption-configuration '{"Rules":[{"
  "ApplyServerSideEncryptionByDefault":{"SSEAlgorithm":"aws:kms",
  "KMSMasterKeyID":"arn:aws:kms:...:key/1a2b3c4d"},
  "BucketKeyEnabled":true}]}'
```

BucketKeyEnabled reduce las llamadas a KMS reutilizando una clave a nivel de bucket:

```
sin clave de bucket: 1 llamada por objeto → 8,4 M llamadas/mes × 0,03/10k = 25,20 USD
con clave de bucket: ~99 % menos → 0,28 USD
```

Prueba negativa con un rol que tiene permiso sobre S3 y no sobre la clave:

```
AccessDenied ... not authorized to perform: kms:Decrypt ✓
```

Hueco 2 — la contraseña en variable de entorno.

```
$ aws ecs describe-task-definition --task-definition cloudshop-api:41 \
--query 'taskDefinition.containerDefinitions[0].environment[?name==`DB_PASSWORD`].name' --output
text
DB_PASSWORD
```

Una variable de entorno es visible en la definición de tarea, en los logs de despliegue y para cualquiera que pueda describirla. Se migra:

```
$ aws secretsmanager create-secret --name cloudshop/db \
--secret-string '{"username":"cloudshop_a","password":"..."}'
$ aws secretsmanager rotate-secret --secret-id cloudshop/db \
--rotation-lambda-arn arn:...:function:SecretsManagerRDSPostgreSQLRotationMultiUser \
--rotation-rules AutomaticallyAfterDays=30
```

La primera rotación cortó el servicio 4 minutos. La causa:

```
SECRETO = obtener_secreto("cloudshop/db") # leído una vez al arrancar
```

La aplicación cacheaba el secreto indefinidamente. Se corrige con caducidad y relectura ante fallo de autenticación, y se repite la prueba:

```
rotación #2: 0 s de indisponibilidad, 0 errores ✓
```

Hueco 3 — nada en el borde. Se añaden reglas gestionadas en modo cuenta:

```
tras 10 días en modo cuenta:
coincidencias totales          18.412
ataques reales (inyección, rastreo) 17.903
FALSOS POSITIVOS              509
```

Los 509 se concentran en un sitio:

```
$ aws wafv2 get-sampled-requests ... | jq -r '.SampledRequests[].Request.URI' \
| sort | uniq -c | sort -rn | head -2
486 /opiniones
23 /soporte/adjuntar
```

Opiniones de clientes con guiones y comillas disparaban SQLiBODY. Se excluye esa regla solo para esa ruta, no globalmente:

```
scope-down statement: NOT (URI = "/opiniones")
→ la regla sigue activa en todo lo demás
```

Y se añade la regla de tasa, que resultó ser la más efectiva:

```
límite 2.000 peticiones / 5 min por IP
bloqueos en la primera semana: 41 IP
tráfico malicioso cortado: 63 % del total de bloqueos
```

Resultado, con las cuatro pruebas:

cifrado con segunda barrera	no	sí	kms:Decrypt denegado ✓
secreto en variable de entorno	sí	no	rotación sin caída ✓
rotación automática	no	30 días	ejecutada 2 veces ✓
filtrado de aplicación	no	sí	exceso bloqueado ✓
tráfico legítimo con caracteres especiales	n/a	pasa	opinión con -- : 200 ✓
coste añadido	—	+38 USD/mes	

Los 509 falsos positivos son el dato que justifica el modo cuenta: activadas en bloqueo desde el principio, habrían impedido escribir opiniones a 486 clientes sin que nadie relacionara la causa.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/035-kms-secrets-manager-waf-y-controles-de-seguridad/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa controles-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye controles-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El cifrado está activado y cualquiera con permiso de lectura ve los datos	Clave gestionada por AWS: no hay política propia que actúe como segunda barrera	Usa clave del cliente con política que restrinja el descifrado a los roles previstos.
La primera rotación automática corta el servicio	La aplicación lee el secreto una vez al arrancar y lo cachea indefinidamente	Caché con caducidad y relectura ante fallo de autenticación.
Activar reglas de WAF bloquea a usuarios legítimos	Se pasó a bloqueo sin medir falsos positivos con tráfico real	Modo cuenta 7-14 días, excluye reglas concretas por ruta y solo entonces bloquea.
La factura de KMS crece con el volumen de objetos	Una llamada a KMS por objeto sin clave de bucket	Activa la clave de bucket: reduce las llamadas en torno al 99 %.
Durante un compromiso no se puede revocar el acceso a los datos	La clave vive en la misma cuenta que el atacante controla	Sitúa la clave en una cuenta separada; desactivarla es la palanca de emergencia.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué aporta una clave gestionada por el cliente que una gestionada por AWS no puede aportar?
2. ¿Recifra la rotación de una clave de KMS los datos existentes? ¿Qué implica eso si el material antiguo se filtró?
3. ¿Por qué la rotación de secretos necesita una fase con dos versiones válidas?
4. ¿Qué código debe tener una aplicación para que una rotación automática le resulte transparente?
5. ¿Por qué una prueba de que el tráfico legítimo pasa es tan necesaria como la de que el ataque se bloquea?

Referencias

- AWS (2024). AWS KMS: key policies — segunda barrera independiente de los permisos sobre el recurso.
<<https://docs.aws.amazon.com/kms/latest/developerguide/key-policies.html>>
- AWS (2024). Rotating AWS KMS keys — qué rota, qué se conserva y por qué no hay recifrado.
<<https://docs.aws.amazon.com/kms/latest/developerguide/rotate-keys.html>>
- AWS (2024). Secrets Manager rotation — los cuatro pasos y la estrategia de usuario alternativo.
<<https://docs.aws.amazon.com/secretsmanager/latest/userguide/rotating-secrets.html>>
- AWS (2024). AWS WAF managed rule groups — modo cuenta, exclusiones y sentencias de reducción de alcance.
<<https://docs.aws.amazon.com/waf/latest/developerguide/waf-managed-rule-groups.html>>
- AWS (2024). S3 Bucket Keys — reducción de llamadas a KMS y su efecto en el coste.
<<https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-key.html>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 035 KMS, Secrets Manager, WAF y controles de seguridad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega controles-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

036 — Proyecto: aplicación de tres capas en AWS

Parte: 02 — AWS: plataforma esencial Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Integrar las once clases anteriores en una plataforma que funcione, se observe, falle de forma controlada y cueste lo que se decidió que costara. No es un ejercicio de montar servicios: es demostrar con evidencia que cada decisión de las clases 025 a 035 produce el efecto que se le atribuyó, incluidas las que resulten estar mal.

Resultados de aprendizaje

Al finalizar podrás:

1. Ensamblar una aplicación de tres capas con las decisiones de las once clases previas, cada una justificada.
2. Demostrar con prueba negativa que los controles de identidad, red y cifrado deniegan lo que deben.
3. Medir una línea base de latencia y coste unitario que sirva de referencia para las partes siguientes.
4. Provocar tres fallos —zona, dependencia y credencial— y documentar el comportamiento observado.
5. Entregar un paquete de evidencia que otra persona pueda verificar sin conocimiento tácito.

Conceptos centrales

Concepto	Comprensión verificable
paquete de evidencia	Conjunto de salidas reproducibles que respaldan cada afirmación del diseño. Sustituye a «está bien configurado» por el comando ejecutado y su resultado.
prueba de fallo	Inyección deliberada de una avería para observar el comportamiento real. Un diseño resiliente que nunca se probó es una hipótesis, no una propiedad.
línea base	Medición inicial de latencia, rendimiento y coste unitario contra la que se comparará todo cambio posterior. Sin ella, «mejoró» es una opinión.
criterio de aceptación	Condición verificable que decide si el trabajo está terminado. Debe poder evaluarla alguien distinto de quien lo construyó.
riesgo residual	Lo que sigue siendo vulnerable tras aplicar los controles. Nombrarlo es parte de la entrega; omitirlo convierte un límite conocido en una sorpresa futura.

Modelo mental

Una cuenta AWS es una frontera de seguridad y facturación; los servicios se combinan mediante contratos, no como una lista de productos aislados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    U["usuarios"] --> WAF["WAF · límite de tasa"]
    WAF --> ALB["ALB · subred pública"]
    ALB --> ASG["ECS/EC2 · subred privada<br/>2 zonas · rol de tarea"]
    ASG --> RDS["RDS Multi-AZ<br/>subred aislada"]
    ASG --> S3["S3 · clave del cliente"]
    ASG --> SM["Secrets Manager<br/>rotación 30 d"]
    ASG --> Q["SQS + cola de fallidos"]
    Q --> W["consumidor idempotente"]
    ALL["CloudTrail · Config"] --> AUD["cuenta de auditoría"]
```


Desarrollo

1. La arquitectura y de dónde sale cada decisión

Nada en este capstone es una elección nueva: todo procede de una clase anterior, y esa trazabilidad es lo que hay que poder defender.

Componente	Decisión	Viene de
Cuentas separadas por entorno	Cuotas y radio de impacto	017, 025
Rol federado para el pipeline	Cero credenciales de larga vida	018, 026
Subred aislada para la base	Sin ruta a internet	027
Endpoints en vez de NAT	86 % del tráfico no va a internet	027
m7g en vez de t3	La media supera la línea base	028
Seguimiento de objetivo	Histéresis automática	016, 029
Drenado de 60 s	Mayor que la petición más larga	029
Bloqueo de objetos	El versionado no frena un borrado con permisos	010, 030
Índice compuesto	El orden estaba fuera del índice	031
Cola con fallidos e idempotencia	Entrega al menos una vez	009, 033
Clave del cliente	Segunda barrera independiente	035
Modo cuenta antes de bloquear	509 falsos positivos medidos	035

La columna derecha es el entregable real. Una arquitectura sin esa columna es una lista de servicios, y es exactamente lo que la clase 021 identificaba como diseño sin compromisos declarados.

Y hay decisiones que se toman en contra de lo que parece mejor, y también se registran: dos zonas en vez de tres porque el requisito es 21,6 min/mes y con dos ya sobran 20; RDS en vez de DynamoDB porque la agregación mensual no tiene equivalente. Justificar lo que no se hizo es tan parte de la defensa como lo que sí.

2. Cada control necesita su prueba negativa

La configuración no demuestra el efecto. El paquete de evidencia se construye ejecutando intentos que deben fallar y intentos que deben funcionar:

```
# Identidad: un repositorio ajeno no puede asumir el rol
$ (desde repo externo) aws sts assume-role-with-web-identity ...
AccessDenied ✓

# Red: la base de datos no alcanza internet
$ aws ssm start-session --target $ID_BASTION
$ curl -m 5 https://example.com
curl: (28) Connection timed out ✓
$ aws secretsmanager get-secret-value --secret-id cloudshop/db --query Name
"cloudshop/db" ✓ el endpoint sí

# Cifrado: sin permiso de clave no se lee, aunque haya permiso de S3
$ aws s3api get-object --bucket cloudshop-facturas --key f.pdf /tmp/x --profile sin-kms
not authorized to perform: kms:Decrypt ✓

# Almacenamiento: no se puede reabrir al público
$ aws s3api put-bucket-acl --bucket cloudshop-facturas --acl public-read
AccessDenied ... blocked by the public access block ✓

# Guardarriail: no se puede desactivar la auditoría
$ aws cloudtrail stop-logging --name org-trail
AccessDenied ... explicit deny in a service control policy ✓
```

Y las positivas, que son las que se olvidan:

```
$ curl -s -o /dev/null -w '%{http_code}\n' https://api.cloudshop.cl/health/ready
200 ✓
$ curl -s -X POST https://api.cloudshop.cl/opiniones \
```

```
-d '{"texto":"llegó rápido -- muy bien"}' -o /dev/null -w '%{http_code}\n'
200 ✓ WAF no bloquea
```

Cinco negativas y dos positivas. Un control que deniega todo también «pasa» las cinco primeras, y solo las positivas distinguen seguridad de indisponibilidad.

3. La línea base es el entregable que más dura

Las partes 03 a 23 compararán contra estos números, así que hay que medirlos bien y guardarlos versionados.

```
$ hey -z 120s -c 50 https://api.cloudshop.cl/productos/A-1042
```

```
Summary:
  Requests/sec: 987.42
Latency distribution:
  50% in 0.0384 secs
  95% in 0.0912 secs
  99% in 0.1847 secs
Status code distribution:
  [200] 118496 responses
```

Se verifica la coherencia con la ley de Little de la clase 011:

```
L = λ × W = 987 × 0,0384 = 37,9 concurrentes
solicitados: 50 → hay holgura; la medida no está limitada por el generador ✓
```

Si L hubiera dado 50 exactos, el número mediría el generador de carga y no el servicio.

Y el coste unitario, con el denominador de la clase 011:

cómputo (3 × m7g.large)	139,00
base de datos (Multi-AZ)	412,00
almacenamiento y endpoints	58,80
balanceador y WAF	47,20
observabilidad	31,40

total	688,40
pedidos/mes	980.000
coste unitario	688,40 / 980.000 = 0,000702 USD/pedido

Se registra como contrato comparable, con el escenario incluido:

```
{
  "escenario": "productos-lectura",
  "conurrencia": 50,
  "duracion_s": 120,
  "rps": 987.4,
  "p50_ms": 38.4,
  "p95_ms": 91.2,
  "p99_ms": 184.7,
  "errores": 0,
  "coste_mensual_usd": 688.40,
  "coste_unitario_usd": 0.000702,
  "zonas": 2,
  "instancias": 3,
  "fecha": "2026-08-01"
}
```

La relación p99/p50 = 4,8× es la métrica a vigilar entre versiones: si crece, la cola se alarga aunque la media no se mueva.

4. Tres fallos provocados y lo que enseña cada uno

Fallo 1 — pérdida de una zona.

```
$ aws ec2 modify-network-acl-entry --network-acl-id $ACL_ZONA_B \
  --rule-number 100 --egress --protocol -1 --rule-action deny --cidr-block 0.0.0.0/0

t+0   se aísla la zona B
t+31  el balanceador marca insanos sus destinos (10 s × 3)
t+31  el tráfico se concentra en la zona A
t+34  RDS conmuta a la réplica de la zona A
t+96  el autoescalado añade 2 instancias en A
p95 durante la ventana: 91 ms → 214 ms → 97 ms
errores: 0
```

Los 31 segundos de detección son el coste de los parámetros de la clase 029. Bajarlos aumentaría los falsos positivos.

Fallo 2 — dependencia opcional degradada. No caída, sino lenta, que es el caso que rompe sistemas:

```
$ aws ssm send-command --targets Key=tag:Servicio,Values=recomendaciones \
  --document-name AWS-RunShellScript \
  --parameters 'commands=["tc qdisc add dev eth0 root netem delay 800ms"]'

p95 sin degradación 91,2 ms
p95 con la dependencia a 800 ms 94,7 ms
```

El plazo de 150 ms corta la llamada. Sin él, cada petición ocuparía un trabajador 800 ms y por la ley de Little la concurrencia necesaria se multiplicaría por ocho.

Fallo 3 — credencial comprometida. Se simula qué obtiene un atacante con el rol de tarea:

```
$ aws iam list-users           AccessDenied    ✓
$ aws rds delete-db-instance ... AccessDenied    ✓
$ aws s3 rm s3://cloudshop-facturas/... AccessDenied    ✓ (bloqueo de objetos)
$ aws s3 ls s3://cloudshop-facturas/    lista objetos    ← Sí puede
$ aws s3 cp s3://cloudshop-facturas/f-1042.pdf .    descarga    ← Sí puede
```

El rol puede leer y exfiltrar las facturas. No es un fallo de configuración: es el permiso que la aplicación necesita. El control que queda es detectivo —alerta por volumen anómalo de lecturas— y hay que declararlo como riesgo residual, no ocultarlo.

5. La entrega: sin conocimiento tácito

El criterio de aceptación no es que funcione, sino que otra persona lo verifique sin preguntarte nada.

```
capstone-aws/
■■■■ README.md           qué es, cómo se despliega, cómo se comprueba, cómo se retira
■■■■ infra/              la infraestructura declarada, no clicada
■■■■ evidencia/
■ ■■■■ negativas.md      los 5 intentos que deben fallar, con su salida
■ ■■■■ positivas.md      los 2 que deben funcionar
■ ■■■■ baseline.json     la línea base con su escenario
■ ■■■■ fallo-zona.md      cronología medida
■ ■■■■ fallo-dep.md       p95 con y sin degradación
■ ■■■■ fallo-cred.md      qué obtiene un atacante con el rol de tarea
■■■■ adr/                las decisiones con sus alternativas descartadas
■■■■ verify.sh           ejecuta todas las comprobaciones y sale != 0 si alguna falla
```

verify.sh es la pieza que convierte la evidencia en algo vivo:

```
$ ./verify.sh
✓ rol federado deniega desde repositorio ajeno
✓ base de datos sin salida a internet
✓ objeto no legible sin permiso de clave
✓ bucket no reabrible al público
✓ CloudTrail no desactivable
✓ salud del servicio: 200
✓ opinión con caracteres especiales: 200
7/7 comprobaciones correctas
```

Un script que imprime «OK» y siempre sale 0 no verifica nada: el código de salida es el contrato, como en la clase 012. Y con él, estas mismas comprobaciones se ejecutan en CI en la parte 08 sin cambiar una línea.

Riesgos residuales que se entregan declarados:

1. El rol de tarea puede leer y exfiltrar facturas. Mitigación detectiva (alerta por volumen), no preventiva. Aceptado por el responsable de datos.
2. No sobrevive a un fallo regional completo. Aceptado: no es requisito hoy.
3. La línea base se midió con datos sintéticos; la distribución real de tamaños de pedido puede diferir.
4. El WAF excluye la regla de inyección en /opiniones. Compensado con validación y consultas parametrizadas en esa ruta.

Esa lista es la parte más valiosa de la entrega. Un capstone sin riesgos residuales declarados no es que no los tenga: es que no los buscó.

Ejemplo trabajado

Entrega del capstone de la parte 02, con las cifras que se llevan a la parte 03.

Verificación completa:

```
$ ./verify.sh
✓ rol federado deniega desde repositorio ajeno    AccessDenied
✓ base de datos sin salida a internet             timeout a los 5 s
✓ objeto no legible sin permiso de clave          kms:Decrypt denegado
✓ bucket no reabrible al público                 bloqueado
✓ CloudTrail no desactivable                     deny de SCP
✓ salud del servicio                             200
✓ opinión con " -- " y ";"                       200
```

7/7 correctas

Línea base:

```
rps 987,4 · p50 38,4 ms · p95 91,2 ms · p99 184,7 ms · errores 0
L = 987 × 0,0384 = 37,9 de 50 solicitados → medida válida
p99/p50 = 4,8×
coste 688,40 USD/mes · 0,000702 USD/pedido
```

Los tres fallos, con lo aprendido en cada uno:

zona aislada	31 s	214 ms	0	el coste de detección
dependencia a 800 ms	—	94,7 ms	0	son los parámetros de salud
credencial comprometida	—	—	—	el plazo hizo el trabajo; sin él, ×8 de concurrencia lee facturas: riesgo residual

Un hallazgo que la prueba de fallo destapó y la configuración no. Durante el aislamiento de la zona B:

```
$ aws logs filter-log-events --log-group-name /aws/ecs/cloudshop \
  --filter-pattern 'ERROR' --start-time ... --query 'length(events)'
0
$ aws sqs get-queue-attributes --queue-url $URL_DLQ \
  --attribute-names ApproximateNumberOfMessagesVisible
{"ApproximateNumberOfMessagesVisible": "14"}
```

14 mensajes en la cola de fallidos sin ningún error en los logs. El consumidor vivía solo en la zona B; al aislarla, sus mensajes agotaron reintentos mientras el servicio HTTP —que sí estaba en dos zonas— seguía en verde.

síntoma observable	ninguno: cero errores, p95 aceptable, alarmas en verde
consecuencia real	14 pedidos sin facturar
causa	el consumidor asíncrono no estaba en dos zonas

Se corrige y se añade lo que faltaba:

1. consumidor desplegado en dos zonas
2. alerta sobre la cola de fallidos con umbral cero (clase 033)
3. la prueba de fallo de zona pasa a incluir la ruta asíncrona, no solo la síncrona

El valor del capstone fue este hallazgo. Todo estaba correctamente configurado, todas las pruebas negativas pasaban, y había un camino —el asíncrono— que nadie había probado porque el fallo no era visible desde fuera.

Se entrega a la parte 03 con:

línea base	para comparar la misma aplicación en Azure
ADR	9 decisiones con sus alternativas descartadas
4 riesgos residuales declarados y aceptados por su responsable	
verify.sh	reutilizable, con código de salida

Y la pregunta que abre la parte siguiente: de estas nueve decisiones, ¿cuáles son de arquitectura y cuáles son de proveedor? Las primeras deberían reaparecer en Azure con otro nombre; las segundas, no reaparecer en absoluto.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-02-aws-core-platform/036-proyecto-aplicacion-de-tres-capas-en-aws/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El capstone es una lista de servicios desplegados	Falta la trazabilidad de cada decisión a la clase y al requisito que la motivó	Cada componente debe citar qué requisito satisface y qué alternativa se descartó.
Todas las pruebas negativas pasan y el sistema no funciona	Un control que deniega todo también las pasa; faltan las pruebas positivas	Empareja cada prueba negativa con una positiva del acceso legítimo.
Un fallo de zona deja trabajo asíncrono sin procesar y nada lo detecta	Solo se probó el camino síncrono, que sí estaba en dos zonas	Incluye colas y consumidores en la prueba de fallo, y alerta sobre la cola de fallidos.
No se puede afirmar si un cambio posterior mejoró o empeoró	No hay línea base con escenario, percentiles y coste unitario	Registra la medición versionada junto al código, con la carga y la duración usadas.
La entrega depende de que su autor explique algo	Conocimiento tácito no escrito y verificación no automatizada	Exige que otra persona ejecute verify.sh sin ayuda; lo que pregunte es lo que falta en el README.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. Elige tres componentes de tu capstone y di de qué clase y de qué requisito procede cada decisión.
2. ¿Por qué cinco pruebas negativas correctas no demuestran que el sistema sea seguro Y usable?
3. Tu medición da $L = 50$ con concurrencia solicitada de 50. ¿Qué estás midiendo en realidad?
4. Durante un fallo de zona no hay errores ni alarmas, pero se pierden pedidos. ¿Dónde buscarías?
5. De las decisiones de tu capstone, ¿cuáles esperas volver a tomar en Azure y cuáles no reaparecerán?

Referencias

- AWS (2024). Well-Architected Framework: the review process — cómo estructurar la revisión de una carga.
<<https://docs.aws.amazon.com/wellarchitected/latest/framework/the-review-process.html>>

- AWS (2024). Fault Injection Service — inyección controlada de fallos de zona, red y recursos.
<<https://docs.aws.amazon.com/fis/latest/userguide/what-is.html>>
- Beyer, B. et al., eds. (2018). The Site Reliability Workbook, cap. 5 — pruebas de fiabilidad y game days.
<<https://sre.google/workbook/testing-reliability/>>
- Nygard, M. (2018). Release It!, 2.ª ed., cap. 5 — plazos, mamparos y degradación bajo dependencia lenta.
- Nygard, M. (2011). Documenting Architecture Decisions — formato de los ADR que acompañan la entrega.
<<https://www.cognitect.com/blog/2011/11/15/documenting-architecture-decisions>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 036 Proyecto: aplicación de tres capas en AWS

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 03 — Azure: plataforma esencial

← Parte 02 · Índice completo · Parte 04 →

Nivel: intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar y operar una carga empresarial en Azure desde su jerarquía de gobierno.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
037	Tenant, management groups, suscripciones y resource groups	governance	4
038	Microsoft Entra ID, RBAC, managed identities y PIM	iam	4
039	Virtual Network, subredes, NSG, UDR, peering y Private Link	network	4
040	Virtual Machines, Scale Sets y Load Balancer	compute	4
041	Blob Storage, Files, redundancia y lifecycle	storage	4
042	Azure SQL, Cosmos DB y Azure Cache for Redis	data	4
043	App Service, Functions y Container Apps	serverless	4
044	Service Bus, Event Grid y Event Hubs	messaging	4
045	Azure Monitor, Log Analytics y Application Insights	observability	4
046	Key Vault, Defender for Cloud y Azure Policy	security	4
047	Bicep, plantillas y despliegues por alcance	iac	4
048	Proyecto: aplicación de tres capas en Azure	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Azure Architecture Center — Microsoft.
- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.

037 — Tenant, management groups, suscripciones y resource groups

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Construir la jerarquía de Azure y descubrir en qué se parece y en qué no a la de AWS. El ejercicio central de esta parte no es aprender nombres nuevos: es distinguir qué decisiones de la parte 02 eran de arquitectura —y reaparecen— y cuáles eran de proveedor, que es la pregunta con la que cerró la clase 036.

Resultados de aprendizaje

Al finalizar podrás:

1. Traducir la jerarquía de Azure a los conceptos neutrales de la clase 017 y señalar dónde no hay equivalencia.
2. Explicar por qué el grupo de recursos no es una frontera de aislamiento y qué sí lo es.
3. Escribir una directiva que restrinja regiones sin bloquear los recursos globales.
4. Distinguir cuota de suscripción de límite de servicio y anticipar cuál se agota antes.
5. Diseñar grupos de administración por régimen de gobierno, no por organigrama.

Conceptos centrales

Concepto	Comprensión verificable
tenant	Instancia de Microsoft Entra ID que contiene identidades y suscripciones. Es la raíz de la jerarquía y el equivalente conceptual de la organización, con una diferencia importante: la identidad vive ahí, no en la suscripción.
suscripción	Frontera de aislamiento, facturación y cuota. Es el equivalente funcional de la cuenta de AWS y del proyecto de Google Cloud.
grupo de recursos	Contenedor de recursos con ciclo de vida común dentro de una suscripción. No aísla nada: es unidad de despliegue y de borrado, y confundirlo con frontera de seguridad es el error más frecuente al llegar de AWS.
directiva	Regla que evalúa recursos y puede auditar, denegar, modificar o desplegar. A diferencia de una SCP, puede corregir además de prohibir, y se aplica también a lo que ya existe.
grupo de administración	Agrupador de suscripciones sobre el que se heredan directivas y asignaciones de rol. Admite hasta seis niveles de profundidad bajo la raíz.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    T["Tenant de Entra ID<br/>identidades y directorio"] --> R["Grupo de administración raíz"]
    R --> P["MG Plataforma"]
    R --> W["MG Cargas de trabajo"]
    R --> S["MG Sandbox"]
    P --> C["suscripción conectividad"]
    P --> ID["suscripción identidad"]
    P --> M["suscripción gestión"]
    W --> PR["MG Producción"] --> SP["suscripción pagos-prod"]
    W --> NP["MG No producción"] --> SD["suscripción pagos-dev"]
    SP --> RG1["grupo de recursos: red"]
    SP --> RG2["grupo de recursos: app"]
    R --> D["directiva: regiones permitidas"]
    W --> D
    RG1 --> F["NO es frontera:<br/>solo ciclo de vida"]
    RG2 --> F
```

Desarrollo

1. El mapa de equivalencias, y dónde se rompe

Concepto neutral (clase 017)	AWS	Azure	Google Cloud
Raíz	Organización	Tenant	Organización
Agrupador	Unidad organizativa	Grupo de administración	Carpeta
Frontera de aislamiento	Cuenta	Suscripción	Proyecto
Agrupador interno	—	Grupo de recursos	—
Política heredada	SCP	Azure Policy	Restricción de organización

Dos diferencias cambian el diseño y no son cosméticas:

La identidad vive en el tenant, no en la suscripción. En AWS cada cuenta tiene su propio IAM y el acceso entre cuentas exige asumir roles. En Azure, un usuario del tenant puede tener asignaciones de rol en varias suscripciones sin ningún salto: la frontera de identidad y la de aislamiento no coinciden. Eso simplifica la operación y hace que una identidad comprometida alcance potencialmente más superficie, así que la separación por asignaciones de rol importa más que en AWS.

Existe un nivel intermedio que AWS no tiene. El grupo de recursos agrupa recursos con ciclo de vida común. Su propiedad más útil es que borrarlo borra todo lo que contiene, lo que lo hace excelente para entornos efímeros y peligroso si se confunde con una frontera de seguridad.

Y una asimetría de las directivas frente a las SCP: Azure Policy puede modificar y desplegar, no solo denegar. Una directiva con efecto DeployIfNotExists puede añadir el diagnóstico que falta a un recurso recién creado. Es más potente y exige más cuidado: una directiva de corrección mal escrita cambia recursos en producción sin que nadie lo pida.

2. El grupo de recursos no aísla

Es el error de traducción más común al llegar de AWS. Un grupo de recursos parece una cuenta pequeña y no lo es:

```
grupo de recursos
✓ ciclo de vida común: se borra entero
✓ ámbito de asignación de roles
✓ ámbito de aplicación de directivas
✗ NO tiene cuotas propias
✗ NO es frontera de facturación
✗ NO impide que un recurso hable con otro de otro grupo
```

Las dos primeras exclusiones son las que producen incidentes. Las cuotas son de la suscripción, así que dos grupos de recursos en la misma suscripción compiten por los mismos núcleos disponibles — exactamente el problema de la clase 017 con otro nombre:

```
$ az vm list-usage --location brazilsouth --query "[?contains(name.value, 'cores')].{n:name.value,u:currentValue,l:limit}" -o table
n          u      l
-----
cores      186    200
```

186 de 200 núcleos consumidos en la suscripción, sin importar en cuántos grupos de recursos estén repartidos. Un experimento en el grupo «desarrollo» deja sin capacidad al grupo «producción» si comparten suscripción.

La tercera exclusión también sorprende: no hay aislamiento de red entre grupos de recursos. Dos máquinas en la misma red virtual se hablan aunque estén en grupos distintos; el aislamiento de red lo dan la red virtual y los grupos de seguridad, no la organización de recursos.

La regla de traducción: donde en AWS separarías por cuenta, en Azure separa por suscripción. El grupo de recursos es la unidad de despliegue, no de aislamiento.

3. Directivas: cuatro efectos y cuándo usar cada uno

Azure Policy va más allá de prohibir:

Efecto	Qué hace	Cuándo
Audit	Marca no conforme, no impide	Siempre primero: medir antes de bloquear
Deny	Impide la creación o el cambio	Controles duros, tras medir
Modify	Añade o cambia propiedades	Etiquetas obligatorias
DeployIfNotExists	Despliega un recurso asociado	Diagnóstico, agentes, copias

La disciplina es la misma que con el WAF de la clase 035: empezar en Audit. Una directiva Deny aplicada sin medir bloquea despliegues legítimos, y el equipo la desactiva en el primer incidente en vez de corregirla.

```
$ az policy assignment create --name regiones-permitidas \
  --scope /providers/Microsoft.Management/managementGroups/mg-cargas \
  --policy "e56962a6-4747-49cd-b67b-bf8b01975c4c" \
  --params '{"listOfAllowedLocations":{"value":["brazilsouth","eastus"]}}' \
  --enforcement-mode DoNotEnforce # equivale a Audit
```

Y la restricción de regiones tiene aquí el mismo problema que en la clase 025, con distinta solución. Varios tipos de recurso son globales y su ubicación declarada es global:

```
Microsoft.Network/trafficManagerProfiles
Microsoft.Network/dnsZones
Microsoft.Cdn/profiles
Microsoft.ManagedIdentity/userAssignedIdentities (en algunos casos)
```

La directiva integrada de ubicaciones permitidas ya excluye los recursos globales, pero una escrita a mano no. Comprobarlo antes de aplicarla evita bloquear la creación de zonas DNS sin entender por qué.

Y un detalle operativo: las directivas evalúan lo existente, no solo lo nuevo. Tras asignar una, aparece un informe de conformidad de todo lo que ya estaba. Es una ventaja sobre las SCP —que solo actúan sobre peticiones nuevas— y también una fuente de ruido inicial que hay que triar.

4. Cuotas: dos límites que se confunden

Azure tiene dos clases de límite y se agotan por motivos distintos:

```
cuota de suscripción    núcleos por familia y región, IP públicas, redes virtuales
                        → ampliable con solicitud, tarda horas o días

límite de servicio      reglas por grupo de seguridad, tamaño de plantilla,
                        recursos por grupo de recursos
                        → normalmente NO ampliable
```

La primera es la que aparece en el autoescalado y la segunda en el despliegue. Un caso frecuente del segundo tipo:

```
límite de reglas por grupo de seguridad de red: 1.000
un equipo genera una regla por cliente autorizado
→ el despliegue falla al llegar a 1.000 y NO se puede ampliar
→ la corrección exige rediseñar: grupos de seguridad de aplicación
  o listas de prefijos en vez de reglas individuales
```

La consulta de cuotas debe formar parte del plan de capacidad, no de la respuesta a incidentes:

```
$ az vm list-usage --location brazilsouth -o json \
  | jq -r '.[ ] | select((.currentValue / .limit) > 0.8)
```

```
| "\(.name.value): \(.currentValue)/\(.limit)"
standardDSv3Family: 142/150
```

El 80 % como umbral de alerta, igual que en la clase 017. Ampliar una cuota tarda horas o días, así que descubrirla durante un pico significa que ya no es una palanca disponible.

Y una diferencia con AWS que conviene saber: en Azure las cuotas de núcleos son por familia de máquina virtual además de por total. Tener 60 núcleos libres del total no sirve si la familia concreta que necesitas está al límite, y el mensaje de error no siempre lo deja claro.

5. Grupos de administración por gobierno, no por organigrama

El mismo criterio de la clase 017: agrupar por qué directiva se aplica, porque los organigramas se reorganizan y mover suscripciones entre grupos cambia las directivas heredadas.

La estructura de referencia de Microsoft para escala empresarial:

raíz	
■ ■ ■ ■ Plataforma	directivas estrictas, la opera el equipo central
■ ■ ■ ■ conectividad	redes virtuales de concentrador, DNS privado
■ ■ ■ ■ identidad	controladores de dominio, servicios de identidad
■ ■ ■ ■ gestión	registros, copias, automatización
■ ■ ■ ■ Cargas de trabajo	
■ ■ ■ ■ Producción	regiones, cifrado obligatorio, diagnóstico forzado
■ ■ ■ ■ No producción	regiones, límite de gasto
■ ■ ■ ■ Sandbox	directivas laxas, presupuesto duro, caducidad
■ ■ ■ ■ Desmantelado	suscripciones en retirada, solo lectura

Dos elementos que AWS no tiene explícitos y aquí conviene aprovechar:

El grupo «Desmantelado» recoge suscripciones que se van a cerrar, con directivas que impiden crear nada nuevo. Evita que una suscripción en retirada siga acumulando recursos durante meses.

La separación de plataforma en tres suscripciones —conectividad, identidad y gestión— responde a que cada una tiene un ciclo de vida y un equipo distintos. Es el equivalente a las cuentas de red, seguridad y auditoría de la clase 025.

Y una restricción práctica: la profundidad máxima es de seis niveles bajo la raíz. Suficiente, pero conviene no gastarlos en reflejar jerarquías organizativas que cambiarán.

Un recurso que no debe alojar cargas: el grupo de administración raíz. Igual que la cuenta de gestión de AWS, es el único punto sin nada por encima, y las asignaciones que se hacen ahí se heredan a todo el tenant.

Ejemplo trabajado

CloudShop despliega en Azure la misma aplicación de la parte 02. El equipo replica la estructura de AWS literalmente: un grupo de recursos por lo que allí era una cuenta. A las tres semanas, un despliegue de pruebas deja producción sin capacidad.

El incidente, idéntico al de la clase 017 con otro vocabulario:

```
$ az vm list-usage --location brazilsouth \
  --query "[?name.value=='standardDSv3Family'].{u:currentValue,l:limit}" -o tsv
148    150
$ az group list --query "[].name" -o tsv
rg-cloudshop-prod
rg-cloudshop-dev
rg-cloudshop-red
```

Tres grupos de recursos en una sola suscripción. El equipo tradujo «cuenta» por «grupo de recursos» y las cuotas no lo respetan:

cuota de la familia DSV3	150 núcleos
producción en régimen	88
experimento en rg-dev	60
total	148 → producción no puede escalar

Se verifica que el grupo de recursos tampoco aísla la red:

```
$ az network vnet subnet list -g rg-cloudshop-red --vnet-name vnet-cloudshop \
  --query "[].name" -o tsv
snet-prod
snet-dev
```

```
# una VM de rg-dev en snet-dev alcanza una de rg-prod en snet-prod:
$ az network watcher test-ip-flow --vm vm-dev-01 --direction Outbound \
  --local 10.30.2.4:0 --remote 10.30.1.7:5432 --protocol TCP -o tsv --query access
Allow
```

Ni cuotas ni red separadas. La traducción era incorrecta.

Rediseño con la equivalencia correcta:

```
tenant cloudshop
  mg-raiz
    mg-plataforma
      sub-conectividad      red de concentrador, DNS privado
    mg-cargas
      mg-produccion
      sub-cloudshop-prod    cuota propia
      mg-no-produccion
      sub-cloudshop-dev     cuota propia
    mg-sandbox
      sub-lab-*             presupuesto duro
```

Directiva de regiones, primero en modo auditoría:

```
$ az policy assignment create --name regiones --display-name "Regiones permitidas" \
  --scope /providers/Microsoft.Management/managementGroups/mg-cargas \
  --policy e56962a6-4747-49cd-b67b-bf8b01975c4c \
  --params '{"listOfAllowedLocations":{"value":["brazilsouth","eastus2"]}}' \
  --enforcement-mode DoNotEnforce
```

Tras 7 días, el informe de conformidad revela algo que nadie había notado:

```
$ az policy state summarize --management-group mg-cargas \
  --query "policyAssignments[0].results.{conformes:compliantResources,no:nonCompliantResources}"
{"conformes": 214, "no": 9}
$ az policy state list --management-group mg-cargas --filter "complianceState eq 'NonCompliant'" \
  --query "[].{r:resourceId,loc:resourceLocation}" -o tsv | awk '{print $2}' | sort | uniq -c
  7 westeurope
  2 global
```

Siete recursos en una región no aprobada —creados durante una prueba y olvidados— y dos globales, que son zonas DNS y no deben bloquearse. La directiva integrada ya los excluye; una escrita a mano los habría bloqueado.

Se migran los siete, se comprueba que la conformidad llega al 100 % y solo entonces se pasa a bloqueo:

```
$ az policy assignment update --name regiones --enforcement-mode Default
$ az group create --name rg-prueba --location westeurope
(RequestDisallowedByPolicy) Resource 'rg-prueba' was disallowed by policy  ✓
$ az network dns zone create -g rg-cloudshop-red -n cloudshop.cl
{... "location": "global" ...}  ✓ no bloqueado
```

Resultado, con la comparación que interesa para el resto del programa:

frontera de aislamiento	cuenta	suscripción
agrupador de gobierno	unidad organizativa	grupo de administración
cuota compartida entre entornos	no	no (tras el rediseño)
política heredada	SCP (solo denegar)	Directiva (auditar, denegar, modificar, desplegar)
identidad	por cuenta	por tenant ← DIFIERE
nivel intermedio	—	grupo de recursos ← DIFIERE

Las dos últimas filas son las decisiones de proveedor; el resto reaparece con otro nombre. Esa distinción es la que pedía la pregunta de cierre de la clase 036, y es la que hará que la parte 13 pueda hablar de portabilidad con criterio.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/037-tenant-management-groups-suscripciones-y-resource-groups/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee `exercise.steps` y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de `negativetest` y explica la señal observada.
4. Materializa jerarquía-azure en `evidence/` usando la plantilla indicada.
5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye jerarquía-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un entorno de pruebas deja sin capacidad a producción pese a estar en otro grupo de recursos	Las cuotas son de la suscripción; el grupo de recursos no aísla	Separa por suscripción donde en AWS separarías por cuenta; el grupo de recursos es ciclo de vida.
Una directiva de regiones bloquea la creación de zonas DNS	Los recursos globales declaran ubicación global y una directiva casera no los excluye	Usa la directiva integrada de ubicaciones permitidas, que ya los contempla.
Se aplica una directiva Deny y se paran despliegues legítimos	No se midió la conformidad del estado existente antes de bloquear	Asigna en modo auditoría, revisa el informe una o dos semanas y solo entonces refuerza.
Hay núcleos libres en la suscripción y la creación falla igual	La cuota es por familia de máquina además de por total	Consulta el uso por familia; el total disponible no garantiza capacidad de la familia que necesitas.
Un despliegue falla al superar el número de reglas de un grupo de seguridad	Es un límite de servicio, no una cuota: no se amplía	Rediseña con grupos de seguridad de aplicación o listas de prefijos en vez de reglas individuales.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Cuál es la frontera de aislamiento en Azure y por qué el grupo de recursos no lo es?
2. Nombra las dos diferencias estructurales entre la jerarquía de AWS y la de Azure que sí cambian el diseño.

3. ¿Qué puede hacer una directiva de Azure que una SCP no puede, y qué cuidado adicional exige?
4. Tienes 60 núcleos libres en la suscripción y la creación de una máquina falla. ¿Qué compruebas?
5. ¿Por qué una directiva se asigna primero en modo auditoría, y qué información produce esa fase?

Referencias

- Microsoft (2024). Organize your Azure resources with management groups — jerarquía, herencia y profundidad máxima. <<https://learn.microsoft.com/en-us/azure/governance/management-groups/overview>>
- Microsoft (2024). Azure Policy: understand policy effects — audit, deny, modify y deployIfNotExists. <<https://learn.microsoft.com/en-us/azure/governance/policy/concepts/effects>>
- Microsoft (2024). Azure subscription and service limits, quotas, and constraints — cuotas ampliables y límites duros. <<https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/azure-subscription-service-limits>>
- Microsoft (2024). Cloud Adoption Framework: enterprise-scale landing zones — estructura de grupos de administración de referencia. <<https://learn.microsoft.com/en-us/azure/cloud-adoption-framework/ready/landing-zone/>>
- Microsoft (2024). Azure Resource Manager: resource groups — ámbito, ciclo de vida y qué no aísla. <<https://learn.microsoft.com/en-us/azure/azure-resource-manager/management/manage-resource-groups-portal>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 037 Tenant, management groups, suscripciones y resource groups

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega jerarquía-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

038 — Microsoft Entra ID, RBAC, managed identities y PIM

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Dominar el modelo de identidad de Azure, que difiere del de AWS en algo estructural: el directorio es del tenant y los permisos sobre recursos son otra cosa. Confundir un rol de Entra ID con uno de Azure RBAC es la causa de la mitad de los problemas de acceso al llegar de AWS, y de una parte de las escaladas de privilegio.

Resultados de aprendizaje

Al finalizar podrás:

1. Distinguir roles de Entra ID de roles de Azure RBAC y saber cuál gobierna cada acción.
2. Explicar por qué una denegación de Azure RBAC no es equivalente a un Deny de IAM y qué la sustituye.
3. Sustituir secretos de aplicación por identidades administradas y por federación desde un pipeline.
4. Aplicar acceso con privilegios mínimos en el tiempo mediante activación con caducidad y aprobación.
5. Diagnosticar un acceso denegado sabiendo en qué ámbito y en qué sistema de roles buscar.

Conceptos centrales

Concepto	Comprensión verificable
rol de Entra ID	Permiso sobre el directorio: crear usuarios, conceder consentimiento a aplicaciones, gestionar el propio tenant. No otorga ningún permiso sobre recursos de Azure.
rol de Azure RBAC	Permiso sobre recursos, asignado en un ámbito —grupo de administración, suscripción, grupo de recursos o recurso—. Se hereda hacia abajo y es acumulativo.
identidad administrada	Identidad que Azure crea y rota por ti, asociada a un recurso. Elimina el secreto: no hay nada que guardar ni que filtrar.
asignación de denegación	Mecanismo que bloquea acciones con independencia de los roles concedidos. A diferencia de IAM, no se puede crear directamente: solo la generan servicios como los blueprints o los entornos gestionados.
acceso con privilegios mínimos en el tiempo	Modelo en el que un rol privilegiado no está activo permanentemente: se activa por un periodo acotado, con justificación y posible aprobación.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    subgraph dir["Plano del directorio · Entra ID"]
        U["usuario"] --> RE["roles de Entra ID<br/>Global Admin, App Admin..."]
        RE --> D["crear usuarios, consentir apps,<br/>gestionar el tenant"]
    end
    subgraph rec["Plano de recursos · Azure RBAC"]
        U --> RA["asignaciones de rol"]
        RA --> MG["ámbito: grupo de administración"]
        MG --> SUB["suscripción"] --> RG["grupo de recursos"] --> RES["recurso"]
    end
    dir -.->|un Global Admin puede<br/>CONCEDERSE acceso a todas<br/>las suscripciones| rec
```

Desarrollo

1. Dos sistemas de roles que no se solapan

Azure tiene dos planos de autorización independientes, y esa es la diferencia estructural con AWS:

	Roles de Entra ID	Roles de Azure RBAC
Gobiernan	El directorio	Los recursos
Ejemplos	Global Administrator, User Administrator	Owner, Contributor, Reader
Ámbito	Todo el tenant o unidades administrativas	Grupo de administración → recurso
Pregunta que responden	¿Puedes crear un usuario?	¿Puedes crear una máquina virtual?

Un Global Administrator no tiene por defecto ningún permiso sobre recursos. Puede crear usuarios y consentir aplicaciones, y no puede leer una base de datos.

Pero hay una conexión peligrosa que conviene conocer:

```
# Un Global Administrator puede concederse a sí mismo acceso a TODAS las suscripciones
$ az rest --method post --url "https://management.azure.com/providers/Microsoft.Authorization/elevateAccess?api-version=2016-07-01"
```

Esa llamada le asigna User Access Administrator en el ámbito raíz. Es una función legítima —recuperar el control de una suscripción huérfana— y una escalada completa. Debe alertarse siempre, y el número de Global Administrators debe ser mínimo: entre dos y cuatro, todos con acceso privilegiado en el tiempo.

Al depurar «no tengo permiso», la primera pregunta es en qué plano está la acción. Crear un grupo de recursos es RBAC; invitar a un usuario externo es Entra ID. Añadir permisos en el plano equivocado no cambia nada, igual que añadir permisos de identidad contra un Deny de SCP en la clase 026.

2. RBAC es aditivo y las denegaciones no se crean a mano

El modelo de evaluación de Azure RBAC es más simple que el de AWS y su simplicidad tiene una consecuencia importante:

permiso efectivo = unión de todas las asignaciones en todos los ámbitos heredados
menos las asignaciones de denegación (que no puedes crear)

Es aditivo. No existe un Deny que puedas escribir para excluir una acción concreta de un rol amplio. Si alguien tiene Contributor en la suscripción, tiene todo lo que ese rol incluye sobre todo lo que hay debajo, y no hay forma de restarle una acción con otra asignación.

Las dos formas reales de acotar:

1. Rol personalizado con NotActions
define exactamente lo que incluye y lo que excluye
2. Azure Policy con efecto Deny
no es autorización, es gobierno: impide la operación aunque el rol la permita

La segunda es la que sustituye funcionalmente al Deny de IAM, y opera en otro plano: el rol dice que puedes y la directiva impide que ocurra. Al diagnosticar, el mensaje de error las distingue:

AuthorizationFailed → falta el rol

RequestDisallowedByPolicy → el rol lo permite y la directiva lo impide

Un rol personalizado con exclusiones:

```
{
  "Name": "Colaborador sin borrado",
  "Actions": ["Microsoft.Compute/*", "Microsoft.Network/*", "Microsoft.Storage/*"],
  "NotActions": [
    "Microsoft.Compute/virtualMachines/delete",
    "Microsoft.Storage/storageAccounts/delete"
  ],
  "AssignableScopes": ["/subscriptions/aaaa-bbbb"]
}
```

Y una precisión que evita sorpresas: NotActions no es una denegación. Si otra asignación concede esa misma acción, se permite. Solo excluye la acción de ese rol.

3. Identidades administradas: el secreto que no existe

Es el equivalente al rol de instancia de AWS, con dos variantes:

	Asignada por el sistema	Asignada por el usuario
Ciclo de vida	Ligado al recurso: se borra con él	Independiente
Compatible	No	Sí, entre varios recursos
Cuándo	Un recurso único	Flotas y despliegues azul-verde

La segunda es la que conviene en una flota: si la identidad estuviera ligada al recurso, cada instancia nueva tendría una identidad distinta y habría que asignarle roles al crearla. Con identidad asignada por el usuario, los roles se conceden una vez y todas las instancias los heredan.

El código no maneja ningún secreto:

```
from azure.identity import DefaultAzureCredential
from azure.keyvault.secrets import SecretClient

cred = DefaultAzureCredential() # detecta la identidad administrada
cliente = SecretClient("https://kv-cloudshop.vault.azure.net", cred)
secreto = cliente.get_secret("db-password").value
```

DefaultAzureCredential prueba varias fuentes en orden: variables de entorno, identidad administrada, credenciales del desarrollador. Eso hace que el mismo código funcione en local y en producción, y también esconde una trampa: si la identidad administrada falla, puede caer silenciosamente a otra credencial y funcionar por el motivo equivocado. En producción conviene fijar la fuente explícitamente.

Para el pipeline, la equivalencia con la federación OIDC de la clase 026:

```
$ az ad app federated-credential create --id $APP_ID --parameters '{
  "name": "github-produccion",
  "issuer": "https://token.actions.githubusercontent.com",
  "subject": "repo:miorg/cloudshop:environment:produccion",
  "audiences": ["api://AzureADTokenExchange"]
}'
```

El campo subject cumple exactamente la misma función crítica que en AWS: sin él acotado, cualquier repositorio puede obtener el token. Y aquí hay un detalle propio: el valor es de coincidencia exacta, no admite comodines, lo que elimina el error de usar StringLike con un patrón demasiado amplio.

4. Privilegio mínimo en el tiempo, no solo en el alcance

El privilegio mínimo clásico acota qué puedes hacer. Añadir la dimensión temporal acota cuándo:

modelo permanente	el rol está activo 24x7 una credencial comprometida a las 3 de la madrugada tiene los mismos permisos que a mediodía
modelo elegible	el rol existe pero está inactivo se activa por 1-8 h, con justificación y a veces aprobación fuera de esa ventana, el permiso no existe

La reducción de superficie es aritmética:

rol permanente: 720 h/mes de exposición
rol elegible activado 4 h/semana: 16 h/mes
reducción: 97,8 %

La configuración añade tres controles a la activación:

duración máxima	8 h, no indefinida
justificación	obligatoria y registrada
aprobación	para los roles más sensibles
notificación	a un segundo par al activarse

La última es la más rentable y la menos usada: si alguien activa un rol privilegiado y otra persona lo ve al instante, un uso indebido se detecta en minutos en vez de en la revisión trimestral.

Los roles que deberían ser siempre elegibles y nunca permanentes:

Global Administrator (Entra ID)
Privileged Role Administrator (Entra ID)
Owner y User Access Administrator en suscripciones de producción (RBAC)

Y una excepción deliberada: conviene mantener dos cuentas de emergencia con acceso permanente, excluidas de acceso condicional y con credenciales en custodia física. Son la salida si el sistema de activación o el proveedor de identidad falla. Su uso debe alertar de inmediato, y su existencia hay que documentarla — porque una cuenta de emergencia olvidada es una puerta trasera.

5. Diagnosticar un acceso denegado

El orden que acota el problema en minutos:

```
# 1. ¿Qué roles tengo y en qué ámbito?  
$ az role assignment list --assignee $UPN --all -o table --include-inherited  
  
# 2. ¿La acción concreta está en alguno de esos roles?  
$ az role definition list --name "Storage Blob Data Reader" \  
  --query "[0].permissions[0].{a:actions,da:dataActions}"  
  
# 3. ¿Lo impide una directiva en vez de faltar el rol?  
$ az policy state list --resource $ID --filter "complianceState eq 'NonCompliant'" \  
  --query "[].policyDefinitionName"
```

El paso 2 esconde la sutileza que más confunde en Azure: existen actions y dataActions, y son planos distintos.

actions	operaciones sobre el recurso: crear la cuenta de almacenamiento, leer sus claves, cambiar su configuración
dataActions	operaciones sobre los DATOS: leer un blob concreto

Un rol Contributor sobre una cuenta de almacenamiento no permite leer los blobs — permite gestionarla y obtener sus claves, que es otra cosa. Leerlos exige un rol con dataActions, como Storage Blob Data Reader.

Eso produce el error más desconcertante para quien llega de AWS: alguien con Owner sobre la suscripción recibe AuthorizationPermissionMismatch al listar blobs. No es un fallo: es que Owner concede actions y no dataActions sobre datos.

Y el mensaje de error distingue los tres casos:

AuthorizationFailed	falta el rol de gestión
AuthorizationPermissionMismatch	falta el rol de DATOS
RequestDisallowedByPolicy	hay rol y una directiva lo impide

Leerlo ahorra la búsqueda en el plano equivocado, igual que en la clase 026.

Ejemplo trabajado

Al desplegar CloudShop en Azure, el equipo replica el rol del pipeline de AWS y se encuentra con tres problemas en dos días.

Problema 1 — el pipeline no puede leer el almacenamiento pese a ser Owner.

```
$ az role assignment list --assignee $APP_ID --all --query "[].{rol:roleDefinitionName,ambito:scope}"  
-o tsv  
Owner /subscriptions/aaaa-bbbb  
$ az storage blob list --account-name stcloudshop --container-name facturas --auth-mode login
```

```
AuthorizationPermissionMismatch
```

Owner sobre la suscripción y no puede listar blobs. La causa es la separación entre actions y dataActions:

```
$ az role definition list --name Owner --query "[0].permissions[0].dataActions" -o tsv
(vacío)
```

Owner no tiene ninguna dataAction. Se asigna el rol de datos, acotado al contenedor:

```
$ az role assignment create --assignee $APP_ID \
  --role "Storage Blob Data Contributor" \
  --scope "/subscriptions/aaaa-bbbb/resourceGroups/rg-prod/providers/\
Microsoft.Storage/storageAccounts/stcloudshop/blobServices/default/containers/facturas"
$ az storage blob list --account-name stcloudshop --container-name facturas --auth-mode login -o tsv
| wc -l
412
```

✓

Y se aprovecha para quitar Owner, que estaba de más:

```
antes: Owner sobre la suscripción entera
después: Storage Blob Data Contributor sobre UN contenedor
        + Website Contributor sobre el grupo de recursos de la app
```

Problema 2 — el pipeline usaba un secreto de aplicación.

```
$ az ad app credential list --id $APP_ID --query "[].{fin:endDateTime}" -o tsv
2027-03-14T10:22:00Z
```

Un secreto con dos años de vigencia, guardado en el repositorio. Se sustituye por federación:

```
$ az ad app federated-credential create --id $APP_ID --parameters '{
  "name":"github-produccion",
  "issuer":"https://token.actions.githubusercontent.com",
  "subject":"repo:miorg/cloudshop:environment:produccion",
  "audiences":["api://AzureADTokenExchange"]}'
$ az ad app credential delete --id $APP_ID --key-id $KEY_ID
```

Pruebas del sujeto, igual que en la clase 026:

```
desde otro repositorio          AADSTS70021: no matching federated identity ✓
desde miorg/cloudshop, rama de trabajo AADSTS70021 ✓
desde miorg/cloudshop, entorno produccion token emitido ✓
```

Problema 3 — un despliegue legítimo bloqueado.

```
RequestDisallowedByPolicy: Resource 'st-cloudshop-tmp' was disallowed by policy
'Storage accounts should restrict network access'
```

El mensaje no dice AuthorizationFailed, así que no falta rol: hay rol y una directiva lo impide. Añadir permisos no habría servido de nada.

```
$ az policy assignment show --name red-almacenamiento --query "parameters" -o json
{"effect":{"value":"Deny"}}
```

La cuenta temporal se creaba sin restricción de red. Se corrige en la plantilla, no en la directiva.

Revisión final de la superficie de identidad:

```
$ az role assignment list --all --query "[?roleDefinitionName=='Owner']\
.{p:principalName,a:scope}" -o tsv | wc -l
11
```

Once asignaciones de Owner. Se reducen a dos y el resto pasa a elegible con activación:

Owner permanentes	11	2 (emergencia, documentadas)
Owner elegibles	0	9 (activación de 8 h con justificación)
Global Administrators	6	3, todos elegibles
secretos de aplicación	1	0
exposición de roles privilegiados	720 h/mes	~22 h/mes (-97 %)

La lección que traslada esta clase al resto del programa: el modelo de identidad de Azure se parece al de AWS en el objetivo y difiere en la mecánica. Traducir literalmente —«Owner es como AdministratorAccess»— produce a la vez permisos de más en el plano de gestión y permisos de menos en el plano de datos.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/038-microsoft-entra-id-rbac-managed-identities-y-pim/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa identidad-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye identidad-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
Un principal con Owner recibe AuthorizationPermissionMismatch al leer datos	Owner concede actions y no dataActions: son planos distintos	Asigna un rol de datos como Storage Blob Data Reader, acotado al contenedor.
Se añaden roles y el despliegue sigue fallando	El error es RequestDisallowedByPolicy: hay rol y una directiva lo impide	Lee el código de error; con directivas, la corrección va en la plantilla o en la directiva, no en los roles.
No se puede excluir una acción concreta de un rol amplio	Azure RBAC es aditivo y las asignaciones de denegación no se crean a mano	Usa un rol personalizado con NotActions o una directiva con efecto Deny.
Un Global Administrator obtiene acceso a todas las suscripciones	La elevación de acceso es una función legítima del rol	Minimiza los Global Administrators, hazlos elegibles y alerta siempre sobre elevateAccess.
Cada instancia nueva de una flota necesita asignaciones de rol propias	Se usó identidad asignada por el sistema, ligada al ciclo de vida del recurso	Usa identidad asignada por el usuario: los roles se conceden una vez y las instancias la comparten.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué puede hacer un Global Administrator sobre los recursos de una suscripción, y cómo puede cambiar eso?
2. ¿Por qué NotActions en un rol personalizado no equivale a un Deny de IAM?
3. Un principal con Owner no puede listar blobs. ¿Cuál es la causa y cuál el arreglo?
4. ¿Qué distingue AuthorizationFailed de RequestDisallowedByPolicy y qué corrige cada uno?
5. ¿Cuánto se reduce la exposición al pasar un rol de permanente a elegible activado 4 h por semana?

Referencias

- Microsoft (2024). Azure RBAC vs Microsoft Entra roles — los dos planos y qué gobierna cada uno.
<<https://learn.microsoft.com/en-us/entra/identity/role-based-access-control/custom-overview>>
- Microsoft (2024). Understand Azure role definitions — actions, notActions, dataActions y su evaluación.
<<https://learn.microsoft.com/en-us/azure/role-based-access-control/role-definitions>>
- Microsoft (2024). Managed identities for Azure resources — asignadas por el sistema y por el usuario.
<<https://learn.microsoft.com/en-us/entra/identity/managed-identities-azure-resources/overview>>
- Microsoft (2024). Workload identity federation — credenciales federadas y coincidencia exacta del sujeto.
<<https://learn.microsoft.com/en-us/entra/workload-id/workload-identity-federation>>
- Microsoft (2024). Privileged Identity Management — activación con caducidad, justificación y aprobación.
<<https://learn.microsoft.com/en-us/entra/id-governance/privileged-identity-management/pim-configure>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 038 Microsoft Entra ID, RBAC, managed identities y PIM

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega identidad-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

039 — Virtual Network, subredes, NSG, UDR, peering y Private Link

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Diseñar la red de Azure sabiendo dónde falla la traducción literal desde una VPC. Tres puntos concretos rompen el modelo mental de la clase 027: no existe la subred privada por defecto, hay dos grupos de seguridad de red evaluándose en cadena y ambos deben permitir, y un punto de conexión privado sin zona DNS no falla — funciona por el camino público mientras crees que es privado. Es la base de las clases 040 a 048 y el origen de la mayoría de incidentes de conectividad de la parte.

Resultados de aprendizaje

Al finalizar podrás:

1. Planificar el espacio de direcciones contando las cinco IP reservadas por subred y las subredes de nombre obligatorio.
2. Explicar por qué una subred de Azure alcanza internet sin ninguna ruta declarada y qué cambió el 30 de septiembre de 2025.
3. Aplicar reglas de NSG sabiendo que subred y NIC se evalúan en cadena y que las reglas por defecto permiten todo dentro de la red virtual.
4. Decidir entre endpoint de servicio y punto de conexión privado con criterio de alcance, origen y costo.
5. Diagnosticar conectividad de abajo hacia arriba distinguiendo ruta, NSG y resolución de nombres.

Conceptos centrales

Concepto	Comprensión verificable
red virtual y espacio de direcciones	Ámbito de direccionamiento privado dentro de una región. Azure reserva cinco direcciones por subred —las cuatro primeras y la última—, así que una /24 ofrece 251 utilizables, no 254.
subred de nombre reservado	Subredes cuyo nombre es parte de la API: GatewaySubnet, AzureFirewallSubnet, AzureBastionSubnet. Escrito de otra forma, el servicio simplemente no se puede crear ahí.
ruta del sistema	Ruta que Azure crea sin que la declares, incluida 0.0.0.0/0 → Internet en toda subred. Es la razón de que no exista una subred privada por omisión.
UDR	Tabla de rutas definida por el usuario. Se elige por prefijo más largo y, a igualdad de prefijo, UDR gana a BGP y BGP gana al sistema.
NSG	Filtro con estado que puede aplicarse a la subred y a la NIC a la vez. Ambos se evalúan y ambos deben permitir; la primera regla que coincide decide.
punto de conexión privado	Interfaz de red con IP privada de tu subred que representa a un recurso concreto. Sin la zona DNS privada correspondiente no da error: el nombre sigue resolviendo a la IP pública.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
C["cliente en snet-app · 10.20.4.10"] --> R{"selección de ruta<br/>prefijo más largo"}
R -->| "UDR · gana a BGP y al sistema" | NVA["0.0.0.0/0 → firewall 10.20.0.4"]
R -->| "sistema" | VN["10.20.0.0/16 → VirtualNetwork"]
R -->| "sistema · /32 del endpoint" | PE["10.20.6.5 → punto de conexión privado"]
NVA --> N1["NSG de la NIC · salida"]
VN --> N1
PE --> N1
N1 --> N2["NSG de la subred · salida"]
N2 --> D["destino"]
C -. ->| "si falta la zona DNS privada<br/>el nombre resuelve a IP pública" | INT["salida a internet:<br/>funciona por el camino equivocado"]
```

Desarrollo

1. Cinco direcciones y tres nombres que no puedes elegir

El direccionamiento se planifica antes de crear nada, por la misma razón que en la clase 027: el emparejamiento entre redes virtuales rechaza espacios solapados, y dos equipos que eligieron 10.0.0.0/16 cada uno por su cuenta ya no pueden conectarse sin rehacer una de las dos.

Azure reserva cinco direcciones por subred, igual que AWS, pero con otros papeles:

```
10.20.1.0    identificador de red
10.20.1.1    puerta de enlace predeterminada
10.20.1.2    servicio DNS de Azure
10.20.1.3    reservada
10.20.1.255  difusión
→ una /24 ofrece 251 direcciones utilizables
```

Hasta aquí, aritmética conocida. Lo que no existe en AWS son las subredes cuyo nombre forma parte del contrato:

Nombre exacto	Tamaño mínimo	Para qué
GatewaySubnet	/27 recomendado	Puertas de enlace de VPN y ExpressRoute
AzureFirewallSubnet	/26	Azure Firewall
AzureFirewallManagementSubnet	/26	Plano de gestión del firewall forzado
AzureBastionSubnet	/26	Bastion

El nombre no es una convención, es la API. Una subred llamada snet-gateway con el tamaño correcto no sirve: la puerta de enlace no se puede crear ahí. Y el orden importa, porque estas subredes deben caber en el espacio que ya repartiste: descubrir que hace falta una /26 para el firewall cuando solo quedan /28 libres obliga a ampliar el espacio de direcciones de la red virtual y a recrear emparejamientos.

Un reparto que deja sitio a lo que vendrá en las clases 040 a 048:

```
vnet-hub    10.20.0.0/22
  AzureFirewallSubnet  10.20.0.0/26
  AzureBastionSubnet   10.20.0.64/26
  GatewaySubnet        10.20.0.128/27
  snet-dns              10.20.0.160/28

vnet-spoke-app  10.20.4.0/22
  snet-app        10.20.4.0/24    (251 utilizables)
  snet-datos      10.20.5.0/24
  snet-endpoints  10.20.6.0/26    (puntos de conexión privados)
  snet-integracion 10.20.6.64/26   (delegada a App Service)
```

La última merece una nota: una subred delegada a un servicio —App Service, Container Apps, bases de datos flexibles— queda bajo control de ese servicio y no admite otros recursos. Se decide al crearla y cambiarla implica vaciarla, así que conviene reservarla desde el principio aunque todavía no se use.

2. No existe la subred privada: toda subred sale a internet

Esta es la diferencia estructural con AWS, y la que más incidentes produce al llegar.

En AWS, «pública» y «privada» son propiedades de la tabla de rutas: una subred es pública porque su tabla apunta a una puerta de enlace de internet, y es privada porque no lo hace. En Azure no hay puerta de enlace que adjuntar. Toda subred nace con esta ruta del sistema:

0.0.0.0/0 → Internet

Una máquina virtual sin IP pública, sin UDR y sin NAT alcanza internet igualmente, mediante una traducción de direcciones implícita con una IP que elige la plataforma. Se llama acceso saliente predeterminado y tiene tres consecuencias:

1. La salida no está controlada: cualquier proceso sale sin que nadie lo declare.
2. La IP de origen no es estable ni tuya: no puede figurar en la lista de permitidos de un tercero, porque cambia sin aviso.
3. Los puertos de traducción son escasos y no se pueden dimensionar.

Qué cambió. Desde el 30 de septiembre de 2025 el acceso saliente predeterminado está retirado para las redes virtuales nuevas: un despliegue nuevo debe declarar cómo sale. Las redes virtuales creadas antes lo conservan, así que la pregunta útil sobre una plataforma existente no es «¿tenemos salida implícita?» sino «¿de qué fecha es esta red virtual?».

Las tres formas explícitas de salir, y por qué no son intercambiables:

Mecanismo	Puertos de traducción	Cuándo
IP pública en la NIC	64.000 por instancia	Una máquina suelta; no escala ni se audita bien
Regla de salida del balanceador	64.000 repartidos entre las instancias	Flotas pequeñas y estables
NAT Gateway	64.512 por IP, asignados bajo demanda, hasta 16 IP	Recomendado por defecto

El reparto de la segunda opción es la trampa aritmética:

64.000 puertos / 100 instancias = 640 puertos por instancia
un servicio que abre 200 conexiones cortas por segundo al mismo destino,
con 240 s de espera antes de reutilizar el puerto → agotamiento en segundos

El síntoma es característico y desorienta: conexiones que fallan por tiempo de espera solo hacia un destino concreto, mientras el resto de la red funciona. El NAT Gateway lo elimina porque asigna puertos bajo demanda en vez de repartirlos por adelantado.

Su costo tiene la misma forma que el de AWS, así que la lección de la clase 027 se traslada entera:

precio por hora ~0,045 USD → 32,85 USD/mes
precio por GB procesado ~0,045 USD

Y para conseguir de verdad una subred sin salida hacen falta tres cosas, no una:

```
# 1. desactivar el acceso saliente predeterminado en la subred
$ az network vnet subnet update -g rg-red --vnet-name vnet-spoke-app -n snet-datos \
  --default-outbound-access false

# 2. ninguna UDR que envíe 0.0.0.0/0 a Internet
# 3. NSG que deniegue la salida a la etiqueta Internet
$ az network nsg rule create -g rg-red --nsg-name nsg-datos -n deny-internet-out \
  --priority 4000 --direction Outbound --access Deny \
  --source-address-prefixes '*' --destination-address-prefixes Internet \
  --destination-port-ranges '*' --protocol '*'
```

Saltarse la primera es el error habitual: la regla de NSG bloquea el tráfico, pero la ruta sigue existiendo y basta con que alguien afloje el NSG para que la salida vuelva sin que nadie lo note.

3. Dos NSG en cadena y una regla por defecto que lo permite todo dentro

El NSG tiene estado, en la subred y en la NIC. Esto elimina de golpe la trampa de las ACL de red de AWS —aquellas reglas de vuelta para los puertos efímeros de la clase 027—, y a cambio introduce otra.

entrada: NSG de la subred → NSG de la NIC → máquina
salida: NSG de la NIC → NSG de la subred → red

Ambos se evalúan y ambos deben permitir. Un NSG de subred impecable no sirve de nada si la NIC tiene otro que deniega, y el mensaje de error no dice cuál de los dos fue. Por eso conviene una regla de equipo simple: NSG en la subred, salvo excepción documentada. Dos capas de filtrado en manos de dos equipos distintos producen incidentes

que nadie sabe reproducir.

Las reglas por defecto, que no se pueden borrar y sí sobrescribir con prioridad menor:

entrada	65000	AllowVnetInBound	VirtualNetwork → VirtualNetwork
	65001	AllowAzureLoadBalancerInBound	AzureLoadBalancer → *
	65500	DenyAllInBound	
salida	65000	AllowVnetOutBound	VirtualNetwork → VirtualNetwork
	65001	AllowInternetOutBound	* → Internet
	65500	DenyAllOutBound	

La primera es la que cambia el modelo mental. VirtualNetwork no significa «esta subred»: incluye toda la red virtual, las redes emparejadas y los rangos anunciados desde la red corporativa. Es decir:

AWS	dos instancias con grupos de seguridad distintos NO se hablan hasta que una regla lo permita
Azure	dos subredes cualesquiera de la red virtual SÍ se hablan, en todos los puertos, desde el primer minuto

La segmentación entre subredes en Azure es trabajo explícito, no un estado inicial. Denegar el tráfico lateral y abrir solo lo necesario:

```
$ az network nsg rule create -g rg-red --nsg-name nsg-datos -n allow-app-5432 \
--priority 100 --direction Inbound --access Allow --protocol Tcp \
--source-address-prefixes 10.20.4.0/24 --destination-port-ranges 5432

$ az network nsg rule create -g rg-red --nsg-name nsg-datos -n deny-lateral \
--priority 200 --direction Inbound --access Deny --protocol '*' \
--source-address-prefixes VirtualNetwork --destination-port-ranges '*'
```

Se evalúa por prioridad ascendente y la primera coincidencia decide: la 100 permite la aplicación y la 200 corta todo lo demás que venga de dentro, incluida la sonda del balanceador si no se tuvo cuidado. Ese es el incidente clásico, porque AllowAzureLoadBalancerInBound vive en la 65001 y cualquier denegación de usuario la sobrescribe:

sonda bloqueada → todas las instancias marcadas como no sanas
→ el balanceador deja de enviar tráfico
→ 502 en el frontal, y las máquinas están perfectamente vivas

La corrección es una regla de permiso con la etiqueta de servicio AzureLoadBalancer por encima de la denegación. Las etiquetas de servicio son la mejora real frente a mantener listas de IP a mano:

VirtualNetwork	la red virtual, las emparejadas y lo anunciado desde on-premises
AzureLoadBalancer	las sondas de estado de la plataforma
Internet	todo lo que no es privado ni de Azure
Storage.WestEurope	los rangos de almacenamiento de una región concreta
AzureMonitor, Sql, AzureKeyVault...	

Microsoft actualiza los rangos; tus reglas no cambian. Y para el equivalente al grupo de seguridad que referencia a otro grupo de seguridad —que en Azure no existe tal cual— está el grupo de seguridad de aplicación: una etiqueta que se asigna a NIC y se usa como origen o destino en las reglas, en lugar de un CIDR. Permite escribir «los servidores web pueden hablar con los de base de datos» sin depender de cómo se repartieron las subredes.

4. UDR, emparejamiento y las dos formas de dejar la red sin salida

La selección de ruta sigue dos criterios, en este orden:

1. prefijo más largo (la ruta más específica gana)
2. a igualdad de prefijo: UDR > BGP > sistema

Forzar todo el tráfico saliente por un firewall es una UDR de una línea sobre la subred de carga de trabajo:

```
$ az network route-table route create -g rg-red --route-table-name rt-app \
-n via-firewall --address-prefix 0.0.0.0/0 \
--next-hop-type VirtualAppliance --next-hop-ip-address 10.20.0.4
```

Y tiene dos formas conocidas de provocar una caída total:

La primera: aplicar esa misma UDR a la subred del firewall. El firewall enruta hacia sí mismo, el tráfico entra en bucle y la salida desaparece para toda la red. La regla es que AzureFirewallSubnet y GatewaySubnet no llevan una ruta por defecto hacia el propio dispositivo; asociar la tabla «a todas las subredes por comodidad» es exactamente cómo ocurre.

La segunda: el enrutamiento asimétrico. El tráfico entra por la IP pública del balanceador y, con la UDR puesta, la respuesta intenta salir por el firewall. El firewall ve un paquete de vuelta de una conexión que nunca vio empezar y lo descarta. El síntoma engaña: la conexión se establece y se queda colgada hasta agotar el tiempo, así que parece un problema de la aplicación. La corrección es una ruta más específica que devuelva ese tráfico por donde entró, o mover la entrada detrás del mismo dispositivo que inspecciona la salida.

Sobre el emparejamiento, tres propiedades que hay que tener presentes a la vez:

no es transitivo	radio A ↔ concentrador ↔ radio B NO da A ↔ B
es bidireccional	hay que crearlo en los dos sentidos o queda a medias
se factura en los dos	se paga el GB que sale y el GB que entra

La no transitividad se resuelve como en AWS, con distinta mecánica: una UDR en cada radio que envíe el rango del otro al firewall del concentrador, más dos interruptores del emparejamiento que casi siempre se olvidan:

allowForwardedTraffic	sin él, la red emparejada descarta el tráfico cuyo origen no le pertenece – es decir, todo lo reenviado
allowGatewayTransit / useRemoteGateways	permiten que los radios usen la puerta de enlace del concentrador en vez de tener una cada uno

El coste conviene calcularlo antes de dibujar el diagrama, porque un concentrador cobra el tránsito dos veces. Para 900 GB mensuales entre dos radios de la misma región:

emparejamiento radio A ↔ concentrador	
900 GB salida × 0,01 + 900 GB entrada × 0,01	18,00
emparejamiento concentrador ↔ radio B	
900 GB salida × 0,01 + 900 GB entrada × 0,01	18,00
proceso de datos del firewall 900 × 0,016	14,40
	■■■■■■■■
	50,40
emparejamiento directo A ↔ B	18,00

Inspeccionar ese tráfico cuesta 32,40 USD al mes. No es una cifra que descalifique el diseño; es una cifra que hay que poder decir en voz alta. La decisión defendible es la que elige el concentrador porque el tráfico entre radios debe inspeccionarse, no la que lo elige porque el diagrama de referencia lo dibujaba así.

5. Endpoint de servicio y punto de conexión privado: el que falla es el DNS

Dos mecanismos para llegar a un servicio de plataforma sin pasar por internet, con nombres parecidos y comportamientos distintos:

	Endpoint de servicio	Punto de conexión privado
Qué hace	Añade una ruta optimizada y presenta la identidad de la subred al servicio	Crea una NIC con IP privada de tu subred que representa al recurso
Dirección de destino	La IP pública del servicio	Una IP privada tuya
DNS	Sin cambios	Hay que cambiarlo
Alcance	El servicio en la región, acotado por el cortafuegos del recurso	Un recurso concreto, e incluso un subrecurso (blob, file, table)
Desde on-premises o red emparejada	No funciona	Sí
Costo	Gratuito	0,01 USD/h + 0,01 USD/GB procesado

La equivalencia con la clase 027 es casi exacta: el endpoint de servicio se comporta como el endpoint de tipo gateway de AWS —gratuito, basado en ruta, regional, inútil desde la red corporativa— y el punto de conexión privado como el endpoint de interfaz: una interfaz de red con IP privada, con costo por hora y por GB, alcanzable desde fuera de la red virtual.

Y aquí está el fallo más peligroso de esta clase, porque no se manifiesta como un error.

Crear el punto de conexión privado no cambia nada para el cliente. El nombre público sigue existiendo y sigue resolviendo hacia fuera:

```
stcloudshop.blob.core.windows.net
→ CNAME stcloudshop.privatelink.blob.core.windows.net
```

```
→ 20.60.x.x      (IP pública, si no hay zona privada)
→ 10.20.6.5      (IP del endpoint, si la zona privada está enlazada)
```

Sin la zona DNS privada, la aplicación sigue funcionando: sale a internet por el NAT, llega al almacenamiento por su IP pública y devuelve datos correctos. El punto de conexión privado está creado, facturado y sin usar. No hay alerta, no hay excepción, no hay traza roja en ningún panel. La única señal es una consulta:

```
$ nslookup stcloudshop.blob.core.windows.net      # desde dentro de snet-app
Address: 10.20.6.5                                ✓
```

Los tres pasos completos, en orden:

```
# 1. la zona privada, con el nombre EXACTO que corresponde al servicio
$ az network private-dns zone create -g rg-red -n privatelink.blob.core.windows.net

# 2. enlazarla a cada red virtual que deba resolverlo
$ az network private-dns link create -g rg-red -n link-spoke-app \
  -z privatelink.blob.core.windows.net -v vnet-spoke-app -e false

# 3. cerrar la puerta pública del recurso
$ az storage account update -n stcloudshop --public-network-access Disabled
```

El tercero es el que convierte esto en un control. Con la puerta pública abierta, el camino privado es solo un camino más: bonito en el diagrama e irrelevante para el auditor, porque cualquiera con la clave sigue entrando desde internet.

Dos detalles que ahorran una tarde de depuración:

Las zonas se centralizan. Una zona por nombre de servicio, alojada en la suscripción de conectividad y enlazada a todas las redes virtuales. Si cada equipo crea la suya, dos endpoints del mismo servicio en redes distintas producen resoluciones incoherentes según desde dónde preguntes. Lo sostenible es una directiva de Azure que registre automáticamente cada punto de conexión nuevo en la zona correcta — el mismo patrón de gobierno de la clase 037: la regla se aplica sola en vez de recordarse.

El NSG sobre el endpoint puede estar sin efecto. La subred tiene un interruptor, `privateEndpointNetworkPolicies`, que decide si los NSG y las UDR se aplican al punto de conexión. Si está deshabilitado, tus reglas se ignoran en silencio y el panel las muestra igualmente. Comprobarlo forma parte de la evidencia:

```
$ az network vnet subnet show -g rg-red --vnet-name vnet-spoke-app -n snet-endpoints \
  --query privateEndpointNetworkPolicies -o tsv
Enabled                                ✓
```

Ejemplo trabajado

CloudShop traslada a Azure la VPC diseñada en la clase 027. El plano se copia en una tarde y produce cuatro incidentes en tres semanas — cada uno en uno de los puntos donde la traducción no era válida.

El punto de partida, traducido literalmente:

```
vnet-spoke-app 10.20.4.0/22
snet-app       10.20.4.0/24   sin IP pública → «privada»
snet-datos     10.20.5.0/24   sin IP pública → «privada»
```

Incidente 1 — las subredes privadas no eran privadas.

Una revisión de dependencias detecta que el servicio de catálogo descarga paquetes durante el arranque. No debería tener salida.

```
$ ssh app-01 'curl -s ifconfig.io'
20.103.44.187
```

Hay salida, y con una IP que nadie declaró. Es el acceso saliente predeterminado: la red virtual se creó en 2024 y lo conserva. Se corrige con NAT Gateway explícito y se cierra la subred de datos:

```
$ az network nat gateway create -g rg-red -n natgw-spoke-app \
  --public-ip-addresses pip-nat --idle-timeout 10
$ az network vnet subnet update -g rg-red --vnet-name vnet-spoke-app -n snet-app \
  --nat-gateway natgw-spoke-app
$ az network vnet subnet update -g rg-red --vnet-name vnet-spoke-app -n snet-datos \
  --default-outbound-access false
$ ssh datos-01 'curl -s --max-time 5 ifconfig.io; echo rc=$?'
rc=28                                ✓
```

El tráfico de salida medido es de 3,2 TB/mes, de los cuales 2,4 TB son lecturas y escrituras contra el almacenamiento:

NAT Gateway 32,85 + 3.200 × 0,045 176,85

Incidente 2 — 502 en el frontal con todas las máquinas vivas.

Al aplicar segmentación se añadió una denegación del tráfico lateral:

```
200 deny-lateral Inbound VirtualNetwork → * Deny
```

A los diez minutos, el balanceador devuelve 502 y las cuatro instancias figuran como no sanas. Ninguna se ha caído.

```
$ az network watcher test-ip-flow --vm app-01 --direction Inbound --protocol TCP \
--local 10.20.4.10:8080 --remote 168.63.129.16:60000
Access: Deny RuleName: deny-lateral
```

La sonda de estado llega desde AzureLoadBalancer, permitida por defecto en la prioridad 65001 — y la regla 200 la sobrescribe. Se antepone el permiso explícito:

```
$ az network nsg rule create -g rg-red --nsg-name nsg-app -n allow-lb-probe \
--priority 100 --direction Inbound --access Allow --protocol '*' \
--source-address-prefixes AzureLoadBalancer --destination-port-ranges '*'
$ az network lb show -g rg-red -n lb-app --query "probes[0].name" -o tsv && \
curl -s -o /dev/null -w '%{http_code}\n' https://cloudshop.example
200 ✓
```

Incidente 3 — dos radios que no se ven, y la factura de arreglarlo.

El servicio de pedidos, en vnet-spoke-app, debe llamar al de facturación, en vnet-spoke-fin. Ambos están emparejados con el concentrador y no se alcanzan entre sí: el emparejamiento no es transitivo. Se añaden las UDR hacia el firewall y sigue sin funcionar.

```
$ az network vnet peering show -g rg-red --vnet-name vnet-spoke-app -n to-hub \
--query "{fwd:allowForwardedTraffic}" -o tsv
False
```

El concentrador descartaba el tráfico reenviado. Con el interruptor activado en los cuatro emparejamientos —dos por sentido— la ruta se completa. El volumen entre radios es de 900 GB/mes:

vía concentrador con inspección	50,40
emparejamiento directo entre radios, sin inspección	18,00
diferencia: el precio de inspeccionar	32,40

Se deja el paso por el concentrador y se anota el motivo: son llamadas que cruzan una frontera de datos de pago y deben registrarse. La cifra queda escrita en la decisión, no descubierta en la factura.

Incidente 4 — el punto de conexión privado que llevaba cuatro meses sin usarse.

Una auditoría pide evidencia de que el acceso al almacenamiento no cruza internet. El endpoint existe desde el primer despliegue y la aplicación funciona sin fallos.

```
$ ssh app-01 'nslookup stcloudshop.blob.core.windows.net | tail -2'
Address: 20.60.191.132
```

Una IP pública. Nunca se creó la zona DNS privada, así que el tráfico salía por el NAT y volvía por la puerta pública del almacenamiento. Todo funcionaba, y ninguna de las dos afirmaciones del diagrama era cierta.

```
$ az network private-dns zone create -g rg-hub -n privatelink.blob.core.windows.net
$ az network private-dns link vnet create -g rg-hub -n link-spoke-app \
-z privatelink.blob.core.windows.net -v vnet-spoke-app -e false
$ az storage account update -n stcloudshop --public-network-access Disabled

$ ssh app-01 'nslookup stcloudshop.blob.core.windows.net | tail -2'
Address: 10.20.6.5 ✓
$ curl -s -o /dev/null -w '%{http_code}\n' \
https://stcloudshop.blob.core.windows.net/facturas # desde fuera de la red
403 ✓
```

La prueba negativa es la mitad que faltaba: que resuelva a una IP privada demuestra que el camino existe; que desde fuera devuelva 403 demuestra que es el único.

Y se recuperan 2,4 TB del NAT:

tráfico por NAT Gateway	3.200 GB	800 GB	
costo del NAT 32,85 + GB × 0,045	176,85	68,85	
punto de conexión privado (7,30 + 2.400 × 0,01)		0,00 → ya facturado	31,30
■■■■■■■■■■	■■■■■■■■■■		
176,85	100,15	(-43 %)	

El punto de conexión ya se estaba pagando desde el primer día. Enlazar la zona DNS no añadió costo: hizo que el gasto sirviera para algo.

Resumen del rediseño:

subredes con salida no declarada	2	0
camino de datos al almacenamiento	internet	privado
acceso público al almacenamiento	habilitado	deshabilitado
tráfico lateral permitido por defecto	todos los puertos	5432 desde snet-app
salida mensual por NAT	3,2 TB	0,8 TB
costo mensual de red	227,25 USD	150,55 USD

La lección que esta clase traslada al resto de la parte: en Azure, la conectividad es el estado por defecto y el aislamiento es trabajo explícito. Al revisar un diseño heredado, la pregunta útil no es «¿qué hemos abierto?» sino «¿qué hemos cerrado, y cómo lo demostramos?».

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/039-virtual-network-subredes-nsg-udr-peering-y-private-link/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa vnet-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye vnet-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Una máquina sin IP pública en una subred «privada» descarga desde internet	La ruta del sistema 0.0.0.0/0 → Internet existe en toda subred y el acceso saliente predeterminado la resuelve	Declara la salida con NAT Gateway y desactiva defaultOutboundAccess en las subredes que no deben salir; el NSG por sí solo no basta.
El balanceador devuelve 502 y todas las instancias figuran como no sanas, pero están vivas	Una regla de denegación de usuario sobrescribe AllowAzureLoadBalancerInBound, que vive en la prioridad 65001	Añade un permiso explícito para la etiqueta AzureLoadBalancer con prioridad menor que la denegación.
Dos subredes se hablan en todos los puertos sin que nadie lo haya permitido	AllowVnetInBound permite por defecto todo el tráfico de la red virtual, las emparejadas y lo anunciado desde on-premises	La segmentación es explícita: permite lo necesario y deniega el resto del origen VirtualNetwork.
El punto de conexión privado existe, la aplicación funciona y el tráfico sigue saliendo a internet	Falta la zona DNS privada enlazada, así que el nombre resuelve a la IP pública del servicio	Crea y enlaza privatelink.<servicio>, verifica con nslookup desde dentro y deshabilita el acceso público del recurso.
La conexión se establece y se queda colgada hasta agotar el tiempo de espera	Enrutamiento asimétrico: la entrada llega por el balanceador y la UDR devuelve la respuesta por el firewall, que nunca vio empezar la conexión	Añade una ruta más específica que devuelva ese tráfico por donde entró, o unifica entrada y salida en el mismo dispositivo.
Se pierde la salida de toda la red al aplicar una tabla de rutas	La UDR con 0.0.0.0/0 → dispositivo virtual se asoció también a AzureFirewallSubnet o a GatewaySubnet	Asocia las tablas de rutas subred por subred; esas dos nunca llevan la ruta por defecto hacia el propio dispositivo.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Por qué una máquina virtual sin IP pública y sin UDR alcanza internet, y qué tres cambios hacen falta para impedirlo?
2. Un NSG de subred permite el tráfico y la conexión falla. ¿Dónde más hay que mirar y por qué el error no lo indica?
3. ¿Qué permite AllowVnetInBound exactamente, y en qué se diferencia del comportamiento por defecto entre grupos de seguridad de AWS?
4. ¿Cuándo elegirías un endpoint de servicio en vez de un punto de conexión privado, y qué requisito descarta al primero?
5. El nslookup de una cuenta de almacenamiento devuelve una IP pública desde dentro de la red virtual y la aplicación funciona. ¿Qué está ocurriendo y qué evidencia demuestra la corrección?

Referencias

- Microsoft (2025). Default outbound access in Azure — la salida implícita y su retirada para redes virtuales nuevas. <<https://learn.microsoft.com/en-us/azure/virtual-network/ip-services/default-outbound-access>>
- Microsoft (2025). Network security groups — reglas por defecto, evaluación en subred y NIC, y etiquetas de servicio. <<https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview>>
- Microsoft (2025). Virtual network traffic routing — rutas del sistema y precedencia entre UDR, BGP y sistema. <<https://learn.microsoft.com/en-us/azure/virtual-network/virtual-networks-udr-overview>>
- Microsoft (2025). Private endpoint DNS configuration — zonas privatelink, enlaces a redes virtuales y resolución desde on-premises. <<https://learn.microsoft.com/en-us/azure/private-link/private-endpoint-dns>>

- Microsoft (2025). Hub-spoke network topology — transitividad, tráfico reenviado y tránsito de puerta de enlace.
<<https://learn.microsoft.com/en-us/azure/architecture/networking/architecture/hub-spoke>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 039 Virtual Network, subredes, NSG, UDR, peering y Private Link

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega vnet-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

040 — Virtual Machines, Scale Sets y Load Balancer

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: compute · Estado: EXECUTABLECORE

Propósito

Dimensionar cómputo en Azure sabiendo que hay dos límites de disco y casi siempre manda el que no compraste: el de la máquina. Las clases 028 y 029 dejaron el criterio —familia, IOPS, elasticidad, comprobación de estado—; aquí cambian las piezas y aparecen decisiones que en AWS no existen: los dominios de actualización, las dos orquestaciones de un conjunto de escalado, cuatro balanceadores con un reparto distinto y un aviso de desalojo de 30 segundos en vez de dos minutos.

Resultados de aprendizaje

Al finalizar podrás:

1. Leer el nombre de un tamaño de máquina virtual como una especificación y detectar qué falta cuando no lleva s ni d.
2. Distinguir el límite de IOPS del disco del límite de la máquina y localizar cuál es el cuello de botella real.
3. Elegir entre conjunto de disponibilidad, zonas y conjunto de escalado según el fallo que se quiere sobrevivir.
4. Configurar un conjunto de escalado con orquestación flexible, mezcla de prioridades y política de reducción explícita.
5. Seleccionar entre los cuatro balanceadores de Azure a partir del ámbito, la capa y el tiempo de conmutación aceptable.

Conceptos centrales

Concepto	Comprensión verificable
tamaño de máquina virtual	Cadena con estructura: familia, número de vCPU y letras aditivas. s habilita discos premium, d añade disco temporal local, a indica AMD, p indica ARM. Sin la letra, la capacidad no existe.
límite de la máquina frente al del disco	Cada tamaño tiene un tope propio de IOPS y de rendimiento de disco, distinto del que ofrece el disco. El caudal real es el menor de los dos.
almacenamiento en caché del host	Modo de caché del disco en el anfitrión: None, ReadOnly o ReadWrite. Cambia el límite aplicable y, en discos de registro de transacciones, ReadWrite puede costar datos.
dominio de actualización	Grupo de máquinas que el mantenimiento planificado de la plataforma reinicia a la vez. Un conjunto de disponibilidad con cinco garantiza que como mucho una quinta parte se reinicie de golpe.
orquestación flexible	Modo de conjunto de escalado en el que las instancias son máquinas virtuales reales, gestionables una a una y con tamaños o prioridades mezcladas. Es el modo por defecto.
aviso de desalojo	Notificación de eventos programados que anuncia el desalojo de una máquina de excedente. En Azure son 30 segundos, no dos minutos: solo sirve para abortar, no para drenar.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart TB
    REQ["requisito medible<br/>IOPS · latencia · SLO · presupuesto"] --> T{"¿qué fallo hay que sobrevivir?"}
    T --> M["mantenimiento del anfitrión"]
    AS["conjunto de disponibilidad<br/>dominios de error y actualización"]
    T --> C["caída de un centro de datos"]
    AZ["zonas de disponibilidad"]
    T --> D["demanda variable"]
    SS["conjunto de escalado<br/>orquestación flexible"]
    AS --> D1["caudal de disco"]
    AZ --> D
    SS --> D
    D --> MIN["real = min(límite del disco,<br/>límite de la máquina)"]
    MIN --> LB1["ámbito del reparto"]
    LB --> C4["capa 4 regional"]
    L4["Load Balancer"]
    LB --> C7["capa 7 regional"]
    L7["Application Gateway"]
    LB --> G["global, conmuta en segundos"]
    FD["Front Door"]
    LB --> G2["global por DNS,<br/>conmuta según TTL"]
    TM["Traffic Manager"]
```

Desarrollo

1. Las letras del tamaño son la especificación

La clase 028 dejó establecido que el nombre de una instancia es una especificación, no una etiqueta. En Azure el alfabeto es otro y el orden importa:

```
Standard_D8ads_v6
■ ■■■ generación
■ ■■■■■ s : admite discos premium
■ ■■■■■ d : incluye disco temporal local
■ ■■■■■ a : procesador AMD
■ ■■■■■ D8 : familia de propósito general, 8 vCPU
```

Las familias, en una línea cada una:

B	ráfaga con créditos	cargas ociosas la mayor parte del tiempo
D	propósito general	4 GB de memoria por vCPU
E	memoria	8 GB por vCPU — bases de datos, cachés
F	cómputo	2 GB por vCPU — cálculo, compilación
L	almacenamiento	NVMe local grande y muy rápido
M	memoria extrema	cargas SAP y bases de datos monolíticas
N	GPU	

Las dos letras que más problemas dan son las que no están:

Sin s, el tamaño no admite discos premium. Se descubre al desplegar, con un error de compatibilidad que no menciona la letra, y obliga a cambiar el tamaño —y por tanto a reiniciar— con la plantilla ya en producción.

Sin d, no hay disco temporal local. Y aquí está el error silencioso: mucha automatización heredada asume que existe /mnt en Linux o D: en Windows porque «siempre estuvo ahí». En un D8sv5 no está. El servicio arranca, escribe en la raíz del disco del sistema operativo, y semanas después el disco se llena o el rendimiento cae, porque el disco del sistema no está dimensionado para eso.

Y una advertencia sobre el disco temporal cuando sí existe: es efímero de verdad. Se borra al desasignar la máquina y al moverla de anfitrión, sin aviso y sin copia. Sirve para archivos de intercambio, cachés reconstruibles y ficheros temporales de compilación. Poner ahí datos de una base de datos es la versión de Azure del problema que cerró la clase 028: el estado local es el enemigo de la elasticidad, y en este caso ni siquiera hay que escalar para perderlo.

Sobre la familia B basta con una nota, porque el mecanismo de créditos ya se trabajó en 028: es el mismo desplome, con distinta contabilidad. La diferencia práctica es que la señal se llama CPU Credits Remaining y hay que graficarla junto al porcentaje de CPU. Sin ella, un servicio que agota créditos aparece en el panel como una CPU al 20 % —el límite base— mientras las latencias se disparan, exactamente el falso negativo que describía aquella clase.

2. Dos límites de disco, y casi siempre manda el que no compraste

Este es el punto donde Azure se aparta más de AWS, y produce la conversación de «duplicamos el disco y no cambió nada».

En el SSD premium clásico, el rendimiento se compra comprando tamaño. No hay un parámetro de IOPS que ajustar: cada escalón de tamaño trae su cifra:

P10	128 GiB	500 IOPS	100 MB/s
P20	512 GiB	2.300 IOPS	150 MB/s
P30	1.024 GiB	5.000 IOPS	200 MB/s
P40	2.048 GiB	7.500 IOPS	250 MB/s

Quien viene de gp3 —donde IOPS y tamaño se compran por separado— compra tamaño que no necesita para conseguir IOPS que sí. Las alternativas modernas lo desacoplan: SSD premium v2 y Ultra Disk permiten fijar tamaño, IOPS y rendimiento de forma independiente, que es el comportamiento esperado; a cambio, no admiten caché del host y v2 tiene restricciones de zona.

Pero la cifra del disco es solo la mitad. Cada tamaño de máquina tiene su propio tope de IOPS y de MB/s, y el caudal real es el menor de los dos:

Standard_D2s_v5	3.750	85
Standard_D4s_v5	6.400	145
Standard_D8s_v5	12.800	290
Standard_D16s_v5	25.600	600

P40 (7.500 IOPS) en un D2s_v5 (3.750) → real: 3.750
 cambiar a P50 no mueve la cifra
 cambiar a D8s_v5 la sube a 7.500

La regla operativa es corta: antes de comprar disco, mira la fila de la máquina. Y para diagnosticar, la señal que lo distingue es la profundidad de cola junto al porcentaje de uso del disco: si el disco no está saturado y la cola crece, el techo es de la máquina.

La caché del host añade un tercer factor y una decisión con consecuencias:

None	lecturas y escrituras van al disco → discos de registro de transacciones y cargas de escritura intensa
ReadOnly	lecturas desde la memoria del anfitrión, escrituras al disco → discos de datos con lectura predominante; suele ser la mejor opción
ReadWrite	escrituras confirmadas en la caché del anfitrión → solo el disco del sistema operativo

ReadWrite en un disco de datos o de registro es un error con consecuencias reales: si el anfitrión se reinicia de forma no ordenada, las escrituras confirmadas a la aplicación y aún no bajadas al disco se pierden, y un motor transaccional se encuentra con un registro incoherente. El motivo por el que ocurre es prosaico: ReadWrite es el valor por defecto del disco del sistema, y quien copia la plantilla lo arrastra al resto.

Y un detalle contable: la caché cambia el límite aplicable. Un D8sv5 tiene un tope con caché mayor que sin ella, pero ese tope solo aplica a lo que la caché sirve. Contar con el número grande para dimensionar una carga de escritura es contar con capacidad que no se va a usar.

3. Tres formas de sobrevivir y no todas se combinan

Azure separa en tres construcciones lo que AWS resuelve casi todo con zonas:

	Qué fallo sobrevive	Ámbito	SLA orientativo
Máquina única con disco premium	Ninguno estructural	Un anfitrión	99,9 %
Conjunto de disponibilidad	Fallo de bastidor y mantenimiento planificado	Un centro de datos	99,95 %
Zonas de disponibilidad	Caída de un centro de datos completo	Una región	99,99 %

El conjunto de disponibilidad no tiene equivalente directo en AWS y aporta algo que las zonas no dan: los dominios de actualización. Son la unidad del mantenimiento planificado de la plataforma:

dominios de error (hasta 3) bastidores distintos: alimentación y red
 dominios de actualización (hasta 20) grupos que se reinician por separado
 al actualizar el anfitrión

Con cinco dominios de actualización y diez máquinas, un mantenimiento reinicia como mucho dos a la vez. Sin conjunto de disponibilidad, nada impide que la plataforma reinicie las diez.

La restricción que hay que conocer antes de dibujar: conjunto de disponibilidad y zonas son excluyentes. Una máquina está en uno o en otras, y cambiar de decisión implica recrearla. En una región con zonas, la elección casi siempre es zonas; el conjunto de disponibilidad queda para regiones sin ellas.

El conjunto de escalado es la pieza que integra ambas cosas con elasticidad, y tiene dos modos:

	Uniforme	Flexible
Las instancias son	Objetos del conjunto	Máquinas virtuales reales
Tamaños mezclados	No	Sí
Excedente y bajo demanda mezclados	No	Sí
Gestión individual	Limitada	az vm funciona sobre ellas
Reparto por dominios de error	Implícito	Explícito y configurable

Flexible es el modo por defecto y el que conviene salvo que se necesite escalar a miles de instancias idénticas muy rápido. La razón práctica: permite mezclar prioridades —una base bajo demanda y el pico en excedente— y deja diagnosticar una instancia concreta con las mismas herramientas que cualquier otra máquina.

El autoescalado vive en Azure Monitor, no en el conjunto. Los cuatro parámetros y el efecto de la histéresis ya se trabajaron en la clase 029; lo que cambia aquí son dos cosas concretas:

política de reducción	Default NewestVM OldestVM decide a quién apagar; con Default, el reparto entre zonas se mantiene equilibrado
perfiles	por defecto + perfiles con horario para carga predecible, un perfil horario evita escalar por reacción

Y la trampa compartida con AWS, que conviene repetir porque cuesta una noche: la memoria no es una métrica del anfitrión. El porcentaje de CPU se publica sin instalar nada; la memoria y el espacio en disco exigen el agente de Azure Monitor. Una regla de escalado por memoria configurada sin agente no falla: no dispara nunca.

4. Cuatro balanceadores y la conmutación que depende del TTL

El reparto de Azure no coincide con el de AWS, y traducir «esto es el ALB» produce elecciones equivocadas:

Servicio	Capa	Ámbito	Conmuta en	Equivalente aproximado
Load Balancer	4	Regional	Segundos	Network Load Balancer
Application Gateway	7	Regional	Segundos	Application Load Balancer + WAF
Front Door	7	Global, anycast	Segundos	CloudFront + enrutamiento global
Traffic Manager	DNS	Global	Lo que dure el TTL y la caché del cliente	Políticas de enrutamiento de Route 53

La fila que hay que leer dos veces es la última. Traffic Manager no ve el tráfico: responde consultas de DNS. Cuando una región cae, la conmutación no ocurre hasta que cada cliente vuelve a resolver el nombre, y los clientes no respetan el TTL de forma uniforme:

TTL de 60 s + caché del resolutor + caché de la JVM o del navegador
→ conmutación observada de varios minutos, con una cola larga de clientes que siguen yendo a la región caída

Para un requisito de conmutación en segundos, la respuesta es Front Door: el cliente conecta siempre a la misma dirección anycast y el borde decide a qué origen enviar. Traffic Manager sigue siendo útil para lo que sí resuelve bien: dirigir por proximidad geográfica, repartir entre destinos que no son HTTP y encaminar hacia recursos fuera de Azure.

Sobre el Load Balancer estándar hay cuatro hechos que producen incidentes:

Es denegar por defecto. El SKU básico se retiró el 30 de septiembre de 2025, y el estándar no acepta tráfico entrante si un NSG no lo permite explícitamente — el básico sí lo aceptaba. Una migración literal deja el servicio inalcanzable con toda la configuración aparentemente correcta.

No da salida por tener entrada. Una regla de balanceo entrante no proporciona conectividad saliente. Unido a la retirada del acceso saliente predeterminado de la clase 039, un despliegue nuevo se queda literalmente sin salida hasta que se declara un NAT Gateway o una regla de salida.

Las sondas llegan desde 168.63.129.16. Es una dirección virtual de la plataforma, la misma que sirve DHCP, DNS y el canal del agente de la máquina. Bloquearla no solo tumba las sondas: deja al agente sin comunicación, así que las extensiones dejan de responder y las operaciones desde el portal se quedan colgadas. Es un fallo con dos síntomas que parecen no tener relación.

La sonda debe reflejar disponibilidad real. Vale aquí íntegra la lección de la clase 029: un / que devuelve 200 mientras la base de datos no responde mantiene en rotación a una instancia que no puede servir. La forma correcta es un punto de comprobación que verifique las dependencias críticas, con un umbral de fallos consecutivos que tolere un parpadeo.

Un detalle propio de Azure que aparece en migraciones de bases de datos con clúster: la IP flotante —retorno directo desde el servidor— hace que el destino vea la dirección del frontal en lugar de la suya. Es imprescindible para grupos de disponibilidad de SQL Server y rompe cualquier servicio que no espere ese comportamiento. Y la regla de puertos de alta disponibilidad, que balancea todos los puertos y protocolos a la vez, existe para poner dispositivos virtuales de red en alta disponibilidad detrás de un balanceador interno: es la pieza que faltaba en el diseño de concentrador y radio de la clase 039.

5. Excedente: 30 segundos cambian el diseño

Las máquinas de excedente cuestan entre un 60 % y un 90 % menos y se retiran cuando Azure necesita la capacidad. La clase 028 fijó el criterio de los modelos de compra; lo que cambia aquí es un número:

AWS	aviso de interrupción de excedente	~120 s
Azure	aviso de desalojo	~30 s

Treinta segundos no dan para drenar conexiones, terminar una petición larga ni volcar un estado grande. Dan para abortar de forma ordenada: dejar de aceptar trabajo, confirmar o descartar la unidad en curso y salir. Cualquier diseño que dependa de un drenado educado sobre excedente en Azure está apostando a que el aviso llegue antes de lo prometido.

El aviso se consulta contra el servicio de metadatos, no llega solo:

```
$ curl -s -H Metadata:true \
  "http://169.254.169.254/metadata/scheduledevents?api-version=2020-07-01" | jq -r \
  '.Events[] | "\(.EventType) \(.NotBefore)'"
Preempt Mon, 03 Aug 2026 11:42:07 GMT
```

Y la política de desalojo decide qué queda después:

Deallocate	la máquina se detiene y se conserva; sigue pagando el disco y puede volver a arrancar cuando haya capacidad
Delete	se elimina; no queda nada que pagar ni que recuperar

Para un conjunto de escalado, Delete es casi siempre lo correcto: máquinas desasignadas que nadie recuerda son una partida silenciosa en la factura y una lista de recursos huérfanos. Deallocate tiene sentido en una estación de trabajo con estado en su disco.

La forma sensata de usar excedente es mezclarlo, y para eso hace falta orquestación flexible:

```
$ az vmss create -g rg-app -n vmss-catalogo --orchestration-mode Flexible \
  --instance-count 4 --zones 1 2 3 \
  --priority Spot --eviction-policy Delete --max-price -1

base bajo demanda  capacidad mínima que sostiene el SLO aunque se vaya todo el excedente
pico en excedente  el resto
```

El dimensionado de la base no es una preferencia: es la carga que el servicio debe seguir atendiendo si el excedente desaparece entero en el mismo minuto, que es un escenario real cuando la región se queda sin capacidad. Y --max-price -1 significa «acepto pagar hasta el precio bajo demanda», que evita un desalojo por precio además del desalojo por capacidad: fijar un máximo bajo añade una segunda causa de interrupción que casi nunca compensa.

Ejemplo trabajado

CloudShop traslada la capa de cómputo a Azure con la arquitectura de las clases 028 y 029. La traducción es correcta en el papel y produce cuatro problemas medibles.

Punto de partida:

base de datos Standard_D2s_v5 + disco P40 (2 TiB, 7.500 IOPS), caché ReadWrite
catálogo 4 × Standard_D4s_v5 en conjunto de escalado uniforme
reparto Load Balancer básico + Traffic Manager entre dos regiones

Problema 1 — se duplicó el disco y el rendimiento no se movió.

La base de datos se queda en 3.750 IOPS bajo carga, con la cola de disco creciendo.

```
$ az monitor metrics list --resource $VM_ID --metric "Data Disk IOPS Consumed Percentage" \
  --aggregation Maximum --interval PT1M --query "value[0].timeseries[0].data[-3:].maximum"
[49.8, 50.1, 49.9]
```

El disco está al 50 %. No es el disco. La fila de la máquina lo explica:

P40	7.500 IOPS	
D2s_v5	3.750 IOPS sin caché	← el techo real

Se corrige por el lado correcto y se aprovecha para dejar de pagar tamaño que no se usa:

máquina	D2s_v5 (3.750 IOPS)	D8s_v5 (12.800 IOPS)
disco	P40 · 2 TiB · ~259	P30 · 1 TiB · ~135
IOPS reales	3.750	5.000
costo de disco	259 USD/mes	135 USD/mes

El volumen ocupado eran 640 GiB: la P40 se había comprado por sus IOPS, que la máquina nunca dejó alcanzar.

Problema 2 — un reinicio del anfitrión deja el registro de transacciones incoherente.

Durante un mantenimiento no planificado, el motor no arranca y reporta escrituras confirmadas que no están en disco.

```
$ az vm show -g rg-datos -n vm-db-01 \
  --query "storageProfile.dataDisks[0].{lun:lun,cache:caching}" -o tsv
0  ReadWrite
1  ReadWrite
```

Ambos discos de datos con ReadWrite, heredado de la plantilla del disco del sistema. Se corrige por función:

```
$ az vm update -g rg-datos -n vm-db-01 --disk-caching 0=ReadOnly 1=None

LUN 0  datos      ReadOnly  lectura predominante, la caché ayuda
LUN 1  registro   None      ninguna escritura confirmada fuera del disco
```

Y se añade la consecuencia operativa que faltaba: la restauración se prueba, no se supone. La prueba de recuperación pasa a ejecutarse mensualmente con evidencia, siguiendo el criterio de la clase 030 — replicación no es copia de seguridad.

Problema 3 — la migración del balanceador deja el servicio inalcanzable.

Al sustituir el balanceador básico por el estándar, el frontal deja de responder. La configuración es idéntica.

```
$ az network watcher test-ip-flow --vm cat-01 --direction Inbound --protocol TCP \
  --local 10.20.4.11:8080 --remote 168.63.129.16:62000
Access: Deny RuleName: defaultSecurityRules/DenyAllInBound
```

Dos cosas a la vez: el estándar es denegar por defecto y no había NSG que permitiera nada, porque el básico nunca lo había exigido. Además, no había salida.

```
$ az network nsg rule create -g rg-red --nsg-name nsg-app -n allow-lb-probe \
  --priority 100 --direction Inbound --access Allow --protocol '*' \
  --source-address-prefixes AzureLoadBalancer --destination-port-ranges '*'
$ az network nsg rule create -g rg-red --nsg-name nsg-app -n allow-appgw-8080 \
  --priority 110 --direction Inbound --access Allow --protocol Tcp \
  --source-address-prefixes 10.20.4.128/26 --destination-port-ranges 8080
$ az network vnet subnet update -g rg-red --vnet-name vnet-spoke-app -n snet-app \
  --nat-gateway natgw-spoke-app
```

Y se corrige la sonda, que apuntaba a /:

antes	GET /	200 mientras la base de datos no respondía
después	GET /readyz	comprueba base de datos y caché; 3 fallos consecutivos para retirar de rotación

Problema 4 — una conmutación de región que tardó nueve minutos.

Un simulacro corta la región primaria. Traffic Manager marca el destino como degradado en 40 s; el tráfico tarda mucho más en moverse.


```

t+0:00 se corta la región primaria
t+0:40 Traffic Manager marca el destino como degradado
t+1:10 el DNS ya devuelve la región secundaria
t+9:20 el último cliente deja de intentar contra la región caída

```

Los nueve minutos no son del servicio: son de la caché de DNS de los clientes, que no respetan el TTL de 60 s. Se antepone Front Door para el tráfico HTTP y se conserva Traffic Manager solo para el enrutamiento geográfico de un servicio que no es HTTP:

```

simulacro repetido con Front Door
t+0:00 se corta la región primaria
t+0:35 el borde deja de enviar al origen caído
t+0:41 error observado por el cliente: ninguno tras el reintento

```

Y una revisión de costo que no estaba en el plan. El conjunto de escalado uniforme no permitía mezclar prioridades. Al pasarlo a flexible:

orquestración	uniforme	flexible
capacidad base	4 × D4s_v5 bajo demanda	2 × D4s_v5 bajo demanda
pico	escala en bajo demanda	hasta 6 × D4s_v5 excedente
costo mensual de cómputo	~702 USD	~412 USD
disco de base de datos	259 USD	135 USD
	■■■■■■■■■■	■■■■■■■■■■
	961 USD	547 USD (-43 %)

La base de dos instancias bajo demanda no es un número redondeado a ojo: es la capacidad que sostiene el percentil 95 de tráfico observado si todo el excedente desaparece en el mismo minuto. Se comprobó apagando las seis instancias de excedente a la vez y midiendo la latencia resultante:

```

con 2 bajo demanda y 0 excedente, a carga de p95
p99 de latencia 340 ms (SLO: 500 ms) ✓

```

La lección que esta clase traslada al resto de la parte: en Azure el cuello de botella rara vez está donde se compró la capacidad. El disco, la máquina y la caché tienen topes distintos, y el balanceador y el DNS tienen tiempos de reacción distintos. La cifra que importa siempre es el mínimo de la cadena, y es la que hay que medir antes de firmar un SLO.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/040-virtual-machines-scale-sets-y-load-balancer/lab.py
```

El laboratorio selecciona el motor de práctica compute y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-elastico-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una selección de capacidad justificada y observable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye servicio-elastico-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.

- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se aumenta el tamaño del disco y el rendimiento no cambia	El tope efectivo es el de la máquina virtual, no el del disco	Consulta las IOPS y MB/s sin caché del tamaño elegido; el caudal real es el mínimo de los dos y se corrige cambiando de tamaño.
La automatización falla porque /mnt o D: no existe	El tamaño elegido no lleva la letra d, así que no tiene disco temporal local	Elige un tamaño con d o retira la dependencia del disco temporal; nunca guardes ahí datos que deban sobrevivir.
Tras un reinicio no ordenado del anfitrión, el motor transaccional reporta escrituras perdidas	El disco de registro tenía caché ReadWrite, heredada de la plantilla del disco del sistema	None en el disco de registro, ReadOnly en los de datos con lectura predominante; ReadWrite solo en el disco del sistema.
Al migrar del balanceador básico al estándar, el servicio queda inalcanzable	El estándar deniega el tráfico entrante salvo que un NSG lo permita, y no proporciona salida por tener reglas de entrada	Añade reglas de NSG para la etiqueta AzureLoadBalancer y para el origen real, y declara la salida con NAT Gateway.
La regla de autoescalado por memoria nunca dispara	La memoria no es una métrica del anfitrión: requiere el agente de Azure Monitor	Instala el agente y verifica que la métrica llega antes de confiar la elasticidad a esa regla.
La conmutación entre regiones tarda minutos pese a que la sonda detecta el fallo en segundos	Traffic Manager conmuta por DNS y los clientes cachean más allá del TTL	Usa Front Door para el tráfico HTTP; reserva Traffic Manager para enrutamiento geográfico y destinos que no son HTTP.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué capacidades pierdes con un tamaño sin s y sin d, y cuándo lo descubrirías en cada caso?
2. Un disco P40 rinde 3.750 IOPS. ¿Qué medirías para decidir si el problema es el disco o la máquina?
3. ¿Qué aporta un conjunto de disponibilidad que las zonas no dan, y por qué no se pueden combinar?
4. ¿Por qué la orquestación flexible permite una estrategia de costo que la uniforme no?
5. El aviso de desalojo de excedente es de 30 segundos. ¿Qué diseños quedan descartados y cómo se dimensiona la capacidad base?

Referencias

- Microsoft (2025). Sizes for virtual machines in Azure — nomenclatura, letras aditivas y límites por tamaño.
<<https://learn.microsoft.com/en-us/azure/virtual-machines/sizes/overview>>
- Microsoft (2025). Azure managed disk types — escalones de rendimiento, SSD premium v2 y Ultra Disk.
<<https://learn.microsoft.com/en-us/azure/virtual-machines/disks-types>>
- Microsoft (2025). Virtual Machine Scale Sets orchestration modes — flexible frente a uniforme.
<<https://learn.microsoft.com/en-us/azure/virtual-machine-scale-sets/virtual-machine-scale-sets-orchestration-modes>>
- Microsoft (2025). Load-balancing options — Load Balancer, Application Gateway, Front Door y Traffic Manager.
<<https://learn.microsoft.com/en-us/azure/architecture/guide/technology-choices/load-balancing-overview>>

- Microsoft (2025). Azure Spot Virtual Machines — aviso de desalojo, políticas y precio máximo. <<https://learn.microsoft.com/en-us/azure/virtual-machines/spot-vms>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 040 Virtual Machines, Scale Sets y Load Balancer

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega servicio-elastico-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

041 — Blob Storage, Files, redundancia y lifecycle

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

Propósito

Operar el almacenamiento de Azure sabiendo que la unidad de configuración no es el contenedor sino la cuenta, y que casi todo lo que protege los datos son interruptores separados que hay que activar uno a uno. La clase 030 dejó el criterio —versionado, clases, público bloqueado, replicación no es copia—; aquí cambia dónde vive cada control, y aparecen tres decisiones que en AWS no existen: la región emparejada no se elige, la conmutación la inicias tú, y borrar un contenedor no es lo mismo que borrar sus blobs.

Resultados de aprendizaje

Al finalizar podrás:

1. Dimensionar cuentas de almacenamiento sabiendo qué límites son de la cuenta y qué configuraciones arrastra todo lo que hay dentro.
2. Elegir redundancia distinguiendo lo que protege de lo que cuesta, y sabiendo qué ocurre con la cuenta después de una conmutación.
3. Calcular el punto de equilibrio de un nivel de acceso frío antes de mover datos a él.
4. Configurar los interruptores de protección —versionado, eliminación temporal de blob y de contenedor, inmutabilidad— y demostrar cada uno con una prueba de recuperación.
5. Sustituir claves de cuenta y firmas irrevocables por identidad y firmas de delegación de usuario.

Conceptos centrales

Concepto	Comprensión verificable
cuenta de almacenamiento	Unidad de configuración, facturación y límites. Contiene blobs, archivos, colas y tablas a la vez, y su nombre es una etiqueta DNS global: 3 a 24 caracteres, minúsculas y dígitos, sin guiones.
región emparejada	Destino de la replicación geográfica, asignado por Microsoft y no elegible. Si el dato no puede salir de una jurisdicción, esa asignación puede descartar la redundancia geográfica.
conmutación de cuenta	Cambio de la región principal a la secundaria. La inicia el cliente, pierde lo no replicado y deja la cuenta como LRS, lo que obliga a reconfigurar la redundancia después.
eliminación temporal	Dos interruptores independientes: uno para blobs y otro para contenedores. Activar el primero no protege de borrar el contenedor entero.
firma de delegación de usuario	SAS firmada con una clave derivada de Entra ID en vez de con la clave de la cuenta. Caduca en 7 días como máximo y se puede revocar sin romper el resto.
regla de ciclo de vida	Directiva JSON de la cuenta que mueve o borra por antigüedad. Si solo apunta a baseBlob, las versiones e instantáneas se quedan y la factura no baja.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart TB

```
SA["cuenta de almacenamiento<br/>límites, redundancia y red se fijan AQUÍ"] --> B["blobs"]
SA --> F["archivos"]
SA --> Q["colas"]
SA --> T["tablas"]
B --> P{"interruptores de protección<br/>independientes entre sí"}
P --> V["versionado"]
P --> SB["eliminación temporal de blob"]
P --> SC["eliminación temporal de contenedor"]
P --> IM["inmutabilidad · WORM"]
B --> A{"quién accede"}
A -->|"peor"| K["clave de cuenta<br/>acceso total, sin identidad"]
A -->|"irrevocable"| S1["SAS firmada con la clave"]
A -->|"correcto"| S2["SAS de delegación de usuario"]
A -->|"correcto"| RB["Entra ID + roles de datos"]
```

Desarrollo

1. La cuenta es la unidad, y arrastra todo lo que hay dentro

En AWS el bucket es la unidad: sus políticas, su cifrado y su bloqueo de acceso público valen para él y para nadie más. En Azure la unidad es la cuenta de almacenamiento, y dentro de ella conviven cuatro servicios:

```
cuenta stcloudshopprod
■ ■ blobs      objetos
■ ■ archivos   recursos compartidos SMB y NFS
■ ■ colas      mensajería simple
■ ■ tablas     clave-valor
```

Lo que se decide en la cuenta y no se puede matizar por contenedor:

```

redundancia (LRS, ZRS, GRS, GZRS)
reglas de red y punto de conexión privado
versión mínima de TLS
acceso con clave compartida habilitado o no
acceso público anónimo permitido o no
nivel de acceso por defecto
cifrado y clave gestionada por el cliente

```

Esa lista es la razón por la que una cuenta para todo es un error de diseño, aunque parezca ordenado. Datos con requisitos distintos de residencia, de retención o de exposición terminan compartiendo la misma configuración, y el único modo de separarlos después es copiar terabytes.

Además la cuenta tiene límites propios que un bucket no tiene:

capacidad	5 PiB (ampliable por solicitud)
frecuencia de solicitudes	~20.000 por segundo
salida en la región primaria	límite por cuenta, no por contenedor

Un servicio ruidoso puede limitar a los demás inquilinos de la misma cuenta. El síntoma es un 503 ServerBusy intermitente en un servicio que no cambió nada, causado por otro que sí. La respuesta a x-ms-request-id y el contador Throttling Error lo identifican en un minuto; el arreglo es repartir en varias cuentas, no reintentar más fuerte.

Y el nombre importa más de lo que parece: es una etiqueta DNS global, de 3 a 24 caracteres, minúsculas y dígitos, sin guiones ni puntos. Una convención de nombres que incluya guiones —muy común al llegar de AWS— no se puede aplicar aquí, así que conviene fijarla antes de crear la primera:

```

st  cloudshop  prod  weu  01
■  ■          ■    ■    ■■■ secuencia
■  ■          ■    ■■■■■■■■ región
■  ■          ■■■■■■■■■■■■■■ entorno
■  ■■■■■■■■■■■■■■■■■■■■■■■■ sistema
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ tipo de recurso

```

2. Redundancia: la región no se elige y la conmutación la inicias tú

Cinco opciones, dos ejes: cuántas copias y dónde.

	Copias	Sobrevive a	Lectura en la secundaria
LRS	3 en un centro de datos	Fallo de disco o bastidor	—
ZRS	3 en tres zonas	Caída de un centro de datos	—
GRS	3 locales + 3 remotas	Caída de la región	No
GZRS	3 zonas + 3 remotas	Caída de la región, con zonas en la principal	No
RA-GRS / RA-GZRS	Igual	Igual	Sí, con un nombre -secondary

Tres hechos cambian cómo se diseña con esto, y ninguno tiene equivalente directo en la replicación entre regiones de AWS:

La región emparejada la asigna Microsoft. No se elige el destino. Para una carga sujeta a residencia de datos, eso puede descalificar la redundancia geográfica entera: si el par asignado está fuera de la jurisdicción aprobada, la opción correcta es ZRS más una replicación de objetos explícita hacia una cuenta en la región que sí lo está. Es más trabajo y es la única que se puede documentar ante un auditor.

La replicación es asíncrona y su retraso no es un compromiso de servicio. El indicador es Last Sync Time, y es la única fuente honesta del RPO real:

```
$ az storage account show -n stcloudshopprod -g rg-datos \
  --expand geoReplicationStats --query "geoReplicationStats.lastSyncTime" -o tsv
2026-08-01T09:47:12Z
```

Todo lo escrito después de esa marca no está en la secundaria. Un RPO declarado de 15 minutos que no se contrasta con esta métrica es una aspiración, no un dato.

La conmutación no es automática y deja la cuenta en LRS. Nadie la ejecuta por ti: es una operación que inicia el cliente, y después de completarse la cuenta queda como LRS en la que era región secundaria. Es decir:

```
antes del incidente  GRS  principal weu, secundaria nue
durante la conmutación  se pierde lo posterior a lastSyncTime
después              LRS  una sola región, sin redundancia geográfica
acción pendiente      reconfigurar a GRS y esperar la replicación inicial
```

Ese último renglón es el que sorprende: justo después de un incidente regional, la plataforma está menos protegida que antes, y volver a estarlo tarda lo que tarde copiar todo el volumen. Un plan de recuperación que no incluya ese paso está incompleto.

Y el cambio entre modelos tampoco es un interruptor: pasar de LRS a ZRS exige una conversión —solicitada o mediante copia manual—, con ventana y sin garantía de instantaneidad. Conviene elegir bien al crear la cuenta, porque corregirlo después cuesta tiempo y tráfico.

3. Niveles de acceso: calcula el punto de equilibrio antes de mover nada

Cuatro niveles, con mínimos de permanencia que son la mitad de la decisión:

	Almacenamiento aprox.	Permanencia mínima	Recuperación
Caliente	0,0184 USD/GiB/mes	—	Inmediata
Frío (cool)	0,0100	30 días	Inmediata, con cargo por GiB
Muy frío (cold)	0,0036	90 días	Inmediata, con cargo mayor
Archivo	0,0010	180 días	Hay que rehidratar: hasta 15 h

La clase 030 ya estableció que el ahorro por nivel a veces no lo es. Aquí está la aritmética con las cifras de Azure, para 1 TiB:

```
ahorro mensual al pasar de caliente a frío
(0,0184 - 0,0100) × 1.024 GiB = 8,60 USD/mes

coste de leer ese TiB una vez al mes
1.024 GiB × 0,01 USD/GiB de recuperación = 10,24 USD
```

Leer el conjunto entero una sola vez al mes ya destruye el ahorro. El punto de equilibrio está en releer menos del 84 % del volumen cada mes; por encima, el nivel frío sale más caro que el caliente además de más lento.

Y encima está la penalización por eliminación temprana, que se cobra prorrateada: borrar a los 10 días un blob en nivel frío factura los 20 días restantes de los 30 mínimos. En un flujo con mucha rotación —archivos temporales, informes diarios que se sustituyen— mover al nivel frío puede multiplicar la factura en lugar de reducirla.

El nivel archivo es una decisión distinta, no un escalón más: el blob no se puede leer. Hay que rehidratarlo, y eso tiene tiempo y precio:

prioridad estándar	hasta 15 h
prioridad alta	menos de 1 h para blobs de menos de 10 GiB, con sobrecoste

Quince horas es incompatible con casi cualquier RTO de servicio y perfectamente compatible con una obligación legal de conservar siete años. Esa es la frontera: archivo es para lo que hay que guardar, no para lo que hay que poder leer.

Las reglas de ciclo de vida automatizan el movimiento, y tienen dos comportamientos que hay que anticipar:

```
{
  "rules": [{
    "name": "facturas",
    "type": "Lifecycle",
    "definition": {
      "filters": {"blobTypes": ["blockBlob"], "prefixMatch": ["facturas/"]},
      "actions": {
        "baseBlob": {
          "tierToCool": {"daysAfterModificationGreaterThan": 30},
          "tierToArchive": {"daysAfterModificationGreaterThan": 365}
        },
        "version": {"delete": {"daysAfterCreationGreaterThan": 90}},
        "snapshot": {"delete": {"daysAfterCreationGreaterThan": 90}}
      }
    }
  ]
}
```

Primero: las secciones version y snapshot son las que casi nadie escribe, y son la causa de que la factura no baje después de una limpieza. Con versionado activo, borrar un blob no libera nada: crea una versión. Es el mismo mecanismo que el marcador de borrado de la clase 030, con otro nombre y la misma consecuencia contable.

Segundo: la directiva se evalúa una vez al día y la primera ejecución puede tardar hasta 48 horas. No es un fallo de configuración, es su cadencia; comprobarla a los diez minutos y volver a escribirla es la forma habitual de perder una tarde.

Y daysAfterLastAccessTimeGreaterThan —mover por último acceso en vez de por última modificación— exige activar el seguimiento de acceso, que tiene su propio costo por transacción. En un contenedor con lecturas muy frecuentes puede costar más que lo que ahorra el cambio de nivel.

4. Cuatro interruptores para proteger lo mismo, y ninguno cubre al otro

Aquí está la trampa más cara de esta clase. Los mecanismos de protección son independientes, y activar el evidente deja el hueco:

versionado	conserva una versión por cada sobrescritura o borrado
eliminación temporal de blob	recupera blobs borrados durante N días
eliminación temporal de contenedor	recupera CONTENEDORES borrados durante N días
restauración a un punto anterior	devuelve un rango de blobs a un instante
inmutabilidad	impide modificar o borrar durante un plazo

La eliminación temporal de blob no protege de borrar el contenedor. Es literal: con eliminación temporal de blob a 7 días y eliminación temporal de contenedor desactivada, un az storage container delete se lleva todo el contenido de forma irreversible. El interruptor que parecía cubrirlo cubría otra cosa.

La configuración completa, y su dependencia entre piezas:

```
$ az storage account blob-service-properties update -n stcloudshopprod -g rg-datos \
  --enable-versioning true \
  --enable-delete-retention true --delete-retention-days 30 \
  --enable-container-delete-retention true --container-delete-retention-days 30 \
  --enable-change-feed true \
  --enable-restore-policy true --restore-days 29
```

La restauración a un punto anterior exige versionado, fuente de cambios y eliminación temporal de blob, y su ventana debe ser menor que la de retención. Activarla suelta da un error que no explica cuál de las tres falta.

La inmutabilidad es otra cosa y responde a otro requisito. Se aplica a contenedor o a versión, en dos formas:

retención por tiempo	ningún borrado ni modificación durante N días
retención legal	indefinida hasta que se retire la etiqueta

Y el matiz que la hace útil ante un auditor: mientras la directiva está bloqueada, el plazo se puede alargar y no se puede acortar ni eliminar — tampoco por el propietario de la suscripción, tampoco por un administrador global. Es la diferencia entre una política y un control: la política la cambia quien tiene permiso, el control no lo cambia nadie. La contrapartida hay que aceptarla de antemano: la cuenta no se puede borrar mientras haya datos bajo retención, ni siquiera si se creó por error con un plazo de siete años.

Una última pieza que la clase 030 dejó dicha y aquí tiene dos interruptores en vez de uno: el acceso público anónimo se controla en la cuenta y en el contenedor, y basta con que el de la cuenta esté abierto y alguien marque un contenedor como público para exponerlo:

```
$ az storage account update -n stcloudshopprod -g rg-datos \
  --allow-blob-public-access false
```

Con eso, ningún contenedor puede ser público aunque su configuración diga que lo es. Es el equivalente al bloqueo en el nivel de la cuenta, y es el que hay que auditar: comprobar contenedor por contenedor es trabajo que se deshace solo.

5. Claves, firmas y la que no se puede revocar

Cuatro formas de acceder, ordenadas de peor a mejor:

Clave de cuenta. Dos claves, rotables. Conceden acceso total a los cuatro servicios de la cuenta y no llevan identidad: el registro dice qué se hizo, no quién. Una clave filtrada es la cuenta entera. La instrucción es desactivarlas:

```
$ az storage account update -n stcloudshopprod -g rg-datos --allow-shared-key-access false
```

SAS firmada con la clave de cuenta. Es cómoda y tiene un defecto que define la decisión: no se puede revocar. Una firma emitida con vencimiento en 2029 sigue siendo válida hasta 2029, y la única forma de invalidarla es rotar la clave con la que se firmó — lo que rompe a la vez a todos los demás que la usan. El repositorio se limpia, la firma sigue funcionando.

Un paliativo parcial es la directiva de acceso almacenada: la firma referencia una directiva del contenedor y borrar la directiva invalida las firmas asociadas. Requiere haberlo previsto al emitirlas.

SAS de delegación de usuario. Se firma con una clave derivada de Entra ID, no con la clave de cuenta:

```
$ az storage blob generate-sas --account-name stcloudshopprod \
  --container-name facturas --name 2026-07.pdf \
  --permissions r --expiry 2026-08-01T18:00Z --as-user --auth-mode login -o tsv
```

Caduca en 7 días como máximo, hereda los permisos del principal que la emite —no puede conceder más de lo que ese principal tiene— y se revoca retirando el rol o revocando las claves de delegación, sin tocar nada más. Es la opción correcta para dar acceso temporal a un objeto concreto.

Entra ID con roles de datos. Es la vía por defecto para servicios. Y aquí vuelve, con consecuencias, la distinción de la clase 038:

Storage Account Contributor	gestiona la cuenta y LEE SUS CLAVES → sin dataActions, pero con la clave se accede a todo
Storage Blob Data Reader	lee los datos, no gestiona la cuenta
Storage Blob Data Contributor	lee y escribe los datos

La primera fila es una escalada disfrazada de permiso administrativo: quien puede leer las claves puede hacer todo lo que hacen las claves, sin necesitar ningún rol de datos y sin dejar rastro de identidad. Con allowSharedKeyAccess desactivado, ese camino se cierra: la clave existe y no sirve para autenticarse.

Y para Azure Files conviene anticipar una decisión de identidad, porque no es simétrica con los blobs:

	Estándar	Premium
Soporte	HDD, cobro por transacción	SSD, capacidad aprovisionada

	Estándar	Premium
Rendimiento	Variable	IOPS en función de los GiB aprovisionados
NFS	No	Sí, con punto de conexión privado
Se factura	Lo usado	Lo aprovisionado

La última fila es la que produce facturas inesperadas: un recurso compartido premium de 5 TiB aprovisionado y 300 GiB usados factura 5 TiB. Y para SMB con identidad de dominio hace falta integrar con AD DS o con Entra Domain Services; sin eso, el acceso es con clave de cuenta, que es justo lo que se acaba de desactivar. Conviene resolverlo antes de migrar un recurso compartido de archivos, no después.

Ejemplo trabajado

CloudShop lleva a Azure el almacenamiento diseñado en la clase 030. La configuración inicial parece completa: versionado activo, replicación geográfica y niveles de acceso. Cuatro sucesos demuestran que ninguna de las tres cosas hacía lo que el equipo creía.

Estado inicial:

```
cuenta stcloudshopprod GRS · una sola cuenta para facturas, medios y registros
versionado activo
eliminación temporal de blob 7 días
eliminación temporal de contenedor desactivada
acceso con clave compartida habilitado
```

Suceso 1 — un contenedor borrado por error se lleva 412 GiB.

Un script de limpieza mal parametrizado borra facturas en lugar de facturas-tmp.

```
$ az storage container restore -n facturas --account-name stcloudshopprod
ERROR: Container soft delete is not enabled for this account.
```

La eliminación temporal de blob estaba activa y no cubría esto. No hay recuperación. Se reconstruye desde el sistema de facturación —tres días de trabajo— y se cierran los cuatro interruptores, no uno:

```
$ az storage account blob-service-properties update -n stcloudshopprod -g rg-datos \
  --enable-versioning true \
  --enable-delete-retention true --delete-retention-days 30 \
  --enable-container-delete-retention true --container-delete-retention-days 30 \
  --enable-change-feed true --enable-restore-policy true --restore-days 29
```

Y se prueba la recuperación en lugar de suponerla:

```
$ az storage container delete -n prueba-recuperacion --account-name stcloudshopprod
$ az storage container restore -n prueba-recuperacion --account-name stcloudshopprod
$ az storage blob list -c prueba-recuperacion --account-name stcloudshopprod -o tsv | wc -l
50 ✓
```

Además, las facturas tienen obligación legal de siete años, así que se añade inmutabilidad bloqueada — con la consecuencia aceptada por escrito de que la cuenta ya no se podrá eliminar durante ese plazo.

Suceso 2 — se borran 3 TiB y la factura no baja.

```
datos visibles 1,8 TiB
facturado 4,9 TiB

$ az storage account show -n stcloudshopprod -g rg-datos \
  --query "{v:blobServiceProperties.isVersioningEnabled}"
$ az storage account management-policy show --account-name stcloudshopprod -g rg-datos \
  --query "policy.rules[0].definition.actions" -o json
{"baseBlob": {"tierToCool": {"daysAfterModificationGreaterThan": 30}}}
```

La regla solo tocaba baseBlob. Con versionado activo, cada borrado había creado una versión que nadie retiraba nunca. Es el marcador de borrado de la clase 030 con otro nombre. Se completa la directiva con version y snapshot, y se espera a su cadencia diaria en lugar de reescribirla:

```
facturado 4,9 TiB 1,9 TiB
costo mensual de blobs 92,20 USD 35,80 USD
```

Suceso 3 — una firma en una aplicación móvil, válida hasta 2029.

Una revisión de seguridad encuentra una SAS incrustada en el paquete de la aplicación.

```
se=2029-01-01T00%3A00%3A00Z&sp=racwdl&sr=c&sig=...
```

Permisos de lectura, escritura, borrado y listado sobre el contenedor entero, durante tres años. Firmada con la clave de la cuenta, así que revocarla exige rotar la clave, y la clave la usan seis servicios más.

La secuencia, en este orden y no en otro:

1. migrar los seis servicios a identidad administrada + roles de datos
2. verificar que ninguno usa ya la clave (métrica de autenticación por clave = 0)
3. rotar las dos claves → la firma de 2029 deja de valer
4. desactivar el acceso con clave compartida
5. la aplicación móvil pasa a pedir una SAS de delegación de usuario al backend, de 15 minutos y para un solo blob

```
$ az storage account update -n stcloudshopprod -g rg-datos --allow-shared-key-access false
$ az storage blob list -c facturas --account-name stcloudshopprod \
  --account-key $CLAVE_ANTIGUA -o tsv
KeyBasedAuthenticationNotPermitted ✓
```

Suceso 4 — el simulacro de región revela dos supuestos falsos.

El plan decía «GRS conmuta a la región emparejada». El simulacro mide otra cosa:

```
$ az storage account show -n stcloudshopprod -g rg-datos --expand geoReplicationStats \
  --query "geoReplicationStats.{estado:status,ultima:lastSyncTime}" -o tsv
Live 2026-08-01T09:36:00Z
```

Con la hora del corte a las 09:47, el RPO real de ese momento era de 11 minutos, no de cero. Y la conmutación no ocurre sola: hay que ejecutarla, y al terminar la cuenta queda como LRS.

conmutación	automática	manual, ~1 h
RPO	0 min	11 min
redundancia después de conmutar	GRS	LRS

Además, la región emparejada asignada estaba fuera de la jurisdicción aprobada para los datos de facturación. La corrección separa lo que nunca debió compartir cuenta:

facturas	■	GZRS + replicación de objetos
medios	■■ una cuenta GRS	a una región elegida y aprobada
registros	■	medios: GRS, ciclo de vida a frío a 30 días
		registros: LRS, borrado a 90 días

Resumen del rediseño:

cuentas de almacenamiento	1	3
interruptores de protección activos	2 de 5	5 de 5
recuperación de contenedor probada	no	sí, mensual
acceso con clave compartida	sí	no
firmas irrevocables en circulación	1	0
RPO documentado y medido	no	sí, 11 min
datos facturados	4,9 TiB	1,9 TiB
costo mensual de almacenamiento	92,20 USD	41,60 USD

La lección que esta clase traslada al resto de la parte: en Azure, la protección de datos no es una propiedad que se activa sino una lista de interruptores independientes, y ninguno de ellos cubre el hueco del de al lado. La única forma honesta de saber cuáles están activos es borrar algo a propósito y recuperarlo.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/041-blob-storage-files-redundancia-y-lifecycle/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa storage-gobernado-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye storage-gobernado-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
Se borra un contenedor y no hay forma de recuperarlo pese a tener eliminación temporal	La eliminación temporal de blob y la de contenedor son interruptores distintos	Activa ambos y demuéstalo borrando y restaurando un contenedor de prueba cada mes.
Se liberan terabytes y la factura no cambia	Con versionado activo, borrar crea una versión, y la regla de ciclo de vida solo apuntaba a baseBlob	Añade las secciones version y snapshot a la directiva y espera a su evaluación diaria.
Una SAS filtrada no se puede revocar sin romper otros servicios	Se firmó con la clave de la cuenta, así que solo se invalida rotando esa clave	Migra los servicios a identidad, rota las claves, desactiva el acceso con clave compartida y emite SAS de delegación de usuario.
Tras una conmutación regional la cuenta ya no tiene redundancia geográfica	La conmutación deja la cuenta como LRS en la región secundaria	Incluye en el plan de recuperación el paso de reconfigurar la redundancia y el tiempo de la replicación inicial.
Mover datos al nivel frío aumenta el costo en vez de reducirlo	Los cargos por recuperación y la penalización por eliminación temprana superan el ahorro de almacenamiento	Calcula el punto de equilibrio con el volumen releído al mes antes de aplicar la regla.
Un principal sin roles de datos accede a todos los blobs	Tenía un rol de gestión que permite leer las claves de la cuenta	Desactiva allowSharedKeyAccess: la clave sigue existiendo y deja de servir para autenticarse.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué configuraciones se deciden en la cuenta y no se pueden matizar por contenedor, y qué implica eso para el diseño?

2. ¿Por qué la redundancia geográfica puede quedar descartada por un requisito de residencia de datos?
3. Con 1 TiB, ¿a partir de qué volumen leído al mes deja de compensar el nivel frío?
4. Tienes versionado y eliminación temporal de blob activos. ¿De qué pérdida NO estás protegido?
5. ¿Qué diferencia operativa concreta hay entre una SAS firmada con la clave de cuenta y una de delegación de usuario?

Referencias

- Microsoft (2025). Azure Storage redundancy — LRS a GZRS, región emparejada y lectura en la secundaria. <<https://learn.microsoft.com/en-us/azure/storage/common/storage-redundancy>>
- Microsoft (2025). Disaster recovery and storage account failover — conmutación iniciada por el cliente, lastSyncTime y estado posterior. <<https://learn.microsoft.com/en-us/azure/storage/common/storage-disaster-recovery-guidance>>
- Microsoft (2025). Access tiers for blob data — niveles, permanencia mínima y rehidratación desde archivo. <<https://learn.microsoft.com/en-us/azure/storage/blobs/access-tiers-overview>>
- Microsoft (2025). Data protection overview — versionado, eliminación temporal, restauración a un punto anterior e inmutabilidad. <<https://learn.microsoft.com/en-us/azure/storage/blobs/data-protection-overview>>
- Microsoft (2025). Grant limited access with shared access signatures — tipos de SAS y delegación de usuario. <<https://learn.microsoft.com/en-us/azure/storage/common/storage-sas-overview>>
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 041 Blob Storage, Files, redundancia y lifecycle

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega storage-gobernado-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

042 — Azure SQL, Cosmos DB y Azure Cache for Redis

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar azure sql, cosmos db y azure cache for redis dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar azure sql, cosmos db y azure cache for redis con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-datos-azure con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
azure	Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real.
sql	Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real.
cosmos	Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real.
db	Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real.
azure	Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real.
cache	Define su papel en azure sql, cosmos db y azure cache for redis y cómo observarlo en un sistema real.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Azure SQL, Cosmos DB y Azure Cache for Redis"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar azure sql, cosmos db y azure cache for redis. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/042-azure-sql-cosmos-db-y-azure-cache-for-redis/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-datos-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-datos-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 042 Azure SQL, Cosmos DB y Azure Cache for Redis

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-datos-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

043 — App Service, Functions y Container Apps

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar app service, functions y container apps dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar app service, functions y container apps con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar api-plataforma-azure con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
app	Define su papel en app service, functions y container apps y cómo observarlo en un sistema real.
service	Define su papel en app service, functions y container apps y cómo observarlo en un sistema real.
functions	Define su papel en app service, functions y container apps y cómo observarlo en un sistema real.
container	Define su papel en app service, functions y container apps y cómo observarlo en un sistema real.
apps	Define su papel en app service, functions y container apps y cómo observarlo en un sistema real.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: App Service, Functions y Container Apps"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar app service, functions y container apps. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/043-app-service-functions-y-container-apps/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa api-plataforma-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `api-plataforma-azure` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 043 App Service, Functions y Container Apps

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega api-plataforma-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

044 — Service Bus, Event Grid y Event Hubs

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar service bus, event grid y event hubs dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar service bus, event grid y event hubs con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar flujo-eventos-azure con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
service	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
bus	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
event	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
grid	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
event	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
hubs	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Service Bus, Event Grid y Event Hubs"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar service bus, event grid y event hubs. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/044-service-bus-event-grid-y-event-hubs/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa flujo-eventos-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye flujo-eventos-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 044 Service Bus, Event Grid y Event Hubs

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega flujo-eventos-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

045 — Azure Monitor, Log Analytics y Application Insights

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar azure monitor, log analytics y application insights dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar azure monitor, log analytics y application insights con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar evidencia-operativa-azure con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
azure	Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real.
monitor	Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real.
log	Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real.
analytics	Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real.
application	Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real.
insights	Define su papel en azure monitor, log analytics y application insights y cómo observarlo en un sistema real.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Azure Monitor, Log Analytics y Application Insights"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar azure monitor, log analytics y application insights. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/045-azure-monitor-log-analytics-y-application-insights/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa evidencia-operativa-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye evidencia-operativa-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 045 Azure Monitor, Log Analytics y Application Insights

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega evidencia-operativa-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

046 — Key Vault, Defender for Cloud y Azure Policy

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar key vault, defender for cloud y azure policy dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar key vault, defender for cloud y azure policy con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar controles-azure con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
key	Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real.
vault	Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real.
defender	Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real.
for	Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real.
cloud	Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real.
azure	Define su papel en key vault, defender for cloud y azure policy y cómo observarlo en un sistema real.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Key Vault, Defender for Cloud y Azure Policy"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar key vault, defender for cloud y azure policy. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/046-key-vault-defender-for-cloud-y-azure-policy/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa controles-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye controles-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 046 Key Vault, Defender for Cloud y Azure Policy

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega controles-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

047 — Bicep, plantillas y despliegues por alcance

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar bicep, plantillas y despliegues por alcance dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bicep, plantillas y despliegues por alcance con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar infraestructura-bicep con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
bicep	Define su papel en bicep, plantillas y despliegues por alcance y cómo observarlo en un sistema real.
plantillas	Define su papel en bicep, plantillas y despliegues por alcance y cómo observarlo en un sistema real.
despliegues	Define su papel en bicep, plantillas y despliegues por alcance y cómo observarlo en un sistema real.
alcance	Define su papel en bicep, plantillas y despliegues por alcance y cómo observarlo en un sistema real.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Bicep, plantillas y despliegues por alcance"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bicep, plantillas y despliegues por alcance. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/047-bicep-plantillas-y-despliegues-por-alcance/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa infraestructura-bicep en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye infraestructura-bicep para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 047 Bicep, plantillas y despliegues por alcance

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega infraestructura-bicep con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

048 — Proyecto: aplicación de tres capas en Azure

Parte: 03 — Azure: plataforma esencial Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: aplicación de tres capas en azure dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: aplicación de tres capas en azure con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-azure con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real.
aplicación	Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real.
tres	Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real.
capas	Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real.
azure	Define su papel en proyecto: aplicación de tres capas en azure y cómo observarlo en un sistema real.

Modelo mental

En Azure, identidad y jerarquía organizacional preceden al recurso: tenant, management group, suscripción y resource group determinan el alcance de control.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Proyecto: aplicación de tres capas en Azure"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: aplicación de tres capas en azure. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-03-azure-core-platform/048-proyecto-aplicacion-de-tres-capas-en-azure/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-azure en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.

- Azure for Architects — Ritesh Modi.
- Exam Ref AZ-305 — Microsoft Press.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 048 Proyecto: aplicación de tres capas en Azure

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 04 — Google Cloud: plataforma esencial

← Parte 03 · Índice completo · Parte 05 →

Nivel: intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar y operar una carga en Google Cloud con proyectos, identidades y datos bien delimitados.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
049	Organización, folders, proyectos, billing y cuotas	governance	4
050	IAM, service accounts y Workload Identity Federation	iam	4
051	VPC global, subredes regionales, firewall y Cloud NAT	network	4
052	Compute Engine, managed instance groups y load balancing	compute	4
053	Cloud Storage, clases, lifecycle y replicación	storage	4
054	Cloud SQL, Spanner, Firestore y Memorystore	data	4
055	Cloud Run, Cloud Functions y API Gateway	serverless	4
056	Pub/Sub, Cloud Tasks y Workflows	messaging	4
057	Cloud Logging, Monitoring, Trace y Audit Logs	observability	4
058	Cloud KMS, Secret Manager y Security Command Center	security	4
059	Terraform y despliegues reproducibles en GCP	iac	4
060	Proyecto: aplicación de tres capas en Google Cloud	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.

049 — Organización, folders, proyectos, billing y cuotas

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar organización, folders, proyectos, billing y cuotas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar organización, folders, proyectos, billing y cuotas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar jerarquía-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
organización	Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real.
folders	Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real.
proyectos	Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real.
billing	Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real.
cuotas	Define su papel en organización, folders, proyectos, billing y cuotas y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Organización, folders, proyectos, billing y cuotas"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar organización, folders, proyectos, billing y cuotas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/049-organizacion-folders-proyectos-billing-y-cuotas/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa jerarquia-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `jerarquia-gcp` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 049 Organización, folders, proyectos, billing y cuotas

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega jerarquía-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

050 — IAM, service accounts y Workload Identity Federation

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar iam, service accounts y workload identity federation dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar iam, service accounts y workload identity federation con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar identidad-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
iam	Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real.
service	Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real.
accounts	Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real.
workload	Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real.
identity	Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real.
federation	Define su papel en iam, service accounts y workload identity federation y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: IAM, service accounts y Workload Identity Federation"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar iam, service accounts y workload identity federation. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/050-iam-service-accounts-y-workload-identity-federation/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa identidad-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye identidad-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 050 IAM, service accounts y Workload Identity Federation

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega identidad-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

051 — VPC global, subredes regionales, firewall y Cloud NAT

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar vpc global, subredes regionales, firewall y cloud nat dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar vpc global, subredes regionales, firewall y cloud nat con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar vpc-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
vpc	Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real.
global	Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real.
subredes	Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real.
regionales	Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real.
firewall	Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real.
cloud	Define su papel en vpc global, subredes regionales, firewall y cloud nat y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: VPC global, subredes regionales, firewall y Cloud NAT"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar vpc global, subredes regionales, firewall y cloud nat. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/051-vpc-global-subredes-regionales-firewall-y-cloud-nat/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa vpc-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye vpc-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 051 VPC global, subredes regionales, firewall y Cloud NAT

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega vpc-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

052 — Compute Engine, managed instance groups y load balancing

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: compute · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar compute engine, managed instance groups y load balancing dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar compute engine, managed instance groups y load balancing con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar servicio-elastico-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
compute	Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real.
engine	Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real.
managed	Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real.
instance	Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real.
groups	Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real.
load	Define su papel en compute engine, managed instance groups y load balancing y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Compute Engine, managed instance groups y load balancing"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar compute engine, managed instance groups y load balancing. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/052-compute-engine-managed-instance-groups-y-load-balancing/
```

lab.py

El laboratorio selecciona el motor de práctica compute y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa servicio-elastico-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una selección de capacidad justificada y observable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye servicio-elastico-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 052 Compute Engine, managed instance groups y load balancing

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega servicio-elastico-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

053 — Cloud Storage, clases, lifecycle y replicación

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloud storage, clases, lifecycle y replicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud storage, clases, lifecycle y replicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar bucket-gobernado-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloud	Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real.
storage	Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real.
clases	Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real.
lifecycle	Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real.
replicación	Define su papel en cloud storage, clases, lifecycle y replicación y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Cloud Storage, clases, lifecycle y replicación"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud storage, clases, lifecycle y replicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/053-cloud-storage-clases-lifecycle-y-replicacion/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa bucket-gobernado-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `bucket-gobernado-gcp` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 053 Cloud Storage, clases, lifecycle y replicación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega bucket-gobernado-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

054 — Cloud SQL, Spanner, Firestore y Memorystore

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloud sql, spanner, firestore y memorystore dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud sql, spanner, firestore y memorystore con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-datos-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloud	Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real.
sql	Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real.
spanner	Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real.
firestore	Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real.
memorystore	Define su papel en cloud sql, spanner, firestore y memorystore y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Cloud SQL, Spanner, Firestore y Memorystore"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud sql, spanner, firestore y memystore. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/054-cloud-sql-spanner-firestore-y-memystore/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-datos-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `matriz-datos-gcp` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 054 Cloud SQL, Spanner, Firestore y Memorystore

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-datos-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

055 — Cloud Run, Cloud Functions y API Gateway

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloud run, cloud functions y api gateway dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud run, cloud functions y api gateway con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar api-serverless-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloud	Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real.
run	Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real.
cloud	Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real.
functions	Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real.
api	Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real.
gateway	Define su papel en cloud run, cloud functions y api gateway y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Cloud Run, Cloud Functions y API Gateway"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud run, cloud functions y api gateway. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/055-cloud-run-cloud-functions-y-api-gateway/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa api-serverless-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `api-serverless-gcp` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 055 Cloud Run, Cloud Functions y API Gateway

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega api-serverless-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

056 — Pub/Sub, Cloud Tasks y Workflows

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar pub/sub, cloud tasks y workflows dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pub/sub, cloud tasks y workflows con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar flujo-eventos-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
pub	Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real.
sub	Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real.
cloud	Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real.
tasks	Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real.
workflows	Define su papel en pub/sub, cloud tasks y workflows y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Pub/Sub, Cloud Tasks y Workflows"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pub/sub, cloud tasks y workflows. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/056-pub-sub-cloud-tasks-y-workflows/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa flujo-eventos-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye flujo-eventos-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 056 Pub/Sub, Cloud Tasks y Workflows

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega flujo-eventos-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

057 — Cloud Logging, Monitoring, Trace y Audit Logs

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloud logging, monitoring, trace y audit logs dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud logging, monitoring, trace y audit logs con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar evidencia-operativa-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloud	Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real.
logging	Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real.
monitoring	Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real.
trace	Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real.
audit	Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real.
logs	Define su papel en cloud logging, monitoring, trace y audit logs y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Cloud Logging, Monitoring, Trace y Audit Logs"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud logging, monitoring, trace y audit logs. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/057-cloud-logging-monitoring-trace-y-audit-logs/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa evidencia-operativa-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye evidencia-operativa-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 057 Cloud Logging, Monitoring, Trace y Audit Logs

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega evidencia-operativa-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

058 — Cloud KMS, Secret Manager y Security Command Center

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloud kms, secret manager y security command center dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud kms, secret manager y security command center con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar controles-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloud	Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real.
kms	Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real.
secret	Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real.
manager	Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real.
security	Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real.
command	Define su papel en cloud kms, secret manager y security command center y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Cloud KMS, Secret Manager y Security Command Center"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud kms, secret manager y security command center. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/058-cloud-kms-secret-manager-y-security-command-center/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa controles-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye controles-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 058 Cloud KMS, Secret Manager y Security Command Center

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega controles-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

059 — Terraform y despliegues reproducibles en GCP

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar terraform y despliegues reproducibles en gcp dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar terraform y despliegues reproducibles en gcp con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar infraestructura-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
terraform	Define su papel en terraform y despliegues reproducibles en gcp y cómo observarlo en un sistema real.
despliegues	Define su papel en terraform y despliegues reproducibles en gcp y cómo observarlo en un sistema real.
reproducibles	Define su papel en terraform y despliegues reproducibles en gcp y cómo observarlo en un sistema real.
gcp	Define su papel en terraform y despliegues reproducibles en gcp y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Terraform y despliegues reproducibles en GCP"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar terraform y despliegues reproducibles en gcp. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/059-terraform-y-despliegues-reproducibles-en-gcp/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa infraestructura-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye infraestructura-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 059 Terraform y despliegues reproducibles en GCP

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega infraestructura-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

060 — Proyecto: aplicación de tres capas en Google Cloud

Parte: 04 — Google Cloud: plataforma esencial Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: aplicación de tres capas en google cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: aplicación de tres capas en google cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real.
aplicación	Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real.
tres	Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real.
capas	Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real.
google	Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real.
cloud	Define su papel en proyecto: aplicación de tres capas en google cloud y cómo observarlo en un sistema real.

Modelo mental

Un proyecto de Google Cloud es la unidad práctica de API, cuota, IAM y facturación; la organización aporta la política heredable.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Proyecto: aplicación de tres capas en Google Cloud"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: aplicación de tres capas en google cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-04-gcp-core-platform/060-proyecto-aplicacion-de-tres-capas-en-google-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-gcp en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `plataforma-gcp` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.

- Official Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud Platform — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 060 Proyecto: aplicación de tres capas en Google Cloud

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 05 — Contenedores, Docker y OCI

← Parte 04 · Índice completo · Parte 06 →

Nivel: intermedio · Clases: 12 · Duración sugerida: 6–8 semanas

Empaquetar servicios portables, observables y endurecidos.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
061	Imágenes, capas, registros y estándar OCI	container	4
062	Dockerfile reproducible y builds multi-stage	container	4
063	Namespaces, cgroups y runtime de contenedores	container	4
064	Volúmenes, bind mounts y persistencia	storage	4
065	Redes bridge, DNS interno y publicación de puertos	network	4
066	Docker Compose y aplicaciones multiservicio	orchestration	4
067	Registros, SBOM, firma y procedencia de imágenes	supply-chain	4
068	Límites, health checks y apagado ordenado	reliability	4
069	Rootless, capabilities, seccomp y secretos	security	4
070	Diagnóstico de CPU, memoria, red y filesystem	observability	4
071	Migración de una aplicación legacy a contenedores	migration	4
072	Proyecto: stack OCI endurecido y observable	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.

061 — Imágenes, capas, registros y estándar OCI

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: container · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar imágenes, capas, registros y estándar oci dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar imágenes, capas, registros y estándar oci con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar inspeccion-imagen con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
imágenes	Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real.
capas	Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real.
registros	Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real.
estándar	Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real.
oci	Define su papel en imágenes, capas, registros y estándar oci y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Imágenes, capas, registros y estándar OCI"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar imágenes, capas, registros y estándar oci. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/061-imagenes-capas-registros-y-estandar-oci/lab.py
```

El laboratorio selecciona el motor de práctica container y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa inspeccion-imagen en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una imagen mínima, escaneada y ejecutada sin privilegios. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye inspeccion-imagen para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 061 Imágenes, capas, registros y estándar OCI

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega inspeccion-imagen con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

062 — Dockerfile reproducible y builds multi-stage

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: container · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar dockerfile reproducible y builds multi-stage dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar dockerfile reproducible y builds multi-stage con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar imagen-minima con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
dockerfile	Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real.
reproducible	Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real.
builds	Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real.
multi	Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real.
stage	Define su papel en dockerfile reproducible y builds multi-stage y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Dockerfile reproducible y builds multi-stage"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar dockerfile reproducible y builds multi-stage. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/062-dockerfile-reproducible-y-builds-multi-stage/lab.py
```

El laboratorio selecciona el motor de práctica container y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa imagen-minima en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una imagen mínima, escaneada y ejecutada sin privilegios. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye imagen-minima para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 062 Dockerfile reproducible y builds multi-stage

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega imagen-minima con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

063 — Namespaces, cgroups y runtime de contenedores

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: container · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar namespaces, cgroups y runtime de contenedores dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar namespaces, cgroups y runtime de contenedores con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-aislamiento con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
namespaces	Define su papel en namespaces, cgroups y runtime de contenedores y cómo observarlo en un sistema real.
cgroups	Define su papel en namespaces, cgroups y runtime de contenedores y cómo observarlo en un sistema real.
runtime	Define su papel en namespaces, cgroups y runtime de contenedores y cómo observarlo en un sistema real.
contenedores	Define su papel en namespaces, cgroups y runtime de contenedores y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Namespaces, cgroups y runtime de contenedores"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E["¿Cumple seguridad, SLO y costo?"]
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar namespaces, cgroups y runtime de contenedores. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/063-namespaces-cgroups-y-runtime-de-contenedores/lab.py
```

El laboratorio selecciona el motor de práctica container y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mapa-aislamiento en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una imagen mínima, escaneada y ejecutada sin privilegios. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye mapa-aislamiento para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 063 Namespaces, cgroups y runtime de contenedores

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega mapa-aislamiento con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

064 — Volúmenes, bind mounts y persistencia

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar volúmenes, bind mounts y persistencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar volúmenes, bind mounts y persistencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar persistencia-contenedor con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
volúmenes	Define su papel en volúmenes, bind mounts y persistencia y cómo observarlo en un sistema real.
bind	Define su papel en volúmenes, bind mounts y persistencia y cómo observarlo en un sistema real.
mounts	Define su papel en volúmenes, bind mounts y persistencia y cómo observarlo en un sistema real.
persistencia	Define su papel en volúmenes, bind mounts y persistencia y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Volúmenes, bind mounts y persistencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar volúmenes, bind mounts y persistencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/064-volumenes-bind-mounts-y-persistencia/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa persistencia-contenedor en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye persistencia-contenedor para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 064 Volúmenes, bind mounts y persistencia

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega persistencia-contenedor con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

065 — Redes bridge, DNS interno y publicación de puertos

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar redes bridge, dns interno y publicación de puertos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar redes bridge, dns interno y publicación de puertos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar red-contenedores con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
redes	Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real.
bridge	Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real.
dns	Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real.
interno	Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real.
publicación	Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real.
puertos	Define su papel en redes bridge, dns interno y publicación de puertos y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Redes bridge, DNS interno y publicación de puertos"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar redes bridge, dns interno y publicación de puertos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/065-redes-bridge-dns-interno-y-publicacion-de-puertos/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa red-contenedores en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye red-contenedores para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 065 Redes bridge, DNS interno y publicación de puertos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega red-contenedores con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

066 — Docker Compose y aplicaciones multiservicio

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: orchestration · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar docker compose y aplicaciones multiservicio dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar docker compose y aplicaciones multiservicio con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar stack-compose con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
docker	Define su papel en docker compose y aplicaciones multiservicio y cómo observarlo en un sistema real.
compose	Define su papel en docker compose y aplicaciones multiservicio y cómo observarlo en un sistema real.
aplicaciones	Define su papel en docker compose y aplicaciones multiservicio y cómo observarlo en un sistema real.
multiservicio	Define su papel en docker compose y aplicaciones multiservicio y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Docker Compose y aplicaciones multiservicio"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar docker compose y aplicaciones multiservicio. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/066-docker-compose-y-aplicaciones-multiservicio/lab.py
```

El laboratorio selecciona el motor de práctica orchestration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa stack-compose en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es servicios coordinados con health checks y apagado limpio. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye stack-compose para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 066 Docker Compose y aplicaciones multiservicio

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega stack-compose con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

067 — Registros, SBOM, firma y procedencia de imágenes

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: supply-chain · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar registros, sbom, firma y procedencia de imágenes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar registros, sbom, firma y procedencia de imágenes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cadena-suministro-imagen con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
registros	Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real.
sbom	Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real.
firma	Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real.
procedencia	Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real.
imágenes	Define su papel en registros, sbom, firma y procedencia de imágenes y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Registros, SBOM, firma y procedencia de imágenes"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar registros, sbom, firma y procedencia de imágenes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/067-registros-sbom-firma-y-procedencia-de-imagenes/lab.py
```

El laboratorio selecciona el motor de práctica supply-chain y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cadena-suministro-imagen en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es procedencia, inventario y verificación del artefacto. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cadena-suministro-imagen para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 067 Registros, SBOM, firma y procedencia de imágenes

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cadena-suministro-imagen con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

068 — Límites, health checks y apagado ordenado

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar límites, health checks y apagado ordenado dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar límites, health checks y apagado ordenado con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contenedor-operable con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
límites	Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real.
health	Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real.
checks	Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real.
apagado	Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real.
ordenado	Define su papel en límites, health checks y apagado ordenado y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Límites, health checks y apagado ordenado"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar límites, health checks y apagado ordenado. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/068-limites-health-checks-y-apagado-ordenado/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa contenedor-operable en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye contenedor-operable para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 068 Límites, health checks y apagado ordenado

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega contenedor-operable con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

069 — Rootless, capabilities, seccomp y secretos

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar rootless, capabilities, seccomp y secretos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar rootless, capabilities, seccomp y secretos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contenedor-endurecido con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
rootless	Define su papel en rootless, capabilities, seccomp y secretos y cómo observarlo en un sistema real.
capabilities	Define su papel en rootless, capabilities, seccomp y secretos y cómo observarlo en un sistema real.
seccomp	Define su papel en rootless, capabilities, seccomp y secretos y cómo observarlo en un sistema real.
secretos	Define su papel en rootless, capabilities, seccomp y secretos y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Rootless, capabilities, seccomp y secretos"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar rootless, capabilities, seccomp y secretos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/069-rootless-capabilities-seccomp-y-secretos/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa contenedor-endurecido en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye contenedor-endurecido para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 069 Rootless, capabilities, seccomp y secretos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega contenedor-endurecido con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

070 — Diagnóstico de CPU, memoria, red y filesystem

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar diagnóstico de cpu, memoria, red y filesystem dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar diagnóstico de cpu, memoria, red y filesystem con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar informe-diagnostico con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
diagnóstico	Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real.
cpu	Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real.
memoria	Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real.
red	Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real.
filesystem	Define su papel en diagnóstico de cpu, memoria, red y filesystem y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Diagnóstico de CPU, memoria, red y filesystem"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar diagnóstico de cpu, memoria, red y filesystem. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/070-diagnostico-de-cpu-memoria-red-y-filesystem/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa informe-diagnostico en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye informe-diagnostico para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 070 Diagnóstico de CPU, memoria, red y filesystem

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega informe-diagnostico con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

071 — Migración de una aplicación legacy a contenedores

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 4 Laboratorio: migration · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar migración de una aplicación legacy a contenedores dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar migración de una aplicación legacy a contenedores con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plan-modernización con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
migración	Define su papel en migración de una aplicación legacy a contenedores y cómo observarlo en un sistema real.
aplicación	Define su papel en migración de una aplicación legacy a contenedores y cómo observarlo en un sistema real.
legacy	Define su papel en migración de una aplicación legacy a contenedores y cómo observarlo en un sistema real.
contenedores	Define su papel en migración de una aplicación legacy a contenedores y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Migración de una aplicación legacy a contenedores"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar migración de una aplicación legacy a contenedores. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/071-migracion-de-una-aplicacion-legacy-a-contenedores/lab.py
```

El laboratorio selecciona el motor de práctica migration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plan-modernizacion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un inventario, dependencias, riesgo y oleadas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plan-modernización para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.
- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 071 Migración de una aplicación legacy a contenedores

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plan-modernización con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

072 — Proyecto: stack OCI endurecido y observable

Parte: 05 — Contenedores, Docker y OCI Nivel: intermedio · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: stack oci endurecido y observable dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: stack oci endurecido y observable con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-contenedores con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real.
stack	Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real.
oci	Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real.
endurecido	Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real.
observable	Define su papel en proyecto: stack oci endurecido y observable y cómo observarlo en un sistema real.

Modelo mental

Una imagen es un artefacto inmutable; un contenedor es un proceso aislado con límites y dependencias explícitas, no una máquina virtual pequeña.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: stack OCI endurecido y observable"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: stack oci endurecido y observable. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-05-containers-docker-oci/072-proyecto-stack-oci-endurecido-y-observable/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-contenedores en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-contenedores para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Docker Deep Dive — Nigel Poulton.

- Container Security — Liz Rice.
- Cloud Native Patterns — Cornelia Davis.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 072 Proyecto: stack OCI endurecido y observable

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-contenedores con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 06 — Kubernetes y plataformas administradas

← Parte 05 · Índice completo · Parte 07 →

Nivel: intermedio-avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Operar cargas declarativas y portables sobre EKS, AKS o GKE.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
073	API server, etcd, scheduler, controllers y kubelet	kubernetes	4
074	Pods, ReplicaSets, Deployments y Jobs	kubernetes	4
075	Services, DNS, Ingress y Gateway API	network	4
076	ConfigMaps, Secrets y configuración externa	configuration	4
077	Volumes, PersistentVolumes, CSI y StatefulSets	storage	4
078	Requests, limits, scheduling y autoscaling	capacity	4
079	Probes, rollouts, rollback y PodDisruptionBudget	reliability	4
080	Namespaces, RBAC, NetworkPolicy y admission	security	4
081	Helm, Kustomize y gestión de paquetes	configuration	4
082	Logs, métricas, eventos y depuración	observability	4
083	EKS, AKS y GKE: similitudes y diferencias	decision	4
084	Proyecto: plataforma Kubernetes portable	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..

073 — API server, etcd, scheduler, controllers y kubelet

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: kubernetes · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar api server, etcd, scheduler, controllers y kubelet dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar api server, etcd, scheduler, controllers y kubelet con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-control-plane con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
api	Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real.
server	Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real.
etcd	Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real.
scheduler	Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real.
controllers	Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real.
kubelet	Define su papel en api server, etcd, scheduler, controllers y kubelet y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: API server, etcd, scheduler, controllers y kubelet"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar api server, etcd, scheduler, controllers y kubelet. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/073-api-server-etcd-scheduler-controllers-y-kubelet/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa mapa-control-plane en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye mapa-control-plane para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Benda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 073 API server, etcd, scheduler, controllers y kubelet

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega mapa-control-plane con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

074 — Pods, ReplicaSets, Deployments y Jobs

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: kubernetes · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar pods, replicasets, deployments y jobs dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pods, replicasets, deployments y jobs con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar workload-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
pods	Define su papel en pods, replicasets, deployments y jobs y cómo observarlo en un sistema real.
replicasets	Define su papel en pods, replicasets, deployments y jobs y cómo observarlo en un sistema real.
deployments	Define su papel en pods, replicasets, deployments y jobs y cómo observarlo en un sistema real.
jobs	Define su papel en pods, replicasets, deployments y jobs y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Pods, ReplicaSets, Deployments y Jobs"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pods, replicasets, deployments y jobs. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/074-pods-replicasets-deployments-y-jobs/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa workload-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye workload-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 074 Pods, ReplicaSets, Deployments y Jobs

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega workload-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

075 — Services, DNS, Ingress y Gateway API

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar services, dns, ingress y gateway api dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar services, dns, ingress y gateway api con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar exposicion-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
services	Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real.
dns	Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real.
ingress	Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real.
gateway	Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real.
api	Define su papel en services, dns, ingress y gateway api y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Services, DNS, Ingress y Gateway API"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar services, dns, ingress y gateway api. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/075-services-dns-ingress-y-gateway-api/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa exposicion-kubernetes en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `exposicion-kubernetes` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.

- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 075 Services, DNS, Ingress y Gateway API

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega exposicion-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

076 — ConfigMaps, Secrets y configuración externa

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: configuration · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar configmaps, secrets y configuración externa dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar configmaps, secrets y configuración externa con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar configuracion-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
configmaps	Define su papel en configmaps, secrets y configuración externa y cómo observarlo en un sistema real.
secrets	Define su papel en configmaps, secrets y configuración externa y cómo observarlo en un sistema real.
configuración	Define su papel en configmaps, secrets y configuración externa y cómo observarlo en un sistema real.
externa	Define su papel en configmaps, secrets y configuración externa y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: ConfigMaps, Secrets y configuración externa"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar configmaps, secrets y configuración externa. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/076-configmaps-secrets-y-configuracion-externa/lab.py
```

El laboratorio selecciona el motor de práctica configuration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa configuracion-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es configuración separada, validada y promovible. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye configuración-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 076 ConfigMaps, Secrets y configuración externa

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega configuracion-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

077 — Volumes, PersistentVolumes, CSI y StatefulSets

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: storage · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar volumes, persistentvolumes, csi y statefulsets dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar volumes, persistentvolumes, csi y statefulsets con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar estado-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
volumes	Define su papel en volumes, persistentvolumes, csi y statefulsets y cómo observarlo en un sistema real.
persistentvolumes	Define su papel en volumes, persistentvolumes, csi y statefulsets y cómo observarlo en un sistema real.
csi	Define su papel en volumes, persistentvolumes, csi y statefulsets y cómo observarlo en un sistema real.
statefulsets	Define su papel en volumes, persistentvolumes, csi y statefulsets y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Volumes, PersistentVolumes, CSI y StatefulSets"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar volumes, persistentvolumes, csi y statefulsets. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/077-volumes-persistentvolumes-csi-y-statefulsets/lab.py
```

El laboratorio selecciona el motor de práctica storage y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa estado-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una política de durabilidad, acceso, retención y costo. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye estado-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 077 Volumes, PersistentVolumes, CSI y StatefulSets

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega estado-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

078 — Requests, limits, scheduling y autoscaling

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: capacity · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar requests, limits, scheduling y autoscaling dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar requests, limits, scheduling y autoscaling con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar capacidad-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
requests	Define su papel en requests, limits, scheduling y autoscaling y cómo observarlo en un sistema real.
limits	Define su papel en requests, limits, scheduling y autoscaling y cómo observarlo en un sistema real.
scheduling	Define su papel en requests, limits, scheduling y autoscaling y cómo observarlo en un sistema real.
autoscaling	Define su papel en requests, limits, scheduling y autoscaling y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Requests, limits, scheduling y autoscaling"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar requests, limits, scheduling y autoscaling. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/078-requests-limits-scheduling-y-autoscaling/lab.py
```

El laboratorio selecciona el motor de práctica capacity y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa capacidad-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es requests, límites y una decisión de escalado medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye capacidad-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 078 Requests, limits, scheduling y autoscaling

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega capacidad-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

079 — Probes, rollouts, rollback y PodDisruptionBudget

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar probes, rollouts, rollback y poddisruptionbudget dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar probes, rollouts, rollback y poddisruptionbudget con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar despliegue-resiliente con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
probes	Define su papel en probes, rollouts, rollback y poddisruptionbudget y cómo observarlo en un sistema real.
rollouts	Define su papel en probes, rollouts, rollback y poddisruptionbudget y cómo observarlo en un sistema real.
rollback	Define su papel en probes, rollouts, rollback y poddisruptionbudget y cómo observarlo en un sistema real.
poddisruptionbudget	Define su papel en probes, rollouts, rollback y poddisruptionbudget y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Probes, rollouts, rollback y PodDisruptionBudget"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar probes, rollouts, rollback y poddisruptionbudget. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/079-probes-rollouts-rollback-y-poddisruptionbudget/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa despliegue-resiliente en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye despliegue-resiliente para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Benda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 079 Probes, rollouts, rollback y PodDisruptionBudget

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega despliegue-resiliente con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

080 — Namespaces, RBAC, NetworkPolicy y admission

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar namespaces, rbac, networkpolicy y admission dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar namespaces, rbac, networkpolicy y admission con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar tenant-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
namespaces	Define su papel en namespaces, rbac, networkpolicy y admission y cómo observarlo en un sistema real.
rbac	Define su papel en namespaces, rbac, networkpolicy y admission y cómo observarlo en un sistema real.
networkpolicy	Define su papel en namespaces, rbac, networkpolicy y admission y cómo observarlo en un sistema real.
admission	Define su papel en namespaces, rbac, networkpolicy y admission y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Namespaces, RBAC, NetworkPolicy y admission"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar namespaces, rbac, networkpolicy y admission. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/080-namespaces-rbac-networkpolicy-y-admission/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa tenant-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye tenant-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 080 Namespaces, RBAC, NetworkPolicy y admission

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega tenant-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

081 — Helm, Kustomize y gestión de paquetes

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: configuration · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar helm, kustomize y gestión de paquetes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar helm, kustomize y gestión de paquetes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar paquete-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
helm	Define su papel en helm, kustomize y gestión de paquetes y cómo observarlo en un sistema real.
kustomize	Define su papel en helm, kustomize y gestión de paquetes y cómo observarlo en un sistema real.
gestión	Define su papel en helm, kustomize y gestión de paquetes y cómo observarlo en un sistema real.
paquetes	Define su papel en helm, kustomize y gestión de paquetes y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Helm, Kustomize y gestión de paquetes"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar helm, kustomize y gestión de paquetes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/081-helm-kustomize-y-gestion-de-paquetes/lab.py
```

El laboratorio selecciona el motor de práctica configuration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa paquete-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es configuración separada, validada y promovible. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye paquete-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 081 Helm, Kustomize y gestión de paquetes

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega paquete-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

082 — Logs, métricas, eventos y depuración

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar logs, métricas, eventos y depuración dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar logs, métricas, eventos y depuración con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar diagnostico-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
logs	Define su papel en logs, métricas, eventos y depuración y cómo observarlo en un sistema real.
métricas	Define su papel en logs, métricas, eventos y depuración y cómo observarlo en un sistema real.
eventos	Define su papel en logs, métricas, eventos y depuración y cómo observarlo en un sistema real.
depuración	Define su papel en logs, métricas, eventos y depuración y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Logs, métricas, eventos y depuración"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar logs, métricas, eventos y depuración. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/082-logs-metricas-eventos-y-depuracion/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa diagnostico-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye diagnostico-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 082 Logs, métricas, eventos y depuración

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega diagnóstico-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

083 — EKS, AKS y GKE: similitudes y diferencias

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar eks, aks y gke: similitudes y diferencias dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar eks, aks y gke: similitudes y diferencias con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-kubernetes-administrado con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
eks	Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real.
aks	Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real.
gke	Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real.
similitudes	Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real.
diferencias	Define su papel en eks, aks y gke: similitudes y diferencias y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: EKS, AKS y GKE: similitudes y diferencias"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar eks, aks y gke: similitudes y diferencias. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/083-eks-aks-y-gke-similitudes-y-diferencias/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-kubernetes-administrado en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-kubernetes-administrado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.

- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 083 EKS, AKS y GKE: similitudes y diferencias

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-kubernetes-administrado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

084 — Proyecto: plataforma Kubernetes portable

Parte: 06 — Kubernetes y plataformas administradas Nivel: intermedio-avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: plataforma kubernetes portable dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: plataforma kubernetes portable con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: plataforma kubernetes portable y cómo observarlo en un sistema real.
plataforma	Define su papel en proyecto: plataforma kubernetes portable y cómo observarlo en un sistema real.
kubernetes	Define su papel en proyecto: plataforma kubernetes portable y cómo observarlo en un sistema real.
portable	Define su papel en proyecto: plataforma kubernetes portable y cómo observarlo en un sistema real.

Modelo mental

Kubernetes es un sistema de reconciliación: declaras estado deseado y controladores reducen continuamente la diferencia con el estado observado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: plataforma Kubernetes portable"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: plataforma kubernetes portable. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-06-kubernetes-managed-platforms/084-proyecto-plataforma-kubernetes-portable/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Kubernetes: Up and Running — Burns, Beda y Hightower.
- Kubernetes Patterns — Ibryam y Huß.
- Production Kubernetes — Rosso et al..
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 084 Proyecto: plataforma Kubernetes portable

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 07 — Infraestructura como código y configuración

← Parte 06 · Índice completo · Parte 08 →

Nivel: intermedio-avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Convertir infraestructura y políticas en cambios revisables, probables y repetibles.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
085	Declarativo, imperativo, idempotencia y convergencia	iac	4
086	Terraform: HCL, providers, resources y grafo	iac	4
087	Estado remoto, locking, cifrado y recuperación	iac	4
088	Módulos, contratos, versiones y composición	iac	4
089	Variables, outputs, locals y data sources	iac	4
090	Plan, apply, drift, import y refactor con moved	iac	4
091	Validación, lint, pruebas y policy as code	testing	4
092	Secretos y datos sensibles en IaC	security	4
093	CloudFormation, Bicep, Pulumi y Terraform	decision	4
094	Ansible e imagen dorada para configuración	configuration	4
095	Plantillas, golden paths y catálogo interno	platform	4
096	Proyecto: infraestructura multiambiente promovible	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.

085 — Declarativo, imperativo, idempotencia y convergencia

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar declarativo, imperativo, idempotencia y convergencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar declarativo, imperativo, idempotencia y convergencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar comparativa-iac con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
declarativo	Define su papel en declarativo, imperativo, idempotencia y convergencia y cómo observarlo en un sistema real.
imperativo	Define su papel en declarativo, imperativo, idempotencia y convergencia y cómo observarlo en un sistema real.
idempotencia	Define su papel en declarativo, imperativo, idempotencia y convergencia y cómo observarlo en un sistema real.
convergencia	Define su papel en declarativo, imperativo, idempotencia y convergencia y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Declarativo, imperativo, idempotencia y convergencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar declarativo, imperativo, idempotencia y convergencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/085-declarativo-imperativo-idempotencia-y-convergencia/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa comparativa-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye comparativa-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 085 Declarativo, imperativo, idempotencia y convergencia

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega comparativa-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

086 — Terraform: HCL, providers, resources y grafo

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar terraform: hcl, providers, resources y grafo dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar terraform: hcl, providers, resources y grafo con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar stack-terraform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
terraform	Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real.
hcl	Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real.
providers	Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real.
resources	Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real.
grafo	Define su papel en terraform: hcl, providers, resources y grafo y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Terraform: HCL, providers, resources y grafo"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar terraform: hcl, providers, resources y grafo. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infraestructure-as-code-configuration/086-terraform-hcl-providers-resources-y-grafo/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa stack-terraform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye stack-terraform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 086 Terraform: HCL, providers, resources y grafo

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega stack-terraform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

087 — Estado remoto, locking, cifrado y recuperación

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar estado remoto, locking, cifrado y recuperación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar estado remoto, locking, cifrado y recuperación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar backend-terraform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
estado	Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real.
remoto	Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real.
locking	Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real.
cifrado	Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real.
recuperación	Define su papel en estado remoto, locking, cifrado y recuperación y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Estado remoto, locking, cifrado y recuperación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar estado remoto, locking, cifrado y recuperación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infraestructure-as-code-configuration/087-estado-remoto-locking-cifrado-y-recuperacion/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa backend-terraform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye backend-terraform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 087 Estado remoto, locking, cifrado y recuperación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega backend-terraform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

088 — Módulos, contratos, versiones y composición

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar módulos, contratos, versiones y composición dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar módulos, contratos, versiones y composición con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modulo-terraform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
módulos	Define su papel en módulos, contratos, versiones y composición y cómo observarlo en un sistema real.
contratos	Define su papel en módulos, contratos, versiones y composición y cómo observarlo en un sistema real.
versiones	Define su papel en módulos, contratos, versiones y composición y cómo observarlo en un sistema real.
composición	Define su papel en módulos, contratos, versiones y composición y cómo observarlo en un sistema real.

Modelo mental

laC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Módulos, contratos, versiones y composición"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar módulos, contratos, versiones y composición. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/088-modulos-contratos-versiones-y-composicion/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modulo-terraform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modulo-terraform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 088 Módulos, contratos, versiones y composición

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modulo-terraform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

089 — Variables, outputs, locals y data sources

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar variables, outputs, locals y data sources dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar variables, outputs, locals y data sources con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar interfaz-infraestructura con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
variables	Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real.
outputs	Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real.
locals	Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real.
data	Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real.
sources	Define su papel en variables, outputs, locals y data sources y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Variables, outputs, locals y data sources"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar variables, outputs, locals y data sources. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/089-variables-outputs-locals-y-data-sources/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa interfaz-infraestructura en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye interfaz-infraestructura para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 089 Variables, outputs, locals y data sources

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega interfaz-infraestructura con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

090 — Plan, apply, drift, import y refactor con moved

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar plan, apply, drift, import y refactor con moved dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar plan, apply, drift, import y refactor con moved con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ciclo-cambio-iac con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
plan	Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real.
apply	Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real.
drift	Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real.
import	Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real.
refactor	Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real.
moved	Define su papel en plan, apply, drift, import y refactor con moved y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Plan, apply, drift, import y refactor con moved"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar plan, apply, drift, import y refactor con moved. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/090-plan-apply-drift-import-y-refactor-con-moved/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa ciclo-cambio-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye ciclo-cambio-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 090 Plan, apply, drift, import y refactor con moved

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega ciclo-cambio-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

091 — Validación, lint, pruebas y policy as code

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: testing · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar validación, lint, pruebas y policy as code dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar validación, lint, pruebas y policy as code con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar pipeline-iac con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
validación	Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real.
lint	Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real.
pruebas	Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real.
policy	Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real.
as	Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real.
code	Define su papel en validación, lint, pruebas y policy as code y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Validación, lint, pruebas y policy as code"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar validación, lint, pruebas y policy as code. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/091-validacion-lint-pruebas-y-policy-as-code/lab.py
```

El laboratorio selecciona el motor de práctica testing y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa pipeline-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es pruebas automatizadas con fallos diagnósticos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye pipeline-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 091 Validación, lint, pruebas y policy as code

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega pipeline-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

092 — Secretos y datos sensibles en IaC

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar secretos y datos sensibles en iac dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar secretos y datos sensibles en iac con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-secretos-iac con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
secretos	Define su papel en secretos y datos sensibles en iac y cómo observarlo en un sistema real.
datos	Define su papel en secretos y datos sensibles en iac y cómo observarlo en un sistema real.
sensibles	Define su papel en secretos y datos sensibles en iac y cómo observarlo en un sistema real.
iac	Define su papel en secretos y datos sensibles en iac y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Secretos y datos sensibles en IaC"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar secretos y datos sensibles en iac. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/092-secretos-y-datos-sensibles-en-iac/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-secretos-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modelo-secretos-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 092 Secretos y datos sensibles en IaC

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modelo-secretos-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

093 — CloudFormation, Bicep, Pulumi y Terraform

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloudformation, bicep, pulumi y terraform dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloudformation, bicep, pulumi y terraform con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar adr-herramienta-iac con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloudformation	Define su papel en cloudformation, bicep, pulumi y terraform y cómo observarlo en un sistema real.
bicep	Define su papel en cloudformation, bicep, pulumi y terraform y cómo observarlo en un sistema real.
pulumi	Define su papel en cloudformation, bicep, pulumi y terraform y cómo observarlo en un sistema real.
terraform	Define su papel en cloudformation, bicep, pulumi y terraform y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: CloudFormation, Bicep, Pulumi y Terraform"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloudformation, bicep, pulumi y terraform. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/093-cloudformation-bicep-pulumi-y-terraform/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa adr-herramienta-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye adr-herramienta-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 093 CloudFormation, Bicep, Pulumi y Terraform

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega adr-herramienta-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

094 — Ansible e imagen dorada para configuración

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: configuration · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ansible e imagen dorada para configuración dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ansible e imagen dorada para configuración con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar configuracion-servidor con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ansible	Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real.
e	Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real.
imagen	Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real.
dorada	Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real.
configuración	Define su papel en ansible e imagen dorada para configuración y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Ansible e imagen dorada para configuración"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ansible e imagen dorada para configuración. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/094-ansible-e-imagen-dorada-para-configuracion/lab.py
```

El laboratorio selecciona el motor de práctica configuration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa configuracion-servidor en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es configuración separada, validada y promovible. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye configuracion-servidor para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 094 Ansible e imagen dorada para configuración

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega configuracion-servidor con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

095 — Plantillas, golden paths y catálogo interno

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 4 Laboratorio: platform · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar plantillas, golden paths y catálogo interno dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar plantillas, golden paths y catálogo interno con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plantilla-plataforma con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
plantillas	Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real.
golden	Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real.
paths	Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real.
catálogo	Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real.
interno	Define su papel en plantillas, golden paths y catálogo interno y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Plantillas, golden paths y catálogo interno"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar plantillas, golden paths y catálogo interno. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/095-plantillas-golden-paths-y-catalogo-interno/lab.py
```

El laboratorio selecciona el motor de práctica platform y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plantilla-plataforma en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una capacidad autoservicio con contrato y golden path. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plantilla-plataforma para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 095 Plantillas, golden paths y catálogo interno

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plantilla-plataforma con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

096 — Proyecto: infraestructura multiambiente promovible

Parte: 07 — Infraestructura como código y configuración Nivel: intermedio-avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: infraestructura multiambiente promovible dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: infraestructura multiambiente promovible con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-iac con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: infraestructura multiambiente promovible y cómo observarlo en un sistema real.
infraestructura	Define su papel en proyecto: infraestructura multiambiente promovible y cómo observarlo en un sistema real.
multiambiente	Define su papel en proyecto: infraestructura multiambiente promovible y cómo observarlo en un sistema real.
promovible	Define su papel en proyecto: infraestructura multiambiente promovible y cómo observarlo en un sistema real.

Modelo mental

IaC trata la infraestructura como producto versionado: el plan explica intención, el estado conecta intención y realidad, y la revisión reduce cambios accidentales.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: infraestructura multiambiente promovible"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: infraestructura multiambiente promovible. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-07-infrastructure-as-code-configuration/096-proyecto-infraestructura-multiambiente-promovible/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-iac en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-iac para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Infrastructure as Code — Kief Morris.
- Terraform: Up & Running — Yevgeniy Brikman.
- Ansible for DevOps — Jeff Geerling.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 096 Proyecto: infraestructura multiambiente promovible

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-iac con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 08 — Entrega continua y platform engineering

← Parte 07 · Índice completo · Parte 09 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar un flujo de entrega rápido, seguro y observable para equipos de producto.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
097	Integración continua, trunk-based development y feedback	delivery	4
098	GitHub Actions: workflows, runners, permisos y caché	delivery	4
099	Artefactos inmutables, semver y promoción	supply-chain	4
100	Pruebas, calidad y puertas de cambio	testing	4
101	SAST, SCA, secretos, SBOM y firma en pipeline	security	4
102	Rolling, blue-green, canary y rollback	delivery	4
103	GitOps con Argo CD o Flux	gitops	4
104	Ambientes efímeros y promoción entre entornos	delivery	4
105	Feature flags y separación deploy-release	delivery	4
106	Platform engineering e Internal Developer Platform	platform	4
107	Developer experience, DORA y carga cognitiva	metrics	4
108	Proyecto: fábrica de software multi-cloud	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.

097 — Integración continua, trunk-based development y feedback

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar integración continua, trunk-based development y feedback dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar integración continua, trunk-based development y feedback con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar flujo-ci con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
integración	Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real.
continua	Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real.
trunk	Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real.
based	Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real.
development	Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real.
feedback	Define su papel en integración continua, trunk-based development y feedback y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Integración continua, trunk-based development y feedback"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar integración continua, trunk-based development y feedback. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/097-integracion-continua-trunk-based-
```

development-y-feedback/lab.py

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa flujo-ci en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye flujo-ci para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 097 Integración continua, trunk-based development y feedback

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega flujo-ci con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

098 — GitHub Actions: workflows, runners, permisos y caché

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar github actions: workflows, runners, permisos y caché dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar github actions: workflows, runners, permisos y caché con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar pipeline-github-actions con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
github	Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real.
actions	Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real.
workflows	Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real.
runners	Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real.
permisos	Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real.
caché	Define su papel en github actions: workflows, runners, permisos y caché y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: GitHub Actions: workflows, runners, permisos y caché"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar github actions: workflows, runners, permisos y caché. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/098-github-actions-workflows-runners-permisos-y-cache/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa pipeline-github-actions en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye pipeline-github-actions para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 098 GitHub Actions: workflows, runners, permisos y caché

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega pipeline-github-actions con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

099 — Artefactos inmutables, semver y promoción

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: supply-chain · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar artefactos inmutables, semver y promoción dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar artefactos inmutables, semver y promoción con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar registro-artefactos con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
artefactos	Define su papel en artefactos inmutables, semver y promoción y cómo observarlo en un sistema real.
inmutables	Define su papel en artefactos inmutables, semver y promoción y cómo observarlo en un sistema real.
semver	Define su papel en artefactos inmutables, semver y promoción y cómo observarlo en un sistema real.
promoción	Define su papel en artefactos inmutables, semver y promoción y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Artefactos inmutables, semver y promoción"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar artefactos inmutables, semver y promoción. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/099-artefactos-inmutables-semver-y-promocion/lab.py
```

El laboratorio selecciona el motor de práctica supply-chain y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa registro-artefactos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es procedencia, inventario y verificación del artefacto. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye registro-artefactos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 099 Artefactos inmutables, semver y promoción

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega registro-artefactos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

100 — Pruebas, calidad y puertas de cambio

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: testing · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar pruebas, calidad y puertas de cambio dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pruebas, calidad y puertas de cambio con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar quality-gates con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
pruebas	Define su papel en pruebas, calidad y puertas de cambio y cómo observarlo en un sistema real.
calidad	Define su papel en pruebas, calidad y puertas de cambio y cómo observarlo en un sistema real.
puertas	Define su papel en pruebas, calidad y puertas de cambio y cómo observarlo en un sistema real.
cambio	Define su papel en pruebas, calidad y puertas de cambio y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Pruebas, calidad y puertas de cambio"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pruebas, calidad y puertas de cambio. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/100-pruebas-calidad-y-puertas-de-cambio/lab.py
```

El laboratorio selecciona el motor de práctica testing y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa quality-gates en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es pruebas automatizadas con fallos diagnósticos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye quality-gates para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 100 Pruebas, calidad y puertas de cambio

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega quality-gates con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

101 — SAST, SCA, secretos, SBOM y firma en pipeline

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar sast, sca, secretos, sbom y firma en pipeline dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sast, sca, secretos, sbom y firma en pipeline con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar pipeline-devsecops con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
sast	Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real.
sca	Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real.
secretos	Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real.
sbom	Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real.
firma	Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real.
pipeline	Define su papel en sast, sca, secretos, sbom y firma en pipeline y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: SAST, SCA, secretos, SBOM y firma en pipeline"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sast, sca, secretos, sbom y firma en pipeline. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/101-sast-sca-secretos-sbom-y-firma-en-pipeline/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa pipeline-devsecops en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye pipeline-devsecops para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 101 SAST, SCA, secretos, SBOM y firma en pipeline

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega pipeline-devsecops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

102 — Rolling, blue-green, canary y rollback

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar rolling, blue-green, canary y rollback dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar rolling, blue-green, canary y rollback con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar estrategia-despliegue con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
rolling	Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real.
blue	Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real.
green	Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real.
canary	Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real.
rollback	Define su papel en rolling, blue-green, canary y rollback y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Rolling, blue-green, canary y rollback"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar rolling, blue-green, canary y rollback. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/102-rolling-blue-green-canary-y-rollback/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa estrategia-despliegue en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye estrategia-despliegue para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 102 Rolling, blue-green, canary y rollback

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega estrategia-despliegue con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

103 — GitOps con Argo CD o Flux

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: gitops · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar gitops con argo cd o flux dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar gitops con argo cd o flux con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar reconciliacion-gitops con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
gitops	Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real.
argo	Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real.
cd	Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real.
o	Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real.
flux	Define su papel en gitops con argo cd o flux y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: GitOps con Argo CD o Flux"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar gitops con argo cd o flux. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/103-gitops-con-argo-cd-o-flux/lab.py
```

El laboratorio selecciona el motor de práctica gitops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa reconciliacion-gitops en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es reconciliación declarativa y evidencia de drift. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye reconciliación-gitops para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 103 GitOps con Argo CD o Flux

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega reconciliación-gitops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

104 — Ambientes efímeros y promoción entre entornos

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ambientes efímeros y promoción entre entornos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ambientes efímeros y promoción entre entornos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar flujo-ambientes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ambientes	Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real.
efímeros	Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real.
promoción	Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real.
entre	Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real.
entornos	Define su papel en ambientes efímeros y promoción entre entornos y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Ambientes efímeros y promoción entre entornos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ambientes efímeros y promoción entre entornos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/104-ambientes-efimeros-y-promocion-entre-entornos/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa flujo-ambientes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye flujo-ambientes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 104 Ambientes efímeros y promoción entre entornos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega flujo-ambientes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

105 — Feature flags y separación deploy-release

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar feature flags y separación deploy-release dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar feature flags y separación deploy-release con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plan-feature-flags con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
feature	Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real.
flags	Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real.
separación	Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real.
deploy	Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real.
release	Define su papel en feature flags y separación deploy-release y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Feature flags y separación deploy-release"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar feature flags y separación deploy-release. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/105-feature-flags-y-separacion-deploy-release/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plan-feature-flags en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plan-feature-flags para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 105 Feature flags y separación deploy-release

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plan-feature-flags con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

106 — Platform engineering e Internal Developer Platform

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: platform · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar platform engineering e internal developer platform dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar platform engineering e internal developer platform con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-capacidades-plataforma con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
platform	Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real.
engineering	Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real.
e	Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real.
internal	Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real.
developer	Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real.
platform	Define su papel en platform engineering e internal developer platform y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Platform engineering e Internal Developer Platform"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar platform engineering e internal developer platform. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/106-platform-engineering-e-internal-developer-platform/lab.py
```

El laboratorio selecciona el motor de práctica platform y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa mapa-capacidades-plataforma en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una capacidad autoservicio con contrato y golden path. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye mapa-capacidades-plataforma para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 106 Platform engineering e Internal Developer Platform

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega mapa-capacidades-plataforma con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

107 — Developer experience, DORA y carga cognitiva

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 4 Laboratorio: metrics · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar developer experience, dora y carga cognitiva dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar developer experience, dora y carga cognitiva con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar tablero-dora con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
developer	Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real.
experience	Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real.
dora	Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real.
carga	Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real.
cognitiva	Define su papel en developer experience, dora y carga cognitiva y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Developer experience, DORA y carga cognitiva"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar developer experience, dora y carga cognitiva. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/107-developer-experience-dora-y-carga-cognitiva/lab.py
```

El laboratorio selecciona el motor de práctica metrics y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa tablero-dora en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es métricas definidas, consultables y vinculadas a una decisión. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye tablero-dora para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 107 Developer experience, DORA y carga cognitiva

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega tablero-dora con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

108 — Proyecto: fábrica de software multi-cloud

Parte: 08 — Entrega continua y platform engineering Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: fábrica de software multi-cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: fábrica de software multi-cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-entrega con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real.
fábrica	Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real.
software	Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real.
multi	Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real.
cloud	Define su papel en proyecto: fábrica de software multi-cloud y cómo observarlo en un sistema real.

Modelo mental

Una plataforma interna ofrece capacidades como productos y reduce carga cognitiva mediante caminos dorados, sin quitar autonomía donde importa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Proyecto: fábrica de software multi-cloud"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: fábrica de software multi-cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-08-continuous-delivery-platform-engineering/108-proyecto-fabrica-de-software-multi-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plataforma-entrega en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-entrega para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Continuous Delivery — Humble y Farley.
- Accelerate — Forsgren, Humble y Kim.
- Team Topologies — Skelton y Pais.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 108 Proyecto: fábrica de software multi-cloud

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-entrega con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 09 — Datos, mensajería, serverless e integración

← Parte 08 · Índice completo · Parte 10 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Elegir servicios de datos e integración por sus garantías y patrones de acceso.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
109	Bases relacionales administradas y pooling	data	4
110	NoSQL: clave-valor, documento, columna y grafo	data	4
111	Caché, invalidación, TTL y consistencia	data	4
112	Object storage, data lake y formatos columnares	data	4
113	Colas, entrega, reintentos y dead-letter queues	messaging	4
114	Pub/sub, streams, particiones y orden	messaging	4
115	Arquitectura dirigida por eventos y contratos	architecture	4
116	Sagas, outbox, idempotencia y deduplicación	distributed	4
117	Serverless: límites, cold starts y concurrencia	serverless	4
118	API management, cuotas, versiones y monetización	api	4
119	Workflows y orquestación durable	orchestration	4
120	Proyecto: pipeline de pedidos orientado a eventos	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.

109 — Bases relacionales administradas y pooling

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar bases relacionales administradas y pooling dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bases relacionales administradas y pooling con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-relacional con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
bases	Define su papel en bases relacionales administradas y pooling y cómo observarlo en un sistema real.
relacionales	Define su papel en bases relacionales administradas y pooling y cómo observarlo en un sistema real.
administradas	Define su papel en bases relacionales administradas y pooling y cómo observarlo en un sistema real.
pooling	Define su papel en bases relacionales administradas y pooling y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Bases relacionales administradas y pooling"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bases relacionales administradas y pooling. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/109-bases-relacionales-administradas-y-pooling/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-relacional en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modelo-relacional para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 109 Bases relacionales administradas y pooling

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modelo-relacional con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

110 — NoSQL: clave-valor, documento, columna y grafo

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar nosql: clave-valor, documento, columna y grafo dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar nosql: clave-valor, documento, columna y grafo con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-nosql con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
nosql	Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real.
clave	Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real.
valor	Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real.
documento	Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real.
columna	Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real.
grafo	Define su papel en nosql: clave-valor, documento, columna y grafo y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: NoSQL: clave-valor, documento, columna y grafo"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar nosql: clave-valor, documento, columna y grafo. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/110-nosql-clave-valor-documento-columna-y-grafo/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa matriz-nosql en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-nosql para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 110 NoSQL: clave-valor, documento, columna y grafo

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-nosql con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

111 — Caché, invalidación, TTL y consistencia

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar caché, invalidación, ttl y consistencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar caché, invalidación, ttl y consistencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar estrategia-cache con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
caché	Define su papel en caché, invalidación, ttl y consistencia y cómo observarlo en un sistema real.
invalidación	Define su papel en caché, invalidación, ttl y consistencia y cómo observarlo en un sistema real.
ttl	Define su papel en caché, invalidación, ttl y consistencia y cómo observarlo en un sistema real.
consistencia	Define su papel en caché, invalidación, ttl y consistencia y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Caché, invalidación, TTL y consistencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar caché, invalidación, ttl y consistencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/111-cache-invalidacion-ttl-y-consistencia/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa estrategia-cache en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye estrategia-cache para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 111 Caché, invalidación, TTL y consistencia

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega estrategia-cache con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

112 — Object storage, data lake y formatos columnares

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar object storage, data lake y formatos columnares dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar object storage, data lake y formatos columnares con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar diseno-data-lake con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
object	Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real.
storage	Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real.
data	Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real.
lake	Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real.
formatos	Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real.
columnares	Define su papel en object storage, data lake y formatos columnares y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Object storage, data lake y formatos columnares"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar object storage, data lake y formatos columnares. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/112-object-storage-data-lake-y-formatos-columnares/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa diseno-data-lake en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye diseno-data-lake para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 112 Object storage, data lake y formatos columnares

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega diseño-data-lake con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

113 — Colas, entrega, reintentos y dead-letter queues

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar colas, entrega, reintentos y dead-letter queues dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar colas, entrega, reintentos y dead-letter queues con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cola-resiliente con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
colas	Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real.
entrega	Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real.
reintentos	Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real.
dead	Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real.
letter	Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real.
queues	Define su papel en colas, entrega, reintentos y dead-letter queues y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Colas, entrega, reintentos y dead-letter queues"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar colas, entrega, reintentos y dead-letter queues. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/113-colas-entrega-reintentos-y-dead-letter-queues/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa cola-resiliente en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cola-resiliente para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 113 Colas, entrega, reintentos y dead-letter queues

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cola-resiliente con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

114 — Pub/sub, streams, particiones y orden

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar pub/sub, streams, particiones y orden dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pub/sub, streams, particiones y orden con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar stream-particionado con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
pub	Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real.
sub	Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real.
streams	Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real.
particiones	Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real.
orden	Define su papel en pub/sub, streams, particiones y orden y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Pub/sub, streams, particiones y orden"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pub/sub, streams, particiones y orden. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/114-pub-sub-streams-particiones-y-orden/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa stream-particionado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye stream-particionado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 114 Pub/sub, streams, particiones y orden

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega stream-particionado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

115 — Arquitectura dirigida por eventos y contratos

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar arquitectura dirigida por eventos y contratos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar arquitectura dirigida por eventos y contratos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar catalogo-eventos con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
arquitectura	Define su papel en arquitectura dirigida por eventos y contratos y cómo observarlo en un sistema real.
dirigida	Define su papel en arquitectura dirigida por eventos y contratos y cómo observarlo en un sistema real.
eventos	Define su papel en arquitectura dirigida por eventos y contratos y cómo observarlo en un sistema real.
contratos	Define su papel en arquitectura dirigida por eventos y contratos y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Arquitectura dirigida por eventos y contratos"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar arquitectura dirigida por eventos y contratos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/115-arquitectura-dirigida-por-eventos-y-contratos/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa catalogo-eventos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye catalogo-eventos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 115 Arquitectura dirigida por eventos y contratos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega catalogo-eventos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

116 — Sagas, outbox, idempotencia y deduplicación

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: distributed · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar sagas, outbox, idempotencia y deduplicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sagas, outbox, idempotencia y deduplicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar transaccion-distribuida con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
sagas	Define su papel en sagas, outbox, idempotencia y deduplicación y cómo observarlo en un sistema real.
outbox	Define su papel en sagas, outbox, idempotencia y deduplicación y cómo observarlo en un sistema real.
idempotencia	Define su papel en sagas, outbox, idempotencia y deduplicación y cómo observarlo en un sistema real.
deduplicación	Define su papel en sagas, outbox, idempotencia y deduplicación y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Sagas, outbox, idempotencia y deduplicación"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sagas, outbox, idempotencia y deduplicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/116-sagas-outbox-idempotencia-y-deduplicacion/lab.py
```

El laboratorio selecciona el motor de práctica distributed y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa transaccion-distribuida en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una traza de consistencia, reintento o fallo parcial. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye transaccion-distribuida para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 116 Sagas, outbox, idempotencia y deduplicación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega transaccion-distribuida con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

117 — Serverless: límites, cold starts y concurrencia

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar serverless: límites, cold starts y concurrencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar serverless: límites, cold starts y concurrencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar funcion-operable con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
serverless	Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real.
límites	Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real.
cold	Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real.
starts	Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real.
concurrencia	Define su papel en serverless: límites, cold starts y concurrencia y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Serverless: límites, cold starts y concurrencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar serverless: límites, cold starts y concurrencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/117-serverless-limit-cold-starts-y-concurrencia/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa funcion-operable en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye funcion-operable para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 117 Serverless: límites, cold starts y concurrencia

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega funcion-operable con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

118 — API management, cuotas, versiones y monetización

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: api · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar api management, cuotas, versiones y monetización dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar api management, cuotas, versiones y monetización con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar producto-api con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
api	Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real.
management	Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real.
cuotas	Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real.
versiones	Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real.
monetización	Define su papel en api management, cuotas, versiones y monetización y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: API management, cuotas, versiones y monetización"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar api management, cuotas, versiones y monetización. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/118-api-management-cuotas-versiones-y-monetizacion/lab.py
```

El laboratorio selecciona el motor de práctica api y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

- 4. Materializa producto-api en evidence/ usando la plantilla indicada.
- 5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un contrato versionado con pruebas positivas y negativas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye producto-api para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 118 API management, cuotas, versiones y monetización

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega producto-api con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

119 — Workflows y orquestación durable

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 4 Laboratorio: orchestration · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar workflows y orquestación durable dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar workflows y orquestación durable con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar workflow-durable con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
workflows	Define su papel en workflows y orquestación durable y cómo observarlo en un sistema real.
orquestación	Define su papel en workflows y orquestación durable y cómo observarlo en un sistema real.
durable	Define su papel en workflows y orquestación durable y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Workflows y orquestación durable"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar workflows y orquestación durable. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/119-workflows-y-orquestacion-durable/lab.py
```

El laboratorio selecciona el motor de práctica orchestration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa workflow-durable en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es servicios coordinados con health checks y apagado limpio. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye workflow-durable para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 119 Workflows y orquestación durable

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega workflow-durable con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

120 — Proyecto: pipeline de pedidos orientado a eventos

Parte: 09 — Datos, mensajería, serverless e integración Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: pipeline de pedidos orientado a eventos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: pipeline de pedidos orientado a eventos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-eventos con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real.
pipeline	Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real.
pedidos	Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real.
orientado	Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real.
eventos	Define su papel en proyecto: pipeline de pedidos orientado a eventos y cómo observarlo en un sistema real.

Modelo mental

La base de datos correcta depende del patrón de acceso y de las garantías necesarias; distribuir datos convierte latencia, consistencia y fallos parciales en decisiones de diseño.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Proyecto: pipeline de pedidos orientado a eventos"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: pipeline de pedidos orientado a eventos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-09-data-messaging-serverless-integration/120-proyecto-pipeline-de-pedidos-orientado-a-eventos/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plataforma-eventos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-eventos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Data-Intensive Applications — Martin Kleppmann.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Building Event-Driven Microservices — Adam Bellemare.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 120 Proyecto: pipeline de pedidos orientado a eventos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-eventos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 10 — Observabilidad, SRE y confiabilidad

← Parte 09 · Índice completo · Parte 11 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Operar sistemas mediante señales, objetivos y aprendizaje de incidentes.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
121	Logs, métricas, trazas y eventos como señales	observability	4
122	Logging estructurado, correlación y retención	observability	4
123	Métricas, cardinalidad y modelos RED y USE	metrics	4
124	Tracing distribuido y OpenTelemetry	observability	4
125	Dashboards, alertas accionables y fatiga	observability	4
126	SLI, SLO, SLA y presupuesto de error	sre	4
127	Incidentes, severidad, comando y comunicación	incident	4
128	Runbooks, playbooks y automatización operativa	operations	4
129	Capacidad, rendimiento y pruebas de carga	performance	4
130	Timeouts, retries, backoff, circuit breaker y bulkhead	reliability	4
131	Chaos engineering y game days	chaos	4
132	Proyecto: operación SRE de CloudShop	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.

121 — Logs, métricas, trazas y eventos como señales

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar logs, métricas, trazas y eventos como señales dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar logs, métricas, trazas y eventos como señales con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-telemetria con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
logs	Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real.
métricas	Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real.
trazas	Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real.
eventos	Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real.
señales	Define su papel en logs, métricas, trazas y eventos como señales y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Logs, métricas, trazas y eventos como señales"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar logs, métricas, trazas y eventos como señales. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/121-logs-metricas-trazas-y-eventos-como-senales/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa mapa-telemetría en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye mapa-telemetría para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 121 Logs, métricas, trazas y eventos como señales

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega mapa-telemetría con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

122 — Logging estructurado, correlación y retención

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar logging estructurado, correlación y retención dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar logging estructurado, correlación y retención con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contrato-logs con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
logging	Define su papel en logging estructurado, correlación y retención y cómo observarlo en un sistema real.
estructurado	Define su papel en logging estructurado, correlación y retención y cómo observarlo en un sistema real.
correlación	Define su papel en logging estructurado, correlación y retención y cómo observarlo en un sistema real.
retención	Define su papel en logging estructurado, correlación y retención y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Logging estructurado, correlación y retención"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar logging estructurado, correlación y retención. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/122-logging-estructurado-correlacion-y-retencion/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa contrato-logs en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye contrato-logs para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 122 Logging estructurado, correlación y retención

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega contrato-logs con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

123 — Métricas, cardinalidad y modelos RED y USE

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: metrics · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar métricas, cardinalidad y modelos red y use dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar métricas, cardinalidad y modelos red y use con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar catalogo-metricas con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
métricas	Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real.
cardinalidad	Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real.
modelos	Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real.
red	Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real.
use	Define su papel en métricas, cardinalidad y modelos red y use y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Métricas, cardinalidad y modelos RED y USE"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar métricas, cardinalidad y modelos red y use. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/123-metricas-cardinalidad-y-modelos-red-y-use/lab.py
```

El laboratorio selecciona el motor de práctica metrics y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa catalogo-metricas en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es métricas definidas, consultables y vinculadas a una decisión. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye catalogo-metricas para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 123 Métricas, cardinalidad y modelos RED y USE

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega catalogo-metricas con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

124 — Tracing distribuido y OpenTelemetry

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar tracing distribuido y opentelemetry dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar tracing distribuido y opentelemetry con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar traza-distribuida con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
tracing	Define su papel en tracing distribuido y opentelemetry y cómo observarlo en un sistema real.
distribuido	Define su papel en tracing distribuido y opentelemetry y cómo observarlo en un sistema real.
opentelemetry	Define su papel en tracing distribuido y opentelemetry y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Tracing distribuido y OpenTelemetry"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar tracing distribuido y opentelemetry. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/124-tracing-distribuido-y-opentelemetry/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa traza-distribuida en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye traza-distribuida para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 124 Tracing distribuido y OpenTelemetry

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega traza-distribuida con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

125 — Dashboards, alertas accionables y fatiga

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar dashboards, alertas accionables y fatiga dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar dashboards, alertas accionables y fatiga con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar tablero-operativo con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
dashboards	Define su papel en dashboards, alertas accionables y fatiga y cómo observarlo en un sistema real.
alertas	Define su papel en dashboards, alertas accionables y fatiga y cómo observarlo en un sistema real.
accionables	Define su papel en dashboards, alertas accionables y fatiga y cómo observarlo en un sistema real.
fatiga	Define su papel en dashboards, alertas accionables y fatiga y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Dashboards, alertas accionables y fatiga"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar dashboards, alertas accionables y fatiga. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/125-dashboards-alertas-accionables-y-fatiga/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa tablero-operativo en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye tablero-operativo para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 125 Dashboards, alertas accionables y fatiga

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega tablero-operativo con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

126 — SLI, SLO, SLA y presupuesto de error

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: sre · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar sli, slo, sla y presupuesto de error dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sli, slo, sla y presupuesto de error con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar slo-servicio con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
sli	Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real.
slo	Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real.
sla	Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real.
presupuesto	Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real.
error	Define su papel en sli, slo, sla y presupuesto de error y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: SLI, SLO, SLA y presupuesto de error"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sli, slo, sla y presupuesto de error. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/126-sli-slo-sla-y-presupuesto-de-error/lab.py
```

El laboratorio selecciona el motor de práctica sre y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa slo-servicio en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un SLO con presupuesto de error y política de acción. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye slo-servicio para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 126 SLI, SLO, SLA y presupuesto de error

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega slo-servicio con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

127 — Incidentes, severidad, comando y comunicación

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: incident · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar incidentes, severidad, comando y comunicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar incidentes, severidad, comando y comunicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plan-incidente con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
incidentes	Define su papel en incidentes, severidad, comando y comunicación y cómo observarlo en un sistema real.
severidad	Define su papel en incidentes, severidad, comando y comunicación y cómo observarlo en un sistema real.
comando	Define su papel en incidentes, severidad, comando y comunicación y cómo observarlo en un sistema real.
comunicación	Define su papel en incidentes, severidad, comando y comunicación y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Incidentes, severidad, comando y comunicación"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar incidentes, severidad, comando y comunicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/127-incidentes-severidad-comando-y-comunicacion/lab.py
```

El laboratorio selecciona el motor de práctica incident y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plan-incidente en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una cronología, roles, comunicación y aprendizaje. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plan-incidente para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 127 Incidentes, severidad, comando y comunicación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plan-incidente con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

128 — Runbooks, playbooks y automatización operativa

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: operations · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar runbooks, playbooks y automatización operativa dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar runbooks, playbooks y automatización operativa con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar runbook-verificado con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
runbooks	Define su papel en runbooks, playbooks y automatización operativa y cómo observarlo en un sistema real.
playbooks	Define su papel en runbooks, playbooks y automatización operativa y cómo observarlo en un sistema real.
automatización	Define su papel en runbooks, playbooks y automatización operativa y cómo observarlo en un sistema real.
operativa	Define su papel en runbooks, playbooks y automatización operativa y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Runbooks, playbooks y automatización operativa"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar runbooks, playbooks y automatización operativa. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/128-runbooks-playbooks-y-automatizacion-operativa/lab.py
```

El laboratorio selecciona el motor de práctica operations y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa runbook-verificado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un runbook probado por otra persona. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye runbook-verificado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 128 Runbooks, playbooks y automatización operativa

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega runbook-verificado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

129 — Capacidad, rendimiento y pruebas de carga

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: performance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capacidad, rendimiento y pruebas de carga dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capacidad, rendimiento y pruebas de carga con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar informe-carga con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capacidad	Define su papel en capacidad, rendimiento y pruebas de carga y cómo observarlo en un sistema real.
rendimiento	Define su papel en capacidad, rendimiento y pruebas de carga y cómo observarlo en un sistema real.
pruebas	Define su papel en capacidad, rendimiento y pruebas de carga y cómo observarlo en un sistema real.
carga	Define su papel en capacidad, rendimiento y pruebas de carga y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Capacidad, rendimiento y pruebas de carga"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capacidad, rendimiento y pruebas de carga. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/129-capacidad-rendimiento-y-pruebas-de-carga/lab.py
```

El laboratorio selecciona el motor de práctica performance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa informe-carga en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una prueba de carga con baseline y cuello de botella. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye informe-carga para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 129 Capacidad, rendimiento y pruebas de carga

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega informe-carga con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

130 — Timeouts, retries, backoff, circuit breaker y bulkhead

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar timeouts, retries, backoff, circuit breaker y bulkhead dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar timeouts, retries, backoff, circuit breaker y bulkhead con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cliente-resiliente con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
timeouts	Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real.
retries	Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real.
backoff	Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real.
circuit	Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real.
breaker	Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real.
bulkhead	Define su papel en timeouts, retries, backoff, circuit breaker y bulkhead y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Timeouts, retries, backoff, circuit breaker y bulkhead"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar timeouts, retries, backoff, circuit breaker y bulkhead. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/130-timeouts-retries-backoff-circuit-breaker-y-bulkhead/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa cliente-resiliente en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cliente-resiliente para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 130 Timeouts, retries, backoff, circuit breaker y bulkhead

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cliente-resiliente con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

131 — Chaos engineering y game days

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 4 Laboratorio: chaos · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar chaos engineering y game days dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar chaos engineering y game days con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar experimento-caos con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
chaos	Define su papel en chaos engineering y game days y cómo observarlo en un sistema real.
engineering	Define su papel en chaos engineering y game days y cómo observarlo en un sistema real.
game	Define su papel en chaos engineering y game days y cómo observarlo en un sistema real.
days	Define su papel en chaos engineering y game days y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Chaos engineering y game days"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar chaos engineering y game days. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/131-chaos-engineering-y-game-days/lab.py
```

El laboratorio selecciona el motor de práctica chaos y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa experimento-caos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una hipótesis de resiliencia y criterio de abortar. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye experimento-caos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 131 Chaos engineering y game days

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega experimento-caos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

132 — Proyecto: operación SRE de CloudShop

Parte: 10 — Observabilidad, SRE y confiabilidad Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: operación sre de cloudshop dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: operación sre de cloudshop con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-sre con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: operación sre de cloudshop y cómo observarlo en un sistema real.
operación	Define su papel en proyecto: operación sre de cloudshop y cómo observarlo en un sistema real.
sre	Define su papel en proyecto: operación sre de cloudshop y cómo observarlo en un sistema real.
cloudshop	Define su papel en proyecto: operación sre de cloudshop y cómo observarlo en un sistema real.

Modelo mental

La confiabilidad es una característica del servicio percibido por usuarios; SRE la administra con objetivos explícitos y presupuestos de error.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: operación SRE de CloudShop"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: operación sre de cloudshop. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-10-observability-sre-reliability/132-proyecto-operacion-sre-de-cloudshop/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa plataforma-sre en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-sre para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Site Reliability Engineering — Beyer et al..
- The Site Reliability Workbook — Beyer et al..
- Observability Engineering — Majors, Fong-Jones y Miranda.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 132 Proyecto: operación SRE de CloudShop

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-sre con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 11 — Seguridad, gobierno, cumplimiento y FinOps

← Parte 10 · Índice completo · Parte 12 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Construir guardrails que hagan segura y económicamente sostenible la autonomía.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
133	Zero Trust y defensa en profundidad	security	4
134	Mínimo privilegio, acceso temporal y separación de funciones	iam	4
135	Segmentación, perímetro, WAF, DDoS y egress	network	4
136	Cifrado, KMS, HSM, rotación y envelope encryption	security	4
137	Gestión de secretos y credenciales de workloads	security	4
138	Vulnerabilidades, imágenes y cadena de suministro	supply-chain	4
139	CSPM, postura, policy as code y remediación	governance	4
140	Threat modeling con STRIDE y attack paths	security	4
141	Cumplimiento, residencia, privacidad y evidencia	compliance	4
142	FinOps: showback, chargeback, budgets y anomalías	finops	4
143	Optimización de costo, capacidad y sostenibilidad	finops	4
144	Proyecto: landing zone con guardrails	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.

133 — Zero Trust y defensa en profundidad

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar zero trust y defensa en profundidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar zero trust y defensa en profundidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-zero-trust con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
zero	Define su papel en zero trust y defensa en profundidad y cómo observarlo en un sistema real.
trust	Define su papel en zero trust y defensa en profundidad y cómo observarlo en un sistema real.
defensa	Define su papel en zero trust y defensa en profundidad y cómo observarlo en un sistema real.
profundidad	Define su papel en zero trust y defensa en profundidad y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Zero Trust y defensa en profundidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E["¿Cumple seguridad, SLO y costo?"]
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar zero trust y defensa en profundidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/133-zero-trust-y-defensa-en-profundidad/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-zero-trust en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modelo-zero-trust para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 133 Zero Trust y defensa en profundidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modelo-zero-trust con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

134 — Mínimo privilegio, acceso temporal y separación de funciones

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar mínimo privilegio, acceso temporal y separación de funciones dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar mínimo privilegio, acceso temporal y separación de funciones con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar revision-accesos con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
mínimo	Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real.
privilegio	Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real.
acceso	Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real.
temporal	Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real.
separación	Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real.
funciones	Define su papel en mínimo privilegio, acceso temporal y separación de funciones y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Mínimo privilegio, acceso temporal y separación de funciones"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar mínimo privilegio, acceso temporal y separación de funciones. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/134-minimo-privilegio-acceso-temporal-y-separacion-
```

de-funciones/lab.py

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa revision-accesos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye revision-accesos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 134 Mínimo privilegio, acceso temporal y separación de funciones

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega revision-accesos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

135 — Segmentación, perímetro, WAF, DDoS y egress

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar segmentación, perímetro, waf, ddos y egress dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar segmentación, perímetro, waf, ddos y egress con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar arquitectura-defensiva con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
segmentación	Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real.
perímetro	Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real.
waf	Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real.
ddos	Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real.
egress	Define su papel en segmentación, perímetro, waf, ddos y egress y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Segmentación, perímetro, WAF, DDoS y egress"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar segmentación, perímetro, waf, ddos y egress. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/135-segmentacion-perimetro-waf-ddos-y-egress/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa arquitectura-defensiva en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye arquitectura-defensiva para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.

- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 135 Segmentación, perímetro, WAF, DDoS y egress

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega arquitectura-defensiva con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

136 — Cifrado, KMS, HSM, rotación y envelope encryption

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cifrado, kms, hsm, rotación y envelope encryption dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cifrado, kms, hsm, rotación y envelope encryption con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar jerarquía-claves con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cifrado	Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real.
kms	Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real.
hsm	Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real.
rotación	Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real.
envelope	Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real.
encryption	Define su papel en cifrado, kms, hsm, rotación y envelope encryption y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cifrado, KMS, HSM, rotación y envelope encryption"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cifrado, kms, hsm, rotación y envelope encryption. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/136-cifrado-kms-hsm-rotacion-y-envelope-encryption/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa jerarquía-claves en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye jerarquía-claves para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 136 Cifrado, KMS, HSM, rotación y envelope encryption

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega jerarquía-claves con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

137 — Gestión de secretos y credenciales de workloads

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar gestión de secretos y credenciales de workloads dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar gestión de secretos y credenciales de workloads con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ciclo-secretos con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
gestión	Define su papel en gestión de secretos y credenciales de workloads y cómo observarlo en un sistema real.
secretos	Define su papel en gestión de secretos y credenciales de workloads y cómo observarlo en un sistema real.
credenciales	Define su papel en gestión de secretos y credenciales de workloads y cómo observarlo en un sistema real.
workloads	Define su papel en gestión de secretos y credenciales de workloads y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Gestión de secretos y credenciales de workloads"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar gestión de secretos y credenciales de workloads. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/137-gestion-de-secretos-y-credenciales-de-workloads/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa ciclo-secretos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye ciclo-secretos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 137 Gestión de secretos y credenciales de workloads

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega ciclo-secretos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

138 — Vulnerabilidades, imágenes y cadena de suministro

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: supply-chain · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar vulnerabilidades, imágenes y cadena de suministro dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar vulnerabilidades, imágenes y cadena de suministro con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar informe-supply-chain con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
vulnerabilidades	Define su papel en vulnerabilidades, imágenes y cadena de suministro y cómo observarlo en un sistema real.
imágenes	Define su papel en vulnerabilidades, imágenes y cadena de suministro y cómo observarlo en un sistema real.
cadena	Define su papel en vulnerabilidades, imágenes y cadena de suministro y cómo observarlo en un sistema real.
suministro	Define su papel en vulnerabilidades, imágenes y cadena de suministro y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Vulnerabilidades, imágenes y cadena de suministro"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar vulnerabilidades, imágenes y cadena de suministro. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/138-vulnerabilidades-imagenes-y-cadena-de-suministro/lab.py
```

El laboratorio selecciona el motor de práctica supply-chain y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa informe-supply-chain en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es procedencia, inventario y verificación del artefacto. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye informe-supply-chain para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 138 Vulnerabilidades, imágenes y cadena de suministro

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega informe-supply-chain con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

139 — CSPM, postura, policy as code y remediación

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cspm, postura, policy as code y remediación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cspm, postura, policy as code y remediación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar guardrail-policy con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
csdm	Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real.
postura	Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real.
policy	Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real.
as	Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real.
code	Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real.
remediación	Define su papel en cspm, postura, policy as code y remediación y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: CSPM, postura, policy as code y remediación"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cspm, postura, policy as code y remediación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/139-cspm-postura-policy-as-code-y-remediacion/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa guardrail-policy en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye guardrail-policy para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.

- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 139 CSPM, postura, policy as code y remediación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega guardrail-policy con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

140 — Threat modeling con STRIDE y attack paths

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar threat modeling con stride y attack paths dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar threat modeling con stride y attack paths con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-amenazas con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
threat	Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real.
modeling	Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real.
stride	Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real.
attack	Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real.
paths	Define su papel en threat modeling con stride y attack paths y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Threat modeling con STRIDE y attack paths"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar threat modeling con stride y attack paths. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/140-threat-modeling-con-stride-y-attack-paths/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-amenazas en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modelo-amenazas para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.

- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 140 Threat modeling con STRIDE y attack paths

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modelo-amenazas con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

141 — Cumplimiento, residencia, privacidad y evidencia

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: compliance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cumplimiento, residencia, privacidad y evidencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cumplimiento, residencia, privacidad y evidencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-controles con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cumplimiento	Define su papel en cumplimiento, residencia, privacidad y evidencia y cómo observarlo en un sistema real.
residencia	Define su papel en cumplimiento, residencia, privacidad y evidencia y cómo observarlo en un sistema real.
privacidad	Define su papel en cumplimiento, residencia, privacidad y evidencia y cómo observarlo en un sistema real.
evidencia	Define su papel en cumplimiento, residencia, privacidad y evidencia y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cumplimiento, residencia, privacidad y evidencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cumplimiento, residencia, privacidad y evidencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/141-cumplimiento-residencia-privacidad-y-evidencia/lab.py
```

El laboratorio selecciona el motor de práctica compliance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-controles en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control mapeado a evidencia y responsable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-controles para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 141 Cumplimiento, residencia, privacidad y evidencia

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-controles con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

142 — FinOps: showback, chargeback, budgets y anomalías

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar finops: showback, chargeback, budgets y anomalías dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar finops: showback, chargeback, budgets y anomalías con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-asignacion-costos con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
finops	Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real.
showback	Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real.
chargeback	Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real.
budgets	Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real.
anomalías	Define su papel en finops: showback, chargeback, budgets y anomalías y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: FinOps: showback, chargeback, budgets y anomalías"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar finops: showback, chargeback, budgets y anomalías. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/142-finops-showback-chargeback-budgets-y-anomalias/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa modelo-asignacion-costos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modelo-asignacion-costos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 142 FinOps: showback, chargeback, budgets y anomalías

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modelo-asignacion-costos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

143 — Optimización de costo, capacidad y sostenibilidad

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar optimización de costo, capacidad y sostenibilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar optimización de costo, capacidad y sostenibilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar backlog-optimizacion con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
optimización	Define su papel en optimización de costo, capacidad y sostenibilidad y cómo observarlo en un sistema real.
costo	Define su papel en optimización de costo, capacidad y sostenibilidad y cómo observarlo en un sistema real.
capacidad	Define su papel en optimización de costo, capacidad y sostenibilidad y cómo observarlo en un sistema real.
sostenibilidad	Define su papel en optimización de costo, capacidad y sostenibilidad y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Optimización de costo, capacidad y sostenibilidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar optimización de costo, capacidad y sostenibilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/143-optimizacion-de-costo-capacidad-y-sostenibilidad/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa backlog-optimizacion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye backlog-optimización para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 143 Optimización de costo, capacidad y sostenibilidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega backlog-optimización con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

144 — Proyecto: landing zone con guardrails

Parte: 11 — Seguridad, gobierno, cumplimiento y FinOps Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: landing zone con guardrails dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: landing zone con guardrails con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar landing-zone-gobernada con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: landing zone con guardrails y cómo observarlo en un sistema real.
landing	Define su papel en proyecto: landing zone con guardrails y cómo observarlo en un sistema real.
zone	Define su papel en proyecto: landing zone con guardrails y cómo observarlo en un sistema real.
guardrails	Define su papel en proyecto: landing zone con guardrails y cómo observarlo en un sistema real.

Modelo mental

Gobernar no significa aprobar cada cambio: significa codificar límites, evidencia y responsabilidades para que los equipos puedan avanzar con seguridad.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: landing zone con guardrails"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: landing zone con guardrails. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-11-security-governance-finops/144-proyecto-landing-zone-con-guardrails/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa landing-zone-gobernada en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye landing-zone-gobernada para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Practical Cloud Security — Chris Dotson.
- Cloud Security Handbook — Eyal Estrin.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 144 Proyecto: landing zone con guardrails

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega landing-zone-gobernada con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 12 — Arquitectura cloud-native y sistemas distribuidos

← Parte 11 · Índice completo · Parte 13 →

Nivel: avanzado-experto · Clases: 12 · Duración sugerida: 6–8 semanas

Tomar decisiones de arquitectura con compensaciones explícitas y evidencia.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
145	Requisitos, restricciones y atributos de calidad	architecture	4
146	Twelve-Factor App y configuración cloud-native	architecture	4
147	DDD, bounded contexts y ownership de datos	architecture	4
148	Monolito modular, microservicios y función	decision	4
149	CAP, PACELC y consistencia por operación	distributed	4
150	Replicación, particionado y consenso	distributed	4
151	Fallos parciales y patrones de resiliencia	reliability	4
152	Service discovery, malla y comunicación	network	4
153	Contratos API, compatibilidad y evolución	api	4
154	Multi-tenancy, aislamiento y noisy neighbor	architecture	4
155	Rendimiento, costo, seguridad y operabilidad	decision	4
156	Proyecto: revisión de arquitectura con ADR	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.

145 — Requisitos, restricciones y atributos de calidad

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar requisitos, restricciones y atributos de calidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar requisitos, restricciones y atributos de calidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar quality-attribute-scenarios con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
requisitos	Define su papel en requisitos, restricciones y atributos de calidad y cómo observarlo en un sistema real.
restricciones	Define su papel en requisitos, restricciones y atributos de calidad y cómo observarlo en un sistema real.
atributos	Define su papel en requisitos, restricciones y atributos de calidad y cómo observarlo en un sistema real.
calidad	Define su papel en requisitos, restricciones y atributos de calidad y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Requisitos, restricciones y atributos de calidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar requisitos, restricciones y atributos de calidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/145-requisitos-restricciones-y-atributos-de-calidad/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa quality-attribute-scenarios en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye quality-attribute-scenarios para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 145 Requisitos, restricciones y atributos de calidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega quality-attribute-scenarios con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

146 — Twelve-Factor App y configuración cloud-native

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar twelve-factor app y configuración cloud-native dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar twelve-factor app y configuración cloud-native con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar revision-twelve-factor con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
twelve	Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real.
factor	Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real.
app	Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real.
configuración	Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real.
cloud	Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real.
native	Define su papel en twelve-factor app y configuración cloud-native y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Twelve-Factor App y configuración cloud-native"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar twelve-factor app y configuración cloud-native. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/146-twelve-factor-app-y-configuracion-cloud-native/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa revision-twelve-factor en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye revision-twelve-factor para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 146 Twelve-Factor App y configuración cloud-native

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega revision-twelve-factor con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

147 — DDD, bounded contexts y ownership de datos

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ddd, bounded contexts y ownership de datos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ddd, bounded contexts y ownership de datos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar context-map con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ddd	Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real.
bounded	Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real.
contexts	Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real.
ownership	Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real.
datos	Define su papel en ddd, bounded contexts y ownership de datos y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: DDD, bounded contexts y ownership de datos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ddd, bounded contexts y ownership de datos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/147-ddd-bounded-contexts-y-ownership-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa context-map en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye context-map para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 147 DDD, bounded contexts y ownership de datos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega context-map con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

148 — Monolito modular, microservicios y función

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar monolito modular, microservicios y función dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar monolito modular, microservicios y función con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar adr-descomposicion con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
monolito	Define su papel en monolito modular, microservicios y función y cómo observarlo en un sistema real.
modular	Define su papel en monolito modular, microservicios y función y cómo observarlo en un sistema real.
microservicios	Define su papel en monolito modular, microservicios y función y cómo observarlo en un sistema real.
función	Define su papel en monolito modular, microservicios y función y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Monolito modular, microservicios y función"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar monolito modular, microservicios y función. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/148-monolito-modular-microservicios-y-funcion/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa adr-descomposicion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye adr-descomposicion para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 148 Monolito modular, microservicios y función

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega adr-descomposicion con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

149 — CAP, PACELC y consistencia por operación

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: distributed · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cap, pacelc y consistencia por operación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cap, pacelc y consistencia por operación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-consistencia con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cap	Define su papel en cap, pacelc y consistencia por operación y cómo observarlo en un sistema real.
pacelc	Define su papel en cap, pacelc y consistencia por operación y cómo observarlo en un sistema real.
consistencia	Define su papel en cap, pacelc y consistencia por operación y cómo observarlo en un sistema real.
operación	Define su papel en cap, pacelc y consistencia por operación y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: CAP, PACELC y consistencia por operación"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cap, pacelc y consistencia por operación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/149-cap-pacelc-y-consistencia-por-operacion/lab.py
```

El laboratorio selecciona el motor de práctica distributed y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa matriz-consistencia en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una traza de consistencia, reintento o fallo parcial. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-consistencia para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 149 CAP, PACELC y consistencia por operación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-consistencia con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

150 — Replicación, particionado y consenso

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: distributed · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar replicación, particionado y consenso dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar replicación, particionado y consenso con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-replicacion con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
replicación	Define su papel en replicación, particionado y consenso y cómo observarlo en un sistema real.
particionado	Define su papel en replicación, particionado y consenso y cómo observarlo en un sistema real.
consenso	Define su papel en replicación, particionado y consenso y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Replicación, particionado y consenso"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar replicación, particionado y consenso. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/150-replicacion-particionado-y-consenso/lab.py
```

El laboratorio selecciona el motor de práctica distributed y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa modelo-replicacion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una traza de consistencia, reintento o fallo parcial. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modelo-replicación para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 150 Replicación, particionado y consenso

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modelo-replicación con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

151 — Fallos parciales y patrones de resiliencia

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar fallos parciales y patrones de resiliencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar fallos parciales y patrones de resiliencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mapa-fallos con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
fallos	Define su papel en fallos parciales y patrones de resiliencia y cómo observarlo en un sistema real.
parciales	Define su papel en fallos parciales y patrones de resiliencia y cómo observarlo en un sistema real.
patrones	Define su papel en fallos parciales y patrones de resiliencia y cómo observarlo en un sistema real.
resiliencia	Define su papel en fallos parciales y patrones de resiliencia y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Fallos parciales y patrones de resiliencia"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar fallos parciales y patrones de resiliencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/151-fallos-parciales-y-patrones-de-resiliencia/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mapa-fallos en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye mapa-fallos para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 151 Fallos parciales y patrones de resiliencia

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega mapa-fallos con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

152 — Service discovery, malla y comunicación

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar service discovery, malla y comunicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar service discovery, malla y comunicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar topología-servicios con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
service	Define su papel en service discovery, malla y comunicación y cómo observarlo en un sistema real.
discovery	Define su papel en service discovery, malla y comunicación y cómo observarlo en un sistema real.
malla	Define su papel en service discovery, malla y comunicación y cómo observarlo en un sistema real.
comunicación	Define su papel en service discovery, malla y comunicación y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Service discovery, malla y comunicación"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar service discovery, malla y comunicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/152-service-discovery-malla-y-comunicacion/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa topología-servicios en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye topología-servicios para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 152 Service discovery, malla y comunicación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega topología-servicios con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

153 — Contratos API, compatibilidad y evolución

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: api · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar contratos api, compatibilidad y evolución dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar contratos api, compatibilidad y evolución con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contrato-versionado con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
contratos	Define su papel en contratos api, compatibilidad y evolución y cómo observarlo en un sistema real.
api	Define su papel en contratos api, compatibilidad y evolución y cómo observarlo en un sistema real.
compatibilidad	Define su papel en contratos api, compatibilidad y evolución y cómo observarlo en un sistema real.
evolución	Define su papel en contratos api, compatibilidad y evolución y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Contratos API, compatibilidad y evolución"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar contratos api, compatibilidad y evolución. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/153-contratos-api-compatibilidad-y-evolucion/lab.py
```

El laboratorio selecciona el motor de práctica api y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa contrato-versionado en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un contrato versionado con pruebas positivas y negativas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye contrato-versionado para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 153 Contratos API, compatibilidad y evolución

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega contrato-versionado con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

154 — Multi-tenancy, aislamiento y noisy neighbor

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar multi-tenancy, aislamiento y noisy neighbor dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar multi-tenancy, aislamiento y noisy neighbor con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-tenancy con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
multi	Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real.
tenancy	Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real.
aislamiento	Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real.
noisy	Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real.
neighbor	Define su papel en multi-tenancy, aislamiento y noisy neighbor y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Multi-tenancy, aislamiento y noisy neighbor"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar multi-tenancy, aislamiento y noisy neighbor. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/154-multi-tenancy-aislamiento-y-noisy-neighbor/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa modelo-tenancy en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modelo-tenancy para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 154 Multi-tenancy, aislamiento y noisy neighbor

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modelo-tenancy con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

155 — Rendimiento, costo, seguridad y operabilidad

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar rendimiento, costo, seguridad y operabilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar rendimiento, costo, seguridad y operabilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar tradeoff-analysis con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
rendimiento	Define su papel en rendimiento, costo, seguridad y operabilidad y cómo observarlo en un sistema real.
costo	Define su papel en rendimiento, costo, seguridad y operabilidad y cómo observarlo en un sistema real.
seguridad	Define su papel en rendimiento, costo, seguridad y operabilidad y cómo observarlo en un sistema real.
operabilidad	Define su papel en rendimiento, costo, seguridad y operabilidad y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Rendimiento, costo, seguridad y operabilidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar rendimiento, costo, seguridad y operabilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/155-rendimiento-costo-seguridad-y-operabilidad/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa tradeoff-analysis en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye tradeoff-analysis para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 155 Rendimiento, costo, seguridad y operabilidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega tradeoff-analysis con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

156 — Proyecto: revisión de arquitectura con ADR

Parte: 12 — Arquitectura cloud-native y sistemas distribuidos Nivel: avanzado-experto · Horas estimadas: 8
Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: revisión de arquitectura con adr dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: revisión de arquitectura con adr con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar architecture-review con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: revisión de arquitectura con adr y cómo observarlo en un sistema real.
revisión	Define su papel en proyecto: revisión de arquitectura con adr y cómo observarlo en un sistema real.
arquitectura	Define su papel en proyecto: revisión de arquitectura con adr y cómo observarlo en un sistema real.
adr	Define su papel en proyecto: revisión de arquitectura con adr y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones costosas de cambiar; su calidad se juzga contra escenarios concretos, no por la cantidad de servicios usados.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: revisión de arquitectura con ADR"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: revisión de arquitectura con adr. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-12-cloud-native-distributed-architecture/156-proyecto-revision-de-arquitectura-con-adr/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa architecture-review en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye architecture-review para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Native Patterns — Cornelia Davis.
- Building Microservices — Sam Newman.
- Release It! — Michael Nygard.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 156 Proyecto: revisión de arquitectura con ADR

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega architecture-review con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 13 — Multi-cloud, híbrido, migración y recuperación

← Parte 12 · Índice completo · Parte 14 →

Nivel: experto · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar continuidad y portabilidad sin ocultar complejidad, latencia ni egress.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
157	Motivaciones y anti-patrones de multi-cloud	decision	4
158	Portabilidad, capas de abstracción y lock-in	architecture	4
159	Federación de identidad entre nubes	iam	4
160	Conectividad, tránsito, DNS y service discovery	network	4
161	Replicación de datos, soberanía y costos de egress	data	4
162	Observabilidad y operación entre proveedores	observability	4
163	Terraform multi-provider y separación de estados	iac	4
164	Flotas Kubernetes y políticas comunes	kubernetes	4
165	Nube híbrida, edge y conectividad privada	hybrid	4
166	Backup, RTO, RPO y patrones de disaster recovery	reliability	4
167	Las 7R de migración y oleadas	migration	4
168	Proyecto: continuidad activa-pasiva entre nubes	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.

157 — Motivaciones y anti-patrones de multi-cloud

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar motivaciones y anti-patrones de multi-cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar motivaciones y anti-patrones de multi-cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar decision-multicloud con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
motivaciones	Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real.
anti	Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real.
patrones	Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real.
multi	Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real.
cloud	Define su papel en motivaciones y anti-patrones de multi-cloud y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Motivaciones y anti-patrones de multi-cloud"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar motivaciones y anti-patrones de multi-cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/157-motivaciones-y-anti-patrones-de-multi-cloud/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa decision-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye decision-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 157 Motivaciones y anti-patrones de multi-cloud

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega decision-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

158 — Portabilidad, capas de abstracción y lock-in

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar portabilidad, capas de abstracción y lock-in dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar portabilidad, capas de abstracción y lock-in con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar matriz-portabilidad con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
portabilidad	Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real.
capas	Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real.
abstracción	Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real.
lock	Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real.
in	Define su papel en portabilidad, capas de abstracción y lock-in y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Portabilidad, capas de abstracción y lock-in"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar portabilidad, capas de abstracción y lock-in. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/158-portabilidad-capas-de-abstraccion-y-lock-in/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa matriz-portabilidad en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye matriz-portabilidad para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 158 Portabilidad, capas de abstracción y lock-in

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega matriz-portabilidad con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

159 — Federación de identidad entre nubes

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar federación de identidad entre nubes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar federación de identidad entre nubes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar federacion-multicloud con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
federación	Define su papel en federación de identidad entre nubes y cómo observarlo en un sistema real.
identidad	Define su papel en federación de identidad entre nubes y cómo observarlo en un sistema real.
entre	Define su papel en federación de identidad entre nubes y cómo observarlo en un sistema real.
nubes	Define su papel en federación de identidad entre nubes y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Federación de identidad entre nubes"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar federación de identidad entre nubes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/159-federacion-de-identidad-entre-nubes/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa federacion-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye federación-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 159 Federación de identidad entre nubes

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega federacion-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

160 — Conectividad, tránsito, DNS y service discovery

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar conectividad, tránsito, dns y service discovery dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar conectividad, tránsito, dns y service discovery con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar red-multicloud con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
conectividad	Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real.
tránsito	Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real.
dns	Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real.
service	Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real.
discovery	Define su papel en conectividad, tránsito, dns y service discovery y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Conectividad, tránsito, DNS y service discovery"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar conectividad, tránsito, dns y service discovery. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/160-conectividad-transito-dns-y-service-discovery/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa red-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye red-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 160 Conectividad, tránsito, DNS y service discovery

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega red-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

161 — Replicación de datos, soberanía y costos de egress

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar replicación de datos, soberanía y costos de egress dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar replicación de datos, soberanía y costos de egress con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar estrategia-datos-multicloud con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
replicación	Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real.
datos	Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real.
soberanía	Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real.
costos	Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real.
egress	Define su papel en replicación de datos, soberanía y costos de egress y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Replicación de datos, soberanía y costos de egress"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar replicación de datos, soberanía y costos de egress. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/161-replicacion-de-datos-soberania-y-costos-de-egress/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa estrategia-datos-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye estrategia-datos-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 161 Replicación de datos, soberanía y costos de egress

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega estrategia-datos-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

162 — Observabilidad y operación entre proveedores

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar observabilidad y operación entre proveedores dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar observabilidad y operación entre proveedores con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar telemetría-multicloud con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
observabilidad	Define su papel en observabilidad y operación entre proveedores y cómo observarlo en un sistema real.
operación	Define su papel en observabilidad y operación entre proveedores y cómo observarlo en un sistema real.
entre	Define su papel en observabilidad y operación entre proveedores y cómo observarlo en un sistema real.
proveedores	Define su papel en observabilidad y operación entre proveedores y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Observabilidad y operación entre proveedores"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar observabilidad y operación entre proveedores. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/162-observabilidad-y-operacion-entre-proveedores/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa telemetría-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye telemetría-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 162 Observabilidad y operación entre proveedores

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega telemetria-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

163 — Terraform multi-provider y separación de estados

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar terraform multi-provider y separación de estados dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar terraform multi-provider y separación de estados con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar iac-multicloud con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
terraform	Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real.
multi	Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real.
provider	Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real.
separación	Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real.
estados	Define su papel en terraform multi-provider y separación de estados y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Terraform multi-provider y separación de estados"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar terraform multi-provider y separación de estados. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/163-terraform-multi-provider-y-separacion-de-estados/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa iac-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye iac-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 163 Terraform multi-provider y separación de estados

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega iac-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

164 — Flotas Kubernetes y políticas comunes

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: kubernetes · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar flotas kubernetes y políticas comunes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar flotas kubernetes y políticas comunes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar fleet-kubernetes con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
flotas	Define su papel en flotas kubernetes y políticas comunes y cómo observarlo en un sistema real.
kubernetes	Define su papel en flotas kubernetes y políticas comunes y cómo observarlo en un sistema real.
políticas	Define su papel en flotas kubernetes y políticas comunes y cómo observarlo en un sistema real.
comunes	Define su papel en flotas kubernetes y políticas comunes y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Flotas Kubernetes y políticas comunes"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar flotas kubernetes y políticas comunes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/164-flotas-kubernetes-y-politicas-comunes/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa fleet-kubernetes en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye fleet-kubernetes para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 164 Flotas Kubernetes y políticas comunes

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega fleet-kubernetes con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

165 — Nube híbrida, edge y conectividad privada

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: hybrid · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar nube híbrida, edge y conectividad privada dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar nube híbrida, edge y conectividad privada con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar arquitectura-híbrida con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
nube	Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real.
híbrida	Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real.
edge	Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real.
conectividad	Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real.
privada	Define su papel en nube híbrida, edge y conectividad privada y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Nube híbrida, edge y conectividad privada"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar nube híbrida, edge y conectividad privada. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/165-nube-hibrida-edge-y-conectividad-privada/lab.py
```

El laboratorio selecciona el motor de práctica hybrid y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa arquitectura-hibrida en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología híbrida con dependencia y modo degradado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye arquitectura-hibrida para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 165 Nube híbrida, edge y conectividad privada

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega arquitectura-híbrida con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

166 — Backup, RTO, RPO y patrones de disaster recovery

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar backup, rto, rpo y patrones de disaster recovery dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar backup, rto, rpo y patrones de disaster recovery con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plan-dr con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
backup	Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real.
rto	Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real.
rpo	Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real.
patrones	Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real.
disaster	Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real.
recovery	Define su papel en backup, rto, rpo y patrones de disaster recovery y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Backup, RTO, RPO y patrones de disaster recovery"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar backup, rto, rpo y patrones de disaster recovery. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/166-backup-rto-rpo-y-patrones-de-disaster-recovery/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plan-dr en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plan-dr para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 166 Backup, RTO, RPO y patrones de disaster recovery

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plan-dr con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

167 — Las 7R de migración y oleadas

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 4 Laboratorio: migration · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar las 7r de migración y oleadas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar las 7r de migración y oleadas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar roadmap-migracion con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
7r	Define su papel en las 7r de migración y oleadas y cómo observarlo en un sistema real.
migración	Define su papel en las 7r de migración y oleadas y cómo observarlo en un sistema real.
oleadas	Define su papel en las 7r de migración y oleadas y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Las 7R de migración y oleadas"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar las 7r de migración y oleadas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/167-las-7r-de-migracion-y-oleadas/lab.py
```

El laboratorio selecciona el motor de práctica migration y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa roadmap-migracion en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un inventario, dependencias, riesgo y oleadas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye roadmap-migración para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 167 Las 7R de migración y oleadas

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega roadmap-migración con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

168 — Proyecto: continuidad activa-pasiva entre nubes

Parte: 13 — Multi-cloud, híbrido, migración y recuperación Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: continuidad activa-pasiva entre nubes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: continuidad activa-pasiva entre nubes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar plataforma-multicloud con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real.
continuidad	Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real.
activa	Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real.
pasiva	Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real.
entre	Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real.
nubes	Define su papel en proyecto: continuidad activa-pasiva entre nubes y cómo observarlo en un sistema real.

Modelo mental

Multi-cloud es una decisión de negocio y riesgo; duplicar cada componente entre proveedores rara vez es la forma más confiable o económica de lograrla.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Proyecto: continuidad activa-pasiva entre nubes"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: continuidad activa-pasiva entre nubes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-13-multicloud-hybrid-disaster-recovery/168-proyecto-continuidad-activa-pasiva-entre-nubes/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa plataforma-multicloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye plataforma-multicloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Cloud Strategy — Gregor Hohpe.
- Seeking SRE — Blank-Edelman.
- Enterprise Integration Patterns — Hohpe y Woolf.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 168 Proyecto: continuidad activa-pasiva entre nubes

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega plataforma-multicloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 14 — Plataformas avanzadas, capstones y carrera

← Parte 13 · Índice completo · Parte 15 →

Nivel: experto-frontera · Clases: 12 · Duración sugerida: 6–8 semanas

Integrar tecnología, operación, gobierno y comunicación en evidencia profesional.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
169	Landing zones empresariales y vending de cuentas	governance	4
170	Gobierno federado y policy as code a escala	governance	4
171	Platform as a Product y roadmap de capacidades	platform	4
172	Modelo operativo FinOps y economía unitaria	finops	4
173	Madurez SRE y confiabilidad organizacional	sre	4
174	Arquitectura de seguridad cloud empresarial	security	4
175	Workloads de IA, GPU, datos y MLOps multi-cloud	ai	4
176	Edge, IoT y procesamiento desconectado	edge	4
177	Soberanía digital y confidential computing	security	4
178	Capstone: descubrimiento y diseño	capstone	8
179	Capstone: implementación y operación	capstone	8
180	Capstone: defensa, portafolio y plan profesional	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.

169 — Landing zones empresariales y vending de cuentas

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar landing zones empresariales y vending de cuentas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar landing zones empresariales y vending de cuentas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar landing-zone-empresarial con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
landing	Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real.
zones	Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real.
empresariales	Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real.
vending	Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real.
cuentas	Define su papel en landing zones empresariales y vending de cuentas y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Landing zones empresariales y vending de cuentas"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar landing zones empresariales y vending de cuentas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/169-landing-zones-empresariales-y-vending-de-cuentas/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa landing-zone-empresarial en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye landing-zone-empresarial para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 169 Landing zones empresariales y vending de cuentas

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega landing-zone-empresarial con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

170 — Gobierno federado y policy as code a escala

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar gobierno federado y policy as code a escala dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar gobierno federado y policy as code a escala con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar modelo-gobierno con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
gobierno	Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real.
federado	Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real.
policy	Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real.
as	Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real.
code	Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real.
escala	Define su papel en gobierno federado y policy as code a escala y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Gobierno federado y policy as code a escala"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar gobierno federado y policy as code a escala. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/170-gobierno-federado-y-policy-as-code-a-escala/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa modelo-gobierno en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye modelo-gobierno para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 170 Gobierno federado y policy as code a escala

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega modelo-gobierno con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

171 — Platform as a Product y roadmap de capacidades

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: platform · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar platform as a product y roadmap de capacidades dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar platform as a product y roadmap de capacidades con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar roadmap-plataforma con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
platform	Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real.
as	Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real.
product	Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real.
roadmap	Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real.
capacidades	Define su papel en platform as a product y roadmap de capacidades y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Platform as a Product y roadmap de capacidades"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar platform as a product y roadmap de capacidades. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/171-platform-as-a-product-y-roadmap-de-capacidades/lab.py
```

El laboratorio selecciona el motor de práctica platform y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa roadmap-plataforma en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una capacidad autoservicio con contrato y golden path. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye roadmap-plataforma para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 171 Platform as a Product y roadmap de capacidades

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega roadmap-plataforma con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

172 — Modelo operativo FinOps y economía unitaria

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: finops · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar modelo operativo finops y economía unitaria dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelo operativo finops y economía unitaria con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar operating-model-finops con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
modelo	Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real.
operativo	Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real.
finops	Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real.
economía	Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real.
unitaria	Define su papel en modelo operativo finops y economía unitaria y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Modelo operativo FinOps y economía unitaria"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar modelo operativo finops y economía unitaria. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/172-modelo-operativo-finops-y-economia-unitaria/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa operating-model-finops en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye operating-model-finops para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 172 Modelo operativo FinOps y economía unitaria

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega operating-model-finops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

173 — Madurez SRE y confiabilidad organizacional

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: sre · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar madurez sre y confiabilidad organizacional dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar madurez sre y confiabilidad organizacional con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar assessment-sre con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
madurez	Define su papel en madurez sre y confiabilidad organizacional y cómo observarlo en un sistema real.
sre	Define su papel en madurez sre y confiabilidad organizacional y cómo observarlo en un sistema real.
confiabilidad	Define su papel en madurez sre y confiabilidad organizacional y cómo observarlo en un sistema real.
organizacional	Define su papel en madurez sre y confiabilidad organizacional y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Madurez SRE y confiabilidad organizacional"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar madurez sre y confiabilidad organizacional. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/173-madurez-sre-y-confiabilidad-organizacional/lab.py
```

El laboratorio selecciona el motor de práctica sre y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa assessment-sre en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un SLO con presupuesto de error y política de acción. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye assessment-sre para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 173 Madurez SRE y confiabilidad organizacional

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega assessment-sre con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

174 — Arquitectura de seguridad cloud empresarial

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar arquitectura de seguridad cloud empresarial dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar arquitectura de seguridad cloud empresarial con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar security-reference-architecture con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
arquitectura	Define su papel en arquitectura de seguridad cloud empresarial y cómo observarlo en un sistema real.
seguridad	Define su papel en arquitectura de seguridad cloud empresarial y cómo observarlo en un sistema real.
cloud	Define su papel en arquitectura de seguridad cloud empresarial y cómo observarlo en un sistema real.
empresarial	Define su papel en arquitectura de seguridad cloud empresarial y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Arquitectura de seguridad cloud empresarial"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar arquitectura de seguridad cloud empresarial. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/174-arquitectura-de-seguridad-cloud-empresarial/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa security-reference-architecture en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye security-reference-architecture para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 174 Arquitectura de seguridad cloud empresarial

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega security-reference-architecture con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

175 — Workloads de IA, GPU, datos y MLOps multi-cloud

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: ai · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar workloads de ia, gpu, datos y mlops multi-cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar workloads de ia, gpu, datos y mlops multi-cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar arquitectura-ia-cloud con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
workloads	Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real.
ia	Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real.
gpu	Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real.
datos	Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real.
mlops	Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real.
multi	Define su papel en workloads de ia, gpu, datos y mlops multi-cloud y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Workloads de IA, GPU, datos y MLOps multi-cloud"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar workloads de ia, gpu, datos y mlops multi-cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/175-workloads-de-ia-gpu-datos-y-mlops-multi-cloud/lab.py
```

El laboratorio selecciona el motor de práctica ai y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa arquitectura-ia-cloud en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una arquitectura de IA con datos, cómputo, costo y seguridad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye arquitectura-ia-cloud para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 175 Workloads de IA, GPU, datos y MLOps multi-cloud

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega arquitectura-ia-cloud con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

176 — Edge, IoT y procesamiento desconectado

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: edge · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar edge, iot y procesamiento desconectado dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar edge, iot y procesamiento desconectado con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar arquitectura-edge con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
edge	Define su papel en edge, iot y procesamiento desconectado y cómo observarlo en un sistema real.
iot	Define su papel en edge, iot y procesamiento desconectado y cómo observarlo en un sistema real.
procesamiento	Define su papel en edge, iot y procesamiento desconectado y cómo observarlo en un sistema real.
desconectado	Define su papel en edge, iot y procesamiento desconectado y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Edge, IoT y procesamiento desconectado"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar edge, iot y procesamiento desconectado. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/176-edge-iot-y-procesamiento-desconectado/lab.py
```

El laboratorio selecciona el motor de práctica edge y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa arquitectura-edge en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo edge que tolera desconexión y sincroniza estado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye arquitectura-edge para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 176 Edge, IoT y procesamiento desconectado

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega arquitectura-edge con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

177 — Soberanía digital y confidential computing

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar soberanía digital y confidential computing dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar soberanía digital y confidential computing con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar decision-soberania con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
soberanía	Define su papel en soberanía digital y confidential computing y cómo observarlo en un sistema real.
digital	Define su papel en soberanía digital y confidential computing y cómo observarlo en un sistema real.
confidential	Define su papel en soberanía digital y confidential computing y cómo observarlo en un sistema real.
computing	Define su papel en soberanía digital y confidential computing y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Soberanía digital y confidential computing"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar soberanía digital y confidential computing. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/177-soberania-digital-y-confidential-computing/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa decision-soberania en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye decision-soberania para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 177 Soberanía digital y confidential computing

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega decision-soberania con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

178 — Capstone: descubrimiento y diseño

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone: descubrimiento y diseño dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone: descubrimiento y diseño con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar dossier-diseno-final con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone: descubrimiento y diseño y cómo observarlo en un sistema real.
descubrimiento	Define su papel en capstone: descubrimiento y diseño y cómo observarlo en un sistema real.
diseño	Define su papel en capstone: descubrimiento y diseño y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone: descubrimiento y diseño"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone: descubrimiento y diseño. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/178-capstone-descubrimiento-y-diseno/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa dossier-diseno-final en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye dossier-diseno-final para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 178 Capstone: descubrimiento y diseño

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega dossier-diseño-final con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

179 — Capstone: implementación y operación

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone: implementación y operación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone: implementación y operación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar evidencia-implementacion-final con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone: implementación y operación y cómo observarlo en un sistema real.
implementación	Define su papel en capstone: implementación y operación y cómo observarlo en un sistema real.
operación	Define su papel en capstone: implementación y operación y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone: implementación y operación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone: implementación y operación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/179-capstone-implementacion-y-operacion/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa evidencia-implementacion-final en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye evidencia-implementacion-final para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 179 Capstone: implementación y operación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega evidencia-implementacion-final con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

180 — Capstone: defensa, portafolio y plan profesional

Parte: 14 — Plataformas avanzadas, capstones y carrera Nivel: experto-frontera · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone: defensa, portafolio y plan profesional dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone: defensa, portafolio y plan profesional con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar defensa-final con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real.
defensa	Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real.
portafolio	Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real.
plan	Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real.
profesional	Define su papel en capstone: defensa, portafolio y plan profesional y cómo observarlo en un sistema real.

Modelo mental

El nivel experto no consiste en conocer más productos, sino en formular mejores preguntas, validar supuestos y sostener decisiones frente a costo, riesgo y operación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone: defensa, portafolio y plan profesional"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone: defensa, portafolio y plan profesional. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-14-advanced-platform-capstones-career/180-capstone-defensa-portafolio-y-plan-profesional/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa defensa-final en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye defensa-final para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Team Topologies — Skelton y Pais.
- Accelerate — Forsgren, Humble y Kim.
- Fundamentals of Software Architecture — Richards y Ford.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 180 Capstone: defensa, portafolio y plan profesional

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega defensa-final con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 15 — Arquitectura de sistemas e ingeniería de requisitos

← Parte 14 · Índice completo · Parte 16 →

Nivel: intermedio-avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Pasar de requisitos ambiguos a arquitecturas comprobables antes de elegir servicios cloud.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
181	Requisitos funcionales, restricciones y atributos de calidad	architecture	4
182	Contexto, contenedores, componentes y código con C4	architecture	4
183	Acoplamiento, cohesión, modularidad y fronteras	architecture	4
184	Arquitectura monolítica, modular y de microservicios	decision	4
185	Disponibilidad, confiabilidad y análisis de puntos de fallo	reliability	4
186	Capacidad, latencia, throughput y teoría de colas	performance	4
187	Consistencia, particiones, relojes y consenso	distributed	4
188	Contratos de API, eventos y compatibilidad evolutiva	api	4
189	Modelado de amenazas y arquitectura de confianza cero	security	4
190	ADRs, fitness functions y gobierno de decisiones	architecture	4
191	Architecture review y comunicación con stakeholders	governance	4
192	Proyecto: arquitectura completa de CloudShop	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.

181 — Requisitos funcionales, restricciones y atributos de calidad

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar requisitos funcionales, restricciones y atributos de calidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar requisitos funcionales, restricciones y atributos de calidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar quality-attribute-scenarios con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
requisitos	Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real.
funcionales	Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real.
restricciones	Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real.
atributos	Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real.
calidad	Define su papel en requisitos funcionales, restricciones y atributos de calidad y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Requisitos funcionales, restricciones y atributos de calidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar requisitos funcionales, restricciones y atributos de calidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/181-requisitos-funcionales-restricciones-y-atributos-de-calidad/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa quality-attribute-scenarios en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye quality-attribute-scenarios para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 181 Requisitos funcionales, restricciones y atributos de calidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega quality-attribute-scenarios con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

182 — Contexto, contenedores, componentes y código con C4

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar contexto, contenedores, componentes y código con c4 dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar contexto, contenedores, componentes y código con c4 con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar c4-model con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
contexto	Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real.
contenedores	Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real.
componentes	Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real.
código	Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real.
c4	Define su papel en contexto, contenedores, componentes y código con c4 y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Contexto, contenedores, componentes y código con C4"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar contexto, contenedores, componentes y código con c4. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/182-contexto-contenedores-componentes-y-codigo-con-c4/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa c4-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye c4-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 182 Contexto, contenedores, componentes y código con C4

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega c4-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

183 — Acoplamiento, cohesión, modularidad y fronteras

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar acoplamiento, cohesión, modularidad y fronteras dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar acoplamiento, cohesión, modularidad y fronteras con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar module-map con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
acoplamiento	Define su papel en acoplamiento, cohesión, modularidad y fronteras y cómo observarlo en un sistema real.
cohesión	Define su papel en acoplamiento, cohesión, modularidad y fronteras y cómo observarlo en un sistema real.
modularidad	Define su papel en acoplamiento, cohesión, modularidad y fronteras y cómo observarlo en un sistema real.
fronteras	Define su papel en acoplamiento, cohesión, modularidad y fronteras y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Acoplamiento, cohesión, modularidad y fronteras"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar acoplamiento, cohesión, modularidad y fronteras. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/183-acoplamiento-cohesion-modularidad-y-fronteras/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa module-map en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye module-map para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 183 Acoplamiento, cohesión, modularidad y fronteras

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega module-map con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

184 — Arquitectura monolítica, modular y de microservicios

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar arquitectura monolítica, modular y de microservicios dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar arquitectura monolítica, modular y de microservicios con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar architecture-style-adr con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
arquitectura	Define su papel en arquitectura monolítica, modular y de microservicios y cómo observarlo en un sistema real.
monolítica	Define su papel en arquitectura monolítica, modular y de microservicios y cómo observarlo en un sistema real.
modular	Define su papel en arquitectura monolítica, modular y de microservicios y cómo observarlo en un sistema real.
microservicios	Define su papel en arquitectura monolítica, modular y de microservicios y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Arquitectura monolítica, modular y de microservicios"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar arquitectura monolítica, modular y de microservicios. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/184-arquitectura-monolitica-modular-y-de-microservicios/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa architecture-style-adr en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye architecture-style-adr para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 184 Arquitectura monolítica, modular y de microservicios

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega architecture-style-adr con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

185 — Disponibilidad, confiabilidad y análisis de puntos de fallo

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar disponibilidad, confiabilidad y análisis de puntos de fallo dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar disponibilidad, confiabilidad y análisis de puntos de fallo con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar failure-mode-analysis con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
disponibilidad	Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real.
confiabilidad	Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real.
análisis	Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real.
puntos	Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real.
fallo	Define su papel en disponibilidad, confiabilidad y análisis de puntos de fallo y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Disponibilidad, confiabilidad y análisis de puntos de fallo"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar disponibilidad, confiabilidad y análisis de puntos de fallo. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/185-disponibilidad-confiabilidad-y-analisis-de-puntos-de-fallo/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa failure-mode-analysis en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye failure-mode-analysis para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 185 Disponibilidad, confiabilidad y análisis de puntos de fallo

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega failure-mode-analysis con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

186 — Capacidad, latencia, throughput y teoría de colas

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: performance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capacidad, latencia, throughput y teoría de colas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capacidad, latencia, throughput y teoría de colas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar capacity-model con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capacidad	Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real.
latencia	Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real.
throughput	Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real.
teoría	Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real.
colas	Define su papel en capacidad, latencia, throughput y teoría de colas y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capacidad, latencia, throughput y teoría de colas"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capacidad, latencia, throughput y teoría de colas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/186-capacidad-latencia-throughput-y-teoria-de-colas/lab.py
```

El laboratorio selecciona el motor de práctica performance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa capacity-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una prueba de carga con baseline y cuello de botella. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye capacity-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 186 Capacidad, latencia, throughput y teoría de colas

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega capacity-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

187 — Consistencia, particiones, relojes y consenso

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: distributed · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar consistencia, particiones, relojes y consenso dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar consistencia, particiones, relojes y consenso con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar consistency-model con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
consistencia	Define su papel en consistencia, particiones, relojes y consenso y cómo observarlo en un sistema real.
particiones	Define su papel en consistencia, particiones, relojes y consenso y cómo observarlo en un sistema real.
relojes	Define su papel en consistencia, particiones, relojes y consenso y cómo observarlo en un sistema real.
consenso	Define su papel en consistencia, particiones, relojes y consenso y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Consistencia, particiones, relojes y consenso"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar consistencia, particiones, relojes y consenso. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/187-consistencia-particiones-relojes-y-consenso/lab.py
```

El laboratorio selecciona el motor de práctica distributed y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa consistency-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una traza de consistencia, reintento o fallo parcial. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye consistency-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 187 Consistencia, particiones, relojes y consenso

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega consistency-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

188 — Contratos de API, eventos y compatibilidad evolutiva

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: api · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar contratos de api, eventos y compatibilidad evolutiva dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar contratos de api, eventos y compatibilidad evolutiva con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar contract-evolution con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
contratos	Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real.
api	Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real.
eventos	Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real.
compatibilidad	Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real.
evolutiva	Define su papel en contratos de api, eventos y compatibilidad evolutiva y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Contratos de API, eventos y compatibilidad evolutiva"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar contratos de api, eventos y compatibilidad evolutiva. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/188-contratos-de-api-eventos-y-compatibilidad-evolutiva/lab.py
```

El laboratorio selecciona el motor de práctica api y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa contract-evolution en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un contrato versionado con pruebas positivas y negativas. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye contract-evolution para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 188 Contratos de API, eventos y compatibilidad evolutiva

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega contract-evolution con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

189 — Modelado de amenazas y arquitectura de confianza cero

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar modelado de amenazas y arquitectura de confianza cero dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelado de amenazas y arquitectura de confianza cero con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar system-threat-model con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
modelado	Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real.
amenazas	Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real.
arquitectura	Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real.
confianza	Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real.
cero	Define su papel en modelado de amenazas y arquitectura de confianza cero y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Modelado de amenazas y arquitectura de confianza cero"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar modelado de amenazas y arquitectura de confianza cero. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/189-modelado-de-amenazas-y-arquitectura-de-confianza-cero/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa system-threat-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye system-threat-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 189 Modelado de amenazas y arquitectura de confianza cero

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega system-threat-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

190 — ADRs, fitness functions y gobierno de decisiones

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar adrs, fitness functions y gobierno de decisiones dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar adrs, fitness functions y gobierno de decisiones con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar adr-library con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
adrs	Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real.
fitness	Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real.
functions	Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real.
gobierno	Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real.
decisiones	Define su papel en adrs, fitness functions y gobierno de decisiones y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: ADRs, fitness functions y gobierno de decisiones"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar adrs, fitness functions y gobierno de decisiones. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/190-adrs-fitness-functions-y-gobierno-de-decisiones/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa adr-library en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye adr-library para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 190 ADRs, fitness functions y gobierno de decisiones

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega adr-library con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

191 — Architecture review y comunicación con stakeholders

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar architecture review y comunicación con stakeholders dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar architecture review y comunicación con stakeholders con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar architecture-review con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
architecture	Define su papel en architecture review y comunicación con stakeholders y cómo observarlo en un sistema real.
review	Define su papel en architecture review y comunicación con stakeholders y cómo observarlo en un sistema real.
comunicación	Define su papel en architecture review y comunicación con stakeholders y cómo observarlo en un sistema real.
stakeholders	Define su papel en architecture review y comunicación con stakeholders y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Architecture review y comunicación con stakeholders"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar architecture review y comunicación con stakeholders. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/191-architecture-review-y-comunicacion-con-stakeholders/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa architecture-review en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye architecture-review para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 191 Architecture review y comunicación con stakeholders

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega architecture-review con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

192 — Proyecto: arquitectura completa de CloudShop

Parte: 15 — Arquitectura de sistemas e ingeniería de requisitos Nivel: intermedio-avanzado · Horas estimadas: 8
Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: arquitectura completa de cloudshop dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: arquitectura completa de cloudshop con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-system-design con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: arquitectura completa de cloudshop y cómo observarlo en un sistema real.
arquitectura	Define su papel en proyecto: arquitectura completa de cloudshop y cómo observarlo en un sistema real.
completa	Define su papel en proyecto: arquitectura completa de cloudshop y cómo observarlo en un sistema real.
cloudshop	Define su papel en proyecto: arquitectura completa de cloudshop y cómo observarlo en un sistema real.

Modelo mental

Una arquitectura es un conjunto de decisiones sobre estructuras, interfaces y atributos de calidad, respaldadas por escenarios y evidencia.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: arquitectura completa de CloudShop"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E["¿Cumple seguridad, SLO y costo?"]
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: arquitectura completa de cloudshop. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-15-systems-architecture-engineering/192-proyecto-arquitectura-completa-de-cloudshop/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-system-design en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloudshop-system-design para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Software Architecture in Practice — Bass, Clements y Kazman.
- Fundamentals of Software Architecture — Richards y Ford.
- Designing Data-Intensive Applications — Martin Kleppmann.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 192 Proyecto: arquitectura completa de CloudShop

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloudshop-system-design con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 16 — Redes cloud avanzadas, conectividad híbrida y edge

← Parte 15 · Índice completo · Parte 17 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar conectividad segura y observable entre regiones, proveedores, centros de datos y edge.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
193	CIDR, subnetting y planificación IP a escala	network	4
194	Routing, BGP, tránsito y propagación de rutas	network	4
195	DNS autoritativo, recursivo, split-horizon y DNSSEC	network	4
196	Balanceo L4/L7, proxies, TLS y gestión de certificados	network	4
197	CDN, caché, origin shielding y edge compute	performance	4
198	VPN, Direct Connect, ExpressRoute e Interconnect	network	4
199	Transit Gateway, Virtual WAN y Network Connectivity Center	network	4
200	Private endpoints, service networking y egress control	security	4
201	Service mesh, mTLS y gestión de tráfico este-oeste	network	4
202	eBPF, flow logs, packet capture y diagnóstico	observability	4
203	SD-WAN, 5G, IoT y operación desconectada	architecture	4
204	Proyecto: red multi-región y multi-cloud	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.

193 — CIDR, subnetting y planificación IP a escala

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cidr, subnetting y planificación ip a escala dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cidr, subnetting y planificación ip a escala con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ipam-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cidr	Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real.
subnetting	Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real.
planificación	Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real.
ip	Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real.
escala	Define su papel en cidr, subnetting y planificación ip a escala y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: CIDR, subnetting y planificación IP a escala"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cidr, subnetting y planificación ip a escala. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/193-cidr-subnetting-y-planificacion-ip-a-escala/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa ipam-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye ipam-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 193 CIDR, subnetting y planificación IP a escala

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega ipam-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

194 — Routing, BGP, tránsito y propagación de rutas

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar routing, bgp, tránsito y propagación de rutas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar routing, bgp, tránsito y propagación de rutas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar routing-lab con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
routing	Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real.
bgp	Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real.
tránsito	Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real.
propagación	Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real.
rutas	Define su papel en routing, bgp, tránsito y propagación de rutas y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Routing, BGP, tránsito y propagación de rutas"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar routing, bgp, tránsito y propagación de rutas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/194-routing-bgp-transito-y-propagacion-de-rutas/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa routing-lab en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye routing-lab para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 194 Routing, BGP, tránsito y propagación de rutas

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega routing-lab con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

195 — DNS autoritativo, recursivo, split-horizon y DNSSEC

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar dns autoritativo, recursivo, split-horizon y dnssec dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar dns autoritativo, recursivo, split-horizon y dnssec con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar dns-design con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
dns	Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real.
autoritativo	Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real.
recursivo	Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real.
split	Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real.
horizon	Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real.
dnssec	Define su papel en dns autoritativo, recursivo, split-horizon y dnssec y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: DNS autoritativo, recursivo, split-horizon y DNSSEC"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar dns autoritativo, recursivo, split-horizon y dnssec. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/195-dns-autoritativo-recursivo-split-horizon-y-dnssec/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa dns-design en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye dns-design para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 195 DNS autoritativo, recursivo, split-horizon y DNSSEC

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega dns-design con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

196 — Balanceo L4/L7, proxies, TLS y gestión de certificados

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar balanceo L4/L7, proxies, TLS y gestión de certificados dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar balanceo L4/L7, proxies, TLS y gestión de certificados con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar traffic-entry con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
balanceo	Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real.
L4	Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real.
L7	Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real.
proxies	Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real.
TLS	Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real.
gestión	Define su papel en balanceo L4/L7, proxies, TLS y gestión de certificados y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Balanceo L4/L7, proxies, TLS y gestión de certificados"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar balanceo l4/l7, proxies, tls y gestión de certificados. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/196-balanceo-l4-l7-proxies-tls-y-gestion-de-certificados/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa traffic-entry en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye traffic-entry para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 196 Balanceo L4/L7, proxies, TLS y gestión de certificados

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega traffic-entry con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

197 — CDN, caché, origin shielding y edge compute

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: performance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cdn, caché, origin shielding y edge compute dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cdn, caché, origin shielding y edge compute con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cdn-experiment con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cdn	Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real.
caché	Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real.
origin	Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real.
shielding	Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real.
edge	Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real.
compute	Define su papel en cdn, caché, origin shielding y edge compute y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: CDN, caché, origin shielding y edge compute"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cdn, caché, origin shielding y edge compute. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/197-cdn-cache-origin-shielding-y-edge-compute/lab.py
```

El laboratorio selecciona el motor de práctica performance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa `cdn-experiment` en `evidence/` usando la plantilla indicada.
5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una prueba de carga con baseline y cuello de botella. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `cdn-experiment` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 197 CDN, caché, origin shielding y edge compute

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cdn-experiment con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

198 — VPN, Direct Connect, ExpressRoute e Interconnect

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar vpn, direct connect, expressroute e interconnect dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar vpn, direct connect, expressroute e interconnect con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar hybrid-connectivity con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
vpn	Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real.
direct	Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real.
connect	Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real.
expressroute	Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real.
e	Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real.
interconnect	Define su papel en vpn, direct connect, expressroute e interconnect y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: VPN, Direct Connect, ExpressRoute e Interconnect"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar vpn, direct connect, expressroute e interconnect. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/198-vpn-direct-connect-expressroute-e-interconnect/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa hybrid-connectivity en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye hybrid-connectivity para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 198 VPN, Direct Connect, ExpressRoute e Interconnect

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega hybrid-connectivity con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

199 — Transit Gateway, Virtual WAN y Network Connectivity Center

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar transit gateway, virtual wan y network connectivity center dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar transit gateway, virtual wan y network connectivity center con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar transit-comparison con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
transit	Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real.
gateway	Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real.
virtual	Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real.
wan	Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real.
network	Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real.
connectivity	Define su papel en transit gateway, virtual wan y network connectivity center y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Transit Gateway, Virtual WAN y Network Connectivity Center"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar transit gateway, virtual wan y network connectivity center. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/199-transit-gateway-virtual-wan-y-network-connectivity-center/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa transit-comparison en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye transit-comparison para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 199 Transit Gateway, Virtual WAN y Network Connectivity Center

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega transit-comparison con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

200 — Private endpoints, service networking y egress control

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar private endpoints, service networking y egress control dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar private endpoints, service networking y egress control con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar private-connectivity con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
private	Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real.
endpoints	Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real.
service	Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real.
networking	Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real.
egress	Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real.
control	Define su papel en private endpoints, service networking y egress control y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Private endpoints, service networking y egress control"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar private endpoints, service networking y egress control. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/200-private-endpoints-service-networking-y-egress-control/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa private-connectivity en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye private-connectivity para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 200 Private endpoints, service networking y egress control

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega private-connectivity con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

201 — Service mesh, mTLS y gestión de tráfico este-oeste

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar service mesh, mTLS y gestión de tráfico este-oeste dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar service mesh, mTLS y gestión de tráfico este-oeste con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar service-mesh con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
service	Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real.
mesh	Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real.
mTLS	Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real.
gestión	Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real.
tráfico	Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real.
este	Define su papel en service mesh, mTLS y gestión de tráfico este-oeste y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Service mesh, mTLS y gestión de tráfico este-oeste"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar service mesh, mTLS y gestión de tráfico este-oeste. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/201-service-mesh-mtls-y-gestion-de-traffic-este-oeste/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa service-mesh en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye service-mesh para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 201 Service mesh, mTLS y gestión de tráfico este-oeste

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega service-mesh con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

202 — eBPF, flow logs, packet capture y diagnóstico

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ebpf, flow logs, packet capture y diagnóstico dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ebpf, flow logs, packet capture y diagnóstico con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar network-evidence con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ebpf	Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real.
flow	Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real.
logs	Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real.
packet	Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real.
capture	Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real.
diagnóstico	Define su papel en ebpf, flow logs, packet capture y diagnóstico y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: eBPF, flow logs, packet capture y diagnóstico"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ebpf, flow logs, packet capture y diagnóstico. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/202-ebpf-flow-logs-packet-capture-y-diagnostico/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa network-evidence en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye network-evidence para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 202 eBPF, flow logs, packet capture y diagnóstico

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega network-evidence con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

203 — SD-WAN, 5G, IoT y operación desconectada

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar sd-wan, 5g, iot y operación desconectada dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sd-wan, 5g, iot y operación desconectada con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar edge-topology con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
sd	Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real.
wan	Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real.
5g	Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real.
iot	Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real.
operación	Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real.
desconectada	Define su papel en sd-wan, 5g, iot y operación desconectada y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: SD-WAN, 5G, IoT y operación desconectada"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sd-wan, 5g, iot y operación desconectada. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/203-sd-wan-5g-iot-y-operacion-desconectada/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa edge-topology en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye edge-topology para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 203 SD-WAN, 5G, IoT y operación desconectada

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega edge-topology con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

204 — Proyecto: red multi-región y multi-cloud

Parte: 16 — Redes cloud avanzadas, conectividad híbrida y edge Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: red multi-región y multi-cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: red multi-región y multi-cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar multicloud-network con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real.
red	Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real.
multi	Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real.
región	Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real.
multi	Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real.
cloud	Define su papel en proyecto: red multi-región y multi-cloud y cómo observarlo en un sistema real.

Modelo mental

La red cloud es un sistema distribuido de rutas, identidades y políticas; cada salto añade latencia, costo y una frontera de fallo.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: red multi-región y multi-cloud"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: red multi-región y multi-cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-16-advanced-cloud-networking-edge/204-proyecto-red-multi-region-y-multi-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa multicloud-network en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye multicloud-network para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Computer Networking: A Top-Down Approach — Kurose y Ross.
- Cloud Native Data Center Networking — Dinesh Dutt.
- AWS Advanced Networking Study Guide — S. Mahalingam.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 204 Proyecto: red multi-región y multi-cloud

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega multicloud-network con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 17 — AWS: arquitectura, automatización y operación en producción

← Parte 16 · Índice completo · Parte 18 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Integrar los casos AWS existentes en una ruta de producción con seguridad, costo, evidencia y destrucción.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
205	Hosting progresivo con Amplify, S3 y CloudFront	delivery	4
206	OIDC de GitHub y GitLab hacia AWS sin secretos	iam	4
207	SAM, Lambda, API Gateway y despliegue serverless	serverless	4
208	DynamoDB por patrones de acceso y single-table design	data	4
209	Cognito, JWT authorizers, WAF y defensa en profundidad	security	4
210	EventBridge, SQS, DLQ, replay e idempotencia	messaging	4
211	CloudWatch, X-Ray y observabilidad como código	observability	4
212	ECR, ECS Fargate, ALB y autoscaling	container	4
213	EKS, IRSA, GitOps y operación de clúster	kubernetes	4
214	Budgets, Cost Explorer, etiquetado y FinOps automatizado	finops	4
215	Multi-región, Route 53, failover y game day	reliability	4
216	Proyecto: CloudShop productivo en AWS	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.

205 — Hosting progresivo con Amplify, S3 y CloudFront

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: delivery · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar hosting progresivo con amplify, s3 y cloudfront dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar hosting progresivo con amplify, s3 y cloudfront con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-static-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
hosting	Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real.
progresivo	Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real.
amplify	Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real.
s3	Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real.
cloudfront	Define su papel en hosting progresivo con amplify, s3 y cloudfront y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Hosting progresivo con Amplify, S3 y CloudFront"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SL0 y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar hosting progresivo con amplify, s3 y cloudfront. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/205-hosting-progresivo-con-amplify-s3-y-cloudfront/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-static-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-static-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 205 Hosting progresivo con Amplify, S3 y CloudFront

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-static-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

206 — OIDC de GitHub y GitLab hacia AWS sin secretos

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar oidc de github y gitlab hacia aws sin secretos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar oidc de github y gitlab hacia aws sin secretos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-oidc-federation con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
oidc	Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real.
github	Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real.
gitlab	Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real.
hacia	Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real.
aws	Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real.
sin	Define su papel en oidc de github y gitlab hacia aws sin secretos y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: OIDC de GitHub y GitLab hacia AWS sin secretos"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar oidc de github y gitlab hacia aws sin secretos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/206-oidc-de-github-y-gitlab-hacia-aws-sin-secretos/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-oidc-federation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-oidc-federation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 206 OIDC de GitHub y GitLab hacia AWS sin secretos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-oidc-federation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

207 — SAM, Lambda, API Gateway y despliegue serverless

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar sam, lambda, api gateway y despliegue serverless dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar sam, lambda, api gateway y despliegue serverless con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-serverless-api con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
sam	Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real.
lambda	Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real.
api	Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real.
gateway	Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real.
despliegue	Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real.
serverless	Define su papel en sam, lambda, api gateway y despliegue serverless y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: SAM, Lambda, API Gateway y despliegue serverless"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar sam, lambda, api gateway y despliegue serverless. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/207-sam-lambda-api-gateway-y-despliegue-serverless/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-serverless-api en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-serverless-api para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 207 SAM, Lambda, API Gateway y despliegue serverless

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-serverless-api con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

208 — DynamoDB por patrones de acceso y single-table design

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar dynamodb por patrones de acceso y single-table design dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar dynamodb por patrones de acceso y single-table design con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-dynamodb-model con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
dynamodb	Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real.
patrones	Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real.
acceso	Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real.
single	Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real.
table	Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real.
design	Define su papel en dynamodb por patrones de acceso y single-table design y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: DynamoDB por patrones de acceso y single-table design"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar dynamodb por patrones de acceso y single-table design. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/208-dynamodb-por-patrones-de-acceso-y-single-table-design/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-dynamodb-model en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-dynamodb-model para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 208 DynamoDB por patrones de acceso y single-table design

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-dynamodb-model con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

209 — Cognito, JWT authorizers, WAF y defensa en profundidad

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cognito, jwt authorizers, waf y defensa en profundidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cognito, jwt authorizers, waf y defensa en profundidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-identity-edge con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cognito	Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real.
jwt	Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real.
authorizers	Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real.
waf	Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real.
defensa	Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real.
profundidad	Define su papel en cognito, jwt authorizers, waf y defensa en profundidad y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cognito, JWT authorizers, WAF y defensa en profundidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cognito, jwt authorizers, waf y defensa en profundidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/209-cognito-jwt-authorizers-waf-y-defensa-en-profundidad/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-identity-edge en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-identity-edge para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 209 Cognito, JWT authorizers, WAF y defensa en profundidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-identity-edge con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

210 — EventBridge, SQS, DLQ, replay e idempotencia

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar eventbridge, sqs, dlq, replay e idempotencia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar eventbridge, sqs, dlq, replay e idempotencia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-event-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
eventbridge	Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real.
sqs	Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real.
dlq	Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real.
replay	Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real.
e	Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real.
idempotencia	Define su papel en eventbridge, sqs, dlq, replay e idempotencia y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: EventBridge, SQS, DLQ, replay e idempotencia"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar eventbridge, sqs, dlq, replay e idempotencia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/210-eventbridge-sqs-dlq-replay-e-idempotencia/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-event-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-event-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 210 EventBridge, SQS, DLQ, replay e idempotencia

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-event-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

211 — CloudWatch, X-Ray y observabilidad como código

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloudwatch, x-ray y observabilidad como código dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloudwatch, x-ray y observabilidad como código con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-observability con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloudwatch	Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real.
x	Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real.
ray	Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real.
observabilidad	Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real.
código	Define su papel en cloudwatch, x-ray y observabilidad como código y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: CloudWatch, X-Ray y observabilidad como código"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloudwatch, x-ray y observabilidad como código. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/211-cloudwatch-x-ray-y-observabilidad-como-codigo/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-observability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-observability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 211 CloudWatch, X-Ray y observabilidad como código

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-observability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

212 — ECR, ECS Fargate, ALB y autoscaling

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: container · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ecr, ecs fargate, alb y autoscaling dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ecr, ecs fargate, alb y autoscaling con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-ecs-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ecr	Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real.
ecs	Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real.
fargate	Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real.
alb	Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real.
autoscaling	Define su papel en ecr, ecs fargate, alb y autoscaling y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: ECR, ECS Fargate, ALB y autoscaling"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ecr, ecs fargate, alb y autoscaling. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/212-ecr-ecs-fargate-alb-y-autoscaling/lab.py
```

El laboratorio selecciona el motor de práctica container y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-ecs-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una imagen mínima, escaneada y ejecutada sin privilegios. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-ecs-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 212 ECR, ECS Fargate, ALB y autoscaling

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-ecs-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

213 — EKS, IRSA, GitOps y operación de clúster

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: kubernetes · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar eks, irsa, gitops y operación de clúster dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar eks, irsa, gitops y operación de clúster con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-eks-gitops con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
eks	Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real.
irsa	Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real.
gitops	Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real.
operación	Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real.
clúster	Define su papel en eks, irsa, gitops y operación de clúster y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: EKS, IRSA, GitOps y operación de clúster"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar eks, irsa, gitops y operación de clúster. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/213-eks-irsa-gitops-y-operacion-de-cluster/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa aws-eks-gitops en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `aws-eks-gitops` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.

- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 213 EKS, IRSA, GitOps y operación de clúster

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-eks-gitops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

214 — Budgets, Cost Explorer, etiquetado y FinOps automatizado

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: finops · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar budgets, cost explorer, etiquetado y finops automatizado dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar budgets, cost explorer, etiquetado y finops automatizado con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-cost-control con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
budgets	Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real.
cost	Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real.
explorer	Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real.
etiquetado	Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real.
finops	Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real.
automatizado	Define su papel en budgets, cost explorer, etiquetado y finops automatizado y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Budgets, Cost Explorer, etiquetado y FinOps automatizado"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar budgets, cost explorer, etiquetado y finops automatizado. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/214-budgets-cost-explorer-etiquetado-y-finops-automatizado/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-cost-control en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-cost-control para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 214 Budgets, Cost Explorer, etiquetado y FinOps automatizado

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-cost-control con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

215 — Multi-región, Route 53, failover y game day

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar multi-región, route 53, failover y game day dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar multi-región, route 53, failover y game day con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aws-dr-drill con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
multi	Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real.
región	Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real.
route	Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real.
53	Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real.
failover	Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real.
game	Define su papel en multi-región, route 53, failover y game day y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Multi-región, Route 53, failover y game day"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E["¿Cumple seguridad, SLO y costo?"]
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar multi-región, route 53, failover y game day. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/215-multi-region-route-53-failover-y-game-day/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aws-dr-drill en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aws-dr-drill para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 215 Multi-región, Route 53, failover y game day

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aws-dr-drill con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

216 — Proyecto: CloudShop productivo en AWS

Parte: 17 — AWS: arquitectura, automatización y operación en producción Nivel: avanzado · Horas estimadas: 8
Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: cloudshop productivo en aws dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: cloudshop productivo en aws con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-aws con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: cloudshop productivo en aws y cómo observarlo en un sistema real.
cloudshop	Define su papel en proyecto: cloudshop productivo en aws y cómo observarlo en un sistema real.
productivo	Define su papel en proyecto: cloudshop productivo en aws y cómo observarlo en un sistema real.
aws	Define su papel en proyecto: cloudshop productivo en aws y cómo observarlo en un sistema real.

Modelo mental

AWS se aprende como una progresión operativa: identidad federada, infraestructura declarativa, entrega, señales, recuperación y costo controlado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: CloudShop productivo en AWS"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: cloudshop productivo en aws. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-17-aws-production-architecture/216-proyecto-cloudshop-productivo-en-aws/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-aws en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloudshop-aws para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- AWS Well-Architected Framework — AWS.
- AWS Cookbook — Culkin y Zazon.
- Serverless Architectures on AWS — Peter Sbarski.
- Cloud FinOps — Storment y Fuller.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 216 Proyecto: CloudShop productivo en AWS

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloudshop-aws con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 18 — Azure: arquitectura empresarial y operación en producción

← Parte 17 · Índice completo · Parte 19 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Operar CloudShop sobre landing zones, PaaS, datos, identidad y observabilidad de Azure.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
217	Enterprise-scale landing zones y management groups	governance	4
218	Entra ID, workload identity, PIM y Conditional Access	iam	4
219	Hub-spoke, Virtual WAN, Private Link y DNS privado	network	4
220	Bicep, deployment stacks y Azure Verified Modules	iac	4
221	App Service, Functions y Container Apps en producción	serverless	4
222	AKS, workload identity, ingress y GitOps	kubernetes	4
223	Azure SQL, Cosmos DB y consistencia distribuida	data	4
224	Service Bus, Event Grid y Event Hubs	messaging	4
225	Azure Monitor, Application Insights y OpenTelemetry	observability	4
226	Defender for Cloud, Policy y Sentinel	security	4
227	Cost Management, Advisor, resiliencia y Chaos Studio	finops	4
228	Proyecto: CloudShop productivo en Azure	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.

217 — Enterprise-scale landing zones y management groups

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar enterprise-scale landing zones y management groups dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar enterprise-scale landing zones y management groups con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-landing-zone con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
enterprise	Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real.
scale	Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real.
landing	Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real.
zones	Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real.
management	Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real.
groups	Define su papel en enterprise-scale landing zones y management groups y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Enterprise-scale landing zones y management groups"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar enterprise-scale landing zones y management groups. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/217-enterprise-scale-landing-zones-y-management-groups/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-landing-zone en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye azure-landing-zone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 217 Enterprise-scale landing zones y management groups

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-landing-zone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

218 — Entra ID, workload identity, PIM y Conditional Access

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar entra id, workload identity, pim y conditional access dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar entra id, workload identity, pim y conditional access con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-identity con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
entra	Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real.
id	Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real.
workload	Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real.
identity	Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real.
pim	Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real.
conditional	Define su papel en entra id, workload identity, pim y conditional access y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Entra ID, workload identity, PIM y Conditional Access"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar entra id, workload identity, pim y conditional access. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/218-entra-id-workload-identity-pim-y-conditional-access/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-identity en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye azure-identity para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 218 Entra ID, workload identity, PIM y Conditional Access

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-identity con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

219 — Hub-spoke, Virtual WAN, Private Link y DNS privado

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar hub-spoke, virtual wan, private link y dns privado dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar hub-spoke, virtual wan, private link y dns privado con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-network con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
hub	Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real.
spoke	Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real.
virtual	Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real.
wan	Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real.
private	Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real.
link	Define su papel en hub-spoke, virtual wan, private link y dns privado y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Hub-spoke, Virtual WAN, Private Link y DNS privado"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar hub-spoke, virtual wan, private link y dns privado. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/219-hub-spoke-virtual-wan-private-link-y-dns-privado/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-network en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye azure-network para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 219 Hub-spoke, Virtual WAN, Private Link y DNS privado

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-network con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

220 — Bicep, deployment stacks y Azure Verified Modules

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar bicep, deployment stacks y azure verified modules dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bicep, deployment stacks y azure verified modules con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-bicep-stack con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
bicep	Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real.
deployment	Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real.
stacks	Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real.
azure	Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real.
verified	Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real.
modules	Define su papel en bicep, deployment stacks y azure verified modules y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Bicep, deployment stacks y Azure Verified Modules"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bicep, deployment stacks y azure verified modules. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/220-bicep-deployment-stacks-y-azure-verified-modules/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-bicep-stack en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye azure-bicep-stack para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 220 Bicep, deployment stacks y Azure Verified Modules

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-bicep-stack con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

221 — App Service, Functions y Container Apps en producción

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar app service, functions y container apps en producción dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar app service, functions y container apps en producción con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-app-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
app	Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real.
service	Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real.
functions	Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real.
container	Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real.
apps	Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real.
producción	Define su papel en app service, functions y container apps en producción y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: App Service, Functions y Container Apps en producción"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar app service, functions y container apps en producción. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/221-app-service-functions-y-container-apps-en-produccion/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-app-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye azure-app-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 221 App Service, Functions y Container Apps en producción

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-app-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

222 — AKS, workload identity, ingress y GitOps

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: kubernetes · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar aks, workload identity, ingress y gitops dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar aks, workload identity, ingress y gitops con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-aks-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
aks	Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real.
workload	Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real.
identity	Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real.
ingress	Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real.
gitops	Define su papel en aks, workload identity, ingress y gitops y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: AKS, workload identity, ingress y GitOps"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar aks, workload identity, ingress y gitops. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/222-aks-workload-identity-ingress-y-gitops/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa azure-aks-platform en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `azure-aks-platform` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.

- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 222 AKS, workload identity, ingress y GitOps

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-aks-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

223 — Azure SQL, Cosmos DB y consistencia distribuida

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar azure sql, cosmos db y consistencia distribuida dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar azure sql, cosmos db y consistencia distribuida con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-data-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
azure	Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real.
sql	Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real.
cosmos	Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real.
db	Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real.
consistencia	Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real.
distribuida	Define su papel en azure sql, cosmos db y consistencia distribuida y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Azure SQL, Cosmos DB y consistencia distribuida"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar azure sql, cosmos db y consistencia distribuida. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/223-azure-sql-cosmos-db-y-consistencia-distribuida/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-data-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye azure-data-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 223 Azure SQL, Cosmos DB y consistencia distribuida

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-data-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

224 — Service Bus, Event Grid y Event Hubs

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar service bus, event grid y event hubs dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar service bus, event grid y event hubs con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-event-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
service	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
bus	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
event	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
grid	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
event	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.
hubs	Define su papel en service bus, event grid y event hubs y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Service Bus, Event Grid y Event Hubs"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar service bus, event grid y event hubs. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/224-service-bus-event-grid-y-event-hubs/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa azure-event-platform en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `azure-event-platform` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.

- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 224 Service Bus, Event Grid y Event Hubs

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-event-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

225 — Azure Monitor, Application Insights y OpenTelemetry

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar azure monitor, application insights y opentelemetry dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar azure monitor, application insights y opentelemetry con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-observability con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
azure	Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real.
monitor	Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real.
application	Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real.
insights	Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real.
opentelemetry	Define su papel en azure monitor, application insights y opentelemetry y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Azure Monitor, Application Insights y OpenTelemetry"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar azure monitor, application insights y opentelemetry. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/225-azure-monitor-application-insights-y-opentelemetry/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-observability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye azure-observability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 225 Azure Monitor, Application Insights y OpenTelemetry

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-observability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

226 — Defender for Cloud, Policy y Sentinel

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar defender for cloud, policy y sentinel dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar defender for cloud, policy y sentinel con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-security-operations con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
defender	Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real.
for	Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real.
cloud	Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real.
policy	Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real.
sentinel	Define su papel en defender for cloud, policy y sentinel y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Defender for Cloud, Policy y Sentinel"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar defender for cloud, policy y sentinel. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/226-defender-for-cloud-policy-y-sentinel/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa azure-security-operations en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `azure-security-operations` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.

- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 226 Defender for Cloud, Policy y Sentinel

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-security-operations con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

227 — Cost Management, Advisor, resiliencia y Chaos Studio

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: finops · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cost management, advisor, resiliencia y chaos studio dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cost management, advisor, resiliencia y chaos studio con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar azure-optimization con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cost	Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real.
management	Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real.
advisor	Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real.
resiliencia	Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real.
chaos	Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real.
studio	Define su papel en cost management, advisor, resiliencia y chaos studio y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cost Management, Advisor, resiliencia y Chaos Studio"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cost management, advisor, resiliencia y chaos studio. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/227-cost-management-advisor-resiliencia-y-chaos-studio/lab.py
```

El laboratorio selecciona el motor de práctica finops y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa azure-optimization en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un cálculo trazable con unidad, supuesto y sensibilidad. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye azure-optimization para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 227 Cost Management, Advisor, resiliencia y Chaos Studio

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega azure-optimization con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

228 — Proyecto: CloudShop productivo en Azure

Parte: 18 — Azure: arquitectura empresarial y operación en producción Nivel: avanzado · Horas estimadas: 8
Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: cloudshop productivo en azure dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: cloudshop productivo en azure con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-azure con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: cloudshop productivo en azure y cómo observarlo en un sistema real.
cloudshop	Define su papel en proyecto: cloudshop productivo en azure y cómo observarlo en un sistema real.
productivo	Define su papel en proyecto: cloudshop productivo en azure y cómo observarlo en un sistema real.
azure	Define su papel en proyecto: cloudshop productivo en azure y cómo observarlo en un sistema real.

Modelo mental

La arquitectura Azure nace del tenant y las políticas heredables, y llega al workload mediante identidades administradas y redes privadas.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: CloudShop productivo en Azure"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: cloudshop productivo en azure. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-18-azure-production-architecture/228-proyecto-cloudshop-productivo-en-azure/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-azure en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloudshop-azure para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Azure Architecture Center — Microsoft.
- Exam Ref AZ-305 — Microsoft Press.
- Designing Distributed Systems — Brendan Burns.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 228 Proyecto: CloudShop productivo en Azure

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloudshop-azure con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 19 — Google Cloud: arquitectura de datos y operación en producción

← Parte 18 · Índice completo · Parte 20 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Operar CloudShop con la red global, servicios administrados, seguridad y plataforma de datos de Google Cloud.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
229	Resource Manager, folders, Shared VPC y guardrails	governance	4
230	Workload Identity Federation, IAM Conditions y PAM	iam	4
231	Red global, load balancing, PSC y Cloud DNS	network	4
232	Terraform, Infrastructure Manager y policy validation	iac	4
233	Cloud Run, Functions, API Gateway y Workflows	serverless	4
234	GKE Autopilot, Workload Identity y Config Sync	kubernetes	4
235	Cloud SQL, Spanner, Firestore y Bigtable	data	4
236	BigQuery, Dataflow, Dataproc y gobernanza de datos	data	4
237	Pub/Sub, Eventarc y entrega exactamente-una-vez	messaging	4
238	Cloud Operations, Trace y OpenTelemetry	observability	4
239	SCC, VPC Service Controls, KMS y FinOps	security	4
240	Proyecto: CloudShop productivo en Google Cloud	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.

229 — Resource Manager, folders, Shared VPC y guardrails

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar resource manager, folders, shared vpc y guardrails dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar resource manager, folders, shared vpc y guardrails con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-foundation con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
resource	Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real.
manager	Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real.
folders	Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real.
shared	Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real.
vpc	Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real.
guardrails	Define su papel en resource manager, folders, shared vpc y guardrails y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Resource Manager, folders, Shared VPC y guardrails"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar resource manager, folders, shared vpc y guardrails. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/229-resource-manager-folders-shared-vpc-y-guardrails/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-foundation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-foundation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 229 Resource Manager, folders, Shared VPC y guardrails

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-foundation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

230 — Workload Identity Federation, IAM Conditions y PAM

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: iam · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar workload identity federation, iam conditions y pam dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar workload identity federation, iam conditions y pam con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-identity con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
workload	Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real.
identity	Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real.
federation	Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real.
iam	Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real.
conditions	Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real.
pam	Define su papel en workload identity federation, iam conditions y pam y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Workload Identity Federation, IAM Conditions y PAM"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar workload identity federation, iam conditions y pam. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/230-workload-identity-federation-iam-conditions-y-pam/lab.py
```

El laboratorio selecciona el motor de práctica iam y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-identity en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de acceso mínimo con prueba de denegación. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-identity para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 230 Workload Identity Federation, IAM Conditions y PAM

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-identity con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

231 — Red global, load balancing, PSC y Cloud DNS

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: network · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar red global, load balancing, psc y cloud dns dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar red global, load balancing, psc y cloud dns con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-network con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
red	Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real.
global	Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real.
load	Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real.
balancing	Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real.
psc	Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real.
cloud	Define su papel en red global, load balancing, psc y cloud dns y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Red global, load balancing, PSC y Cloud DNS"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar red global, load balancing, psc y cloud dns. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/231-red-global-load-balancing-psc-y-cloud-dns/lab.py
```

El laboratorio selecciona el motor de práctica network y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-network en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una topología con rutas, puertos y controles. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-network para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 231 Red global, load balancing, PSC y Cloud DNS

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-network con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

232 — Terraform, Infrastructure Manager y policy validation

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: iac · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar terraform, infrastructure manager y policy validation dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar terraform, infrastructure manager y policy validation con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-iac-stack con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
terraform	Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real.
infrastructure	Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real.
manager	Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real.
policy	Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real.
validation	Define su papel en terraform, infrastructure manager y policy validation y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Terraform, Infrastructure Manager y policy validation"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar terraform, infrastructure manager y policy validation. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/232-terraform-infrastructure-manager-y-policy-validation/lab.py
```

El laboratorio selecciona el motor de práctica iac y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-iac-stack en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un plan reproducible sin secretos ni cambios inesperados. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-iac-stack para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 232 Terraform, Infrastructure Manager y policy validation

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-iac-stack con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

233 — Cloud Run, Functions, API Gateway y Workflows

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: serverless · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloud run, functions, api gateway y workflows dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud run, functions, api gateway y workflows con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-serverless con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloud	Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real.
run	Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real.
functions	Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real.
api	Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real.
gateway	Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real.
workflows	Define su papel en cloud run, functions, api gateway y workflows y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Cloud Run, Functions, API Gateway y Workflows"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud run, functions, api gateway y workflows. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/233-cloud-run-functions-api-gateway-y-workflows/lab.py
```

El laboratorio selecciona el motor de práctica serverless y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-serverless en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una función con límites, reintentos e idempotencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-serverless para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 233 Cloud Run, Functions, API Gateway y Workflows

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-serverless con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

234 — GKE Autopilot, Workload Identity y Config Sync

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: kubernetes · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar gke autopilot, workload identity y config sync dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar gke autopilot, workload identity y config sync con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-gke-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
gke	Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real.
autopilot	Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real.
workload	Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real.
identity	Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real.
config	Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real.
sync	Define su papel en gke autopilot, workload identity y config sync y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: GKE Autopilot, Workload Identity y Config Sync"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar gke autopilot, workload identity y config sync. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/234-gke-autopilot-workload-identity-y-config-sync/lab.py
```

El laboratorio selecciona el motor de práctica kubernetes y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-gke-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es manifiestos declarativos con estado observado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-gke-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 234 GKE Autopilot, Workload Identity y Config Sync

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-gke-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

235 — Cloud SQL, Spanner, Firestore y Bigtable

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloud sql, spanner, firestore y bigtable dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud sql, spanner, firestore y bigtable con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-operational-data con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloud	Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real.
sql	Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real.
spanner	Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real.
firestore	Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real.
bigtable	Define su papel en cloud sql, spanner, firestore y bigtable y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Cloud SQL, Spanner, Firestore y Bigtable"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud sql, spanner, firestore y bigtable. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/235-cloud-sql-spanner-firestore-y-bigtable/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-operational-data en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `gcp-operational-data` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.

- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 235 Cloud SQL, Spanner, Firestore y Bigtable

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-operational-data con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

236 — BigQuery, Dataflow, Dataproc y gobernanza de datos

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar bigquery, dataflow, dataproc y gobernanza de datos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bigquery, dataflow, dataproc y gobernanza de datos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-analytics-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
bigquery	Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real.
dataflow	Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real.
dataproc	Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real.
gobernanza	Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real.
datos	Define su papel en bigquery, dataflow, dataproc y gobernanza de datos y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: BigQuery, Dataflow, Dataproc y gobernanza de datos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bigquery, dataflow, dataproc y gobernanza de datos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/236-bigquery-dataflow-dataproc-y-gobernanza-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-analytics-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-analytics-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 236 BigQuery, Dataflow, Dataproc y gobernanza de datos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-analytics-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

237 — Pub/Sub, Eventarc y entrega exactamente-una-vez

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: messaging · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar pub/sub, eventarc y entrega exactamente-una-vez dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar pub/sub, eventarc y entrega exactamente-una-vez con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-event-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
pub	Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real.
sub	Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real.
eventarc	Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real.
entrega	Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real.
exactamente	Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real.
vez	Define su papel en pub/sub, eventarc y entrega exactamente-una-vez y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Pub/Sub, Eventarc y entrega exactamente-una-vez"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar pub/sub, eventarc y entrega exactamente-una-vez. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/237-pub-sub-eventarc-y-entrega-exactamente-una-vez/lab.py
```

El laboratorio selecciona el motor de práctica messaging y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gcp-event-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un flujo que declara orden, entrega y manejo de errores. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-event-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 237 Pub/Sub, Eventarc y entrega exactamente-una-vez

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-event-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

238 — Cloud Operations, Trace y OpenTelemetry

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar cloud operations, trace y opentelemetry dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar cloud operations, trace y opentelemetry con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-observability con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
cloud	Define su papel en cloud operations, trace y opentelemetry y cómo observarlo en un sistema real.
operations	Define su papel en cloud operations, trace y opentelemetry y cómo observarlo en un sistema real.
trace	Define su papel en cloud operations, trace y opentelemetry y cómo observarlo en un sistema real.
opentelemetry	Define su papel en cloud operations, trace y opentelemetry y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Cloud Operations, Trace y OpenTelemetry"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar cloud operations, trace y opentelemetry. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/238-cloud-operations-trace-y-opentelemetry/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-observability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gcp-observability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 238 Cloud Operations, Trace y OpenTelemetry

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-observability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

239 — SCC, VPC Service Controls, KMS y FinOps

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 4
Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar scc, vpc service controls, kms y finops dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar scc, vpc service controls, kms y finops con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gcp-security-finops con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
scc	Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real.
vpc	Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real.
service	Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real.
controls	Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real.
kms	Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real.
finops	Define su papel en scc, vpc service controls, kms y finops y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: SCC, VPC Service Controls, KMS y FinOps"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar scc, vpc service controls, kms y finops. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/239-scc-vpc-service-controls-kms-y-finops/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa gcp-security-finops en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `gcp-security-finops` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.

- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 239 SCC, VPC Service Controls, KMS y FinOps

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gcp-security-finops con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

240 — Proyecto: CloudShop productivo en Google Cloud

Parte: 19 — Google Cloud: arquitectura de datos y operación en producción Nivel: avanzado · Horas estimadas: 8
Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: cloudshop productivo en google cloud dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: cloudshop productivo en google cloud con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-gcp con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real.
cloudshop	Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real.
productivo	Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real.
google	Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real.
cloud	Define su papel en proyecto: cloudshop productivo en google cloud y cómo observarlo en un sistema real.

Modelo mental

Google Cloud combina una jerarquía estricta de recursos con una red global y servicios data-first; proyecto, cuota e IAM forman una unidad operativa.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Proyecto: CloudShop productivo en Google Cloud"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: cloudshop productivo en google cloud. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-19-gcp-production-architecture/240-proyecto-cloudshop-productivo-en-google-cloud/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa cloudshop-gcp en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloudshop-gcp para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Google Cloud Architecture Framework — Google.
- Google Cloud Certified Professional Cloud Architect Study Guide — Dan Sullivan.
- Data Engineering with Google Cloud — Adi Wijaya.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 240 Proyecto: CloudShop productivo en Google Cloud

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloudshop-gcp con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 20 — Plataformas cloud de datos, analítica, IA y agentes

← Parte 19 · Índice completo · Parte 21 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Diseñar plataformas gobernadas para batch, streaming, ML, IA generativa y agentes.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
241	Lakehouse, warehouse, mesh y contratos de datos	data	4
242	Ingesta batch, CDC y streaming	data	4
243	Orquestación, calidad, lineage y observabilidad de datos	observability	4
244	Feature stores, training pipelines y experiment tracking	platform	4
245	Serving online, batch inference y escalado de modelos	performance	4
246	MLOps, registro, promoción, drift y rollback	delivery	4
247	Modelos fundacionales, tokens, embeddings y RAG	architecture	4
248	Bedrock, Azure AI Foundry y Vertex AI	decision	4
249	Agentes, tools, memoria, permisos y guardrails	security	4
250	Evaluación de IA, red teaming y observabilidad	testing	4
251	Privacidad, gobernanza, sostenibilidad y costo de IA	governance	4
252	Proyecto: asistente operativo de CloudShop	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.

241 — Lakehouse, warehouse, mesh y contratos de datos

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar lakehouse, warehouse, mesh y contratos de datos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar lakehouse, warehouse, mesh y contratos de datos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar data-architecture con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
lakehouse	Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real.
warehouse	Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real.
mesh	Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real.
contratos	Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real.
datos	Define su papel en lakehouse, warehouse, mesh y contratos de datos y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Lakehouse, warehouse, mesh y contratos de datos"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar lakehouse, warehouse, mesh y contratos de datos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/241-lakehouse-warehouse-mesh-y-contratos-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa data-architecture en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye data-architecture para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 241 Lakehouse, warehouse, mesh y contratos de datos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega data-architecture con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

242 — Ingesta batch, CDC y streaming

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: data · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ingesta batch, cdc y streaming dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ingesta batch, cdc y streaming con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ingestion-pipeline con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ingesta	Define su papel en ingesta batch, cdc y streaming y cómo observarlo en un sistema real.
batch	Define su papel en ingesta batch, cdc y streaming y cómo observarlo en un sistema real.
cdc	Define su papel en ingesta batch, cdc y streaming y cómo observarlo en un sistema real.
streaming	Define su papel en ingesta batch, cdc y streaming y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ingesta batch, CDC y streaming"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ingesta batch, cdc y streaming. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/242-ingesta-batch-cdc-y-streaming/lab.py
```

El laboratorio selecciona el motor de práctica data y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa ingestion-pipeline en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un modelo de datos ligado a patrones de acceso. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye ingestion-pipeline para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 242 Ingesta batch, CDC y streaming

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega ingestion-pipeline con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

243 — Orquestación, calidad, lineage y observabilidad de datos

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: observability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar orquestación, calidad, lineage y observabilidad de datos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar orquestación, calidad, lineage y observabilidad de datos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar data-reliability con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
orquestación	Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real.
calidad	Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real.
lineage	Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real.
observabilidad	Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real.
datos	Define su papel en orquestación, calidad, lineage y observabilidad de datos y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Orquestación, calidad, lineage y observabilidad de datos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar orquestación, calidad, lineage y observabilidad de datos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/243-orquestacion-calidad-lineage-y-observabilidad-de-datos/lab.py
```

El laboratorio selecciona el motor de práctica observability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa data-reliability en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es telemetría correlacionada con una pregunta operativa. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye data-reliability para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 243 Orquestación, calidad, lineage y observabilidad de datos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega data-reliability con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

244 — Feature stores, training pipelines y experiment tracking

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: platform · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar feature stores, training pipelines y experiment tracking dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar feature stores, training pipelines y experiment tracking con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ml-platform con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
feature	Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real.
stores	Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real.
training	Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real.
pipelines	Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real.
experiment	Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real.
tracking	Define su papel en feature stores, training pipelines y experiment tracking y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Feature stores, training pipelines y experiment tracking"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar feature stores, training pipelines y experiment tracking. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/244-feature-stores-training-pipelines-y-experiment-tracking/lab.py
```

El laboratorio selecciona el motor de práctica platform y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa ml-platform en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una capacidad autoservicio con contrato y golden path. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye ml-platform para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 244 Feature stores, training pipelines y experiment tracking

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega ml-platform con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

245 — Serving online, batch inference y escalado de modelos

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: performance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar serving online, batch inference y escalado de modelos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar serving online, batch inference y escalado de modelos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar model-serving con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
serving	Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real.
online	Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real.
batch	Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real.
inference	Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real.
escalado	Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real.
modelos	Define su papel en serving online, batch inference y escalado de modelos y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Serving online, batch inference y escalado de modelos"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar serving online, batch inference y escalado de modelos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/245-serving-online-batch-inference-y-escalado-de-modelos/lab.py
```

El laboratorio selecciona el motor de práctica performance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa model-serving en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una prueba de carga con baseline y cuello de botella. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye model-serving para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 245 Serving online, batch inference y escalado de modelos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega model-serving con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

246 — MLOps, registro, promoción, drift y rollback

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: delivery · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar mlops, registro, promoción, drift y rollback dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar mlops, registro, promoción, drift y rollback con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar mlops-pipeline con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
mlops	Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real.
registro	Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real.
promoción	Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real.
drift	Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real.
rollback	Define su papel en mlops, registro, promoción, drift y rollback y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: MLOps, registro, promoción, drift y rollback"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar mlops, registro, promoción, drift y rollback. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/246-mlops-registro-promocion-drift-y-rollback/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa mlops-pipeline en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye mlops-pipeline para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.

- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 246 MLOps, registro, promoción, drift y rollback

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega mlops-pipeline con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

247 — Modelos fundacionales, tokens, embeddings y RAG

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar modelos fundacionales, tokens, embeddings y rag dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar modelos fundacionales, tokens, embeddings y rag con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar rag-system con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
modelos	Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real.
fundacionales	Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real.
tokens	Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real.
embeddings	Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real.
rag	Define su papel en modelos fundacionales, tokens, embeddings y rag y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Modelos fundacionales, tokens, embeddings y RAG"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar modelos fundacionales, tokens, embeddings y rag. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/247-modelos-fundacionales-tokens-embeddings-y-rag/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa rag-system en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye rag-system para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 247 Modelos fundacionales, tokens, embeddings y RAG

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega rag-system con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

248 — Bedrock, Azure AI Foundry y Vertex AI

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar bedrock, azure ai foundry y vertex ai dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar bedrock, azure ai foundry y vertex ai con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar genai-provider-adr con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
bedrock	Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real.
azure	Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real.
ai	Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real.
foundry	Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real.
vertex	Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real.
ai	Define su papel en bedrock, azure ai foundry y vertex ai y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Bedrock, Azure AI Foundry y Vertex AI"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar bedrock, azure ai foundry y vertex ai. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/248-bedrock-azure-ai-foundry-y-vertex-ai/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa genai-provider-adr en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `genai-provider-adr` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.

- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 248 Bedrock, Azure AI Foundry y Vertex AI

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega genai-provider-adr con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

249 — Agentes, tools, memoria, permisos y guardrails

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar agentes, tools, memoria, permisos y guardrails dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar agentes, tools, memoria, permisos y guardrails con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar agent-architecture con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
agentes	Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real.
tools	Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real.
memoria	Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real.
permisos	Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real.
guardrails	Define su papel en agentes, tools, memoria, permisos y guardrails y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Agentes, tools, memoria, permisos y guardrails"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar agentes, tools, memoria, permisos y guardrails. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/249-agentes-tools-memoria-permisos-y-guardrails/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa agent-architecture en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `agent-architecture` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- `Designing Machine Learning Systems` — Chip Huyen.

- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 249 Agentes, tools, memoria, permisos y guardrails

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega agent-architecture con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

250 — Evaluación de IA, red teaming y observabilidad

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: testing · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar evaluación de ia, red teaming y observabilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar evaluación de ia, red teaming y observabilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar ai-evaluation con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
evaluación	Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real.
ia	Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real.
red	Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real.
teaming	Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real.
observabilidad	Define su papel en evaluación de ia, red teaming y observabilidad y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Evaluación de IA, red teaming y observabilidad"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar evaluación de ia, red teaming y observabilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/250-evaluacion-de-ia-red-teaming-y-observabilidad/lab.py
```

El laboratorio selecciona el motor de práctica testing y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa ai-evaluation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es pruebas automatizadas con fallos diagnósticos. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye ai-evaluation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 250 Evaluación de IA, red teaming y observabilidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega ai-evaluation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

251 — Privacidad, gobernanza, sostenibilidad y costo de IA

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 4 Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar privacidad, gobernanza, sostenibilidad y costo de ia dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar privacidad, gobernanza, sostenibilidad y costo de ia con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar responsible-ai-controls con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
privacidad	Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real.
gobernanza	Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real.
sostenibilidad	Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real.
costo	Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real.
ia	Define su papel en privacidad, gobernanza, sostenibilidad y costo de ia y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Privacidad, gobernanza, sostenibilidad y costo de IA"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar privacidad, gobernanza, sostenibilidad y costo de ia. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/251-privacidad-gobernanza-sostenibilidad-y-costo-de-ia/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa responsible-ai-controls en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye responsible-ai-controls para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 251 Privacidad, gobernanza, sostenibilidad y costo de IA

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega responsible-ai-controls con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

252 — Proyecto: asistente operativo de CloudShop

Parte: 20 — Plataformas cloud de datos, analítica, IA y agentes Nivel: avanzado · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: asistente operativo de cloudshop dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: asistente operativo de cloudshop con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-ai-assistant con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: asistente operativo de cloudshop y cómo observarlo en un sistema real.
asistente	Define su papel en proyecto: asistente operativo de cloudshop y cómo observarlo en un sistema real.
operativo	Define su papel en proyecto: asistente operativo de cloudshop y cómo observarlo en un sistema real.
cloudshop	Define su papel en proyecto: asistente operativo de cloudshop y cómo observarlo en un sistema real.

Modelo mental

Una plataforma de IA sigue siendo un sistema de datos: necesita procedencia, evaluación, límites de costo, seguridad y operación antes de una interfaz inteligente.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: asistente operativo de CloudShop"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: asistente operativo de cloudshop. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-20-cloud-data-ai-platforms/252-proyecto-asistente-operativo-de-cloudshop/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-ai-assistant en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloudshop-ai-assistant para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Designing Machine Learning Systems — Chip Huyen.
- Fundamentals of Data Engineering — Reis y Housley.
- Building Machine Learning Powered Applications — Emmanuel Ameisen.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 252 Proyecto: asistente operativo de CloudShop

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloudshop-ai-assistant con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 21 — Operación cloud, automatización y respuesta a incidentes

← Parte 20 · Índice completo · Parte 22 →

Nivel: avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Convertir la operación diaria en procesos observables, repetibles y progresivamente automatizados.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
253	Inventario, etiquetado, CMDB y ownership	governance	4
254	Patching, imágenes doradas y gestión de configuración	operations	4
255	Backups, restore testing, vaults e inmutabilidad	reliability	4
256	Administración remota sin SSH permanente	security	4
257	Alertas, on-call, escalamiento y comunicación	incident	4
258	Triage de red, cómputo, datos y dependencias	incident	4
259	Runbooks ejecutables y auto-remediation	operations	4
260	Change management, ventanas y rollback	delivery	4
261	Game days, chaos engineering y aprendizaje	chaos	4
262	Capacity planning, cuotas y gestión de demanda	capacity	4
263	AIOps, automatización asistida y límites humanos	governance	4
264	Proyecto: centro de operaciones de CloudShop	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.

253 — Inventario, etiquetado, CMDB y ownership

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar inventario, etiquetado, cmdb y ownership dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar inventario, etiquetado, cmdb y ownership con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloud-inventory con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
inventario	Define su papel en inventario, etiquetado, cmdb y ownership y cómo observarlo en un sistema real.
etiquetado	Define su papel en inventario, etiquetado, cmdb y ownership y cómo observarlo en un sistema real.
cmdb	Define su papel en inventario, etiquetado, cmdb y ownership y cómo observarlo en un sistema real.
ownership	Define su papel en inventario, etiquetado, cmdb y ownership y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Inventario, etiquetado, CMDB y ownership"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar inventario, etiquetado, cmdb y ownership. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/253-inventario-etiquetado-cmdb-y-ownership/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloud-inventory en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloud-inventory para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 253 Inventario, etiquetado, CMDB y ownership

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloud-inventory con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

254 — Patching, imágenes doradas y gestión de configuración

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4

Laboratorio: operations · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar patching, imágenes doradas y gestión de configuración dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar patching, imágenes doradas y gestión de configuración con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar patch-strategy con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
patching	Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real.
imágenes	Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real.
doradas	Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real.
gestión	Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real.
configuración	Define su papel en patching, imágenes doradas y gestión de configuración y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Patching, imágenes doradas y gestión de configuración"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar patching, imágenes doradas y gestión de configuración. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/254-patching-imagenes-doradas-y-gestion-de-configuracion/lab.py
```

El laboratorio selecciona el motor de práctica operations y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa patch-strategy en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un runbook probado por otra persona. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye patch-strategy para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 254 Patching, imágenes doradas y gestión de configuración

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega patch-strategy con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

255 — Backups, restore testing, vaults e inmutabilidad

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: reliability · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar backups, restore testing, vaults e inmutabilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar backups, restore testing, vaults e inmutabilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar backup-restore con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
backups	Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real.
restore	Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real.
testing	Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real.
vaults	Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real.
e	Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real.
inmutabilidad	Define su papel en backups, restore testing, vaults e inmutabilidad y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Backups, restore testing, vaults e inmutabilidad"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar backups, restore testing, vaults e inmutabilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/255-backups-restore-testing-vaults-e-inmutabilidad/lab.py
```

El laboratorio selecciona el motor de práctica reliability y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa backup-restore en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un escenario de fallo con objetivo y recuperación medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye backup-restore para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 255 Backups, restore testing, vaults e inmutabilidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega backup-restore con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

256 — Administración remota sin SSH permanente

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: security · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar administración remota sin ssh permanente dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar administración remota sin ssh permanente con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar secure-operations con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
administración	Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real.
remota	Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real.
sin	Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real.
ssh	Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real.
permanente	Define su papel en administración remota sin ssh permanente y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Administración remota sin SSH permanente"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar administración remota sin ssh permanente. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/256-administracion-remota-sin-ssh-permanente/lab.py
```

El laboratorio selecciona el motor de práctica security y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa secure-operations en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un control con amenaza, mitigación y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye secure-operations para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..

- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 256 Administración remota sin SSH permanente

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega secure-operations con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

257 — Alertas, on-call, escalamiento y comunicación

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: incident · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar alertas, on-call, escalamiento y comunicación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar alertas, on-call, escalamiento y comunicación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar oncall-system con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
alertas	Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real.
on	Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real.
call	Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real.
escalamiento	Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real.
comunicación	Define su papel en alertas, on-call, escalamiento y comunicación y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Alertas, on-call, escalamiento y comunicación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar alertas, on-call, escalamiento y comunicación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/257-alertas-on-call-escalamiento-y-comunicacion/lab.py
```

El laboratorio selecciona el motor de práctica incident y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa oncall-system en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una cronología, roles, comunicación y aprendizaje. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye oncall-system para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 257 Alertas, on-call, escalamiento y comunicación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega oncall-system con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

258 — Triage de red, cómputo, datos y dependencias

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: incident · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar triage de red, cómputo, datos y dependencias dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar triage de red, cómputo, datos y dependencias con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar diagnostic-tree con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
triage	Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real.
red	Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real.
cómputo	Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real.
datos	Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real.
dependencias	Define su papel en triage de red, cómputo, datos y dependencias y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Triage de red, cómputo, datos y dependencias"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SL0 y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar triage de red, cómputo, datos y dependencias. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/258-triage-de-red-computo-datos-y-dependencias/lab.py
```

El laboratorio selecciona el motor de práctica incident y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa diagnostic-tree en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una cronología, roles, comunicación y aprendizaje. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye diagnostic-tree para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 258 Triage de red, cómputo, datos y dependencias

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega diagnostic-tree con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

259 — Runbooks ejecutables y auto-remediation

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: operations · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar runbooks ejecutables y auto-remediation dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar runbooks ejecutables y auto-remediation con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar executable-runbook con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
runbooks	Define su papel en runbooks ejecutables y auto-remediation y cómo observarlo en un sistema real.
ejecutables	Define su papel en runbooks ejecutables y auto-remediation y cómo observarlo en un sistema real.
auto	Define su papel en runbooks ejecutables y auto-remediation y cómo observarlo en un sistema real.
remediation	Define su papel en runbooks ejecutables y auto-remediation y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Runbooks ejecutables y auto-remediation"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar runbooks ejecutables y auto-remediation. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/259-runbooks-ejecutables-y-auto-remediation/lab.py
```

El laboratorio selecciona el motor de práctica operations y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa executable-runbook en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un runbook probado por otra persona. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye executable-runbook para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 259 Runbooks ejecutables y auto-remediation

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega ejecutable-runbook con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

260 — Change management, ventanas y rollback

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: delivery · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar change management, ventanas y rollback dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar change management, ventanas y rollback con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar change-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
change	Define su papel en change management, ventanas y rollback y cómo observarlo en un sistema real.
management	Define su papel en change management, ventanas y rollback y cómo observarlo en un sistema real.
ventanas	Define su papel en change management, ventanas y rollback y cómo observarlo en un sistema real.
rollback	Define su papel en change management, ventanas y rollback y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Change management, ventanas y rollback"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar change management, ventanas y rollback. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/260-change-management-ventanas-y-rollback/lab.py
```

El laboratorio selecciona el motor de práctica delivery y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa change-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un pipeline con gates, promoción y rollback. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye change-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 260 Change management, ventanas y rollback

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega change-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

261 — Game days, chaos engineering y aprendizaje

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: chaos · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar game days, chaos engineering y aprendizaje dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que expicite seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar game days, chaos engineering y aprendizaje con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar gameday-report con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
game	Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real.
days	Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real.
chaos	Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real.
engineering	Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real.
aprendizaje	Define su papel en game days, chaos engineering y aprendizaje y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Game days, chaos engineering y aprendizaje"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar game days, chaos engineering y aprendizaje. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/261-game-days-chaos-engineering-y-aprendizaje/lab.py
```

El laboratorio selecciona el motor de práctica chaos y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa gameday-report en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una hipótesis de resiliencia y criterio de abortar. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye gameday-report para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 261 Game days, chaos engineering y aprendizaje

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega gameday-report con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

262 — Capacity planning, cuotas y gestión de demanda

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: capacity · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capacity planning, cuotas y gestión de demanda dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capacity planning, cuotas y gestión de demanda con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar capacity-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capacity	Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real.
planning	Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real.
cuotas	Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real.
gestión	Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real.
demanda	Define su papel en capacity planning, cuotas y gestión de demanda y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capacity planning, cuotas y gestión de demanda"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capacity planning, cuotas y gestión de demanda. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/262-capacity-planning-cuotas-y-gestion-de-demanda/lab.py
```

El laboratorio selecciona el motor de práctica capacity y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa capacity-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es requests, límites y una decisión de escalado medida. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye capacity-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 262 Capacity planning, cuotas y gestión de demanda

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega capacity-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

263 — AIOps, automatización asistida y límites humanos

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 4
Laboratorio: governance · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar aiops, automatización asistida y límites humanos dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar aiops, automatización asistida y límites humanos con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar aiops-control con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
aiops	Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real.
automatización	Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real.
asistida	Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real.
límites	Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real.
humanos	Define su papel en aiops, automatización asistida y límites humanos y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: AIOps, automatización asistida y límites humanos"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar aiops, automatización asistida y límites humanos. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/263-aiops-automatizacion-asistida-y-limites-humanos/lab.py
```

El laboratorio selecciona el motor de práctica governance y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa aiops-control en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una jerarquía de política, ownership y evidencia. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye aiops-control para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 263 AIOps, automatización asistida y límites humanos

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega aiops-control con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

264 — Proyecto: centro de operaciones de CloudShop

Parte: 21 — Operación cloud, automatización y respuesta a incidentes Nivel: avanzado · Horas estimadas: 8
Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: centro de operaciones de cloudshop dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: centro de operaciones de cloudshop con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloudshop-operations con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: centro de operaciones de cloudshop y cómo observarlo en un sistema real.
centro	Define su papel en proyecto: centro de operaciones de cloudshop y cómo observarlo en un sistema real.
operaciones	Define su papel en proyecto: centro de operaciones de cloudshop y cómo observarlo en un sistema real.
cloudshop	Define su papel en proyecto: centro de operaciones de cloudshop y cómo observarlo en un sistema real.

Modelo mental

Operar es controlar cambios bajo incertidumbre: inventario, señal, diagnóstico, acción reversible y aprendizaje deben formar un ciclo cerrado.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Proyecto: centro de operaciones de CloudShop"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: centro de operaciones de cloudshop. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-21-cloud-operations-automation/264-proyecto-centro-de-operaciones-de-cloudshop/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa cloudshop-operations en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloudshop-operations para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Site Reliability Workbook — Beyer et al..
- Seeking SRE — Murphy et al..
- Effective DevOps — Davis y Daniels.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 264 Proyecto: centro de operaciones de CloudShop

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloudshop-operations con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 22 — Especializaciones, certificaciones y práctica profesional

← Parte 21 · Índice completo · Parte 23 →

Nivel: intermedio-avanzado · Clases: 12 · Duración sugerida: 6–8 semanas

Transformar el conocimiento transversal en perfiles demostrables, preparación certificable y comunicación profesional.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Secuencia

```
flowchart LR
  A["Conceptos"] --> B["Implementación guiada"]
  B --> C["Pruebas y fallos"]
  C --> D["Operación"]
  D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
265	Ruta Cloud Engineer y mapa de competencias	decision	4
266	Ruta DevOps y Delivery Engineer	decision	4
267	Ruta Platform Engineer	decision	4
268	Ruta Site Reliability Engineer	decision	4
269	Ruta Cloud Security Engineer	decision	4
270	Ruta FinOps Practitioner	decision	4
271	Ruta Cloud Data y AI Engineer	decision	4
272	Ruta Cloud Solutions Architect	decision	4
273	Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps	assessment	4
274	Preguntas de escenario y estrategia de examen	assessment	4
275	Portafolio, evidencia, README y entrevista de sistemas	assessment	4
276	Proyecto: defensa técnica ante panel	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.

265 — Ruta Cloud Engineer y mapa de competencias

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ruta cloud engineer y mapa de competencias dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta cloud engineer y mapa de competencias con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar cloud-engineer-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ruta	Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real.
cloud	Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real.
engineer	Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real.
mapa	Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real.
competencias	Define su papel en ruta cloud engineer y mapa de competencias y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Ruta Cloud Engineer y mapa de competencias"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta cloud engineer y mapa de competencias. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/265-ruta-cloud-engineer-y-mapa-de-competencias/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa cloud-engineer-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye cloud-engineer-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 265 Ruta Cloud Engineer y mapa de competencias

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega cloud-engineer-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

266 — Ruta DevOps y Delivery Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ruta devops y delivery engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta devops y delivery engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar devops-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ruta	Define su papel en ruta devops y delivery engineer y cómo observarlo en un sistema real.
devops	Define su papel en ruta devops y delivery engineer y cómo observarlo en un sistema real.
delivery	Define su papel en ruta devops y delivery engineer y cómo observarlo en un sistema real.
engineer	Define su papel en ruta devops y delivery engineer y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta DevOps y Delivery Engineer"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta devops y delivery engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/266-ruta-devops-y-delivery-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa devops-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye devops-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 266 Ruta DevOps y Delivery Engineer

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega devops-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

267 — Ruta Platform Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ruta platform engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta platform engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar platform-engineer-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ruta	Define su papel en ruta platform engineer y cómo observarlo en un sistema real.
platform	Define su papel en ruta platform engineer y cómo observarlo en un sistema real.
engineer	Define su papel en ruta platform engineer y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Ruta Platform Engineer"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta platform engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/267-ruta-platform-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa platform-engineer-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye platform-engineer-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 267 Ruta Platform Engineer

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega platform-engineer-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

268 — Ruta Site Reliability Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ruta site reliability engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta site reliability engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar sre-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ruta	Define su papel en ruta site reliability engineer y cómo observarlo en un sistema real.
site	Define su papel en ruta site reliability engineer y cómo observarlo en un sistema real.
reliability	Define su papel en ruta site reliability engineer y cómo observarlo en un sistema real.
engineer	Define su papel en ruta site reliability engineer y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta Site Reliability Engineer"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta site reliability engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/268-ruta-site-reliability-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa sre-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye sre-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 268 Ruta Site Reliability Engineer

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega sre-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

269 — Ruta Cloud Security Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ruta cloud security engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta cloud security engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar security-engineer-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ruta	Define su papel en ruta cloud security engineer y cómo observarlo en un sistema real.
cloud	Define su papel en ruta cloud security engineer y cómo observarlo en un sistema real.
security	Define su papel en ruta cloud security engineer y cómo observarlo en un sistema real.
engineer	Define su papel en ruta cloud security engineer y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta Cloud Security Engineer"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta cloud security engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/269-ruta-cloud-security-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa security-engineer-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye security-engineer-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 269 Ruta Cloud Security Engineer

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega security-engineer-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

270 — Ruta FinOps Practitioner

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ruta finops practitioner dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta finops practitioner con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar finops-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ruta	Define su papel en ruta finops practitioner y cómo observarlo en un sistema real.
finops	Define su papel en ruta finops practitioner y cómo observarlo en un sistema real.
practitioner	Define su papel en ruta finops practitioner y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Ruta FinOps Practitioner"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta finops practitioner. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/270-ruta-finops-practitioner/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa finops-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye finops-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 270 Ruta FinOps Practitioner

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega finops-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

271 — Ruta Cloud Data y AI Engineer

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ruta cloud data y ai engineer dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta cloud data y ai engineer con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar data-ai-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ruta	Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real.
cloud	Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real.
data	Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real.
ai	Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real.
engineer	Define su papel en ruta cloud data y ai engineer y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta Cloud Data y AI Engineer"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta cloud data y ai engineer. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/271-ruta-cloud-data-y-ai-engineer/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa data-ai-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye data-ai-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 271 Ruta Cloud Data y AI Engineer

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega data-ai-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

272 — Ruta Cloud Solutions Architect

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: decision · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar ruta cloud solutions architect dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar ruta cloud solutions architect con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar architect-plan con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
ruta	Define su papel en ruta cloud solutions architect y cómo observarlo en un sistema real.
cloud	Define su papel en ruta cloud solutions architect y cómo observarlo en un sistema real.
solutions	Define su papel en ruta cloud solutions architect y cómo observarlo en un sistema real.
architect	Define su papel en ruta cloud solutions architect y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Ruta Cloud Solutions Architect"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar ruta cloud solutions architect. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/272-ruta-cloud-solutions-architect/lab.py
```

El laboratorio selecciona el motor de práctica decision y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa architect-plan en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una matriz de decisión ponderada y un ADR. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye architect-plan para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 272 Ruta Cloud Solutions Architect

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega architect-plan con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

273 — Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: assessment · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar mapeo aws, azure, google cloud, kubernetes y finops dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar mapeo aws, azure, google cloud, kubernetes y finops con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar certification-map con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
mapeo	Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real.
aws	Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real.
azure	Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real.
google	Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real.
cloud	Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real.
kubernetes	Define su papel en mapeo aws, azure, google cloud, kubernetes y finops y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar mapeo aws, azure, google cloud, kubernetes y finops. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/273-mapeo-aws-azure-google-cloud-kubernetes-y-finops/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa certification-map en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye certification-map para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 273 Mapeo AWS, Azure, Google Cloud, Kubernetes y FinOps

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega certification-map con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

274 — Preguntas de escenario y estrategia de examen

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: assessment · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar preguntas de escenario y estrategia de examen dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar preguntas de escenario y estrategia de examen con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar exam-simulation con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
preguntas	Define su papel en preguntas de escenario y estrategia de examen y cómo observarlo en un sistema real.
escenario	Define su papel en preguntas de escenario y estrategia de examen y cómo observarlo en un sistema real.
estrategia	Define su papel en preguntas de escenario y estrategia de examen y cómo observarlo en un sistema real.
examen	Define su papel en preguntas de escenario y estrategia de examen y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Preguntas de escenario y estrategia de examen"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar preguntas de escenario y estrategia de examen. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/274-preguntas-de-escenario-y-estrategia-de-examen/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa exam-simulation en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye exam-simulation para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 274 Preguntas de escenario y estrategia de examen

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega exam-simulation con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

275 — Portafolio, evidencia, README y entrevista de sistemas

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 4
Laboratorio: assessment · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar portafolio, evidencia, readme y entrevista de sistemas dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar portafolio, evidencia, readme y entrevista de sistemas con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar professional-portfolio con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
portafolio	Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real.
evidencia	Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real.
readme	Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real.
entrevista	Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real.
sistemas	Define su papel en portafolio, evidencia, readme y entrevista de sistemas y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Portafolio, evidencia, README y entrevista de sistemas"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```


Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar portafolio, evidencia, readme y entrevista de sistemas. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/275-portafolio-evidencia-readme-y-entrevista-de-sistemas/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa professional-portfolio en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye professional-portfolio para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 275 Portafolio, evidencia, README y entrevista de sistemas

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega professional-portfolio con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

276 — Proyecto: defensa técnica ante panel

Parte: 22 — Especializaciones, certificaciones y práctica profesional Nivel: intermedio-avanzado · Horas estimadas: 8
Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar proyecto: defensa técnica ante panel dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar proyecto: defensa técnica ante panel con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar technical-defense con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
proyecto	Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real.
defensa	Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real.
técnica	Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real.
ante	Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real.
panel	Define su papel en proyecto: defensa técnica ante panel y cómo observarlo en un sistema real.

Modelo mental

Una especialización combina fundamentos, evidencia de proyectos y juicio bajo restricciones; una insignia sin práctica no sustituye esa combinación.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Proyecto: defensa técnica ante panel"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar proyecto: defensa técnica ante panel. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-22-specializations-certifications-career/276-proyecto-defensa-tecnica-ante-panel/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa technical-defense en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye technical-defense para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- The Staff Engineer's Path — Tanya Reilly.
- The Manager's Path — Camille Fournier.
- Staff Engineer — Will Larson.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 276 Proyecto: defensa técnica ante panel

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega technical-defense con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

Parte 23 — Capstones por industria y defensa final

← Parte 22 · Índice completo · Fin

Nivel: experto · Clases: 12 · Duración sugerida: 6–8 semanas

Integrar arquitectura, implementación, operación, costo y comunicación en casos industriales completos.

Resultados de la parte

Al completar esta parte podrás explicar el dominio con lenguaje independiente del proveedor, implementar sus mecanismos esenciales, diagnosticar fallos y defender decisiones mediante evidencia, costo, seguridad y objetivos de confiabilidad.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Secuencia

```
flowchart LR
    A["Conceptos"] --> B["Implementación guiada"]
    B --> C["Pruebas y fallos"]
    C --> D["Operación"]
    D --> E["Proyecto integrador"]
```

Clases

ID	Clase	Laboratorio	Horas
277	Capstone retail: comercio multi-región	capstone	8
278	Capstone financiero: pagos, auditoría y recuperación	capstone	8
279	Capstone salud: privacidad e interoperabilidad	capstone	8
280	Capstone sector público: soberanía y continuidad	capstone	8
281	Capstone media: streaming y distribución global	capstone	8
282	Capstone industria: IoT, edge y operación desconectada	capstone	8
283	Capstone SaaS: multi-tenancy y unit economics	capstone	8
284	Capstone datos e IA: plataforma gobernada	capstone	8
285	Game day integrado y respuesta a incidentes	chaos	4
286	Revisión Well-Architected multi-proveedor	architecture	4
287	Paquete de evidencia, costos y riesgos residuales	assessment	4
288	Defensa final, retrospectiva y plan de 12 meses	capstone	8

Evaluación de la parte

- 35 % laboratorios y evidencia reproducible.
- 25 % retos verificables.
- 20 % decisiones y ADR.
- 10 % seguridad, confiabilidad y costo.
- 10 % proyecto integrador de la parte.

Se aprueba con 80 % de clases en nivel B o superior y el proyecto integrador reproducible.

Pauta bibliográfica

- Building Evolutionary Architectures — Ford, Parsons y Kua.
- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.

277 — Capstone retail: comercio multi-región

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone retail: comercio multi-región dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone retail: comercio multi-región con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar retail-capstone con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real.
retail	Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real.
comercio	Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real.
multi	Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real.
región	Define su papel en capstone retail: comercio multi-región y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone retail: comercio multi-región"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone retail: comercio multi-región. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/277-capstone-retail-comercio-multi-region/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa retail-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `retail-capstone` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 277 Capstone retail: comercio multi-región

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega retail-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

278 — Capstone financiero: pagos, auditoría y recuperación

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone financiero: pagos, auditoría y recuperación dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone financiero: pagos, auditoría y recuperación con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar finance-capstone con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real.
financiero	Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real.
pagos	Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real.
auditoría	Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real.
recuperación	Define su papel en capstone financiero: pagos, auditoría y recuperación y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone financiero: pagos, auditoría y recuperación"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone financiero: pagos, auditoría y recuperación. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/278-capstone-financiero-pagos-auditoria-y-recuperacion/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa finance-capstone en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye finance-capstone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.
- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 278 Capstone financiero: pagos, auditoría y recuperación

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega finance-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

279 — Capstone salud: privacidad e interoperabilidad

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone salud: privacidad e interoperabilidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone salud: privacidad e interoperabilidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar health-capstone con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real.
salud	Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real.
privacidad	Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real.
e	Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real.
interoperabilidad	Define su papel en capstone salud: privacidad e interoperabilidad y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone salud: privacidad e interoperabilidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone salud: privacidad e interoperabilidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/279-capstone-salud-privacidad-e-interoperabilidad/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa health-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `health-capstone` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 279 Capstone salud: privacidad e interoperabilidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega health-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

280 — Capstone sector público: soberanía y continuidad

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone sector público: soberanía y continuidad dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone sector público: soberanía y continuidad con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar public-sector-capstone con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real.
sector	Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real.
público	Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real.
soberanía	Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real.
continuidad	Define su papel en capstone sector público: soberanía y continuidad y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone sector público: soberanía y continuidad"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone sector público: soberanía y continuidad. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/280-capstone-sector-publico-soberania-y-continuidad/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa public-sector-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `public-sector-capstone` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 280 Capstone sector público: soberanía y continuidad

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega public-sector-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

281 — Capstone media: streaming y distribución global

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone media: streaming y distribución global dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone media: streaming y distribución global con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar media-capstone con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real.
media	Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real.
streaming	Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real.
distribución	Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real.
global	Define su papel en capstone media: streaming y distribución global y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Capstone media: streaming y distribución global"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone media: streaming y distribución global. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/281-capstone-media-streaming-y-distribucion-global/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa media-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye media-capstone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 281 Capstone media: streaming y distribución global

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega media-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

282 — Capstone industria: IoT, edge y operación desconectada

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone industria: iot, edge y operación desconectada dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone industria: iot, edge y operación desconectada con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar industry-capstone con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real.
industria	Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real.
iot	Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real.
edge	Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real.
operación	Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real.
desconectada	Define su papel en capstone industria: iot, edge y operación desconectada y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Capstone industria: IoT, edge y operación desconectada"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone industria: iot, edge y operación desconectada. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/282-capstone-industria-iot-edge-y-operacion-desconectada/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.

4. Materializa industry-capstone en evidence/ usando la plantilla indicada.
5. Para proveedor real, sigue sandbox.requires, registra costo y ejecuta destroy.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye industry-capstone para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] lab.py termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.
- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 282 Capstone industria: IoT, edge y operación desconectada

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega industry-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

283 — Capstone SaaS: multi-tenancy y unit economics

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone saas: multi-tenancy y unit economics dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone saas: multi-tenancy y unit economics con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar saas-capstone con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real.
saas	Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real.
multi	Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real.
tenancy	Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real.
unit	Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real.
economics	Define su papel en capstone saas: multi-tenancy y unit economics y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Capstone SaaS: multi-tenancy y unit economics"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone saas: multi-tenancy y unit economics. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/283-capstone-saas-multi-tenancy-y-unit-economics/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa saas-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `saas-capstone` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 283 Capstone SaaS: multi-tenancy y unit economics

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega saas-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

284 — Capstone datos e IA: plataforma gobernada

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar capstone datos e ia: plataforma gobernada dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar capstone datos e ia: plataforma gobernada con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar data-ai-capstone con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
capstone	Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real.
datos	Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real.
e	Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real.
ia	Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real.
plataforma	Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real.
gobernada	Define su papel en capstone datos e ia: plataforma gobernada y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
  A["Necesidad y restricciones"] --> B["Diseño: Capstone datos e IA: plataforma gobernada"]
  B --> C["Implementación reproducible"]
  C --> D["Estado observado"]
  D --> E{"¿Cumple seguridad, SLO y costo?"}
  E -- "No" --> B
  E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar capstone datos e ia: plataforma gobernada. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/284-capstone-datos-e-ia-plataforma-gobernada/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa data-ai-capstone en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `data-ai-capstone` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 284 Capstone datos e IA: plataforma gobernada

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega data-ai-capstone con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

285 — Game day integrado y respuesta a incidentes

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 4 Laboratorio: chaos · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar game day integrado y respuesta a incidentes dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar game day integrado y respuesta a incidentes con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar final-gameday con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
game	Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real.
day	Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real.
integrado	Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real.
respuesta	Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real.
incidentes	Define su papel en game day integrado y respuesta a incidentes y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Game day integrado y respuesta a incidentes"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar game day integrado y respuesta a incidentes. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/285-game-day-integrado-y-respuesta-a-incidentes/lab.py
```

El laboratorio selecciona el motor de práctica chaos y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa final-gameday en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una hipótesis de resiliencia y criterio de abortar. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `final-gameday` para el caso `CloudShop`. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 285 Game day integrado y respuesta a incidentes

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega final-gameday con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

286 — Revisión Well-Architected multi-proveedor

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 4 Laboratorio: architecture · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar revisión well-architected multi-proveedor dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar revisión well-architected multi-proveedor con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar final-architecture-review con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
revisión	Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real.
well	Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real.
architected	Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real.
multi	Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real.
proveedor	Define su papel en revisión well-architected multi-proveedor y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

```
flowchart LR
    A["Necesidad y restricciones"] --> B["Diseño: Revisión Well-Architected multi-proveedor"]
    B --> C["Implementación reproducible"]
    C --> D["Estado observado"]
    D --> E{"¿Cumple seguridad, SLO y costo?"}
    E -- "No" --> B
    E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar revisión well-architected multi-proveedor. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/286-revision-well-architected-multi-proveedor/lab.py
```

El laboratorio selecciona el motor de práctica architecture y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa final-architecture-review en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un diagrama acompañado por decisiones y trade-offs. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `final-architecture-review` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 286 Revisión Well-Architected multi-proveedor

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega final-architecture-review con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- [] Resultado reproducible por una segunda persona.
- [] Evidencia vinculada a cada afirmación técnica.
- [] Prueba negativa o de fallo incluida.
- [] Costos y permisos expresados con unidades y alcance.
- [] Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

287 — Paquete de evidencia, costos y riesgos residuales

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 4 Laboratorio: assessment · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar paquete de evidencia, costos y riesgos residuales dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explice seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar paquete de evidencia, costos y riesgos residuales con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar final-evidence-pack con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
paquete	Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real.
evidencia	Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real.
costos	Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real.
riesgos	Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real.
residuales	Define su papel en paquete de evidencia, costos y riesgos residuales y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Paquete de evidencia, costos y riesgos residuales"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar paquete de evidencia, costos y riesgos residuales. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/287-paquete-de-evidencia-costos-y-riesgos-residuales/lab.py
```

El laboratorio selecciona el motor de práctica assessment y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa final-evidence-pack en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es una evaluación por escenarios con rúbrica y evidencia trazable. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye `final-evidence-pack` para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de `rollback`.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 287 Paquete de evidencia, costos y riesgos residuales

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega final-evidence-pack con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.

288 — Defensa final, retrospectiva y plan de 12 meses

Parte: 23 — Capstones por industria y defensa final Nivel: experto · Horas estimadas: 8 Laboratorio: capstone · Estado: EXECUTABLECORE

Propósito

Comprender y aplicar defensa final, retrospectiva y plan de 12 meses dentro de una plataforma cloud realista, produciendo evidencia reproducible y una decisión que explicita seguridad, confiabilidad, costo y operación. La meta no es memorizar nombres de servicios: es reconocer el problema, seleccionar una solución proporcional y demostrar qué ocurrió.

Resultados de aprendizaje

Al finalizar podrás:

1. Explicar defensa final, retrospectiva y plan de 12 meses con vocabulario independiente del proveedor.
2. Relacionar sus componentes con el modelo mental de la parte.
3. Ejecutar un laboratorio local determinista y leer su contrato JSON.
4. Evaluar al menos una alternativa y justificar el trade-off elegido.
5. Entregar final-defense con evidencia, límites y criterio de reversión.

Conceptos centrales

Concepto	Comprensión verificable
defensa	Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real.
final	Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real.
retrospectiva	Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real.
plan	Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real.
12	Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real.
meses	Define su papel en defensa final, retrospectiva y plan de 12 meses y cómo observarlo en un sistema real.

Modelo mental

El capstone no premia cantidad de servicios, sino trazabilidad entre contexto, decisiones, implementación, fallos, evidencia y aprendizaje.

Aplicado a esta clase, separa siempre cuatro planos: intención (qué necesita el usuario), configuración (qué declaramos), estado observado (qué existe de verdad) y evidencia (cómo sabemos que cumple). Confundirlos produce diseños que se ven correctos en un diagrama pero fallan al operar.

■ Flujo de razonamiento

flowchart LR

```
A["Necesidad y restricciones"] --> B["Diseño: Defensa final, retrospectiva y plan de 12 meses"]
B --> C["Implementación reproducible"]
C --> D["Estado observado"]
D --> E{"¿Cumple seguridad, SLO y costo?"}
E -- "No" --> B
E -- "Sí" --> F["Evidencia y decisión registrada"]
```

Desarrollo

1. Del requisito al mecanismo

Empieza por una frase medible: quién consume la capacidad, bajo qué carga, desde dónde, con qué datos y qué impacto tendría un fallo. Después identifica el mecanismo de esta clase que satisface cada restricción. Un producto cloud solo es una implementación posible; el requisito permanece aunque cambies de AWS a Azure, Google Cloud o infraestructura propia.

2. Fronteras y responsabilidades

Documenta quién administra identidad, red, datos, runtime y observabilidad. Marca qué queda en manos del proveedor y qué sigue siendo responsabilidad del equipo. Cada frontera debe tener propietario, interfaz, señal operativa y forma de recuperación. Si una responsabilidad no tiene dueño, el diseño todavía está incompleto.

3. Compensaciones que deben quedar visibles

Dimensión	Pregunta de diseño
Confiabilidad	¿Qué falla, cómo se detecta y cuánto tarda en recuperarse?
Seguridad	¿Qué identidad actúa y cuál es el mínimo privilegio necesario?
Costo	¿Cuál es la unidad de consumo y qué hace crecer la factura?
Operación	¿Qué señal permite diagnosticarlo sin entrar manualmente al servidor?
Portabilidad	¿Qué contrato es estándar y qué decisión es específica del proveedor?

La respuesta correcta puede ser más simple que la arquitectura inicialmente imaginada. En cloud, complejidad también consume presupuesto de error, tiempo de equipo y capacidad de respuesta a incidentes.

Ejemplo trabajado

Una plataforma de pedidos necesita aplicar defensa final, retrospectiva y plan de 12 meses. El equipo registra:

- demanda base de 20 solicitudes/s y pico de 120 solicitudes/s;
- SLO mensual de 99,9 % para operaciones de lectura;
- RPO de 15 minutos y RTO de 60 minutos;
- datos personales que no pueden salir de la región aprobada;
- presupuesto inicial de USD 600/mes.

La decisión se acepta solo si explica cómo la propuesta responde a esas cinco restricciones. Se descarta cualquier alternativa que dependa de acceso administrativo permanente, no tenga telemetría o cuyo costo no pueda atribuirse. El resultado esperado no es "usar servicio X", sino una cadena trazable: requisito → mecanismo → prueba → señal → límite.

Laboratorio guiado

Ejecuta desde la raíz:

```
python classes/part-23-industry-capstones/288-defensa-final-retrospectiva-y-plan-de-12-meses/lab.py
```

El laboratorio selecciona el motor de práctica capstone y produce labresult.json. El escenario, sus comprobaciones y el artefacto esperado corresponden a esta clase; no requiere credenciales y deja explícito qué debe revalidarse en un sandbox real.

1. Lee exercise.steps y formula una predicción antes de ejecutar.
2. Ejecuta la práctica y verifica todos los elementos de checks.
3. Provoca el caso de negativetest y explica la señal observada.
4. Materializa final-defense en evidence/ usando la plantilla indicada.

5. Para proveedor real, sigue `sandbox.requires`, registra costo y ejecuta `destroy`.

Evidencia esperada

El artefacto principal es un incremento integrado, demostrable y documentado. Además, la entrega debe incluir el comando ejecutado, la salida estructurada y una conclusión que no exceda lo observado.

Reto verificable

Construye final-defense para el caso CloudShop. Incluye una alternativa descartada, un supuesto que pueda falsarse, una prueba de fallo y una decisión de rollback.

■ Criterio de aceptación

- [] `lab.py` termina con código 0 y genera JSON válido.
- [] La entrega conecta al menos tres requisitos con mecanismos verificables.
- [] Existe una prueba positiva y una prueba negativa con evidencia.
- [] Seguridad, costo y operación aparecen como decisiones, no como anexos.
- [] Se declara una limitación y una condición que obligaría a revisar el diseño.
- [] Otra persona puede repetir el recorrido sin conocimiento tácito.

■ ■ Errores frecuentes

Síntoma	Causa probable	Corrección
El diseño enumera servicios pero no requisitos	Se comenzó por el catálogo del proveedor	Reescribe primero escenarios y restricciones medibles.
La demo funciona una vez y se declara lista	Se confundió ejecución con evidencia operacional	Añade repetición, fallo, telemetría y recuperación.
Todo tiene permisos administrativos	El laboratorio heredó credenciales humanas	Usa identidad de workload y prueba explícitamente la denegación.
No se puede explicar la factura	Faltan unidades y ownership de costo	Etiqueta, estima por unidad y define presupuesto o alerta.
La solución se llama multi-cloud pero replica todo	Portabilidad se confundió con duplicación	Define qué riesgo se mitiga y porta solo el contrato necesario.

■ Seguridad, ética y costo

Trabaja con cuentas propias o sandboxes autorizados. No publiques secretos, identificadores reales ni datos personales. Antes de crear recursos pagos define presupuesto, etiquetas y comando de destrucción; después verifica que no queden recursos huérfanos. Los ejemplos locales enseñan contratos, pero no certifican cumplimiento ni disponibilidad de producción.

■ Preguntas de comprobación

1. ¿Qué parte del diseño seguiría siendo válida en otro proveedor?
2. ¿Qué señal distinguiría saturación, fallo de dependencia y error de configuración?
3. ¿Cuál es la unidad de costo y quién puede actuar sobre ella?
4. ¿Qué permiso puede retirarse sin romper el caso de uso?
5. ¿Qué evidencia falta para afirmar que esto está listo para producción?

Referencias

- Building Evolutionary Architectures — Ford, Parsons y Kua.

- Release It! — Michael Nygard.
- Accelerate — Forsgren, Humble y Kim.
- Documentación oficial vigente del servicio implementado; registra URL y fecha de consulta.

Evaluación — 288 Defensa final, retrospectiva y plan de 12 meses

Preguntas

1. Explica el problema que resuelve esta capacidad sin mencionar un proveedor.
2. Identifica una frontera de responsabilidad y su propietario.
3. Compara dos alternativas mediante confiabilidad, seguridad, costo y operación.
4. ¿Qué demuestra el laboratorio y qué no puede demostrar?
5. Propón un caso límite o una prueba de fallo.

Reto verificable

Entrega final-defense con diagrama o configuración, evidencia de ejecución, alternativa descartada, riesgo residual y criterio de rollback.

Criterio de aceptación

- ☐ Resultado reproducible por una segunda persona.
- ☐ Evidencia vinculada a cada afirmación técnica.
- ☐ Prueba negativa o de fallo incluida.
- ☐ Costos y permisos expresados con unidades y alcance.
- ☐ Limitaciones declaradas sin presentar la simulación como producción.

Escala

Nivel	Evidencia
A — competente	Cumple todo y defiende trade-offs con datos.
B — en desarrollo	Cumple lo esencial; falta profundidad en una dimensión.
C — inicial	Artefacto parcial o conclusión sin evidencia suficiente.
0	Sin entrega reproducible.